Development Tools Guide

24.0.0.1000

NEXACRO

TOBESOFT strives to maintain the accuracy of information provided in this document and will consistently modify and supplement it. However, we cannot guarantee accuracy of information in this document. TOBESOFT assumes no responsibility for errors and omissions. No liability is assumed for damages resulting from the use of information contained herein.

Users of this document implicitly agree that they understand the contents of this disclaimer.

Each trademark herein, including the product name of each company, is a solely registered trademark of each company who developed it, and is protected by patent laws and the copyright laws, and should be used for the purpose of reference only.

TOBESOFT Co.,Ltd.
2-5F INTOPS Building, 617 Bongeunsa-Ro, Gangnam-Gu,
Seoul 06083, Korea
www.tobesoft.com

Table of Contents

Copyright and Disclaimer ii

Table of Contents iii

Part I. Nexacro Studio 1

1. Main Features 2

2. Installing Nexacro Studio, Start Page, Help 4

2.1 Installing Nexacro Studio 4

2.1.1 Selecting the Installation Language 5

2.1.2 Installation Process 5

2.1.3 Installation Options 10

2.1.4 Update 10

2.1.5 Nexacro N V21 SDK 10

2.2 Initial Setup of Nexacro Studio 11

2.2.1 Main Folders and Files 11

2.2.2 Version Check 12

2.2.3 Initial Settings 13

2.2.4 Licence Authentication 14

2.3 Install Nexacro SDK 16

2.4 Start Page 16

2.5 Using Help 18

3. UIs of Nexacro Studio 20

3.1 Menu, Toolbar and Ribbon Interface 21

3.1.1 Menu bar and Toolbar 21

3.1.2 Ribbon Interface 22

3.2 Status Bar 23

3.3 View 24

3.3.1 Arranging Views 24

Tabs 24

Cascade Windows ... 25
Tile Windows ... 26
Arrange Icons ... 27
3.3.2 Form Design ... 27
Design Window ... 27
Source Ediitor ... 28
Script Editor ... 29
3.4 Dockable Windows ... 30
3.4.1 Rearranging Windows ... 30
Displaying/Hiding Windows ... 30
Moving Windows ... 32
Changing Location ... 32
Docking Windows ... 33
Combining Tabs ... 34
4. Workspaces | Explorers, Properties, Message ... 36
4.1 Project Explorer ... 36
4.1.1 Search ... 37
4.1.2 Filter Function ... 38
4.1.3 Copy ... 39
4.2 Resource Explorer ... 39
4.3 Properties Window ... 40
4.3.1 Object List ... 41
4.3.2 Sorting Type ... 41
4.3.3 Editing Target ... 42
4.3.4 empty value ... 44
4.3.5 Computed Style ... 46
4.3.6 User Property ... 47
4.3.7 Editing Complex Property Values ... 49
Color Picker ... 49
gradient ... 51
Set Border ... 52
Set Font ... 54
Entering Multiple Lines of Text ... 55
Editing Shortcut Keys ... 55
Editing the innerdataset Property Value ... 56
Editing the formats Property Value ... 56
4.3.8 Filtering ... 56
4.3.9 Quick Link to Help ... 57
4.3.10 Copy Property Name ... 58
4.3.11 Checking the Meaning of the Font in which Property Value is Displayed ... 58

4.4 Message Window 60
4.4.1 Bookmarks 60
4.4.2 Output 62
4.4.3 Error List 62
4.4.4 Find Result 63
4.4.5 Reference 64
4.5 Debug Window 65
4.5.1 Debug 65
4.5.2 Breakpoints 66
4.5.3 Call Stack 67
4.5.4 Variables 69
4.5.5 Watch 70
4.5.6 Script Editor 70
4.6 Form, Show Thumbnail 72
5. Wizard | Project and Application 73
5.1 Project Wizard 73
5.1.1 Step 1. Location 74
5.1.2 Step 2. Screen 75
5.1.3 Step 3. Frame 76
5.1.4 Step 4 QuickCode 78
5.2 Application Wizard 79
5.3 Project Template 80
5.3.1 Add on Template 81
6. Wizard | Form and Other Files 84
6.1 Form Creation Wizard 84
6.1.1 Step 1 Location 84
6.1.2 Step 2 Layout 85
6.1.3 Step 3 Position 87
6.2 Create Other Files 87
6.2.1 Create Theme Files 88
6.2.2 Create Style Sheet 89
6.2.3 Create Script Files 89
6.2.4 Create XML Files 90
6.2.5 Create InitValue Files 90
6.2.6 Create UserFont Files 92
6.2.7 Creating a StringResource file 93
6.3 Register and Use Form Templates 94
6.3.1 Register a Form Template 95
6.3.2 Create a Form by using a Form Template 96
6.4 Registering & Using ViewSet Template 97

6.4.1 Registering ViewSet Template ... 97
6.4.2 Creating New Form with ViewSet Template ... 99
7. Project Editing | TypeDefinition ... 101
7.1 Objects ... 101
7.1.1 Edit modules ... 102
7.1.2 Change order ... 102
7.1.3 View file source ... 103
7.1.4 List of objects ... 104
7.1.5 Object refactoring ... 105
7.1.6 PrefixID ... 106
7.2 Services ... 107
7.2.1 Edit services ... 107
7.2.2 Service refactoring ... 110
7.3 ProtocolAdaptors ... 110
7.3.1 Editing protocols ... 111
7.3.2 New Protocol Wizard ... 111
7.4 DeviceAdaptors ... 113
7.5 CordovaPlugins ... 114
8. Project Editing | Application Infomation ... 116
8.1 Application Variables ... 116
8.1.1 Datasets ... 116
8.1.2 Variables ... 118
8.1.3 DataObjects ... 118
8.2 Application ... 118
8.2.1 MainFrame ... 119
8.2.2 Trays ... 121
8.2.3 Style ... 122
9. Project Editing | Environment, FDL, InitValueDefiniton ... 123
9.1 Environment ... 123
9.1.1 ScreenDefinition ... 123
Screen Refactoring ... 124
9.1.2 Variables ... 124
9.1.3 Cookies ... 125
9.1.4 HTTP Header ... 125
9.1.5 Accessibility ... 126
9.2 FDL (Form Definition Language) ... 126
9.2.1 Design ... 127
Register TypeDefinition ... 128
9.2.2 Source ... 129
9.2.3 Script ... 130

Go to Declaration 131
Open Include File 132
9.3 InitValueDefiniton 132
10. Project Editing | Theme, XCSS 135
10.1 Theme 135
10.1.1 Image 136
10.1.2 XCSS 137
10.2 XCSS 137
10.2.1 Adding Selector 140
10.2.2 Adding Class Selector 143
10.2.3 Adding Child Selector 144
10.2.4 NavigatorBar 145
10.2.5 Editing XCSS Code 146
10.2.6 Defining Styles Based on the Screen 148
10.2.7 Declaring a variable and using it as a property value (Predefine) 148
Declare a variable 149
Using declared variables as property values (XCSS Editor) 150
Using a declared variable as a property value (property window) 151
Check the generate result 151
Sharing Predefine Settings 152
Usage example: Changing the gray theme color 152
11. Project Editing | Export, Import, Module Installation 154
11.1 Export 154
11.1.1 Selecting & Exporting the Export Item from File Menu 154
11.1.2 Selecting & Exporting the Export Item from Context Menu 156
11.2 Import 156
11.2.1 Selecting & Importing the Import Item from File Menu 157
11.2.2 Selecting & Importing the Import Item from the Context Menu 158
11.2.3 Importing File by Drag & Drop from File Explorer 159
11.3 Install Module Wizard 159
11.3.1 Module (*.xmodule*) Installation 160
11.3.2 ActiveX Control Installation 162
11.3.3 Installing the Cordova plugin 164
12. Form Designing | Arranging, Selecting, Moving and Setting 166
12.1 Component 166
12.1.1 Component Arrangement 166
12.1.2 Selection Change 167
Select Type 168
Select the parent component(Container Selector) 168
Select overlapping components 169

12.1.3 Move Component ... 170
Use mouse ... 170
Use keyboard ... 171
12.1.4 Size Change ... 171
12.1.5 Copy Component ... 173
Mouse ... 173
Keyboard ... 173
12.1.6 Lock Control ... 174
12.1.7 Tab Order ... 174
Tab Order(Z-Order) View ... 174
Tab Order Editor (View Type) ... 176
Tab Order Editor (List Type) ... 178
12.1.8 Set Properties ... 178
12.1.9 Outlineview ... 179
12.1.10 Layout Editor ... 180
Auto Layout ... 182
12.1.11 Other Features ... 183
Paste Special ... 183
Hotkey editor ... 184
Go to Event Handler ... 184
Copy ID ... 185
Change Component ... 185
12.2 Add and Use Component Preset ... 187
12.2.1 Add Preset ... 188
12.2.2 Allocate Preset ... 189
12.2.3 Manage Preset and Category ... 190
Change name ... 190
Delete ... 190
Change order ... 190
Filtering ... 191
Update preset ... 191
12.2.4 Export and Import Component Preset File ... 192
12.2.5 Toolbar on Component Preset Pane ... 193
13. Form Designing | Positions and Guides ... 194
13.1 Position ... 194
13.1.1 Basic Properties ... 194
13.1.2 Tracker ... 195
13.1.3 min-, max- and fittocontents Properties ... 196
13.1.4 PositionBase ... 197
Restrictions ... 198

PositionBase tracker ... 199
Removing a connection ... 201
13.1.5 Position Editor ... 202
13.2 Guides ... 203
13.2.1 Ruler ... 203
13.2.2 Guide Line ... 204
13.2.3 Background Grid ... 205
13.2.4 Snap ... 206
14. Form Designing | MLM, Fluid Layout ... 207
14.1 MLM (Multi Layout Manager) ... 207
14.1.1 Layout ... 207
Layout Toolbar ... 208
Fluid Layout ... 209
Layout Tooltip ... 209
Property Values in Different Font Styles ... 210
14.1.2 Initialize ... 211
14.1.3 Step ... 211
stepcount ... 212
14.2 Fluid Layout ... 213
14.2.1 Setting component placement standards ... 213
Form object ... 213
Container component ... 213
Panel Component ... 214
14.2.2 Placing components (horizontal, vertical) ... 214
Select a components and move with the mouse ... 214
Grouping and moving the components ... 215
When a container component has a Panel component ... 218
14.2.3 Placing components (table) ... 220
Placing a component at a specific cell position ... 220
Add, insert, delete columns and rows ... 223
Placing components in the area set with the same cell name ... 225
14.2.4 Setting component placement standards and spacing ... 226
Standards for placing components ... 226
Setting the component spacing ... 227
Handling line breaks ... 228
Setting the table cell spacing ... 229
14.2.5 Resizing Based on Placed Components ... 229
15. Binding Data, Creating Events and Editing Contents ... 230
15.1 Data Binding ... 230
15.1.1 Configuring Data Binding ... 230

formats Property Value Setting 231
Bind InnerDataset 231
Bind Item Editor 232
Bind Dataset Editor 233
15.1.2 Adding & Editing Bind Item 234
Create Component & Bind Item Editor 234
Displaying & Editing Bind State Icon 235
Editing in the Property Window 235
15.1.3 Checking Data Binding Information 236
Invisible Objects Pane 236
Binding Components List Pane 238
15.2 Contents Editor 238
15.2.1 Grid Contents Editor 238
Main Editor 239
Format 241
Design Source 241
15.2.2 ListView Contents Editor 242
Main Editor 242
Band Editing Mode 243
Format 244
Design Source 244
15.2.3 Json Contents Editor 245
15.2.4 Data Path Editor 246
15.2.5 XML Contents Editor 248
15.2.6 Basic Contents Editor 248
15.3 Adding Events 249
16. Generating and Executing Project 251
16.1 Project Generation 251
16.1.1 Generation 251
Generate Application 251
Generate Theme 251
Generate Service 252
Generate File 252
Regenerate 253
16.1.2 Automatic Generation 254
16.2 Project Execution 254
16.2.1 Execution Mode 254
NRE (Nexacro Runtime Environment) 255
Emulator 255
Web browser 256

16.2.2 Launch . . . 256
16.2.3 QuickView . . . 257
16.3 Managing Bootstrap Template . . . 257
16.3.1 Creating a Template with Script Code Added . . . 259
16.3.2 Cautions when Editing Bootstrap Template . . . 260
Editing or Deleting Keyword . . . 260
Editing the Default Script . . . 261
16.3.3 Note . . . 261
Bootstrap Template Configuration File Path . . . 261
Keyword List . . . 262
Splash Loader . . . 262
17. Creating Installation Files (or Web Documents) for Deployment . . . 264
17.1 Configuring Deploy Environment . . . 264
17.2 Packing- Create Archive Files . . . 265
17.2.1 Launch Packing Wizard to Create Archive Files . . . 265
17.2.2 Select Update Type (for each operating environment) . . . 267
17.2.3 Configurable Options (according to operating environments and update types) . . . 268
Update Info. . . . 268
Setup . . . 269
Module . . . 270
Source . . . 272
Resource . . . 273
17.3 Build App- Create Installation Files . . . 275
17.3.1 Launch Build App Wizard to Create Installation Files . . . 275
17.3.2 Set Options for AppBuilder . . . 282
Create TargetProject (App Builder Project) . . . 283
Register or Change Build Library . . . 283
Changing or Registering Signing as New . . . 285
Register or Change User Library . . . 286
Define Permissions . . . 287
17.3.3 Check Logs when AppBuilder Fails to build . . . 288
17.4 Quick Build with Previous Settings . . . 289
18. Migration . . . 292
18.1 Procedures . . . 292
18.1.1 Opening Project . . . 293
When you performs migration for the first time . . . 293
When you have performed migration before . . . 293
18.1.2 Migration Wizard . . . 294
18.1.3 Fixing Errors . . . 296
Check migration log . . . 296

Fix error ... 297
18.1.4 Ending Migration Project ... 298
18.2 Migration Mode Menu ... 298
18.2.1 Project Explorer ... 299
18.2.2 Migration Log ... 300
18.2.3 Migration DiffView ... 301
19. Development Facilities ... 303
19.1 Assist ... 303
19.1.1 Copy Full Path ... 303
19.1.2 Open Containing Folder ... 303
19.1.3 Open File in Project ... 303
19.1.4 Open Service Folder / Open Folder ... 304
19.1.5 Code Snippet ... 304
Defining a Code Snippet ... 305
Code Snippet Reserved string ... 306
Using Code Snippets ... 307
19.2 Intellisense ... 309
19.2.1 include ... 309
19.2.2 Script ... 309
19.2.3 XCSS ... 310
19.3 Auto Indent ... 311
19.4 find and replace strings ... 312
19.4.1 Find ... 312
19.4.2 Replace ... 312
19.4.3 Find in File ... 313
19.4.4 common options ... 313
Regular expressions ... 314
19.5 Rename ... 315
19.6 Code Line Editing ... 316
19.7 Auto Recover ... 316
19.8 metadata ... 317
19.8.1 Registering metadata Service ... 317
19.8.2 Applying Data Received through metadata Service as id, text Property Values ... 319
19.8.3 Other Functions ... 320
Refresh Filtered ... 320
Search Column ... 320
Display text ... 320
metadata Loading at Project Start ... 321
19.8.4 ServiceList, DataSchema Specifications ... 321
19.9 Developing Add on ... 322

19.10 HTTP authentication ... 326
19.10.1 Operating method ... 327
20. LiteDB: Looking Up Database and Writing Queries ... 328
20.1 Look up database ... 328
20.2 Write queries ... 330
21. Applying Multiple Languages ... 332
21.1 StringResource ... 332
21.1.1 Create a new StringResource ... 332
21.1.2 Adding and Deleting the Locale (Language Column) ... 334
21.1.3 Adding and Deleting Data ... 335
21.1.4 Making Modifications in the Data Source ... 335
21.1.5 Overwriting with external StringResource data ... 336
Import (overwrite) ... 336
pull ... 337
21.2 Applying Multiple Languages ... 337
21.2.1 Reflecting Selected Key Value in Components in the StringResource Pane ... 337
21.2.2 Changing the Language Applied to the Page ... 338
21.2.3 Reflecting the Key Value in Components in the Properties Pane ... 339
21.2.4 TEXT function ... 340
21.3 Importing the Key Value with a Script ... 341
21.3.1 getStringResourceValue method ... 341
21.4 Importing Data from a Remotely Located StringResource ... 341
21.4.1 Remote StringResource Data Types ... 341
21.5 Multiple Language Application Criteria ... 343
21.5.1 Language ... 343
21.5.2 StringResource Data ... 343
22. Supporting Source Control Software ... 345
22.1 Subversion (SVN) ... 345
22.1.1 Settings for Subversion ... 345
22.1.2 Toolbar ... 348
22.1.3 Authentication ... 349
22.1.4 Basic Features ... 350
Import ... 350
Commit ... 351
Add ... 351
Delete ... 352
Rename ... 353
Update ... 354
22.1.5 Lock ... 354
Lock ... 354

UnLock ... 355
22.1.6 Utilizing Log ... 355
Show Log ... 355
Compare with Previous Revision ... 356
Revert ... 357
Refresh Status ... 357
22.1.7 Repository Browser ... 357
Check Out ... 359
Export ... 360
22.1.8 SVN Output ... 360
22.1.9 SVN Explorer ... 361
22.1.10 Subversion Status ... 362
22.2 Git ... 362
22.2.1 Configure Git ... 362
Set options ... 362
Define merge tool arguments ... 365
22.2.2 Start Project ... 366
Add a project to local repository ... 366
Add a project to remote repository ... 366
Clone a project into local repository ... 367
Delete data from remote repository ... 368
22.2.3 Apply Changes ... 369
Add files or change existing files ... 369
Commit changes to local repository ... 369
Push changes to remote repository ... 370
22.2.4 Resolve Conflicts ... 371
When automatic merge is performed ... 371
When conflict occurs ... 371
Resolve conflicts with merge tool ... 372
22.2.5 Create or Switch Branch ... 373
Create branch ... 373
Switch to another branch ... 374
Check branches ... 374
22.2.6 Output Pane / Status ... 375
Git Output ... 375
Git Explorer ... 376
Git Status ... 377
23. Setting Options ... 379
23.1 Options – Project ... 379
23.1.1 General ... 380

23.1.2 SDK ... 381
23.1.3 Startup ... 381
23.1.4 Generate ... 382
Gereral ... 382
CSS ... 382
Event ... 383
23.1.5 QuickCode ... 384
Form Template, Viewset Template ... 385
View Template ... 385
23.1.6 Deploy ... 385
General ... 386
Compress ... 387
23.1.7 Build ... 389
NRE (Windows) ... 389
AppBuilder > Login ... 390
AppBuilder > Environment ... 390
23.1.8 Launch ... 391
23.1.9 Position ... 392
23.1.10 Macros ... 393
23.1.11 Advanced ... 394
23.2 Options – Environment ... 395
23.2.1 General ... 395
23.2.2 Startup ... 396
23.2.3 Auto Recover ... 397
23.2.4 Font and Color ... 398
23.2.5 Show Information ... 399
23.2.6 Script ... 399
General ... 400
Version ... 400
23.2.7 Generate ... 402
23.2.8 External Tools ... 402
23.2.9 Advanced ... 403
23.3 Options – Form Design ... 404
23.3.1 General ... 405
23.3.2 Guide ... 405
23.3.3 Layout ... 407
23.3.4 Paste Special ... 408
23.4 Options - Launch ... 408
23.4.1 General ... 408
23.4.2 Browser ... 409

23.5 Options – Debug 410
23.6 Options – Source Control 410
24. Toolbar, Menu bar, Keyboard shortcut 412
24.1 Toolbar Functions 412
24.1.1 Standard Bar 412
24.1.2 Align 413
24.1.3 Bookmark 414
24.1.4 Build 415
24.1.5 Debug 415
24.1.6 Component 416
Selecting and moving components 416
Basic components 417
Default Component - Add 418
DeviceAPI 419
XPush 420
XAgent 420
Graphics 421
24.1.7 Position 421
24.1.8 QuickCode 422
24.1.9 SVN 422
24.1.10 Git 423
24.2 Menu Bar 424
24.2.1 File 424
24.2.2 Edit 426
24.2.3 Assist 427
24.2.4 View 428
24.2.5 SVN 429
24.2.6 Git 430
24.2.7 Design 432
24.2.8 Layout 434
24.2.9 QuickCode 435
24.2.10 Debug 436
24.2.11 Generate 437
24.2.12 Deploy 438
24.2.13 Tools 439
24.2.14 Window 439
24.2.15 Help 440
24.3 Shortcut keys 440
24.3.1 Common 441
24.3.2 Window 442

24.3.3 Project Explorer ... 442
24.3.4 Properties Window ... 442
24.3.5 Form Design ... 442
24.3.6 Debug ... 443
24.3.7 Migration ... 444
24.3.8 Source/Script ... 444

Part II. QuickCode Core ... 446

25. Create a project with QuickCode samples ... 447
25.1 Running QuickCode Tutorial Wizard ... 447
25.1.1 Select Use 'QuickCode' in the project wizard ... 447
25.1.2 Selecting the Let's Try QuickCode Item from Menu ... 448
25.2 QuickCode Tutorial Wizard Execution Screen ... 448
25.3 QuickCode Tutorial Wizard Execution Results ... 449
25.3.1 QuickCodeTutorialForm.xfdl ... 450

26. Configure the QuickCode development environment for an existing project and create screens ... 452
26.1 Term Definition ... 452
26.2 Configuring Development Environment ... 453
26.2.1 Creating and starting a new project ... 453
Start by importing all the necessary items in an xpackage file ... 453
Start by importing each of the necessary items separately ... 456
26.2.2 Importing ViewSet Template File ... 459
26.2.3 Importing View Template File ... 460
26.2.4 Creating New Form ... 462
26.3 Creating Screen UI with Model & View Template ... 462
26.3.1 Creating Search UI ... 463
26.3.2 Creating Grid UI ... 465
26.3.3 Creating Detailed Information UI ... 466
26.4 Adding Data Search Function Using Action ... 466
26.4.1 Connecting Action ... 466
26.4.2 Creating sample XML File ... 469
26.4.3 Executing Screen ... 470
26.5 Adding Action Processed when Data Search Action is Successful ... 471
26.5.1 Connecting Action ... 471
26.5.2 Changing Data Search Action Property Value ... 473
26.5.3 Executing Screen ... 473
26.6 Adding Only Trigger to Action ... 474
26.6.1 Connecting Action ... 474

26.6.2 Executing Screen ... 477
27. Develop Model, View Template, and Action ... 478
27.1 Creating Model ... 478
27.1.1 Adding a service in TypeDefinition ... 478
27.1.2 Adding Model Service items ... 479
27.1.3 Creating Model in Model Wizard ... 480
Adding User Attribute ... 482
27.1.4 Setting up the Model Service ... 484
27.1.5 Check the model information source code ... 484
27.2 Creating View Template ... 485
27.2.1 Two methods of creating a View template ... 485
Creating View Template in View Template Wizard ... 485
Creating a View Template with a Designed View Component ... 487
27.2.2 Editing View Template Script ... 488
fn_GetViewGenerationResult ... 489
fieldarray Parameter JSON Format ... 490
contents Parameter JSON Format ... 491
generationattr Parameter JSON Format ... 492
Return Value JSON Format ... 493
fn_GetViewAttributeList ... 497
Return Value JSON Format ... 497
fn_GetFieldUserAttributeList ... 499
Handling User Attributes when creating a Model ... 499
Handling User Attributes when applying a Model ... 500
Return Value JSON Format ... 500
27.2.3 View Template Sample ... 501
viewSampleForm ... 501
viewSampleGrid ... 505
27.3 Creating Action ... 508
27.3.1 Creating Action Module Project ... 509
27.3.2 Adding Action Object ... 510
27.3.3 Editing Action Object Property ... 510
27.3.4 Editing Action Object Script ... 511
TestQueryAction Script ... 513
TestAlertAction Script ... 516
27.3.5 Editing Action Object Meta-Info Property ... 518
group ... 520
edittype ... 520
27.3.6 Creating Action Module File ... 521
28. QuickCode detailed specifications and references ... 522

28.1 Action Object Properties, Methods, & Events ... 522
28.1.1 Properties ... 522
28.1.2 Methods ... 522
28.1.3 Events ... 522
28.2 Item Information Available for the Trigger Type ... 523
28.3 Trigger Condition Reserved Words & Samples ... 524
28.4 fn_GetViewGenerationResult Return Value JSON Format ... 525
28.4.1 Basic Format ... 525
28.4.2 Conversion Rules ... 527
28.4.3 Conversion Rule Exception (Container Component) ... 536
28.4.4 Processing Event ... 537
28.5 View Layout Editor ... 538
28.5.1 Set cell properties ... 541
28.5.2 Merge or Unmerge Cells ... 542
28.5.3 value return value JSON format ... 542
28.6 XML <-> JSON Converter ... 543
28.6.1 Where features are available ... 544
28.6.2 Automatically select code to convert ... 544
28.6.3 Direct code entry ... 545
28.7 Edit Attribute ... 546
28.7.1 Edit Field Attributes ... 546
28.7.2 Edit Model Attribute ... 547
28.7.3 Edit Service Attribute ... 548
28.8 QuickCode Reference ... 549
28.8.1 filter ... 550
28.8.2 search ... 550
28.8.3 Move the View component ... 551
28.9 Querying the Model service ... 551
28.9.1 Querying the Model Group, Model service ... 551
28.9.2 Querying the Model ... 553
28.9.3 Response data format - ErrorCode, ErrorMsg ... 554
28.9.4 Response data format - Model Group, Model Service ... 554
28.9.5 Response Data Format - Model ... 555
28.9.6 JSP code example ... 557

Part III. Console Tools ... 561

29. Nexacro Deploy ... 562

29.1 Generating Code ... 562
29.2 Types of Nexacro Deploy ... 562
29.3 Options ... 563

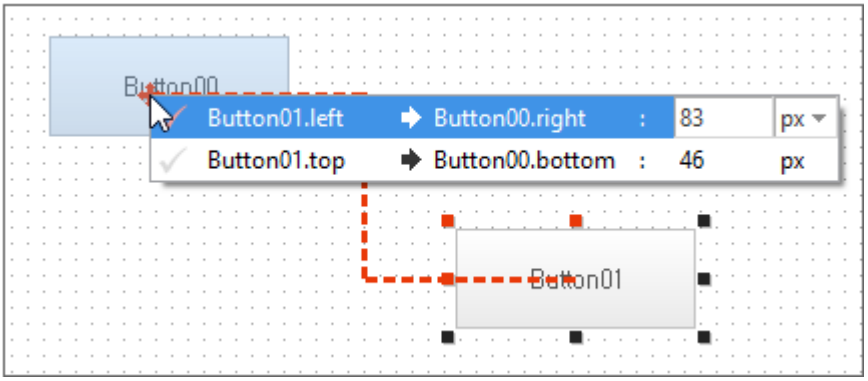
29.3.1 Required Options ... 563
29.3.2 Options for Specifying Target Files ... 564
29.3.3 Generate Options ... 564
29.3.4 Deploy Options ... 566
29.3.5 Other options ... 567
29.4 Examples ... 567
29.4.1 Generate ... 567
Generate entire Application ... 567
Generate only certain file ... 567
When generating only specific services ... 568
Generate only module files ... 568
Generate only bootstrap files ... 568
Generate bootstrap files but not archive them ... 569
29.4.2 Deploy ... 569
Deploy entire Application ... 569
Module merge option ... 569
Compress option ... 570
Compile option ... 571
29.4.3 Others ... 571
Save generation result as log file ... 571

Part I.

Nexacro Studio

1.

Main Features



Intuitive design editor

The form editor of Nexacro Studio provides various features for designers. With the editor, you can make multiple screen layouts and design them. You can also check the data combined to components as well as tab order directly through the editor pane. Moreover, the editor offers the "Arrange Components" feature, which allows you to arrange components based on relative coordinates.

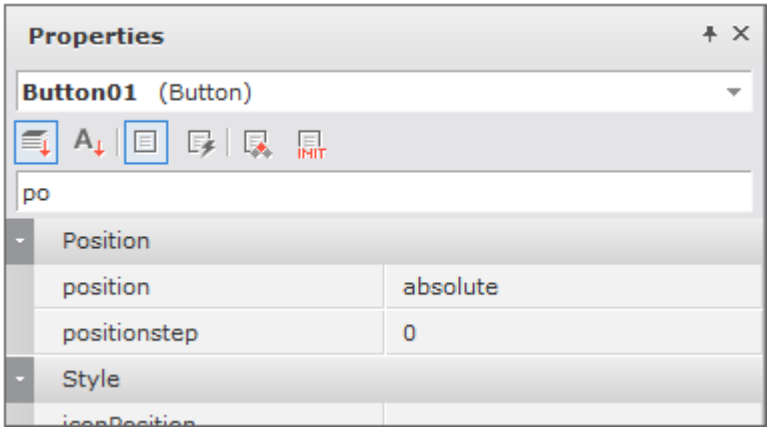
Form Designing | Arranging, Selecting, Moving and Setting PositionBase

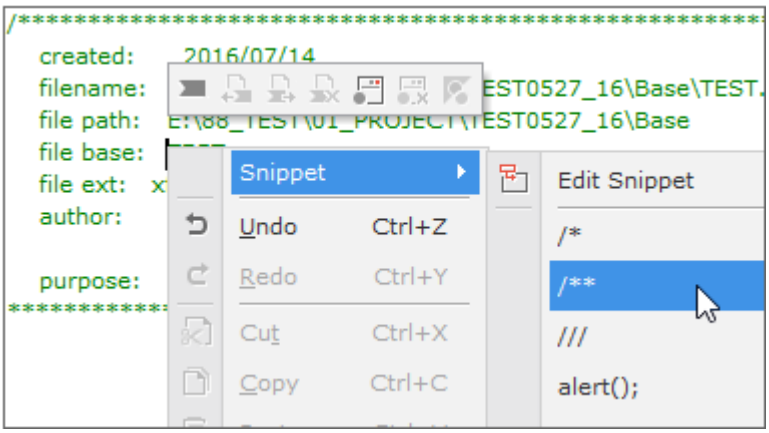
More convenient Properties window

The Properties window allows you to simply edit the properties of settings files and the properties of the components laid on a Form, which all take the form of XML. That is to say, you do not have to access the codes on a script to edit those properties. In addition, the window provides a search feature (filtering) so that you can easily find the properties you want to edit.

Properties Window

Filtering





More powerful code completion

Nexacro Studio offers a code completion feature, which responses to users' inputs. If you define template codes in advance, you can use them as snippets when editing script codes or CSS codes. Furthermore, Nexacro Studio can detect incorrect codes when you are coding.

Intellisense

Code Snippet

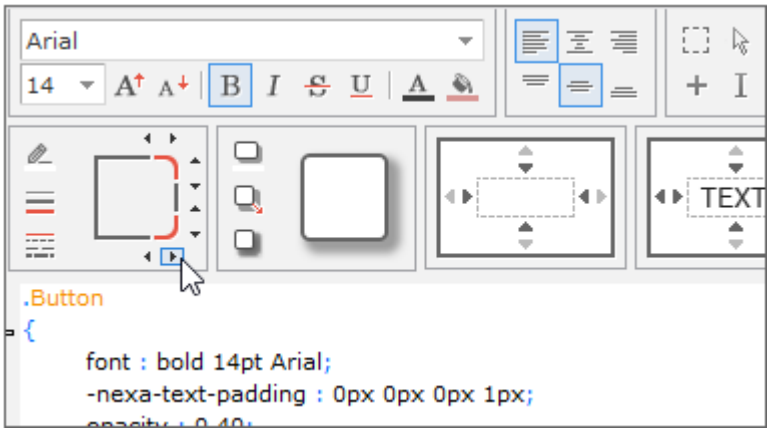
Error List

Improved style sheet editing

With the advent of the version N, Nexacro has aligned its style sheet-related APIs with the official CSS standards. Moreover, the intuitive style editor shows you CSS codes in real time as they appear on actual web browsers.

Theme

XCSS





Link to App Builder

Nexacro Studio provides the connection to the App Builder server, so you can produce deployment files for iOS, Android, and macOS in the tool. The server will create a QR code after building a deployment file of your app, and users can easily download and install your app by scanning the QR code. In addition, the Studio offers an emulator where you can test how your app will work in various devices.

Build App- Create Installation Files

Emulator

2.

Installing Nexacro Studio, Start Page, Help

Nexacro Studio is a powerful integrated development environment (IDE) that supports a series of tasks for developing a Nexacro-based app (Nexacro app, hereinafter). It involves designing, developing, testing, debugging and deploying. The form editor of the tool helps you to design an app screen quickly while you can intuitively see the data-binding state and relationship between components.

The screen management system of the Studio allows you to develop an app of various screen sizes. And App Builder helps you create and deploy installment files of your app for various mobile devices, setting you free from the need for other development tools.

2.1 Installing Nexacro Studio

Nexacro Studio is included in the Nexacro install package. Run the Setup file to start the installation process. When prompted, click **Next**.

Setup files support 32-bit and 64-bit versions.
32-bit: NexacroN_SetupStudio.exe
64-bit: NexacroN_SetupStudio_x64.exe

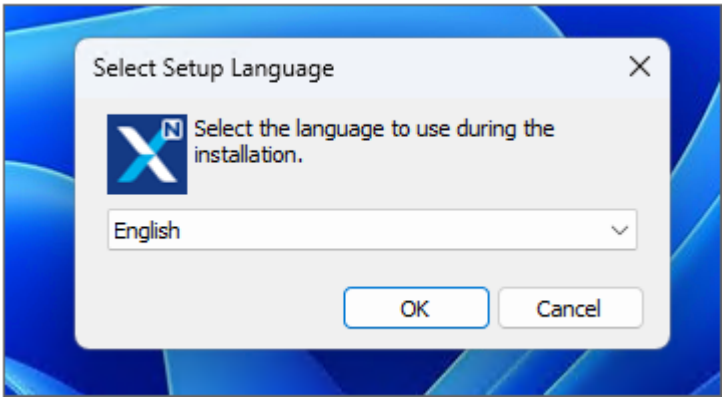
You cannot install both versions.
When updating the 32-bit version of Nexacro Studio that was previously installed to the 64-bit version, you must delete the 32-bit version and install the 64-bit version.
When updating, information such as options and project lists used in the previous version is maintained.

A security warning window may appear when running the setup file. Be sure to select ‘Yes’.

2.1.1 Selecting the Installation Language

Select the installation language. During the installation process, it will be displayed in the selected language, and when running Nexacro Studio after installation, information such as Help will be provided in the selected language.

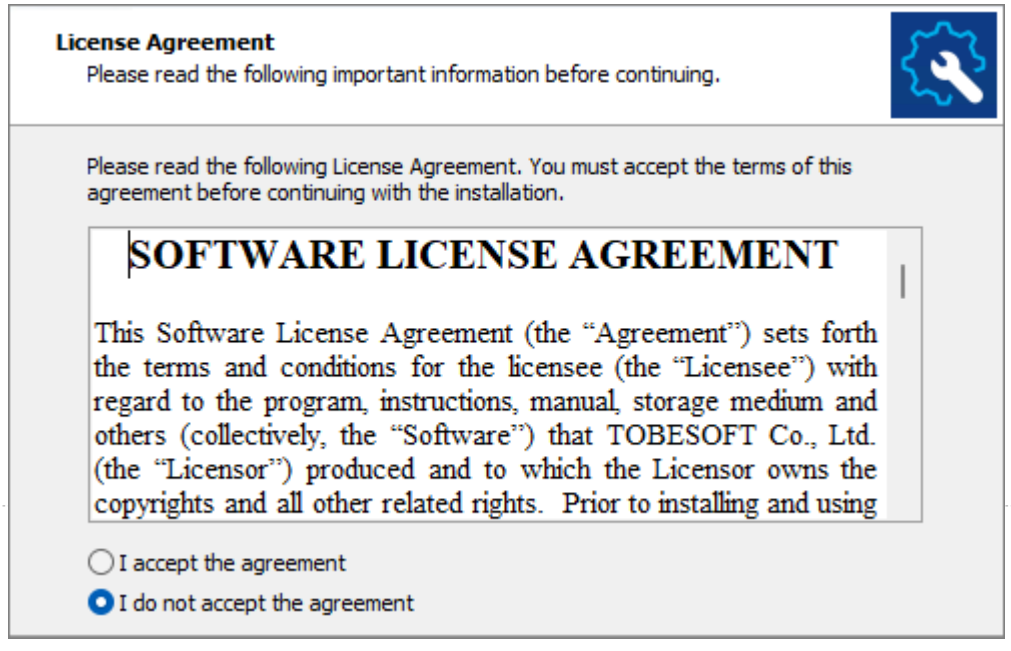
You can select English, Japanese, or Korean. The language used by the operating system will be selected and displayed.

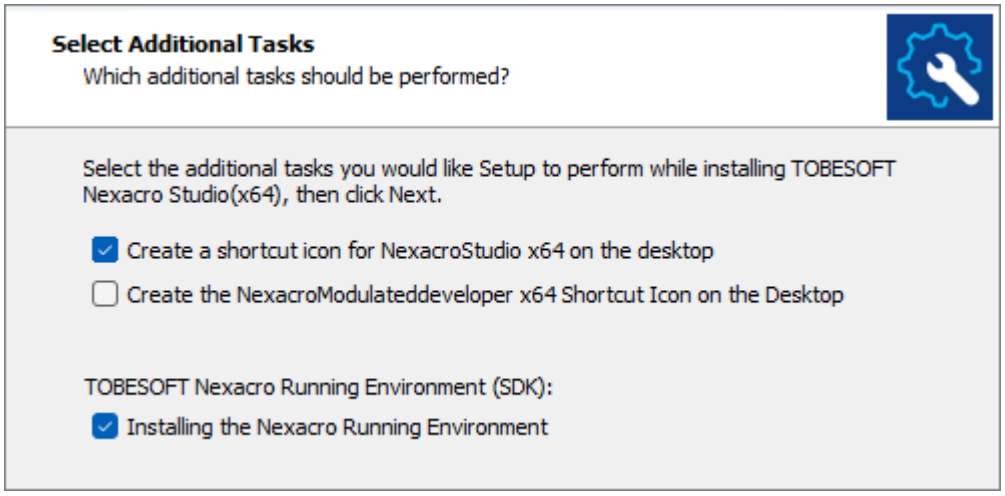


Clicking the [OK] button will start the installation process in the selected language.

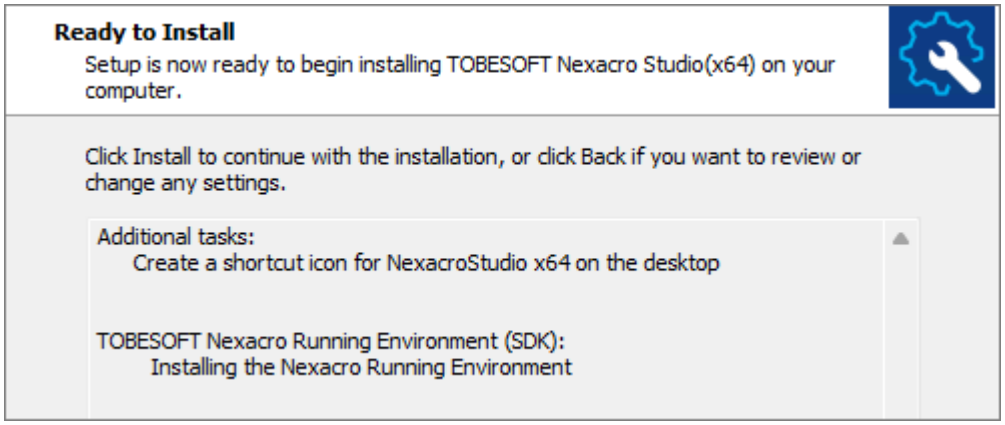
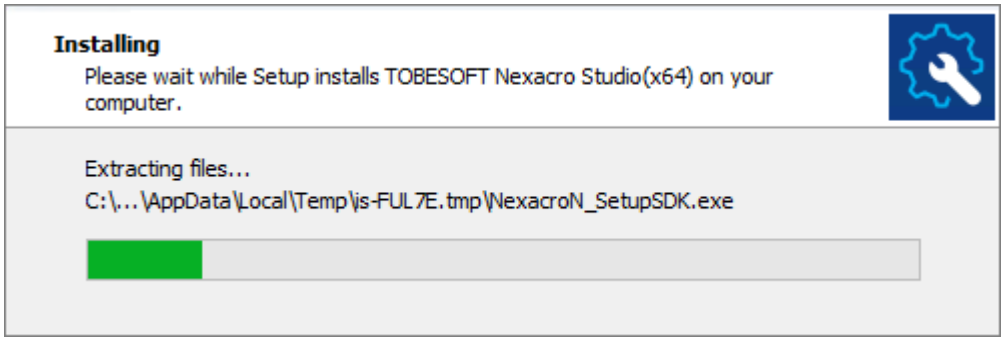
2.1.2 Installation Process

Nexacro Studio installation proceeds in the following order.

Sequence	Function	
1		Check the software licens

Sequence	Function	
		e
2		Decide whether to create shortcut icons on the desktop. Check off to create shortcut icons for the follow

Sequence	Function	
		ing on the desktop: - Nexacro Studio - Nexacro Module Developer Decide whether to install the S

Sequence	Function	
		DK. Check off to install the SDK.
3		Check Option 2
4		Proceed with installation
5		Confirm installati

Sequence	Function
	

on Check whether to run Nexacro Studio If it is checked off, Nexacro Studio will be launched

2.1.3 Installation Options

You can specify additional options when installing Nexacro Studio to configure certain steps in advance or skip certain steps in the installation process.

```
NexacroN_SetupStudio.exe /LANG=[ARG] /SILENT /VERYSILENT
NexacroN_SetupStudio_x64.exe /LANG=[ARG] /SILENT /VERYSILENT
```

Option	Description
/LANG	Select Korean, English, or Japanese as the installation language. Korean | English | Japanese If you don't choose any of these options, it will automatically be set up in English.
/SILENT	Skip the following steps and proceed with the installation process: - Select installation language - Check the software license - Determine whether to create a desktop shortcut icon (for Nexacro Studio) - Determine whether to install the SDK (SDK installation)
/VERYSILENT	Proceed with the installation process without checking if installing a version that is the same as or older than the currently installed version. The rest is the same as the /SILENT option.

You can set the commands as below for the installation process.

```
NexacroN_SetupStudio.exe /LANG=Japanese
NexacroN_SetupStudio.exe /LANG=Japanese /SILENT
NexacroN_SetupStudio.exe /SILENT
NexacroN_SetupStudio.exe /VERYSILENT
```

2.1.4 Update

When updating, the installation language will be set as the language selected at the initial installation.

The update process is the same as the installation process except for the installation language selection.

2.1.5 Nexacro N V21 SDK

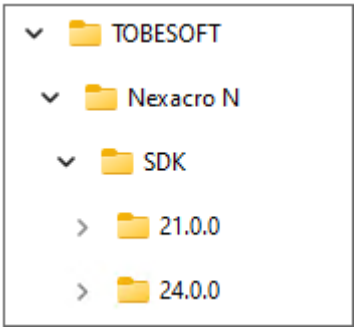
In Nexacro N V24, you can use projects from the previous version (Nexacro N V21) as they are.

- In case of installing Nexacro N V21

When installing Nexacro N V24, the SDK folder will be maintained and only the development tools will be updated.
Existing projects may be used as they are, without any changes, and if necessary, you may change them to V24 projects by changing the SDK.

- In case of newly installing Nexacro N V24
  You can run projects created using previous version after installing the Nexacro N V21 SDK.

It will be installed under the SDK folder, separated by version, as shown below.



2.2 Initial Setup of Nexacro Studio

2.2.1 Main Folders and Files

The below table describes major folders and files created under the installation folder of Nexacro Studio.

SDK > <version>	
Folder name	**Description**
help	- Folder for the help of Nexacro The help is provided in the form of CHM (compiled html help). Korean or English version will be opened in accordance with the language setting of the operating system.
nexacrolib	- Folder for the libraries of Nexacro This folder contains the library files appropriate to the installed platform version. This folder also contains metainfo files, which are used only by Nexacro Studio and not subject to distribution.
embedded	- Folder for Nexacro apps embedded in Nexacro Studio This folder contains the source codes of the Nexacro apps embedded in Nexacro Studio: for example, the Grid Contents Editor and the Nexacro Emulator.
SetupRuntime	Folder for the module files of each OS. Relevant module files among them are referred to when an app is built.

SDK > <version>	
Folder name	**Description**
theme	- Folder for themes This folder contains cjstyles files for the skin of Nexacro Studio and theme files for the Nexacro apps. There are a few default theme files, varying with apps' operating environments.
File name	**Description**
nexacro.exe	- Executable of the NRE
ResourceUpdater_x64.exe ResourceUpdater_x86.exe	- Executable for building a deployment file for the NRE
nexacromigrator.exe	- Executable for migrating a Nexacro Platform 14-based project into a Nexacro N version.
Tools	
Folder name	**Description**
license	- Folder for the license-related documents This folder contains the license-related documents for Nexacro products and open source products.
File name	**Description**
nexacrodeploy.exe	- Executable for converting a project file into a JavaScript file and for obfuscating files.
nexacroemulator.exe	- Executable of Nexacro Emulator
nexacrostudio.exe	- Executable of Nexacro Studio
nexacromoduledeveloper.exe	- Executable of Nexacro Module Developer
caddy.exe	- Executable of the local web server



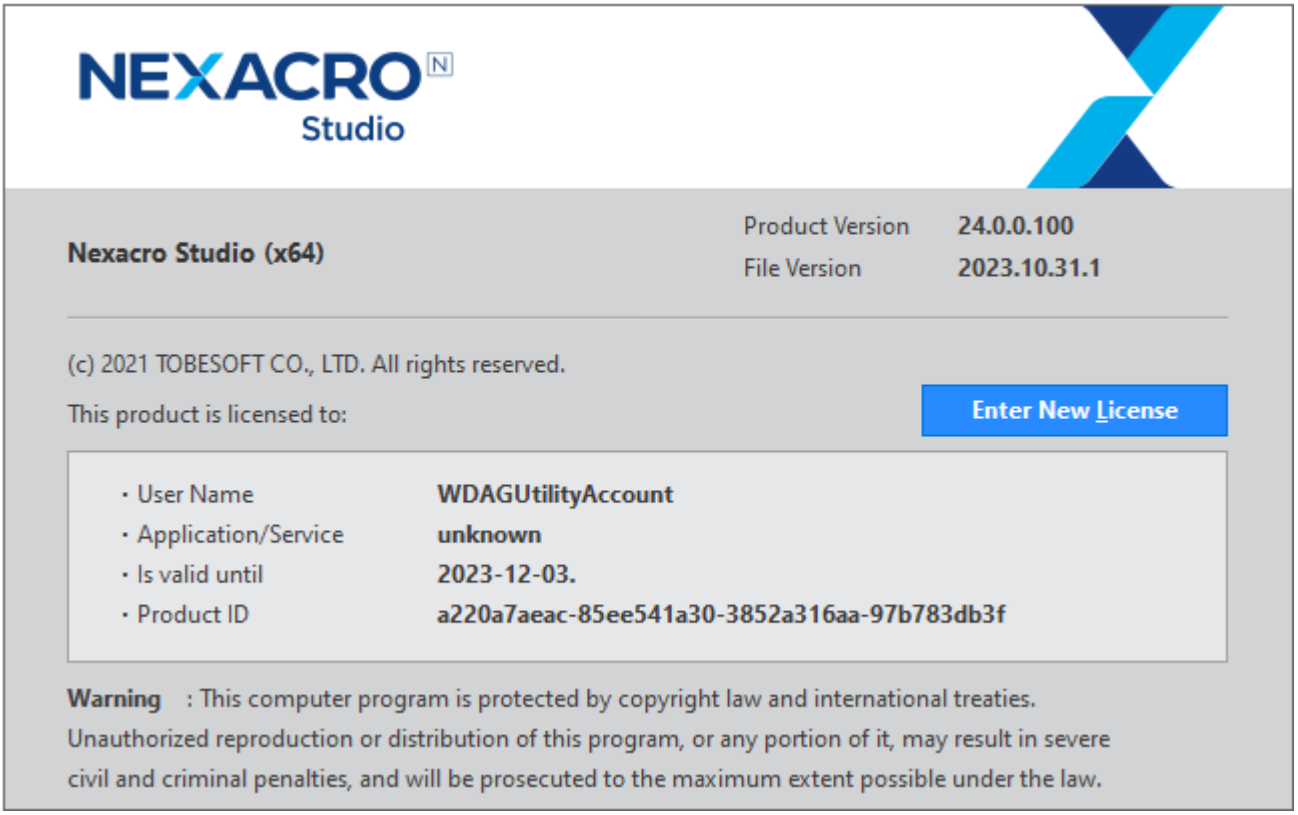
The list of installed files may vary depending upon product version.



If you are using a 64-bit operating system, the 64-bit executables in the SDK folder will also be installed in the %ProgramFiles% folder.

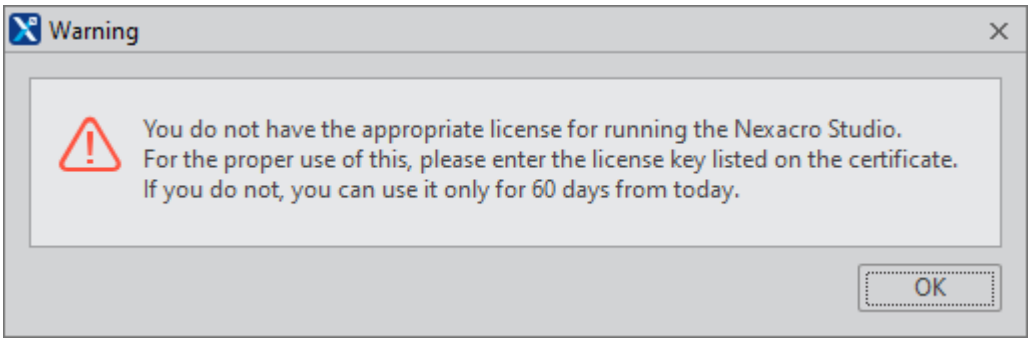
2.2.2 Version Check

To check the information on the registered license and version of Nexacro Studio, select **[Help > About Nexacro Studio]** from the menu.

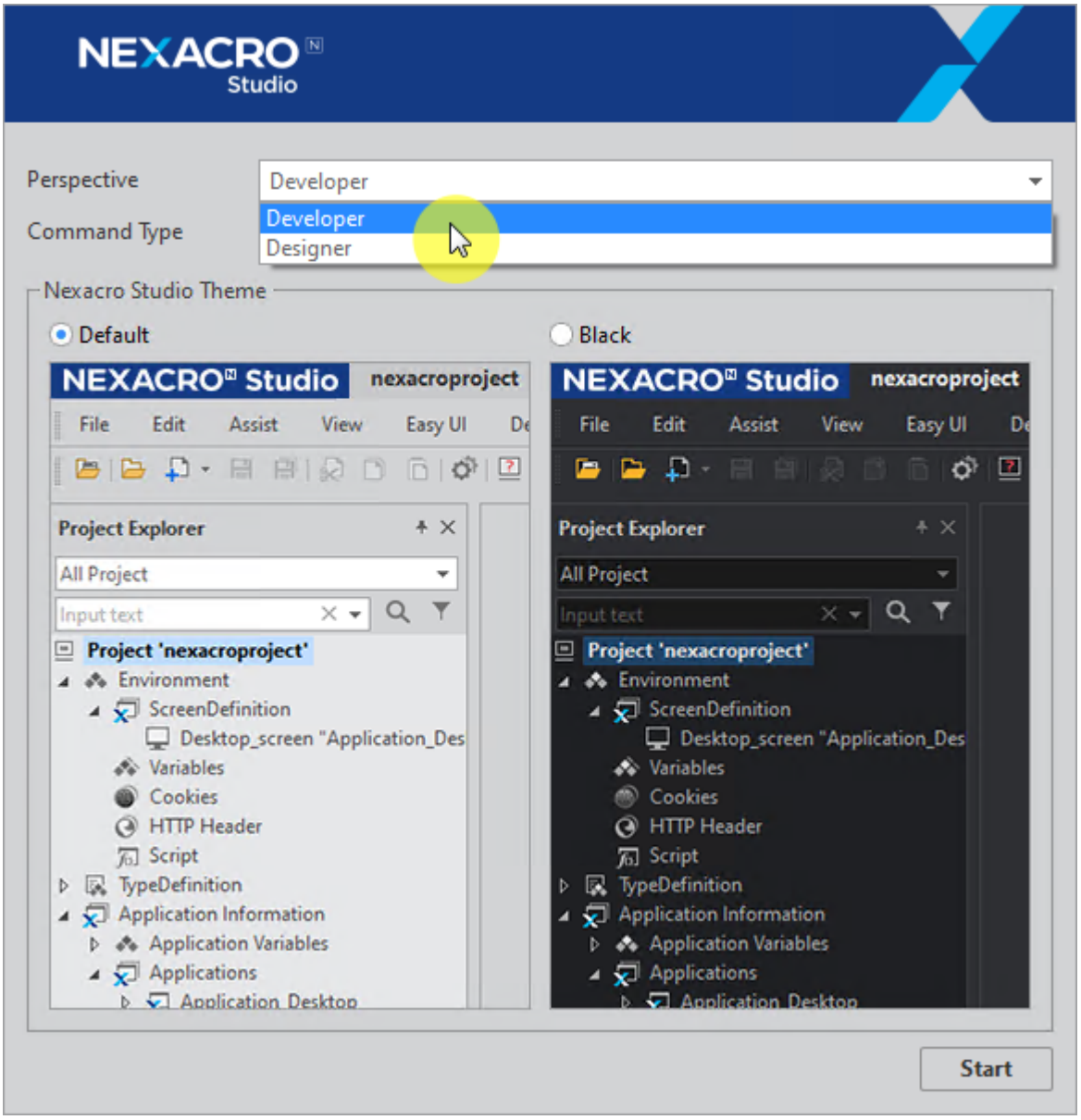


2.2.3 Initial Settings

When you install Nexacro Studio for the first time, the tool will run as a trial version, which lasts for 60 days. With the below warning dialog popping up for the first run, click the [OK] button to proceed to the next step. If you have purchased the product, enter your product key.

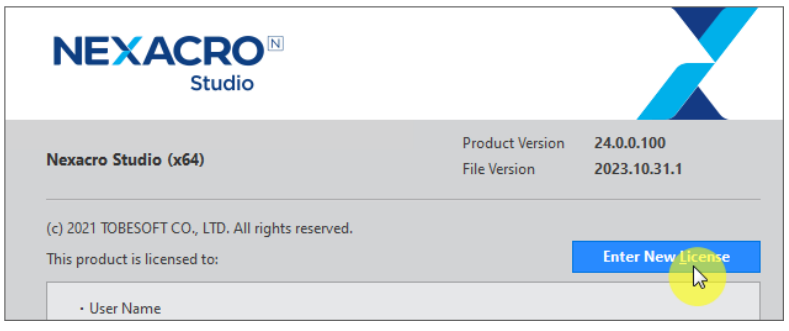


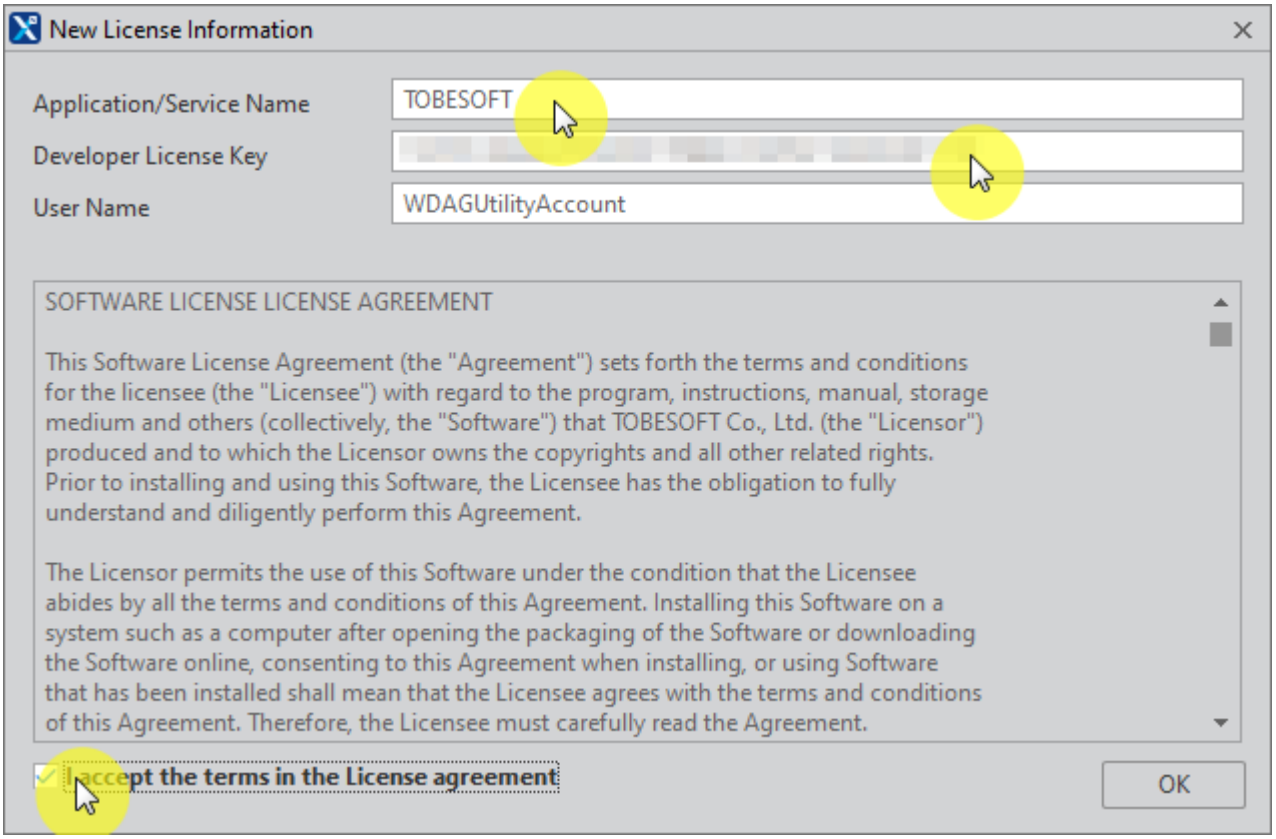
In the next step, you are asked to configure initial settings including Perspective, Command Type and Theme. You can adjust these settings later in the menu [Options > Environment > General > Development Tools].



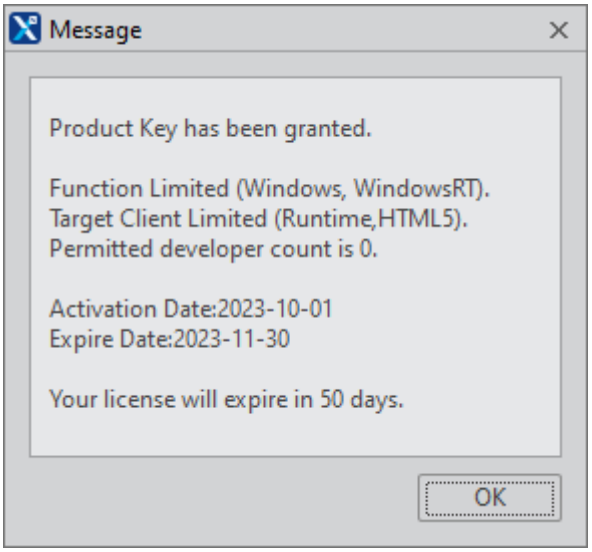
2.2.4 Licence Authentication

Access **Help > About Nexacro Studio** and click **Enter New License** on the version check window. When a pop-up appears, you are required to enter **Application/Service Name** and **Developer License Key**.

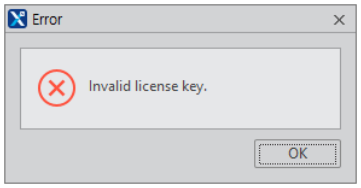




If a correct key is entered, you can see the below message and authentication is done.



If you put your key incorrectly, or the product key is found to have been already approved, the authentication will be failed with the below message on show.



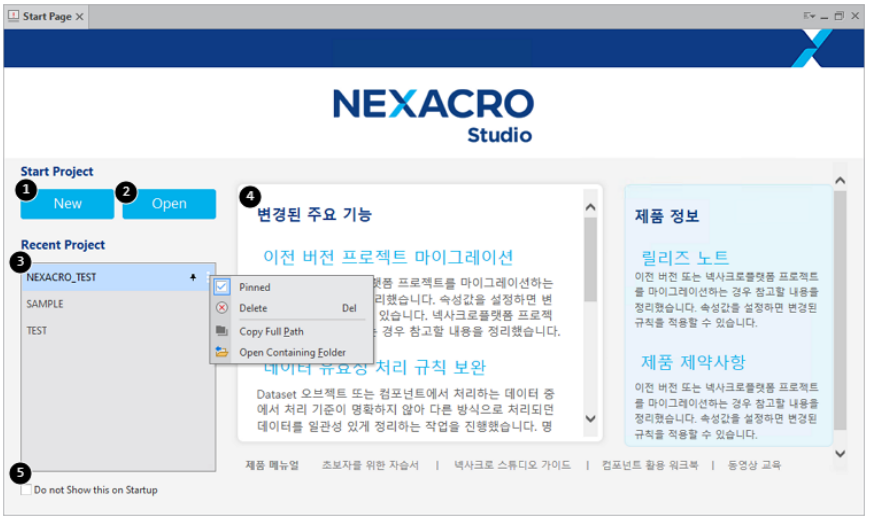
2.3 Install Nexacro SDK

Only Nexacro SDK can be installed or updated without updating Nexacro Studio. Execute the distributed setup file (NexacroN_SetupSDK.exe) to start the installation process.



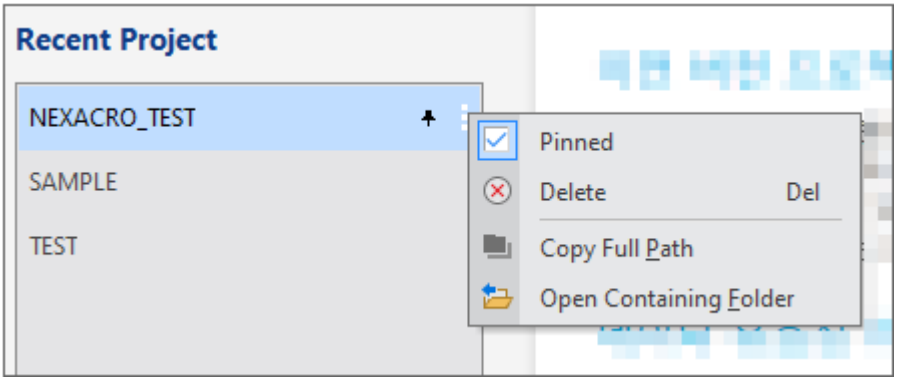
2.4 Start Page

Nexacro Studio provides a start page when it begins. The page can also be accessed through the menu **Help > Start Page**.



	Items	Description
❶	New Project	Creates a new project. Equal to the menu **File > New > Project**.
❷	Open Project	Opens an existing project. Equal to the menu **File > Open > Project**.
❸	Recent Projects	Recent project list. Equal to the menu **File > Recent Projects**. The list shows only project names while displaying the file path for each project when you put the mouse pointer over a certain project name
❹	Contents	Information on new features, product manuals and tip videos
❺	Do not show this on Startup	Whether to display the Start Page when Nexacro Studio begins. If you check the box, Nexacro Studio will not show the Start Page anymore.

You can pin frequently-used projects to Recent Projects through either of the two following ways. The one is selecting the checkbox beside **Pinned** in the context menu while the other is clicking the pin icon next to a certain project on the list. Pinned items are placed at the top of the list.



You can determine the number of items displayed in the Recent Projects list by accessing the menu [Tools > Options > Environment > General > Recent Files]. With the maximum number of the projects in the list amounting to 16, unpinned projects will be removed in the order that they were added if the total number of projects exceeds the maximum. You can delete a certain project in the list by selecting **Delete** in the context menu.

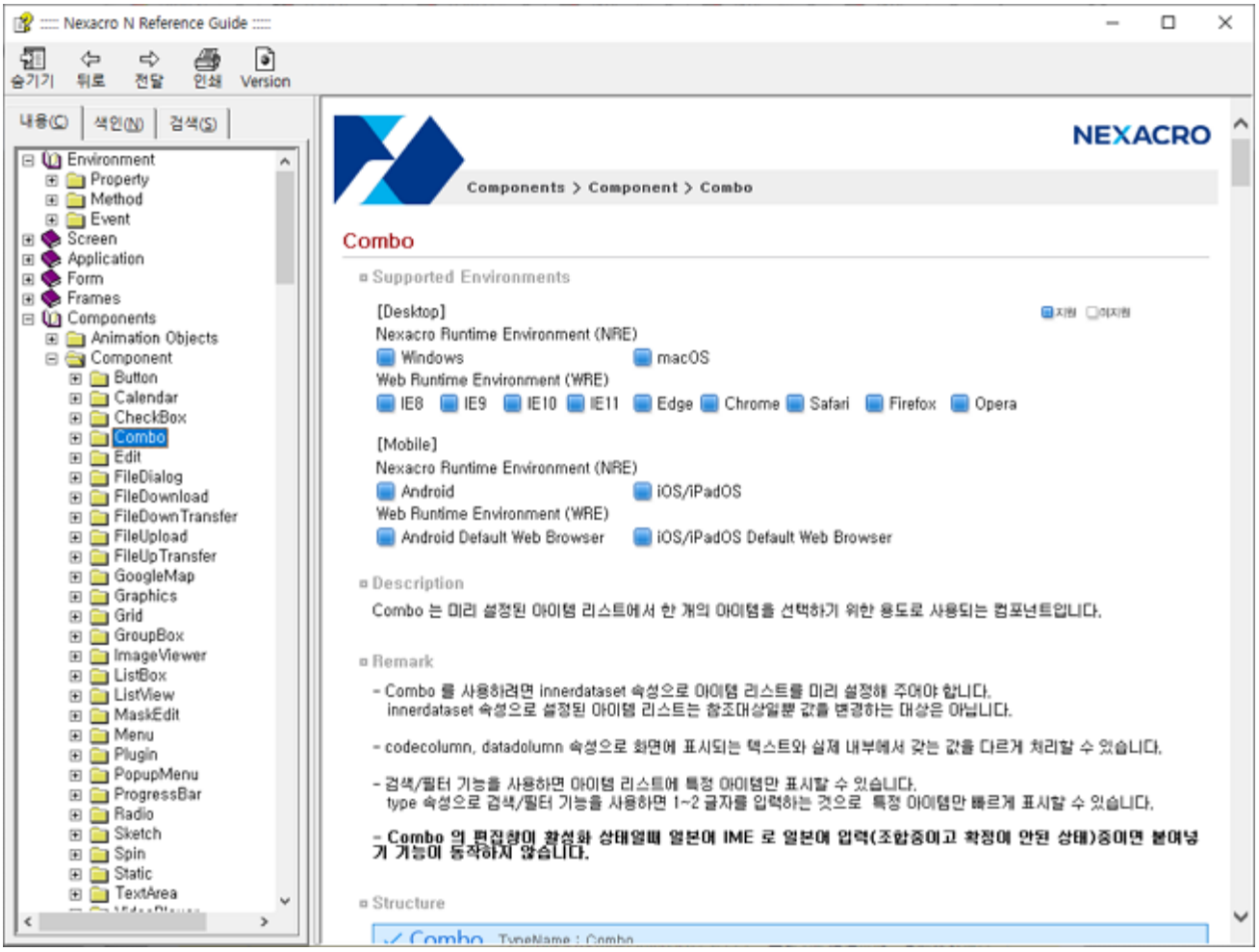
2.5 Using Help

To access the online help, press **F1** or select the menu option **Help – Help**.

The following table lists sample help categories.

Item	Description
Environment	Object that manages the information on the system environment utilized by a project
Screen	Information on the screens utilized by an Application
Application	Application object
Form	Form object
Frames	MainFrame, ChildFrame, FrameSet and etc.
Components	Component: components that serve as user interfaces in an app ContainerComponent: components to connect to a Form or to include other components inside System Component: components that appear to be included in another component but that actually have their own controls
NexacroAPI	Methods for additional features. Those methods begin with the word "nexacro" such as "nexacro.base64Decode()".
Communication Objects	Object used for data transaction. It supports the XPush object.
Data Objects	Dataset, DataObject and ColumnInfo objects
Script	Necessary objects including the ones created by supplementing standard JavaScript objects
Event Objects	Event objects
EventInfo Objects	Objects that help to process events
Misc. Objects	Objects, such as System, used to support certain features. Supported objects may differ according to users' environments.
DeviceAPI	Objects that support the NRE-specific features
Appendix	Various supplements including open source licenses

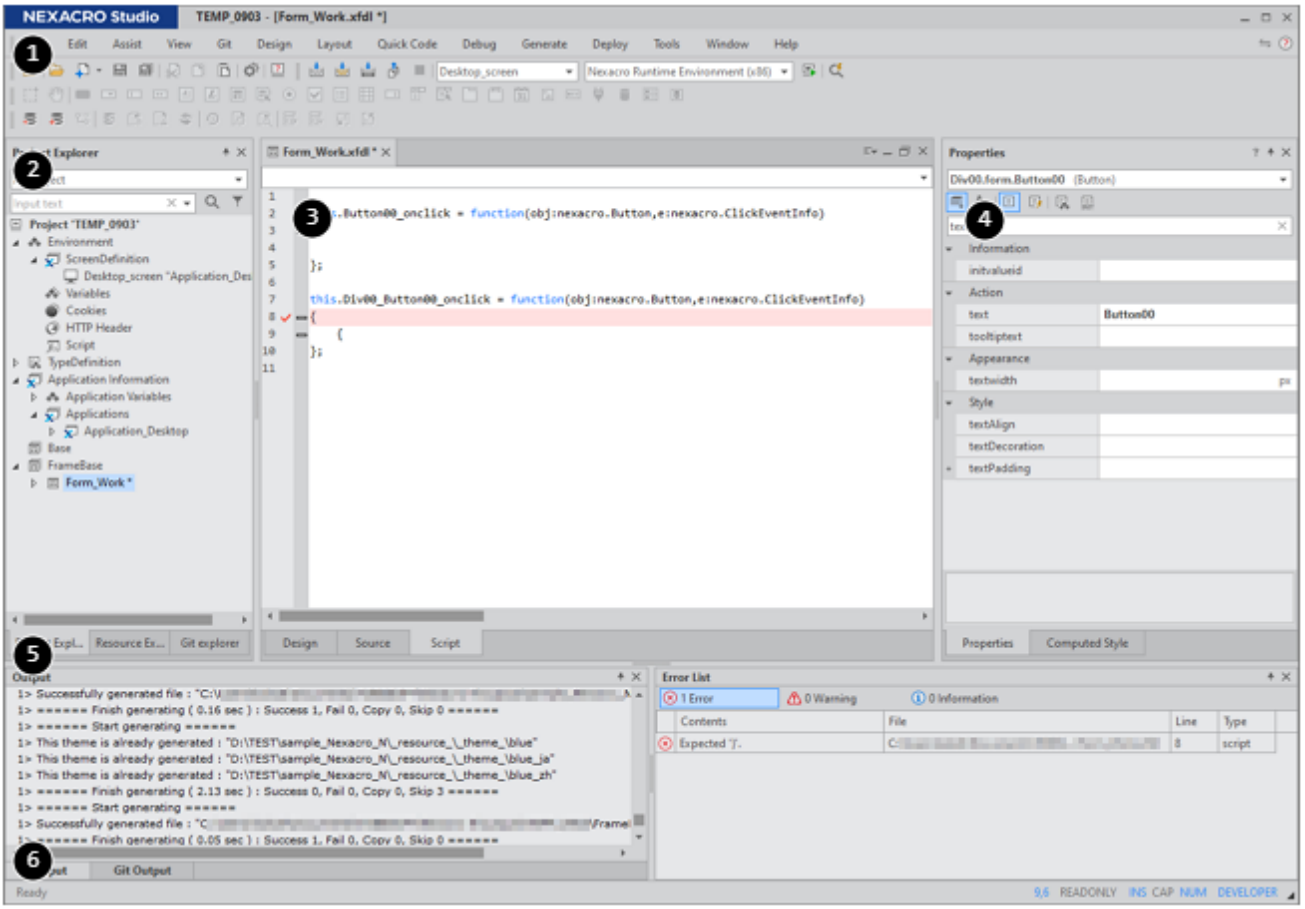
The following figure illustrates sample Help content.


Nexacro N Reference Guide
숨기기
뒤로
전달
인쇄
Version
내용(C)
색인(N)
검색(S)
Environment
Property
Method
Event
Screen
Application
Form
Frames
Components
Animation Objects
Component
Button
Calendar
CheckBox
Combo
Edit
FileDialog
FileDownload
FileDownTransfer
FileUpload
FileUpTransfer
GoogleMap
Graphics
Grid
GroupBox
ImageViewer
ListBox
ListView
MaskEdit
Menu
Plugin
PopupMenu
ProgressBar
Radio
Sketch
Spin
Static
TextArea
NEXACRO
Components > Component > Combo
Combo
Supported Environments
[Desktop]
Nexacro Runtime Environment (NRE)
Windows
macOS
Web Runtime Environment (WRE)
IE8
IE9
IE10
IE11
Edge
Chrome
Safari
Firefox
Opera
[Mobile]
Nexacro Runtime Environment (NRE)
Android
iOS/iPadOS
Web Runtime Environment (WRE)
Android Default Web Browser
iOS/iPadOS Default Web Browser
Description
Combo 는 미리 설정된 아이템 리스트에서 한 개의 아이템을 선택하기 위한 용도로 사용되는 컴포넌트입니다.
Remark
- Combo 를 사용하려면 innerdataset 속성으로 아이템 리스트를 미리 설정해 주어야 합니다.
innerdataset 속성으로 설정된 아이템 리스트는 참조대상일뿐 값을 변경하는 대상은 아닙니다.
- codecolumn, datadolumn 속성으로 화면에 표시되는 텍스트와 실제 내부에서 갖는 값을 다르게 처리할 수 있습니다.
- 검색/필터 기능을 사용하면 아이템 리스트에 특정 아이템만 표시할 수 있습니다.
type 속성으로 검색/필터 기능을 사용하면 1~2 글자를 입력하는 것으로 특정 아이템만 빠르게 표시할 수 있습니다.
- Combo 의 편집창이 활성화 상태일때 일본어 IME 로 일본어 입력(조합중이고 확정이 안된 상태)중이면 본에넣기 기능이 동작하지 않습니다.
Structure
Combo TypeName : Combo

3.

UIs of Nexacro Studio

The most basic tasks you can do through Nexacro Studio constitute drawing a Form and writing codes to program how the Form works. To facilitate such tasks, the Studio offers various features including the clear-cut provision of the information on the properties held by the components on a Form.

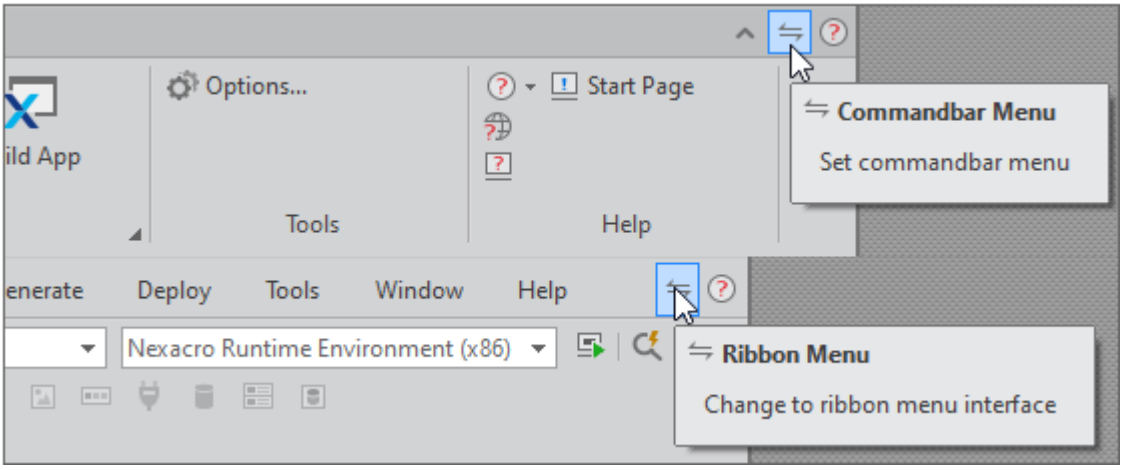


Screen Area		Description
1 Menu, Tool Bar Ribbon interface		Toolbar that controls the menu area, TextStyle, Alignment, and Object.
6 Status Bar		Status bar indicating Nexacro Studio's current state
View	3 Form Design	Editing Design, Source, Script
Dock Windows	2 Explorer	- Project Explorer - Resource Explorer

Screen Area		Description
	❹ Properties	Displays and edits Forms and various components, and associated properties such as Dataset and event properties
	❺ Message Window	Displays various messages and search results

3.1 Menu, Toolbar and Ribbon Interface

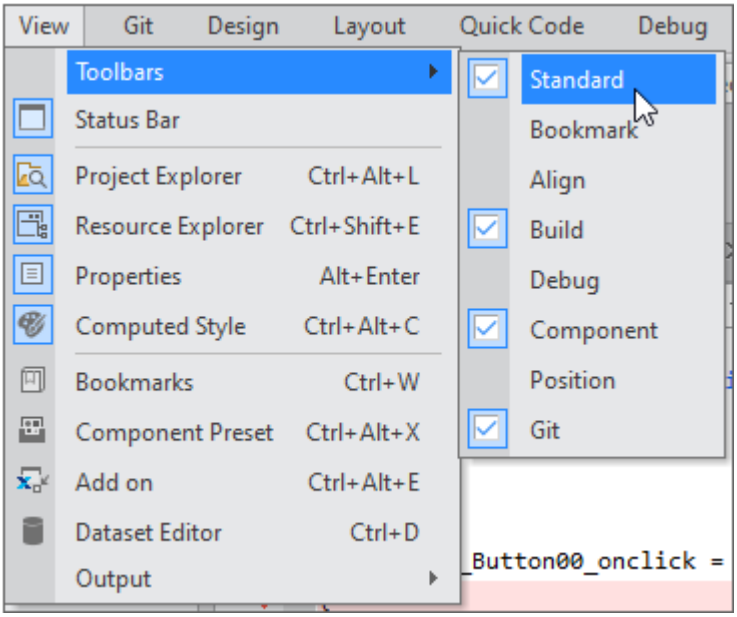
This is an area that accommodates the menu bar as well as either the toolbar or the ribbon bar. You can switch between the toolbar and ribbon bar based on your choice.



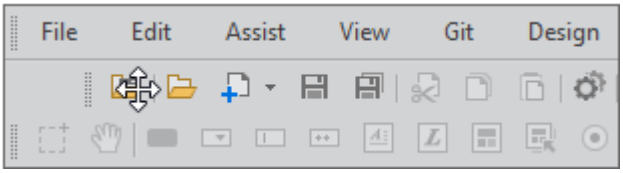
The menu bar and toolbars display or hide their member items according to the status of the relevant project and file.

3.1.1 Menu bar and Toolbar

You can select the display of a certain toolbar by accessing the menu [View > Toolbars]. Checked toolbars will be displayed. You can do the same with the context menu opened by right-clicking the menu or toolbar area.

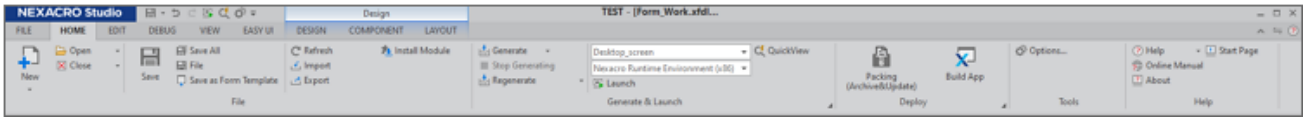


If you move your mouse to the front part of the toolbar, the mouse will change to a move type cursor. You can then click the area and drag it to another location.

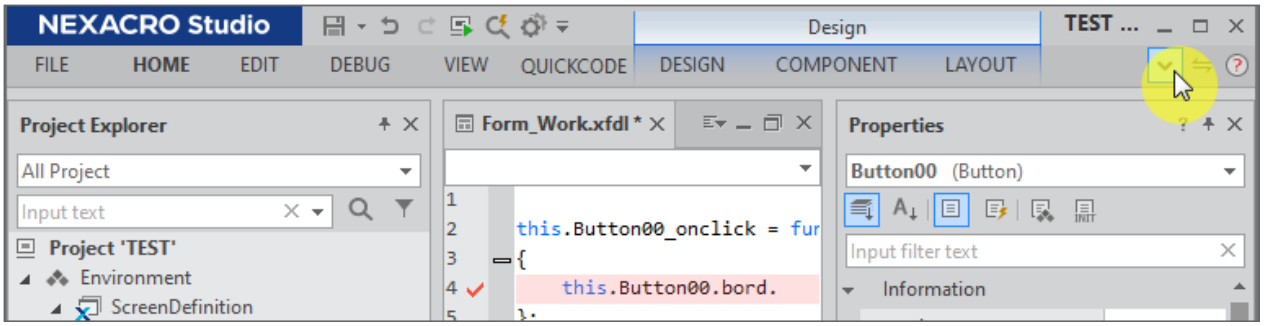


3.1.2 Ribbon Interface

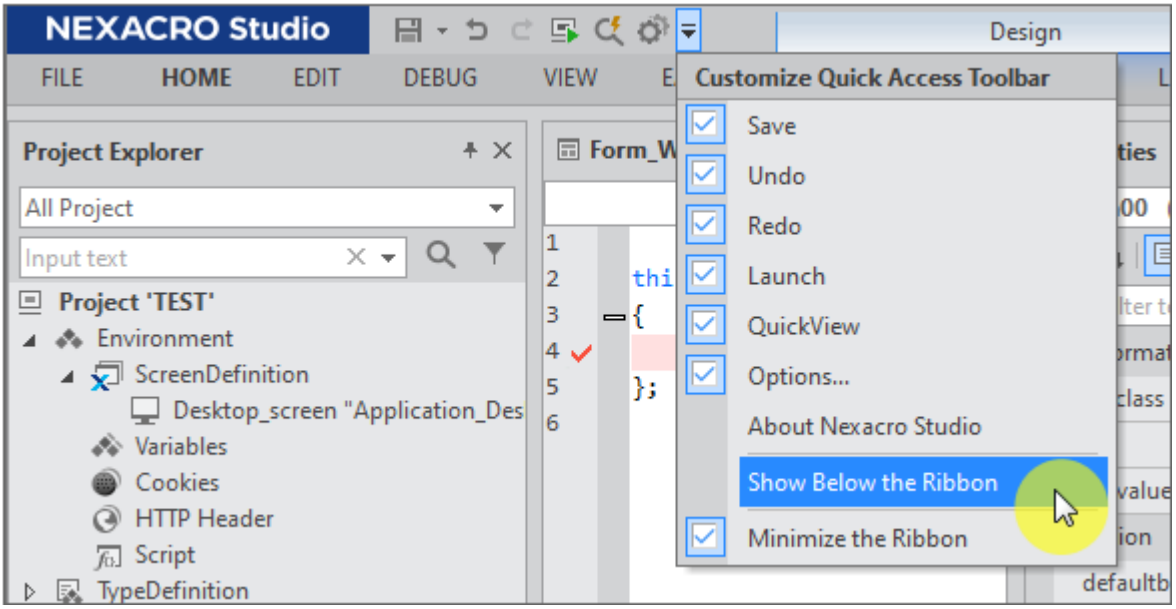
Ribbon menus are a default interface. All the menus under the same category are bundled into one ribbon tab. Clicking a menu in the menu bar will show buttons of relevant features under the bar.



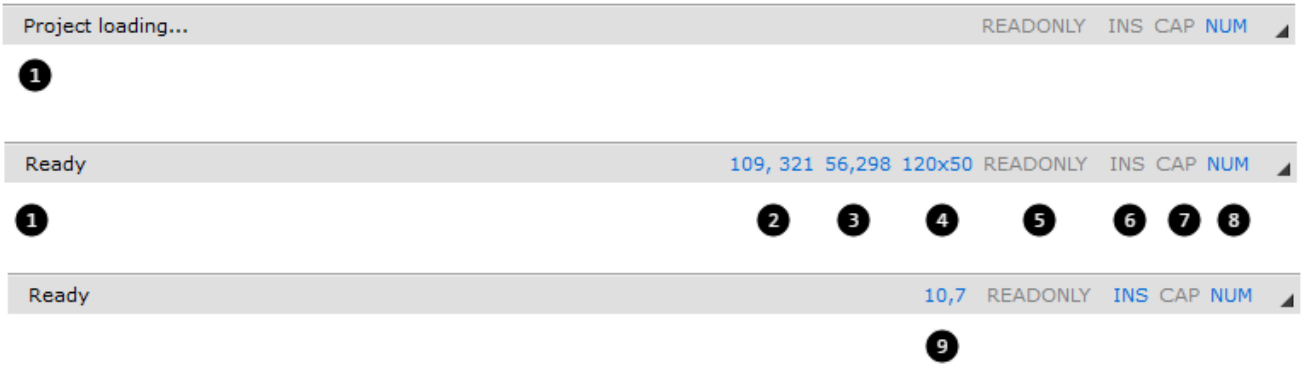
You can expand/hide the ribbon area to secure sufficient workspace.



The Quick Access Toolbar is a customizable toolbar that contains frequently-used commands. It can be situated above or below the ribbon area. The default location is above the ribbon, and select [**Show Below the Ribbon**] to bring it down.



3.2 Status Bar



Different status values are displayed in the Nexacro Studio status bar.

	Function
❶	Message indicating the state of Nexacro Studio - Shows the progress when a project is opened or closed. - Shows the description about a certain menu item when you stop a mouse pointer over the relevant item.
❷	Current coordinates of the mouse cursor
❸	Coordinates of the selected component, calculated within the Form (unit: pixel)

	Function
❹	Size of the selected component (unit: pixel)
❺	The status of the current file. If the file is read-only, the indicator word "READONLY" will light up.
❻	The status of the Insert key (INS, OVR). The indicator will light up if the cursor is placed on the Source or Script editor pane.
❼	On/Off status of the Caps Lock key
❽	On/Off status of the Num Lock key
❾	Location of the cursor when it is placed on the Source or Script editor pane

You can toggle on/off the status bar by accessing the menu [View > Status Bar].

3.3 View

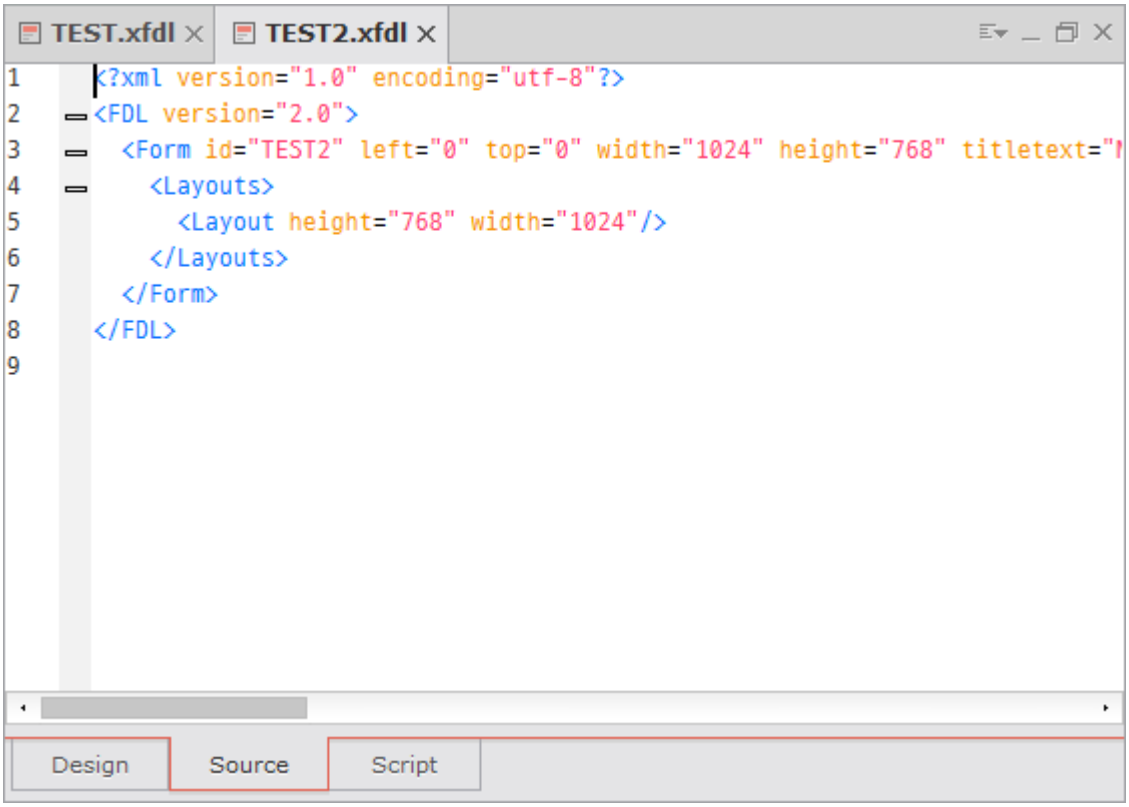
A 'view' is an area where users edit files included in a project. Views are arranged in the form of a multiple-document interface (MDI) and can be laid out at users' discretion.

3.3.1 Arranging Views

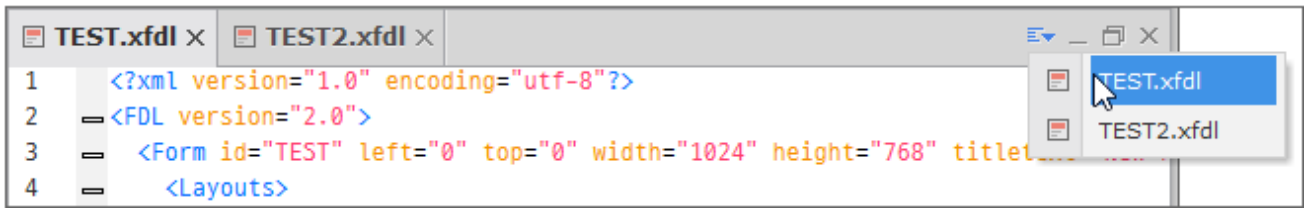
Views can be maximized or minimized while you can change the order of view tabs by dragging them.

Tabs

A view takes the form of a tab. Maximizing only one view will inevitably cause all the other views to be maximized.

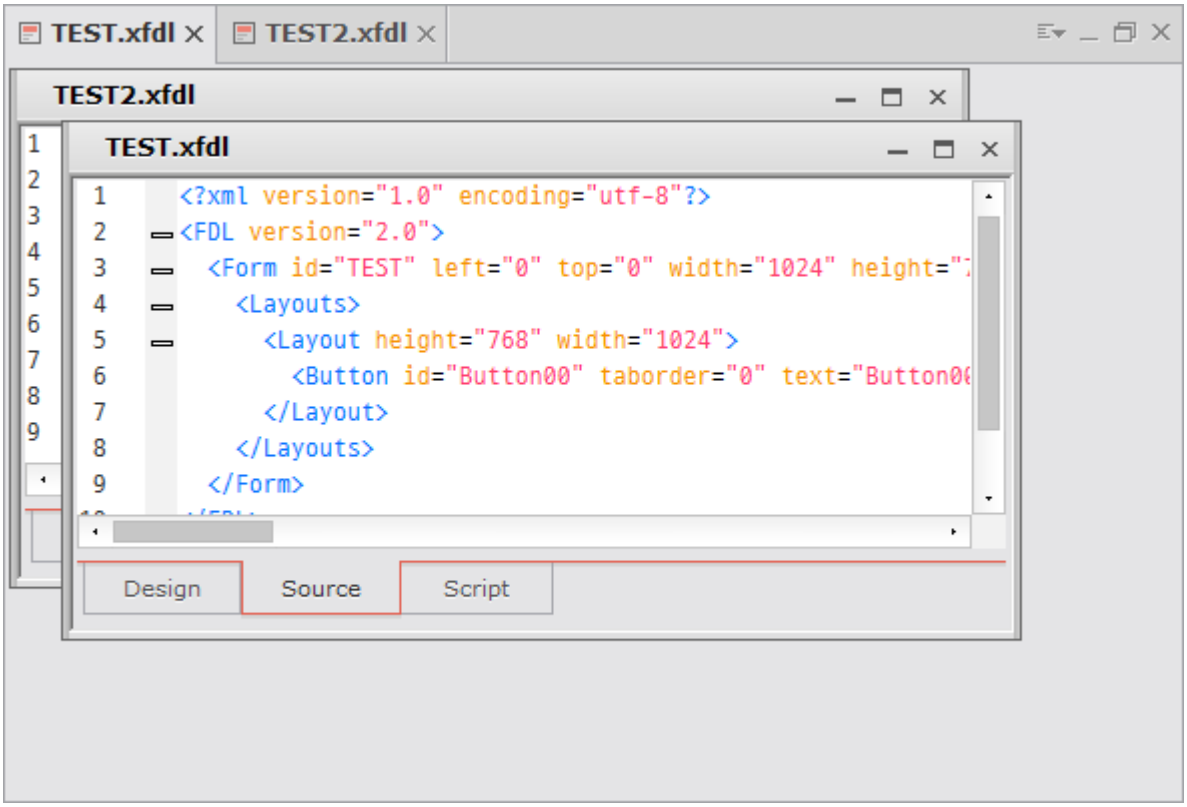


You can change active views by clicking tabs. Otherwise, click the **Active Files** button (a downward triangle icon on the right side of the tab area) and select a view that you want to activate.



Cascade Windows

To apply a cascade layout to the arrangement of views, access [**Window** - **Cascade**] in the command bar interface. Otherwise, right-click the title bar of a view window and select [**Cascade**] from the context menu.



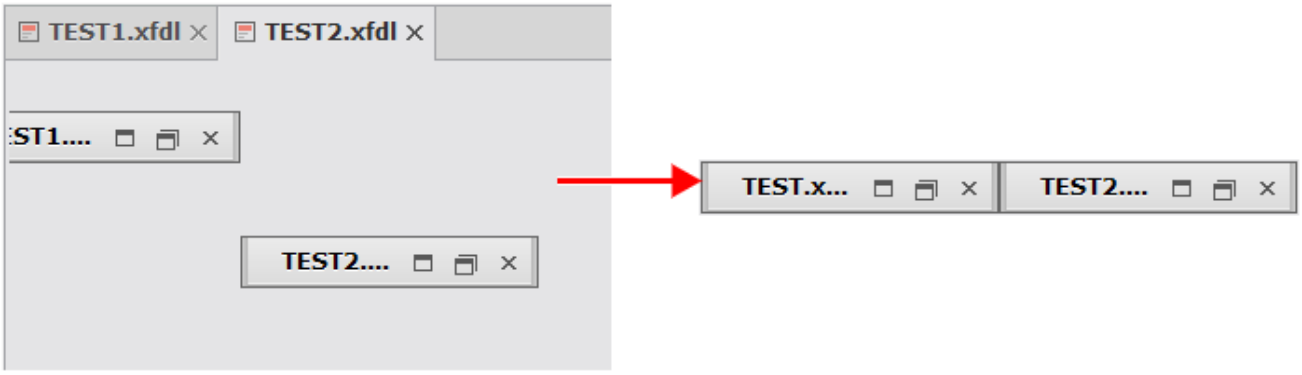
Tile Windows

To apply a tile layout to the arrangement of views, access [**Window** - **Tile**] in the command bar interface. Otherwise, right-click the title bar of a view window and select [**Tile**] from the context menu.



Arrange Icons

Views can be minimized into icons and freely move within the Form Design pane area. When those icons are scattered around, you can arrange all of them on the bottom left corner of the pane at once by accessing the menu [**Window** - **Arrange Icons**]. Otherwise, right-click one icon and select [**Arrange Icons**] from the context menu. You cannot apply This function to non-minimizable views.



3.3.2 Form Design

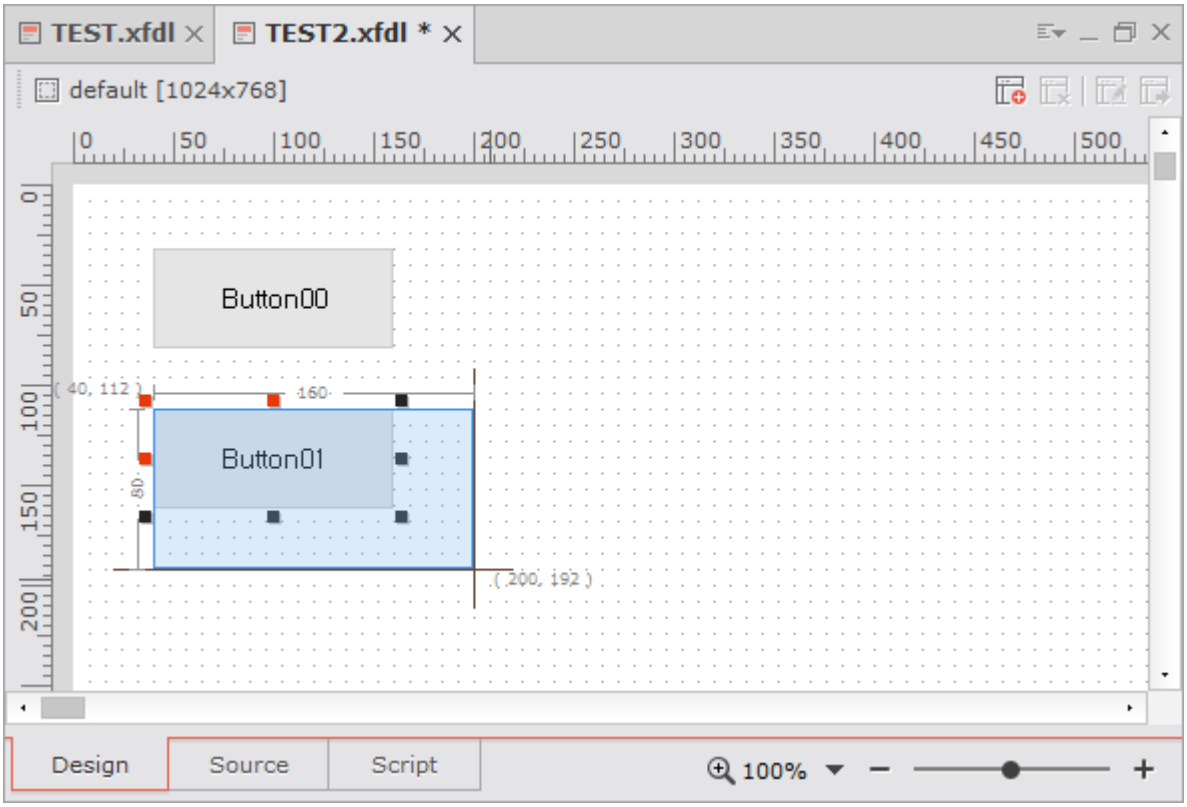
Use the Form Design window to create and edit the screens of an app. It consists of the Design Window, Source Editor, and Script Editor tabs displayed at the bottom of the Form Design Window.

The activation status of a toolbar can change according to various conditions such as the type of editing mode and which object is selected.
For example, the Align toolbar will not be activated until users select the components in the Design window. Furthermore, each item in the Align toolbar may or may not be activated depending on the aspects of selecting components.

Design Window

Access the Design window, by selecting the Design tab at the bottom of the Form Design window.

You can arrange components anywhere in the Design Window by cutting, copying, pasting, and moving them. The Form Canvas area where you arrange components can be changed to whatever the developer wants.



Source Ediitor

Access the Source Editor window by selecting the Source tab at the bottom of the Form Design window.

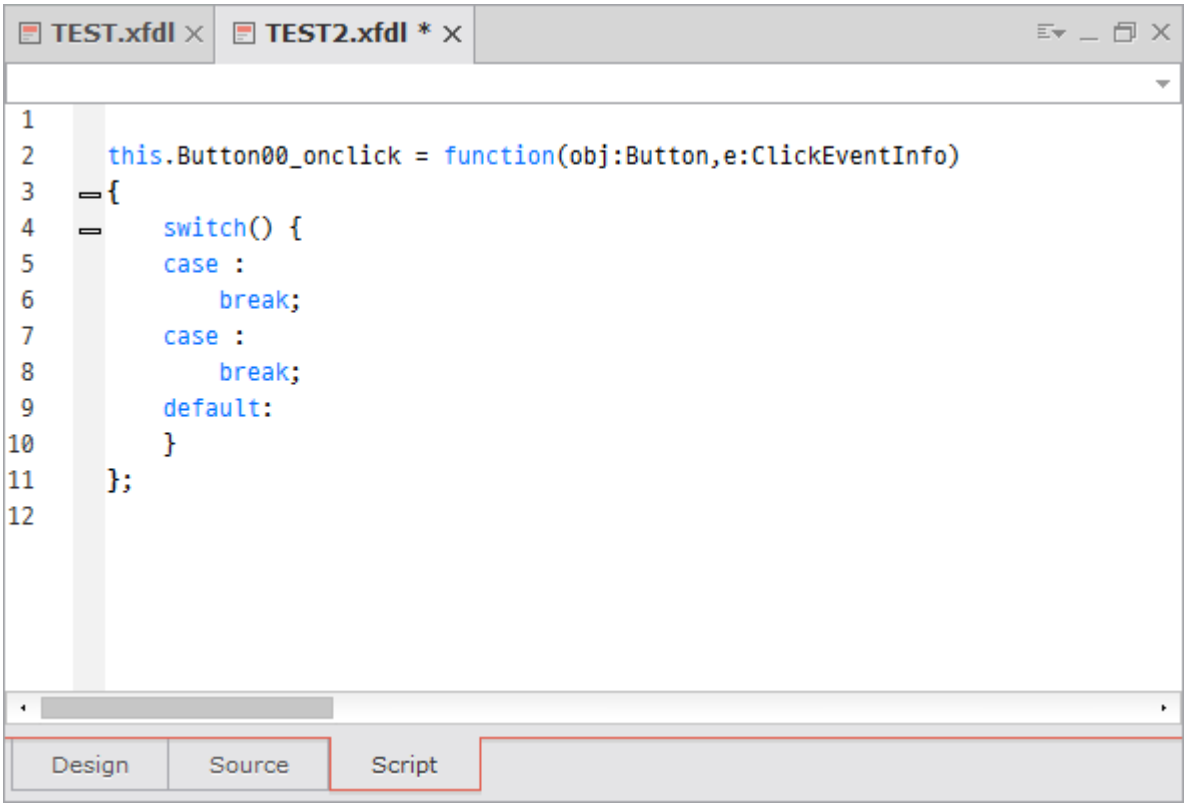
This window shows all form content except for script content in XML format, where it can be edited directly.



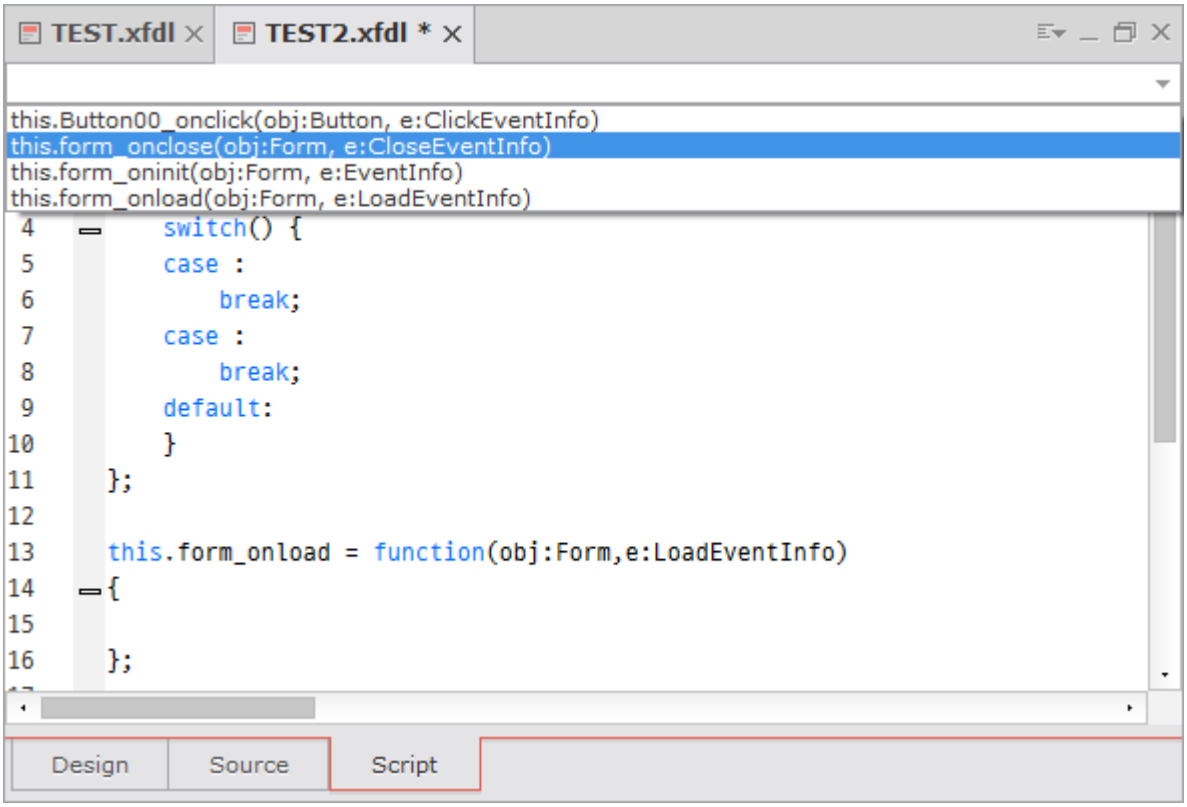
XML source codes will be created automatically when you arrange components or other objects on the Design window, change their properties, or add events. If you correct source codes directly through the Source editor, errors may occur since you can violate the predetermined object creation rules.

Script Editor

In the Script Editor Window, you can edit the Form's JavaScript components, such as event handling functions.



The functions and events currently in use are listed in the combo box on the upper part of the Script Editor window. Select an item to move to the code location.



3.4 Dockable Windows

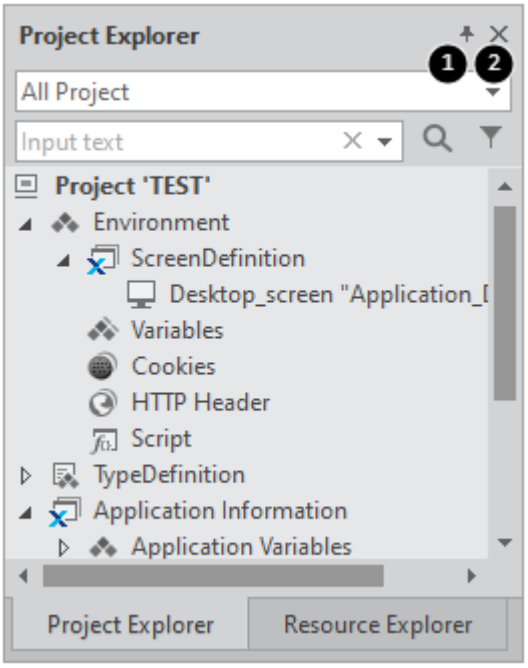
For common features that are used regardless of the types of menus and files, Nexacro Studio encloses them in dockable windows.

3.4.1 Rearranging Windows

All the windows of Nexacro Studio can be rearranged or hidden, except the title bar and Form Design pane.

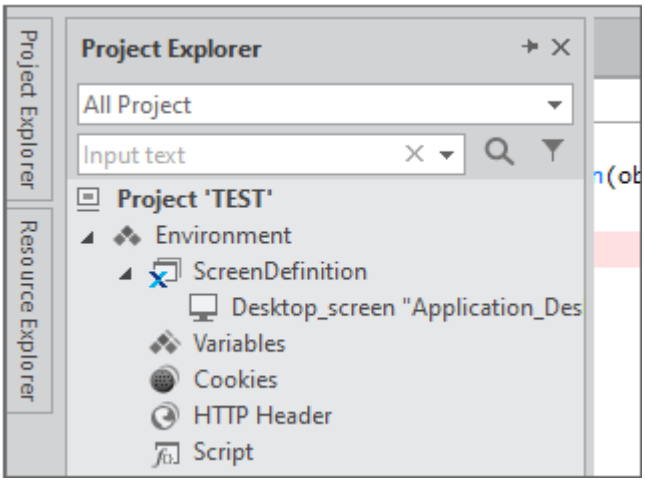
Displaying/Hiding Windows

In the Project Explorer window as shown below, notice that there are icons on the left of the title row. These icons can be used for closing or hiding the corresponding functions.



Callout	Function
❶	It automatically hides a relevant window depending on the setting. When the window is hidden, the window's name is displayed as a tap on the right/left side of the screen. The window will appear again when you move the mouse over the tap or when you choose the window on the [View] menu. Depending on the setting, the icon will be changed as follows. When it is set to fix a window When it is set to hide a window automatically
❷	It closes a relevant window

If there are several layers of windows, the windows can be hidden and displayed by tapping on the right/left side when they are set to be hidden automatically.

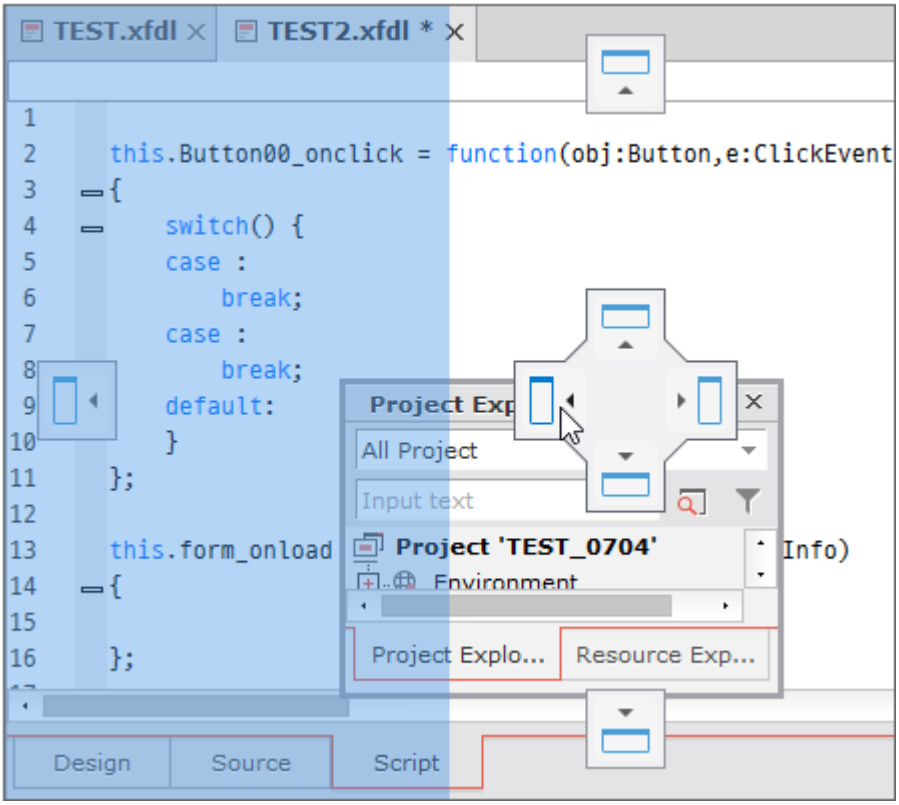


Moving Windows

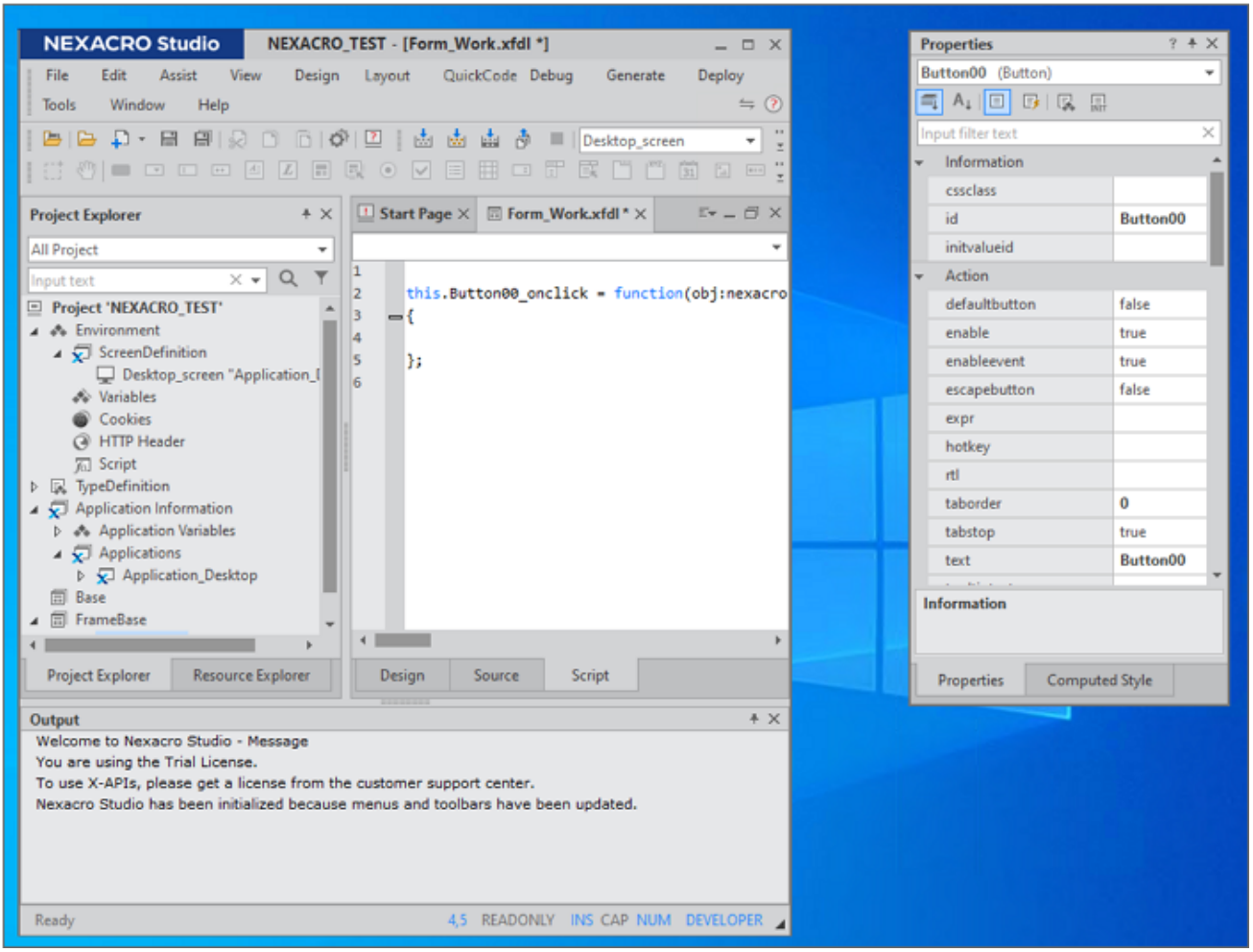
You can move a window anywhere on the Nexacro Studio interface or on your monitor. If windows overlap each other, simply tap on the one you want to pull to the front of the display.

Changing Location

A window can be moved by right-clicking the title bar of the window and dragging it to where you want to place it.



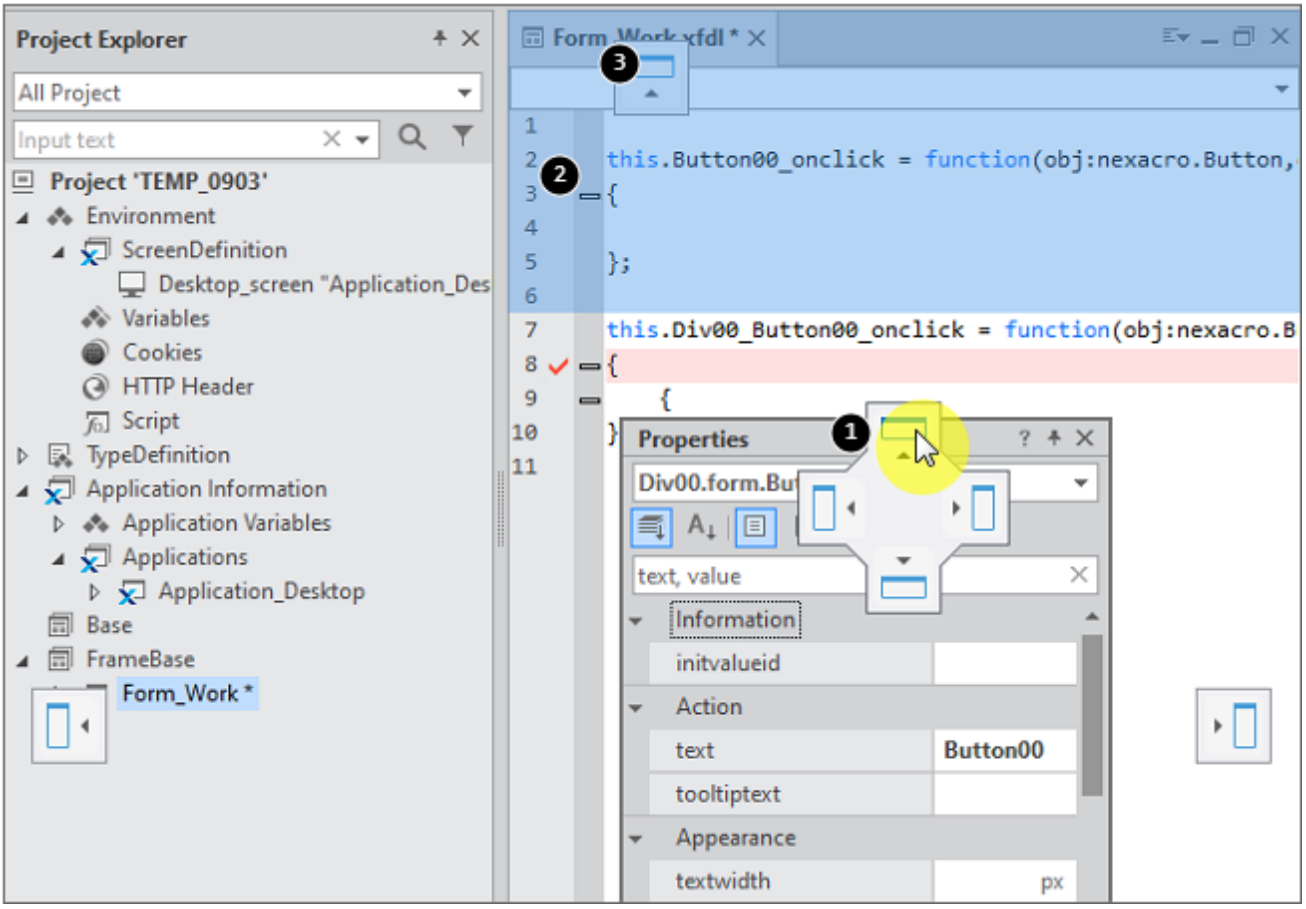
For example, if you use two monitors, you can place the Form design window on the left monitor and the properties window on the right monitor.



Docking Windows

You can also dock a window in the Nexacro Studio UI to another window. You can arrange windows anywhere except the areas for the tool bar and the status bar. Also, you cannot move the Form Design Window.

While dragging a window to another location, if a mouse pointer is placed over another window, you will see an icon that guides you to choose where to re-dock the moving window.

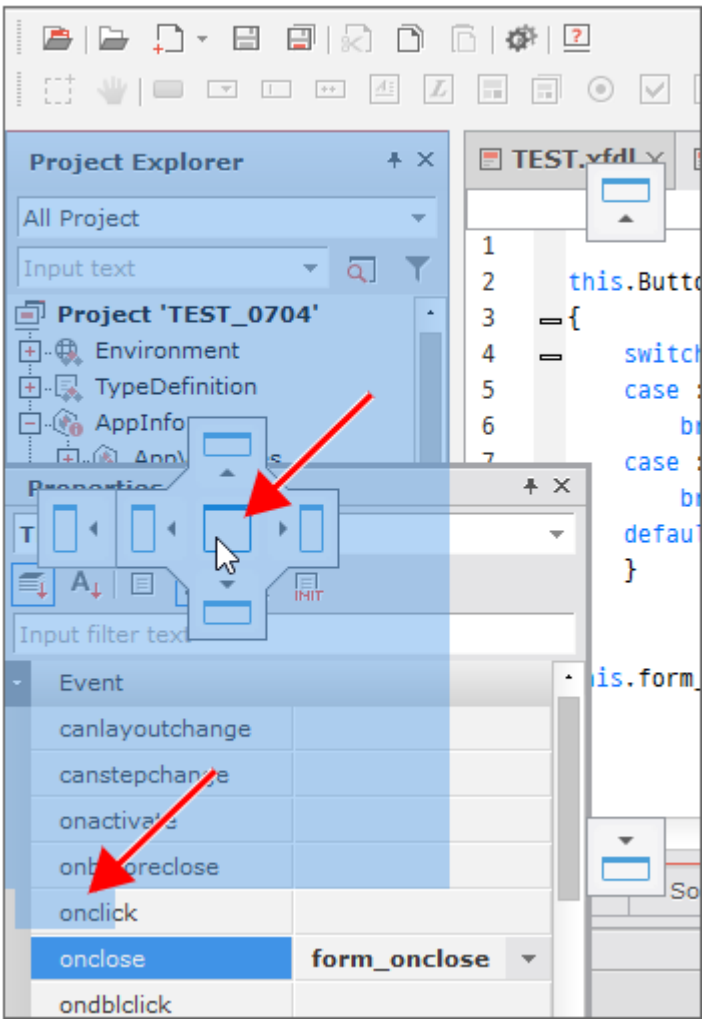


Callout	Docking Function
❶	Based on the window you are moving, the docking functionality determines the possible docking location. As soon as you place the mouse pointer on the direction option, the designated area will be marked as shown in callout ❷.
❸	Not based on the corresponding window, the docking function determines a location for the window being moved on the whole screen. In the example above, when you choose ❶, the properties window is located right above the Form design window. However, when you choose ❸, the properties window is placed right under the tool bar.

Combining Tabs

When re-docking a window, the window can be combined with other windows as a tab, except the Form Design window. A window can be combined with the Form Design window by placing it outside the Form Design window.

Except for the Form Design window, you will see an additional option which tells you that you can choose the middle of the icon when you combine a window with other windows. When you move the pointer to the corresponding area, the area to be combined is highlighted in the corresponding window and you will see the tab on the bottom of the window.



You can move all tabs of the window which includes the combined tab by dragging the title bar of the window. Also, you can move a single tab included in the window by dragging the corresponding tab on the bottom of the window.

4.

Workspaces | Explorers, Properties, Message

Largely, Nexacro Studio has Project Explorer, Properties Windows, Message Window and Debug Window as dockable windows. You can check available dockable windows at [**View**] in the command bar interface or at [**VIEW - Panes**] in the ribbon interface.

4.1 Project Explorer

In the Project Explorer window, all items included in the opened project are displayed in tree form.

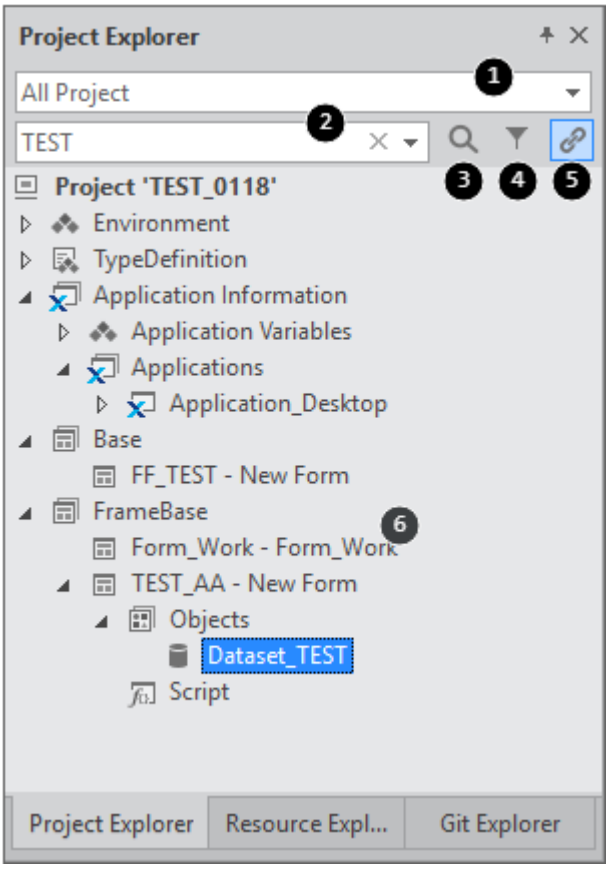
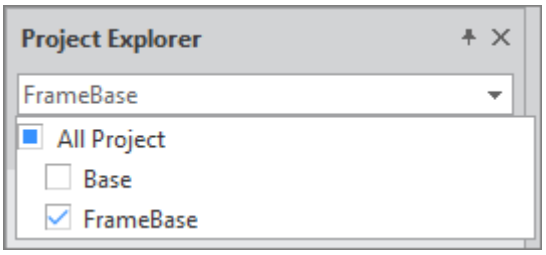


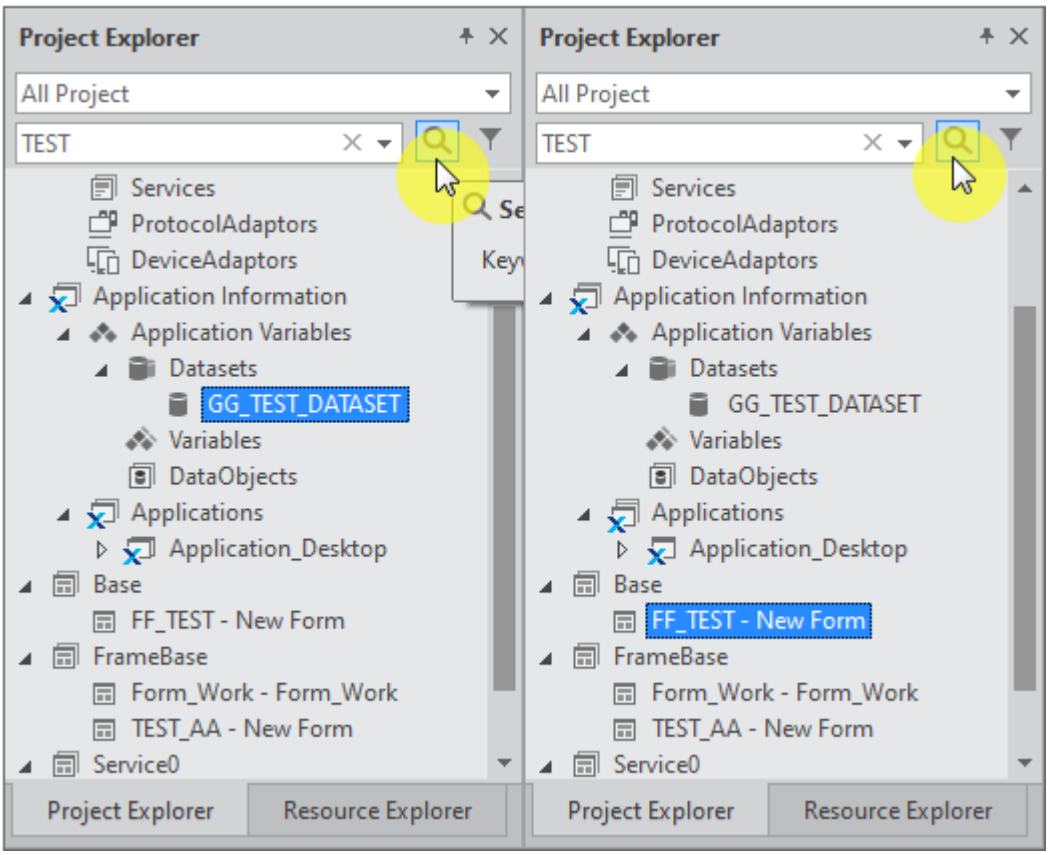
Table 4-1 Project Explorer

Callout	Description
❶	Select a service (or entire project) to apply the search and filter functions ▼ When the button is clicked, the list of services in the project is displayed, and services to be excluded from search and filter can be unchecked.  Environment, Type Definition, and Application Information cannot be excluded from search and filter targets.
❷	Search keyword input window to apply the search and filter functions Enter a search string and press the **Enter** key or **F3**. (It is not case-sensitive)
❸	Sequentially display results identical to the value entered in the search box.
❹	Display only filtered results corresponding to the value entered in the search box. If you click the filter icon after entering a search keyword in the search key input window, then only files containing the character string that matches the search keyword or services containing files will be displayed on the screen.
❺	Choose whether to enable Link with Editor Please refer to the Advanced option items.
❻	Display files and other constituents that are included in a project.

4.1.1 Search

The search for files and sub-elements contained in the project is processed. After entering the search keyword, press the enter key (or F3 key) to proceed with the search immediately, and to proceed with the next search, press the F3 key.

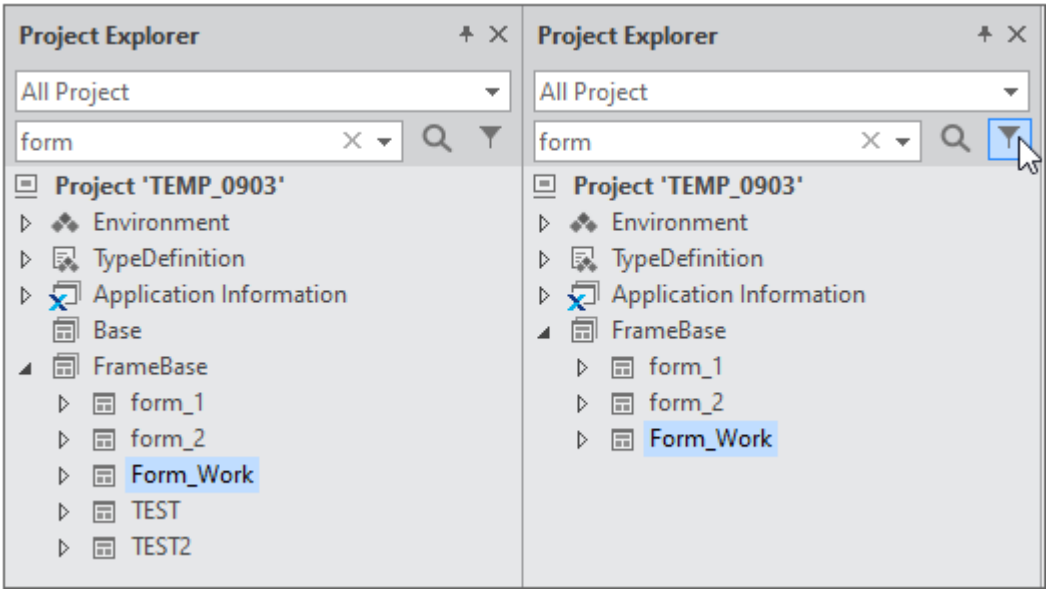
Enter the search keyword 'TEST' and click the search icon to select items containing ‘TEST’ from the Dataset object of Application Variables and from the Base service in order.



4.1.2 Filter Function

Use the filter icon to display filtered files containing an entered item among properties and files in the Service.

in the following example, no form files are currently open. Entering the search word "form" and clicking the Filter icon displays the Form files whose names include "form," but none of the components in those Forms.



By naming files based on tasks in the project, the Filter function can be useful for easily finding files relevant to your task.

If form files included in the service are not opened when you use the Project Search and the Filter, filtering only applies to form file names. However, after form files are opened, it applies to layouts and script-related properties included in the Forms.

Project search targets the items, files, and items included in files in the entire project, but the filter function only targets files created by the user within the service.

4.1.3 Copy

If you copy and paste an item in the Project Explorer or Resource Explorer, a copy of the item will be created with the string "_copy" attached to the end of the file name. The below table presents copyable objects.

Project Explorer	Form
Resource Explorer	InitValue UserFont ImageResource

4.2 Resource Explorer

The Resource Explorer displays the resources held by the current project. In addition, you can create new resources or import existing files through the explorer.

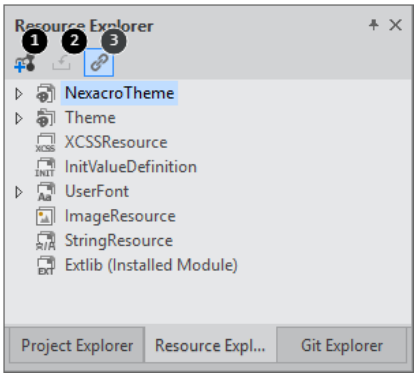
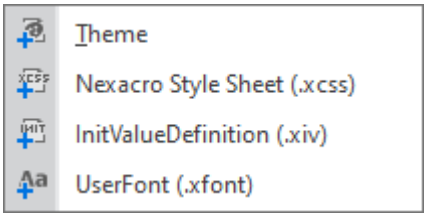
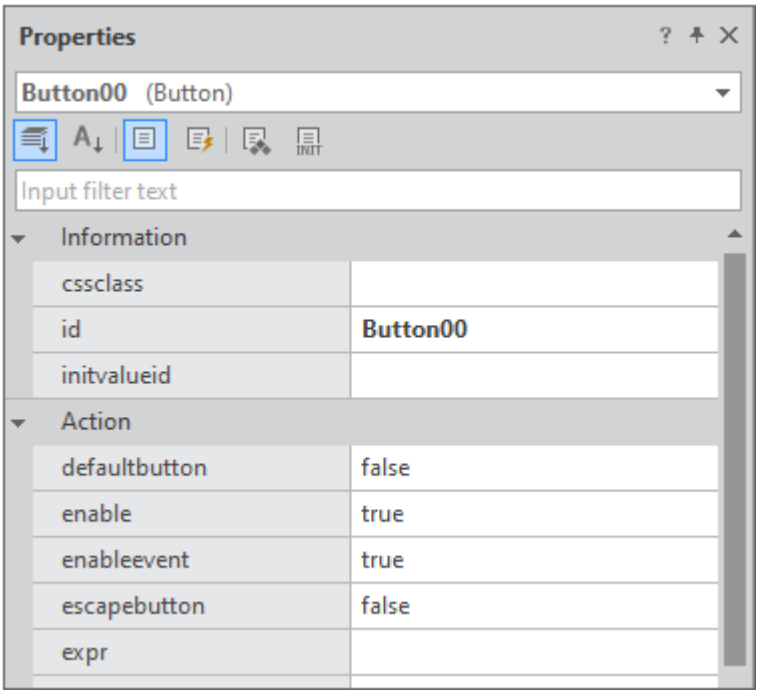


Table 4-2 Project Explorer

	Items	Description
1	New Resource	Creates a new resource file 
2	Import Resource	Adds the previously created resource file to the project or selects the xpackage file and runs the Import Wizard.
3	Link with Editor	Choose whether to enable Link with Editor Please refer to the Advancedoption items.

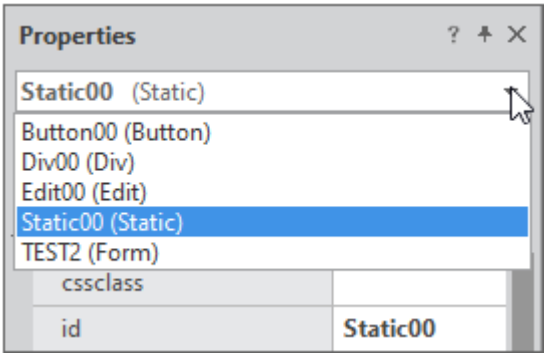
4.3 Properties Window

Select the Properties window to display and edit form components, Dataset properties, and Style Properties.



4.3.1 Object List

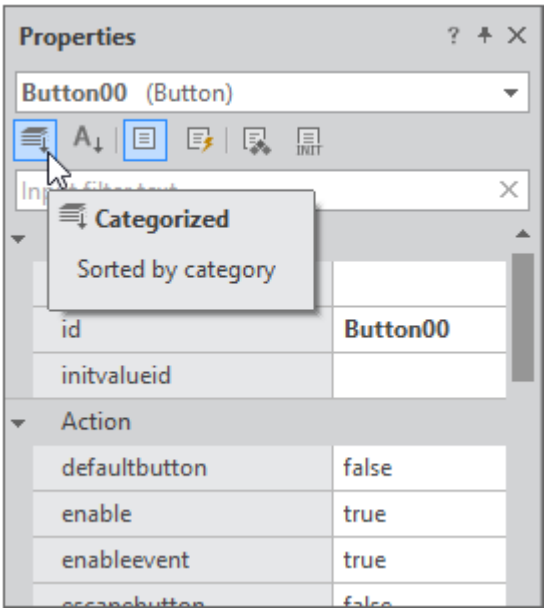
All components, Datasets, etc. included in the Form being edited are shown as a list of id values (object names). Depending on the selected object, it is displayed differently in the list of properties and events below. You can also select specific objects on the design screen.



4.3.2 Sorting Type

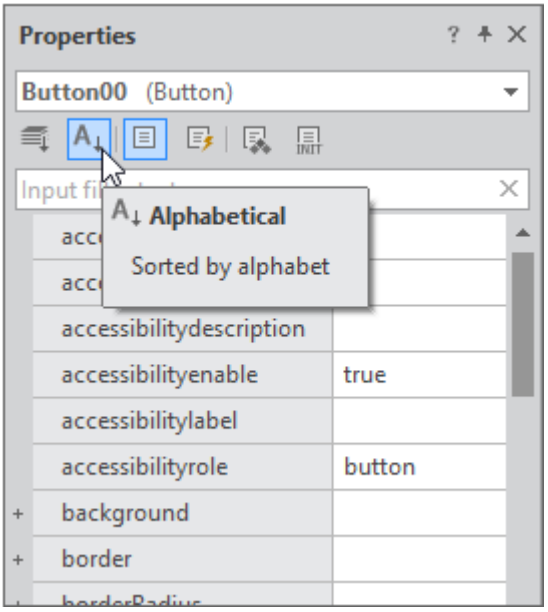
Categorized

Select the Categorized icon to sort the properties or events of a chosen object by category. In this mode, you can expand or collapse each category by clicking the □ or ▼ icon.



Alphabetical

Select the Alphabet Sort icon to organize the properties of a selected object by name.

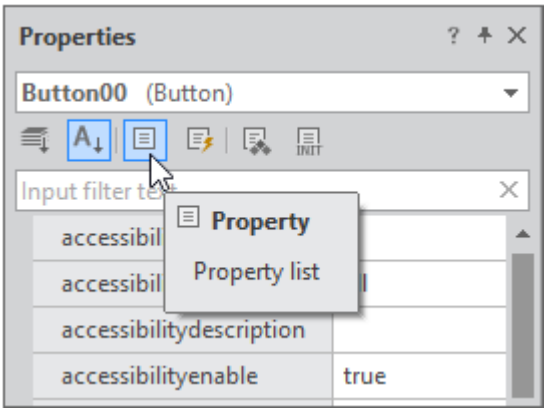


4.3.3 Editing Target

Property

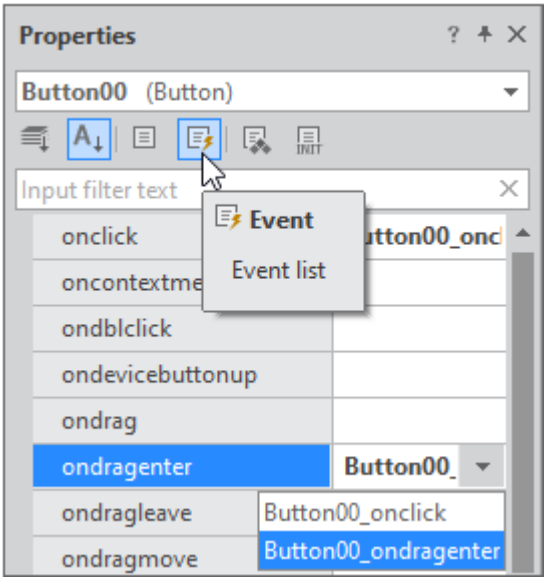
Select the Attribute Info icon to edit properties of selected objects. Nexacro Studio emboldens two types of values: one is those are automatically added to the XML file when an object is created and the other is those are edited by users.

> `<Property mandatory=""/>` in a meta-info file determines the values that are automatically added to the XML file when an object is created.



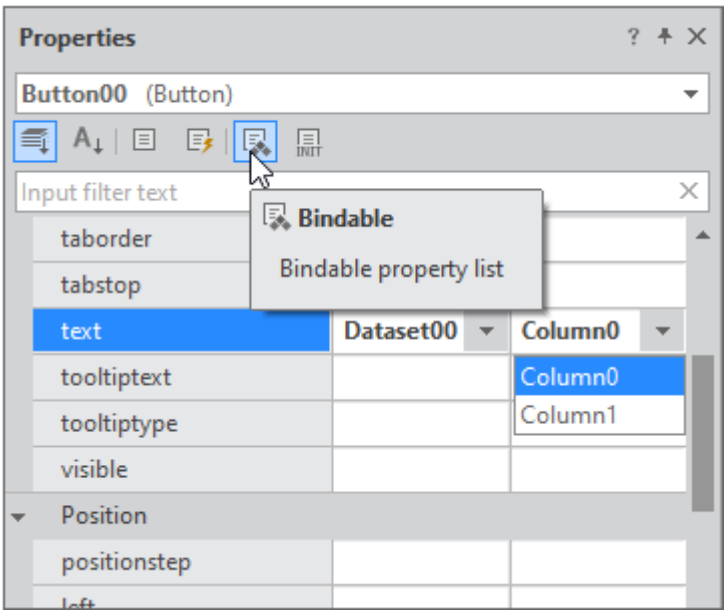
Event

Select the Event Info icon to edit events associated with the selected object. You can create an event either by double-clicking an input box or by selecting an item from a drop-down menu.



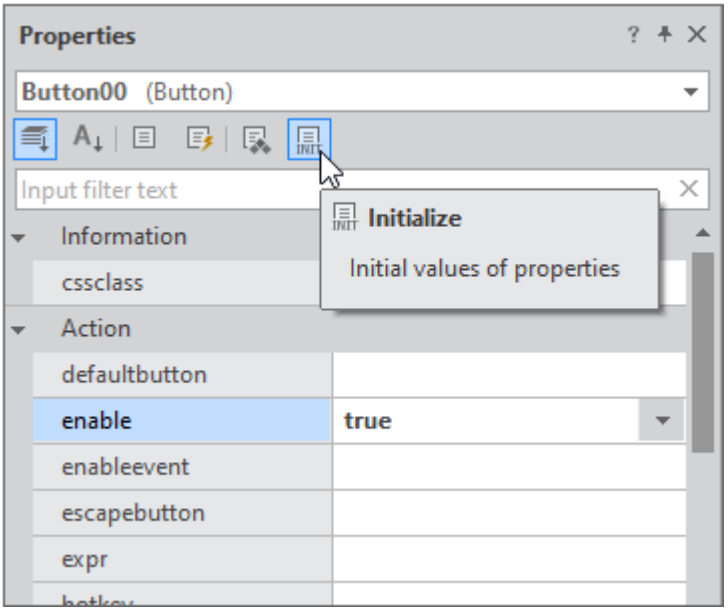
Bindable

Select the Bind Info icon to bind Datasets to properties of the selected object. To perform binding, you need to specify a Dataset ID in the first combo box and a column ID in the second combo box.



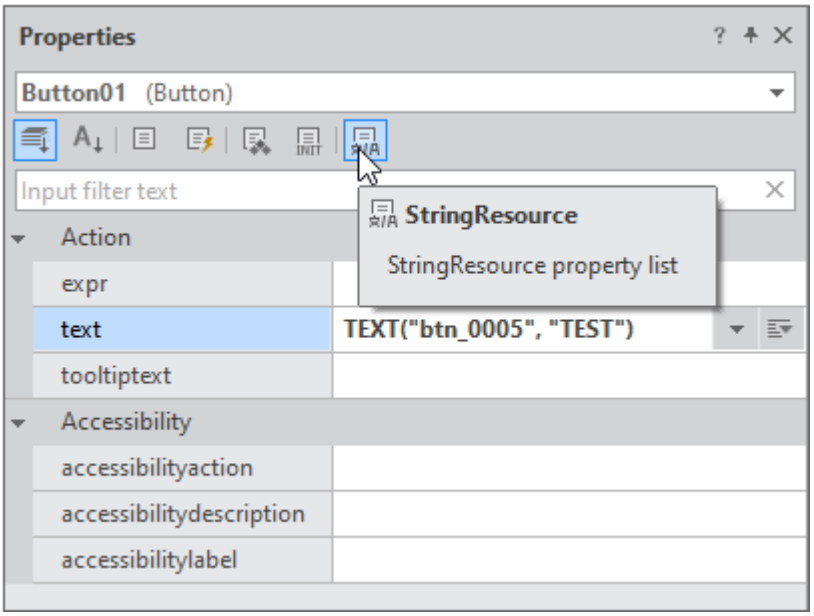
Initialize

There might be a case that you should use designated property values for the initial execution of an app and then use user-input values or changed value while executing the app later on. This kind of value should be separately managed by the Init Info property. Some of these items (except for a value relevant to Layout Information) can be edited by selecting the Init Info icon.



StringResource

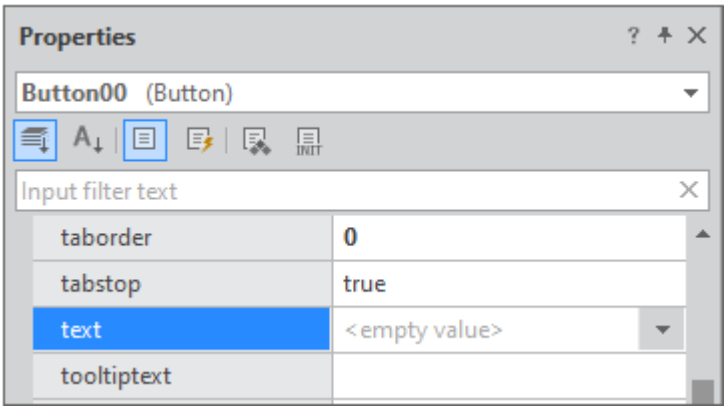
If you want to apply multiple languages, you can set the property value related to texts of the company to be referenced by a StringResource. You will only see a list of properties that support multiple languages.



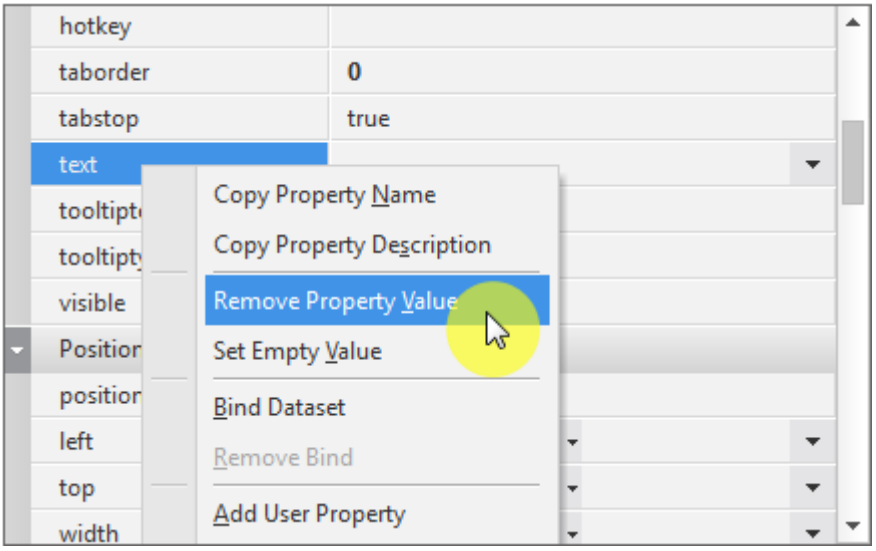
4.3.4 empty value

If a property is given an empty string (“”)as its value, the property will be accompanied by the **<empty value>** tag in the Properties pane. As the tag is intended just for clarity, it will not affect the actual property value. Upon clicking an editable field marked with <empty value>, the tag will disappear as the field has got ready to

accept an input.



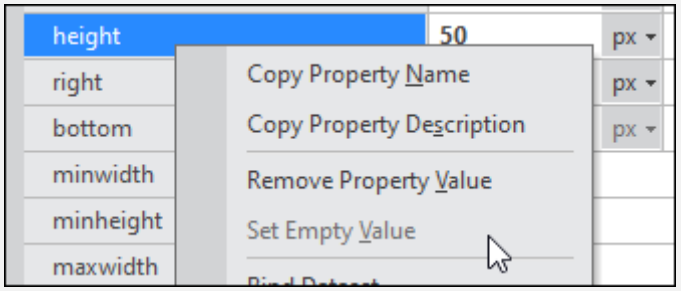
If you want to designate the property value as the empty character string, then select the [Set Empty Value] item from the context menu. When a property is no longer used, select [Remove Property Value] from the context menu.



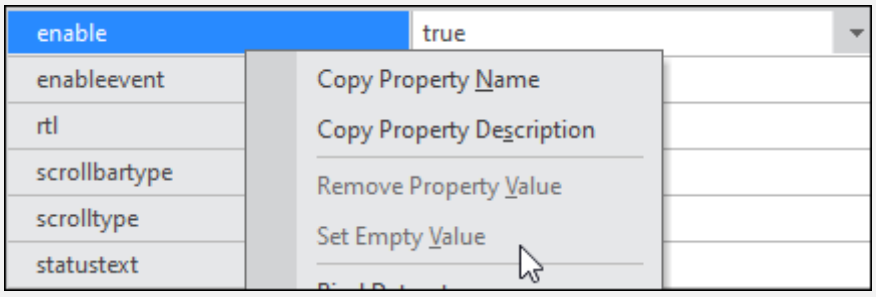
Properties labelled with the <empty value> tag are written in the Source tab of the Form design panel as presented below.

```
<Button id="Button00" taborder="0" left="348" top="124" width="152" height="66"
text="" tooltiptext=""/>
```

The <empty value> tag is marked only for the properties whose values are strings. For example, position properties have numerical values so the context menus from those properties deactivate the items related to <Set Empty Value>.



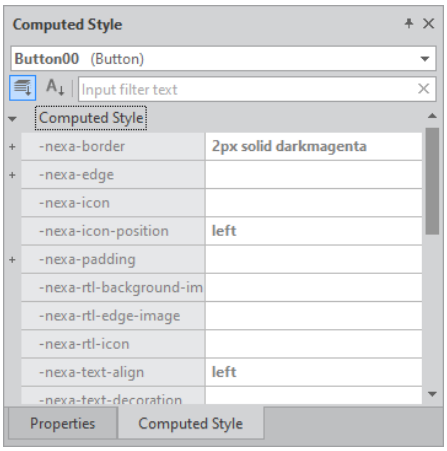
If component properties have default values or definite property values such as boolean values, the Remove Propeory Value, Set Empty Value context menu items are disabled.



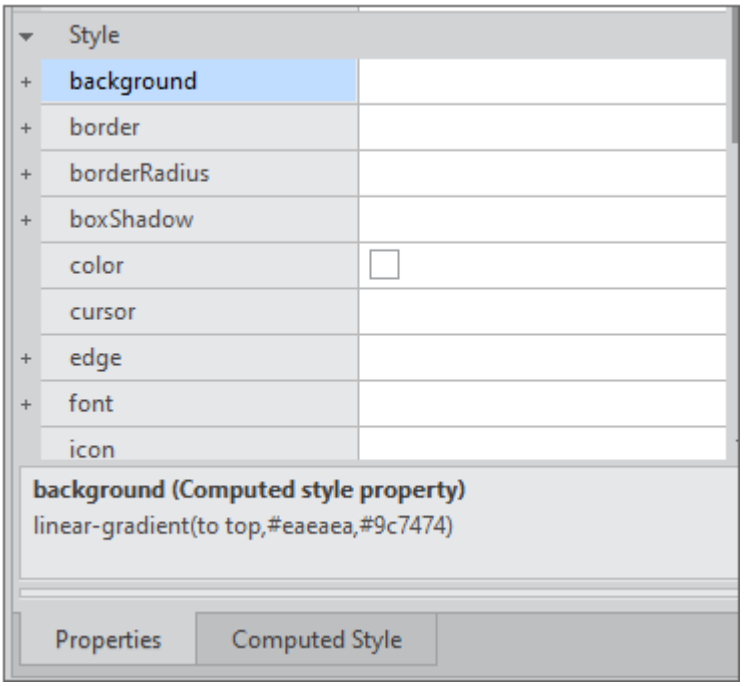
4.3.5 Computed Style

Clicking the Computed Style tab on the bottom of the Properties window switches the pane, allowing you to check the style-related values as calculated after factoring in theme and CSS files. The values presented in the Computed Style window are the ones that actually apply to the application. In the below picture, "beige"—the value of the *background* property—is emphasized in bold type. This indicates that the value has been modified by the user through the Properties window.

You cannot change the values presented in the Computed Style window because they are just the reflections of the results calculated by Nexacro Studio. If you want to change the values, you need to modify relevant theme or CSS files, or change the relevant style-related properties. Then, the Computed Style window will automatically show the changed values.



The Properties window also presents the computed style value on its bottom when you select a certain style-related property.

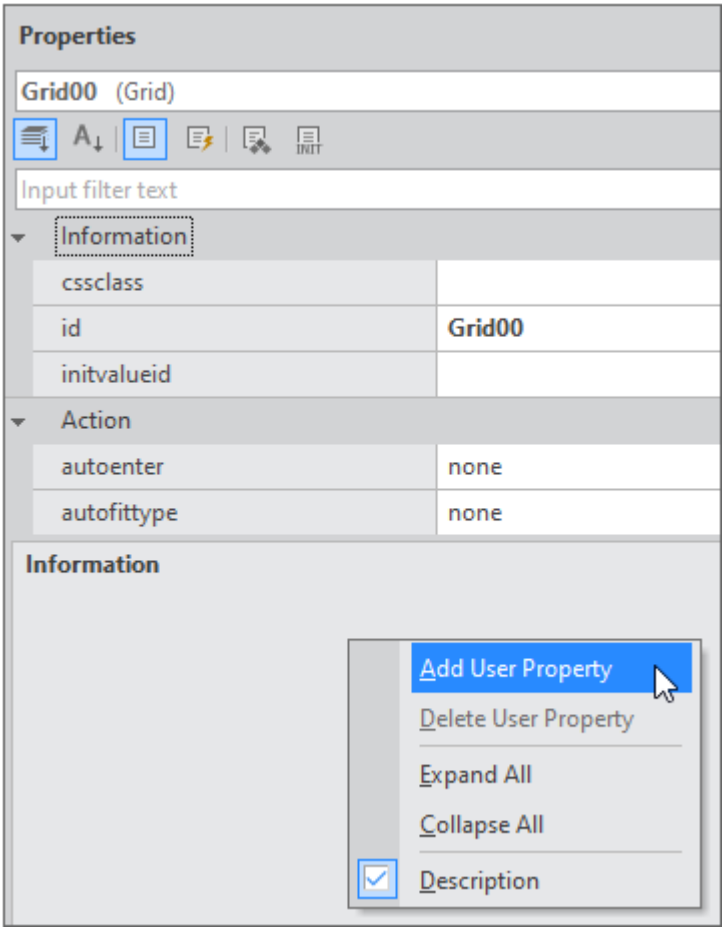


4.3.6 User Property

You can add user properties to components that can be placed on the design surface (Visible Component), Dataset objects, Grid component sub-controls (GridCellControl, CellSubCellControl), and ListView component sub-controls (ListViewCellControl).



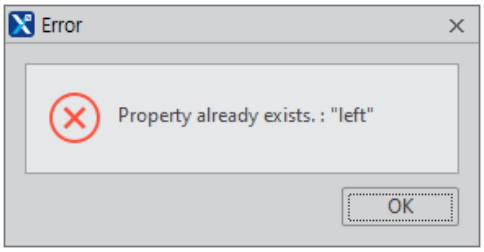
For sub-controls of the Grid component (GridCellControl, CellSubCellControl), user properties can be added in the property window after selecting a cell in the Grid Contents Editor, and for sub-controls of the ListView component (ListViewCellControl), user properties can be added in the property window after selecting a cell in the ListView Contents Editor can be added.



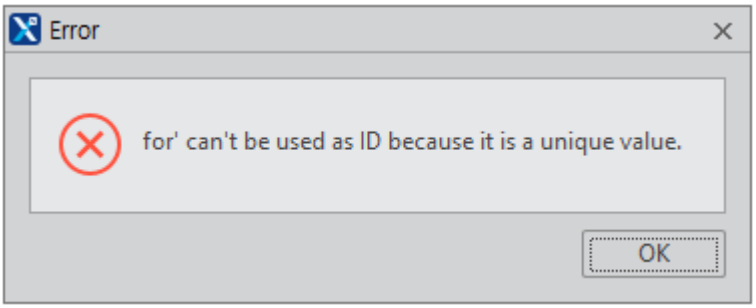
With the "Add User Property" window opened, you will be prompted to enter the "Name" and "Value" values, which are in the form of a string.



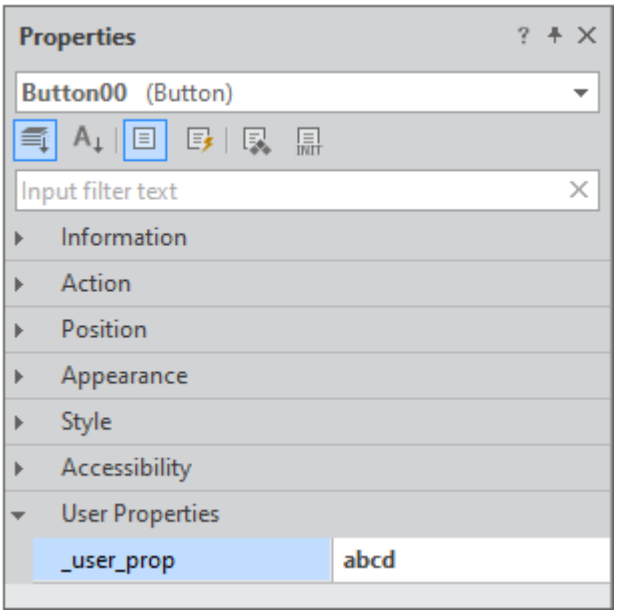
If the relevant component already has a property of the same name, an error message will pop up.



You cannot use JavaScript reserved words for the name of an user property.



The created property will be added under the "**User Properties**" category.

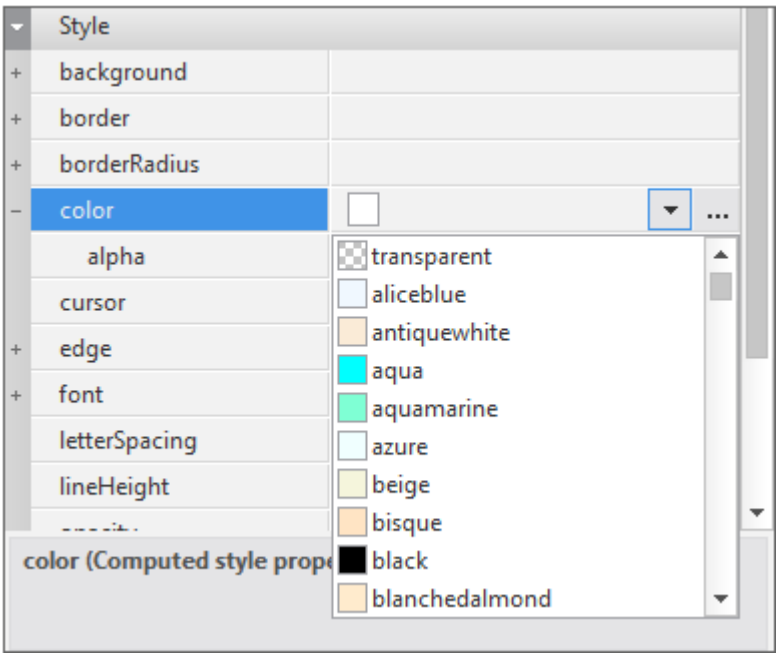


4.3.7 Editing Complex Property Values

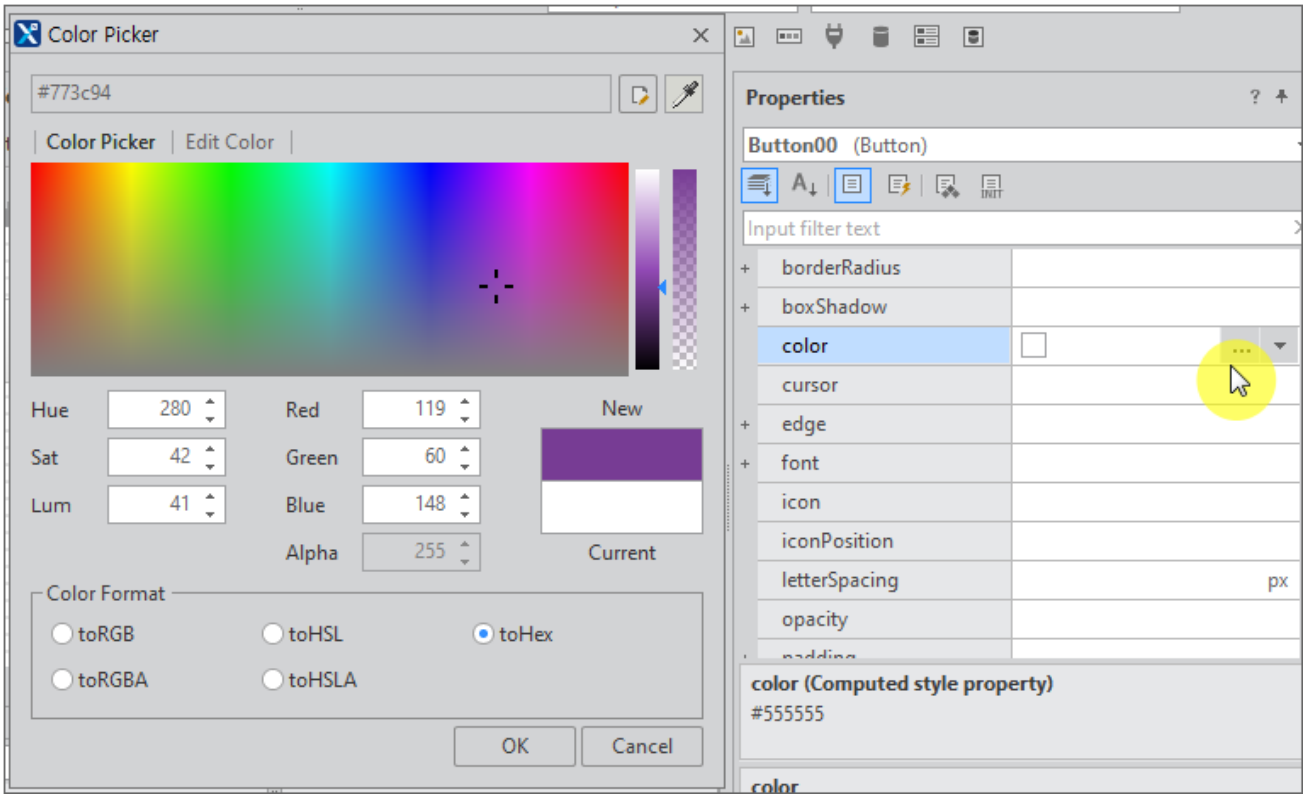
If you know the property value to set, then you can enter it directly into the edit window, but it is difficult to do so if you have to select from a list like color value or if multiple property values need to be reflected in a complex manner. For such properties, a separate edit window for editing property values is supported.

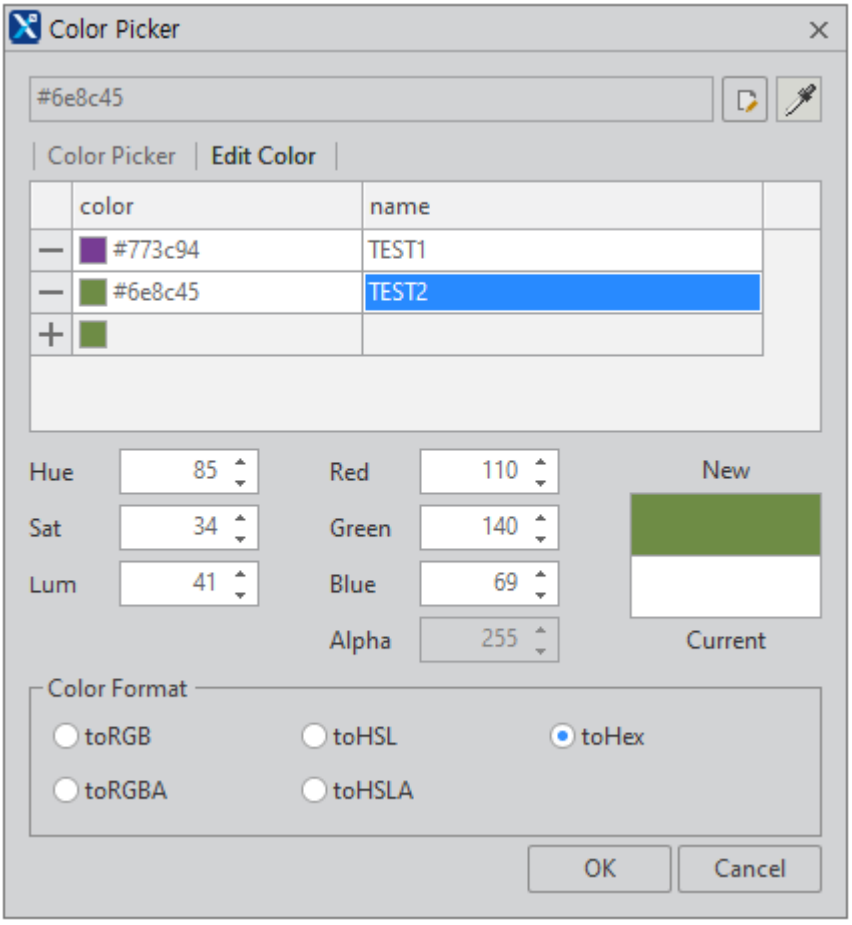
Color Picker

You can select a color from the Combo list or Color Picker.



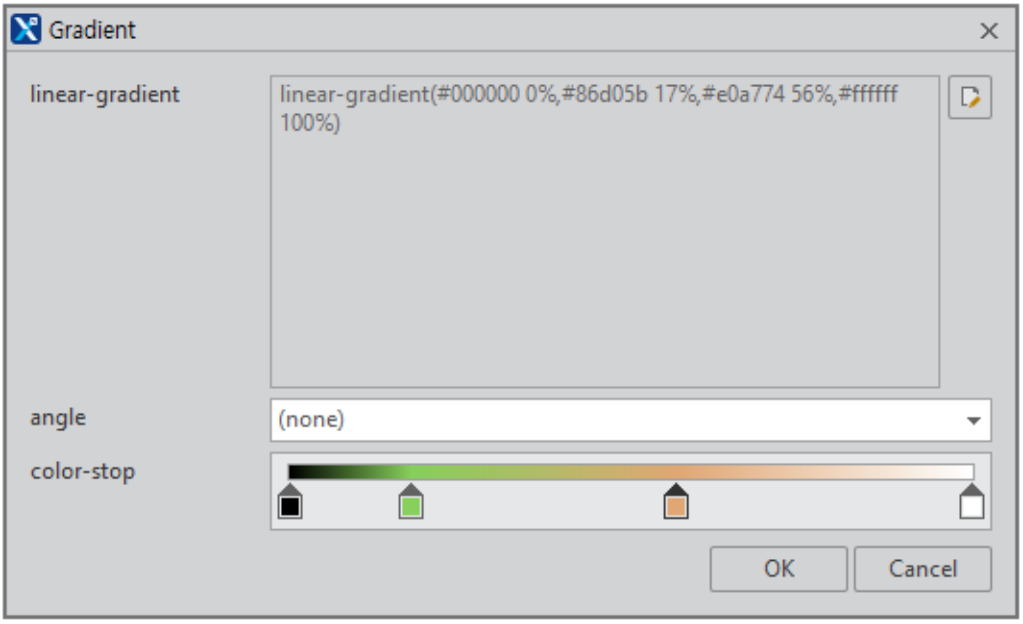
With the Color Picker, you can specify various colors by using multiple techniques as well as define Edit Color for reuse later on.



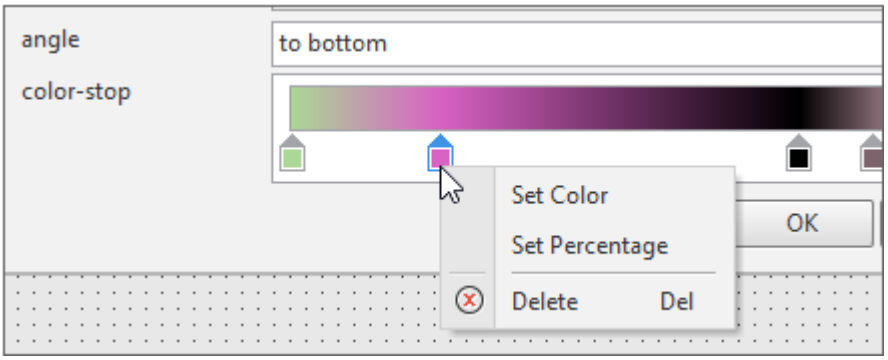


gradient

You can open the Gradient dialog box to edit the *linear-gradient* value of the *background* property. You can specify the direction of a gradient through the [angle] option and gradient colors through the [color-stop] option. You should use the Color Picker to specify gradient colors.

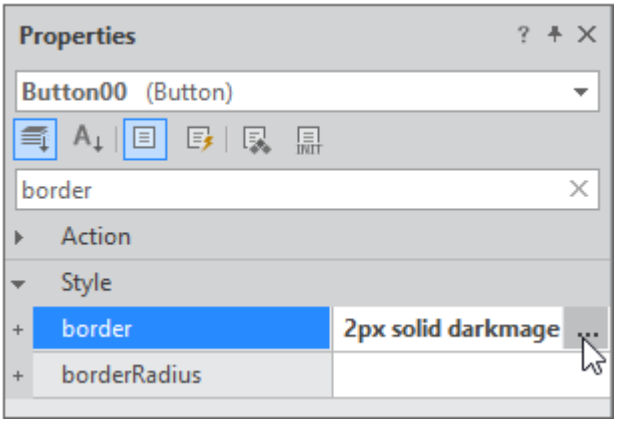


In the [color-stop] option, you can edit or delete colors by right-clicking each color mark.

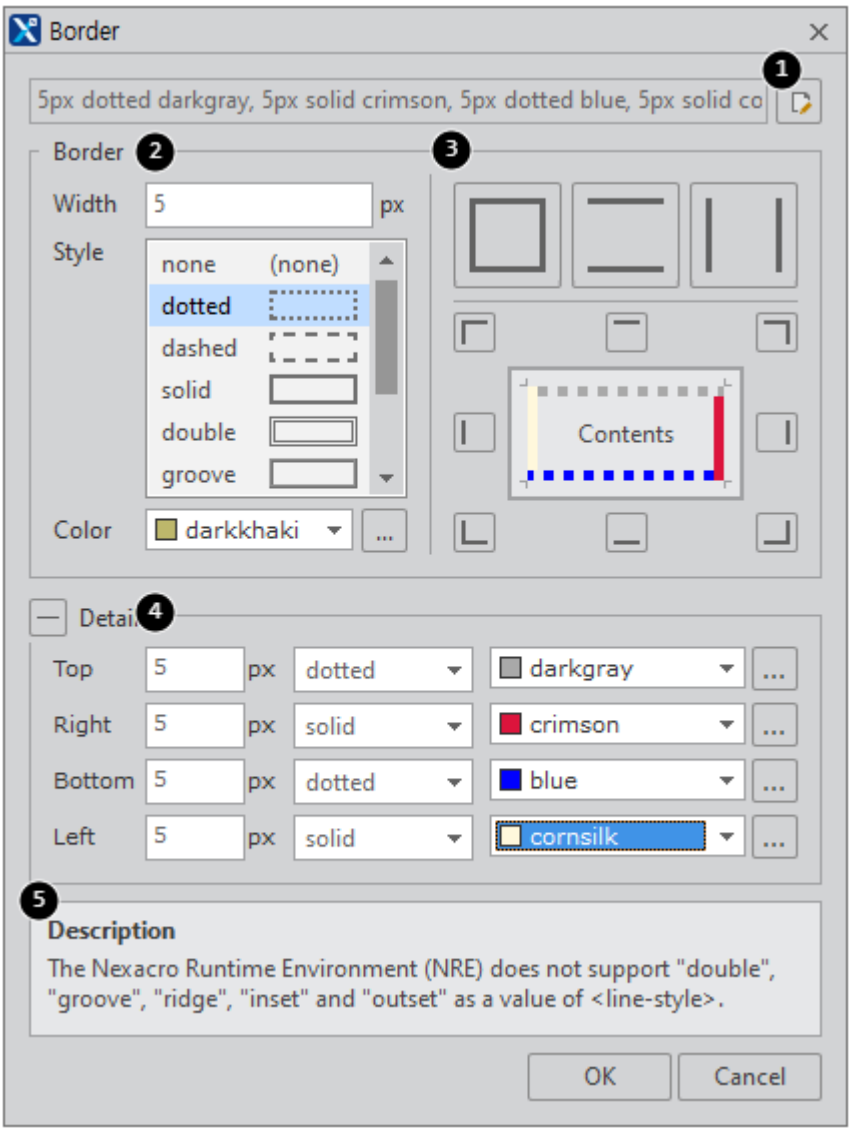


Set Border

You can edit border-related properties of a property through the Border Editor. You can open the editor by clicking the button at the end of the edit field of the *border* property.

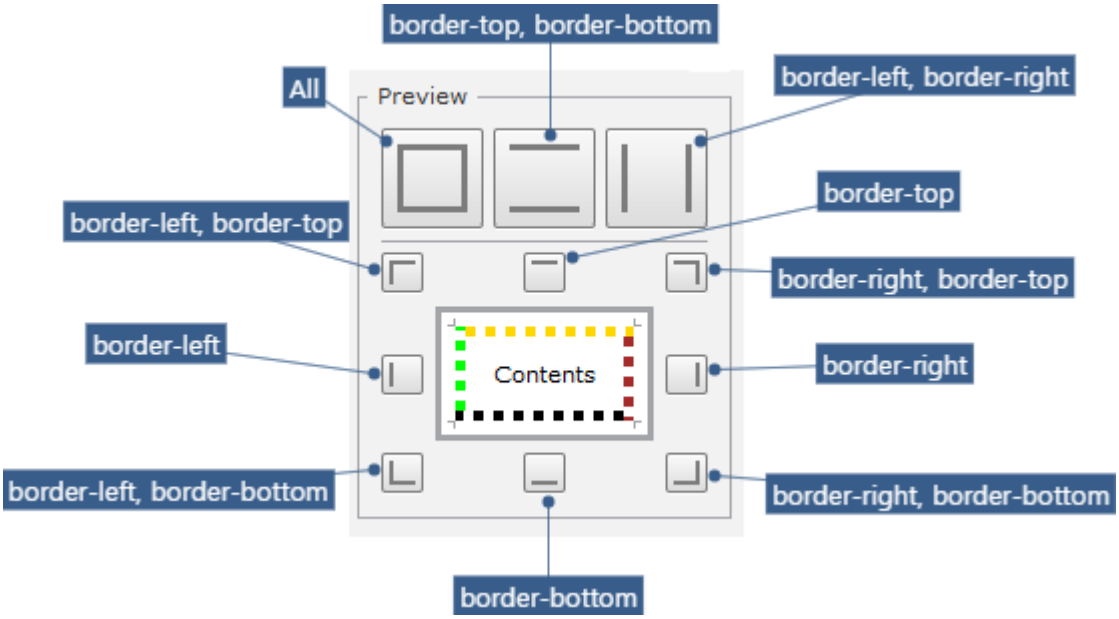


The Border Editor will be opened as a pop-up.



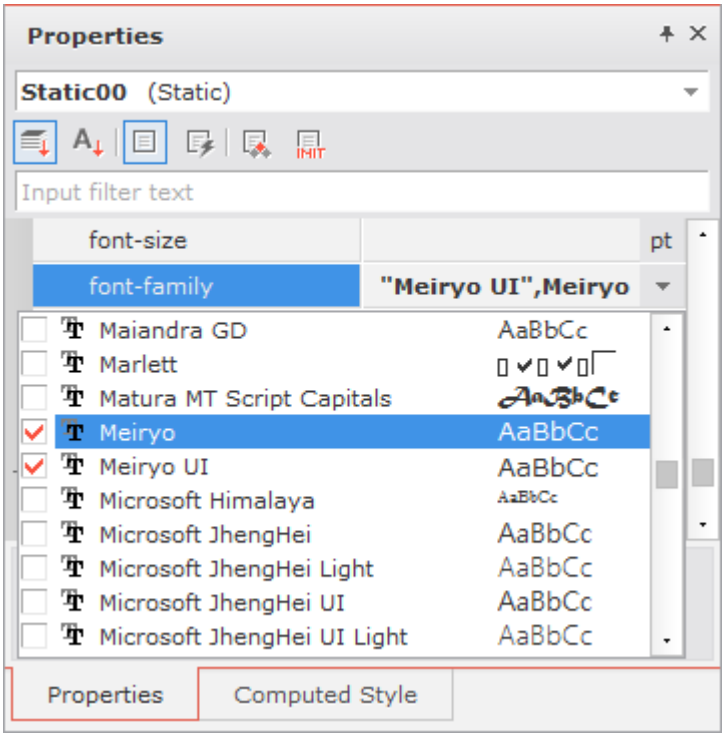
Callout		Description
1	Border value	It is a field where you can see or change specified property values.
2	Border	This section shows property values separately, while you can change those values in this field too.
3	Preview	This section allows you to see specified property values visibly. You can apply what you set in the Border section to intended sides by clicking Preset buttons.
4	Detail	This section allows you to set properties for each side of a component. This section can expand/collapse.
5	Description	This section shows the restrictions pertinent to border-related properties

You can set the below-mentioned border-related properties through Preset buttons.

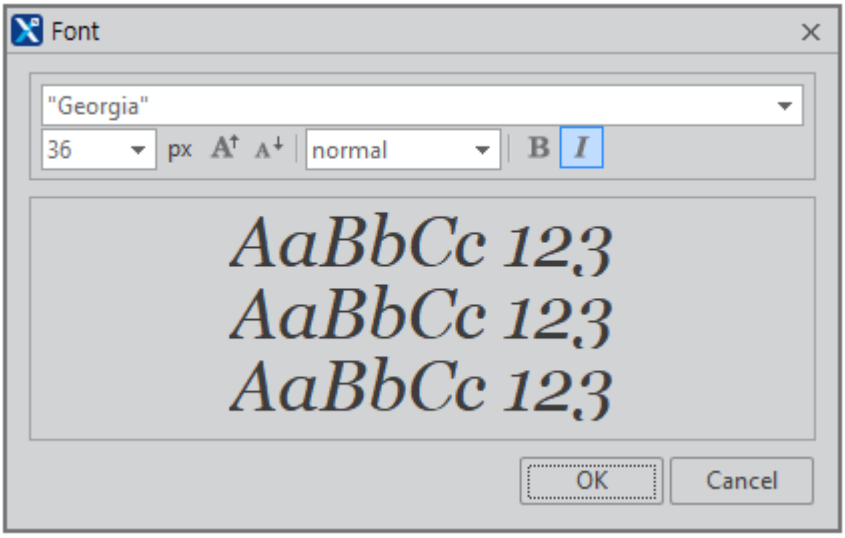


Set Font

Nexacro apps do not have their own fonts. They use the fonts installed with the operating system. You can prioritize available fonts so that you can find them more easily when developing apps.

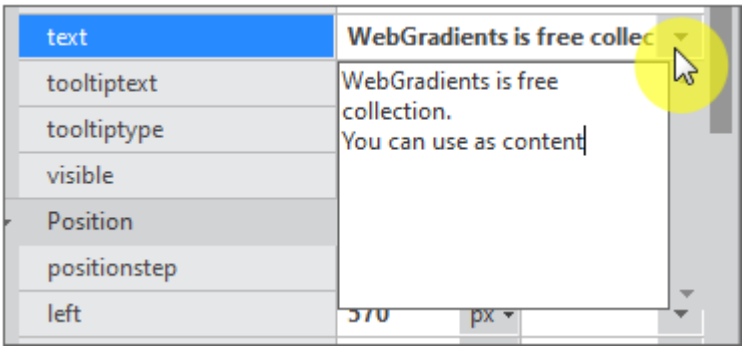


In the font edit window, you can edit property values, such as font, size, and style, and check the applied status in advance.



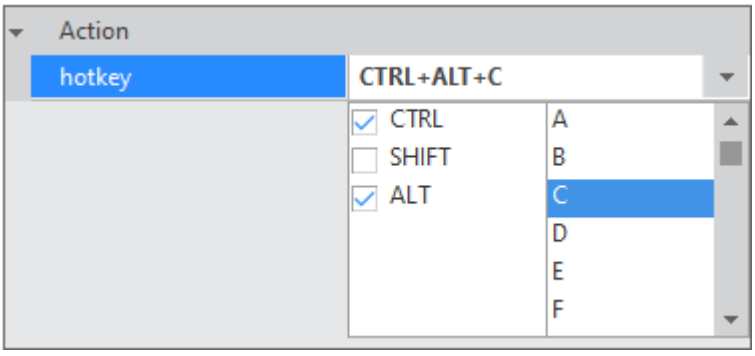
Entering Multiple Lines of Text

When multiple lines of text can be entered for property value, a drop-down button is displayed on the right side of the property input window. A window where you can edit multiple lines of text is displayed by clicking the button. Press the Ctrl+ENTER keys to change lines.



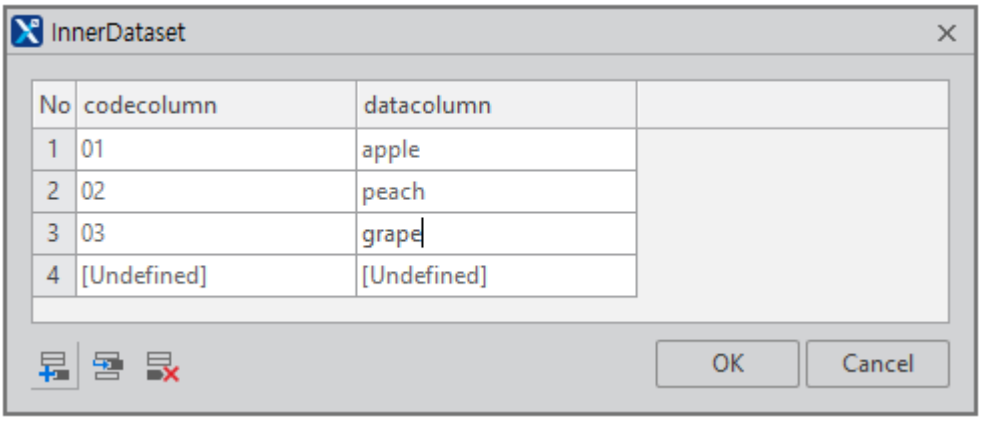
Editing Shortcut Keys

When editing the hotkey property value of the component, an edit window where you can select the key that can be combined for the property value editing is supported.



Editing the innerdataset Property Value

The innerdataset property value can be set by selecting the Dataset object. If you are only using it in the corresponding component and processing simple values, then you can directly edit innerdataset. Functions to add, insert, and delete Row, and to set the codecolumn and datacolumn values are provided.



After adding the innerdataset property value and saving, the codecolumn and datacolumn property values are set to "codecolumn", "datacolumn".

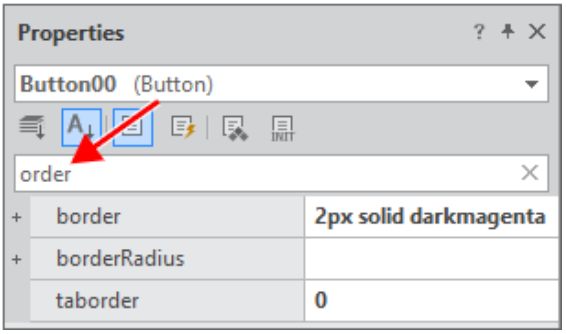
Editing the formats Property Value

The formats property value of the Grid and ListView components can be edited. You can run the edit window by double-clicking the component in the design screen or selecting the formats property value in the property window. Please refer to the link below for more information on the formats property value edit window.

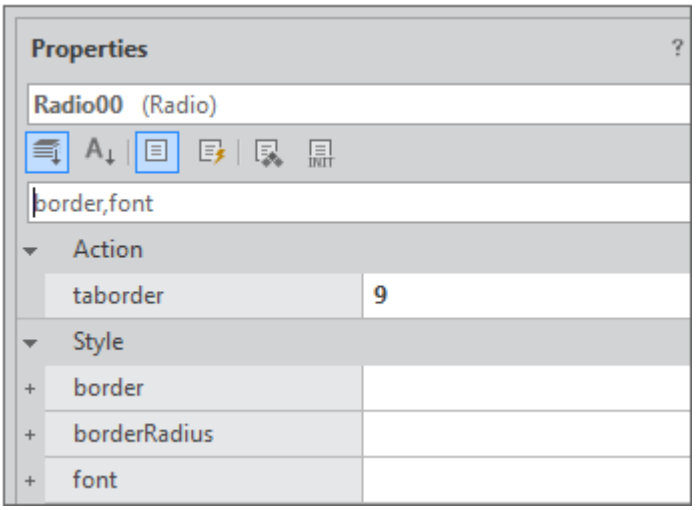
> Grid Contents Editor
> ListView Contents Editor

4.3.8 Filtering

If there too many properties presented in the Properties window, you can quickly find your desired items by using the filtering feature. The pane will show every item that includes the match of your input text.

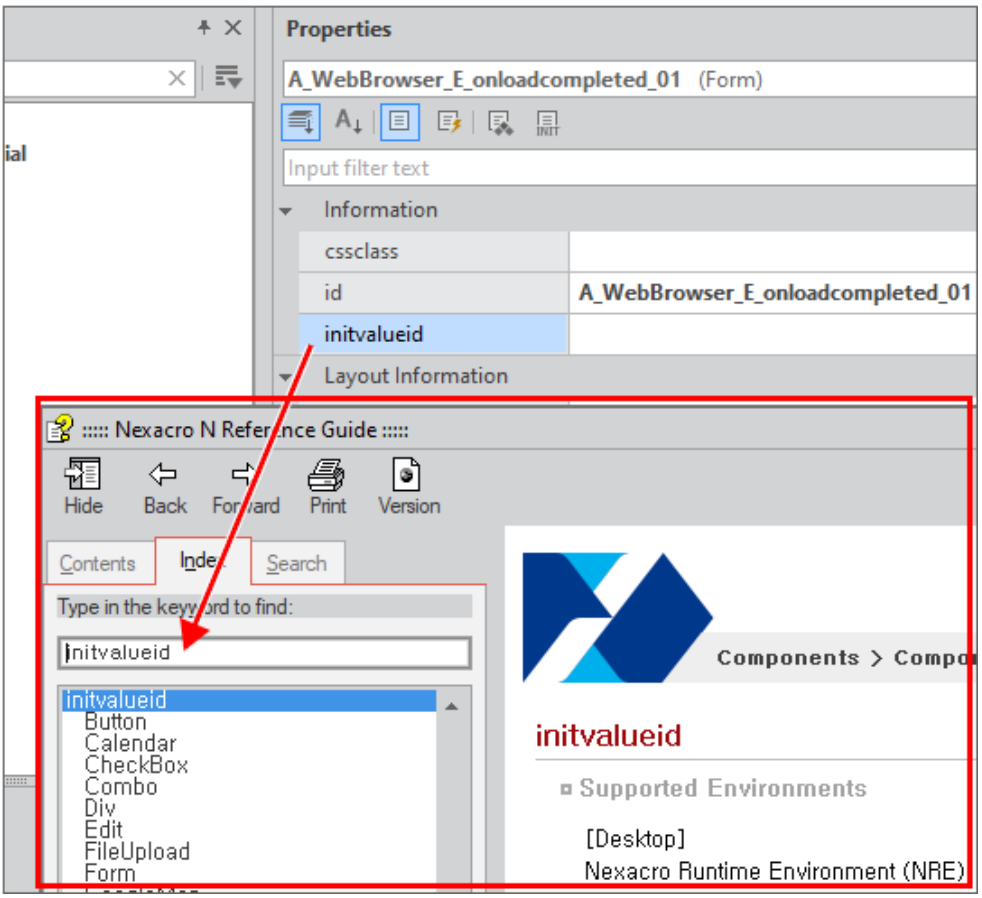


You can use multiple filtering items using the comma (,) separator.



4.3.9 Quick Link to Help

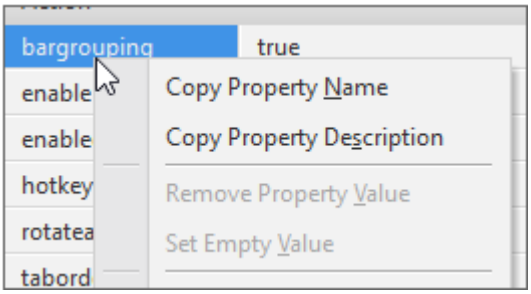
You can use this function by accessing [Help > Help] or pressing the F1 key with a certain property selected. If you do so, the help (Nexacro N Reference Guide) will be called with the property name automatically entered in the input box of the Index section, allowing you to easily find the necessary information.



The first page of the reference guide will be opened if the Properties window is not focused.

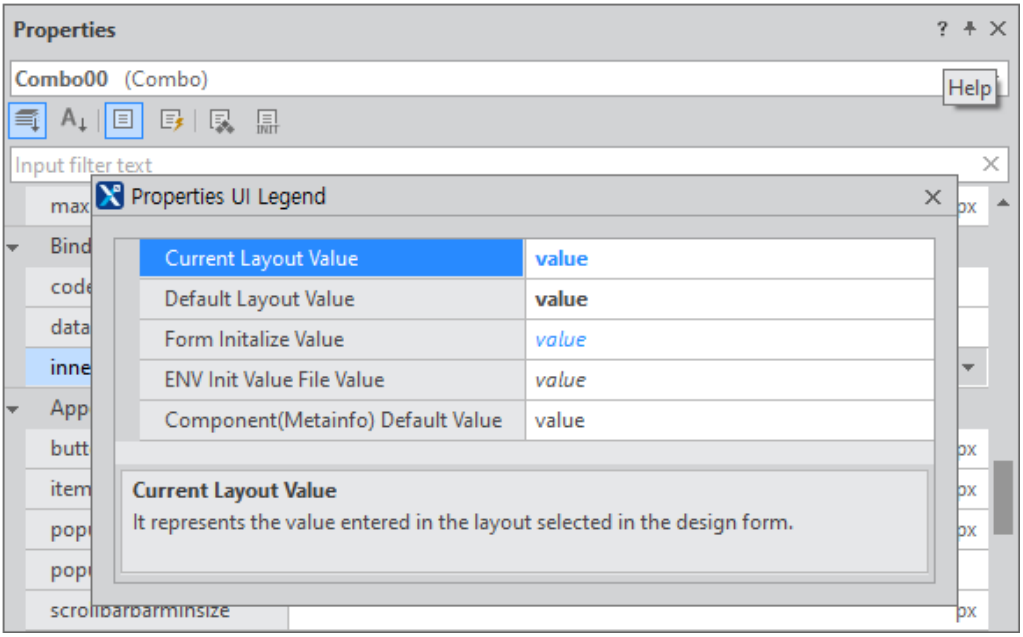
4.3.10 Copy Property Name

Right-click a property name from the Properties pane and select Copy Property Name from the context menu to copy the name onto the clipboard. Likewise, select Copy Property Description to copy the description of a property onto the clipboard.



4.3.11 Checking the Meaning of the Font in which Property Value is Displayed

Depending on the status of the property value, the property value is displayed in different colors and styles. A legend for this is displayed when clicking the [?] button at the top right of the property window. You can see a brief description at the bottom of the window by selecting each item.

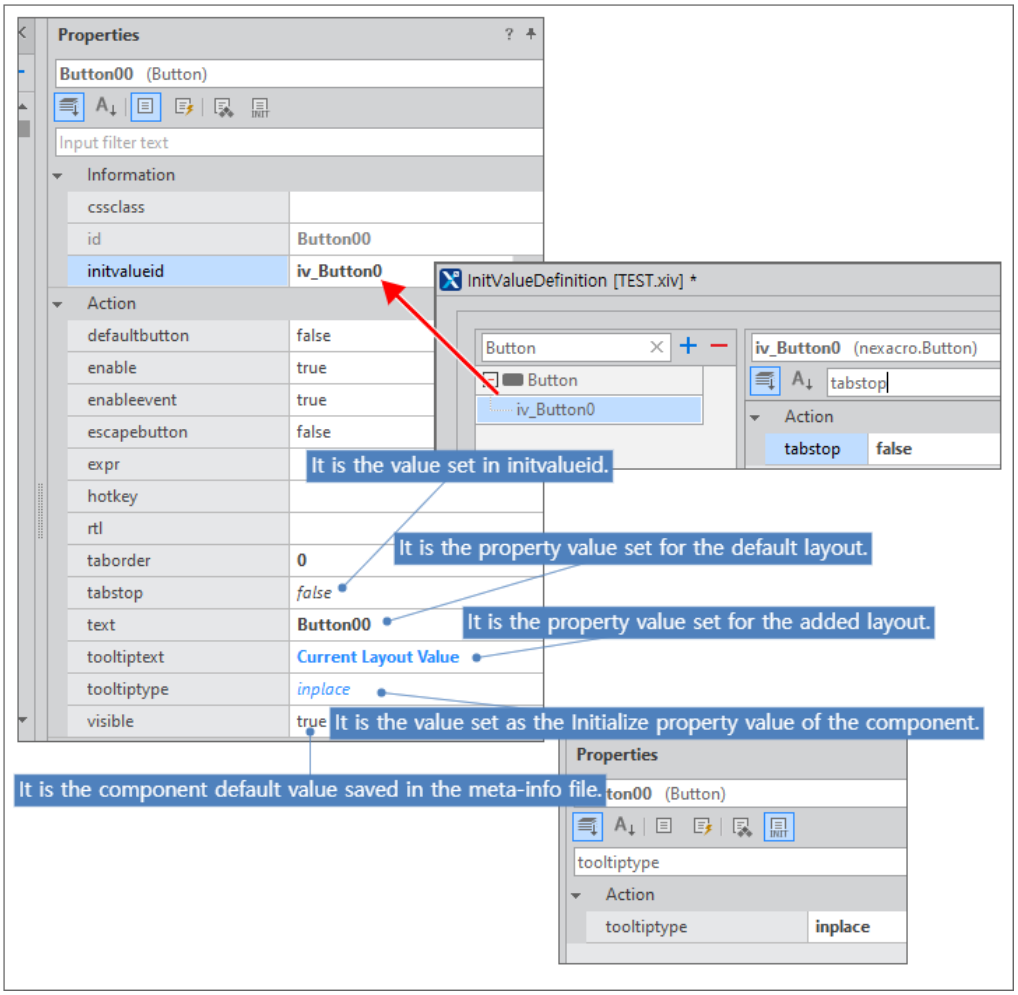


The priority and description of each item are as follows.

Priority	Item	Description
1	Current Layout Value	The value entered in the layout selected in the tool design form
2	Default Layout Value	The value entered in the default layout
3	Form Initialize Value	The Initialize value entered in the form Initialize window
4	ENV Init Value File Value	The value that exists in initvaluefileid entered in Environment
5	Component(Metainfo) Default Value	The component default value saved in the MetaInfo file

The figure below is a case of adding a layout to the form and applying each property value. Fonts are displayed as described in the legend depending on the status of the property value.

In the case all of the above items are applied to the text property value, the final value is reflected according to the priority. For example, if you delete the value entered in the current layout, then the default layout setting value, which is the next priority, is reflected.



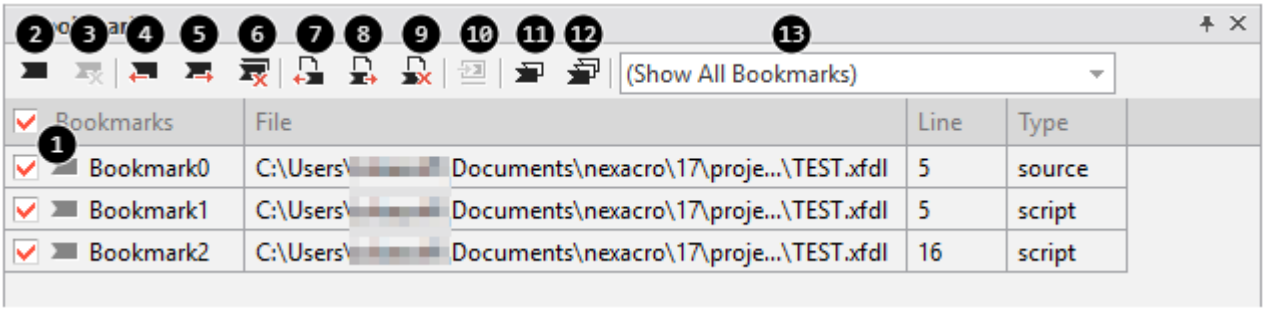
Please refer to the link below for the initvalueid setting.

> InitValueDefiniton

4.4 Message Window

4.4.1 Bookmarks

Select the Bookmarks pane to show the bookmark list that was used in the Source or Script Editor window. If you double-click on the bookmark displayed on the list, the related editing window will appear and the relevant line will be displayed.

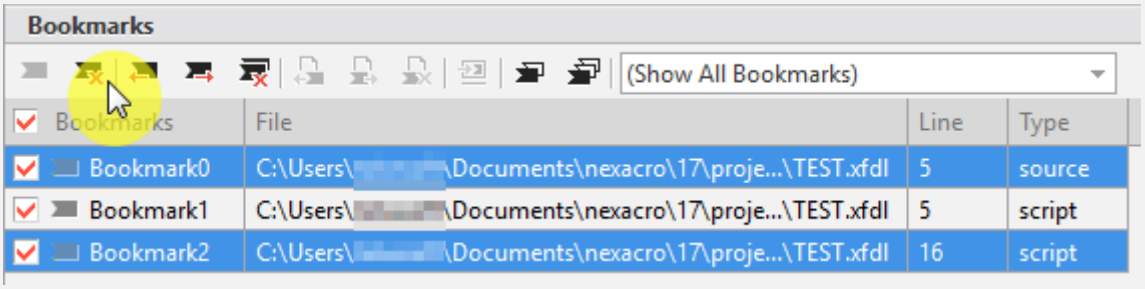


Use the following toolbar icons to perform various bookmark functions.

	Name	Description
1	Check Box	If unselected, the bookmark is excluded from search.
2	Toggle Bookmark	Sets or clears the bookmark at the current cursor location
3	Delete Bookmark	Removes the selected bookmark
4	Previous Bookmark	Moves cursor to the previous bookmark location
5	Next Bookmark	Moves cursor to the next bookmark location
6	Delete All Bookmarks	Clears all bookmarks
7	Previous Bookmark in Document	Moves cursor to the previous bookmark location in the current the current document
8	Next Bookmark in Document	Moves cursor to the next bookmark location in the current the current document
9	Delete All Bookmarks in Document	Clears all bookmarks in the current document
10	Go to Bookmark	Moves the cursor to the line of a selected bookmark
11	Enable/Disable Bookmark	Includes/excludes all the selected bookmarks when you are searching
12	Enable/Disable All Bookmarks	Includes/excludes all the bookmarks when you are

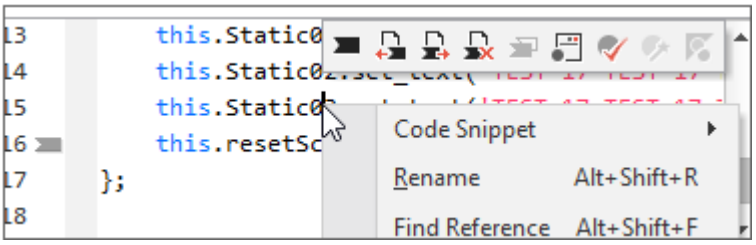
	Name	Description
		searching
13	Project Filter	Filters bookmarks according to services (if you select "Show All Bookmarks", all the bookmarks within a project will be displayed.)

Clicking the Delete Bookmark icon removes the bookmarks whose rows are selected rather than whose checkboxes are selected. In the below picture, for example, rows will turn blue and the Delete Bookmark icon will be activated when they are selected.



When setting or clearing bookmarks in the Bookmark window, the change will be applied directly to the relevant editing window. The bookmark list is managed separately in Nexacro Studio, so when you copy projects done in other locations, bookmark will not be shown.

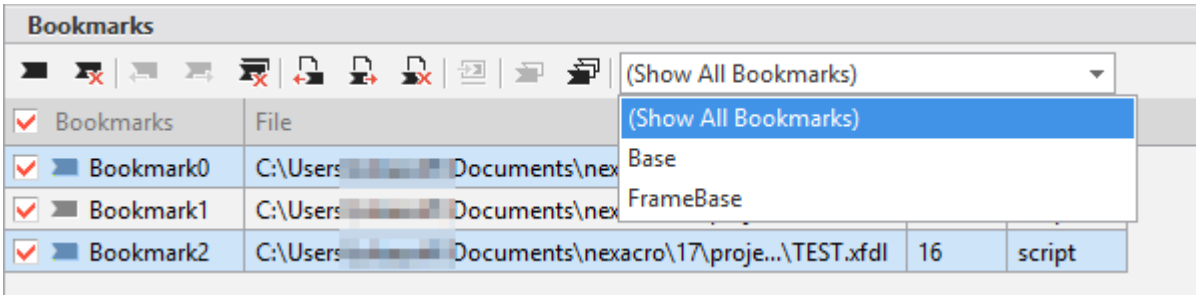
- Add/Delete
  In the Source, Script or XCSS editor, you can open the mini-toolbar for bookmarks by right-clicking a line. Then, selecting [Toggle Bookmark] will add/delete a bookmark. You can do the same task with the Bookmark toolbar on the menu area on the top of Nexacro Studio or with the Bookmarks pane.



- Search
  You can go through bookmarks in the filtered range by clicking the [Previous Bookmark] or [Next Bookmark] buttons. Moreover, you can look over only the bookmarks within the opened editor by clicking the [Previous Bookmark in Document] or [Next Bookmark in Document] buttons. The bookmark-searching features work in the same way as [Go to Bookmark]: they open a corresponding editor window and go to the relevant line.

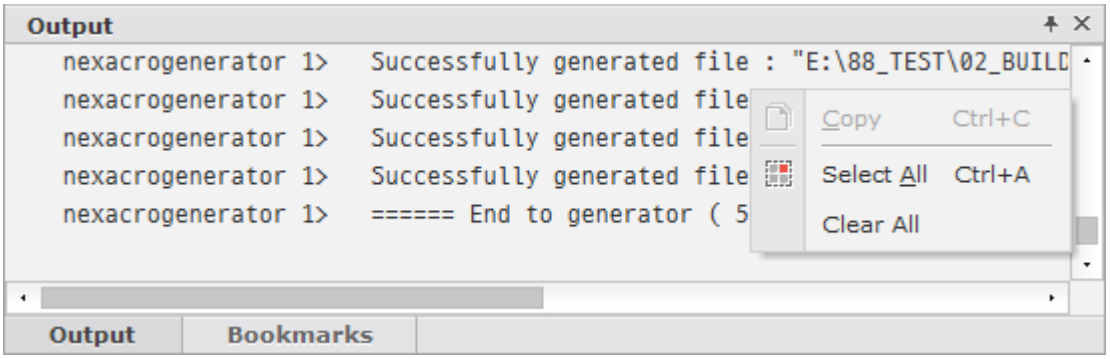
Unchecked bookmarks are excluded from the list to seek.

- Project Filter
  You can filter bookmarks by designating a service.



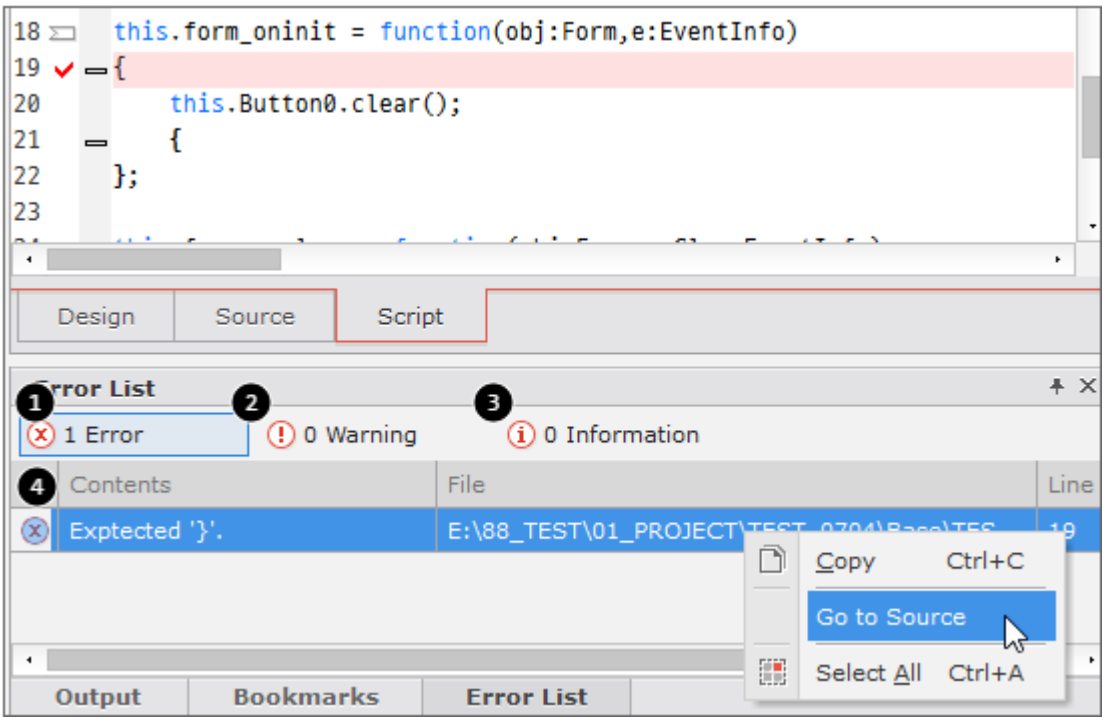
4.4.2 Output

The Output pane displays output messages that may be displayed while working with Nexacro Studio. These messages contain information about errors, processes, and the operation of Nexacro Studio.



4.4.3 Error List

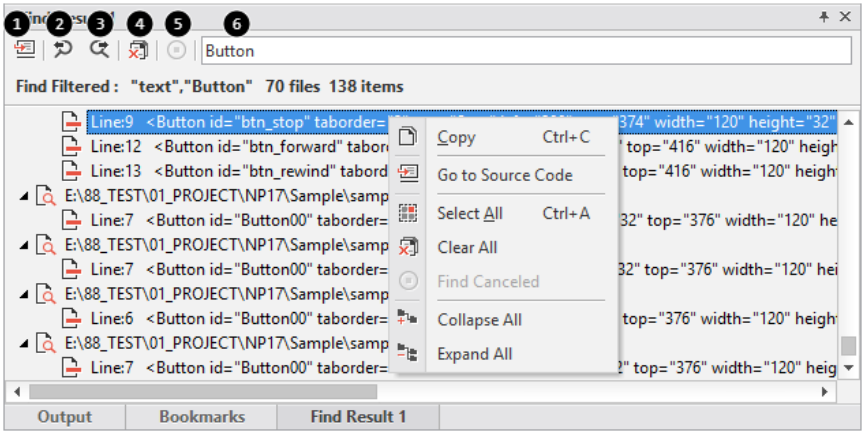
This window displays--on a real time basis--errors, warnings and information that occur when users work on scripts or sources. The three toggle buttons on the toolbar are used to filter messages by types.



Callout	Description
❶	Filtering to show error messages
❷	Filtering to show warning messages
❸	Filtering to show information messages
❹	Space for displaying messages and related information

4.4.4 Find Result

The Find Result displays the results of conducting the menu [Edit > Find in Files]. If you click each file displayed in the window, a related editing window displays the relevant line in the file. Depending on whether you select the option "Find result 2 window," the results will be displayed on "Find Result 1", "Find Result 2" or "Find Result 3".

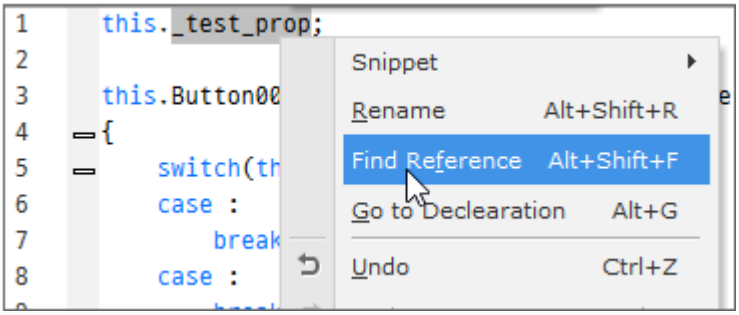


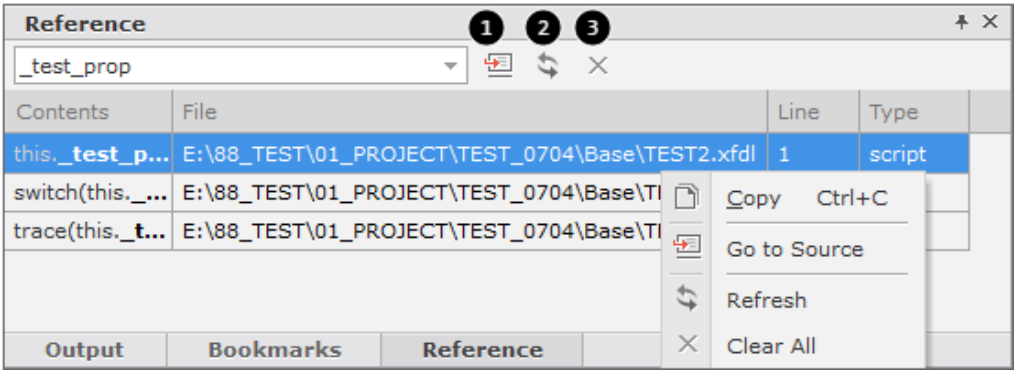
Call-out	Name	Description
❶	Go to Source Code	Goes to the line corresponding to the selected search result
❷	Previous	Goes to the line corresponding to the previous item of the selected search result
❸	Next	Goes to the line corresponding to the next item of the selected search result
❹	Clear All	Deletes all the search results
❺	Find Canceled	Cancels the process of file search (If too many files are being found within a project, you can cancel the search in the middle using this function).
❻	multikey filter	You can search within results by adding keywords. You can enter multiple keywords by dividing them with commas, whitespaces or semicolons.

Clicking the Previous or Next button will select a certain item and move you to the relevant line in source codes. You can conduct the same action by selecting a certain item and then clicking the *Go to Source Code* button.

4.4.5 Reference

You can search for references to a certain character string in a script. First, select a character string you want to find and click [**Find Reference**] from the context menu of the script window. Then, Nexacro Studio will look for the references to the character string within the relevant project and display search results on the **Reference** window. If you double click one of the results, you can go to the relevant location with the pertinent editor opened.





	Name	Description
1	Go To Source	Goes to the relevant reference
2	Refresh	Searches for references again (The refresh commend is needed when values have been changed or when strings have been added or changed).
3	Clear All	Deletes all the search results

4.5 Debug Window

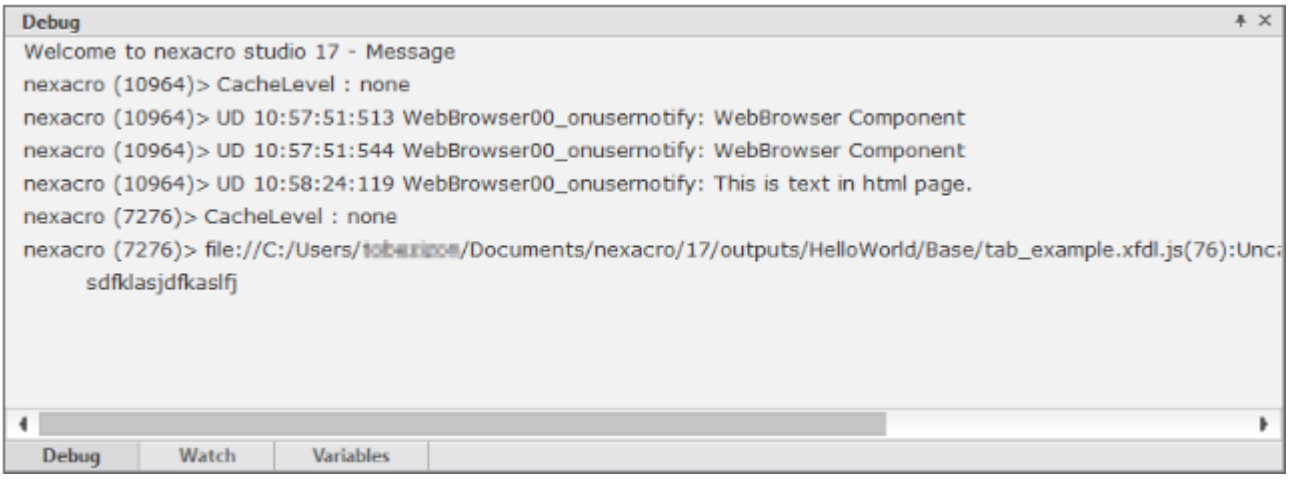
The Debug window provides functionalities to find bugs in code.

Debugging provided by Nexacro Studio only applies to the NRE. If you use a web browser for execution, use the debugging functionalities offered by the web browser.

Refer to the following links for details about the debugging functions offered by web browsers.
Google Chrome
https://developers.google.com/chrome-developer-tools/docs/javascript-debugging
Firefox https://developer.mozilla.org/en-US/docs/Tools/Debugger

4.5.1 Debug

Select the Debug pane to display debugging mode messages.

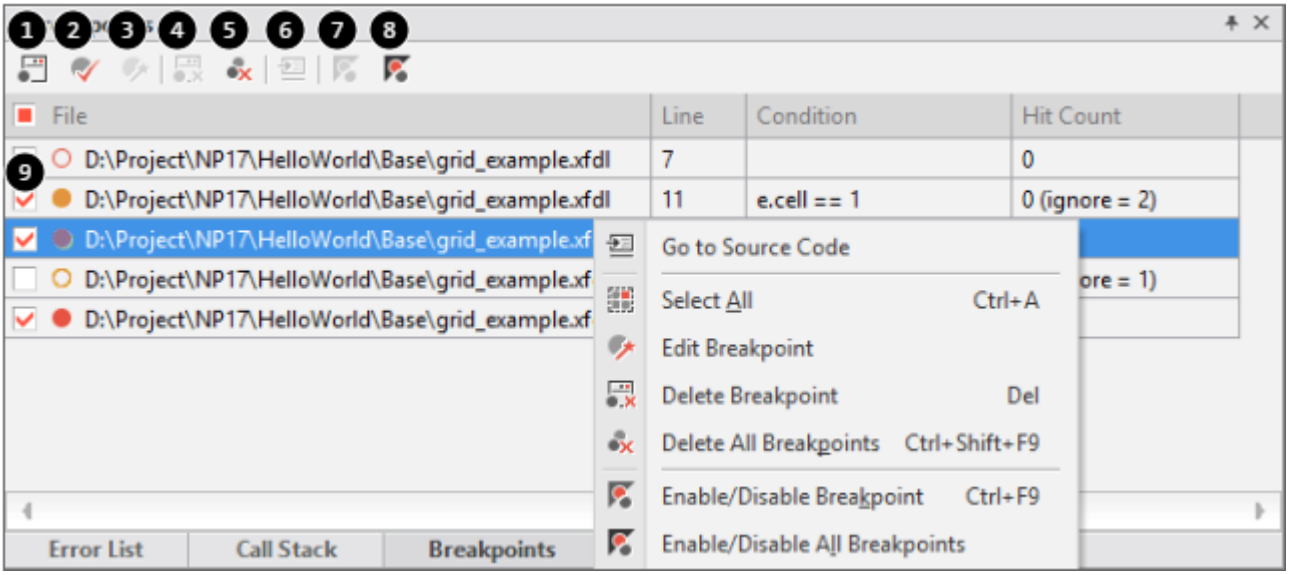


4.5.2 Breakpoints

In debugging mode, you can designate where to pause during script execution. These designated locations are called "Breakpoints." You can see breakpoints listed in this window.

With the breakpoint list, you can move to relevant code lines as well as deactivate or remove breakpoints. Moreover, you can set conditional breakpoints or set the debugger to ignore certain breakpoints.

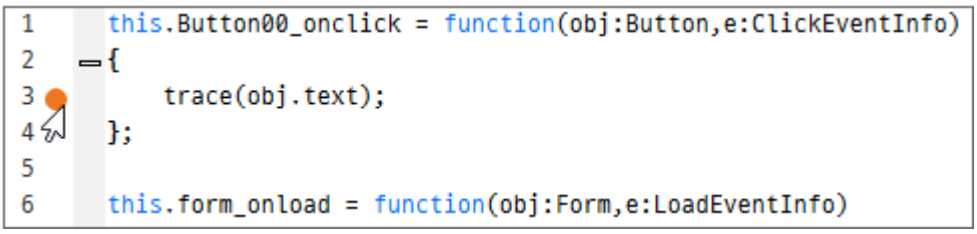
> The data on breakpoints is managed by the Nexacro Studio installed in your computer. Therefore, breakpoints of a certain project will be lost if you open the project in another computer.



Call-out		Description
1	Toggle Breakpoint	Sets or removes a breakpoint
2	Set Conditional Breakpoint	Sets or removes a conditional breakpoint
3	Edit Breakpoint	Edits the condition for activating the selected breakpoint
4	Delete Breakpoint	Removes the selected breakpoint
5	Delete All Breakpoints	Removes all breakpoints
6	Go To Source	Moves to the relevant line of code after the containing script window opened
7	Enable/Disable Breakpoint	Enables/disables the selected breakpoint
8	Enable/Disable All Breakpoints	Enables/disables all breakpoints
9	Breakpoint List	Displays the breakpoint list (You can deselect breakpoints that you will not apply for debugging.)

- Breakpoint Add/Delete

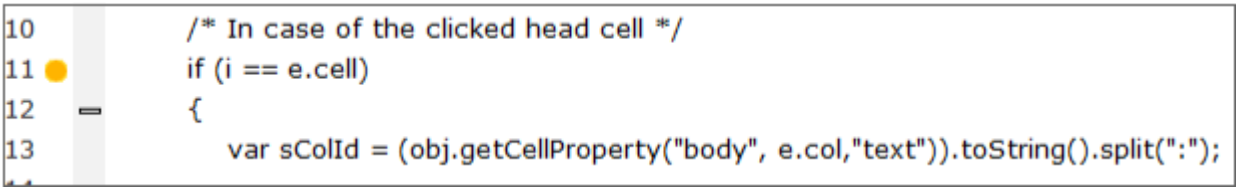
With the script editor opened, clicking the **Toggle Breakpoint** button on the toolbar will leave a marker on the front end of a line where the cursor is placed. Afterward, the relevant item of information will be added to the Breakpoints pane.



If you want to remove a breakpoint, go to the relevant marker and press the Toggle Breakpoint or Delete Breakpoint button.

- Conditional Breakpoint Add/Delete

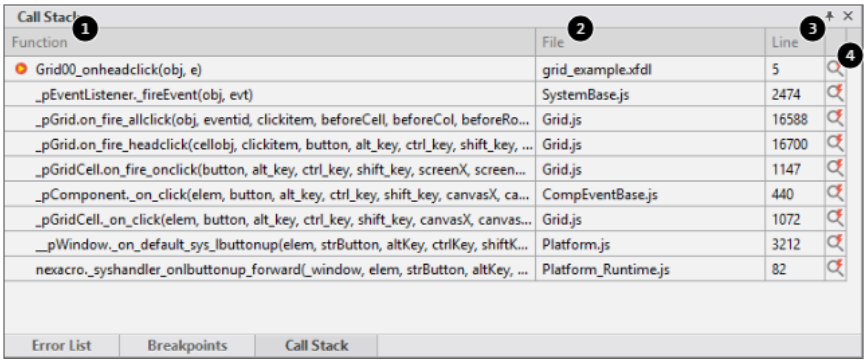
With the script editor opened, clicking the **Set Conditional Breakpoint** button on the toolbar will leave a marker on the front end of a line where the cursor is placed. Afterward, the relevant item of information will be added to the Breakpoints pane. A conditional breakpoint basically works in the same way as an unconditional breakpoint. However, the difference is that you can add a conditional expression and number of *ignore*.



A conditional breakpoint can be deleted in the same way as an unconditional breakpoint. If you want to remove a breakpoint, go to the relevant marker and press the Toggle Breakpoint or Delete Breakpoint button.

4.5.3 Call Stack

The Call Stack pane displays the functions called up to the current line of code in reverse order. When you double-click the relevant item, you can go to the line of code that calls the event.



Call-out		Description
1	Function	Names of the called functions
2	File	Names of the files that contain the functions. A tooltip shows the full path to the file.
3	Line	Numbers of the lines where the functions are declared
4	Quick Show Function	Shows a file that contains the function. The file is presented through an independent window.

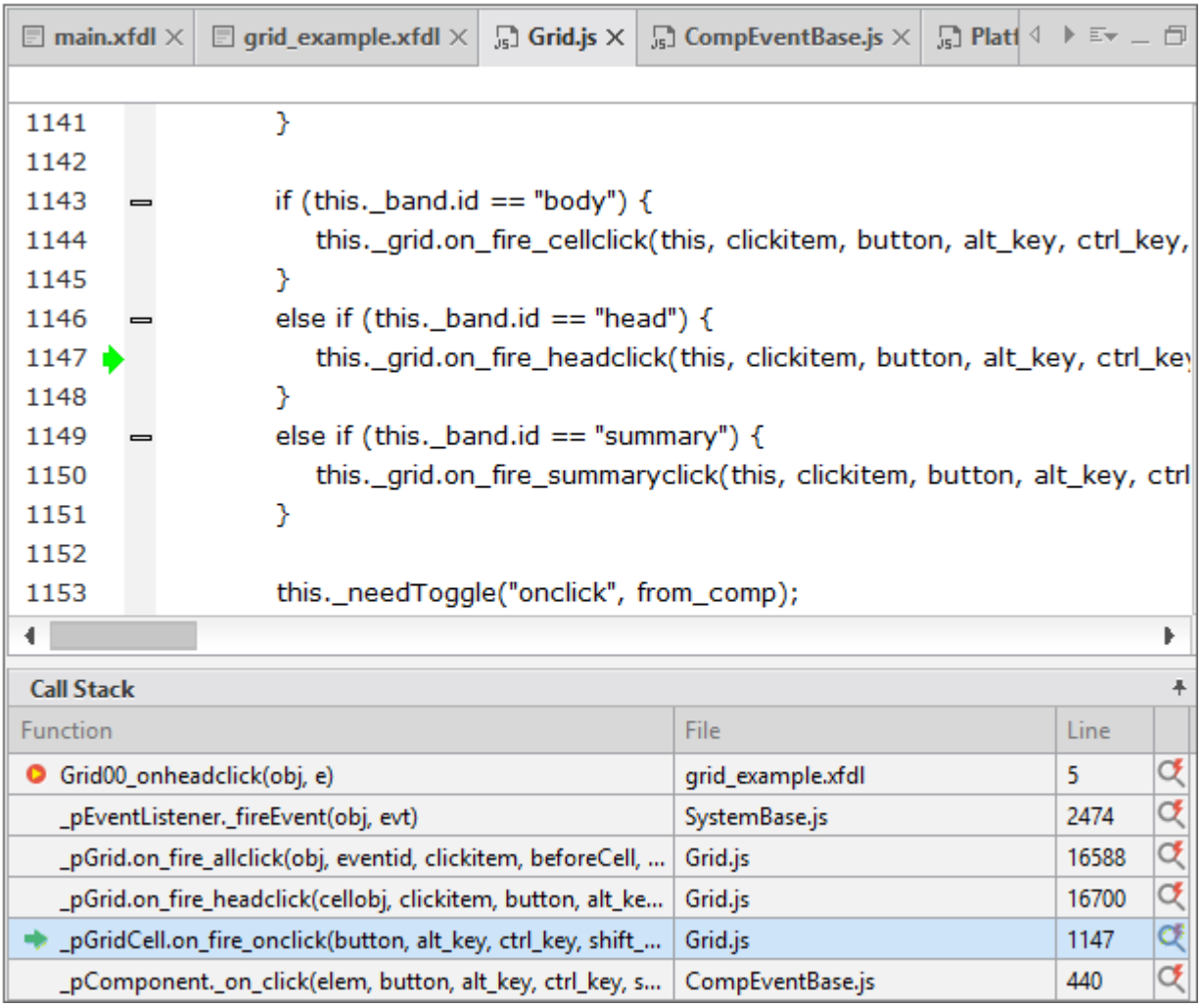
- See sources

  Double-clicking a search (magnifier) icon on the last column opens an independent window that shows the relevant event codes. This is only for reading so you cannot edit the source codes.



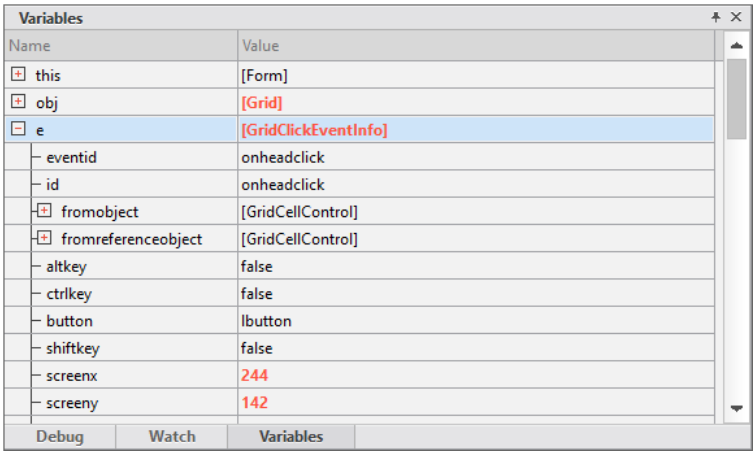
- Call tracking

  You can track operations stretching into a called function within the call stack structure.



4.5.4 Variables

The Variables displays variables or object values accessible from the current location in the code. For the objects that include their members, you can use the expand/collapse button (+/-) to see members. Changed values are marked in bold red.

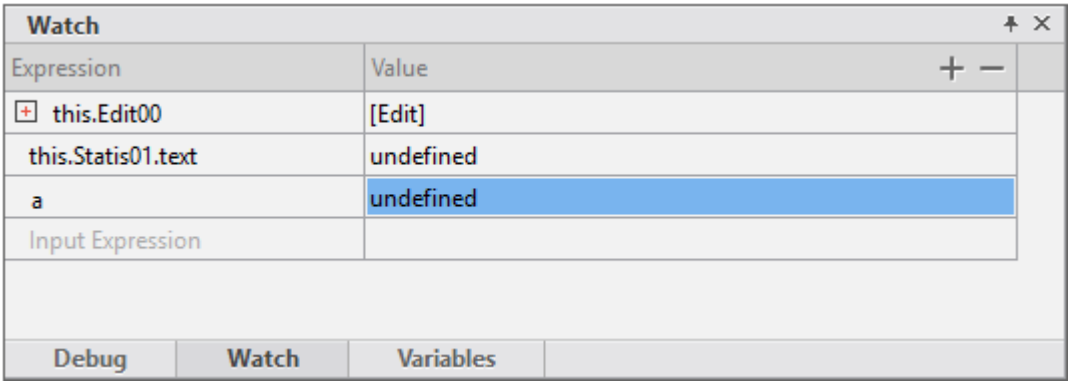


4.5.5 Watch

The Watch pane monitors whether the values of certain variables or objects change. You can add a new item by double-clicking the last row.



The data on a watch list is managed by the Nexacro Studio installed in your computer. Therefore, a watch list of a certain project will be lost if you open the project in another computer.



- Watch Add/Delete

If you want to add an object or variable to the Watch list, click the + button or double-click the last row (which reads "Input Expression"). Moreover, you can conduct the addition from a script editor. After drag-selecting the name of an object or variable, drag and drop the name onto the Watch pane or right-click to select "Add Text to Watch" from the context menu.

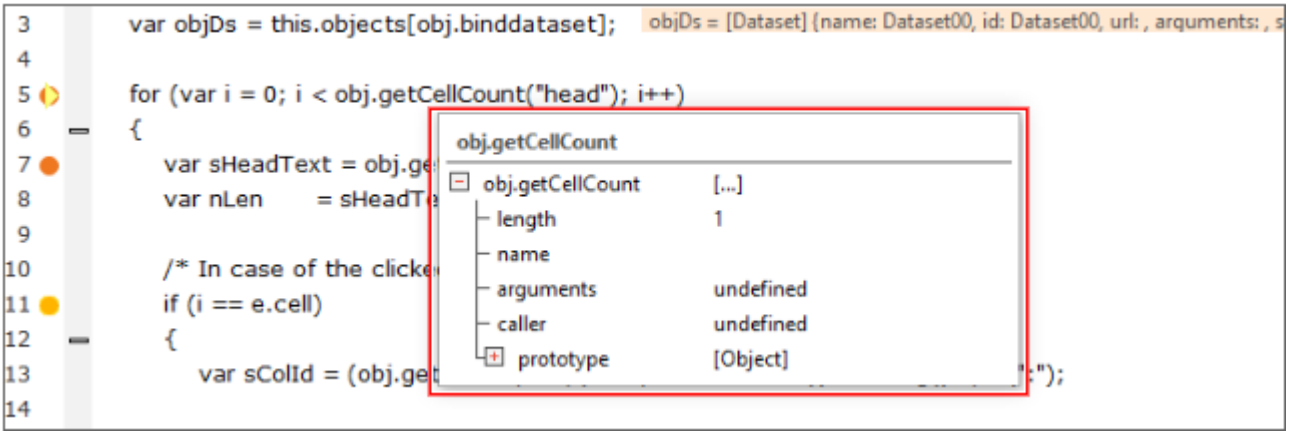
If you want to delete an object or variable from the Watch list, select an item on the Watch pane and click the - button or press the Delete key on the keyboard.

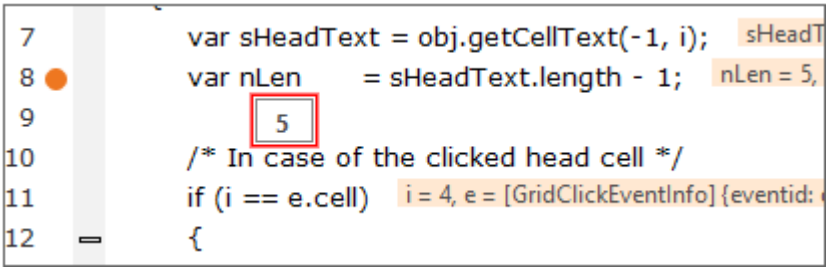
4.5.6 Script Editor

The script editor of Nexacro Studio provides the Assist Tip and Inline Assist Variable features to help users conveniently see the values of objects and variables during debugging.

- Assist Tip

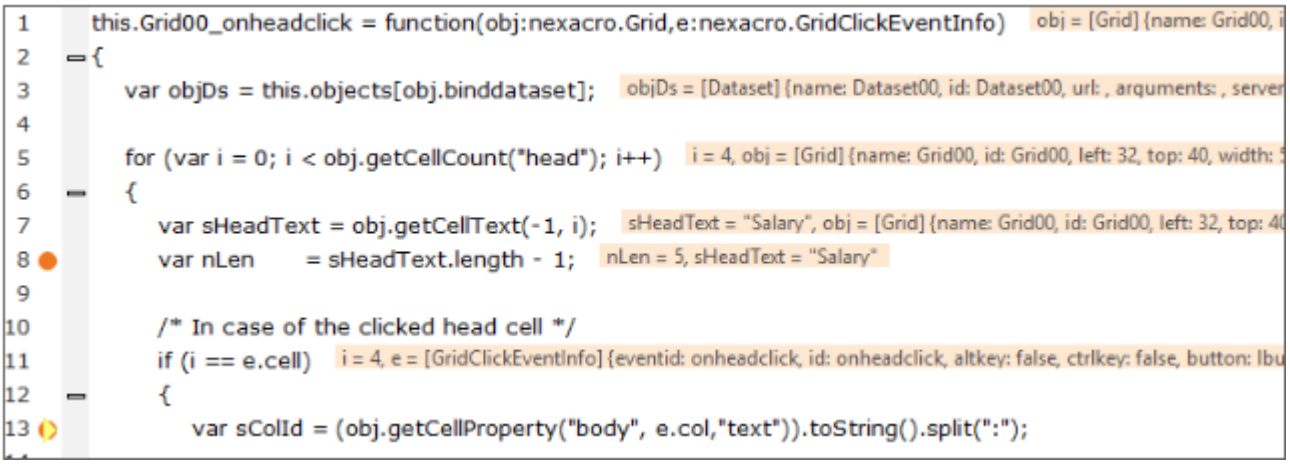
The Assist Tip feature presents the information on an object or variable in the form of a pop-up box when you hover a mouse pointer over the relevant item on the script editor. If the relevant item is a variable, the pop-up will simply show its value. If the item is an object, however, the pop-up will show the members of the object and their values in a tree view.



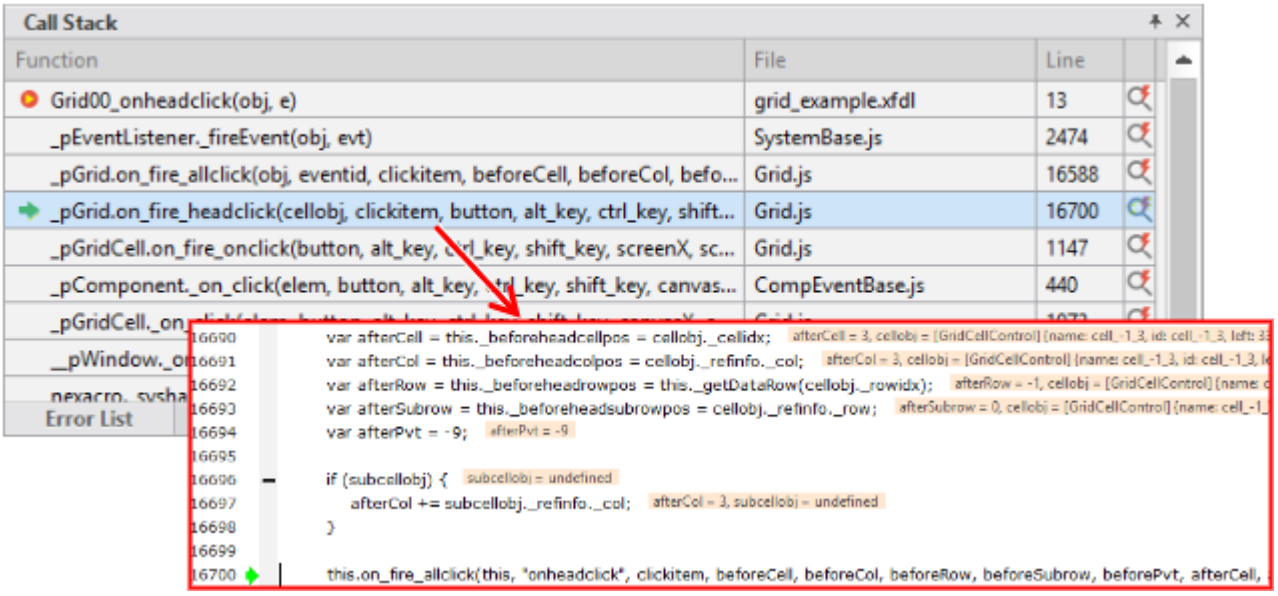


- Inline Assist Variable

The Inline Assist Variable feature presents the information on all the included objects and variables ranging from a function declaration up to the current breakpoint. The information is displayed on the right side of each item in the script editor.



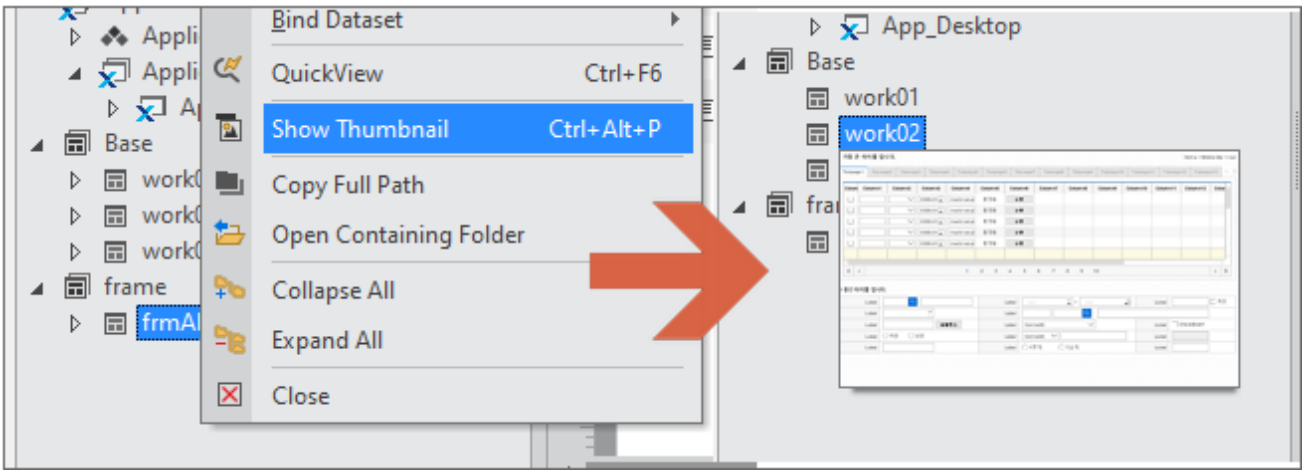
If you double-click the function where a breakpoint is set or a related function from the Call Stack pane, the relevant script editor will be opened, showing the values of all the objects and variables defined until the relevant function.



4.6 Form, Show Thumbnail

To see a thumbnail, open the context menu or press the shortcut (CTRL+ALT+P) after selecting a Form in the Project Explorer or an image in the Resource Explore.

Select a Form and choose [Show Thumnail] from the context menu to take a quick look at the design of the Form. This preview lasts for about two seconds and then disappears.



Since the preview is based on the generated file, a thumbnail will not be displayed if there is no relevant generated file or the Form fails to be loaded.

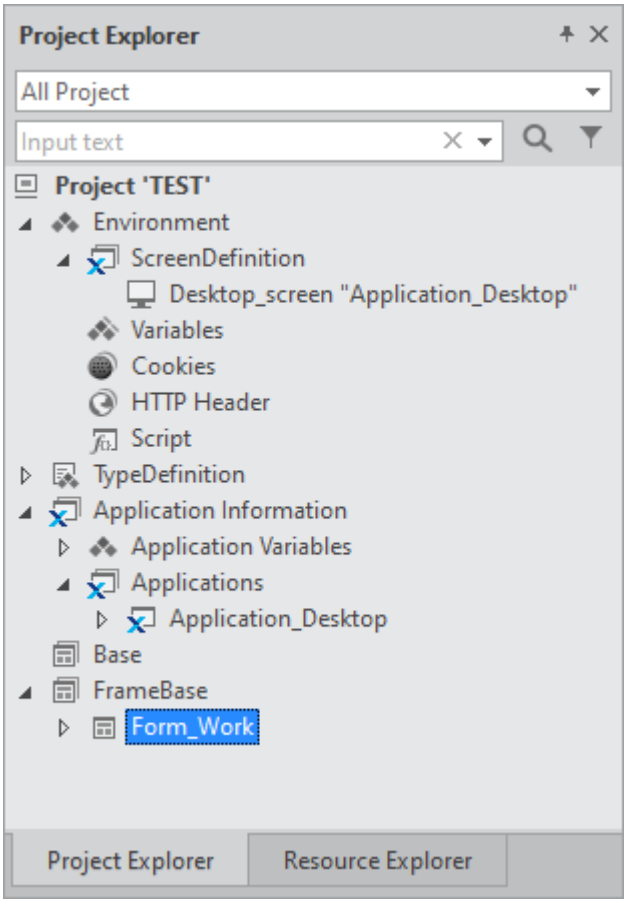
5.

Wizard | Project and Application

You can create a new project by using the Project Wizard. You can launch the wizard by accessing the menu [File > New > Project].

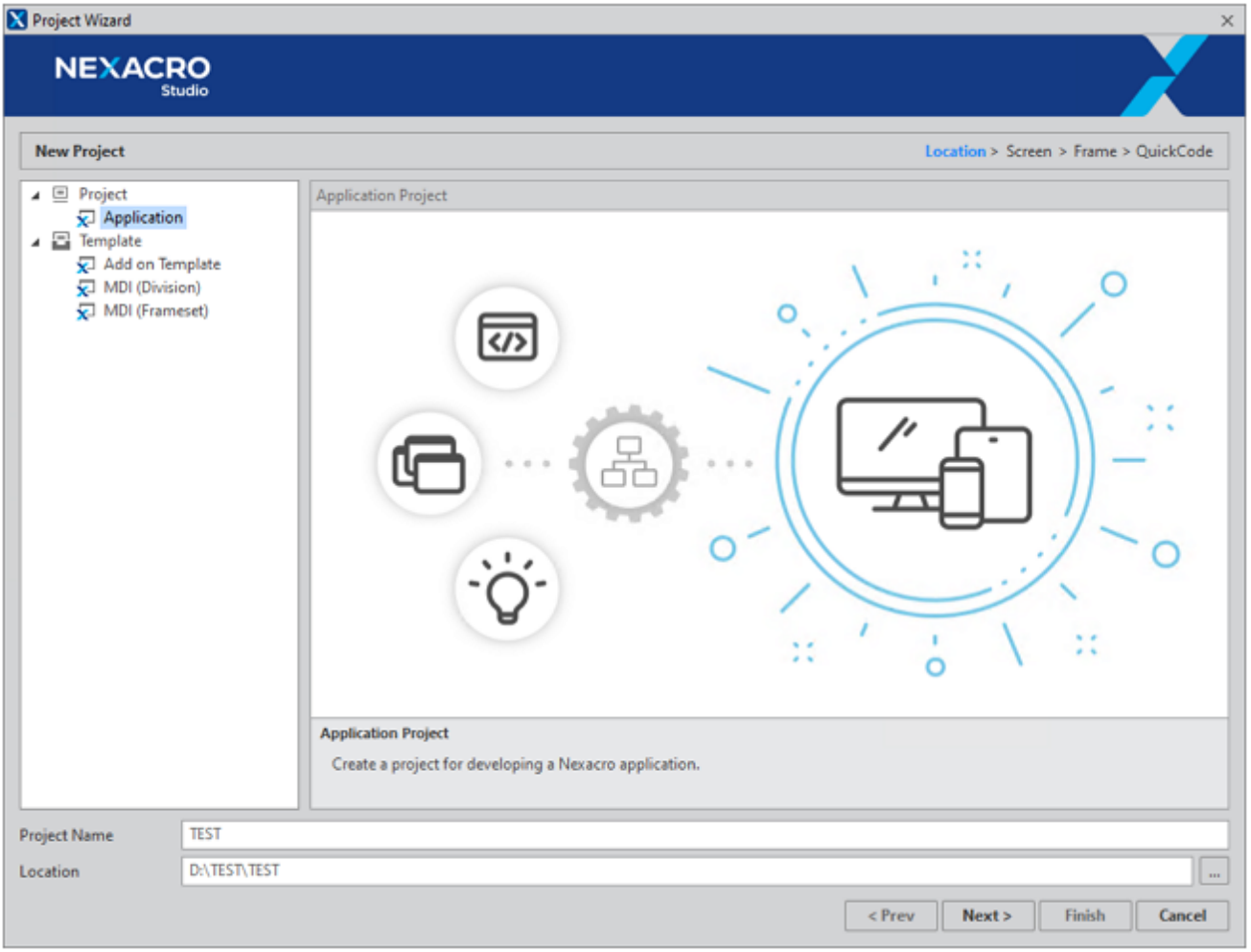
5.1 Project Wizard

Environment, TypeDefinition, Application Information files will be created.

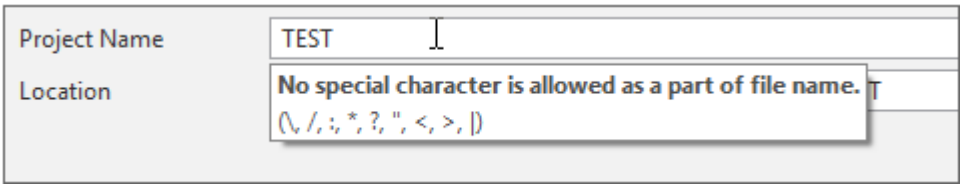


5.1.1 Step 1. Location

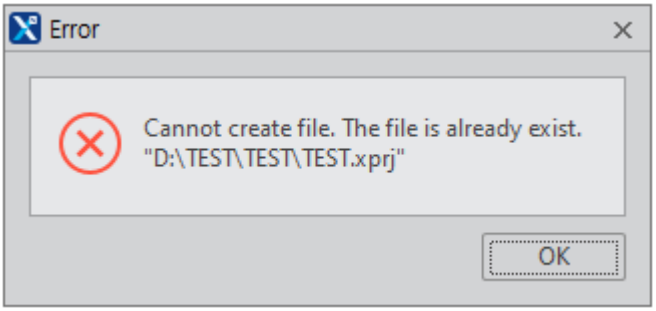
In this step, you can specify the type, name and location of a new project. The default location of a new project is the **Working Folder** designated in [**Tools** > **Options** > **Environment** > **General**].



The special characters presented in the below picture are not allowed as part of a project name.



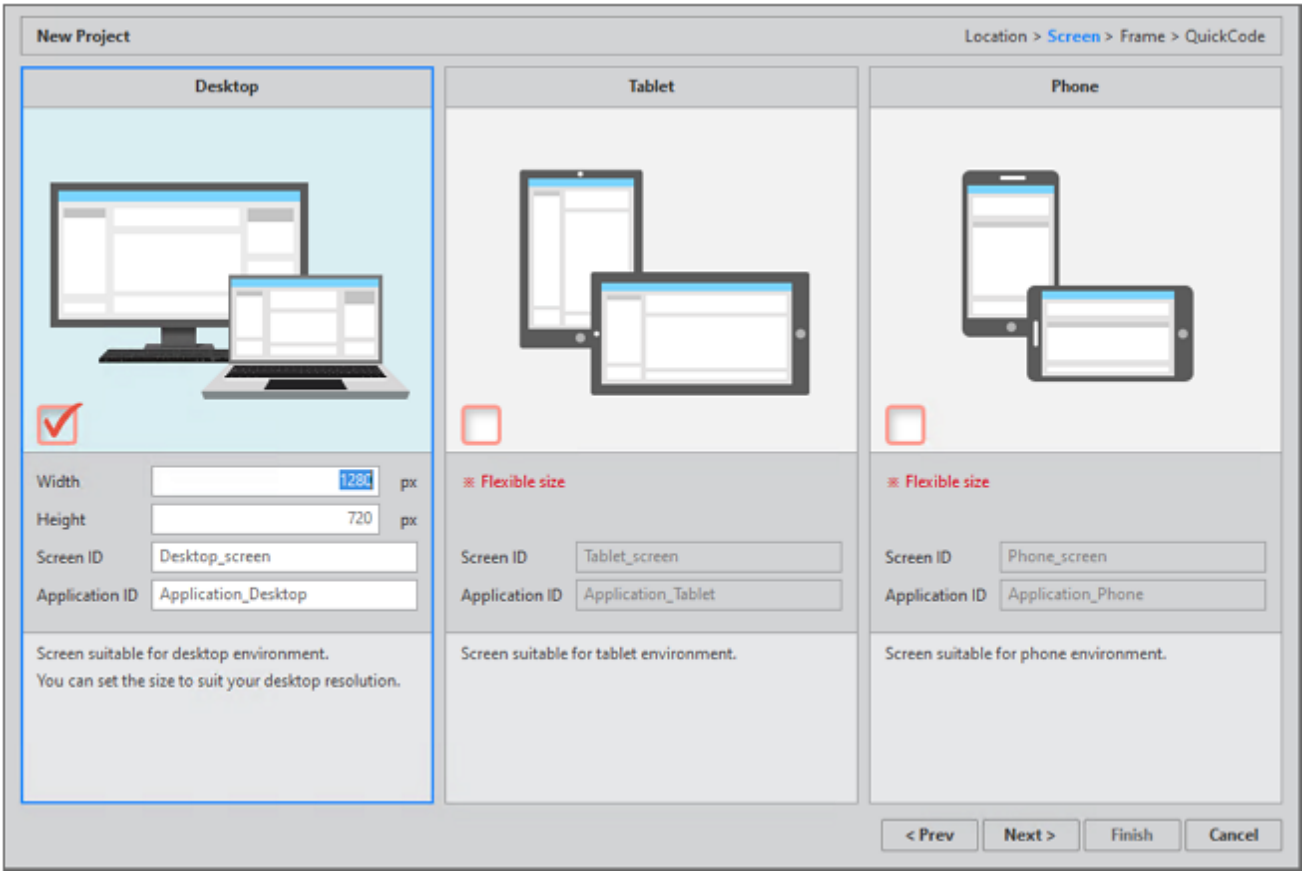
Projects in the same folder must have unique names.



As with any Windows environment, certain file names cannot be used with the Nexacro.
The following site identifies names that cannot be used.
https://msdn.microsoft.com/en-us/library/aa365247.aspx#naming_conventions

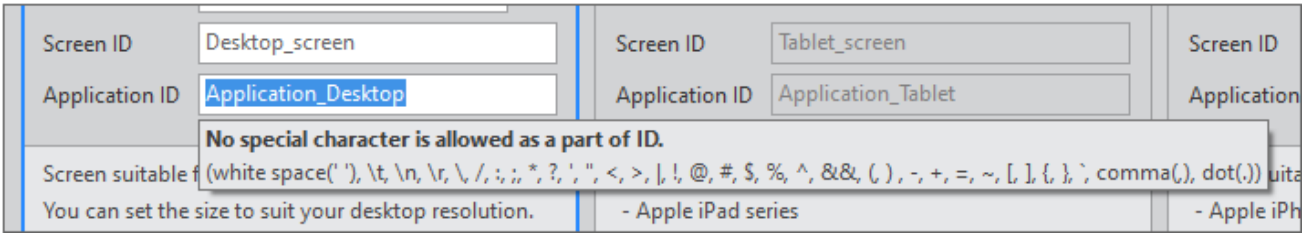
5.1.2 Step 2. Screen

This step is to set up the screen information to be used in a new project. A "screen" refers to a piece of information required to run an application (e.g. a screen size). You can make a list of screens in advance during the process of creating a new project. You can modify the list later through the ScreenDefinition editor.

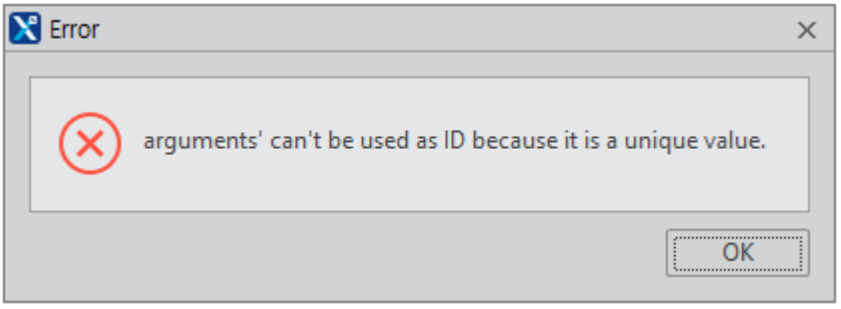


Call-out		Description
1	Width, Height	Width and height of the application
2	Screen ID	ID of a screen
3	Application ID	ID of an application. The ID of a created screen will be the value of the *screenid* property of the corresponding application.

Like the name of a Form, you cannot use the special characters presented in the below picture as part of a Screen ID. Moreover, a Screen ID cannot begin with a number.



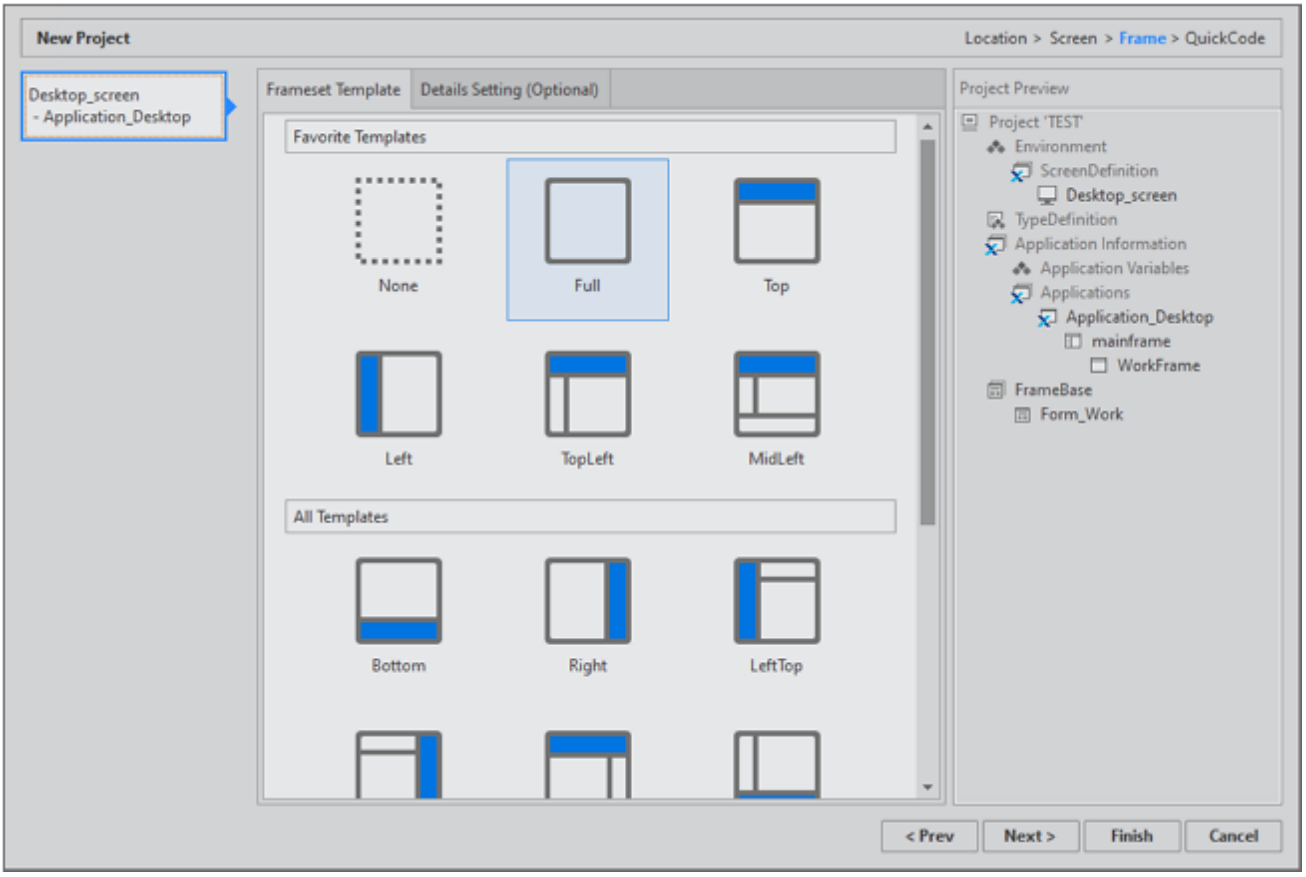
You are also prohibited to use the script reserved words managed by Nexacro Studio, which are presented by the below table.



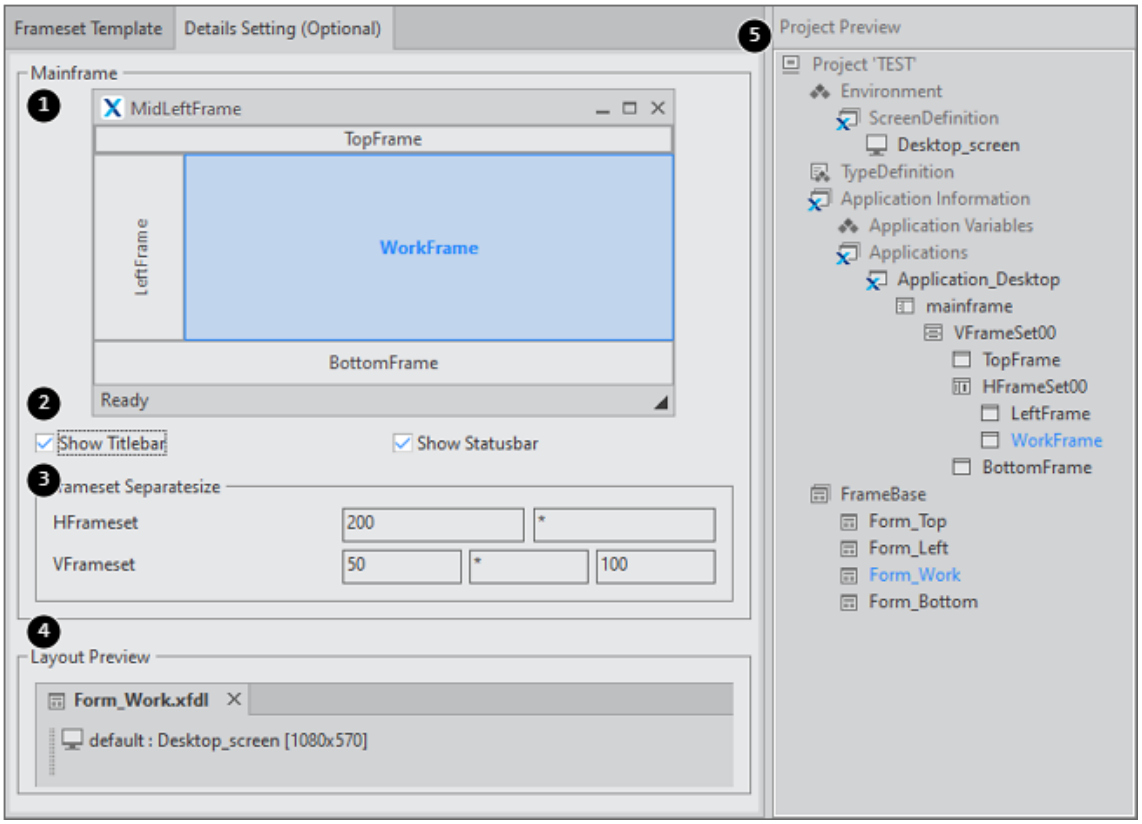
arguments	break	case	catch	class	const	continue
debugger	default	delete	do	else	enum	eval
export	extends	false	finally	for	function	if
implements	import	in	include	instanceof	interface	let
new	null	package	private	protected	public	return
static	super	switch	this	throw	throws	true
try	typeof	var	void	while	with	yield

5.1.3 Step 3. Frame

In this step, you select frames that will be used in the Application. You can select different frames for each screen, and you can change frames after the project is created.



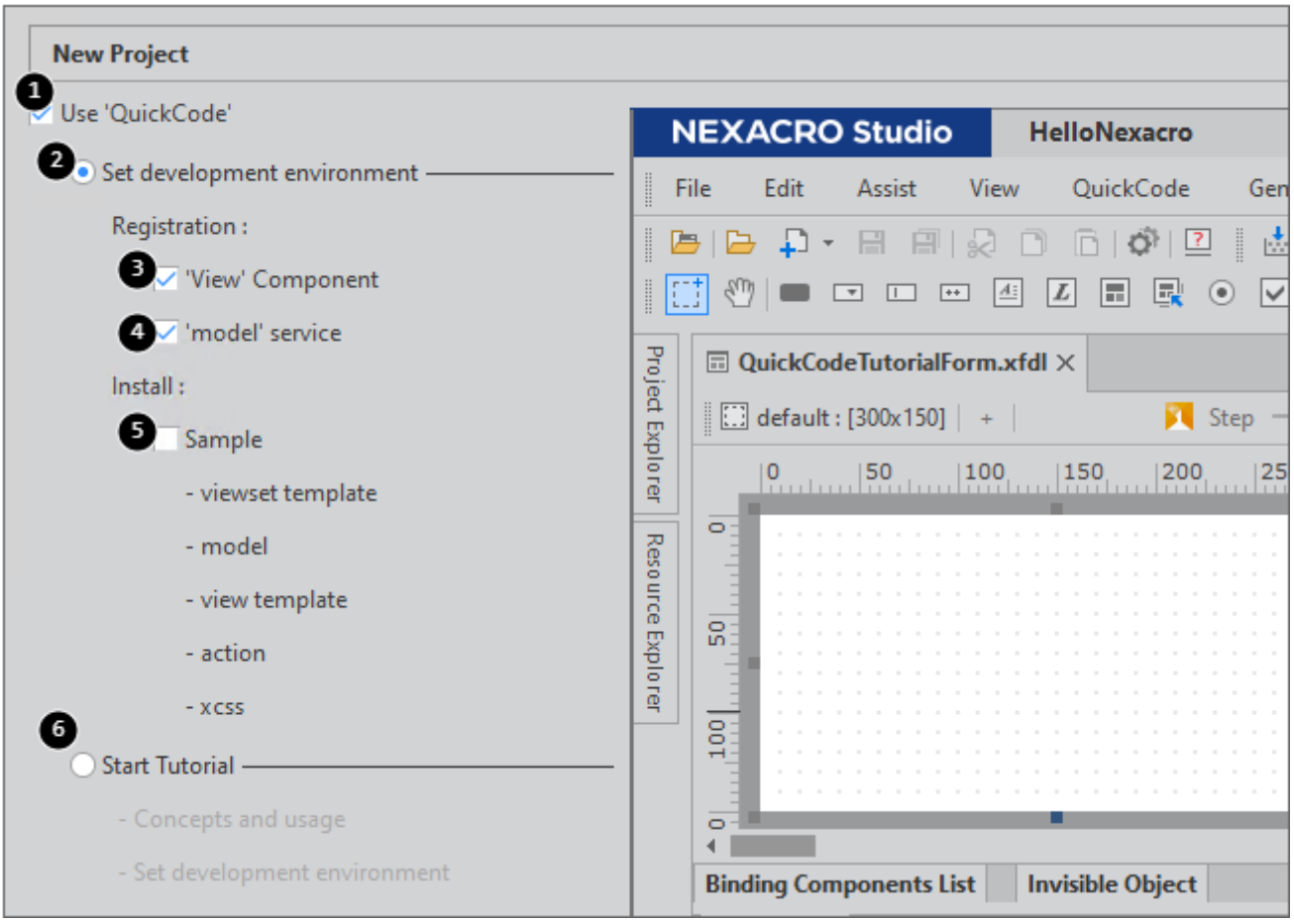
If you select a template other than 'none', Nexacro Studio will create a service named 'Frame' and a Form for each frame automatically. Click the 'Details Setting' tab to adjust detailed settings.



Call-out		Description
1	Frame Preview	Preview of an application's frame structure. You can select a ChildFrame through the Frame Preview, Project Preview or Layout Previw.
2		You can set the *showtitlebar* or *showstatusbar* properties of the MainFrame object.
3		You can set the *separatesize* property of the FrameSet.
4	Layout Preview	Preview of the layouts, each of which structures the Form connected to the relevant ChildFrame
5	Project Preview	Preview of a project's structure. It colors currently-edited elements blue.

5.1.4 Step 4 QuickCode

When using the QuickCode function, you can select the 4-step additional options.



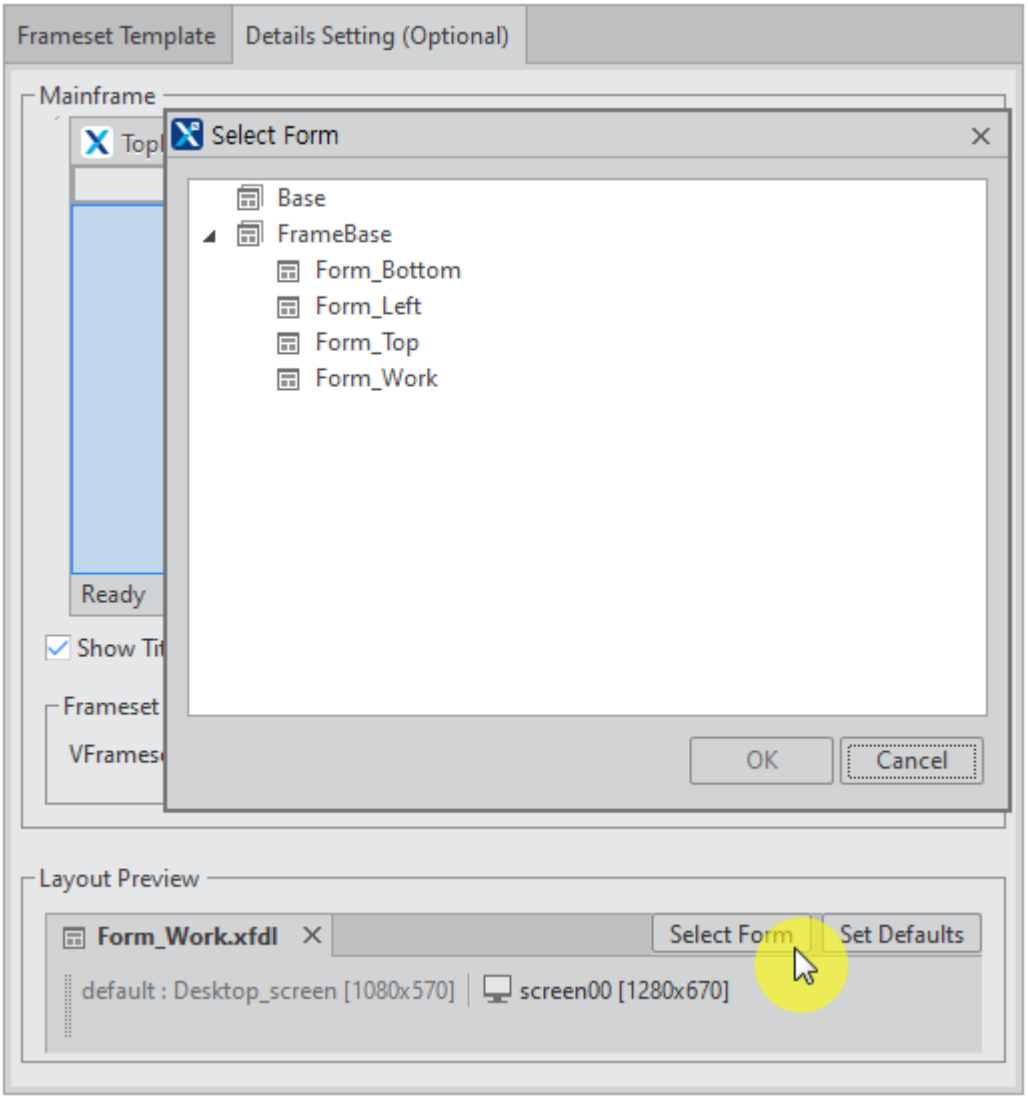
	Item	Description
1	Use 'QuickCode'	Enables the functions required to implement the QuickCode. You can select either the Set development environment or the Start Tutorial item.
2	Set development environment	Enables the QuickCode sub-function selected when creating the project.
3	Registration: View	Adds the View component. You can select the TypeDefinition > Objects item in Project Explorer and control from the ComComp.json list.
4	Registration: Model	Adds the sample Model item. Adds the "model" Type service with the name "svcQuickCodeTutorial" and adds the Model item as the sample.
5	Docking Pane: QuickCode	Enables the QuickCode panel. You can control from the menu [QuickCode > View > QuickCode].
6	Install: Sample	Installs the Viewset template, model, View template, and the Action sample file.
7	Start Tutorial	Runs the QuickCode Tutorial Wizard. If you click the [Cancel] button in the QuickCode Tutorial Wizard, then you can create a project without the QuickCode setting. After creating the project, you can run QuickCode Tutorial Wizard from the menu [QuickCode > Let's Try QuickCode].

5.2 Application Wizard

You can create and add a new application to an existing project through the Application Wizard (Project Wizard for a new application, strictly speaking). You launch this wizard by accessing the menu [File > New >Application].

The process of creating an application through the Application Wizard is almost the same as the Project Wizard.

What makes the Application Wizard different from the Project Wizard is that you can designate existing Forms when you configure ChildFrames.



	Button name	Description
1	Select Form	Designates a Form to connect to the relevant ChildFrame
2	Set Defaults	Designates default Forms for each ChildFrame. The Project Wizard will create such default Forms (e.g. A default Form named 'Form_Left' will be created for a LeftFrame.).

5.3 Project Template

Select the Template item in the Project Wizard to create a preset template project. Just specify Project Name and location for the project to be created immediately.

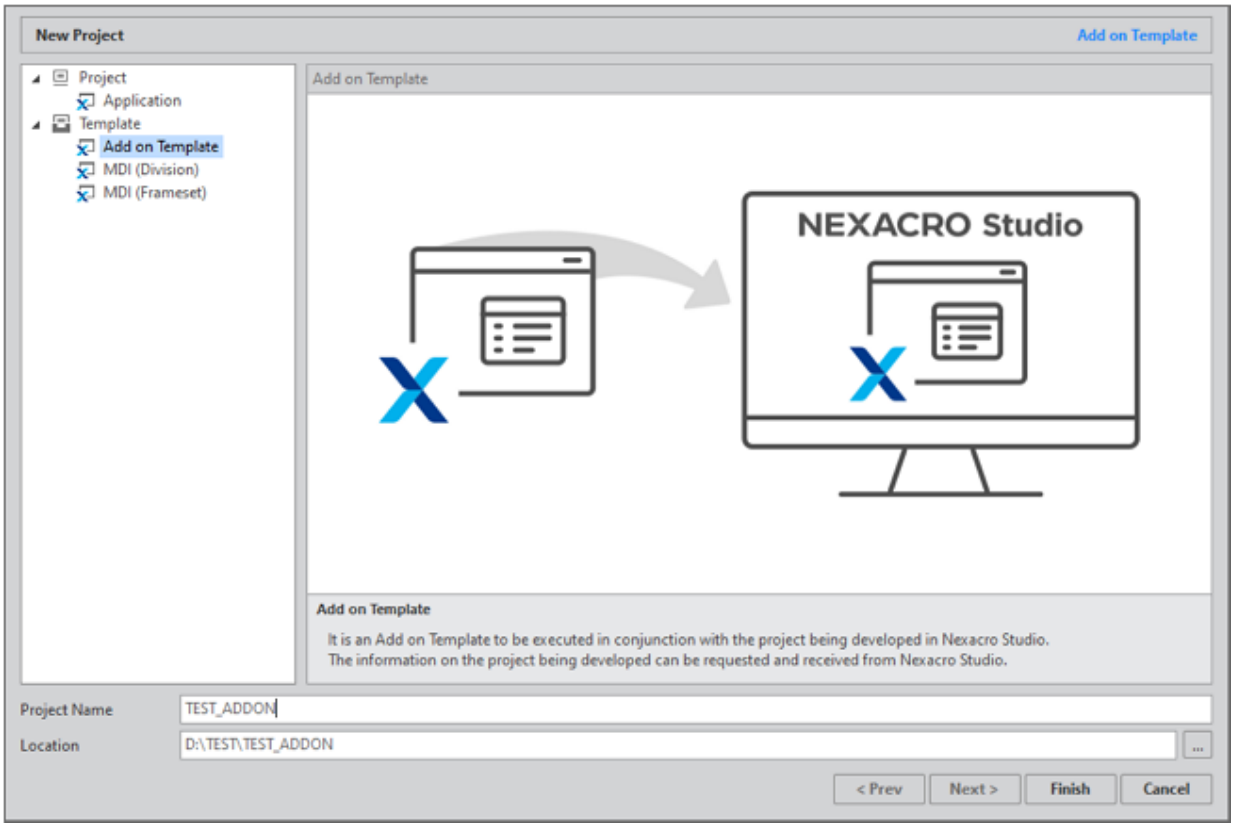
5.3.1 Add on Template

The Add on Template to check open project information in Nexacro Studio is provided. The project created after creating the template project is created and is registered and run in External Tools.

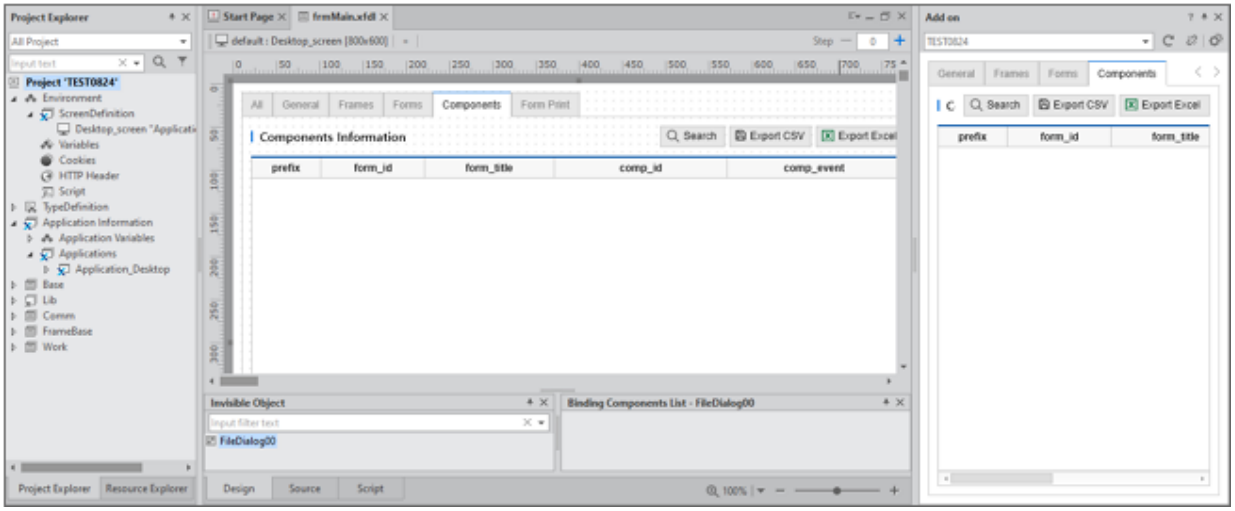
You can modify the provided template project to change it to your desired form. The Add on is not intended to be run alone, but can only be used to check the information of the open project in Nexacro Studio.

The Add on Template can be used as follows.

1. Select the [Template > Add on Template] item in Project Wizard.
2. Enter Project Name.
3. Click the [Finish] button.



As the Add on Template is run, the Add on window is displayed and automatically registered in External Tools.

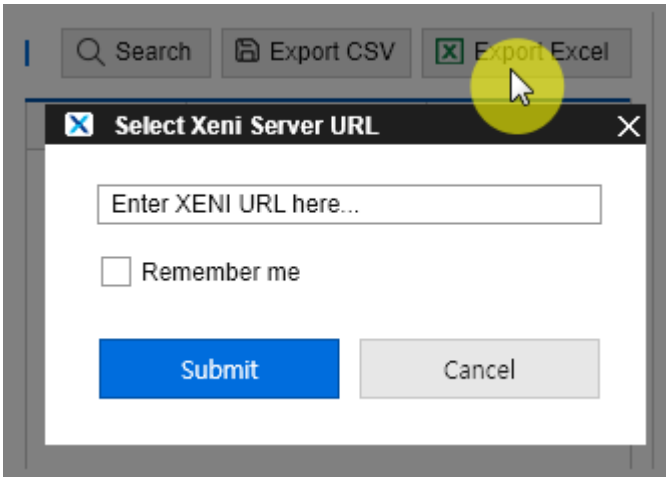


4 Select [Generate > Application] from the menu.

The default screen included in the template project is automatically generated, but the Generate Application task is required to use the entire function.

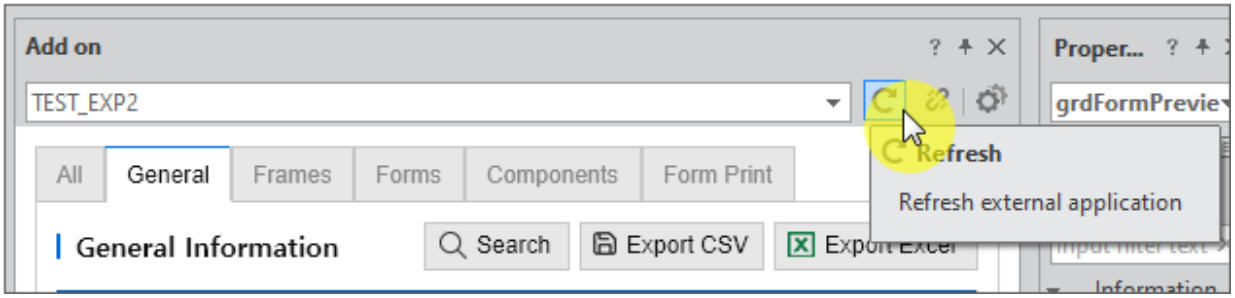
5 You can search project information by selecting the tab on the screen displayed in the Add on window.

The functions of each tab are as follows.

Tab Name	Description
All	Returns full project information. `nexacrostudio.request(requesttype.all)` Displays the returned JSON-formatted string in the TextArea component.
General Frames Forms Components	Returns information of the file used in the project. Puts the returned value in the `nexacrostudio.request(requestType.general, dataObjectID)` format in DataObject to display in the Grid component. The function of exporting output results as the CSV file, Excel file. ● The Excel file export uses XENI, and the XENI server URL connected must be entered. 

Tab Name	Description
Form Print	Returns the selected Form image and prints. `nexacrostudio.getFormImage(strFormurl)`

6 If you have modified the code, then click the Refresh button in the Add on window.



Even if the code is modified and generated, it is not reflected immediately.
For Add on, the generated code is reflected when the Refresh button is clicked.
You can use it when modifying and testing the template project.
You can also use the trace method to check the returned value after running the request method.

6.

Wizard | Form and Other Files

6.1 Form Creation Wizard

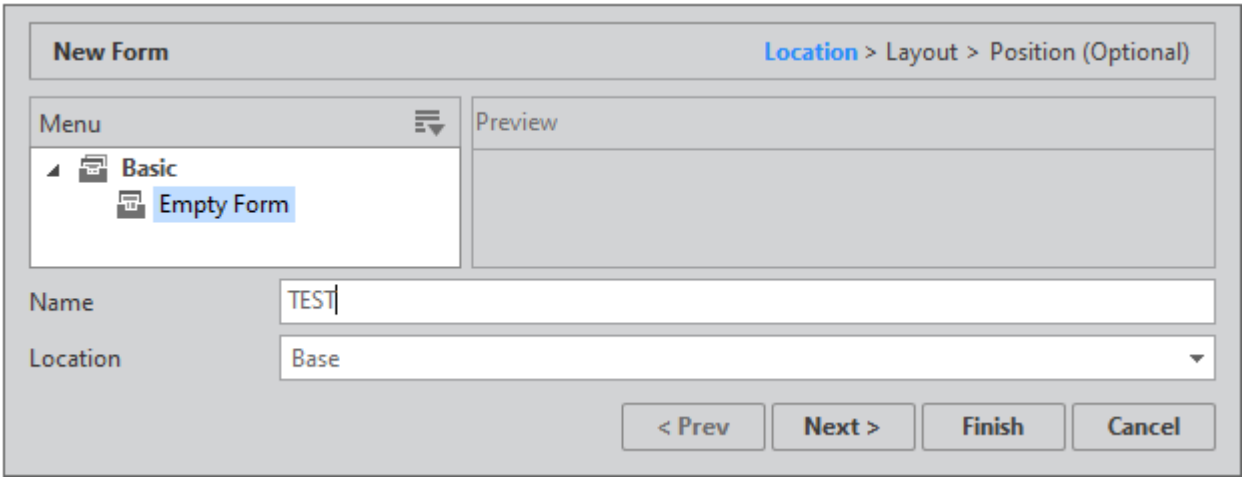
A new form can be created using the New Form Wizard, which is launched using [File > New > Form]



As soon as a Form is created, the corresponding editor view will be opened automatically.

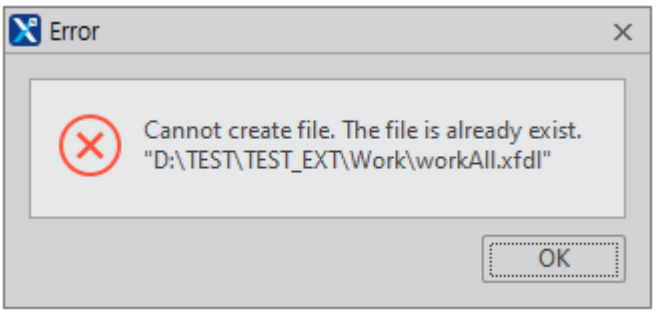
6.1.1 Step 1 Location

The first step is to determine the name and path of a new Form. You must enter a Form's “Name” and “Location.” A Form file is managed by a service, so you cannot designate paths other than a relevant service as a location of a Form file.



You can select among registered Form templates. As a Form template already defines its layout and position, you can click the Finish button to complete the creation of a Form without undergoing the later steps.

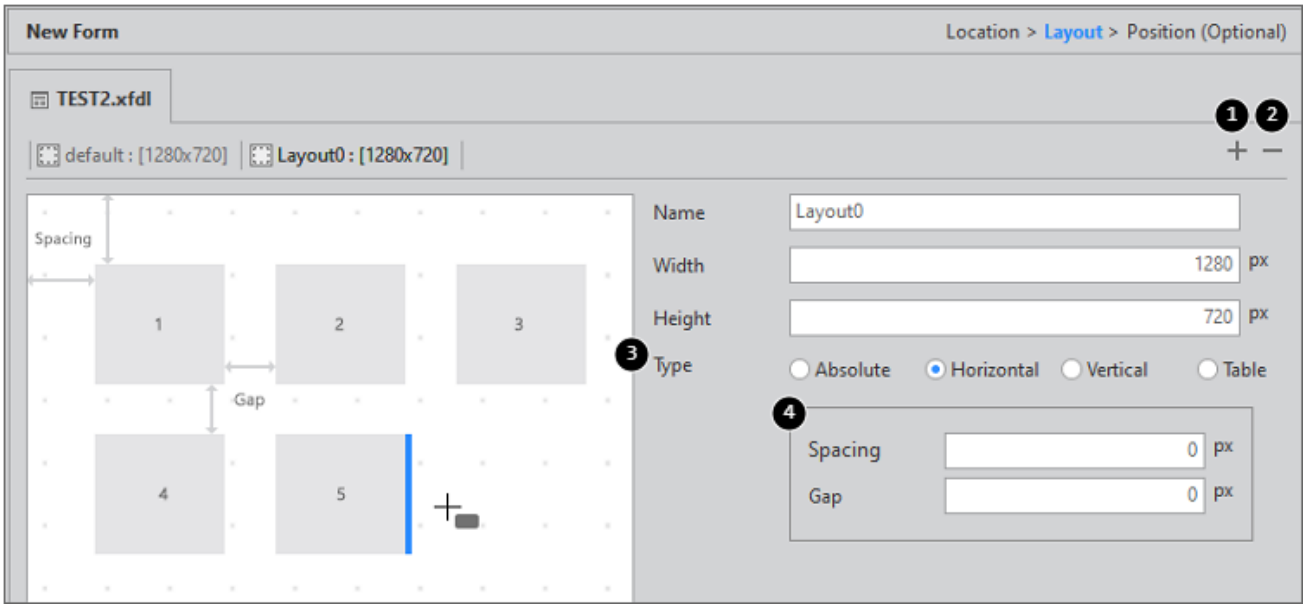
Forms in the same location must have unique names.



6.1.2 Step 2 Layout

In the Layout item, the layout information to be used in the Form is entered. You can add a layout and set property values according to the Layout Type. When the Layout Type is changed, the list of properties that can be set changes, and an example image that can be used as a reference when setting property values is displayed.

The example image is a fixed image and is not dynamically reflected when entering property values.



	call-out	Description
1	Add Layout	Adds a layout.
2	Delete Layout	Delete the selected layout. The default layout cannot be deleted.
3	Layout Type	Select a value for the layout type

	call-out	Description
		property.
4	Additional properties	Set additional properties according to the selected Layout Type.

The basic properties that can be designated when creating the layout information are as follows.

Layout property	Description
name	Layout name (UID) The default layout name cannot be changed.
width	Sets the width of the layout
height	Sets the height of the layout

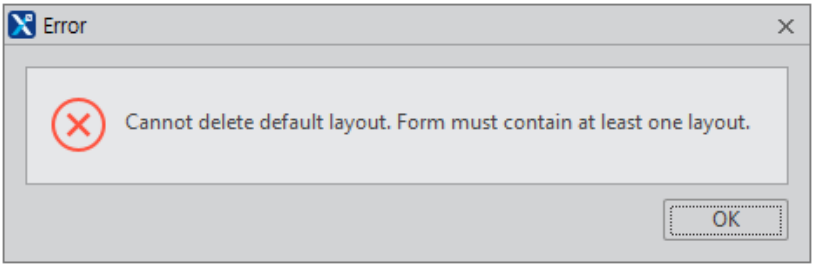
If you select an item from "Horizontal", "Vertical", or "Table" instead of "Absolute" for Layout Type, the additional Fluid Layout properties are displayed. The displayed properties are displayed differently according to the selected Layout Type.



Additional properties are items suggested to simply reflect the actual property values when creating a layout.
For example, when setting the Gap property, it is reflected in the horizontalgap and verticalgap property values of the actual Layout object.

Layout Type	Additional properties	Layout object property
Horizontal Vertical	Spacing	spacing
	Gap	horizontalgap, verticalgap
Table	Row	tabletemplate
	Column	
	Spacing	spacing
	Gap	horizontalgap, verticalgap

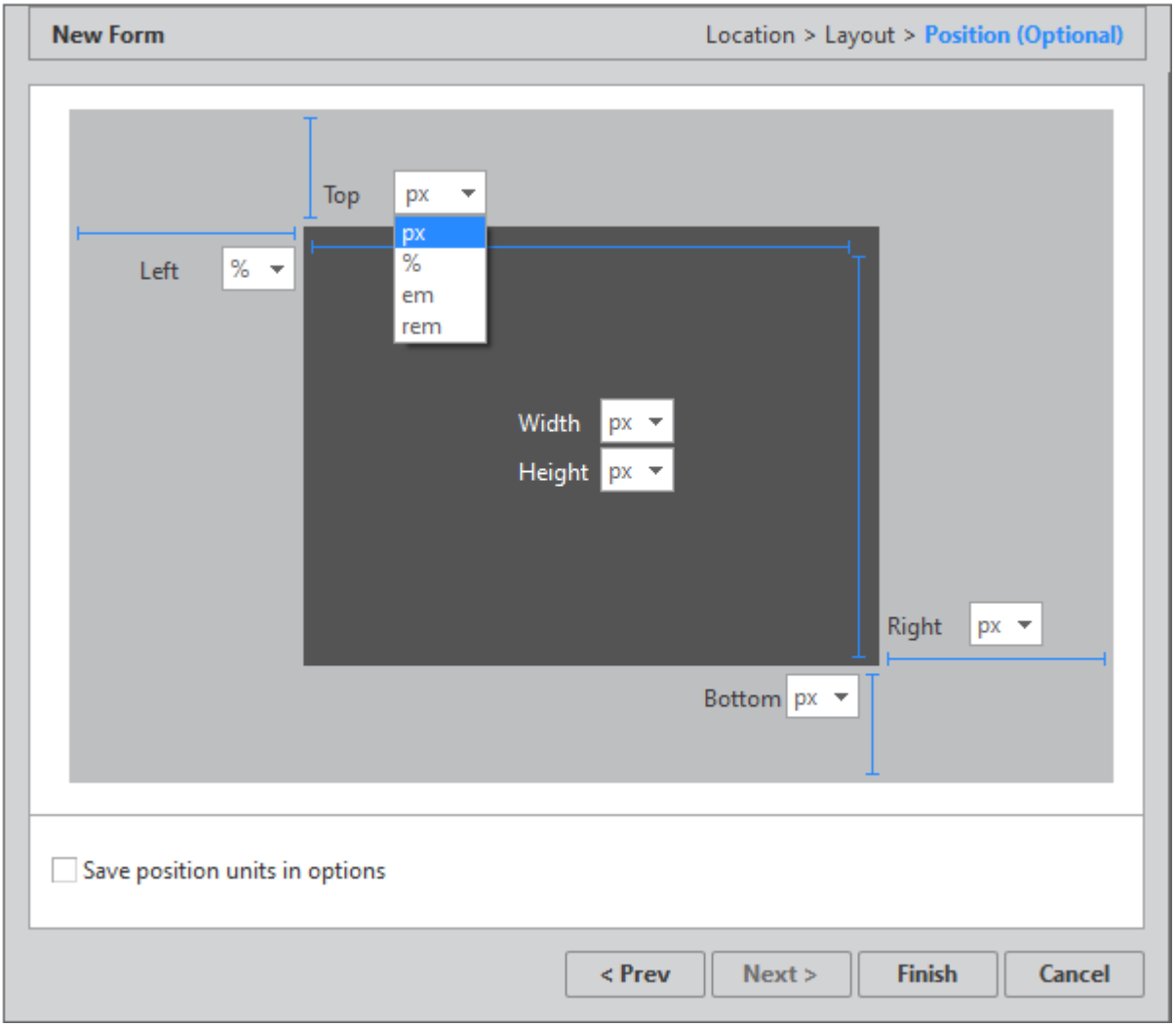
A form must have one layout at least, and you cannot delete a default layout because it is essential.



6.1.3 Step 3 Position

Set up the units for measuring the position when generating a component in a Form. The default is the value specified in options.

After setting up the units, click the checkbox beside 'Save position units in options' on the bottom left corner to save the designated units in options.



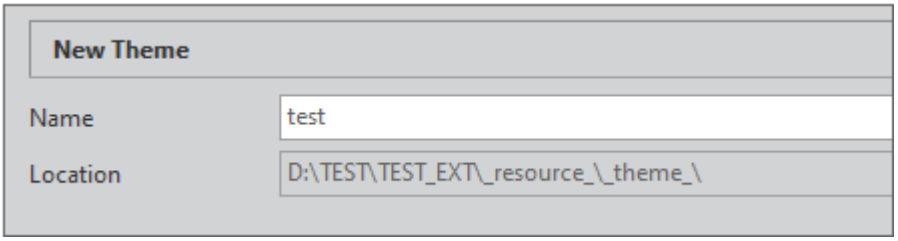
6.2 Create Other Files

Style files (CSS), XML files, script files, and theme files may be created using menu option **File > New**.

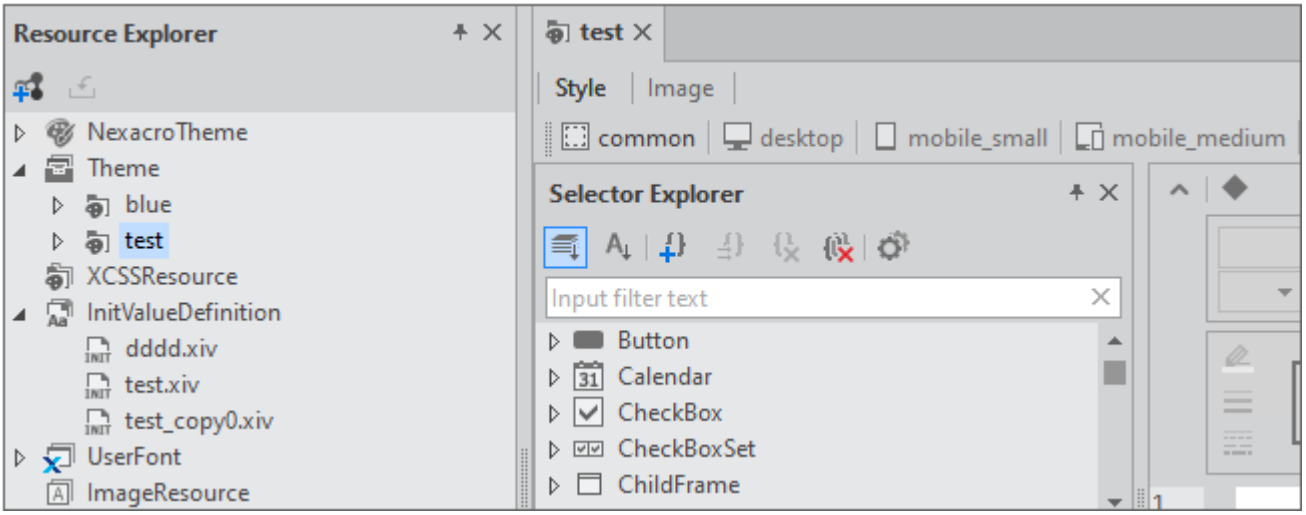
6.2.1 Create Theme Files

You can create a new theme file by accessing [**File** > **New** > **Theme**].

The 'Location' value is auto-populated based on the project path and cannot be changed. The 'Name' value cannot be created when there is a Theme with the same name in the path to be created.



As soon as a theme is created, the corresponding editor view will be opened automatically. A theme file is managed under the **Theme** directory in the Resource Explorer.



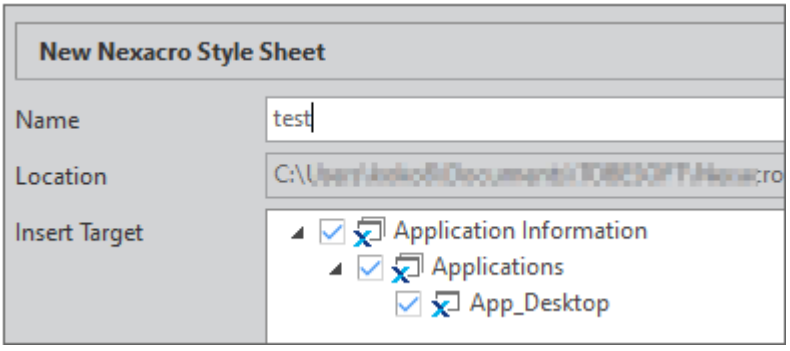
When you finish creating a theme, a folder of the specified name will be created under the relevant project. The folder includes an .xcss file and the *images* folder, the latter of which contains image files.

```
[project]
- [_resource_]
-- [_theme_]
--- [test]
---- theme.xcss
---- [images]
----- btn_CalDrop_D.png
----- btn_CalDrop_N.png
...
```

6.2.2 Create Style Sheet

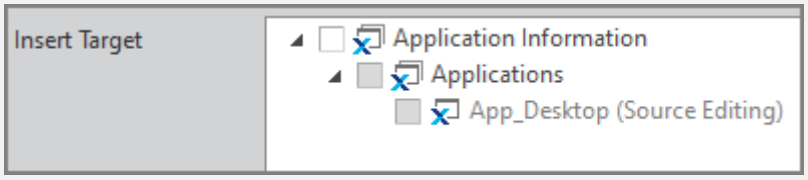
You can create a new style sheet file by accessing [**File** > **New** > **Style Sheet**]. The extension of the file is *.xcss*.

You must enter a XCSS file's "Name" and "Location." Moreover, XCSS files in the same location must have unique names. At "Insert Target," you must specify to which target you apply the XCSS.



As soon as an XCSS file is created, the corresponding editor view will be opened automatically. In the Project Explorer, the XCSS file will be registered in *Styles* of the Application specified as an insert target. In addition, it will be registered in *XCSSResource* in the Resource Explorer.

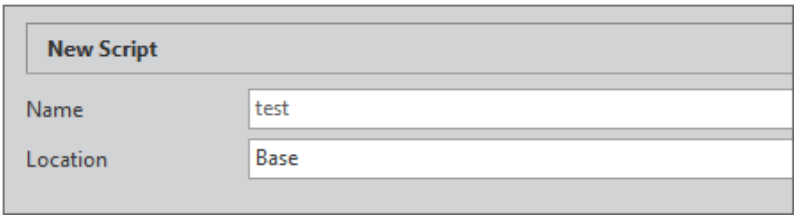
You cannot apply a CSS file to the application that is being edited—i.e. its editor view is currently opened.



6.2.3 Create Script Files

You can create a new script file by accessing [**File** > **New** > **Script**]. The extension of the file is *.xjs*.

You must enter an XJS file's "Name" and "Location." Moreover, XJS files in the same location must have unique names.



As soon as an XJS file is created, the corresponding editor view will be opened automatically. If an XJS file is created in a service path, it will be registered in the relevant service in the Project Explorer.

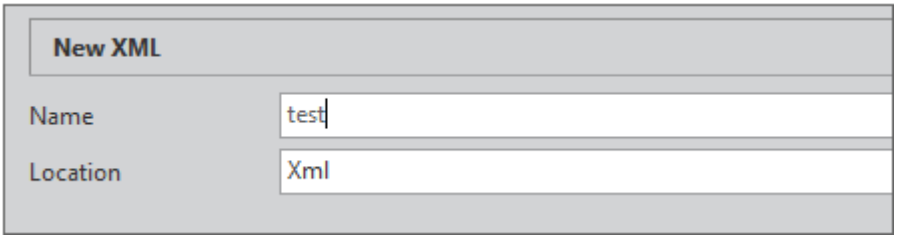
6.2.4 Create XML Files

You can create a new XML file by accessing [**FILE** > **New** > **XML**]. The extension of the file is *.xml*.

You must enter an XML file's "Name" and "Location." Moreover, XML files in the same location must have unique names.



When generating the XML file, only User Services with type set to "file" are shown in the "Location" field.

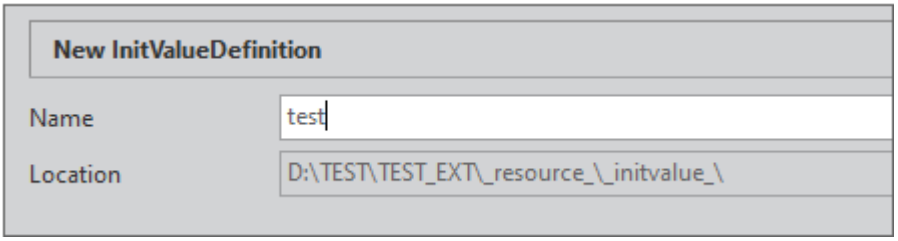


As soon as an XML file is created, the corresponding editor view will be opened automatically. If an XML file is created in a service path, it will be registered in the relevant service in the Project Explorer.

6.2.5 Create InitValue Files

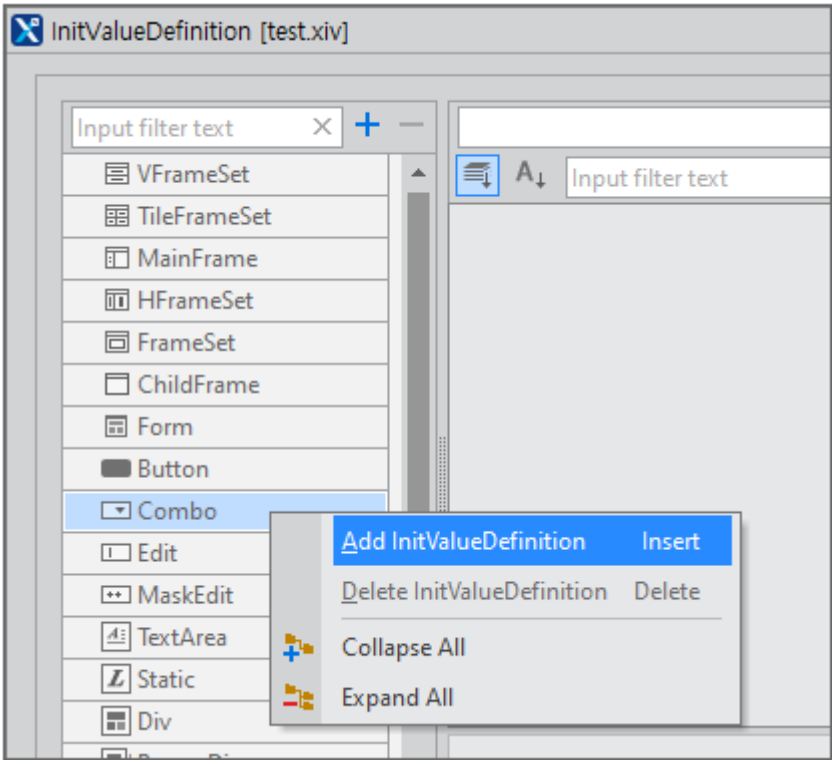
You can create a new InitValue file by accessing the menu [**File** > **New** > **InitValueDefinition**]. The extension of the file is *.xiv* and the created file will be registered in the Resource Explorer.

Nexacro Studio will create a folder named "_initvalue_" under the relevant project folder in your computer. An InitValue file will be saved into this *_resource_/_initvalue_* folder.

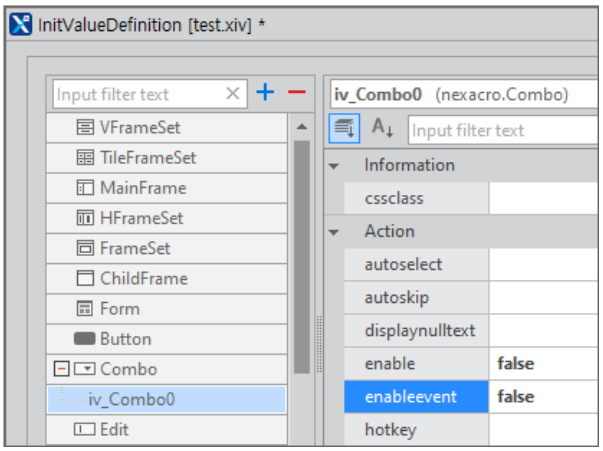


> You cannot change the location for creating an InitValueDefinition file. You have to use the predetermined path.

Creating an InitValue file will prompt the opening of the InitValueDefinition window, in which you can choose objects or components that you want to include in the InitValue file. Then, right-click an object or component to select the context menu [Add InitValueDefinition]. Otherwise, select an object or component and then click the "+" icon at the top of the pane.



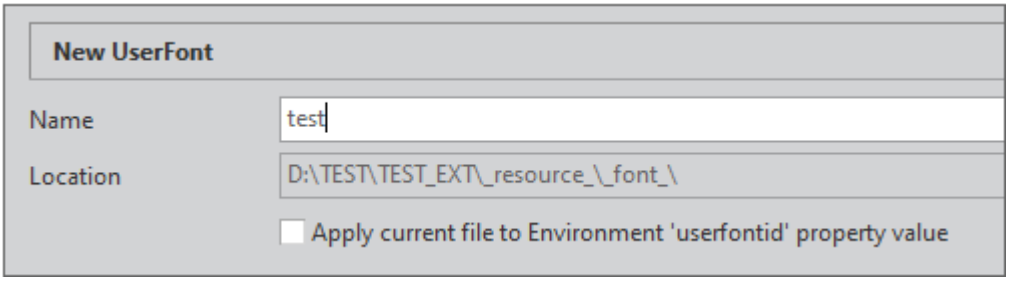
You can see an ID created under the relevant object or component, and also see that the right-hand pane is activated with the list of properties. In these circumstances, you can modify the InitValue file by specifying properties. Clicking the [Source] tab allows you to check the source codes of a created or modified InitValue file.



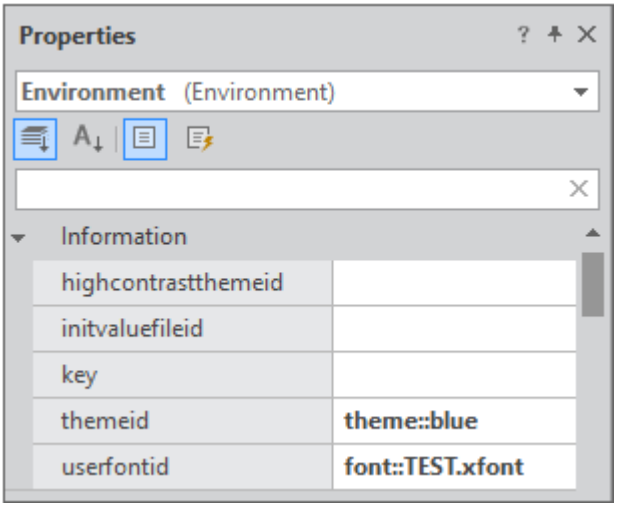
6.2.6 Create UserFont Files

You can create a new UserFont file by accessing the menu [**File** > **New** > **UserFont**]. The extension of the file is *.xfont*, and the created file will be registered in the Resource Explorer.

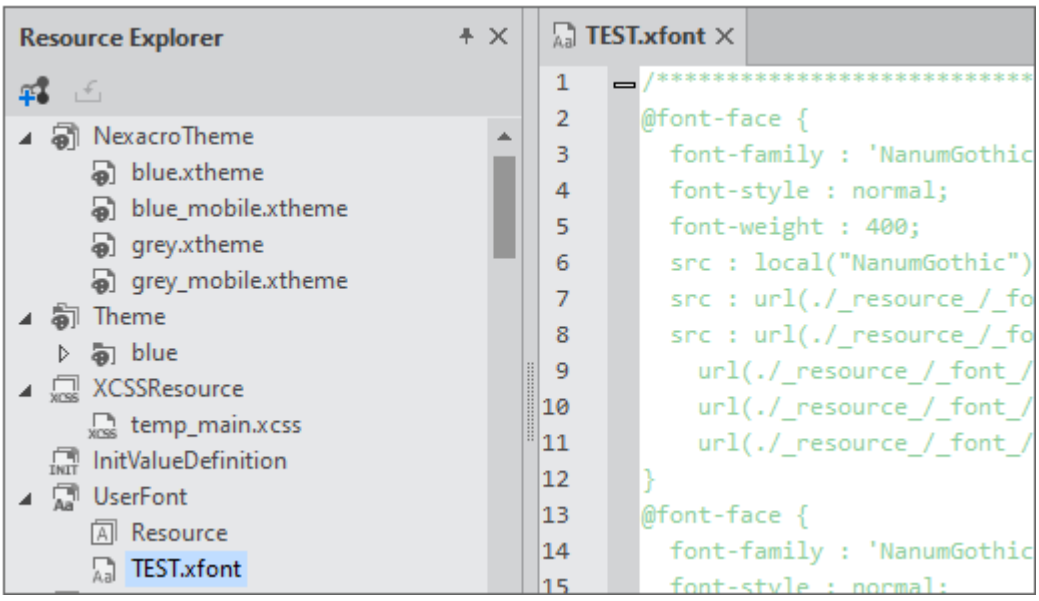
Nexacro Studio will create a folder named "_font_" under the relevant project folder in your computer. A UserFont file will be saved into this *_font_* folder.



If you check the **Apply current file to Environment 'userfontid' property value** option, a created user font will apply to the *userfontid* property of the Environment object. You can change or delete the value of the property later on.



A UserFont file is created with sample codes included in it. You can apply the created user font by manipulating the sample codes as you want and then removing the comments. If you generate the codes included in a UserFont file, they will be incorporated into the inside of the <style> element of the created HTML file.



In case a Nexacro application runs on the NRE, your operating system must have the fonts that are defined as user fonts.

In Nexacro Studio, you cannot find a user font in the Properties pane if the font is not installed in the operating system. In this case, you should enter the name of the font manually to the Properties pane.

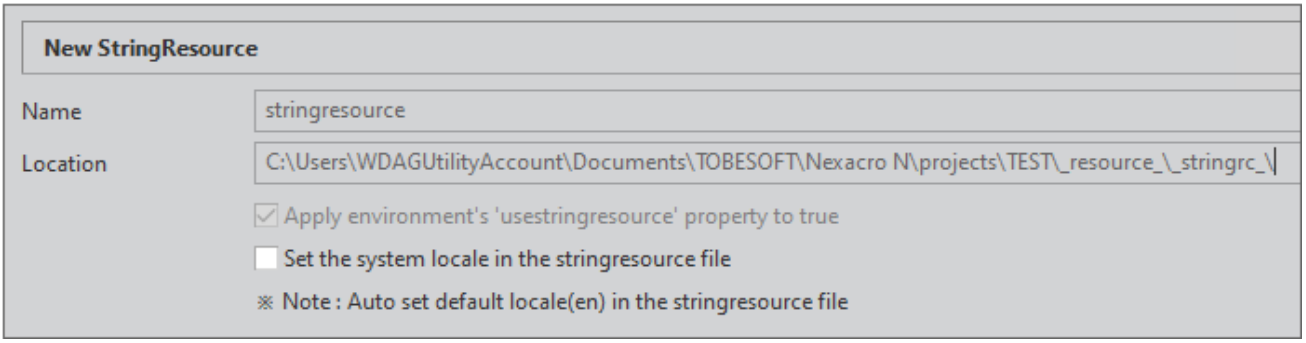
If you want to use an external service like Google Fonts, you can define your user font by specifying the relevant CSS URL, as presented below.

```
@import url(//fonts.googleapis.com/earlyaccess/hannari.css);
```

6.2.7 Creating a StringResource file

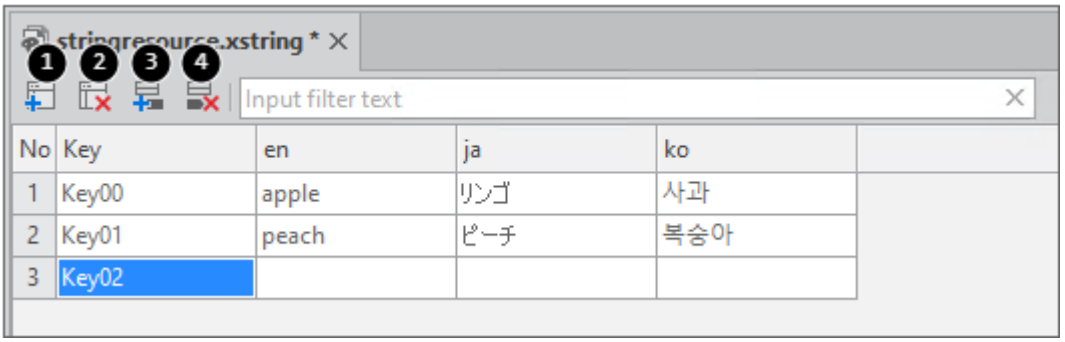
You can create a new StringResource file and run it from the menu [File > New > StringResource]. The file that can be created has the extension, "*.xstring." The StringResource file created will be registered in Resource Explorer.

In the case of the StringResource file, create a [_stringrc_] folder under the project folder and create the file in the folder.



When the StringResource file is created, the value of the usestringresource property of Environment will be set to true.

If the "Set the system locale in the stringresource file" option checked off, the language set in the system locale will be added as the locale item. If this option is not checked off, only the default language, English (en), will be set as the locale item. Locale items may be added or deleted after the StringResource file is created.



	Item	Description
1	Add Locale	Add a locale item to apply when setting up multiple languages.
2	Del Locale	Delete the selected locale item among the set locale items.
3	Add Key	Add words subject to the application of multiple languages.
4	Del Key	Delete the data corresponding to the selected key.

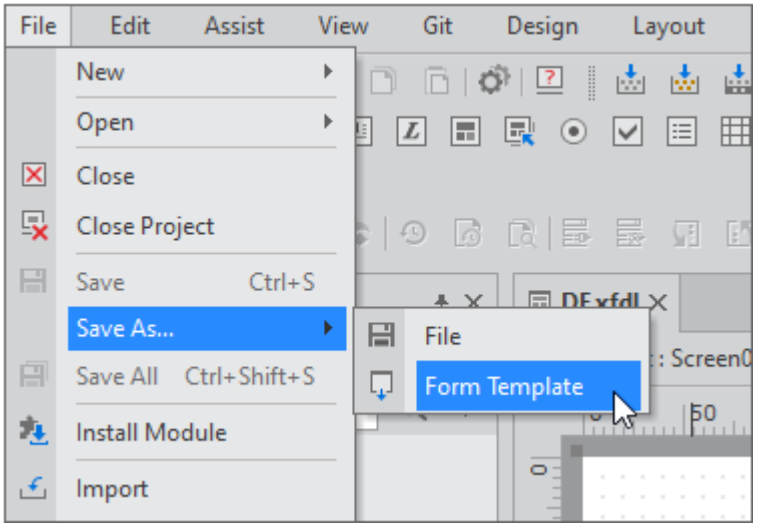
6.3 Register and Use Form Templates

A Form template is such that you designate a standard format used in your project or you register a frequently-used screen type. When you create a Form, you can choose among those templates.

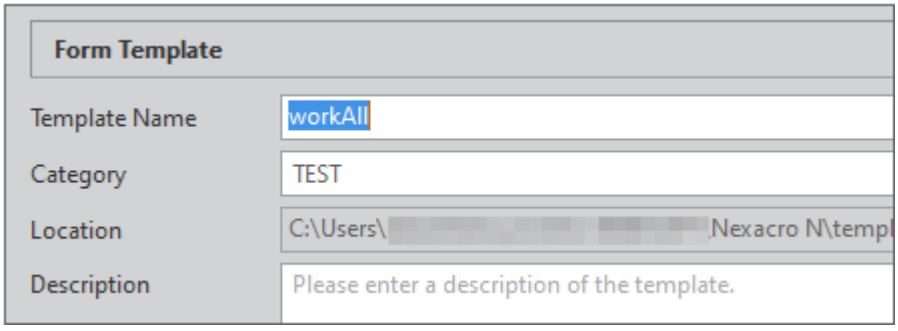
6.3.1 Register a Form Template

You can register a Form template by following the below steps.

1. Create a Form.
2. Arrange components and add necessary properties and script.
3. Select the menu [File > Save As > Form Template].



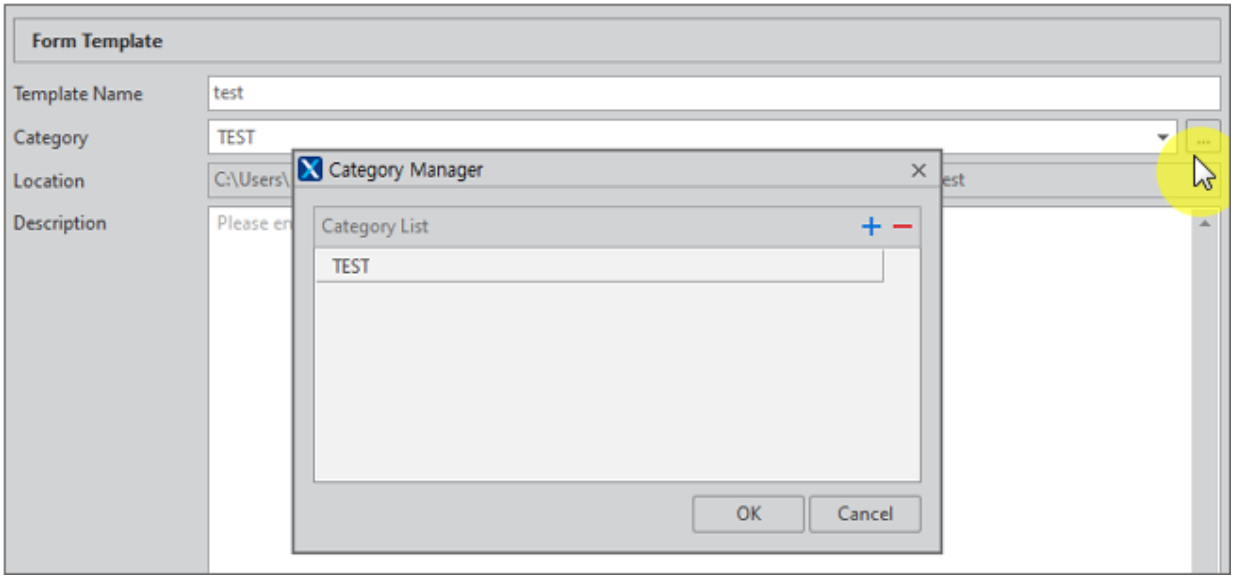
4. Write the name and description of the template.



The default storage path is as follows. After running the menu [Tools > Options], you can change the storage path managed in the project in [Project > QuickCode > Form Template].

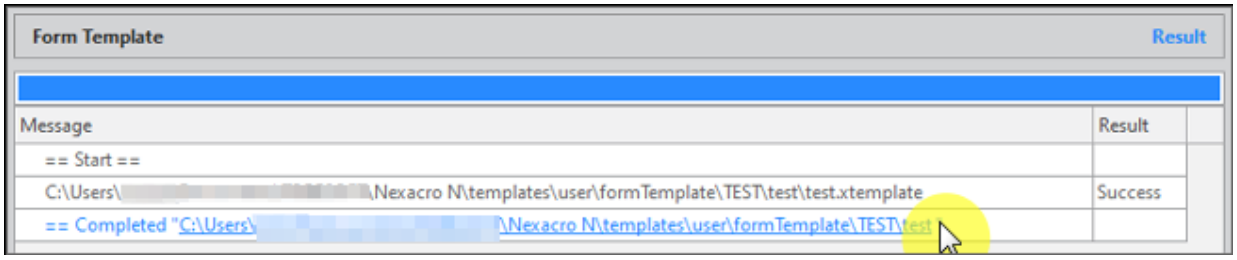
```
C:\Users\[User Name]\Documents\TOBESOFT\Nexacro
N\templates\user\formTemplate\[Category Name]\[Template Name]
C:\Users\[User Name]\Documents\TOBESOFT\Nexacro
N\templates\user\formTemplate\[Template Name]
```

When registering the form template, you can add or change category items. If there is no category item added, then register the form template directly under the Form Templates default path. If there are many form templates to manage, then it is recommended to create categories to manage.



When adding the category. The category name is added in the ctgr0 format. You can edit the category name by selecting the item and then pressing the F2 key.

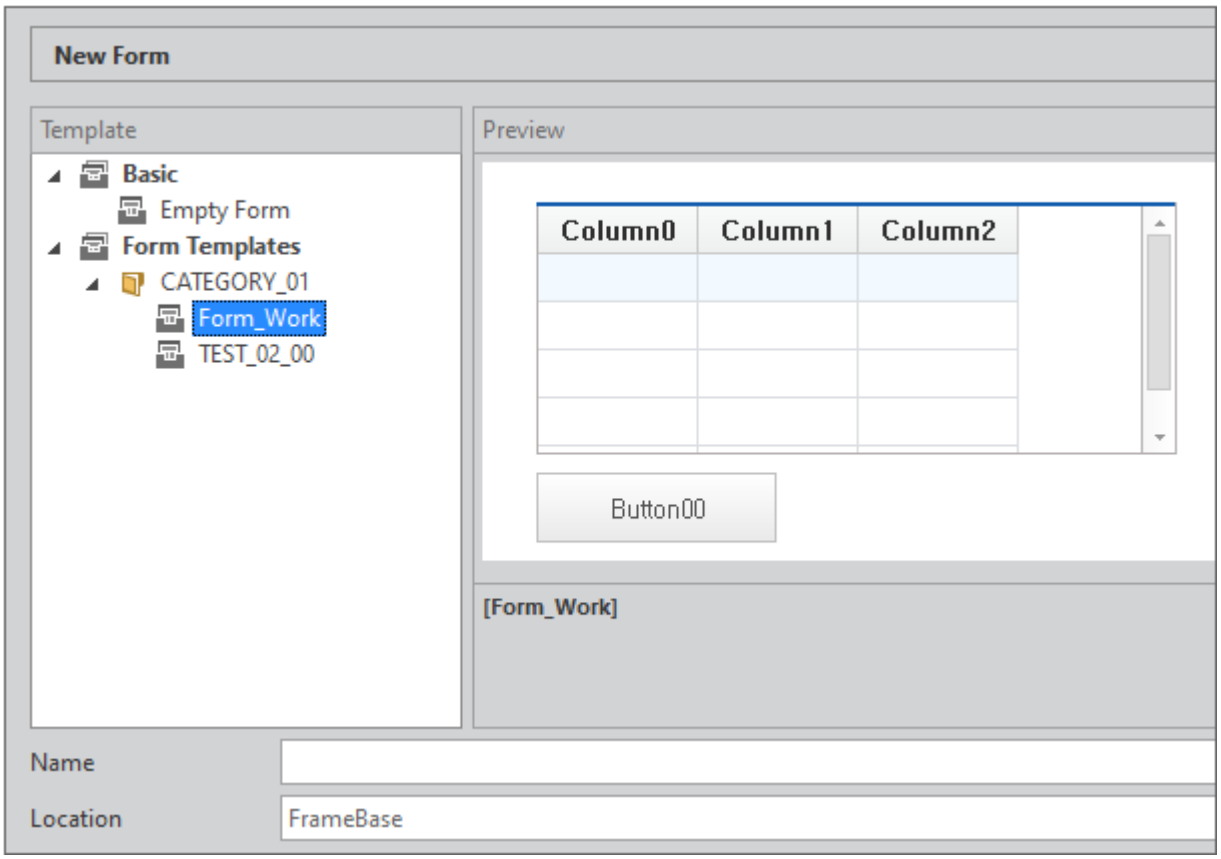
5 Upon the click of the Create button, the template is created and the results are presented. Click the link provided in the results to open the containing folder.



6.3.2 Create a Form by using a Form Template

1 Select the menu [File > New > Form].

2 Choose a template from the Form Wizard. You can see the preview of the selected Form template.



3 A Form is created based on the selected Form template after you designate the location of the Form and click the Finish button.

6.4 Registering & Using ViewSet Template

The ViewSet template can specify standard layouts used within the project by using the View component. The ViewSet template created can be selected when creating a new form.

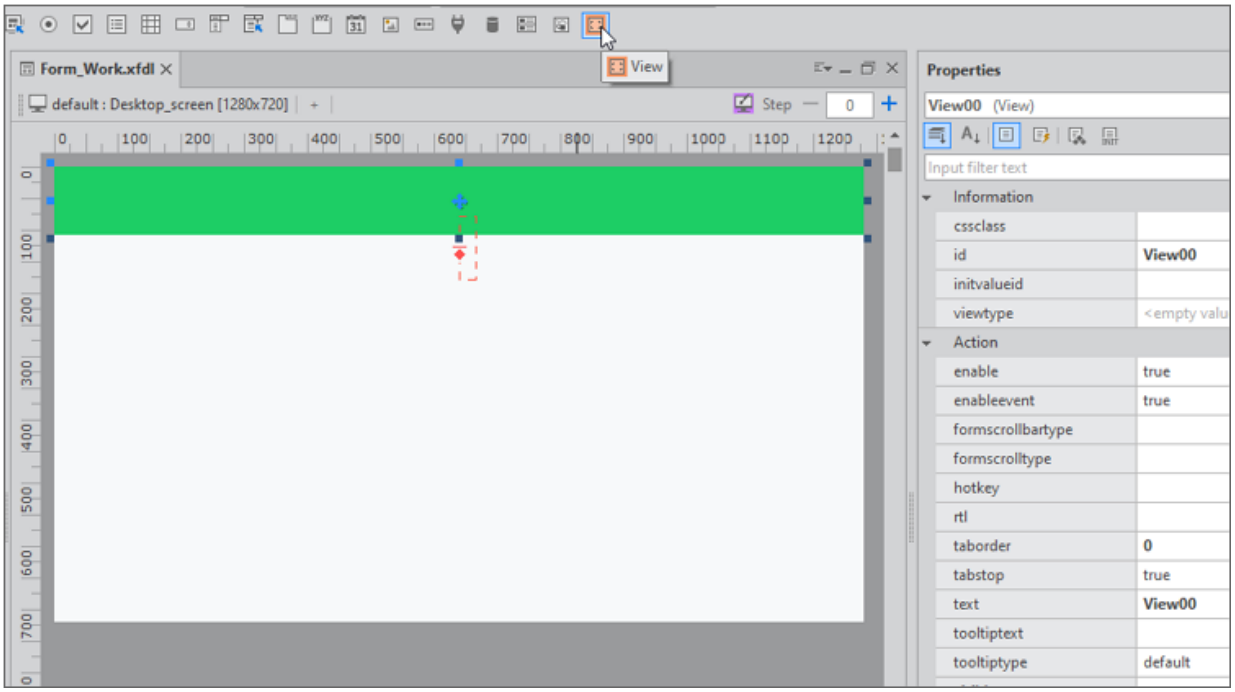
6.4.1 Registering ViewSet Template

The form template can be registered as follows.

1 Create a new form.

2 Configure the screen layout for the ViewSet template using the View component.

The View component holds the layout to place the component that will actually operate. Define the layout type that is frequently used in the entire project first and then make it the ViewSet template.



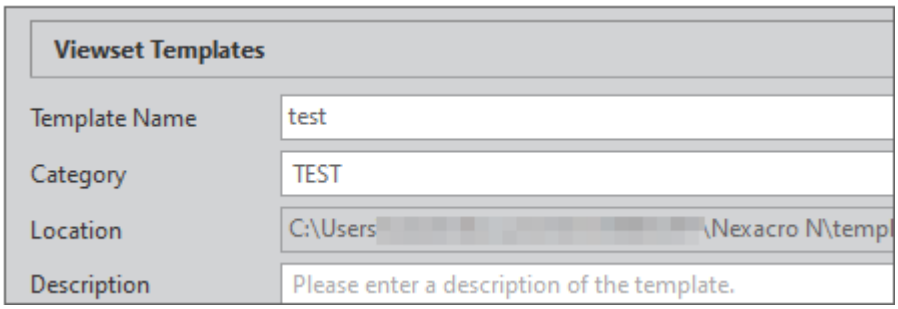
When creating the ViewSet template, other components can be placed in the View component, but the components are deleted from the template code when saved as the ViewSet template. Components to be operated in the View component are automatically created based on the Model.

The View component cannot be placed in ContainerComponent, such as the Div component. The View component can only be registered as the child of the Form object. Even if you place the View component inside the Div component area in Nexacro Studio, it will not be included in the Div component.

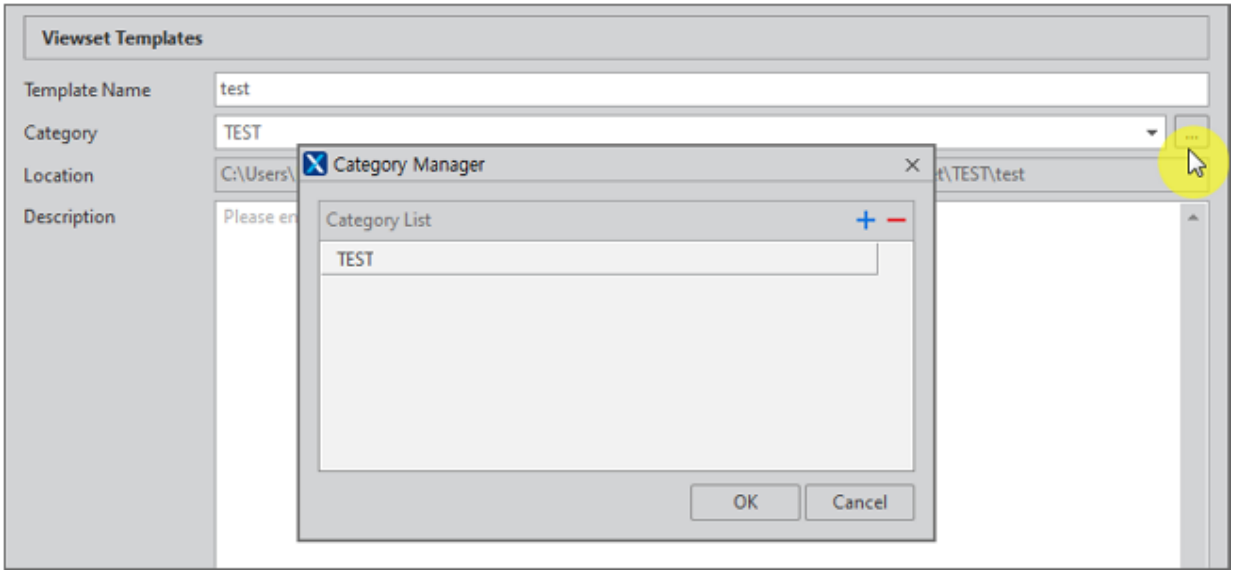
3 Select the menu [File > Save As > Form Template] item.

When saving the Form that includes the View component, ViewSet Template Wizard is executed, and when the View component is not included, Form Template Wizard is executed.

4 Fill in the Template Name and Category items and then click the [Create] button.



Category can be added or changed if necessary. If Category is not selected, then the ViewSet template is displayed directly under the Viewset Templates item. If you have many ViewSet templates that you use, then it is recommended to create Category to manage.



The default storage path is as follows. After running the menu [Tools > Options], you can change the storage path managed in the project in [Project > QuickCode > Form Template]. The "viewset" folder is created based on the set path and the ViewSet form template storage path is created thereunder.

```
C:\Users\[User Name]\Documents\TOBESOFT\Nexacro N\templates\user\formTemplate\viewset\[Category Name]\[Template Name]
C:\Users\[User Name]\Documents\TOBESOFT\Nexacro N\templates\user\formTemplate\viewset\[Template Name]
```

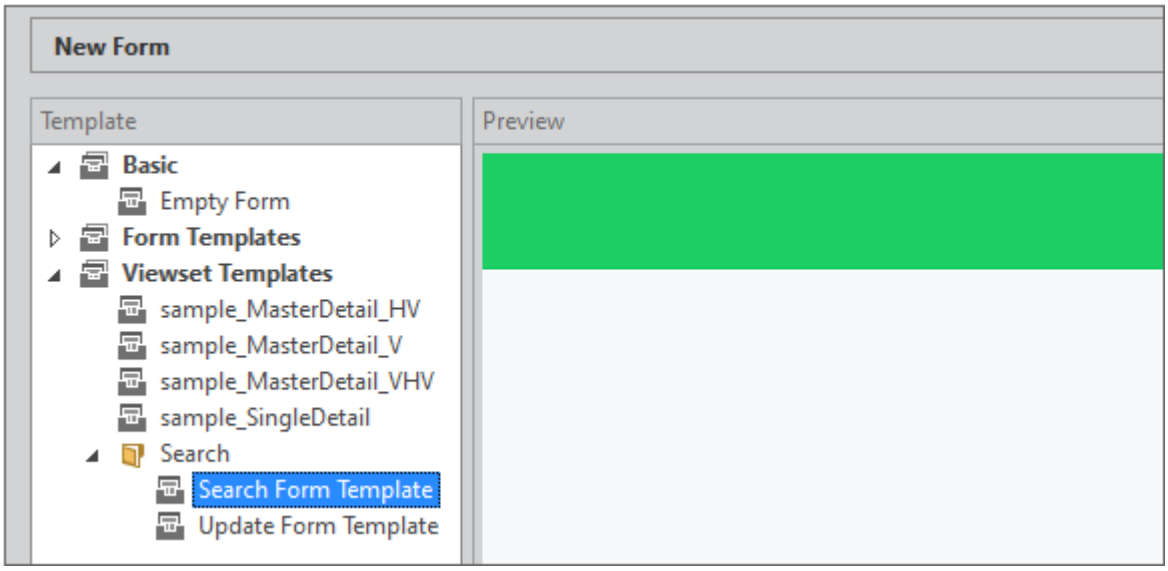
6.4.2 Creating New Form with ViewSet Template

The Viewset Templates item is only displayed when the View component is added to the project TypeDefinition > Objects list.

1. Select the menu [File > New > Form] item.

2. Select the ViewSet template under the Viewset Templates item on the Form Wizard screen. You can preview the configuration of the selected ViewSet template.

   If the Category is set, then you can check the ViewSet template under the corresponding Category folder.



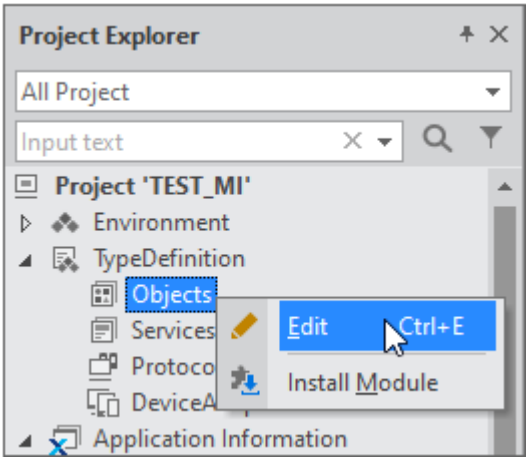
3. Enter Name, select Location, and click the [Finish] button to create a new Form with the selected ViewSet template.

7.

Project Editing | TypeDefinition

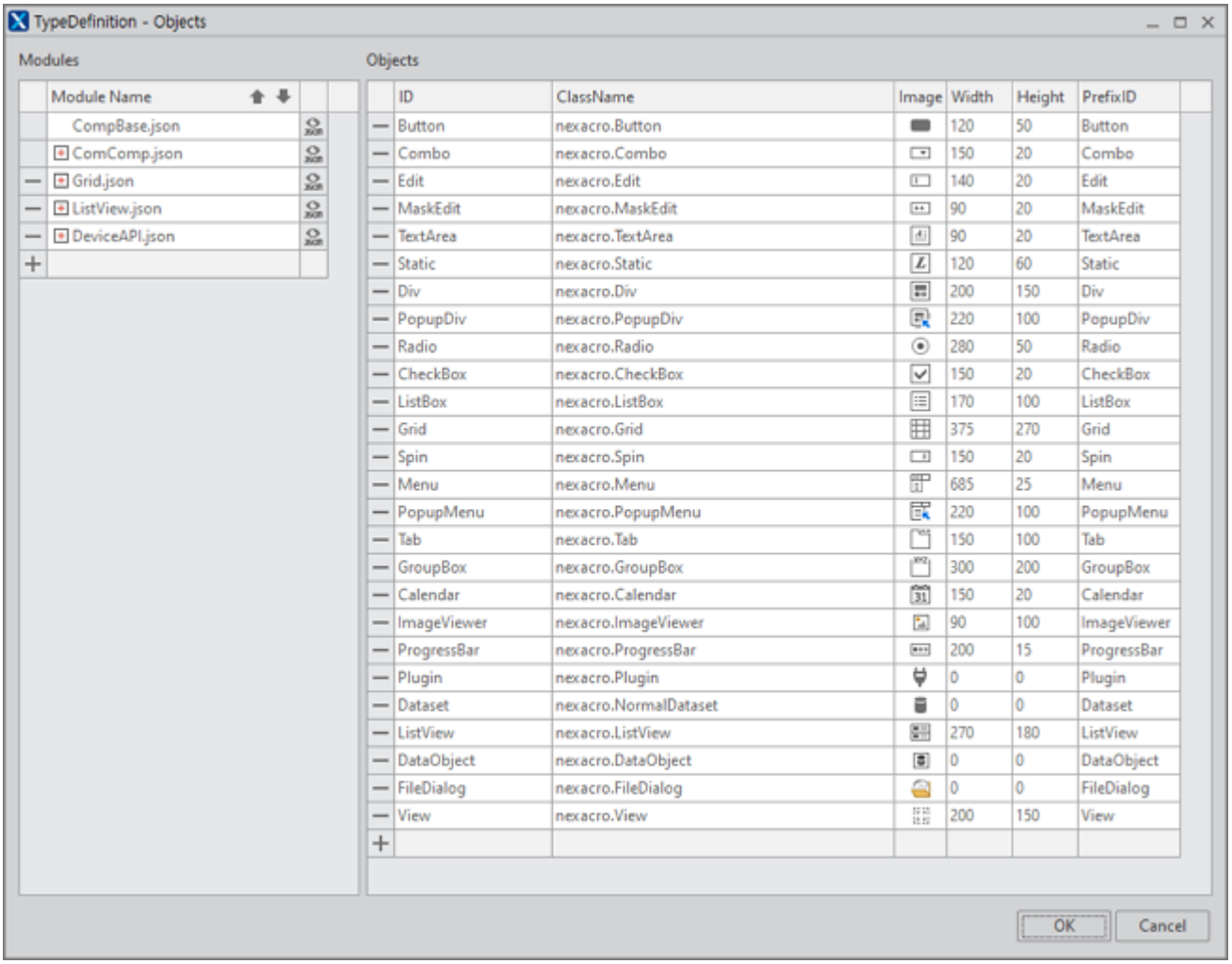
TypeDefinition contains the prefixed information sets required to run Nexacro and Nexacro Studio. The TypeDefinition file is an actual object that is edited and utilized to run a project. You can add, delete and modify items through the TypeDefinition editors, which take the form of dialogue boxes. The TypeDefinition file is formatted in XML.

To open the pop-up TypeDefinition editors, locate the submenu of TypeDefinition in the Project Explorer first. Then, double-click each item. Otherwise, open a context menu by right-clicking each item and select [Edit].



7.1 Objects

The Objects editor manages the libraries used to run Nexacro as well as objects used in a Form.

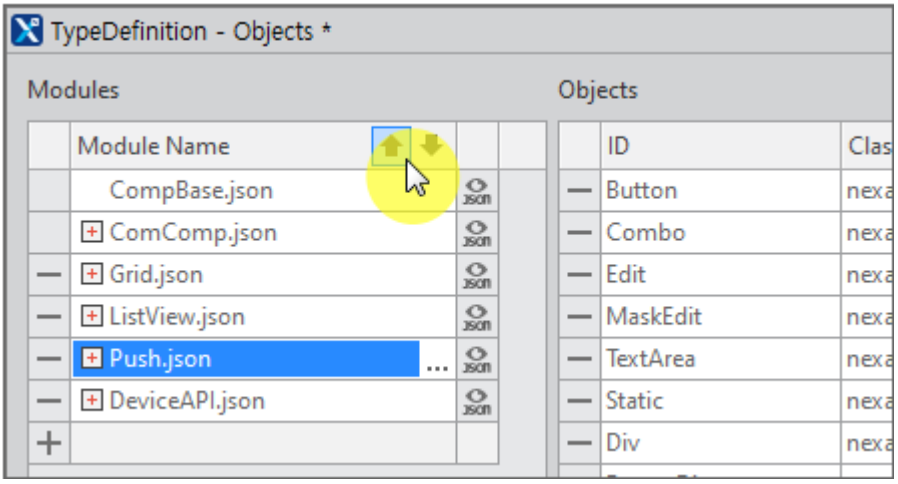


7.1.1 Edit modules

You must enter a module's name, and the name should be unique. Registered module files are managed with relative paths that are determined according to the location specified at "**Base Lib Path**" in [**Tools** > **Options**]. If a specified module file does not exist, the editing process will be aborted with an alert message popping up.

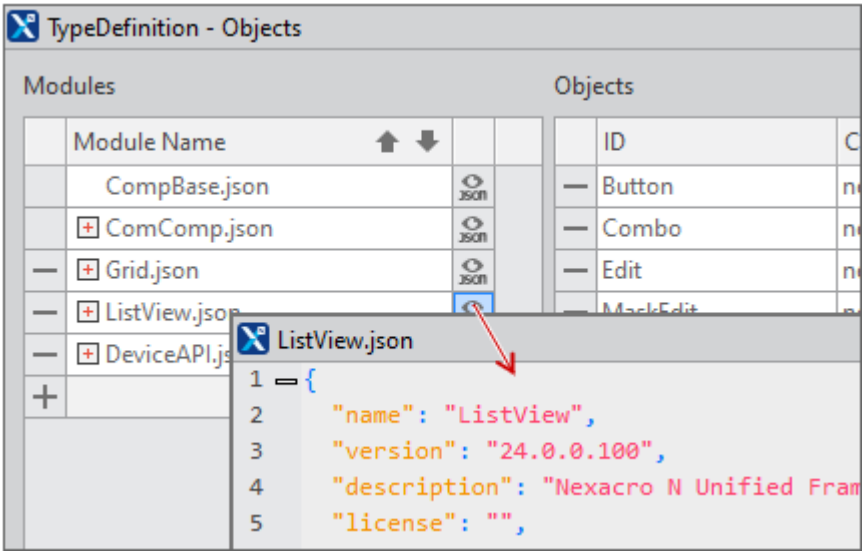
7.1.2 Change order

Order matters when registering modules, which are loaded before an application. As modules are interdependent, loading a project developed by Nexacro Studio may fail if modules are not registered in the correct order. Accordingly, click the Up/Down button if you want to change the order.

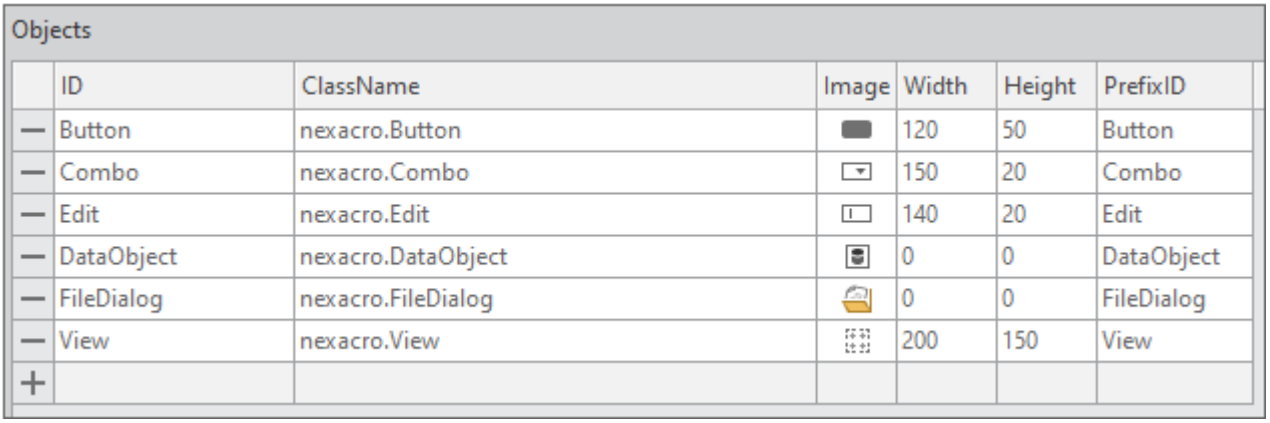


7.1.3 View file source

Selecting a module and clicking the "**Show Json**" button will display the relevant file code by opening a separate window. The presented code is read-only.



7.1.4 List of objects



Objects

	ID	ClassName	Image	Width	Height	PrefixID
—	Button	nexacro.Button		120	50	Button
—	Combo	nexacro.Combo		150	20	Combo
—	Edit	nexacro.Edit		140	20	Edit
—	DataObject	nexacro.DataObject		0	0	DataObject
—	FileDialog	nexacro.FileDialog		0	0	FileDialog
—	View	nexacro.View		200	150	View
+						

	Description
ID	This value is essential and unique.
ClassName	This value is essential and unique.
Image	The image is used as the object's icon displayed on the toolbar.
Width Height	These are the width and height values that will apply when you do not specify any values. That is, these values will apply when you create a component in the Form by mouse clicking rather than dragging.
PrefixID	This value is used to name the ID of a created object. Object IDs consist of PrefixIDs and following double digit numbers in ascending order. (eg. [PrefixID]+"00", [PrefixID]+"01"...)

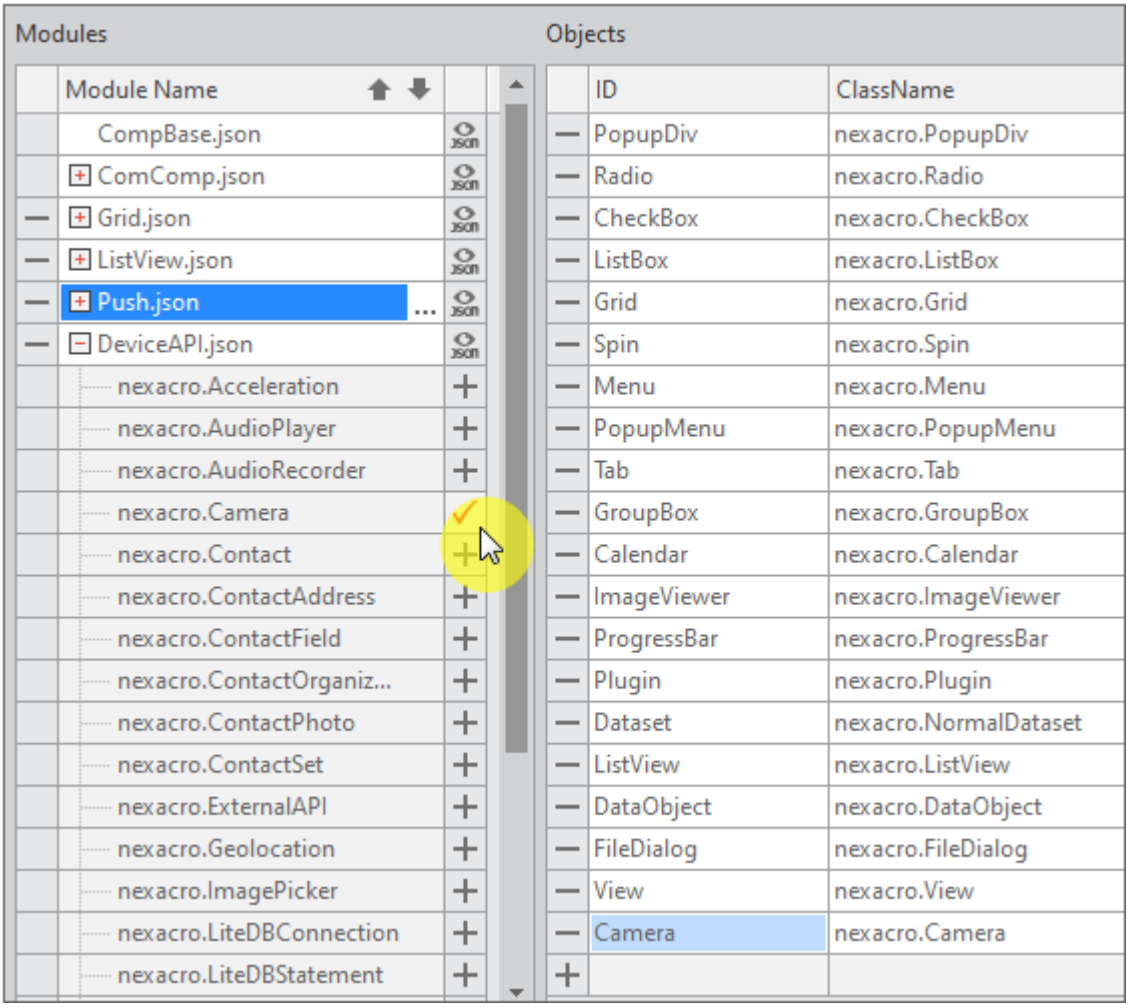


The DefaultWidth and DefaultHeight values are meaningless to invisible objects such as a Dataset.



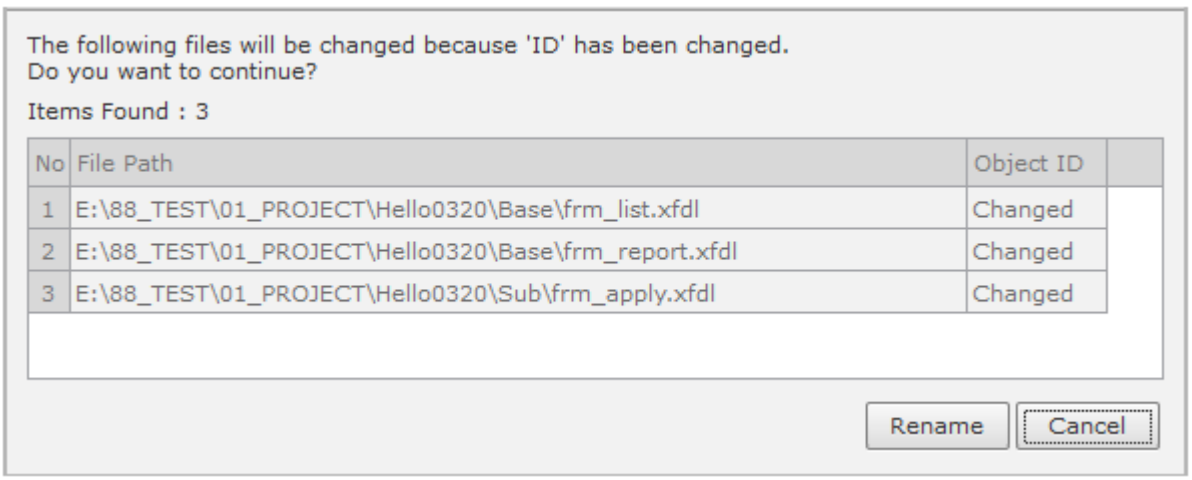
If you enter an invalid value for the Width, Height item, it is treated as 0.
For components that can be placed on the design screen, even if you set the value of the component's Width, Height item to 0 in the TypeDefinition, when you place the component, the value of the width, height property is treated as 100 for screen design convenience.

The default list only includes the frequently-used objects. If you want to add another object, expand the object list of the ComComp, Grid or DeviceAPI module, select the object you want to add and then click the plus (+) button at the end.

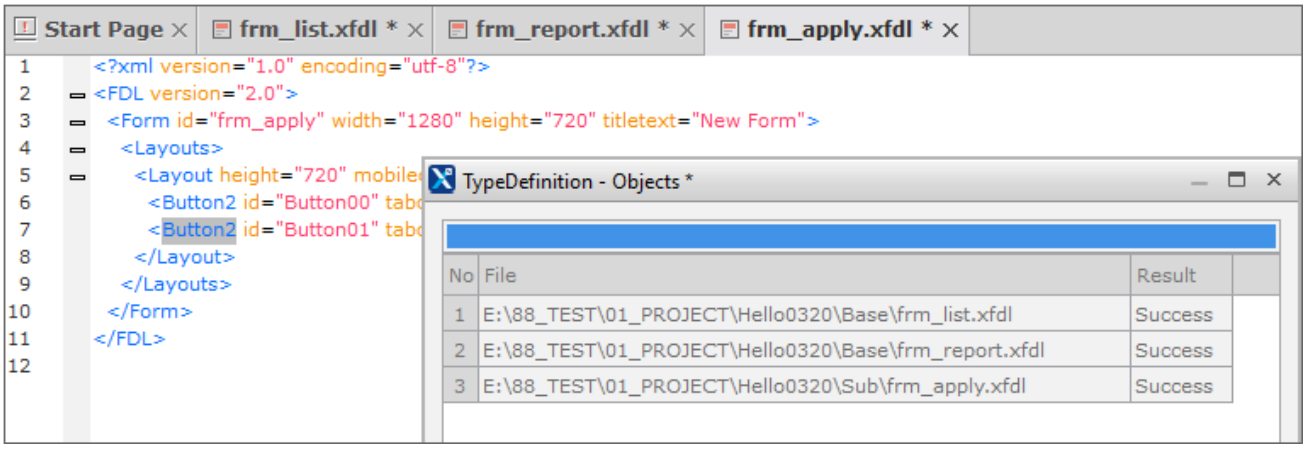


7.1.5 Object refactoring

Nexacro Studio will automatically perform code refactoring when object IDs in TypeDefinition are changed. The tool will search the current file and automatically make proper changes in relevant object IDs.

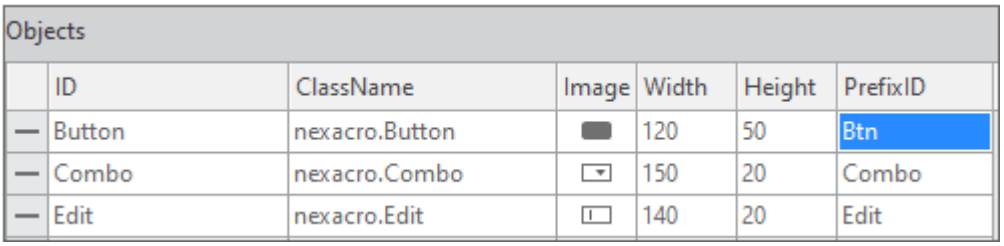


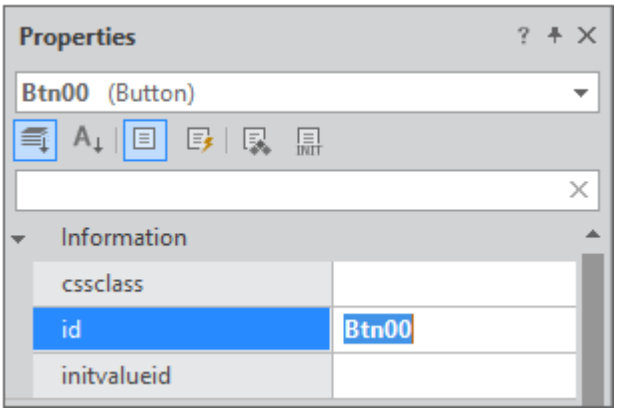
If you change the ID of a button object from *Button* to *Button2*, Nexacro Studio will search the entire project, find all the Forms that use *Button* and list them. If you click the [**Rename**] button thereafter, the Studio will rename all the found IDs and then open the all the relevant Forms to let you determine whether to save the changes.



7.1.6 PrefixID

You can designate a prefix for naming an ID of each component or other object. For instance, if you designate "Btn" as the PrefixID for Button components, the ID of a Button component will begin with "Btn" (eg. "Btn00") when it is created.

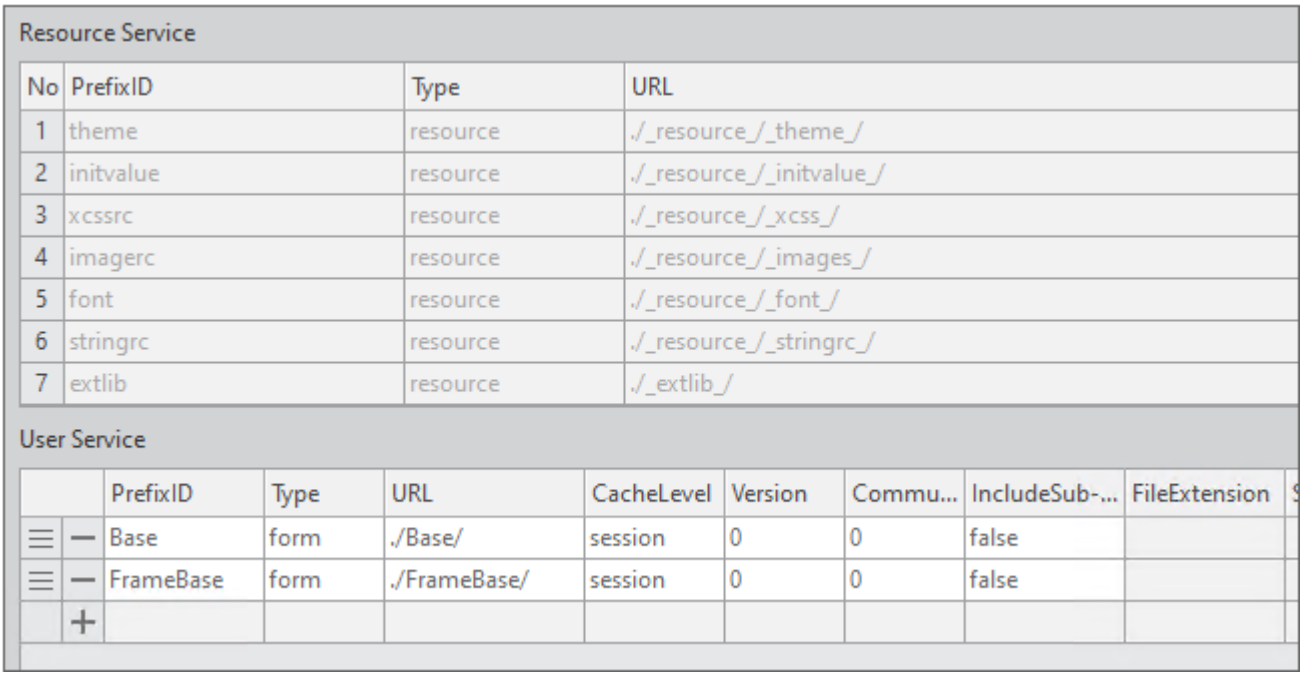




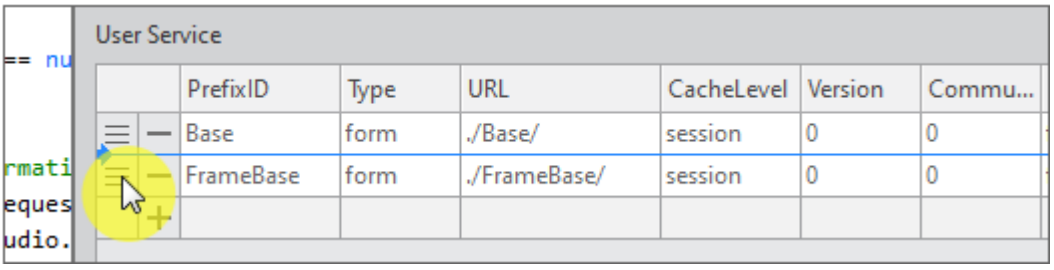
7.2 Services

Access the Services tab for managing service definitions, and the folder path or Internet URL.

Use the Add and Delete button to add new services or delete selected services.



The order of items registered in the User Service can be changed by dragging and dropping the icon displayed in front of the item.



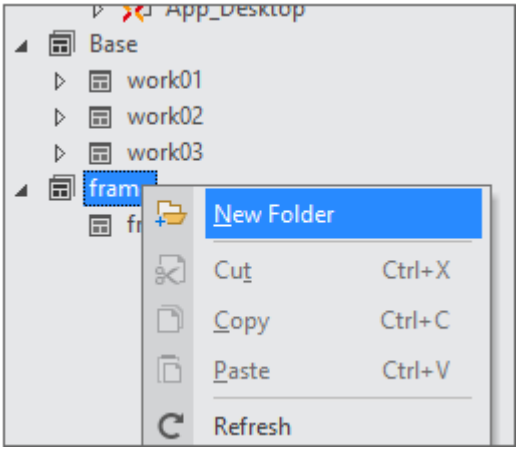
7.2.1 Edit services

You must enter the values of PrefixID, Type, and URL. A Prefix ID value must be unique.

Column		Description
Default	PrefixID	PrefixID of Service

Column		Description
	Type	Service Type
	URL	Url of the Service
	CacheLevel	Whether and what type of caching to use
	Version	Service version information
	Communication version	Communication version information
Extension	Include Sub-directory	Whether to display the subdirectory of the URL if the type is "form", "file" or "js". For theme and imagerc and extlib in Resource Service, this option defaults to "true", which is unchangeable.
	FileExtension	List of the extensions of the files that can be managed in the service (This function is applicable when the service type is file. example: js;txt;xfdl,xml)
	ServiceList	Used when Type is bs, metadata
	DataSchema	Enabled if Type is bs, metadata. model
	Search Column	Enabled if Type is metadata You can designate a column that will be searched when you search for a certain date piece through the metadata dialog. If unspecified, all columns will be searched. Commas (,) or semicolons (;) can be used as dividers.

If you set the Include Sub-directory option to "true" for a certain service, [New Folder] will be activated in the context menu opened from the relevant service in the Project Explorer.



The below table presents the file extensions supported for each type.

Type	File extension
none	nothing
form	*.xfdl, *.xjs, *.js
js	*.js, *.xjs
file	*.*
JSP	nothing

Type	File extension
ASP	nothing
SAP	nothing
bs	XML
metadata	XML
model	*.xmodel



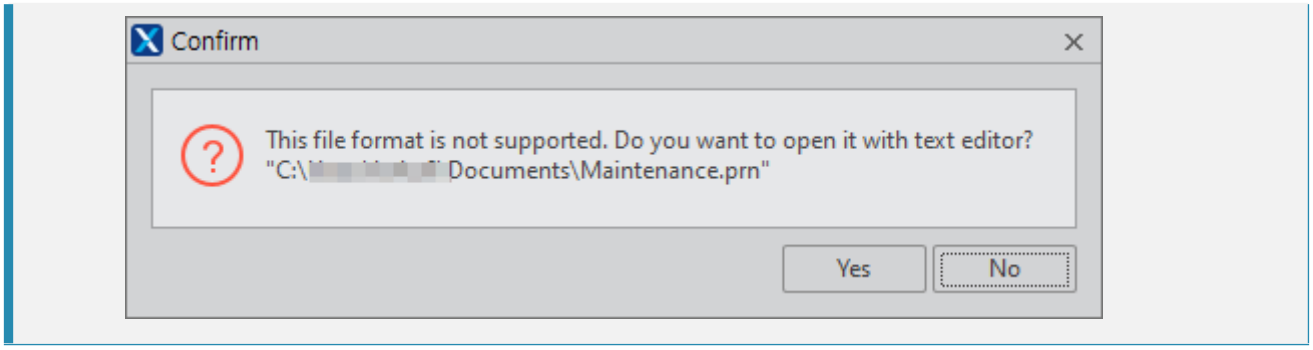
- bs (Business Service)
  This is a service that processes screen data.
  It implements business logic and processes XML-format data used by the screen through the data server.
- metadata
  This is a service that processes list information selectable by values such as component or object id or text properties.
  For example, it can be used by linking a glossary-type service.
  For details, refer to the metadata section.
- model
  This is a service in the QuickCode core that defines data information handled on the screen.
  For details, refer to the Develop Model, View Template, and Action sections.

The below table shows which editor will be launched when you double-click a certain file in the Project Explorer.

Editor	File extension
ADL Editor	*xadl
FDL Editor	*.xfdl
Theme Editor	When a theme is double-clicked
Script Editor	*.js, *.xjs
XML Editor	*.xml
XCSS Editor	*.xcss
Image Viewer	*.bmp, *.jpg, *.jpeg, *.gif, *.png, *.jfif, *.tif, *.tiff, *.svg
Text Editor	*.txt, *.html, *.jsp, *.asp, *.json, *.css

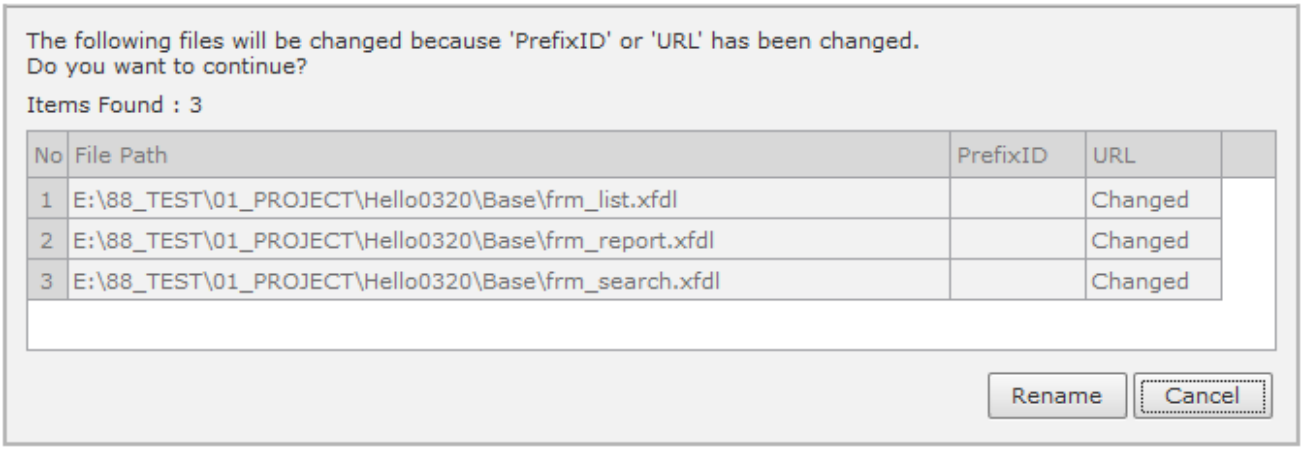


If there is no matching editor for a selected file, an alert will pop up, asking you whether you want to open the file with the Text Editor.



7.2.2 Service refactoring

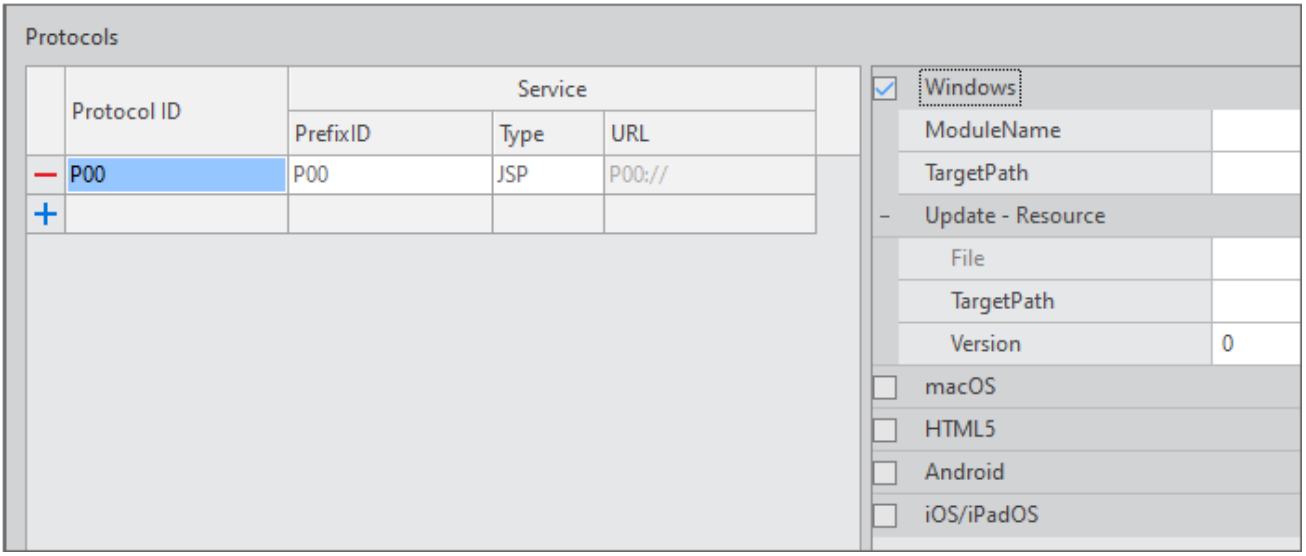
Nexacro Studio will automatically perform code refactoring when you change the PrefixID or URL of a service registered in the TypeDefinition. Code refactoring refers to the process of applying changes automatically.



7.3 ProtocolAdaptors

The Protocols editor is designed to register protocol adapters. Nexacro does not convert data for processing. It uses the standard Hypertext Transfer Protocol (HTTP) to deal with requests and responses. Depending on user environments,. however, you may need to convert or encrypt data for transaction.

A protocol adapter is a tool that connects different data structures. You can develop desired features easily through simple settings.



7.3.1 Editing protocols

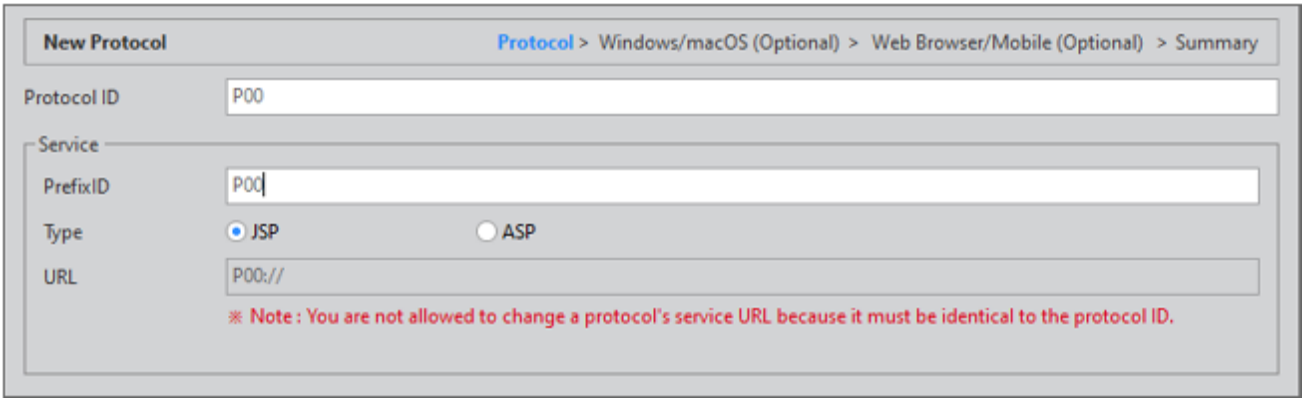
You can add protocols and apply them differently according to execution environments. You can determine the type of protocol to apply by entering information on modules or classes.

7.3.2 New Protocol Wizard

You can add a new protocol in TypeDefinition through New Protocol Wizard. There are two ways to start the wizard.

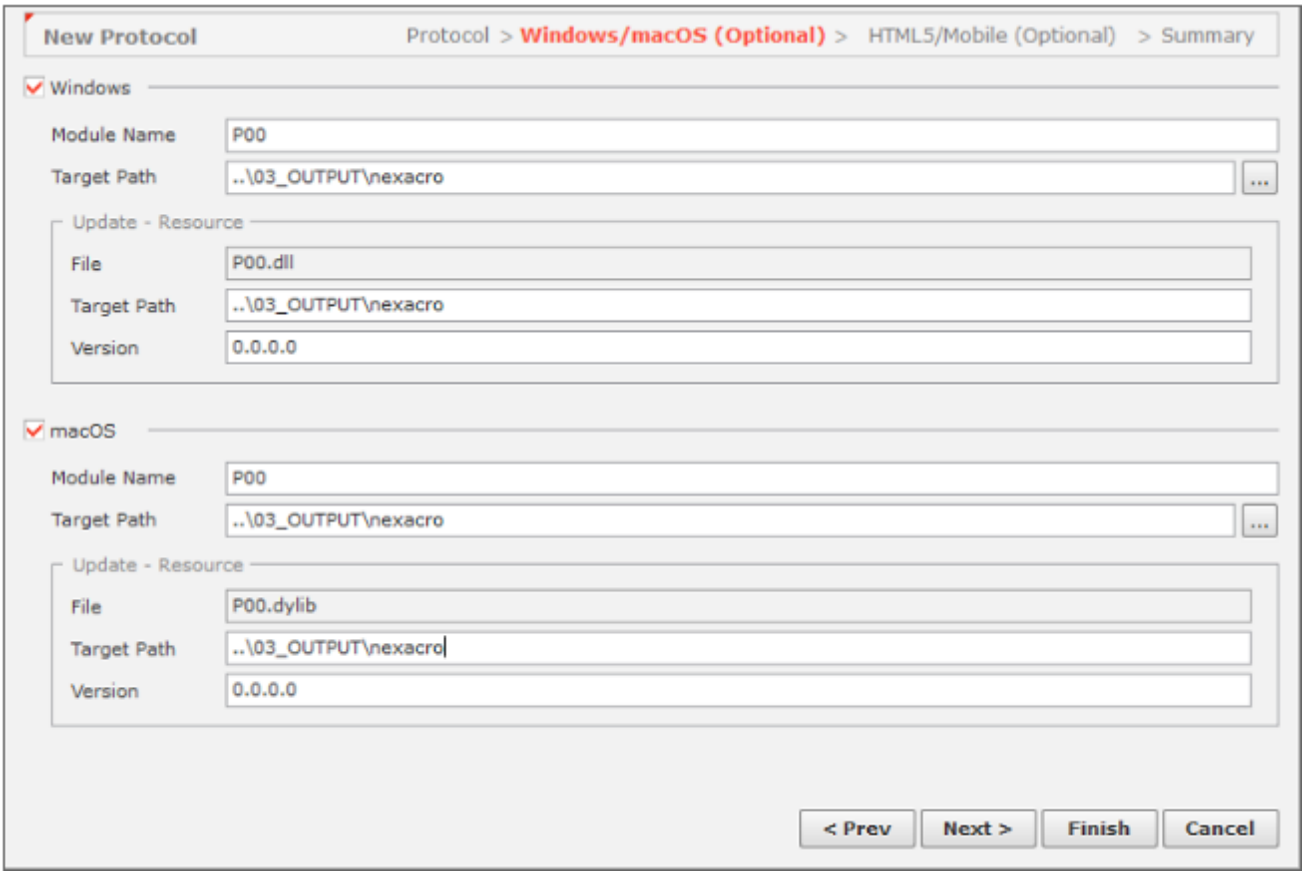
- In the Project Explorer, right-click **TypeDefinition** and access [**Add** > **Protocol**]
- In the Project Explorer, right-click **Protocols** under **TypeDefinition** and select Add Protocol.

Take either way you like in order to add a protocol.



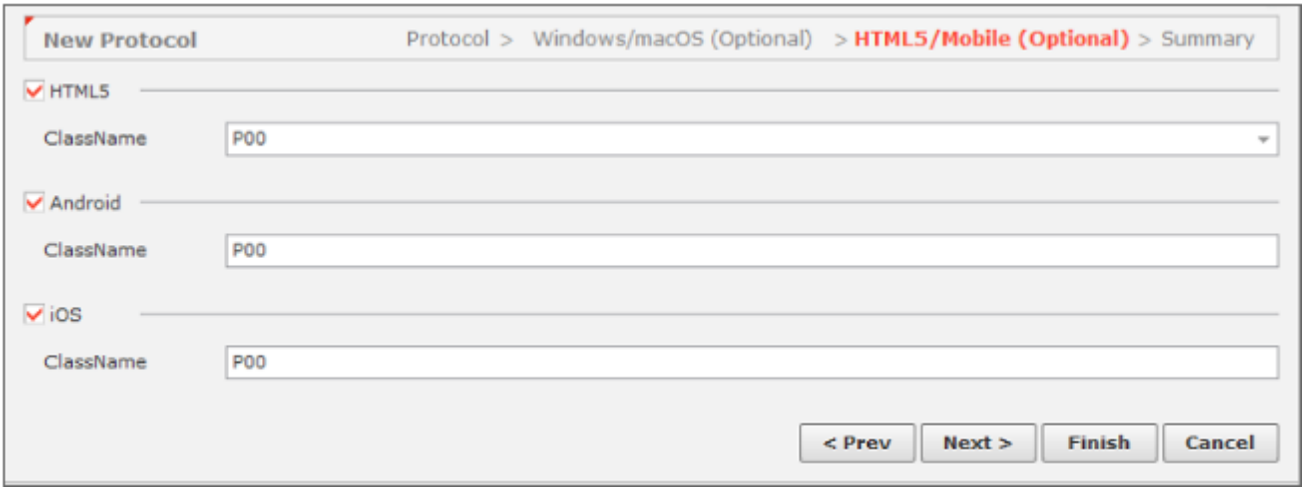
Item	Description
Protocol ID	Protocol ID, which is also used as a service URL ([Protocol ID]://)
PrefixID	Service prefix ID
Type	This indicates the type of a service. You can select either JSP or ASP.

You can select the type of systems to which the protocol will apply. Click the checkbox of a system you want select, and then enter necessary information in relation to the system.



Item		Description
Windows	ModuleName	This is the name of a binary module. Nexacro registers a DLL file with a module name (*module name.dll*) after finding it in the target path. A module name will be added to *start.json*.
	TargetPath	This is the path from which the binary module is obtained. A target path will be added to *start.json*.
Windows > Update	File	Name of a module file (same with the information registered in [TypeDefinition > Updates])
	TargetPath	Path where the module file is updated in the device
	Version	Version of the module file
macOS	ModuleName	Name of a module file for macOS
	TargetPath	This is the location from which Nexacro finds the module before registering it.
macOS	File	Name of a module file (same with the information registered in [TypeDefinition

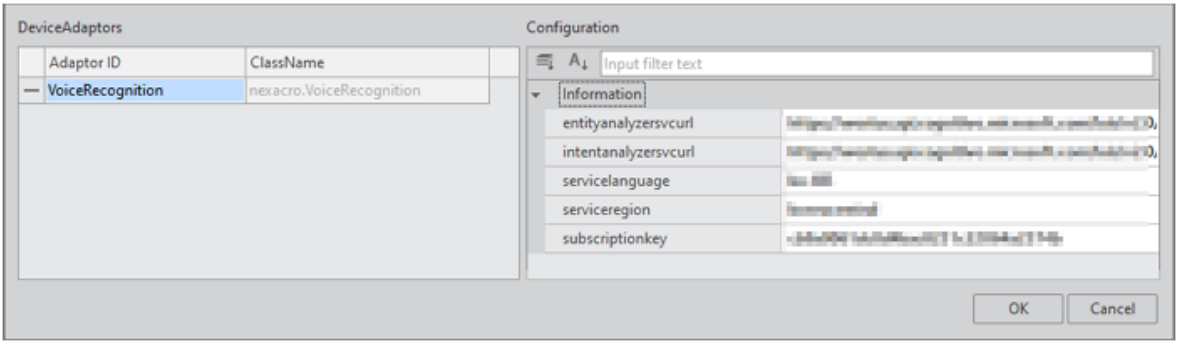
Item		Description
> Update		> Updates])
	TargetPath	Path where the module file is updated in the device
	Version	Version of the module file



Item		Description
HTML5	Classname	This is the class name of a protocol used in the HTML5 environment. The class name will be registered to the *registerclass* array of *environment.xml.js* and the protocol will be loaded with other components.
Android	Classname	This is the class name of a protocol used in the Android environment. The protocol ID and class name will be registered to the *start_android.json* file.
iOS	Classname	This is the class name of a protocol used in the iOS environment. The protocol ID and class name will be registered to the *start_ios.json* file.

7.4 DeviceAdaptors

You can configure a DeviceAdaptor module once you install it.



	Description
Adaptor ID	This is the ID of the module. You can change it when you install the module.
ClassName	This is the class name of the module. You cannot change the value.
Configuration	This is a list of the properties of the module. You can set the values of the properties.

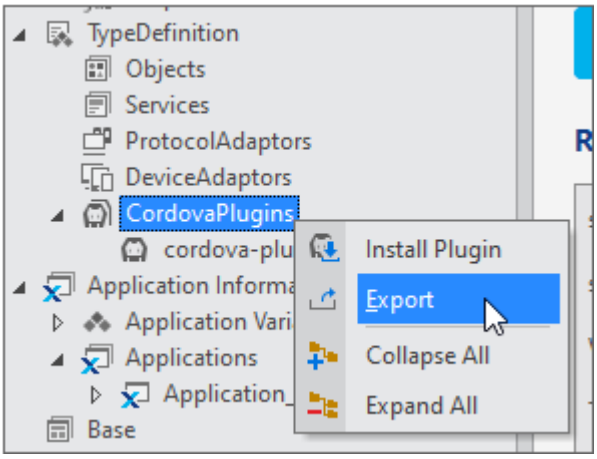
You can obtain device information by using an adaptor ID in a script.

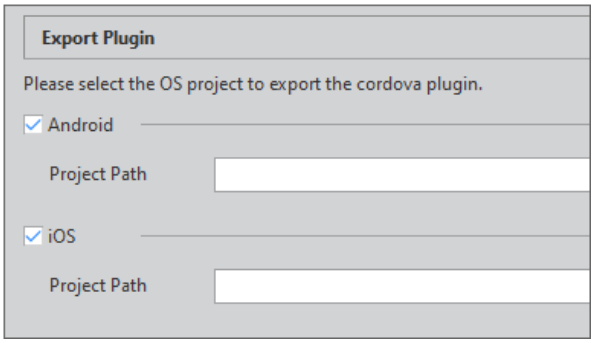
```
var voicerecognition =
nexacro.getEnvironment().getDeviceAdaptor("VoiceRecognition");
```

7.5 CordovaPlugins

A list of Cordova plugins installed through the Install Module Wizard will be displayed.

You can select CordovaPlugins and select [Export] from the Context menu to download the files needed to use the Cordova plugins when building the app.





If you have an app project on your PC, set the root path of that app project as the project path.

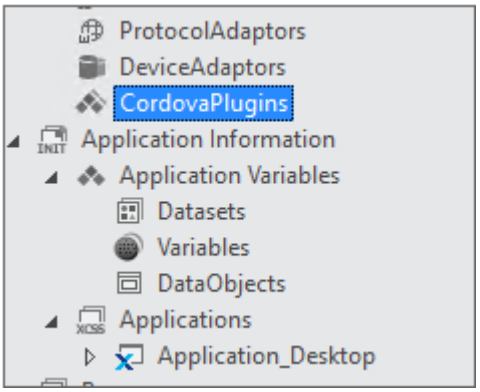
If you are building the app project on another PC, move the file to the exported folder and proceed with the build as shown below.



8.

Project Editing | Application Infomation

As for Application Information, you can set the information managed by the application. Under Application Variables, set the values of parameters that can be used commonly in the application. Under the Applications item, configure the settings.

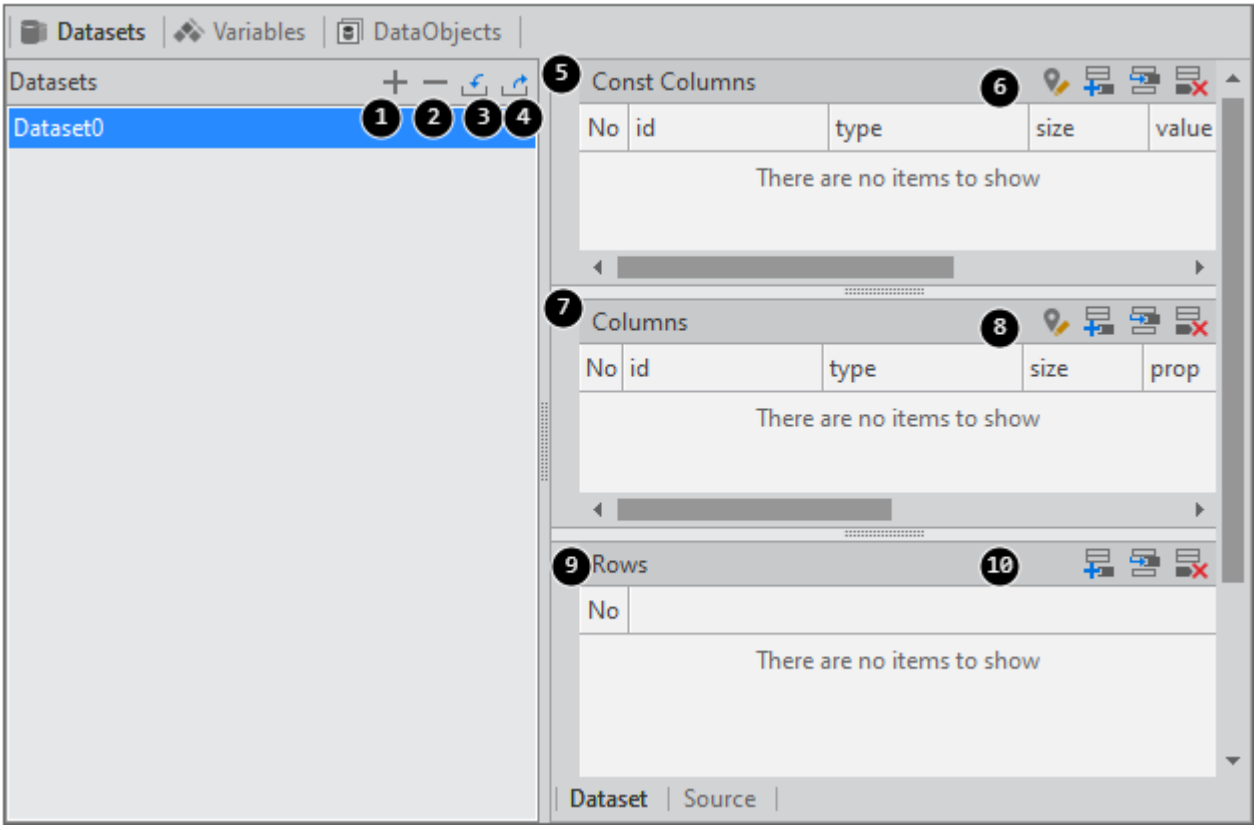


8.1 Application Variables

8.1.1 Datasets

A Dataset is a memory location in a table form used to process data. When a Dataset is bound with components for presentation, automatic synchronization to relevant the component function is provided also.

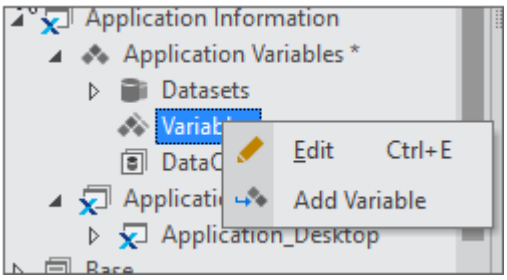
In the Datasets tab, you can add or delete Datasets that are used globally within a project. Moreover, the Import/Export features are provided to allow you to reuse a great deal of data.



	Properties	Description
❶	Add a new Dataset	Adds a Dataset
❷	Delete selected Dataset	Deletes the selected Dataset
❸	Import Dataset	Loads a Dataset from a saved file
❹	Export Dataset	Saves the current Dataset as a file
❺	Const Columns	Sets up columns of constant data
❻	Edit data path Add const column to the Dataset Insert const column to current position Delete selected const column	Run Data path Editor Adds a new constant column Inserts a new constant column above the cursor location Deletes the selected Constant column
❼	Columns	Defines Dataset columns
❽	Edit data path Add column to the Dataset Insert column to current position Delete selected column	Run Data path Editor Adds a new Dataset Column Inserts a new Dataset column above the cursor location Deletes the selected Dataset Column
❾	Rows	Defines Dataset rows
❿	Add row to the Dataset Insert row to current position Delete selected row	Adds a Dataset row Inserts a new Dataset row above the cursor location Delete the selected Dataset rows

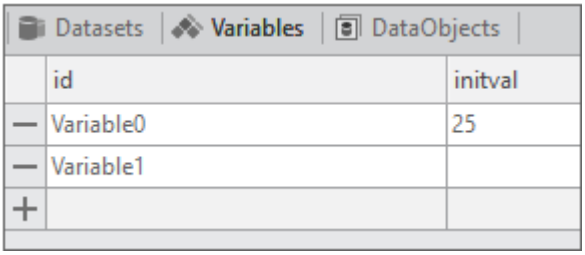
8.1.2 Variables

Variables are used widely in a project. Manage the variables from the Variables context menu.



Menu Function	Description
Edit	Opens the Variable Editor
Add Variable	Adds a new variable and opens the Variable Editor

You can add, delete, or change the construction of the variable items in the Variable Edit Window.

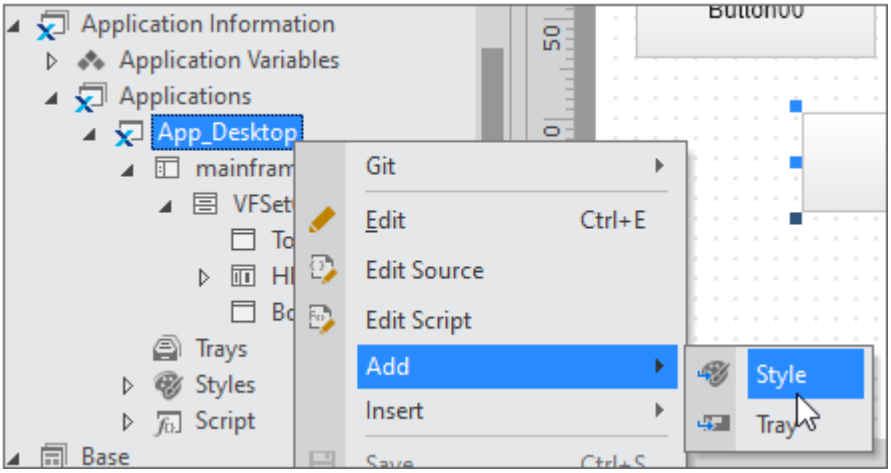


8.1.3 DataObjects

In the DataObjects tab, you can add or delete DataObjects that are used globally within the project.

8.2 Application

In this field, you can manage styles, main frames and trays that are applied to an Application.

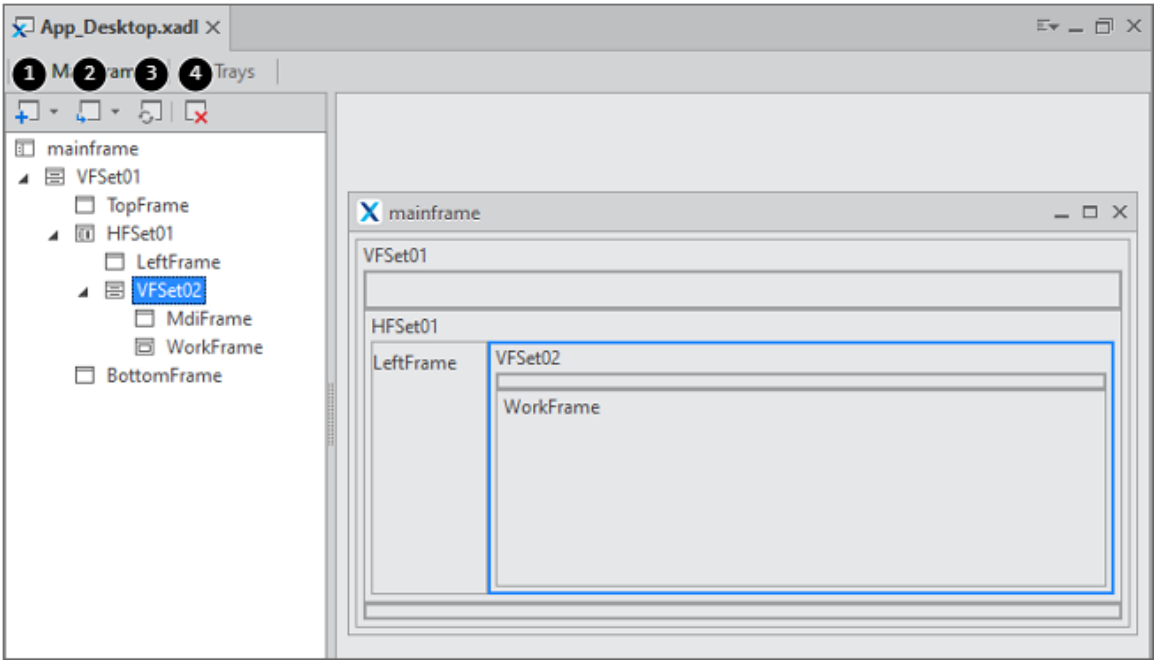


Menu	Function
Edit	Edits an Application through the ADL editor.
Edit Source	Opens the Source Editing window to allow editing the XML of the selected ADL
Edit Script	Opens the Script Editing window to allow editing of the script of the selected ADL
Add	Creates a new style or tray
Insert	Imports an existing style file

8.2.1 MainFrame

MainFrame is an element composing Nexacro app's basic screen. It has ChildFrame as a subordinate element and is created by default when a project is created.

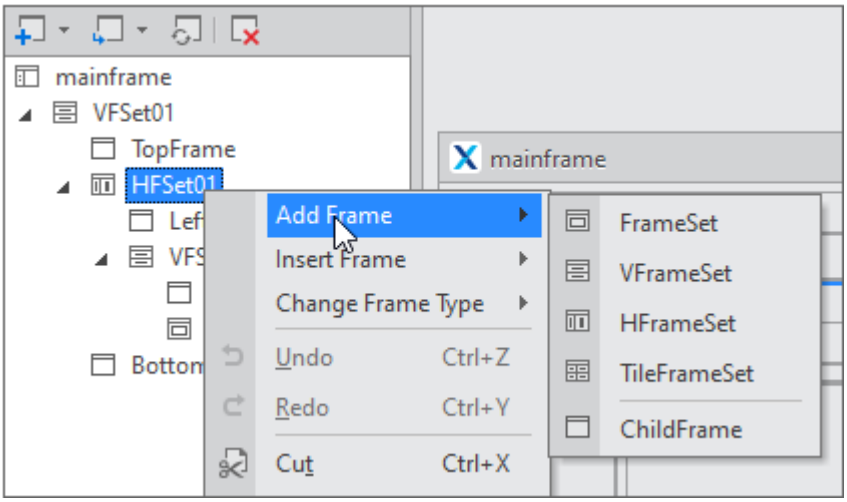
The ADL editor displays a frame structure in a tree view and provides a preview pane.



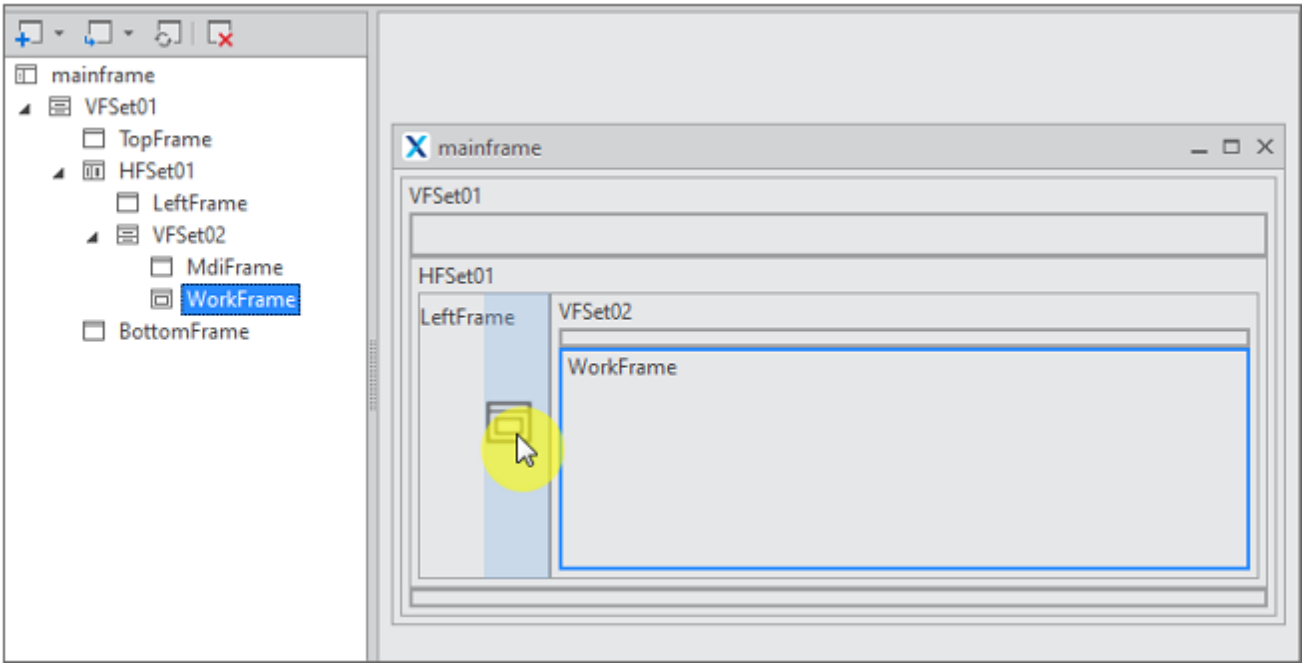
The buttons on the toolbar above the tree are used to change a frame structure.

	Name	Description
1	Add Child Frame	Adds a frame under the selected frame
2	Insert Frame	Adds a frame at the same level as the selected frame
3	Change Frame Type	Changes the type of the selected frame
4	Delete Frame	Removes the selected frame

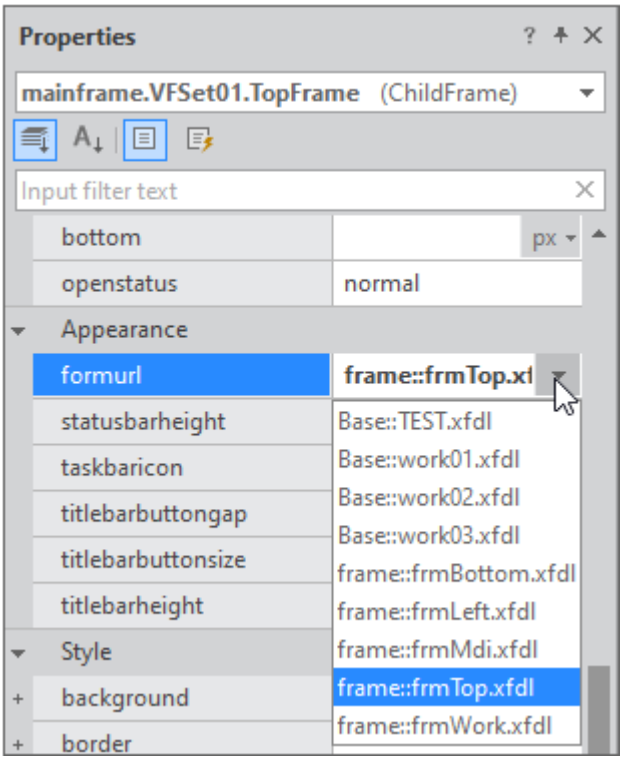
You can also conduct editing through the context menus opened from the tree or preview pane.



In the preview pane, you can move any types of frame other than a main frame by the drag and drop. As you drag a certain frame to a position you intend, the preview pane will show you the validity of the move by coloring the target area.



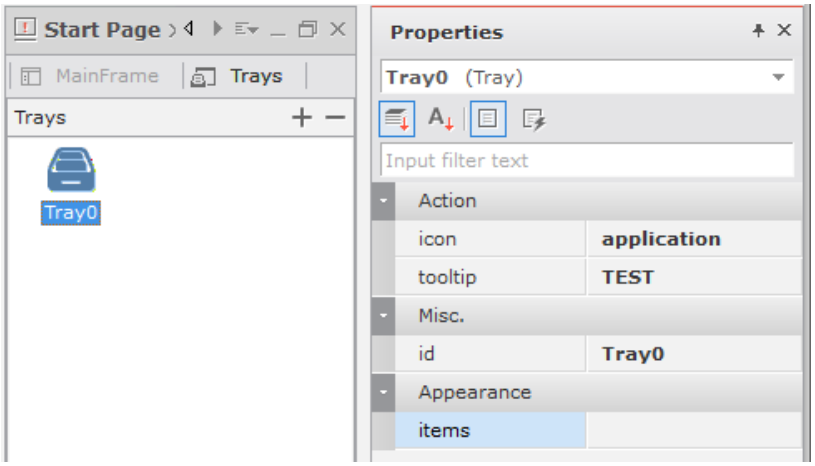
To display Forms when executing Nexacro, you need to link the Forms to the "formurl" property of a child frame.



ChildFrame, a frame that can come at the lower level of MainFrame, has form information in the shape of URL and shows the Form on the screen. It implies the basic unit screen for loading a Form.

8.2.2 Trays

Trays is an element defining the icons and tooltips displayed in the tray area.



8.2.3 Style

You can register XCSS files in an app by opening the context menu from the Application in the Project Explorer. The registered XCSS files are globally applicable.

9.

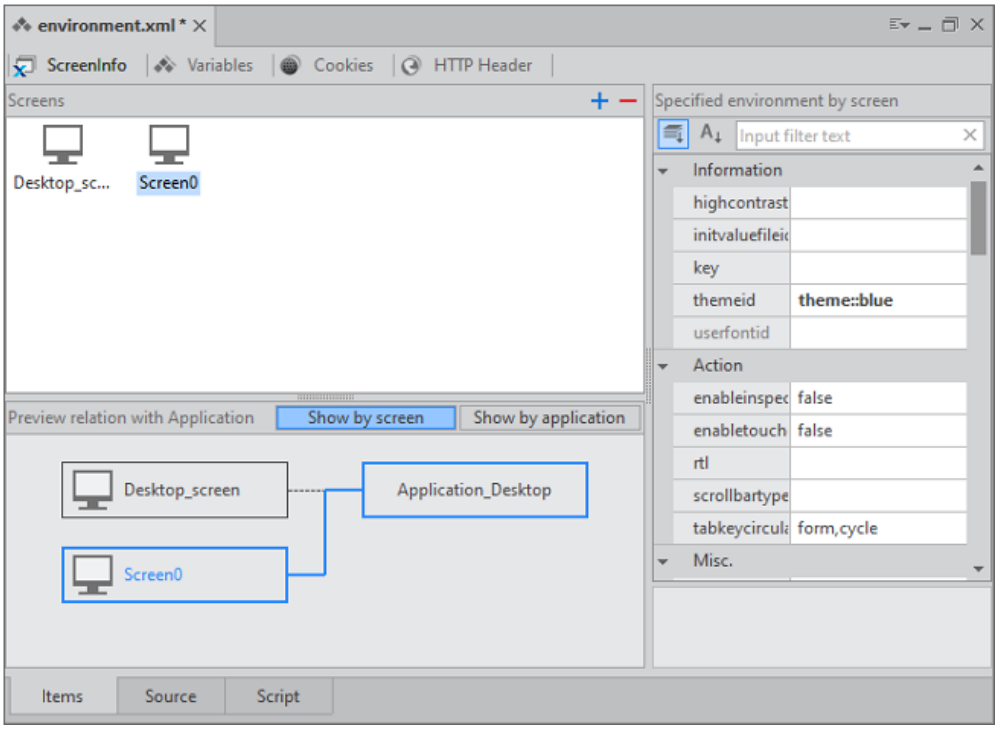
Project Editing | Environment, FDL, InitValueDefiniton

9.1 Environment

Environment is designed to define the settings for individual screens, existing separately from Application Information. You can specify screen definitions, variables and cookies while you can write the script of an Environment file.

9.1.1 ScreenDefinition

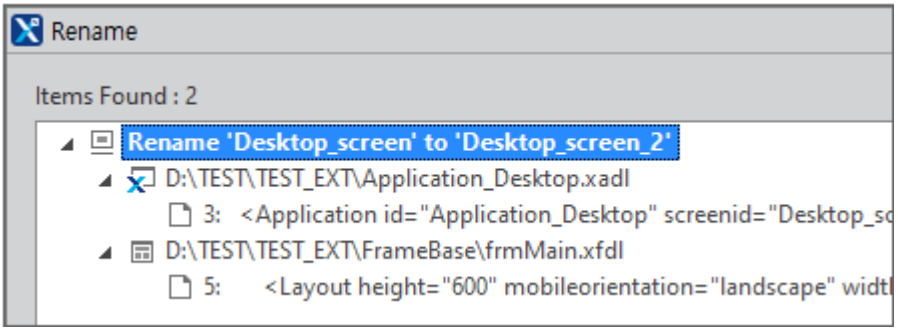
With a ScreenDefinition file, you can add new entries to a piece of screen information specified in the process of creating a project. You can also modify or delete existing entries. A ScreenDefinition file is saved in the form of an XML file. Preview relation with Application presents a intuitive diagram of the relations between screens and applications.



The environment-related properties set for individual screens can override the same properties specified in Environment as the former has priority.

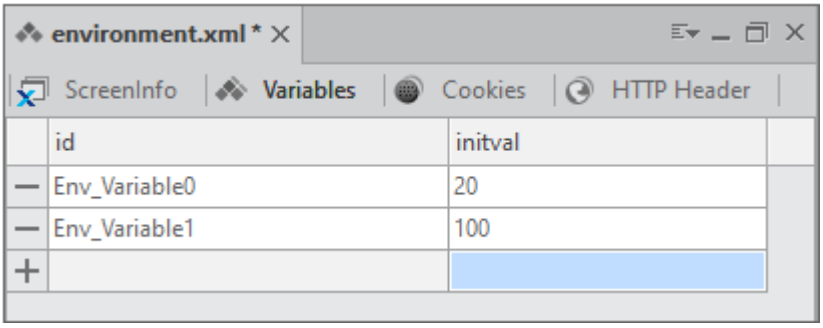
Screen Refactoring

Screen refactoring is conducted when you change the ID of an existing screen. Nexacro Studio will find the applications that adopt the changed screen and then automatically replace the old ID with the new one.



9.1.2 Variables

You can add, modify and delete variables defined in Environment. The variables here can be used globally, regardless of Application Information objects.

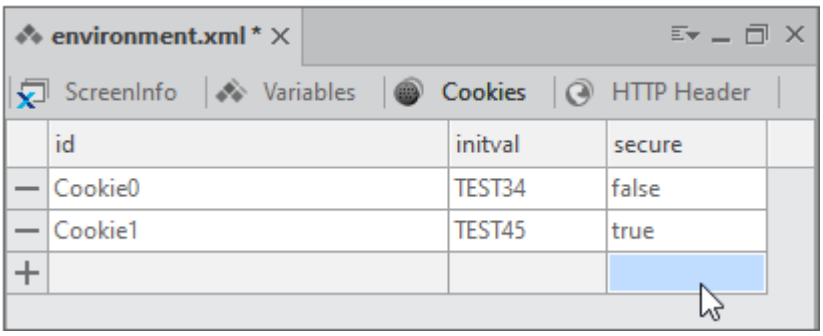


The below syntax set can be used to employ, modify and delete variables in a script.

```
nexacro.getEnvironmentVariable("Env_Variable0");
nexacro.setEnvironmentVariable("Env_Variable0", 30);
nexacro.removeEnvironmentVariable("Env_Variable0");
```

9.1.3 Cookies

You can add, modify and delete cookies defined in Environment.



The below syntax can be used to employ a cookie in a script.

```
var vCookie0 = nexacro.getCookieVariable("Cookie0");
var vCookie1 = nexacro.getCookieVariable("Cookie2", true);

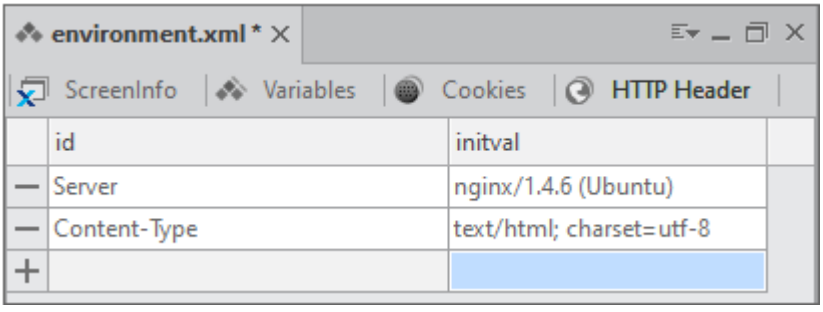
nexacro.setCookieVariable("Cookie0", "test");
nexacro.setCookieVariable("Cookie1", "test", true);
```



As for a cookie whose *secure* is set to "true", the cookie will be sent only in response to an encrypted request if the transaction is conducted under the HTTPS protocol.

9.1.4 HTTP Header

You can add, modify and delete an HTTP header defined in Environment. You can add variables to the header of an HTTP transaction.

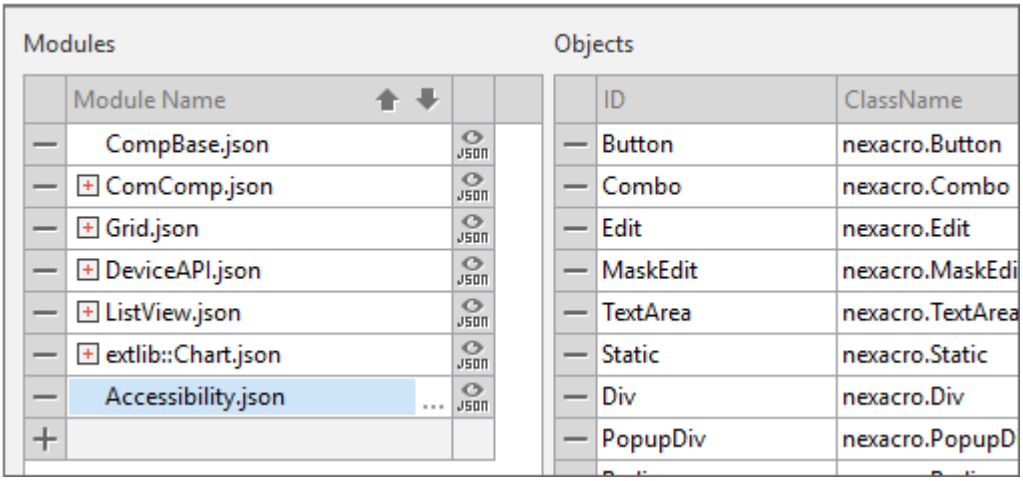


As described below, you can add, check or delete variables to a script.

```
nexacro.setHTTPHeaderVariable("Server", "nginx/1.4.6 (Ubuntu)");
nexacro.getHTTPHeaderVariable("Server");
nexacro.removeHTTPHeaderVariable( "Server" );
```

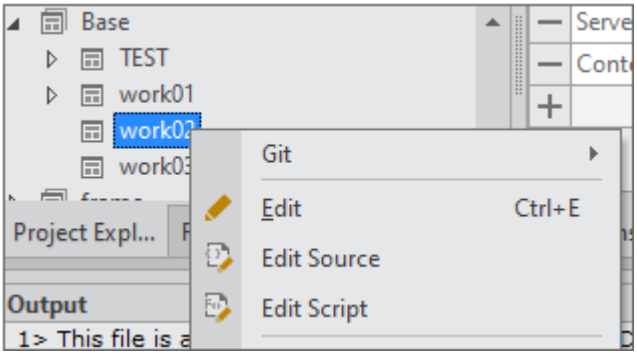
9.1.5 Accessibility

If you want to activate the accessibility features in an app, you need to set the *enableaccessibility* property of the Environment object to "true" and add the *Accessibility.json* file to Modules in TypeDefinition. If you only set *enableaccessibility* to "true" and launch the app, accessibility will not be activated with an alert popping up to inform you of the need to add *Accessibility.json*.



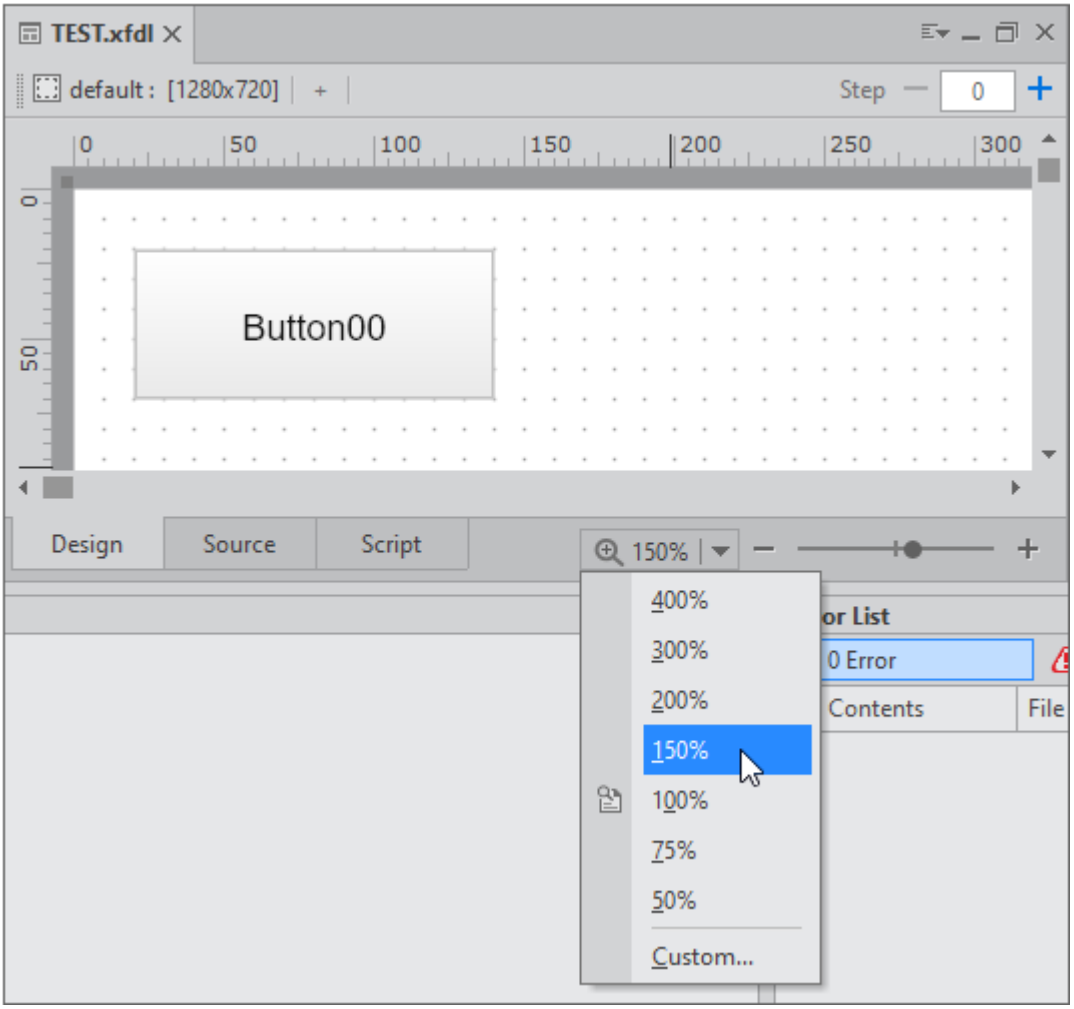
9.2 FDL (Form Definition Language)

You can launch form-related editors by accessing [**Edit**], [**Edit Source**] or [**Edit Script**] in the context menu opened from the Project Explorer.

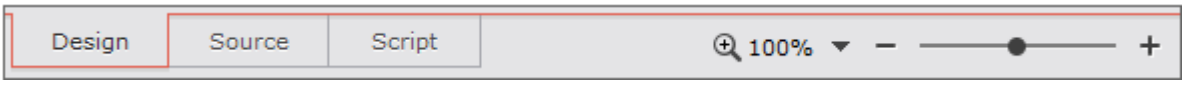


9.2.1 Design

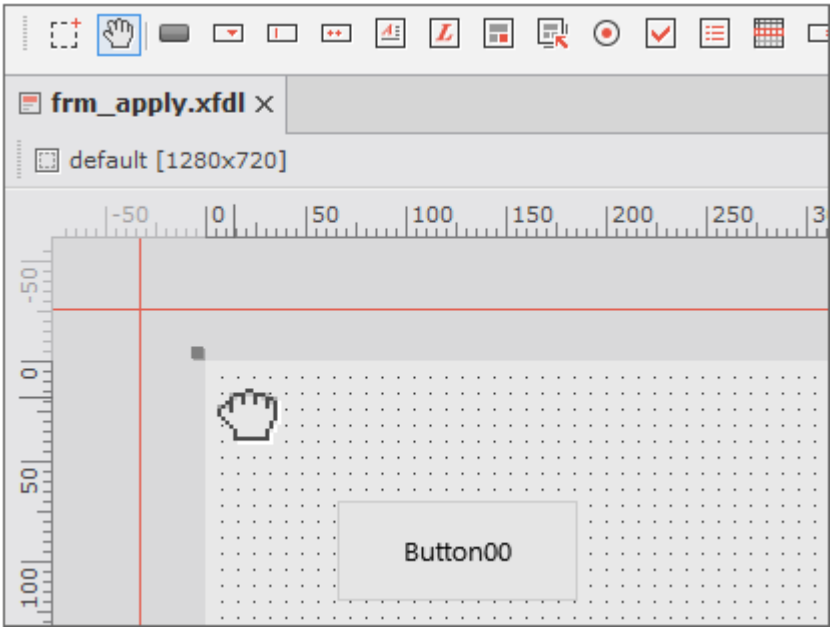
In this tab, you can design components, edit their properties and bind Datasets to them. You can zoom a current form by adjusting the zoom bar on the bottom-right side of the pane.



A zooming feature is supported in the Design tab.



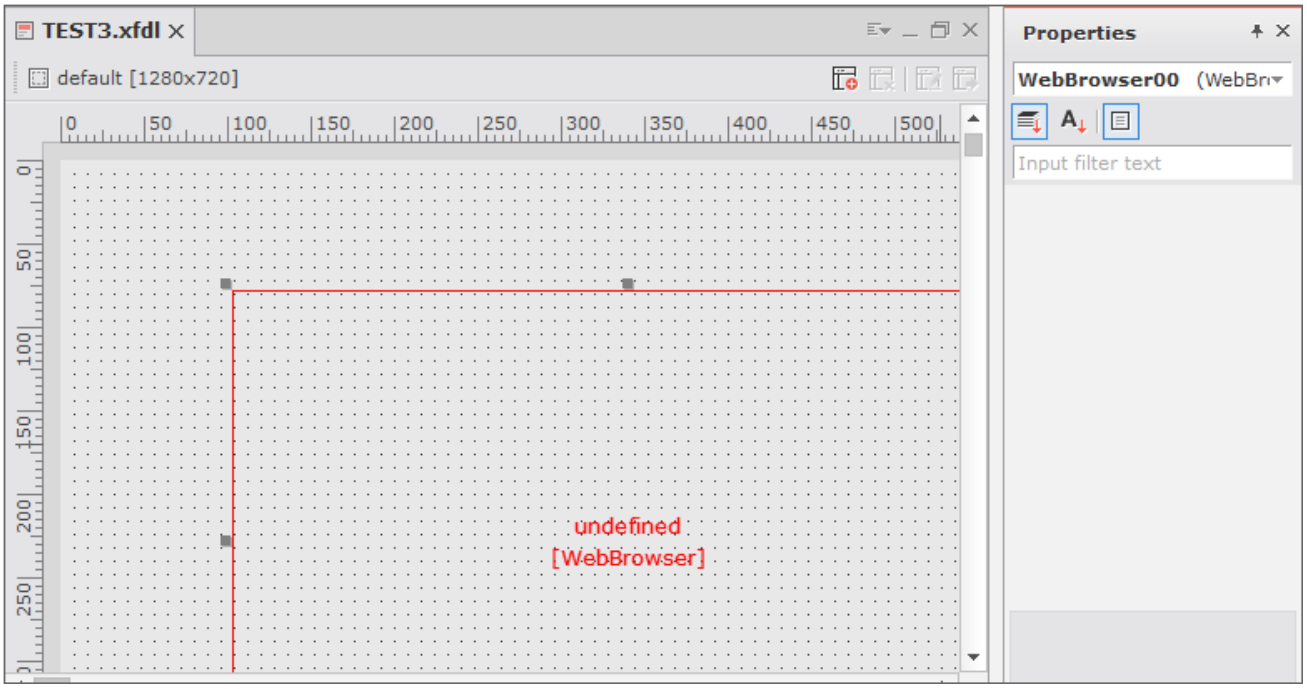
You can move the base coordinate of a Form by selecting the Move icon on the component toolbar. For example, if you want to draw a component outside a Form, you can do so after selecting the Move icon and then moving the base coordinate. As the result of moving a base coordinate, the rulers will display negative areas of their measuring units.



Register TypeDefinition

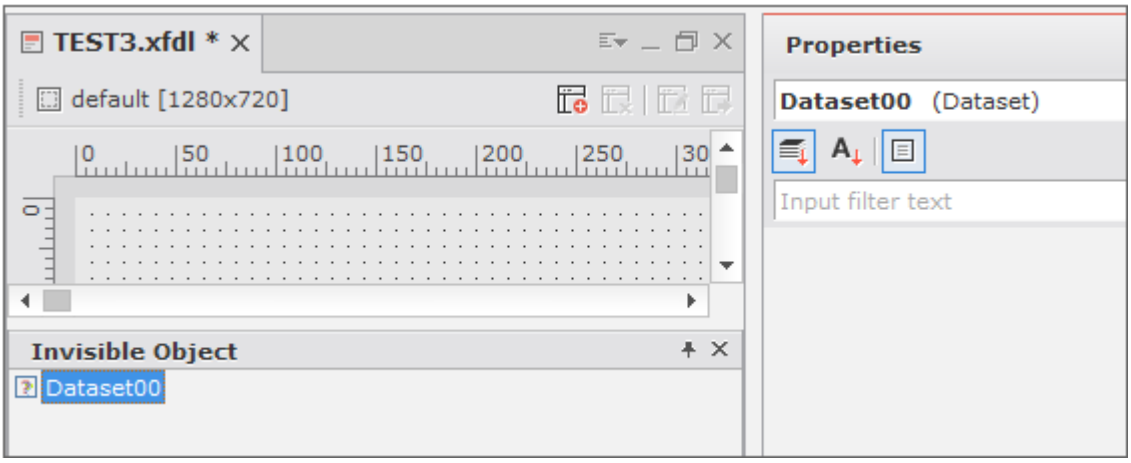
When you import a Form from a different project, some components may not be registered in the TypeDefinition of the current project. In that case, Nexacro Studio will indicate such an absence and help you to register relevant components immediately.

If such a component is visible, its name will be displayed with the word "undefined," all in red. Although you can select the component in this case, you cannot resize it while Studio does not show the properties of the component.

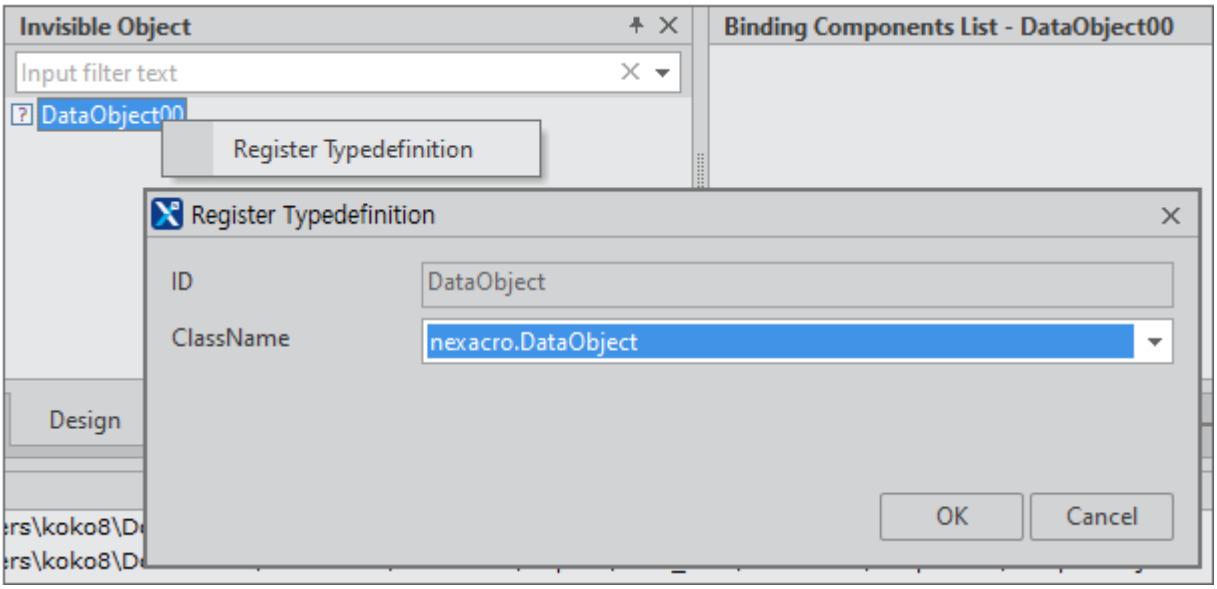


For those that appear in the Invisible Object pane, they will be marked with a question mark icon. Moreover,

no information will be displayed in the Properties pane when you select those objects.



If you right-click a component that is not registered in TypeDefinition, you will see the pop-up menu [**Register TypeDefinition**]. Click it and then select a class name from the combo list opened from the pop-up box in order to register the relevant component.



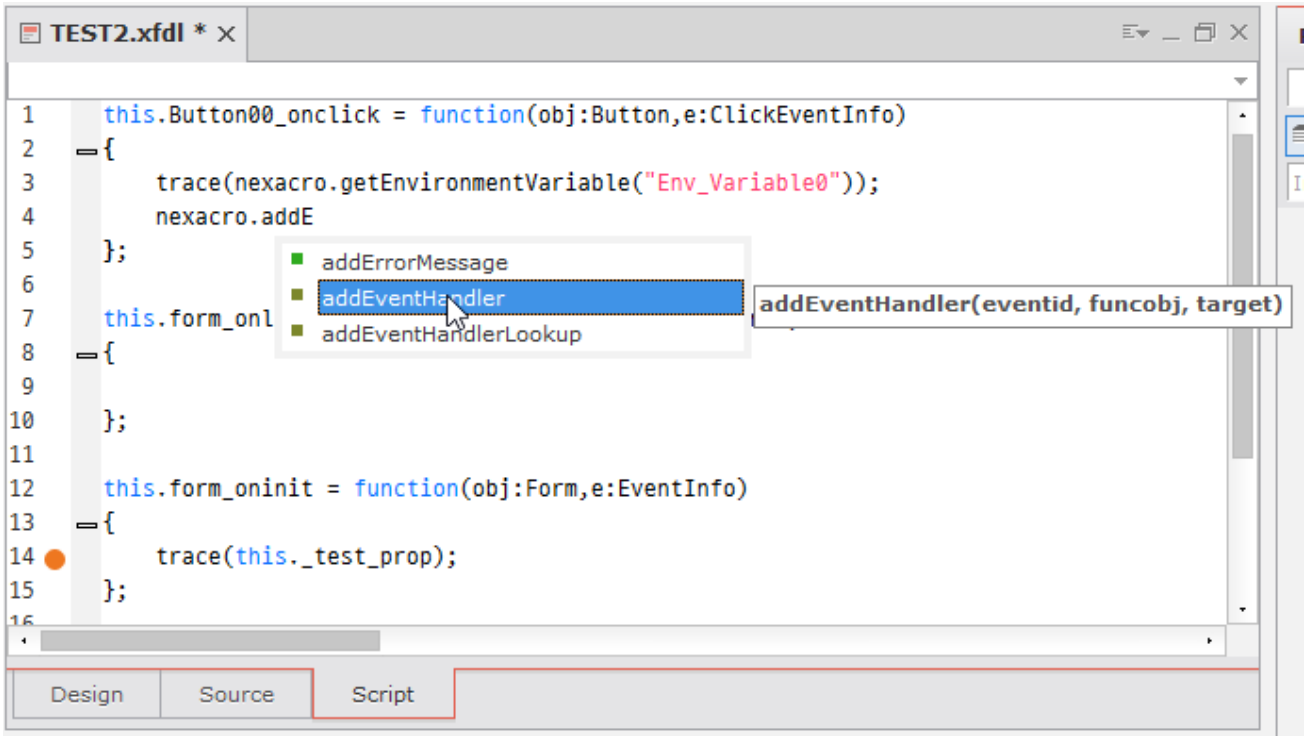
9.2.2 Source

You can see a Form's source code and edit it.



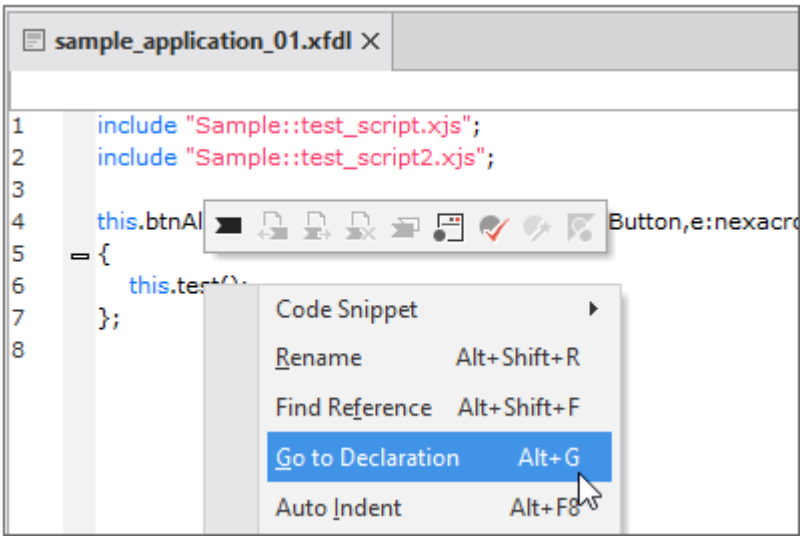
9.2.3 Script

You can see the script of a Form and edit it.

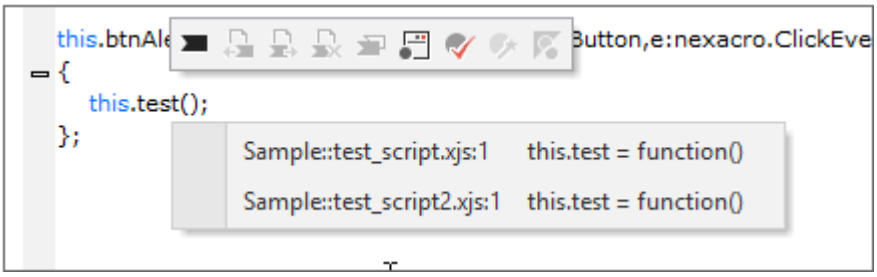


Go to Declaration

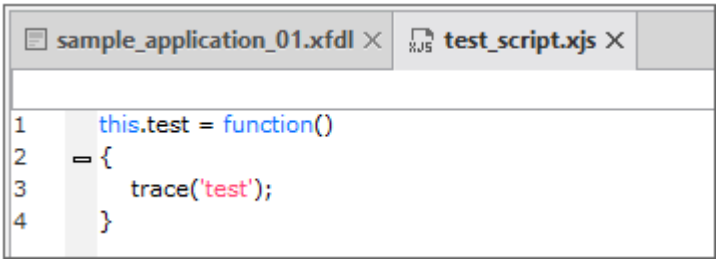
You can find the item [Go to Declaration] from the context menu that you can open by right-clicking a method of an object in the script editor.



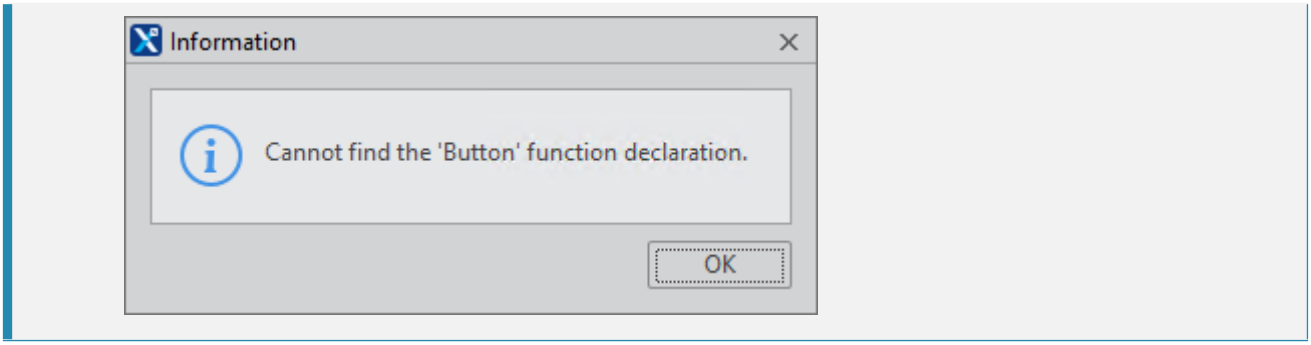
Upon clicking the menu, you will be led to the location where the relevant function is declared. If the function is located in an included script, Nexacro Studio will open the file that the script originates from and bring you to the location of the function. If more than one function are found to have the same name, the list of those functions will be displayed.



If you choose one, Nexacro Studio will show the relevant codes.

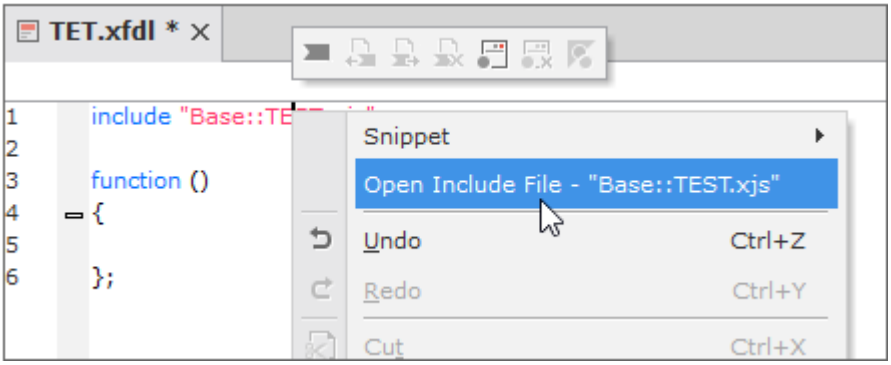


If you try This function over something that is not a method—for example, a property—the below warning message will pop up.



Open Include File

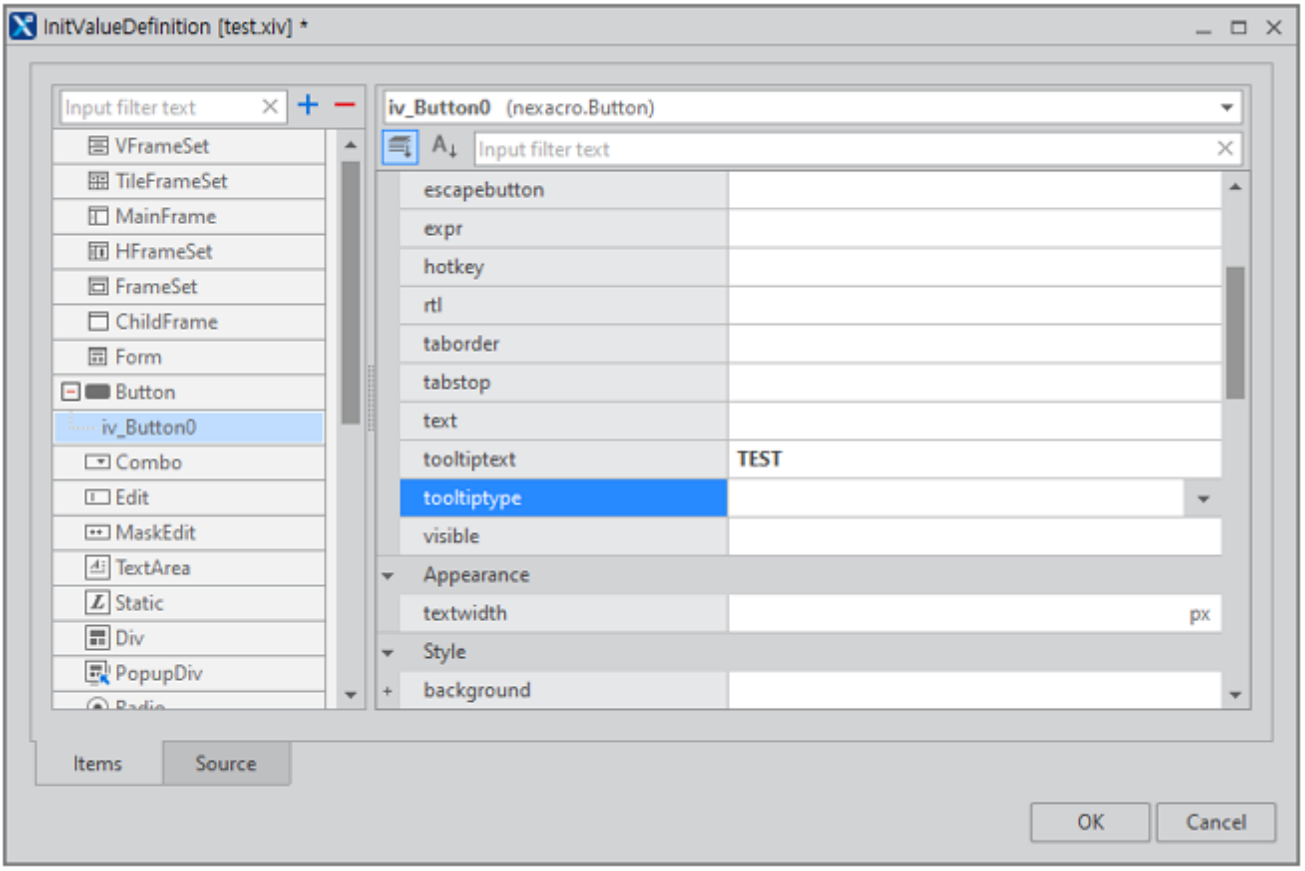
This function will open a script that is referred to in another script through an `include` statement. Right-click an `include` statement and select [Open Include File - XX.xjs] from the context menu.



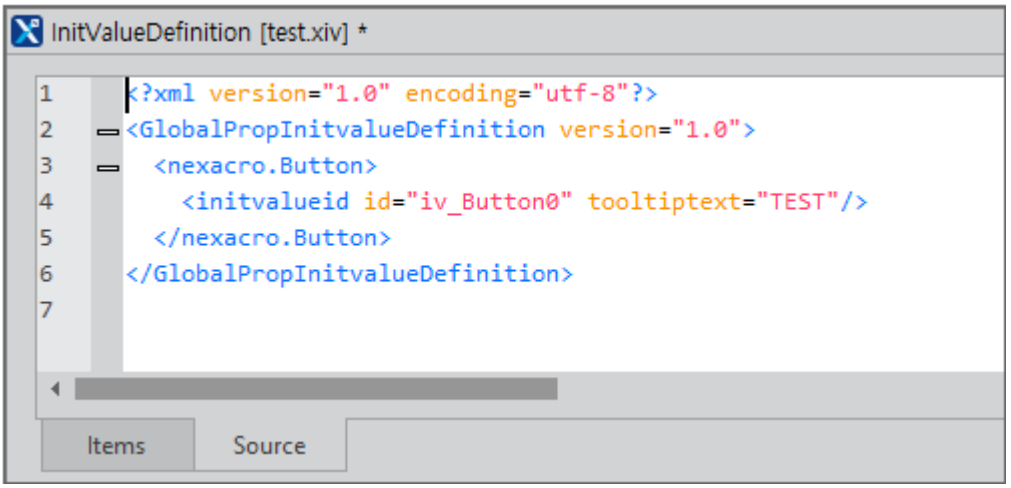
9.3 InitValueDefiniton

An InitValueDefiniton file, which takes an XML file format, contains the information on the initial values of a certain object in a certain app. You can open the editor for an InitValue file by choosing the file in the Resource Explorer and then selecting [**Edit**] from the context menu.

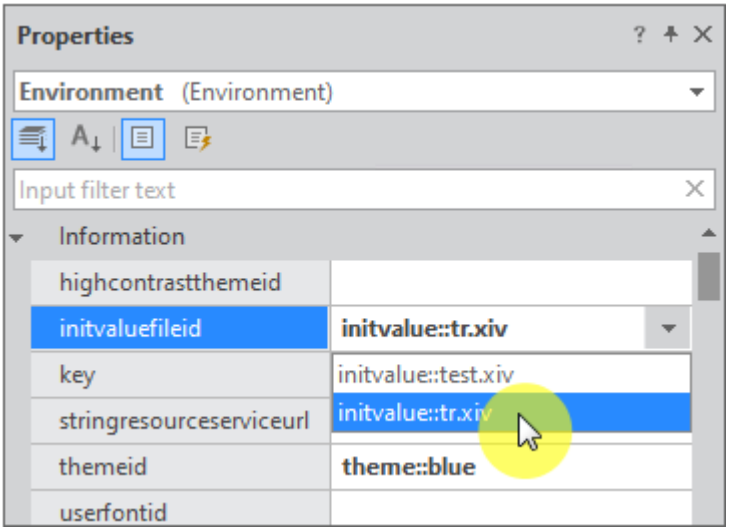
The left pane of the InitValue Editor shows a list of the objects registered in the TypeDefinition file. You can add an initial value by choosing an object and then selecting [**Add**] from the context menu. Afterwards, you can set initial values of the object through the right-handed pane.



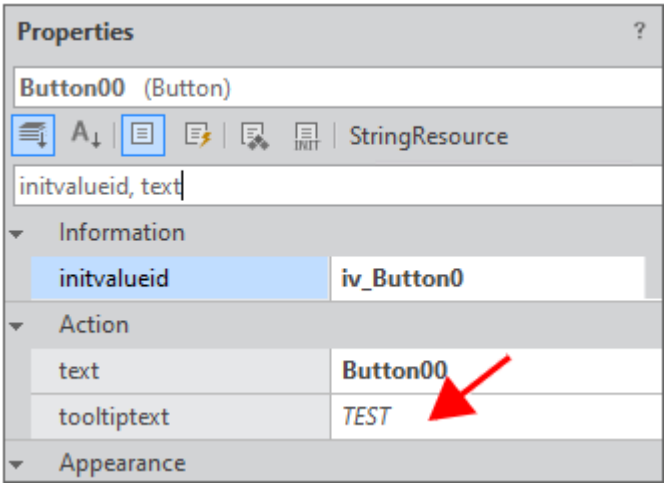
The added initial values will be formatted in XML and saved. You can check the source codes of the InitValue file by clicking the [Source] tab.



If you want to apply an InitValue file, you are required to specify the *initvaluefileid* property in Environment at first.



Then, select a component to apply the file to and specify the *initvalueid* property in the Properties window. After all, the property values saved in the InitValue file will apply to the component, with such values italicized.



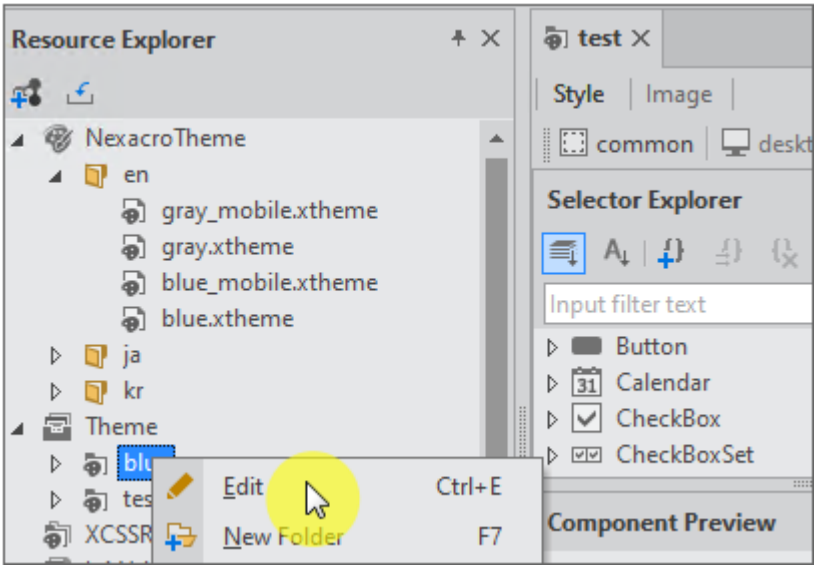
10.

Project Editing | Theme, XCSS

10.1 Theme

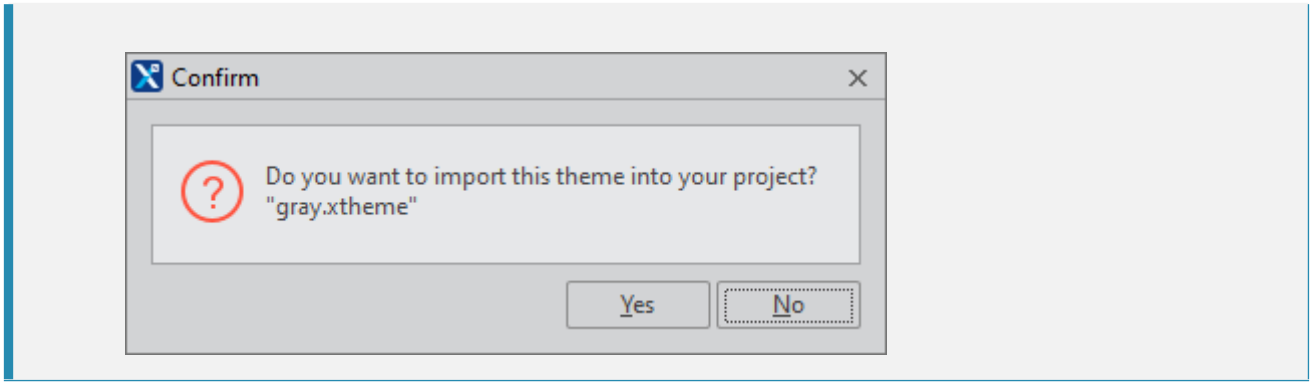
A theme file is composed of images plus XCSS files. While default themes are compressed when provided, you must decompress them if you want to apply them to a project.

To open the Theme Editor, you should double-click a theme, which is located under *Theme* in the Resource Explorer. Otherwise, open the context menu from the theme and select the menu [Edit].



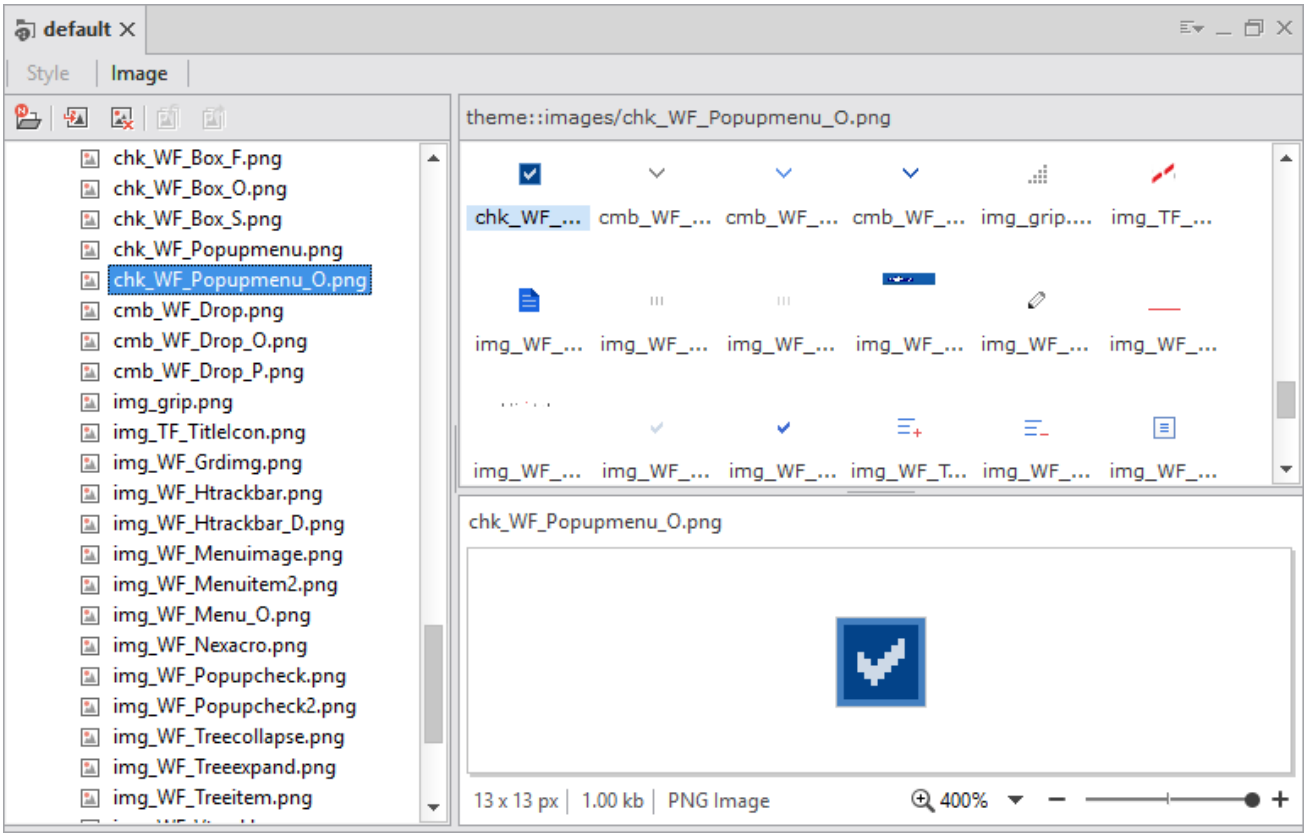
The themes presented under *NexacroTheme* in the Resource Explorer are default themes provided upon the installation of Nexacro Studio. Those themes cannot be edited or deleted as long as they are located under *NexacroTheme*. Therefore, you should copy them to the *Theme* field if you want to edit them. Drag and drop is a way of such copying.

When you try to edit a default theme, you will see the below dialog box asking whether to copy the theme to the relevant project. If you click [Yes], Nexacro Studio will copy the theme to the *Theme* field and open the editor.



10.1.1 Image

The images used in a theme are provided in a tree. You can add or delete images while you can see the visual of an image through the preview pane.



Those images can be applied by writing the images' URLs in the source code of the related CSS file.

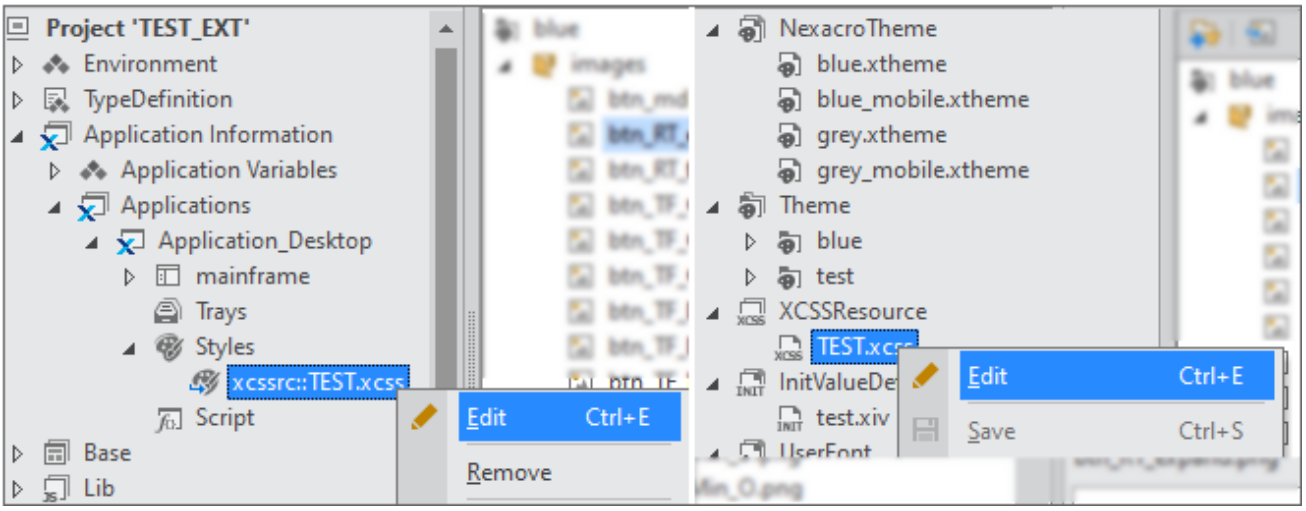
10.1.2 XCSS

You can edit a XCSS file that defines the style of a theme.

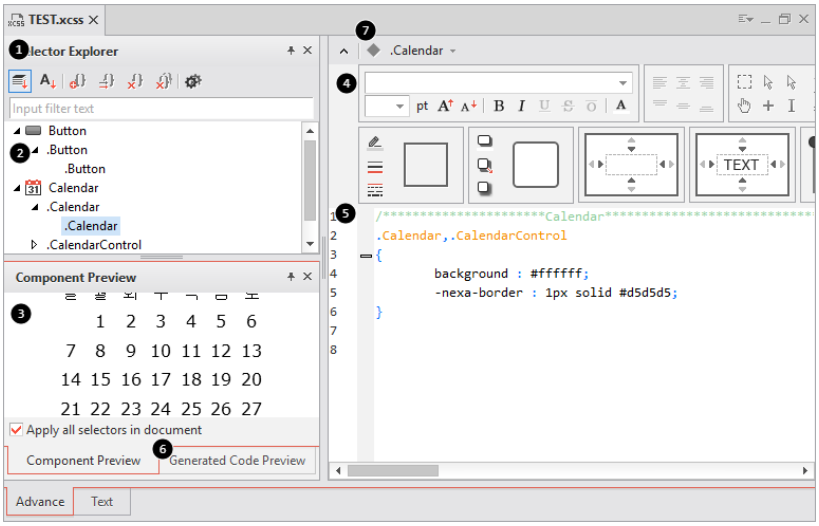
10.2 XCSS

The XCSS here means a feature to design the elements of a screen in Nexacro. The selectors to which a XCSS style can applies include components, Forms, frames, title bars, status bars, and scrollbars.

You can open the XCSS editor by double-clicking an XCSS file after selecting the file in the Project Explorer or Resource Explorer. Otherwise, open a context menu from the XCSS file and then select [Edit].

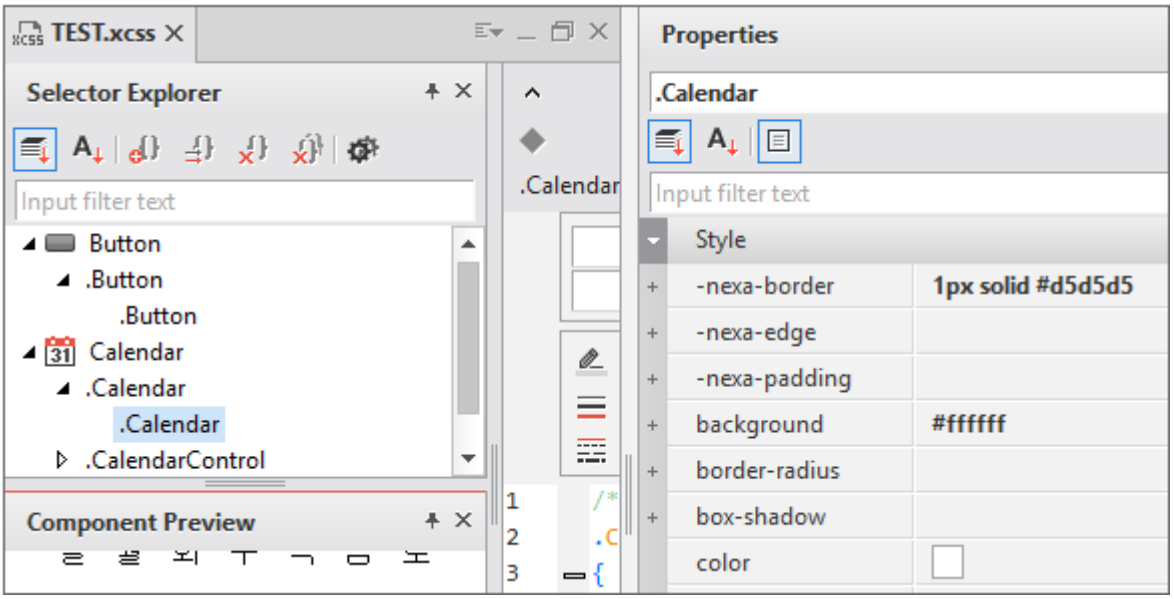


The XCSS editor presents a selector list in a tree form.

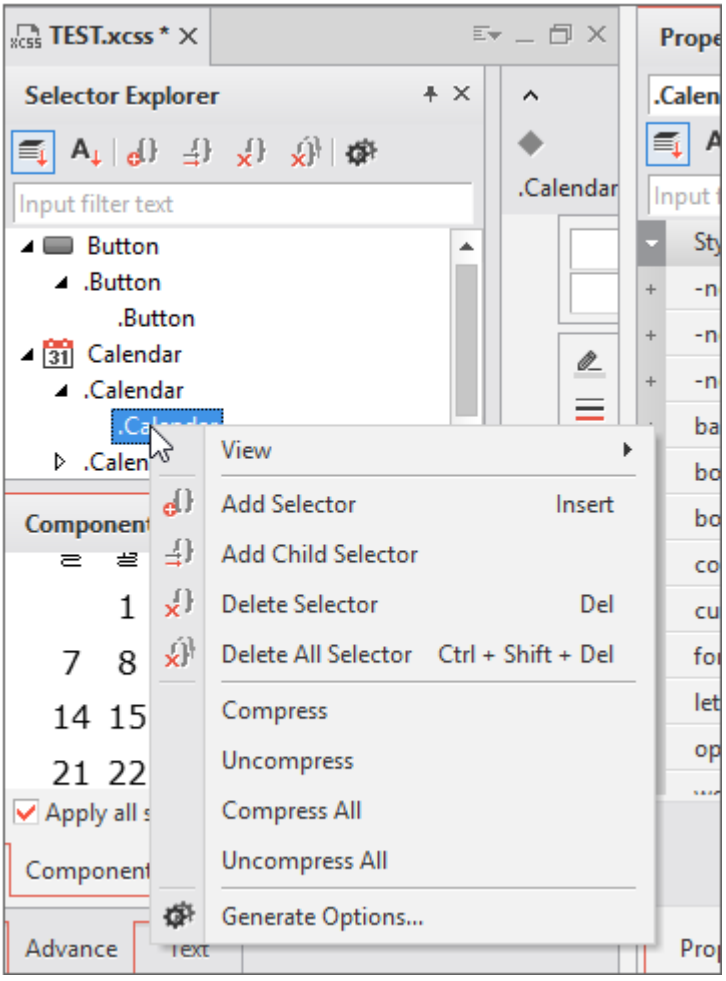


Call-out		Description
1	Sort by Group	Classifies selectors according to component groups
	Sort by Alphabet	Classifies selectors according to the alphabetical order
	Add Selector	Adds a selector
	Add Child Selector	Adds a child to the selected selector
	Delete Selector (Delete)	Removes the selected selector
	Delete All Selector (Ctrl+Shift+Del)	Removes all selectors
	Options	Selects the web browsers to which native CSS will apply after generating it
2	Selector Tree	Shows selectors in the form of a tree
3	Component Preview	Previews a selected item
4	Style Quick Editor	Allows you to quickly edit typical style elements
5	xcss Editor	Nexacro Style Sheet editor
6	Generated Code Preview	Previews a CSS script to which the relevant XCSS file is converted for each NRE/web browser
7	Navigatorbar	Shows a list of selectors or allows you to add a select

When you choose a selector in a tree, Nexacro Studio will display the codes of the relevant XCSS file, native CSS file and preview image. Moreover, you can edit the attributes of the selector through the Properties pane.



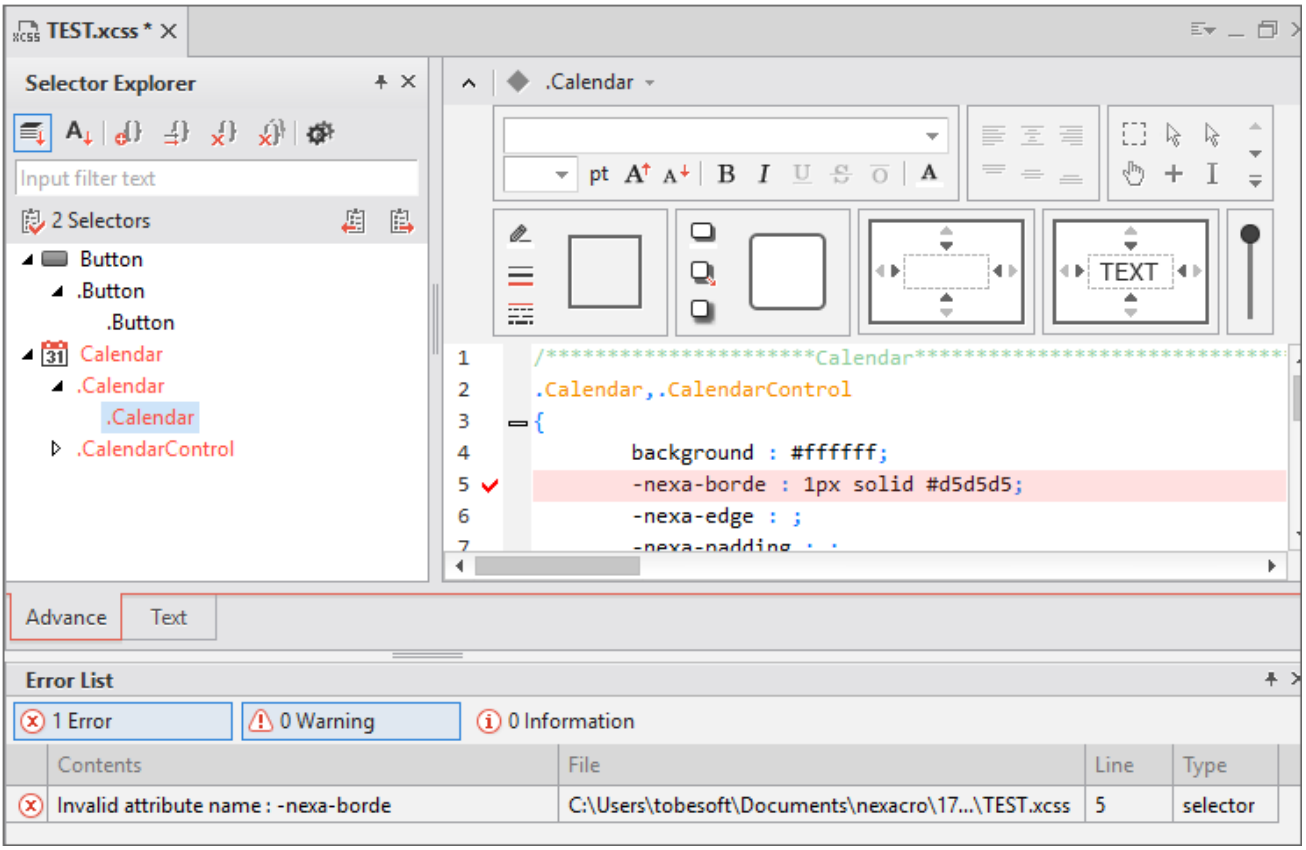
You can open a context menu by right-clicking a select in the tree, and this context menu provides a function to remove unused attributes.



Name	Description
View	Determines the presentation of the panes that can be displayed within the XCSS Editor Selector Explorer Component Preview Generated Code Preview
Add Selector	Adds a selector
Add Child Selector	Adds a child to the selected selector
Delete Selector	Deletes the selected selector
Delete All Selector	Deletes all selectors
Compress	Deletes unused attributes from the selected selector
UnCompress	Restores deleted attributes to the selected selector
Compress All	Deletes unused attributes from all selectors
UnCompress All	Restores deleted attributes to all selectors
Collapse All	Collapses all branches of the tree
Expand All	Expands all branches of the tree

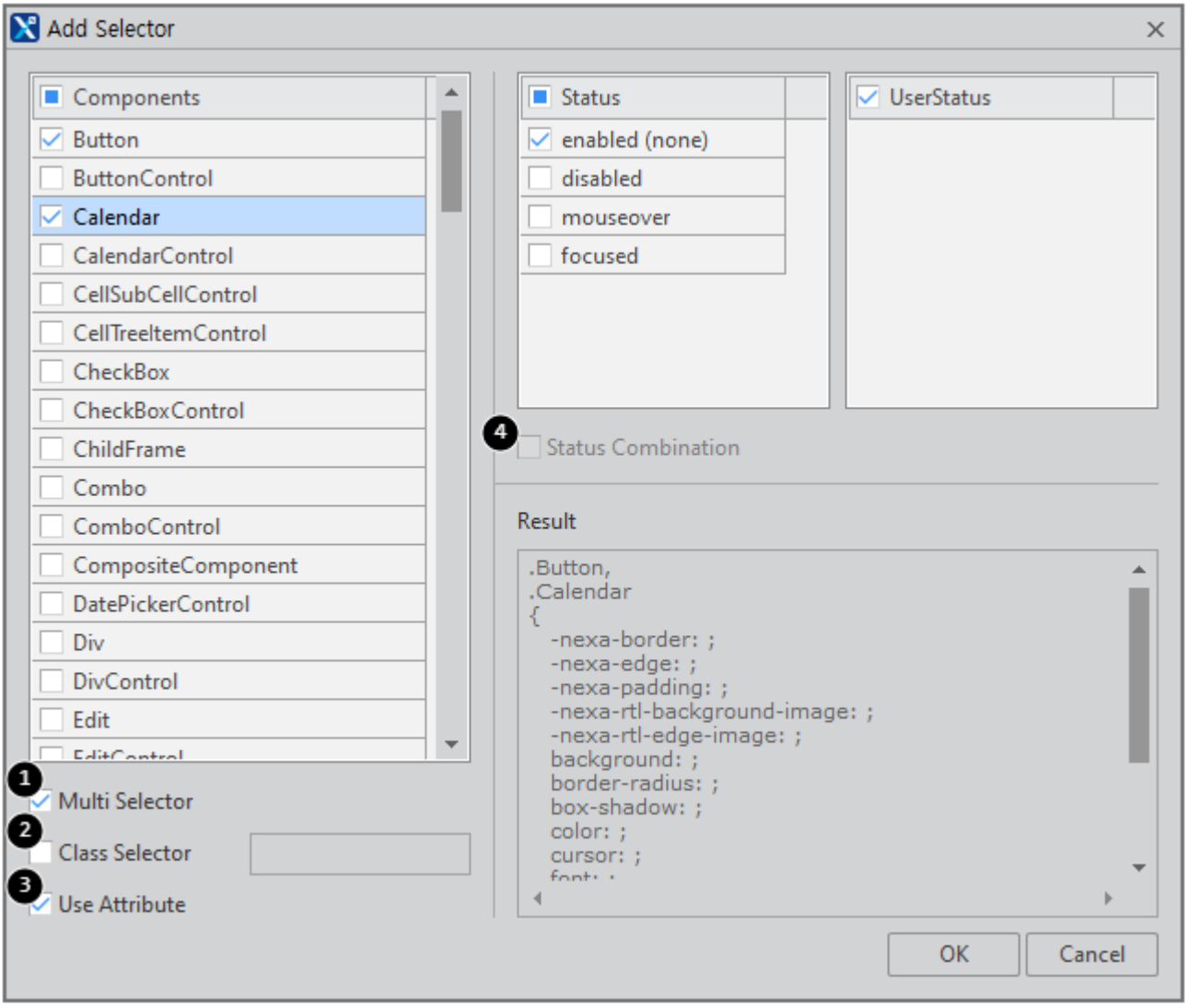
If there are errors in XCSS scripts, the editor will count the number of errors and mark the erroneous

selectors.



10.2.1 Adding Selector

You can add type selectors by clicking **Add Selector** in the toolbar of the CSS editor. To this end, check the desired selectors from the "Components" list.



Call-out		Description

1	Multi Selector	You can define one single style for multiple selectors.
		If you select more than one component or one status, the checkbox for this option will be activated. If you select the checkbox, the involved selectors will be defined by one style.
		For example, the below CSS code will be written if you select a Button and Calendar components and then check this option.
		`.Button,`
		`.Calendar`
		`{`
		`}`

2	Class Selector	Specifies a certain class and applies the style of the current selector to the class. For example, you can define another style by declaring the "TEST" class using a Button component. In this case, you can apply the TEST class by specifying "TEST" as a value of the cssclass property.

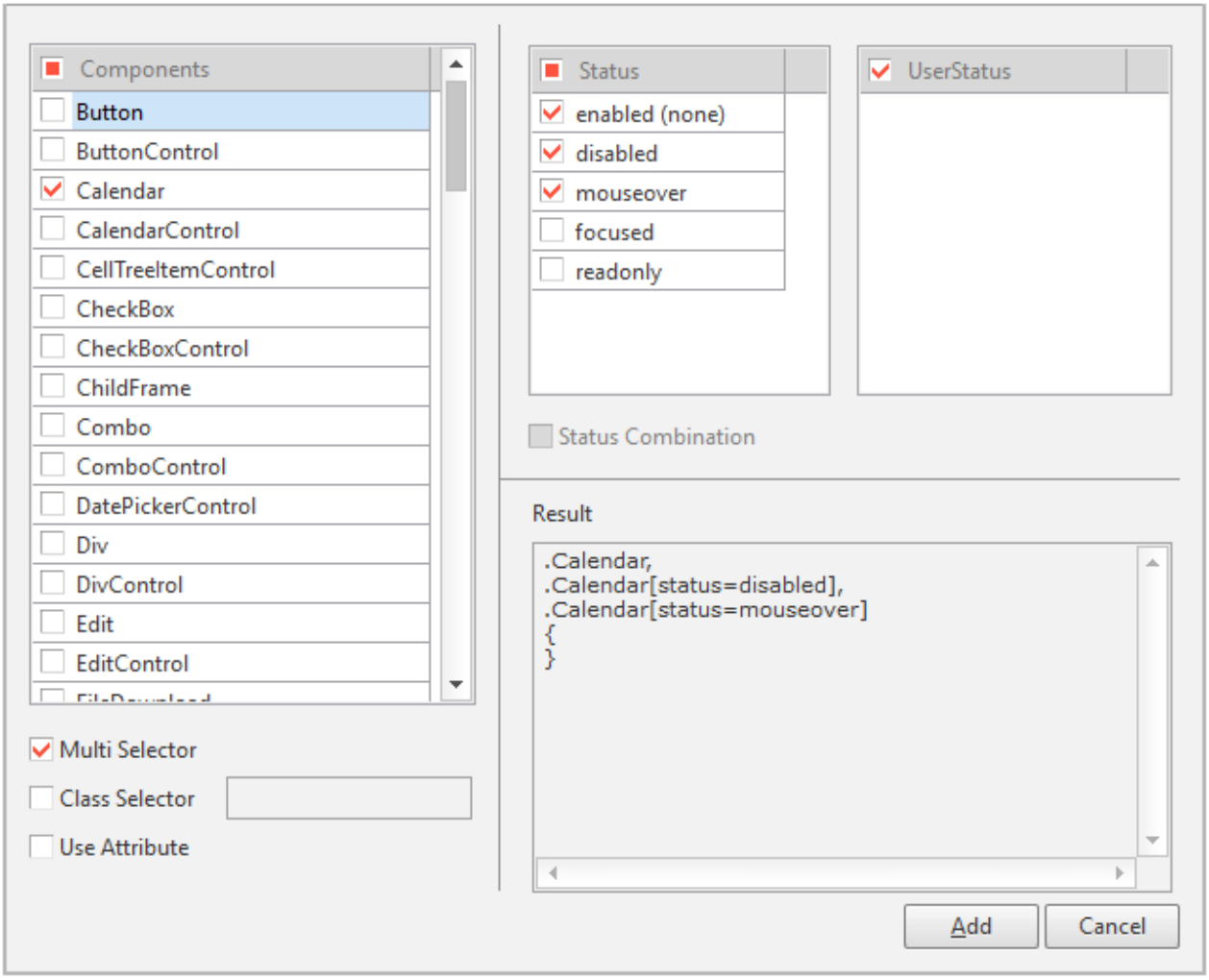
Call-out		Description

		`.Button.TEST`
		`{`
		`background : aqua;`
		`}`

3	Use Attribute	Determines whether to add the attributes of the relevant selector to the editor. If checked, the editor will present the list of attributes applicable to the relevant selector.

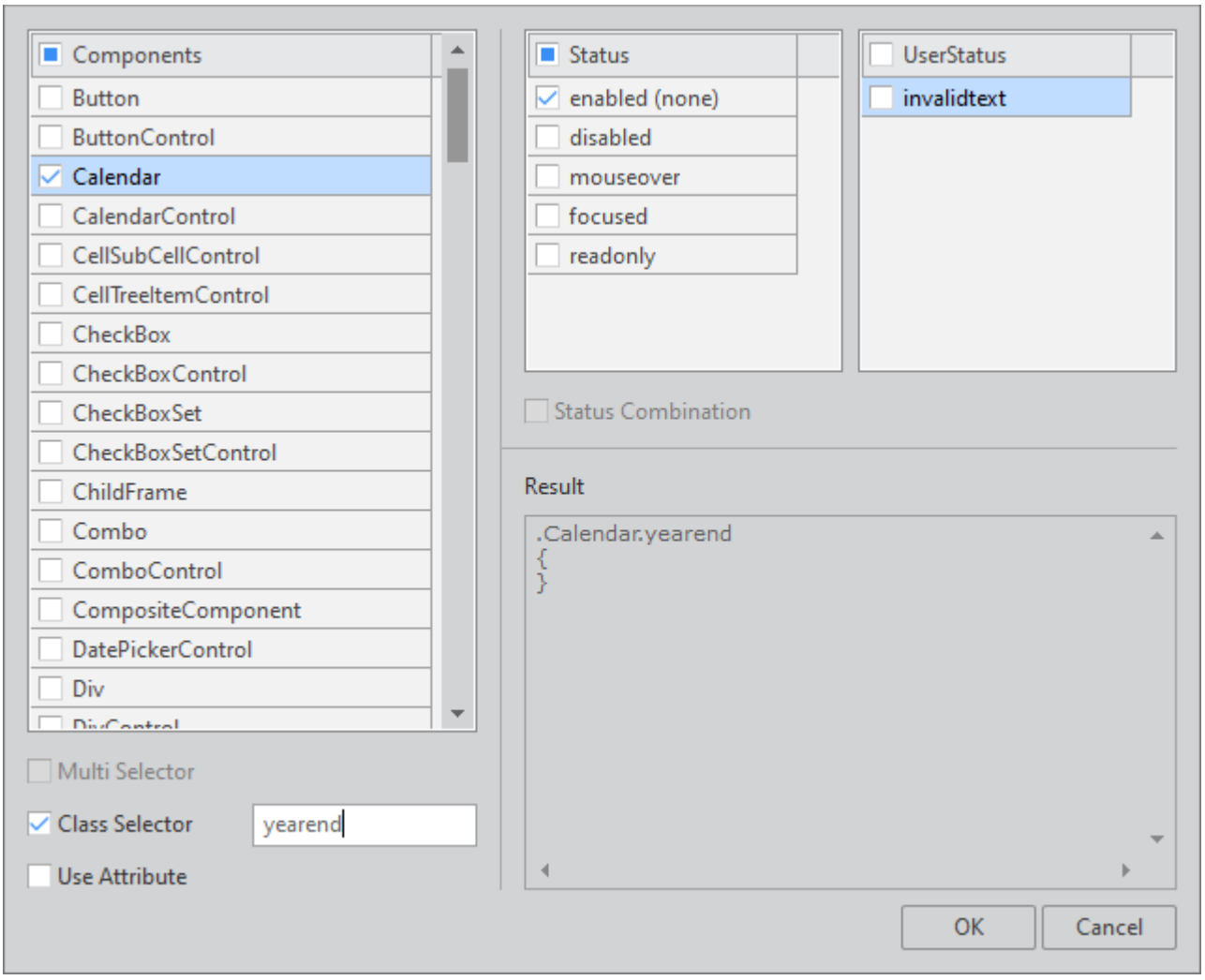
4	Status Combination	You can define one single style for a combination of a Status element and a UserStatus element.
		For example, the below CSS code will be written if you select the *mouseover* Status and *selected* UserStatus and then check this option.
		`.Button[status=mouseover][userstatus=selected]`
		`{`
		`}`

You can style various statuses of a selector through **Status**. The "enabled" status is a default value so you do not need to designate it separately.

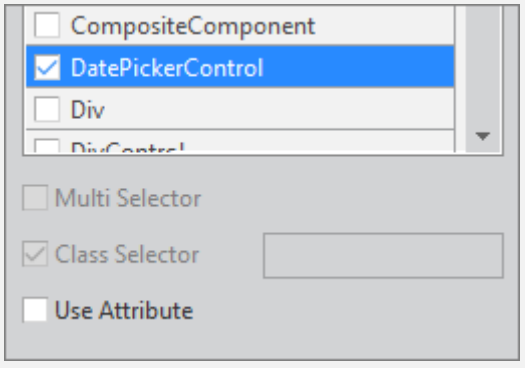


10.2.2 Adding Class Selector

Check **Class Selector** in the Add Selector window and then enter a class name. Check a component if you want to combine a class selector with a component.

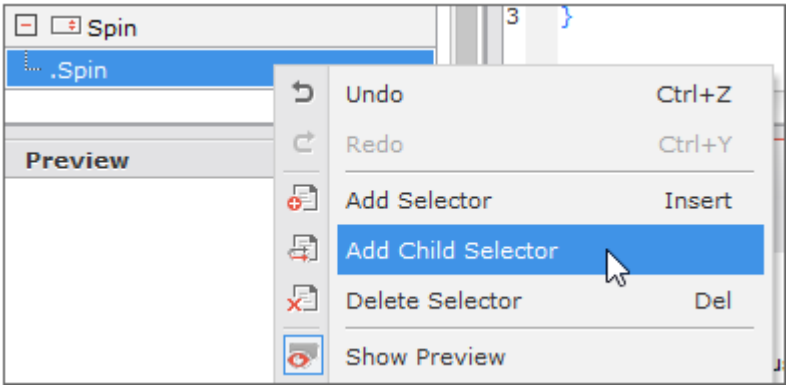


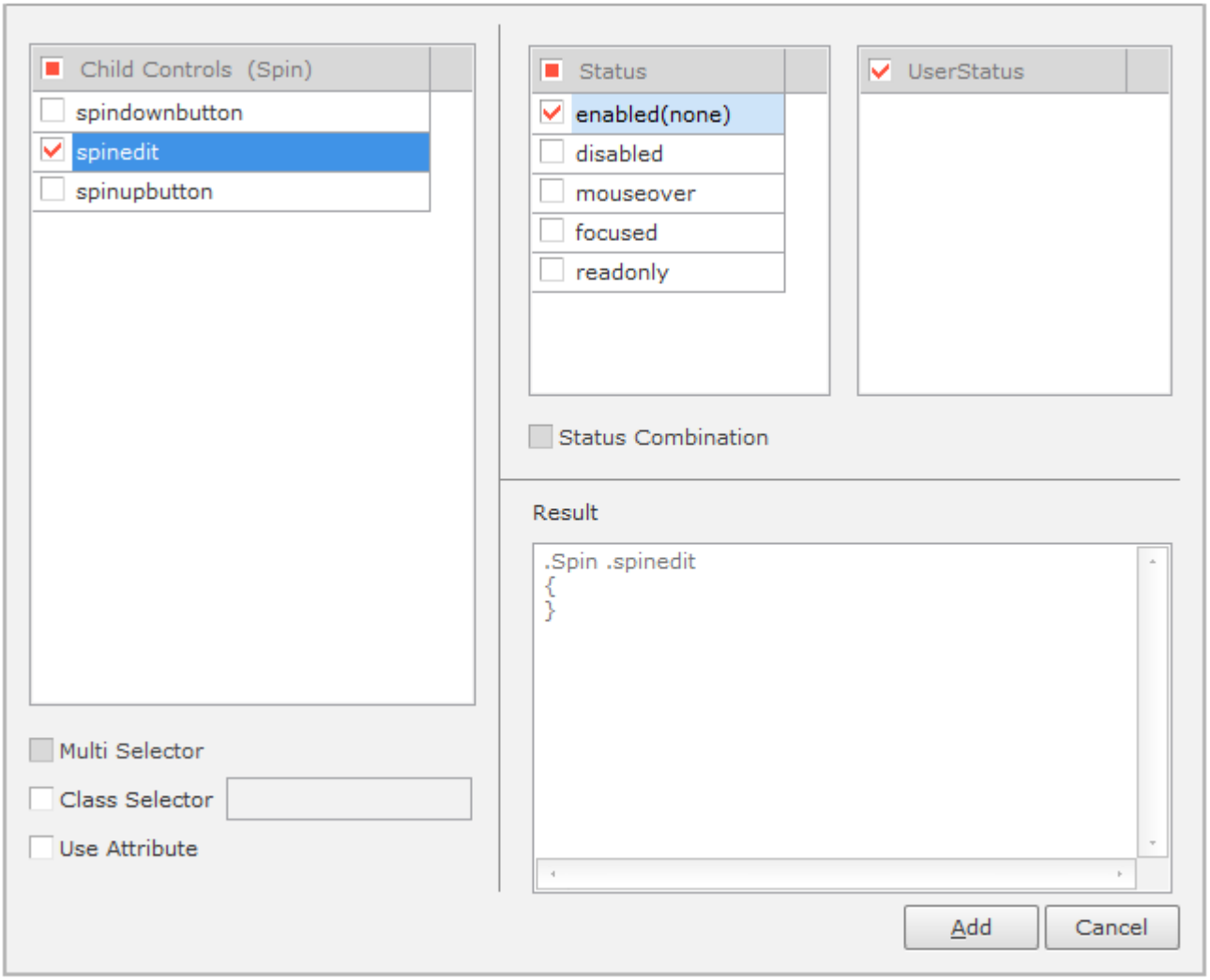
The Class Selector cannot be added when selecting the Control item from the Components list.



10.2.3 Adding Child Selector

You can add a child selector by going to the tree, choosing a selector that has its children and then clicking the **Add Child Selector** button on the toolbar.



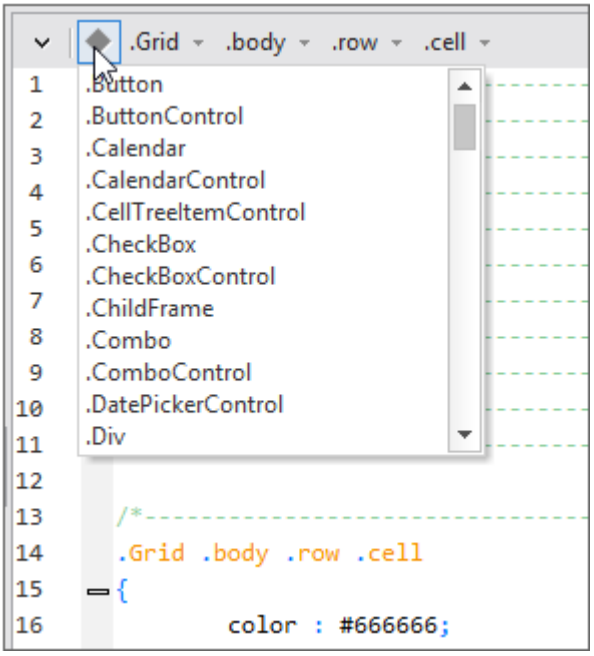




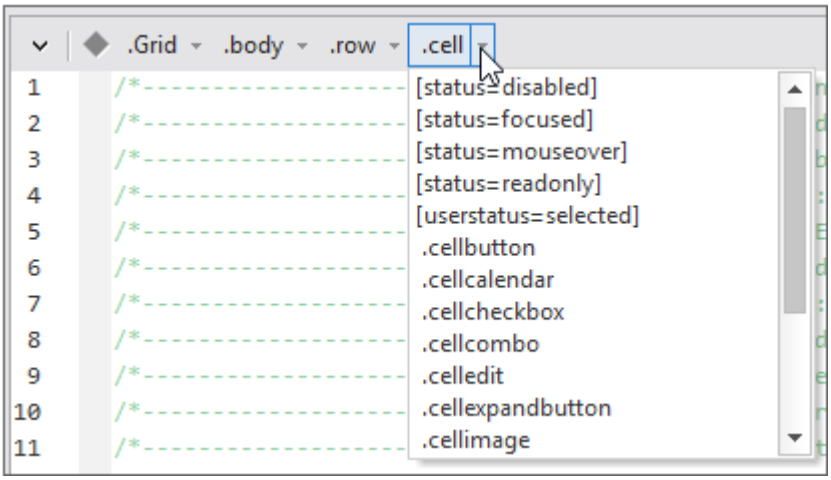
The child selector used in Nexacro is different from the one used in original CSS. It is similar to a descendant selector in original CSS .

10.2.4 NavigatorBar

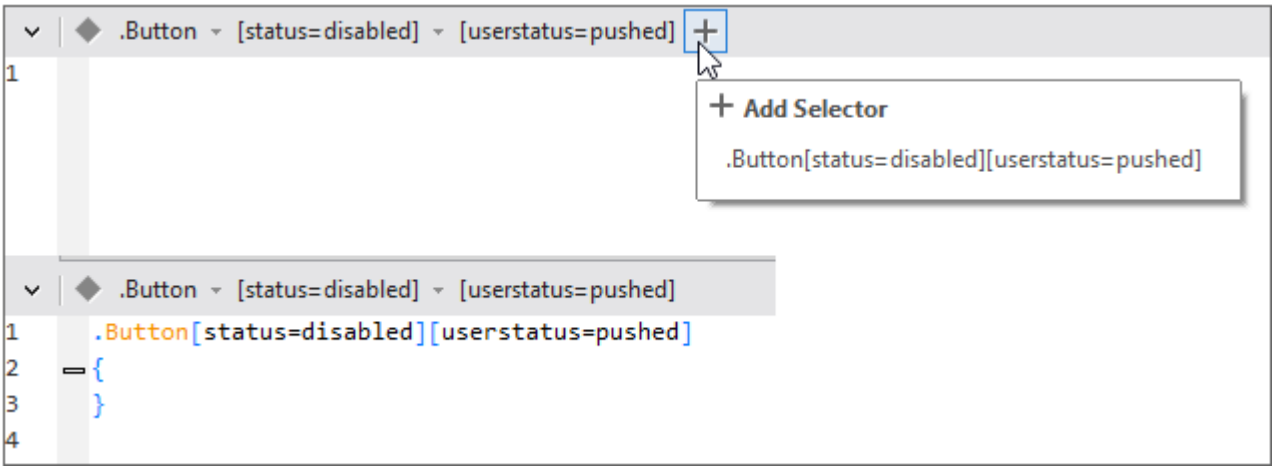
This feature allows you to go directly to a certain component or its sub-element. Clicking the [♦] button will open a list of the elements that you can access. If you click one of them, you will move to the code of the relevant element.



Clicking the drop-down button of a selector or child selector will open the list of the sub-elements. If you select one of them, you will move to the code of the selected sub-element.

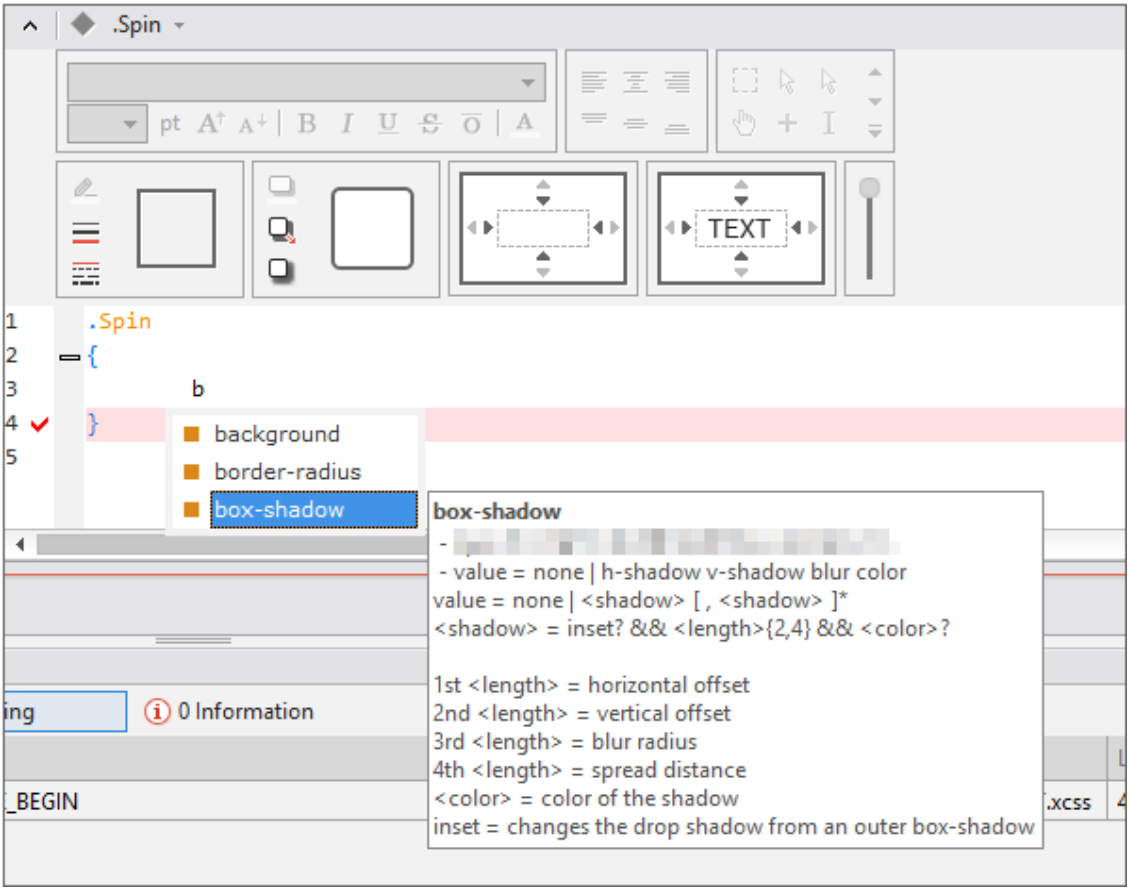


If a selected element does not exist, the [+] button will appear, which you can click to add the relevant element.



10.2.5 Editing XCSS Code

If you type few letters in the editor, Nexacro Studio will automatically present a list of the properties that begin with the input text. After you select one of them by pressing the Enter key, the editor will display the related syntax.

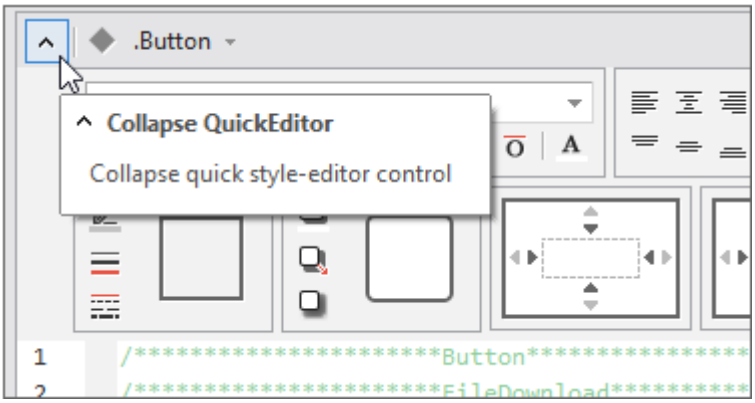


If the syntax involves multiple patterns, you can navigate them with the Up/Down arrow keys. After you select a certain pattern by pressing the Enter key, the editor will display a related sample code.

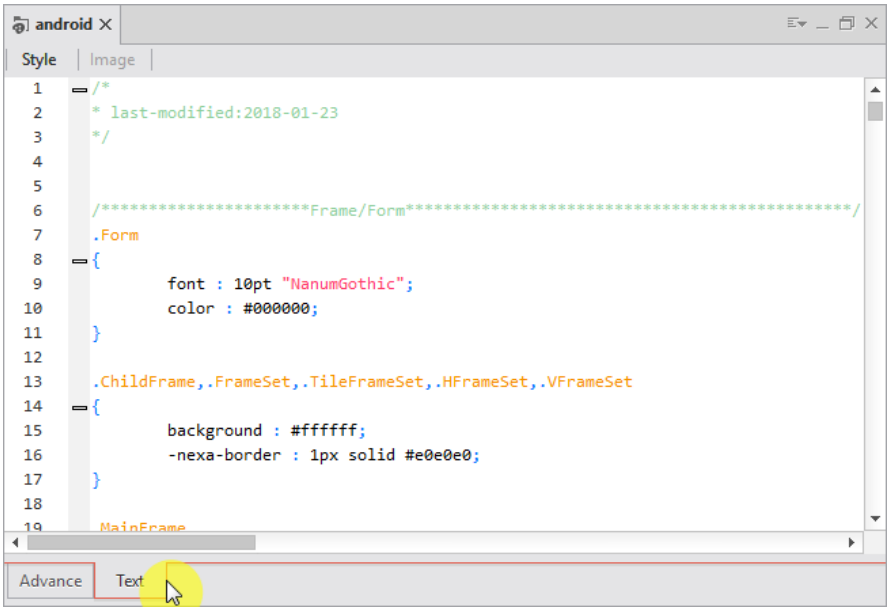




You can collapse or expand the Style Quick Editor field to provide more room to the Text Edit field.

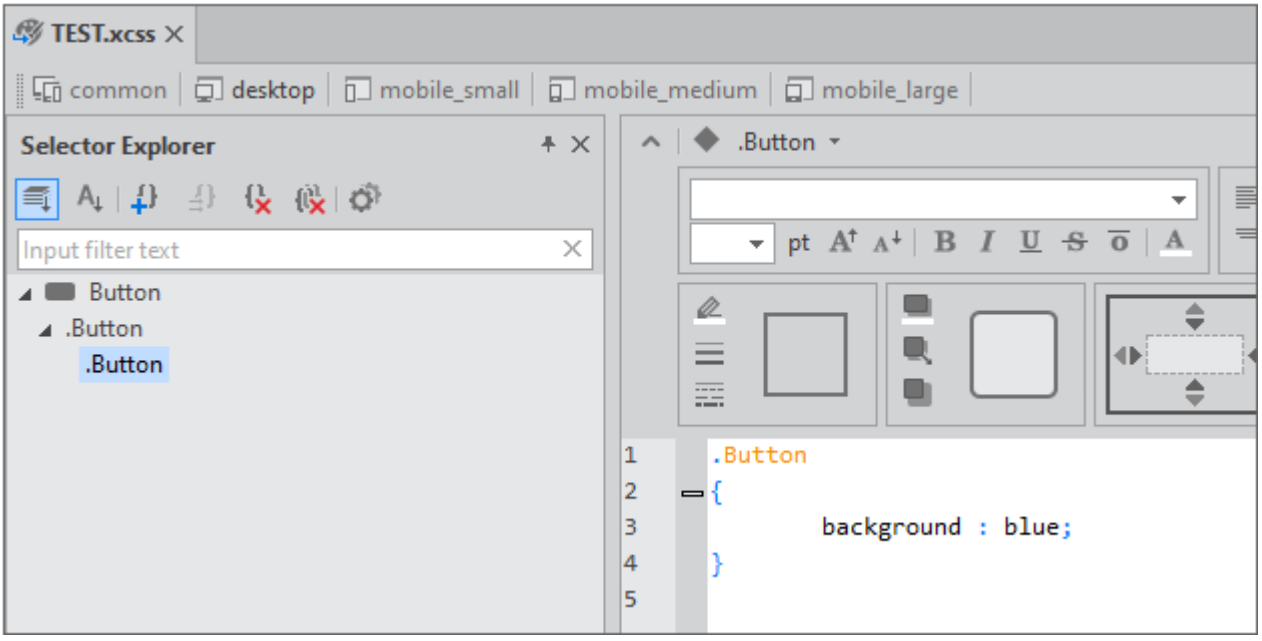


If you click the **Text** tap at the bottom of the XCSS Editor, you can edit the entire code of a certain XCSS file.



10.2.6 Defining Styles Based on the Screen

You can apply different styles to certain selectors depending on the value of the screen type property. When editing the theme or XCSS, you can select type and add a style property or specify a different value.



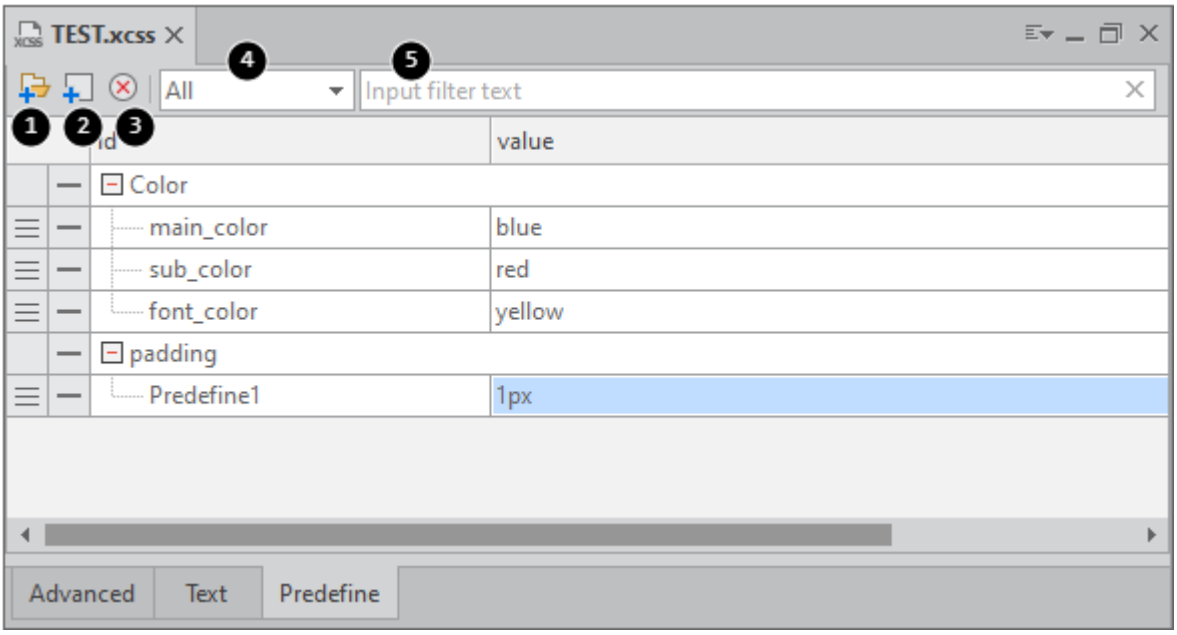
10.2.7 Declaring a variable and using it as a property value (Predefine)

When editing XCSS, you can declare a value that is repeatedly used as a variable in advance and designate it as the variable (id) instead of directly specifying the property value. When you need to modify the property value, you can quickly reflect the property value by modifying the value of the desired item among the Predefine items without directly finding and changing the property value.

The Predefine function is a function that can only be used on the Nexacro Platform.
It can be set only when editing XCSS, and item values cannot be accessed or modified in the script.
This is a different feature than CSS custom properties (variables).
When editing XCSS, CSS custom properties (variables) are not supported.
https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties

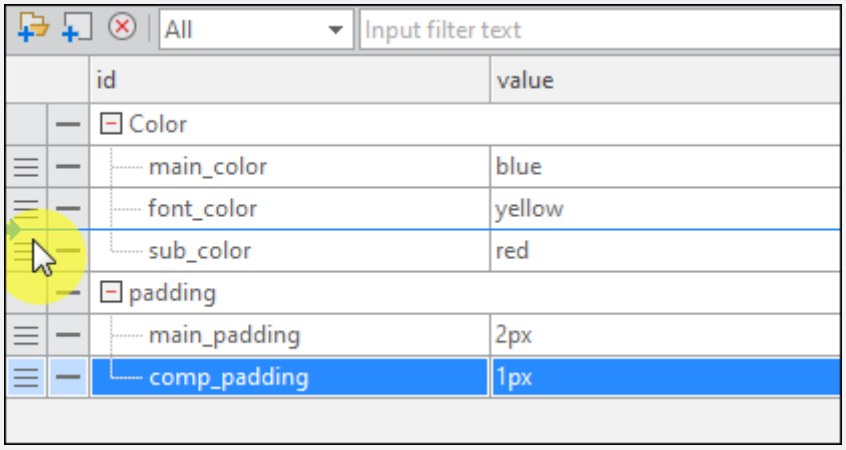
Declare a variable

Select the Predefine tab in the XCSS editor window. Add a Group item and an Item item to be used as a variable.



	Name	Description
1	Add Group	Add a Group item. Group items are used only for classifying Item items by type. When setting Predefine, more than one Group item is required.
2	Add Item	It is a variable composed of id and value. Among the Item items, the id value is used when setting property values in XCSS. Duplicate ids within the same XCSS file are not allowed.
3	Delete	Delete the selected Group or Item. When deleting a group, the items included in the group are also deleted. You can delete after selecting 2 or more items. When deleting an individual group or item, you can delete it with the '-' button in front of each item.
4	Group selection filter	Only items included in the Group selected from the list are displayed. When 'All' is selected, all Groups and Items are displayed.
5	id search filter	Displays only items with id values that contain the string entered in the search bar.

Items can be moved to another Group.

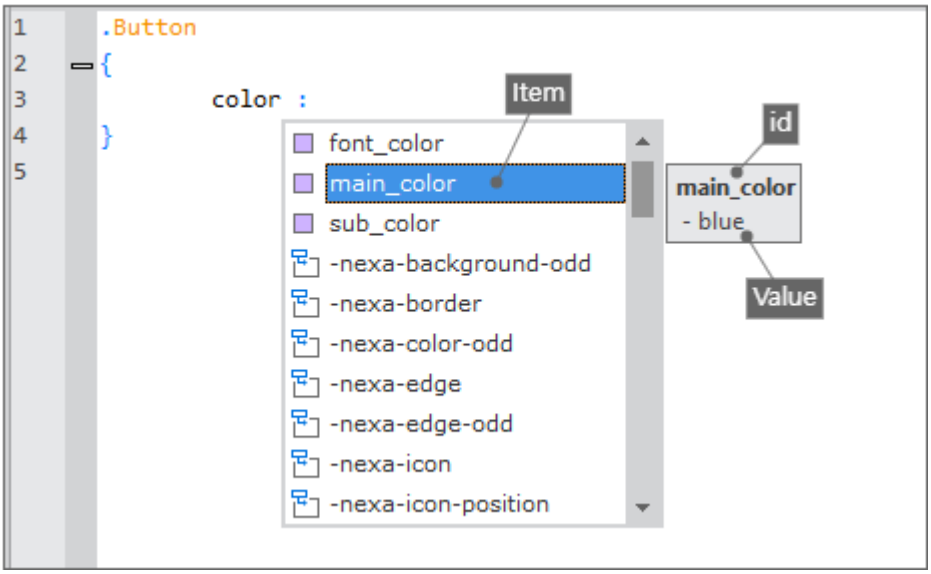


The order of Items declared within a Group has no effect on behavior and cannot be reordered.

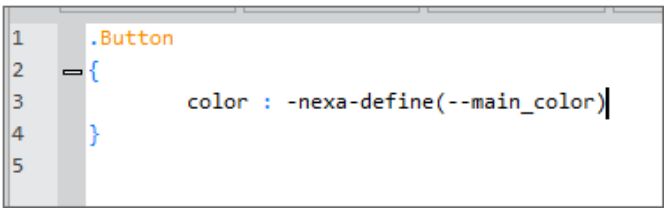
Variables (Item) declared in the Predefine tab can be used when editing XCSS property values.

Using declared variables as property values (XCSS Editor)

When editing XCSS, you can use the IntelliSense function to check and select items declared in the Predefine tab.

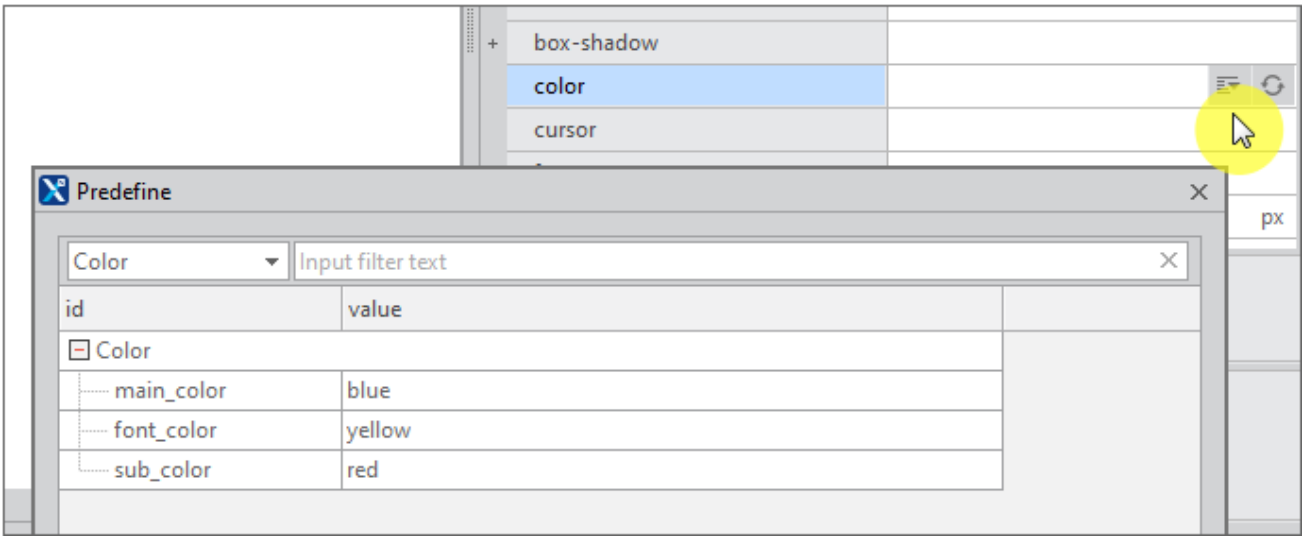


When an item is selected, it is automatically converted into the format shown below and set as a property value.

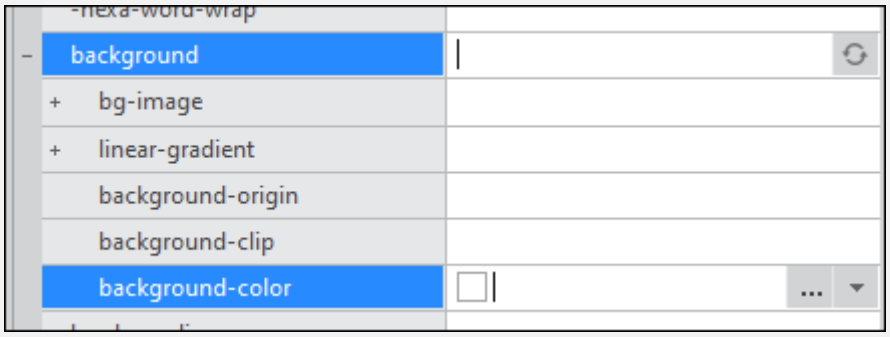


Using a declared variable as a property value (property window)

To select an item declared in the Predefine tab in the property window, click the 'Predefine' button in each item of the property window to switch the state, then click the 'Predefine List' button to open the window and select the item.



Using the Predefine function in the property window can only be set per property unit.
For example, you can use the Predefine function when setting the background property value, but the 'Predefine' button is not displayed and cannot be used for the background-color item under it.



Check the generate result

The value using the Predefine function is converted to the corresponding value during the generate process and reflected. When editing XCSS, you can check the converted value in the Component Preview window or the Computed Style tab of the property window.



Sharing Predefine Settings

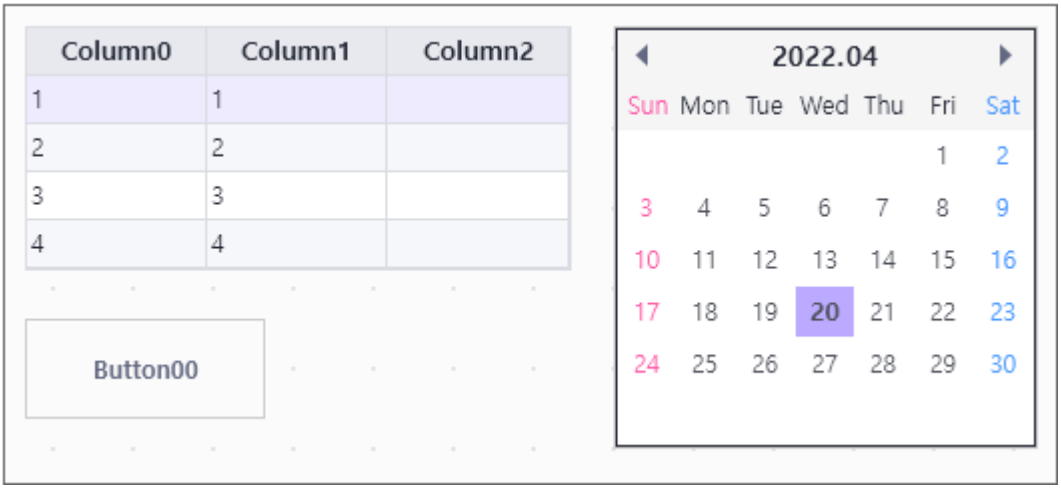
Predefine settings are applied only within the XCSS or XCSS files within the theme. If you want to share the values you set, you can open the XCSS file in an external text editor and copy the entries inside the Predefines tags.

```
<?xml version="1.0" encoding="utf-8"?>
<XCSS version="1.1">
  <Predefines>
    <Group id="Color">
      <Define id="main_color" value="blue"/>
      <Define id="font_color" value="yellow"/>
      <Define id="sub_color" value="red"/>
    </Group>
  </Predefines>
  <CSSData><![CDATA[.Button
{
    background : -nexa-define(--main_color);
}
]]></CSSData>
</XCSS>
```

Usage example: Changing the gray theme color

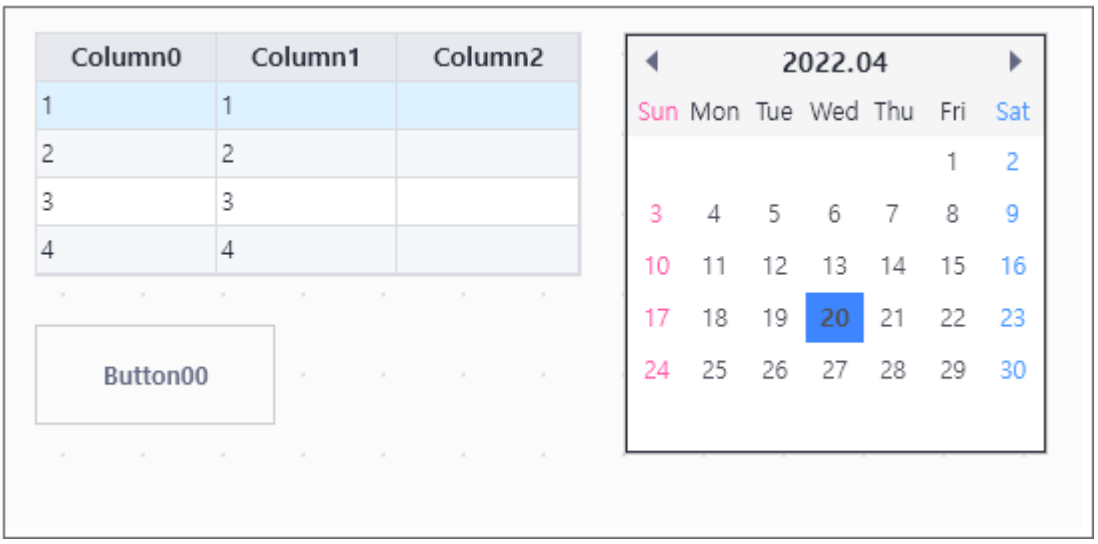
In the gray theme, the color for the selected item is defined in the Predefine item as shown below.

```
maincolor: #BBAAFF
subcolor: #EEEBFF
```



If you change the Predefine item value to the desired value (blue series) and save it, you can see that the changed value is reflected.

```
maincolor: #3E86FF
subcolor: #DEF1FE
```



11.

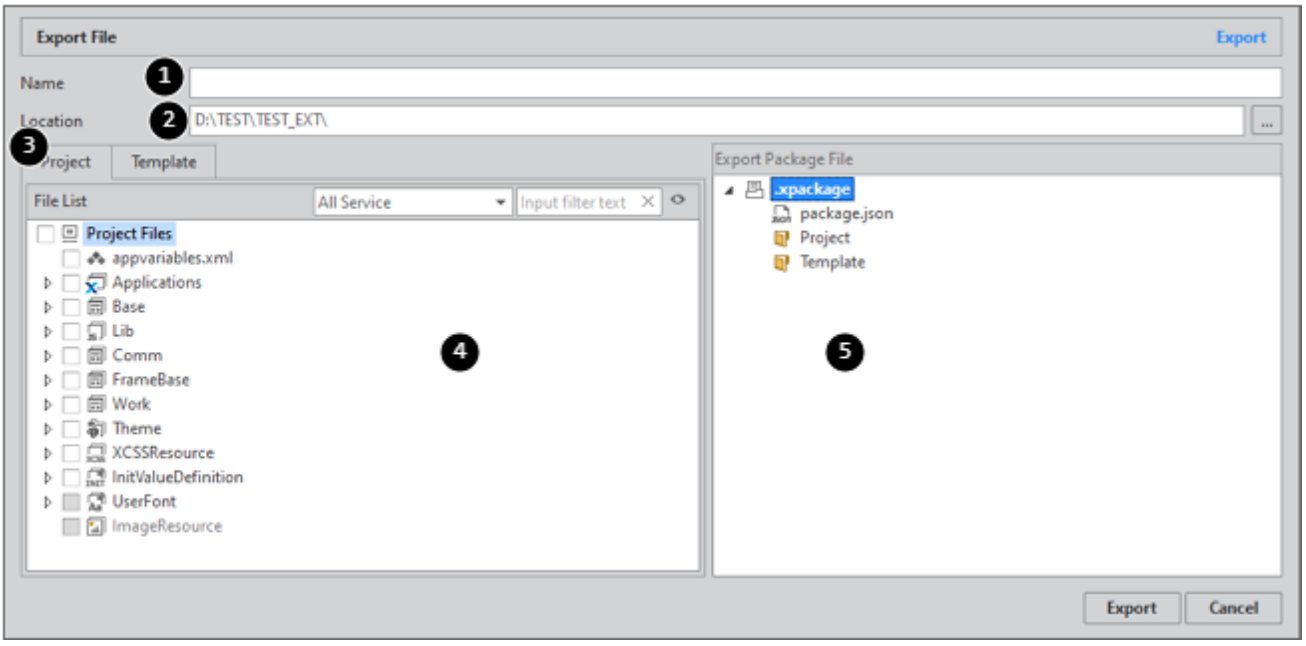
Project Editing | Export, Import, Module Installation

11.1 Export

Information, such as service files, templates, and snippets, in the project can be grouped and exported as a single file. The exported item is created as the file with the xpackage extension and can be imported when executing Import.

11.1.1 Selecting & Exporting the Export Item from File Menu

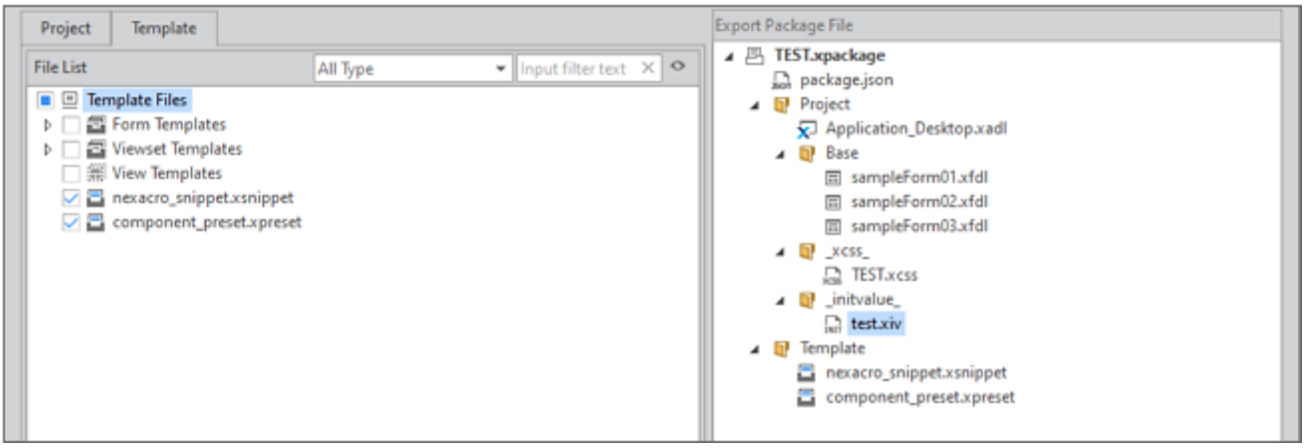
If you select the menu [File > Export] when the project is open, then the Export Wizard is executed.



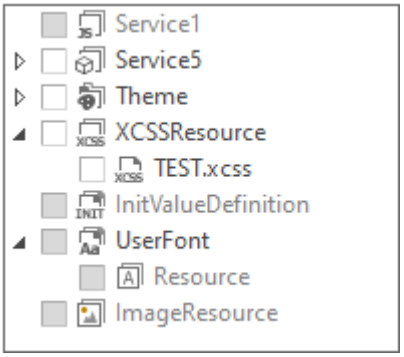
	Item	Description
1	Name	Sets the file name to export.

	Item	Description
		Creates the [file name].xpackage file.
2	Location	Sets the path to create the file to export. The default path is specified as the project path. The path can be changed.
3	Project/Template	When selecting the tab, the lists of Project, Template are displayed in the File List area.
4	File List	Displays a list of files of templates in the project and sets the selected file as the file to export.
5	Export Package File	Displays a list of files of templates in the project selected from the File List.

The items checked in the File List are displayed in the Export Package File window. Only the items displayed in the Export Package File window will be exported.

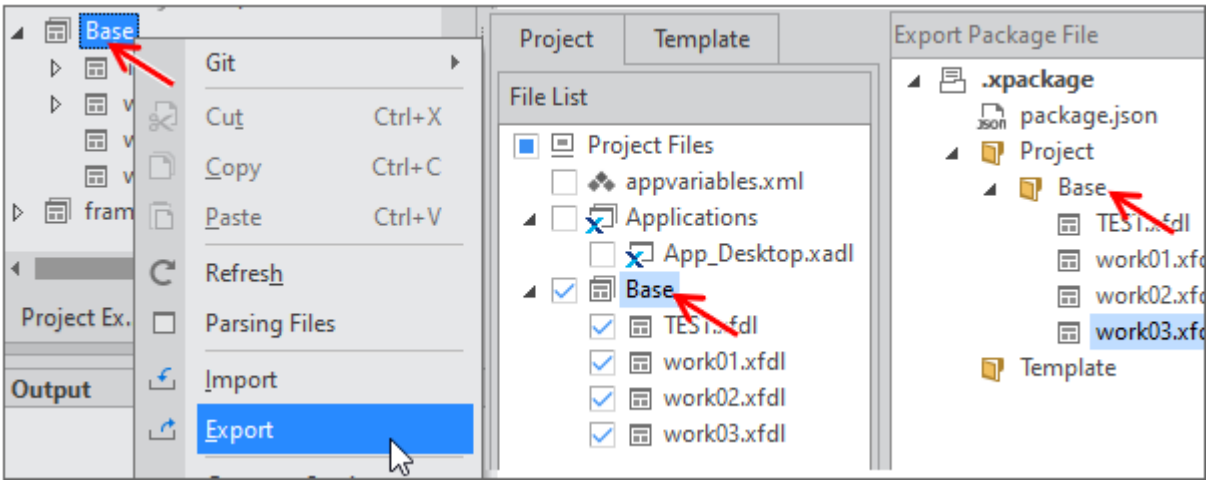


Services or items without the exporting targets cannot be checked.

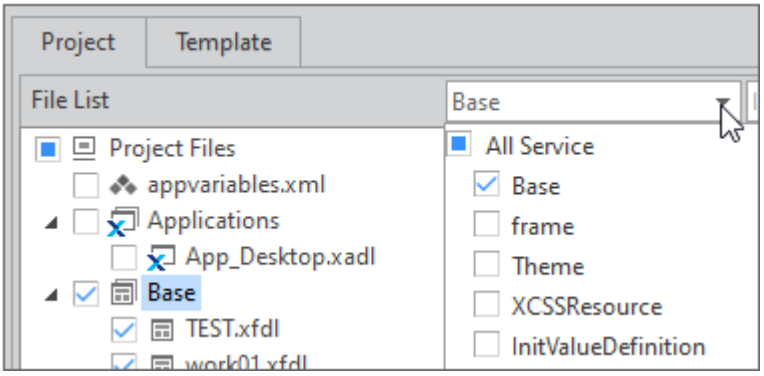


11.1.2 Selecting & Exporting the Export Item from Context Menu

Services or items that support export can select Export from the context menu. As when selecting the menu [File > Export], Export Wizard is executed and the selected item is checked and displayed in the Export Package File list.



When selected from the context menu, only the items for which the service or template type of the project is filtered are displayed.

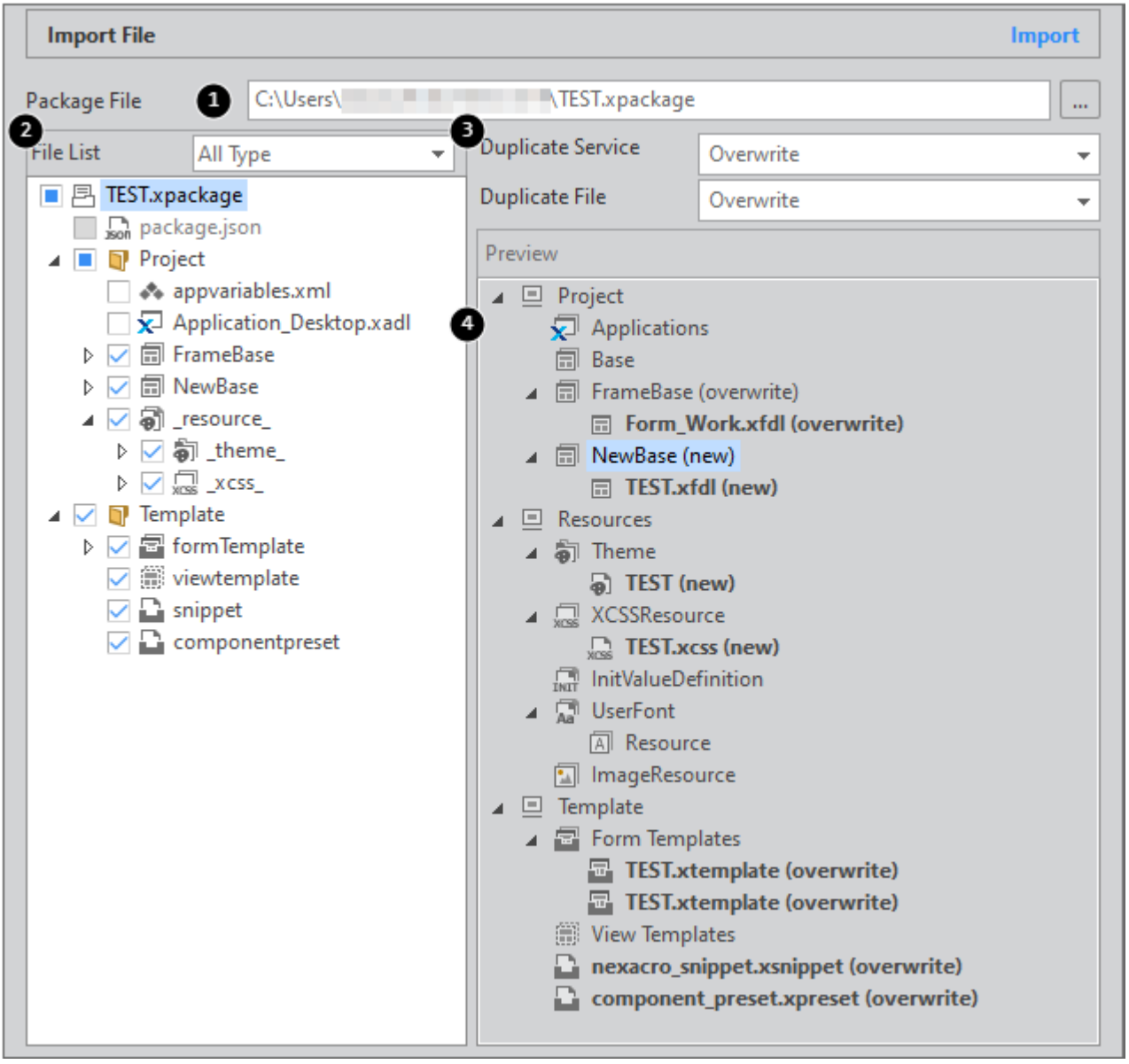


11.2 Import

You can import the xpackage files exported using the Export function or the individual files.

11.2.1 Selecting & Importing the Import Item from File Menu

If you select the menu [File > Import] when the project is open, then the Import Wizard is executed.

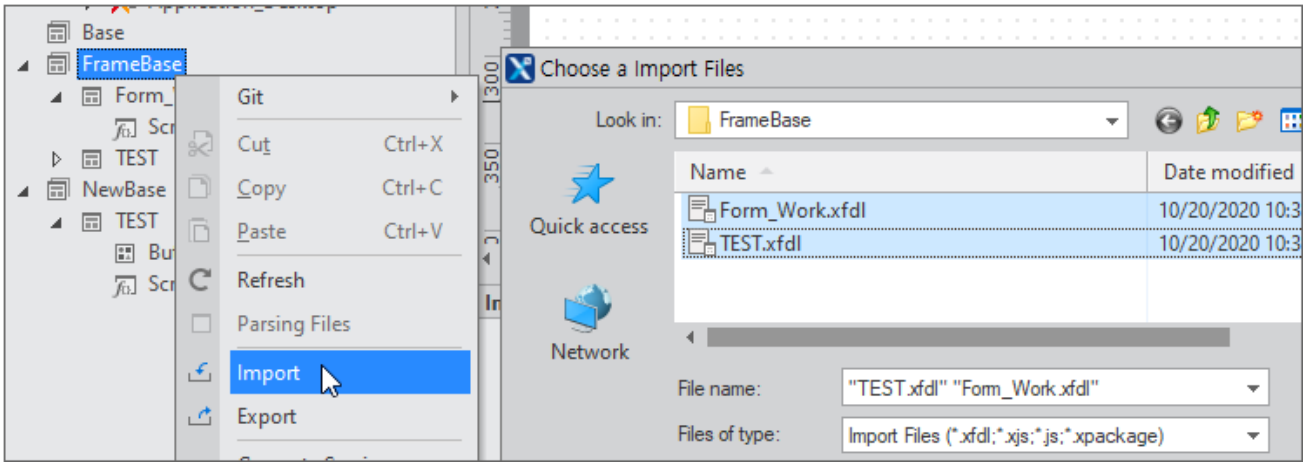


	Item	Description
1	Package File	Selects the file to import (xpackage).
2	File List	Displays the items included in the Package File. Selects the target to import. If you do not select Package File, then nothing is displayed.
3	Duplicate Service Duplicate File	Selects the processing method if the service of the file has the same name as the item that already exists.

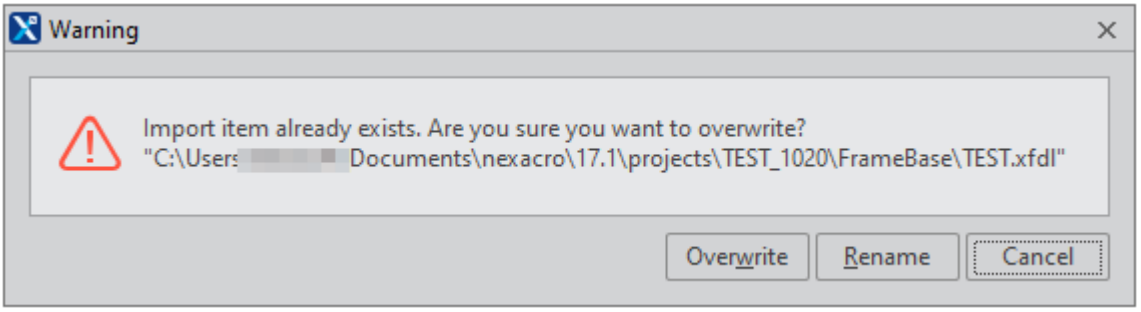
	Item	Description
		Rename & Add: Keeps the existing items and imports the selected items from the File List with different names. The name is changed in the form of "_copy0" after the item name and it is displayed as (new). **Overwrite**: Overwrites the existing item. It is displayed as (overwrite) after the item name. - Files that are not duplicate are displayed as (new), regardless of the selected processing method.
4	Preview	Shows the project and template structure to be reflected when the Import is executed.

11.2.2 Selecting & Importing the Import Item from the Context Menu

Services or items that support import can select Import from the context menu. You can select individual files to import or select the xpackage file to run the Import Wizard. Individual files can be imported by selecting multiple files.



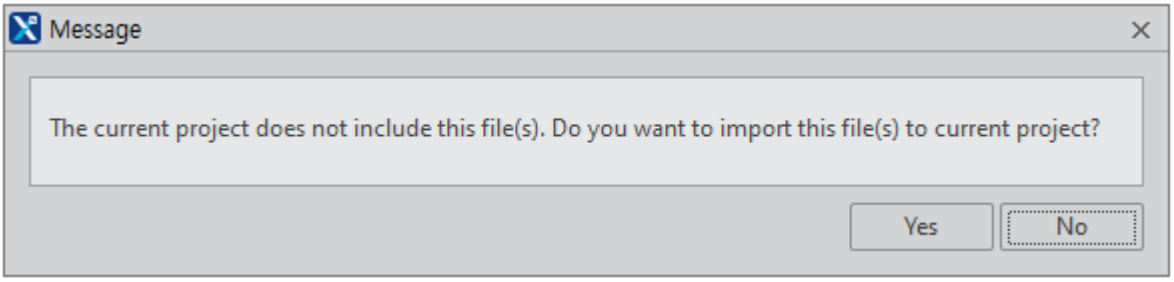
If there is a file with a duplicate name, then it will check if you want to overwrite or change it to a different name.



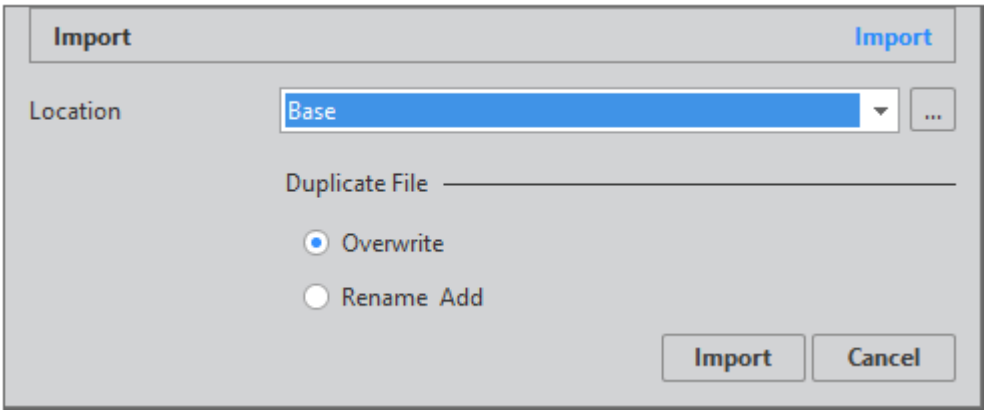
When the xpackage is selected, the Import Wizard is executed as in the case of selecting the menu [File > Import]. The Import Wizard is executed with the selected xpackage item selected as the file to import.

11.2.3 Importing File by Drag & Drop from File Explorer

You can select and import the xfdl file from the file explorer using drag and drop. If you select a file from the file explorer and drag and drop it to the Nexacro Studio area, then the following message will be displayed.



If you select [Yes], then the Import Wizard will be executed, and if you select [No], then the file will be opened without being included in the project and the source code will be shown.

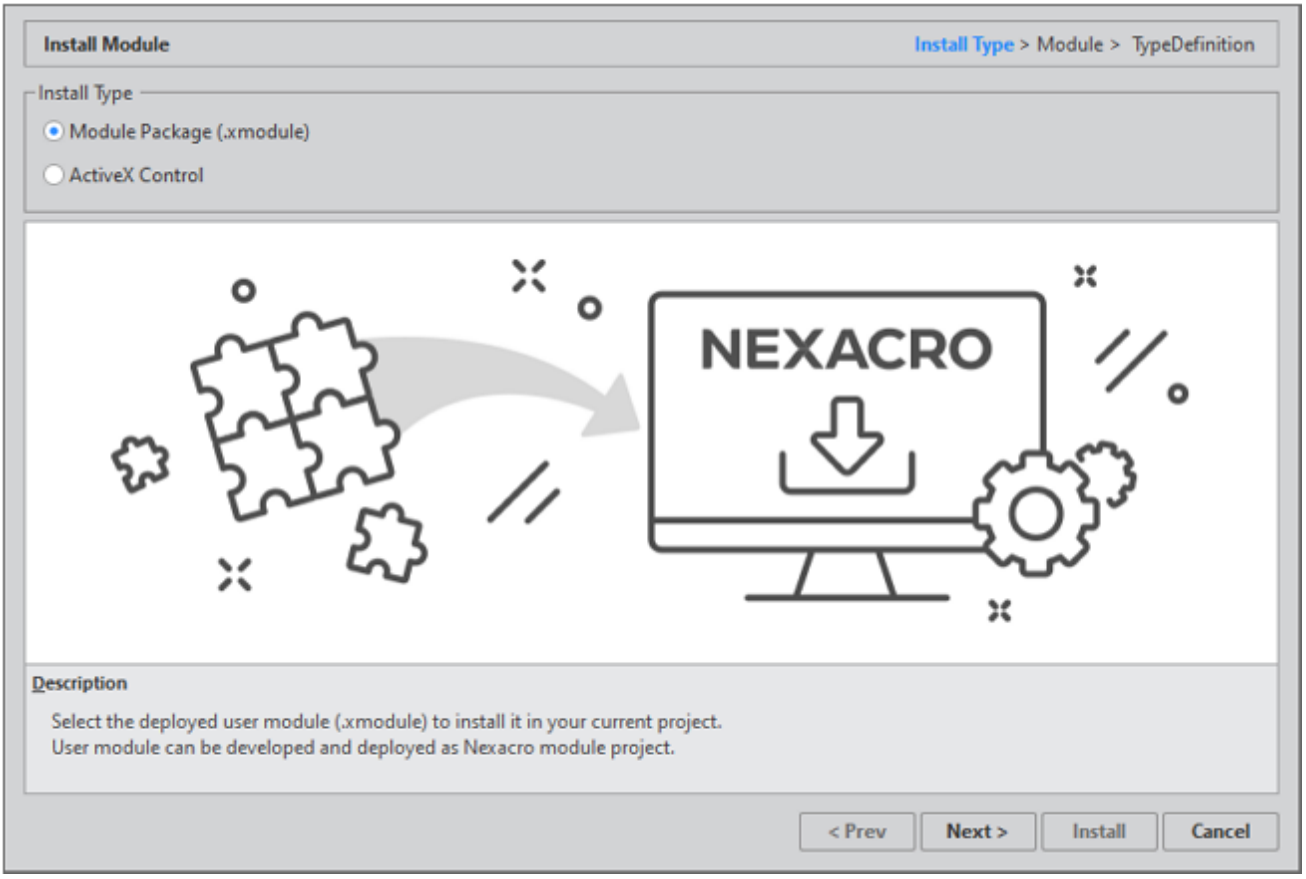


The Import Wizard selects the service to import the file from and selects the processing method on duplicate files to import the file.

11.3 Install Module Wizard

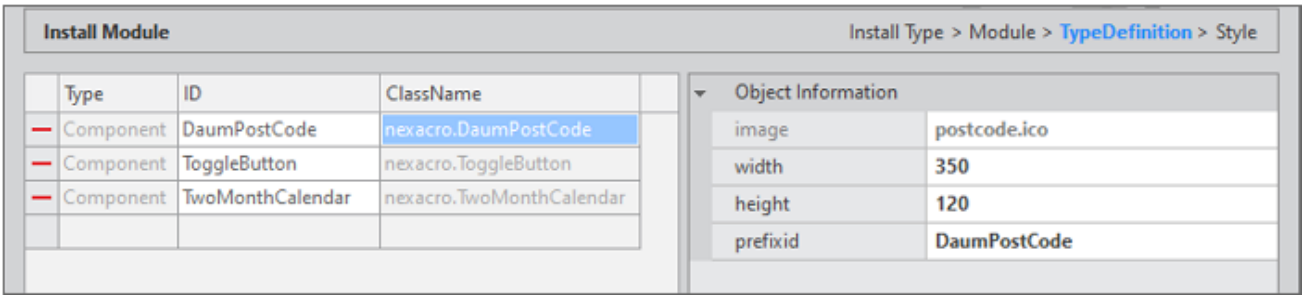
11.3.1 Module (*.xmodule*) Installation

You can bring an *.xmodule* file from a module project and then install it. To this end, go to the menu [File > Install Module] and specify the path to an *.xmodule* file.

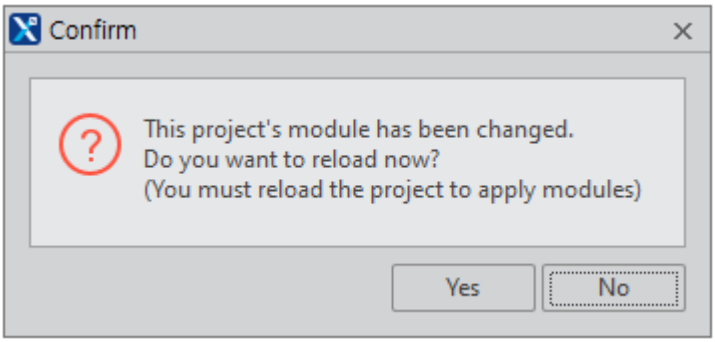




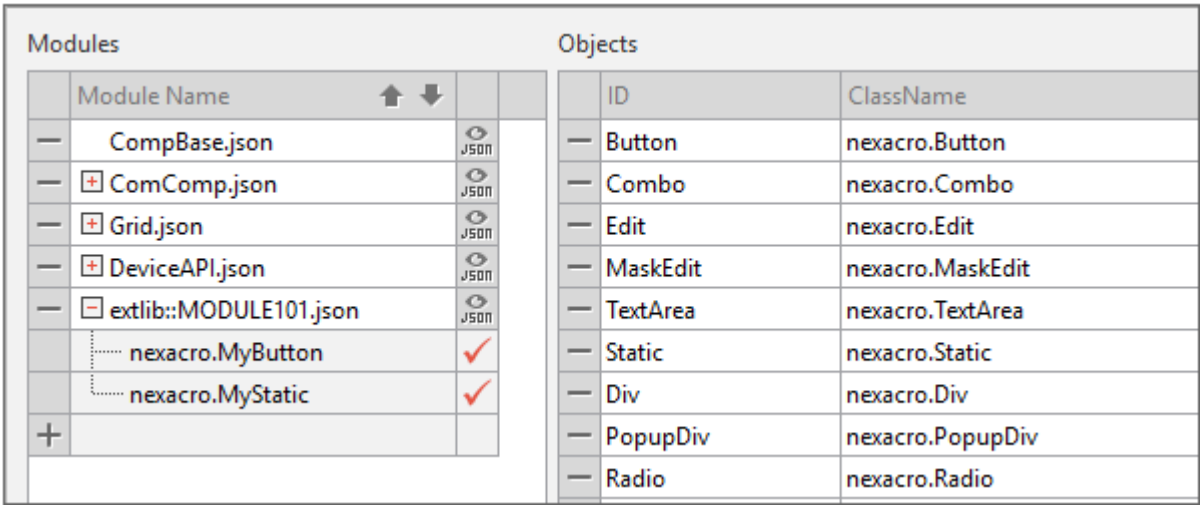
Look over the list of objects included in the module and then add necessary objects. You can later determine whether to use added objects by accessing TypeDefinition.



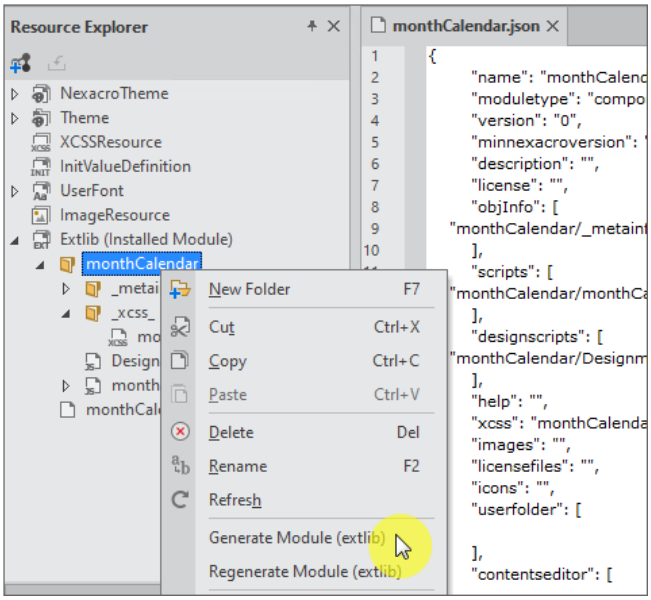
You must restart the project if you are to complete the module installation.



After the restart, you should check whether you can search the installed module in TypeDefinition.



The installed modules can be checked in the Extlib (Installed Module) section of the Resource Explorer. You can open the module source codes and, if necessary, edit the source code and generate only the selected module.

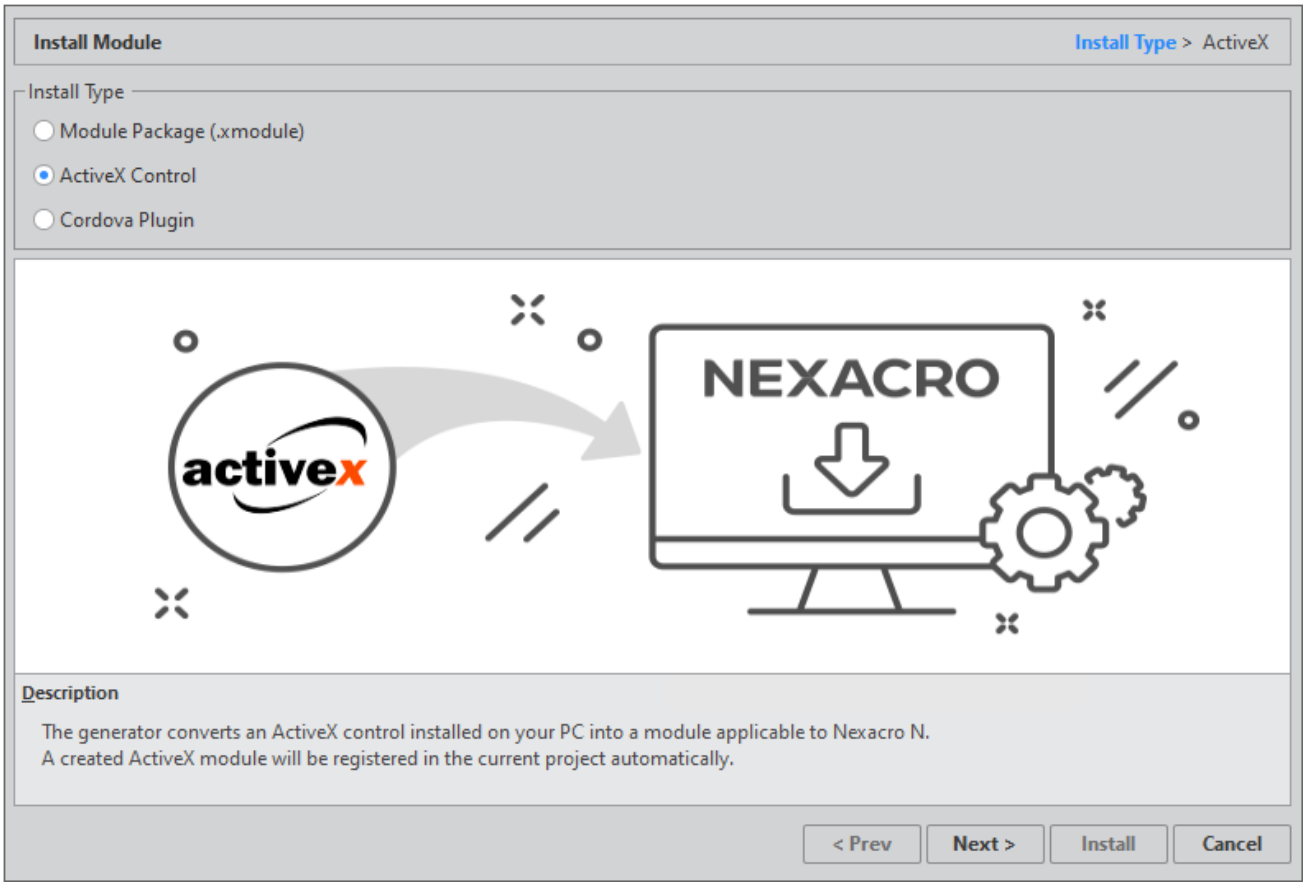


11.3.2 ActiveX Control Installation

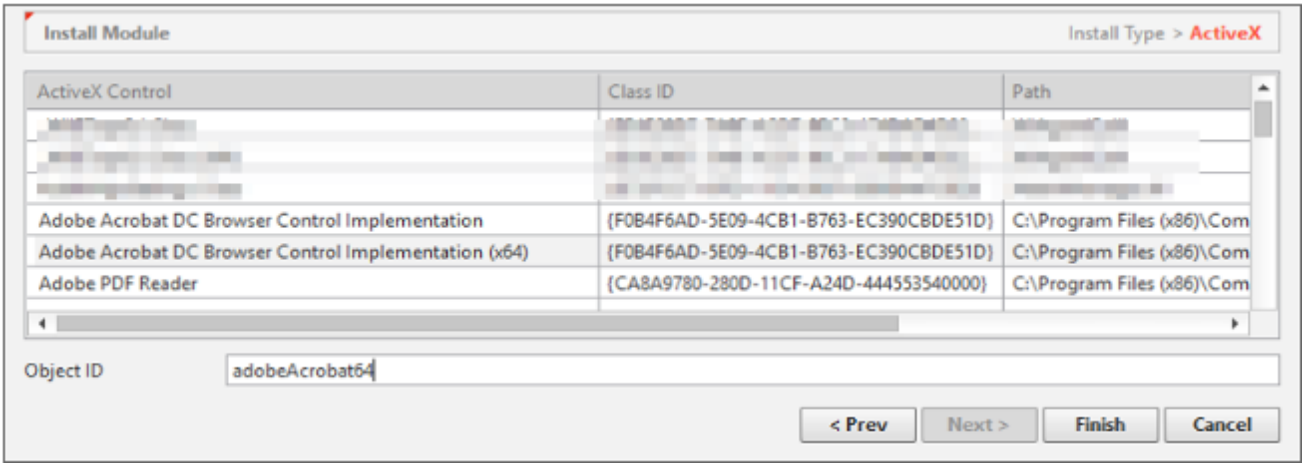
A Plugin component allows you to apply plug-in functions to your app. However, a Plugin component cannot supply the APIs used in an individual ActiveX control because it only provides common properties, methods and events.

> While not all browsers support an ActiveX control, the below browsers do so.
> **NRE (Windows)**

An ActiveX control can be installed like a module. You need to create a meta tag while you can see the properties, methods and events that are contained by a certain ActiveX control. To begin, access the menu [File > Install Module] and select "ActiveX Control" from Install Type.

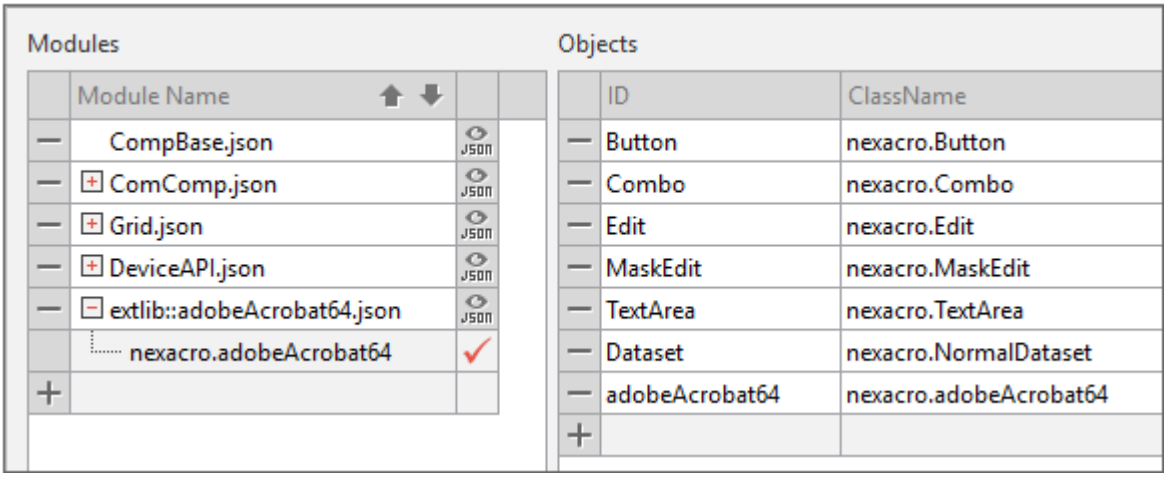


Select an ActiveX control you want to install and then enter the object ID, which will be registered as the ClassName of the component.

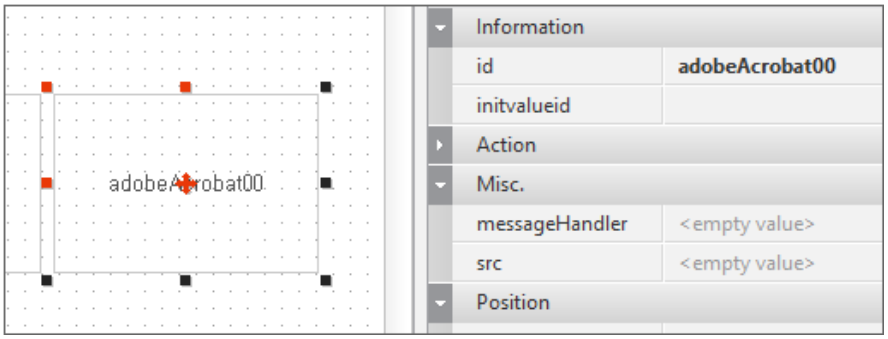


In the next step, Nexacro Studio produces the files required to install the ActiveX control in the form of a module. Those files will be located under the *[Project Path]*_*extlib*_\ path. Afterward, click the Register TypeDefinition button at the bottom of the wizard to register the ActiveX control as a module in TypeDefinition and then restart the project.

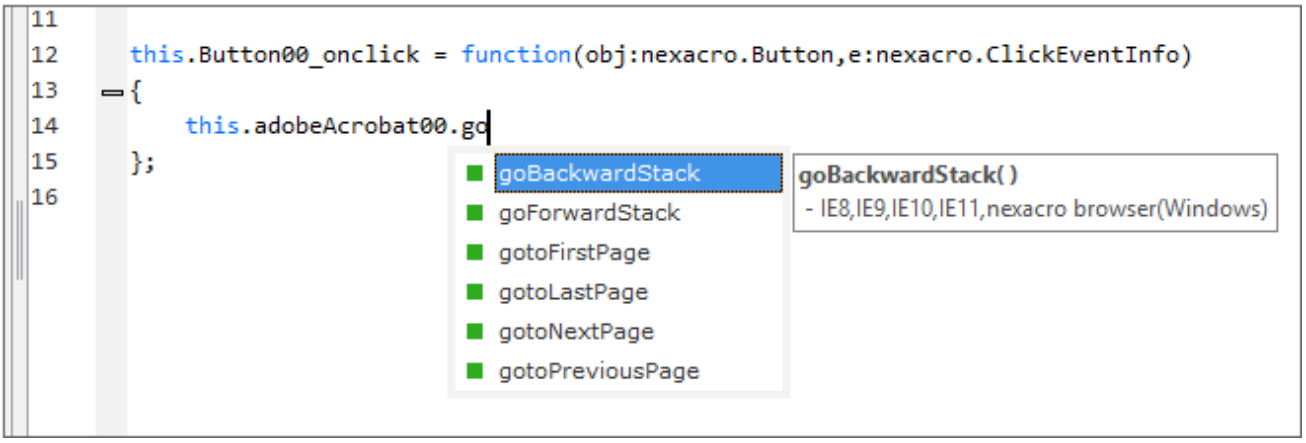
Access the menu [TypeDefinition > Objects] in the Project Explorer to see the object list and then you can see the ActiveX control installed as a module. You can put the created module on a Form like any other component.



When you place the module on a Form, you can see the properties and events provided by the ActiveX control through the Properties pane.



You can see the methods provided by the ActiveX control through the script editor.



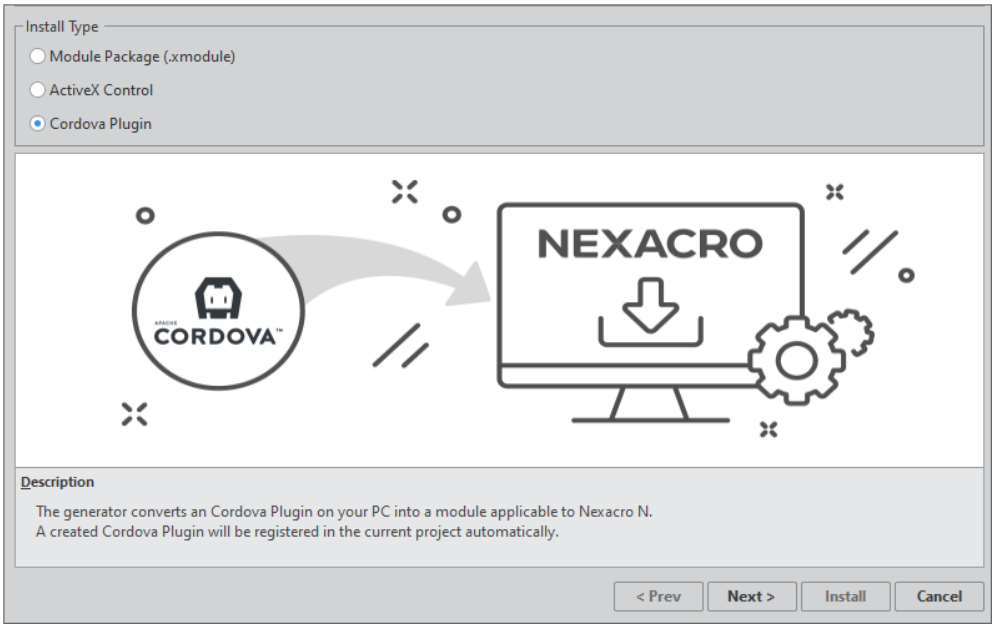
We recommend using the *getProperty* method when you want to obtain a property value of an ActiveX control through the script editor. If you directly access the property value, the return value may differ from the actual value.

```
this.adobeAcrobat00.gotoLastPage();
this.adobeAcrobat00.getProperty("src");
```

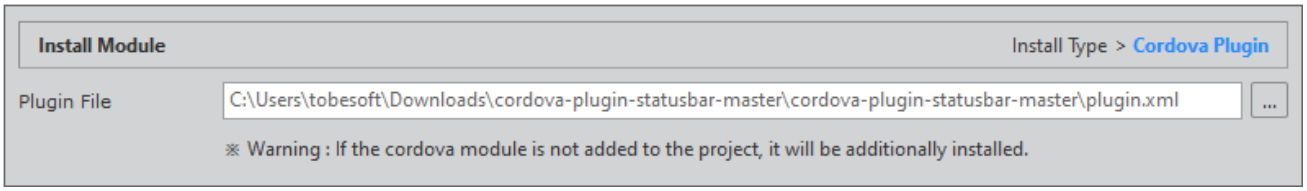
11.3.3 Installing the Cordova plugin

You can install the Cordova plugin and apply the plugin features that are available for Android and iOS apps.

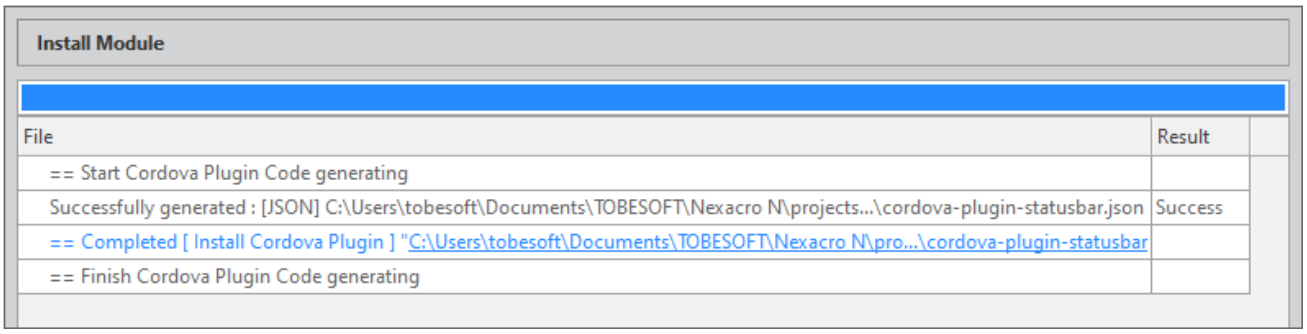
Select [File > Install Module] on the menu and select "Cordiva Plugin" from the Install Type field.



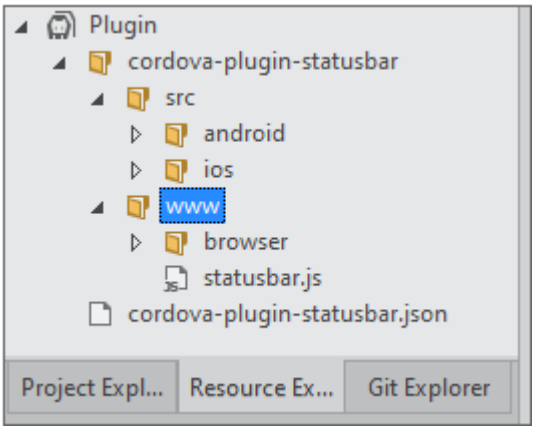
Select the plugin.xml file in the folder of plugins to be installed.



Check the message that the installation is complete.



The plugin-related sources will have been installed inside the _plugins_ folder, and you will be able toview the installed plugin sources in Resource Explorer.



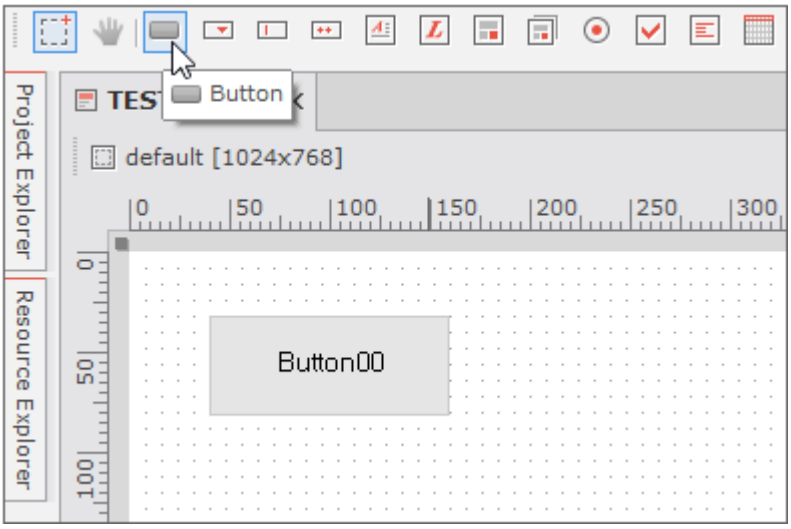
12.

Form Designing | Arranging, Selecting, Moving and Setting

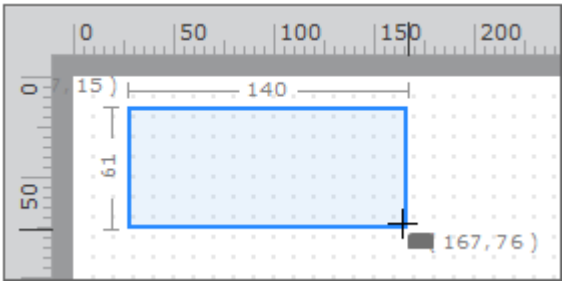
12.1 Component

12.1.1 Component Arrangement

In the Form Design editor, components can be added to the Form by selecting them on the Component bar.

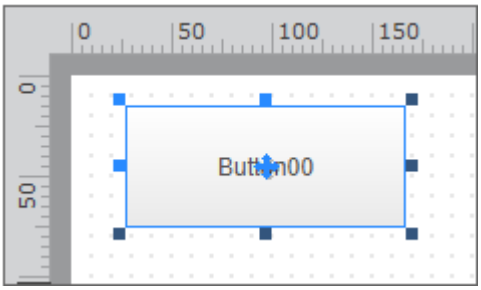


When you drag the mouse on the empty space of the Form Design window, a rectangular region will be selected. The starting and ending points, width and height of the rectangle, in pixels, will be displayed.

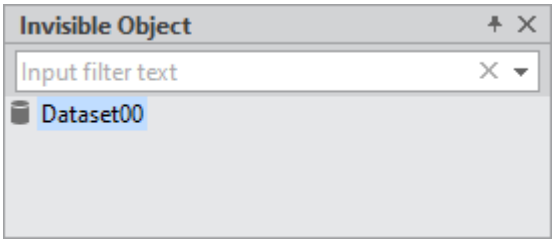


If a component has been selected from the Component Toolbox or toolbar, the component will be created in

the identified area. If you do not drag the mouse and just click on the screen, the component will created at the default size specified in the TypeDefinition.



The Form Design toolbox includes components that are not displayed to the user. These components may be displayed on the screen during app development so that component properties can be managed.



12.1.2 Selection Change

When designing a Form, you can select one or more component by using various key combinations.

Key combination	Description
`Tab`	You can select the component that comes next in the designated tab order. If the currently-selected object is a Form, the first component in the tab order will be selected.
`Shift` + `Tab`	You can select the component that comes one step earlier in the designated tab order. If the currently-selected object is a Form, the last component in the tab order will be selected.
`Ctrl` or `Shift`	You can select multiple components with `Ctrl` or `Shift` key pressed.
`Esc`	You can select the component that is one-level higher than the current component.
`Ctrl` + `a`	You can select all components except a Form.
`Alt`	When selecting a specific component by clicking the mouse while pressing the `Alt` key, all components of the same type are selected.

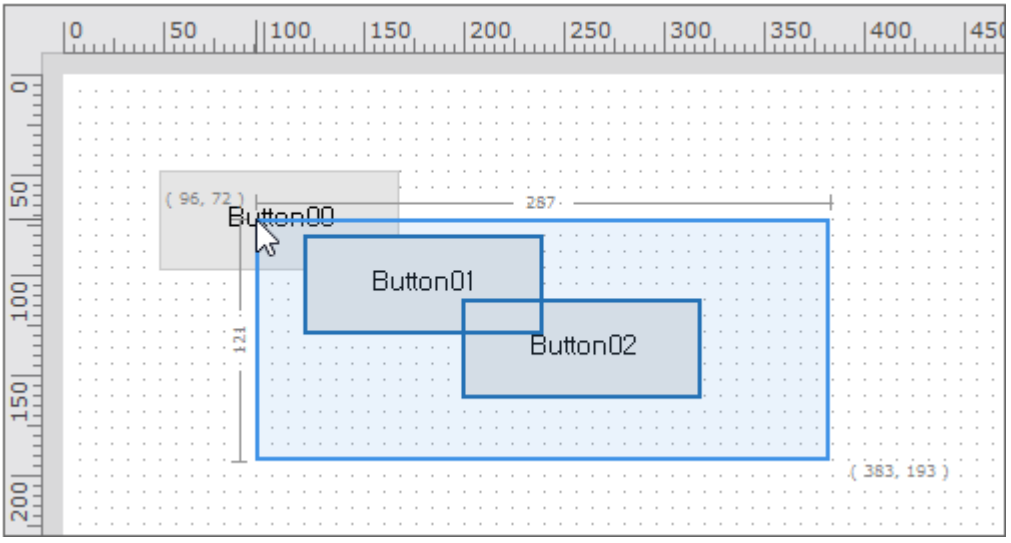


The tab order does not include a Form. Therefore, you cannot select a Form with the `Tab` key while a component is selected.

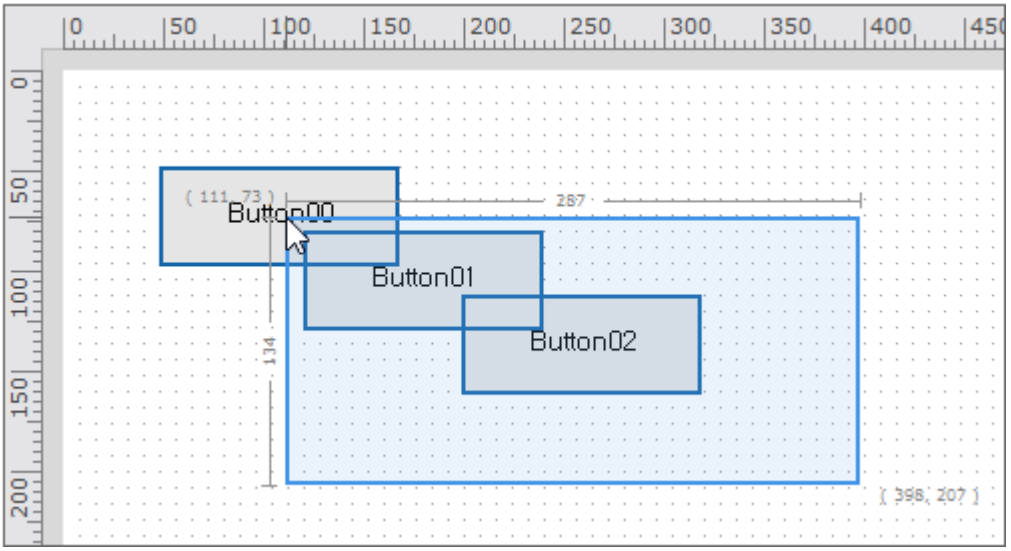
Select Type

The following instruction is how you change the way in which you drag a rectangle on the Form design panel to select components: adjust the setting in [**Tools** > **Options** > **Form Design** > **General** > **Select Type**]

- **Select All**: the option in which a rectangle must include the whole area of a component in order to select the component.



- **Select Part**: the option in which you can select a certain component even if a rectangle includes just a part of the component.

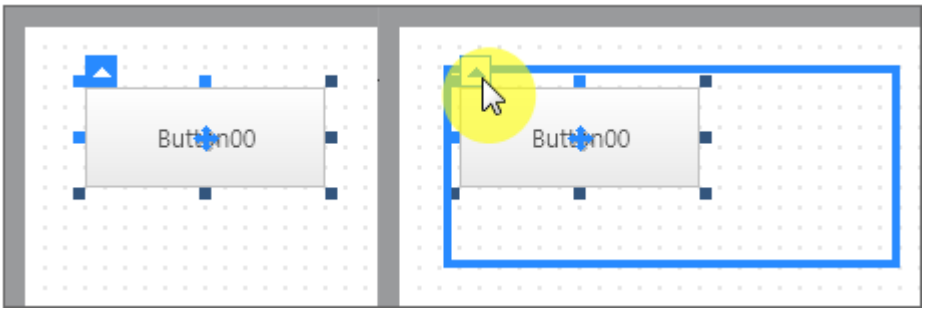


Select the parent component(Container Selector)

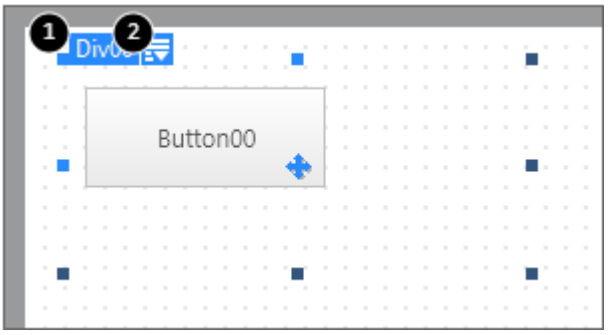
If there are components placed within a container component or Panel component, an additional UI (container selector) is provided for convenience in selecting the component.

When a placed component is selected, the "▲" button is displayed in the upper area of the component. Hover

the mouse pointer over the “▲” button and the parent component area is displayed with a blue border.



Click “▲” and it switches to the state where the parent component is selected. For Div, PopupDiv, View, and Panel components, the UI for moving the component or executing the context menu is provided.



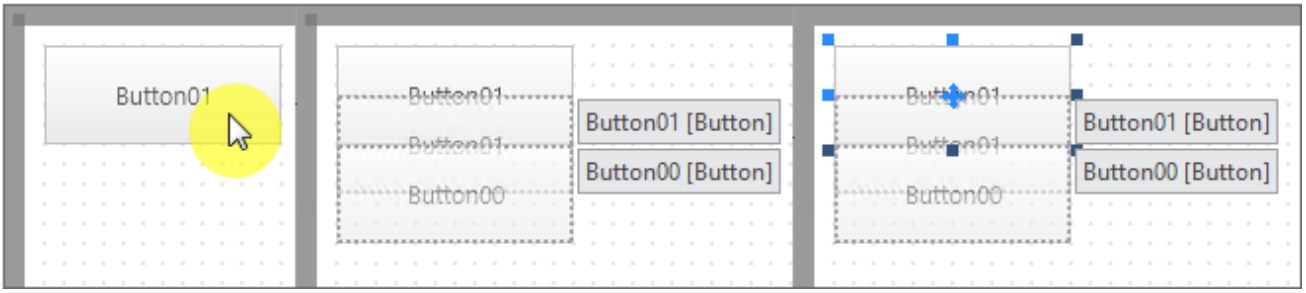
	Description
1	The component id property value is displayed. When you move the mouse pointer over the area, the component becomes movable and you can be drag and move it. If there are other parent container components to the container component, the "▲" button is displayed and the components are displayed next to it.
2	Run the context menu. Click the button with the mouse to run the context menu.



You can set the container selector UI to be displayed or not.
Options > Form Design > Guide > Container Selector

Select overlapping components

When components have the same size and position values, they are displayed overlapping. In the design screen, when the mouse pointer is moved over the overlapping components, the overlapping components are expanded, and you can select a component you want.



12.1.3 Move Component

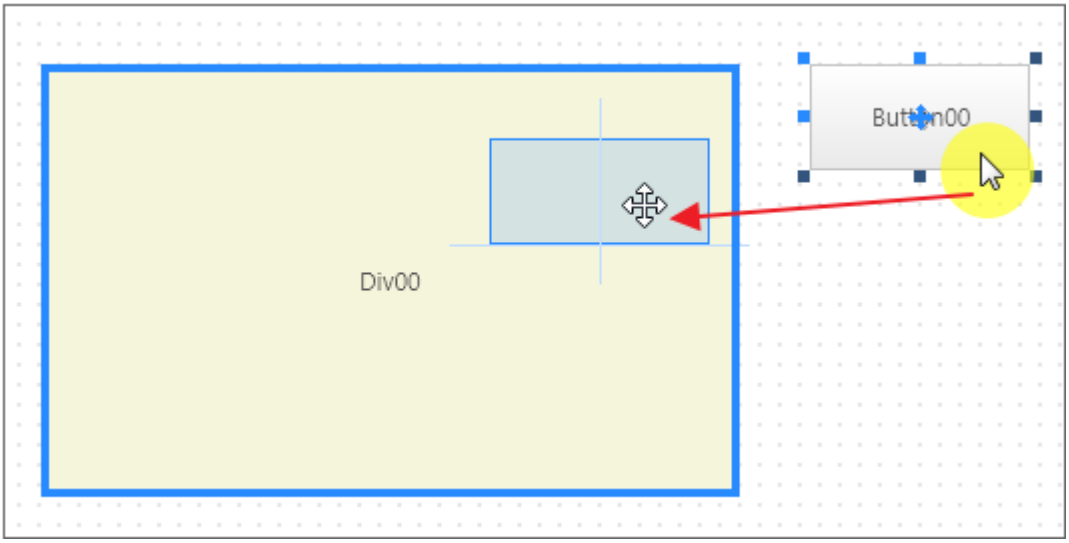
You can move a selected component to the place you want.

Use mouse

You can move a selected component through mouse drag and drop. While using a mouse, you can press a few keys to employ special features.

Combining key	Description
`Ctrl`	You can copy and move a component.
`Shift`	You can make only straight horizontal (leftward/rightward) or vertical (upward/downward) movement.
`Alt`	You can deactivate the Snap feature.

Drag and drop the selected component into the container component area and a blue border will appear and it will be added as a child of the container component.



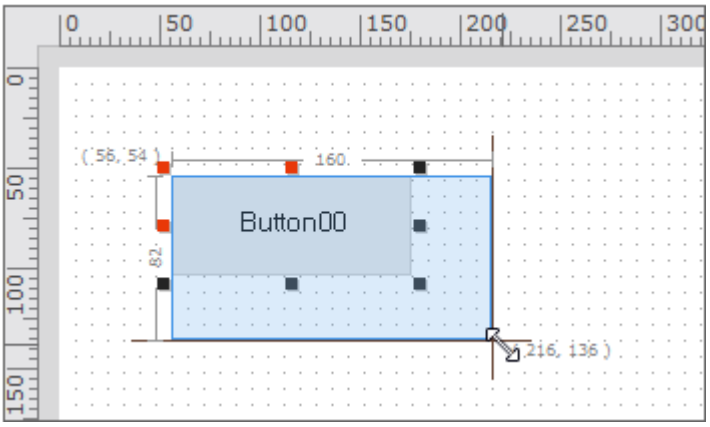
Use keyboard

You can move a selected component with arrow keys. Whenever you enter an arrow key, the component will move as far as you specified in the menu [Options > Form Design > Grid > Dot grid size]. If you enter an arrow key with the Ctrl key pressed, the component will move for the minimum length, which is 1px.

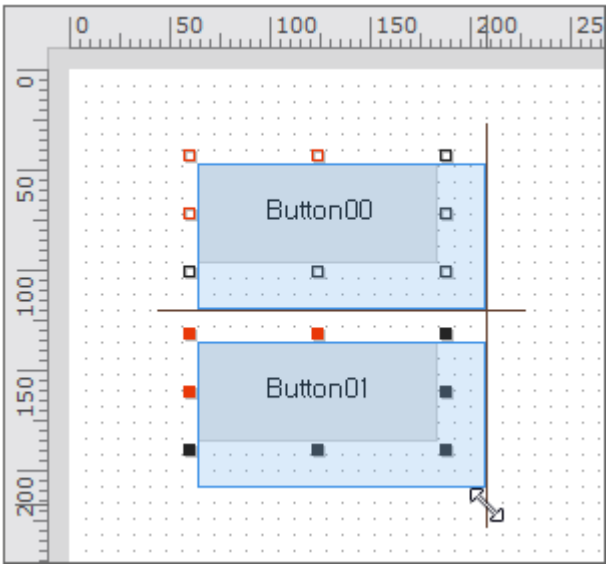
12.1.4 Size Change

When component tracker points are activated, dragging and dropping one point changes the size of the component.

If you adjust the size of just one component, Nexacro Studio will display the information on the size. In this case, the unit for measuring the size will follow the unit of the ruler.

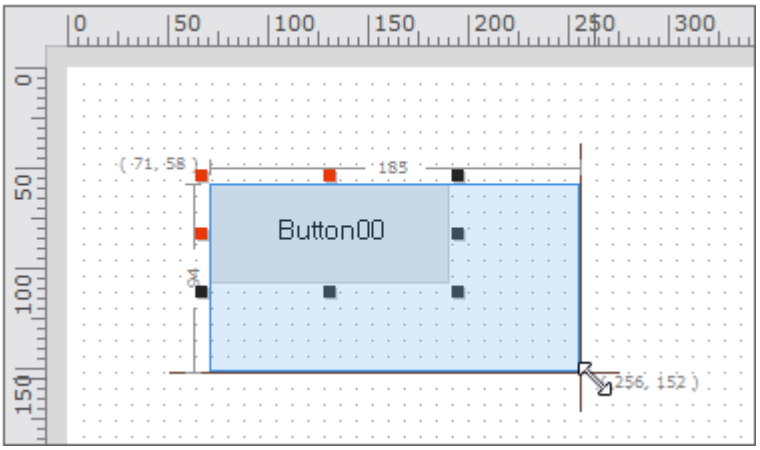


If several components are selected, all of the component sizes will change in the same proportion.

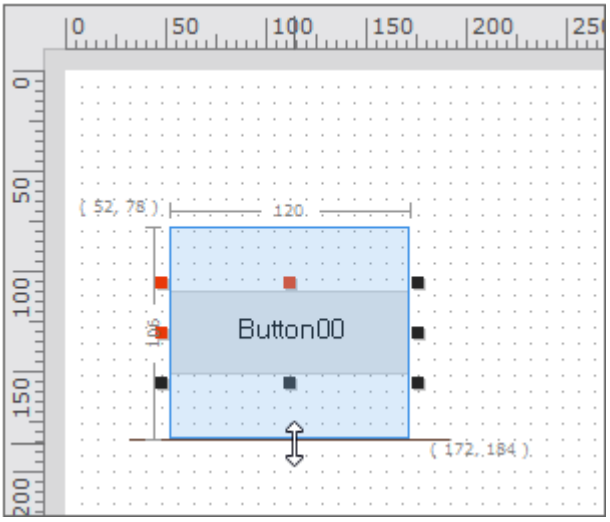


Pressing `Shift` while changing a component's size will modify the width and height in fixed proportion to

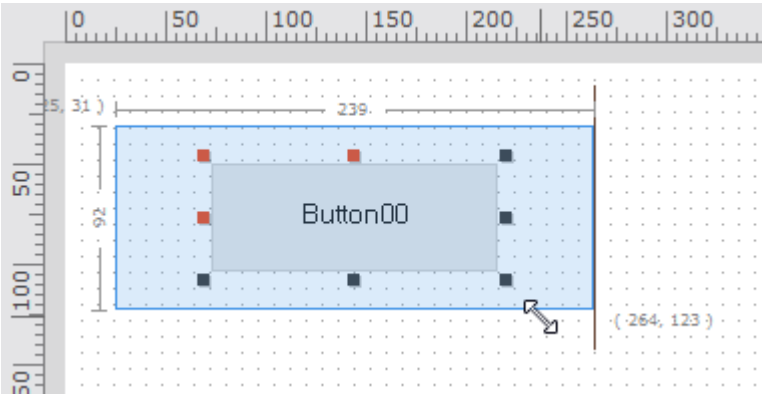
each other.



Pressing `Ctrl` while changing a component's size, will change the size equally on both sides of the middle of the component.



Pressing `Shift`+`Ctrl` while changing a component's size will increase both the width and the height in proportion to one another and equally on both sides of the center of the component.



You can also change the component size by pressing the direction keys while pressing `Shift` key. In this

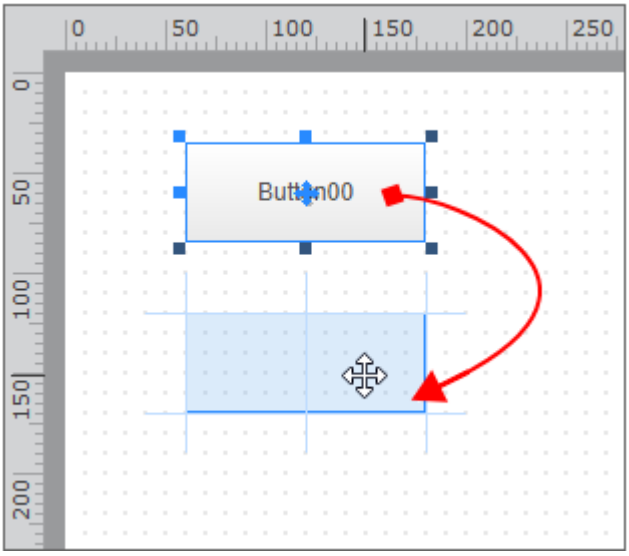
case, pressing a direction key once will change the size as much as the value of the **dot grid size** specified in the menu [Tools > Options > Form Design > Guide].

12.1.5 Copy Component

The selected component can be copied and pasted to a desired location.

Mouse

When a component is selected and dragged with the mouse while holding down the Ctrl key, the component is copied to the dragged location.



Keyboard

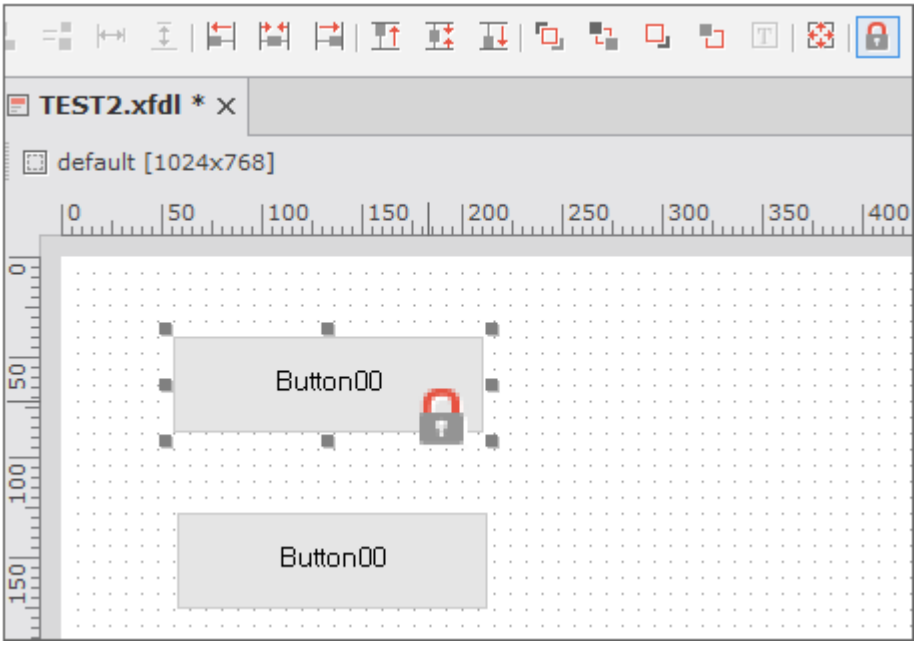
When a component is selected and copied (Ctrl+C) and pasted (Ctrl+V), the component is copied.

When a component is selected and copied, then pasted while another component is selected, the behavior varies as follows depending on the Layout type property.

- default
  The component is pasted at the next z-order after the selected component.
- horizontal □□ vertical
  The component is pasted at the next z-order (or PanelItem) after the selected component.
  The pasted component has the same layoutorder property value as the selected component.
- table
  The component is pasted into the same tablecellarea area as the selected component.
  The pasted component has the same tablecellarea property value as the selected component.

12.1.6 Lock Control

You can prevent components from being moved or resized by setting Lock Components. If you want to set or lift Lock Components, access [**DESIGN** > **Lock** > **Lock Components**] in the ribbon interface or [**Design** > **Lock Components**] in the command bar interface. Otherwise, open the context menu from the Form design panel and select [**Lock Components**].



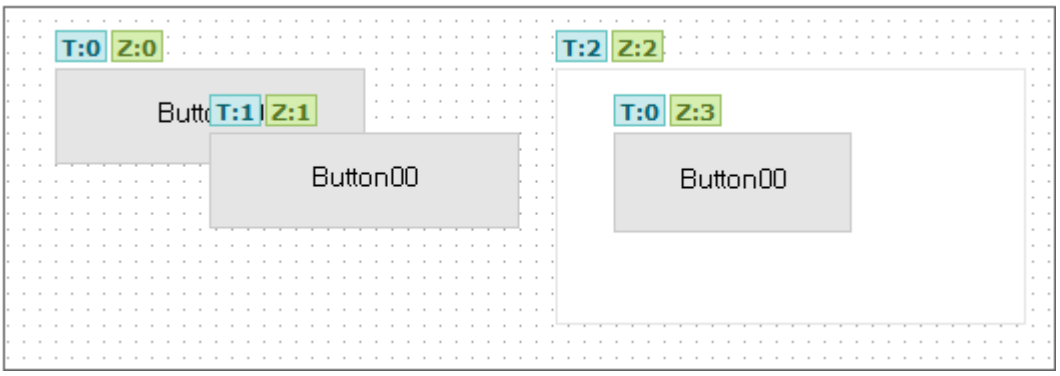
The Lock Components feature only can prevent the moves by a mouse drag. The feature will not work for the moves by arrow keys or by changing position properties.

12.1.7 Tab Order

The tab order refers to the sequence in which you make moves among components by pressing the tab key in the middle of running an app.

Tab Order(Z-Order) View

The Tab Order view is used for checking the tab order defined on the current screen. You can toggle the function by selecting the menu [Design > State View > Tab Order]. When the function is activated, you can see two numbers on the top left of the component, as shown in the picture below.

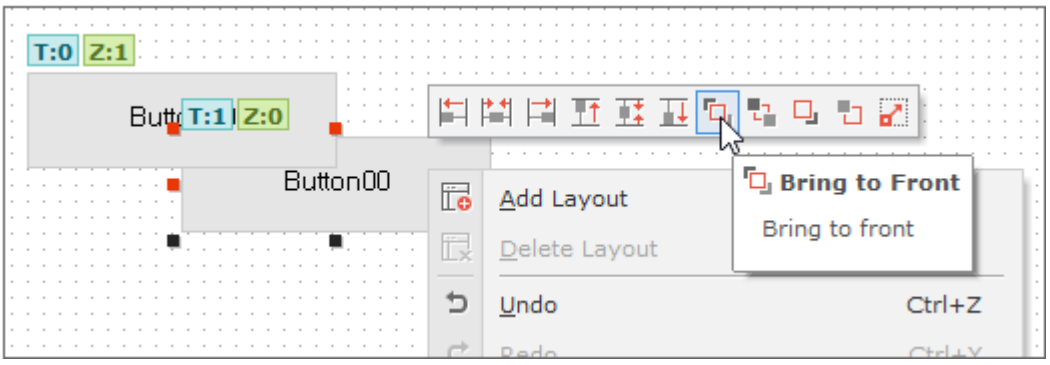


The numbers in the blue boxes indicate the order of tabs. Change the order by revising the tab order property of components or by using the **Tab Order Editor** function.

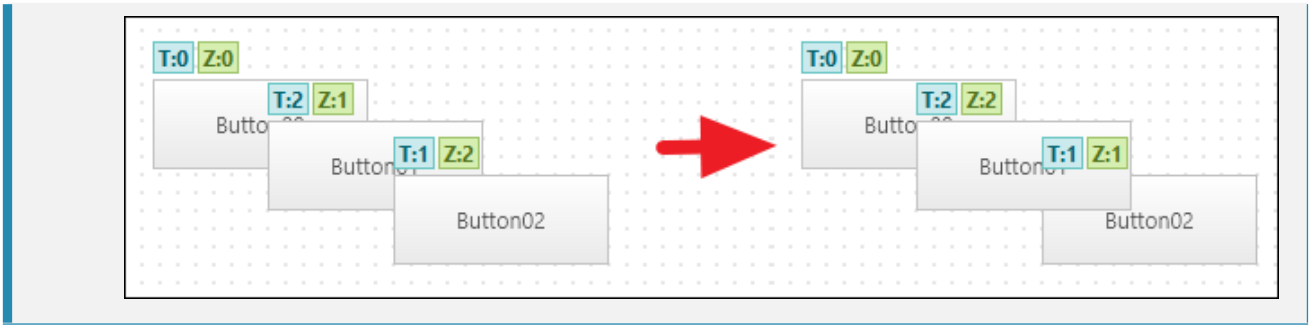
The numbers in the green boxes indicate the Z-Order of components. Z-Order defines the display order of components when the components are overlapped. A component that has a bigger number is moves to the front.

Change the Z-Order by using one of the four functions on Align toolbar, or by changing the order of arranged XML code on Source tab:

- Bring to Front
- Send to Back
- Bring Forward
- Send Backward



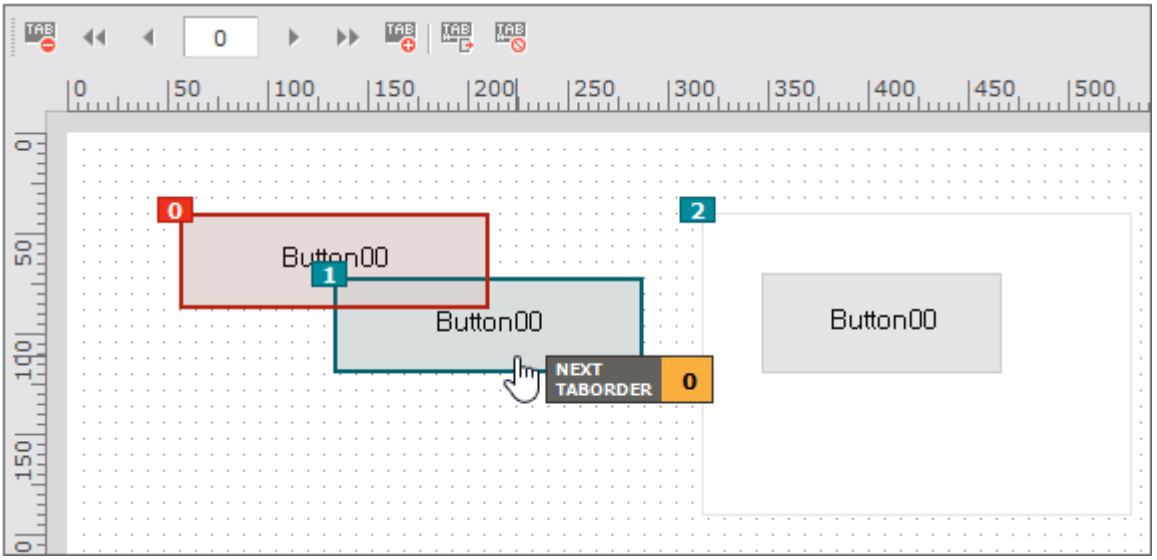
If you want to change the Z-Order according to the tab order after changing the tab order in the Tab Order Editor, you can change the Z-Order by selecting the menu [Design > Arrange to Tab Order].



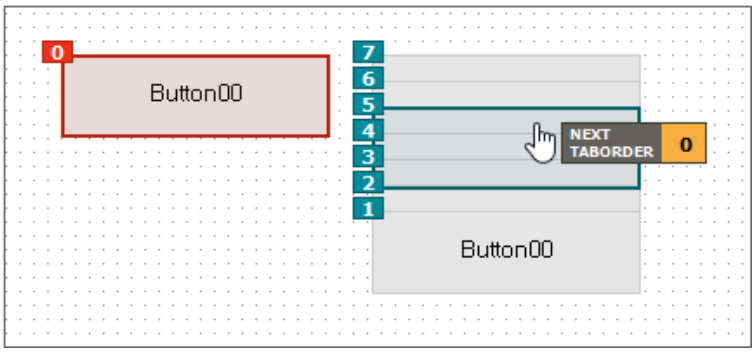
Tab Order Editor (View Type)

You can control the tab order for components by pressing the `Ctrl` + `T` key in the Design pane. Otherwise, you should access the menu [Design > Tab Order Editor(View Type)].

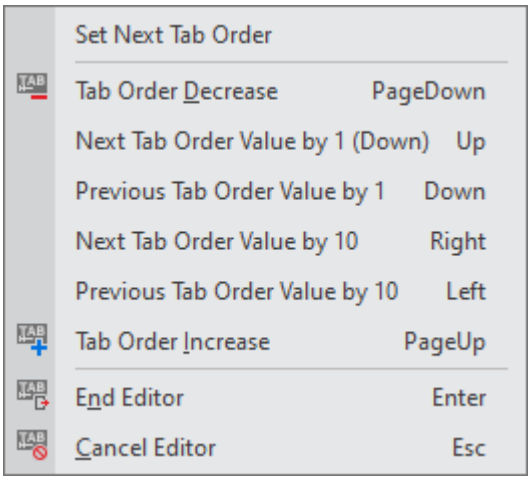
The mouse cursor is labelled with an index number that indicates a next tab order to be set. If you click a certain component, the foregoing tab order will be given to the component, with the tab orders of other components adjusted automatically.

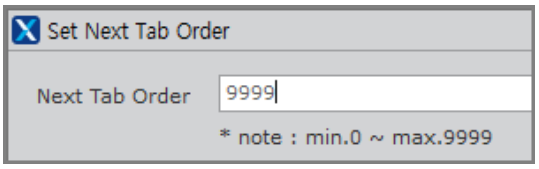


When you move a mouse cursor over a certain component, its color will change to green while the component of the next tab order will be highlighted in orange. If multiple components overlap, those components will be spread out for easy editing.



The Tab Order Edit mode supports the below context menu.

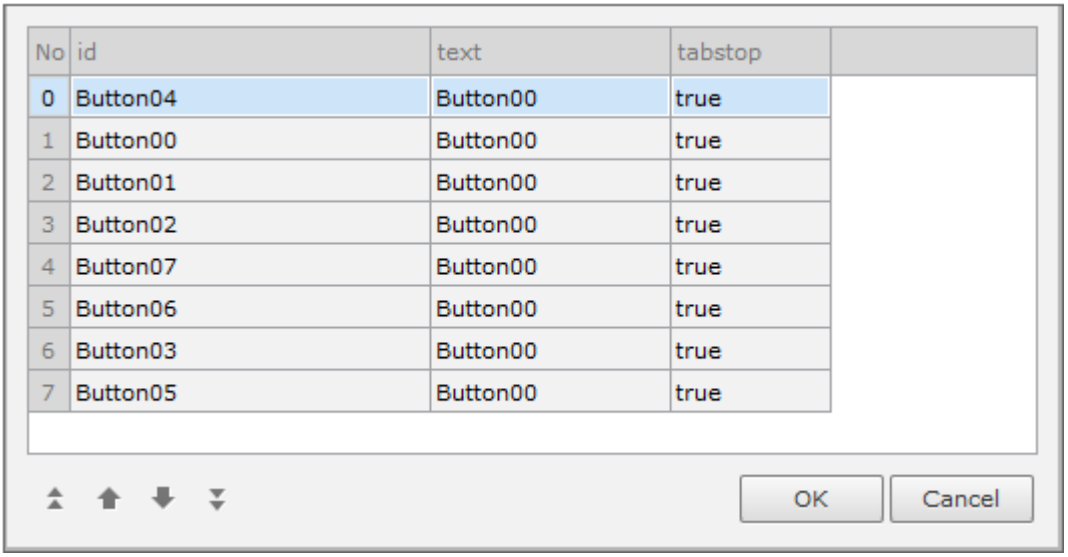


Menu	Shortcut	Description
Set Next Tab Order		Specifies the value of Next Tab Order 
Tab Order Decrease	`PageDown`	Minimizes the gap between current tab orders. Adjusts the value of the first tab order to 0 and arranges the others in consecutive sequence.
Next Tab Order Value by 1	`Up`	Increases the Next Tab Order value by 1
Previous Tab Order Value by 1	`Down`	Decreases the Next Tab Order value by 1. The minimum value is 0
Next Tab Order Value by 10	`Right`	Increases the Next Tab Order value by 10
Previous Tab Order Value by 10	`Left`	Decreases the Next Tab Order value by 10. If the current Next Tab Order value is 10 or smaller, the value will become 0.
Tab Order Increase	`PageUp`	Increases all the tab order values by 10 times. This function is useful when you intend to insert components between tab orders. For instance, assume that there are two components whose tab order values are 3 and 4, respectively. You can insert other components between them by multiplying their values to 30 and 40—in this case, you can insert up to 9 components ranging from 31 to 39.
End Edit	`Enter`	Ends the Tab Order Edit mode after saving changes
Cancel Edit	`Esc`	Ends the Tab Order Edit mode without saving changes

Tab Order Editor (List Type)

The Tab Order List window presents tab order in the form of an editable. You can change tab order by selecting one or more items and then pressing one of the below four buttons.

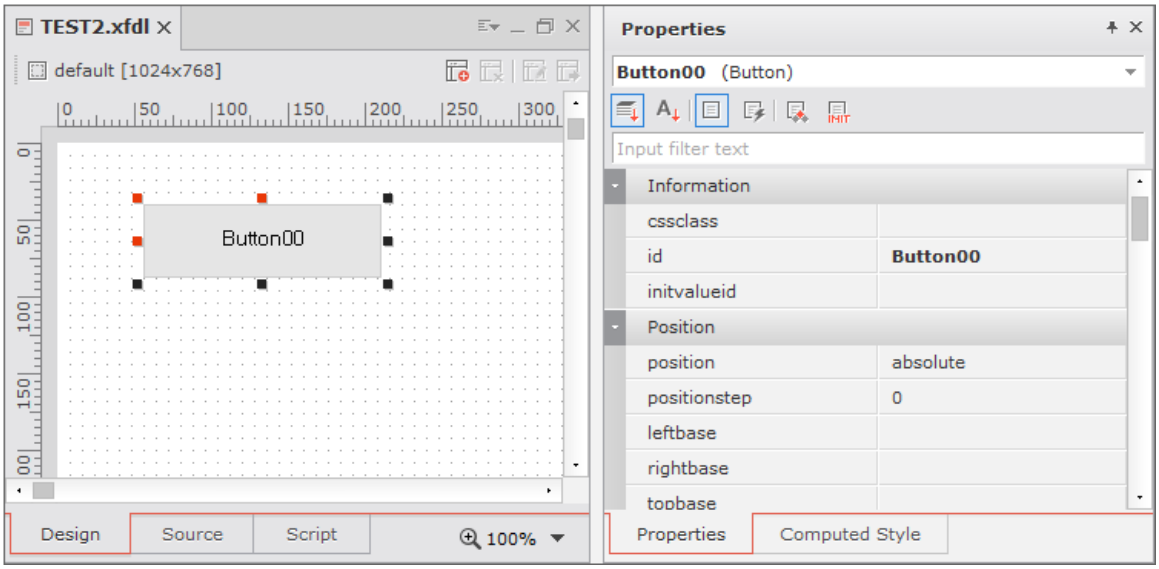
You can open the Tab Order List window by accessing the menu [Design > Tab Order Editor (List Type)].



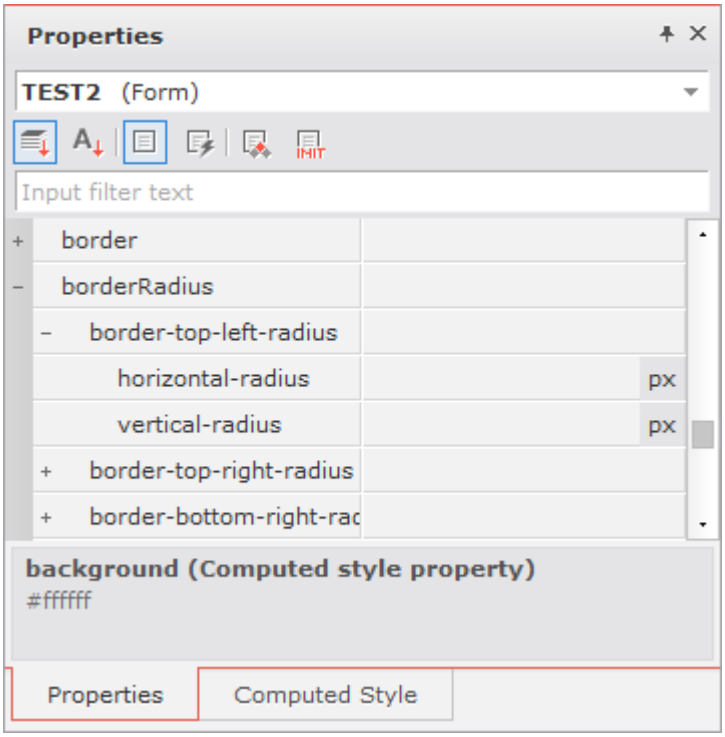
You cannot edit the other information items (i.e. id, text, tabstop) in the list than tab order.

12.1.8 Set Properties

The properties for a selected component are displayed in the Properties Window. When you modify the property values, they are immediately applied to the component.

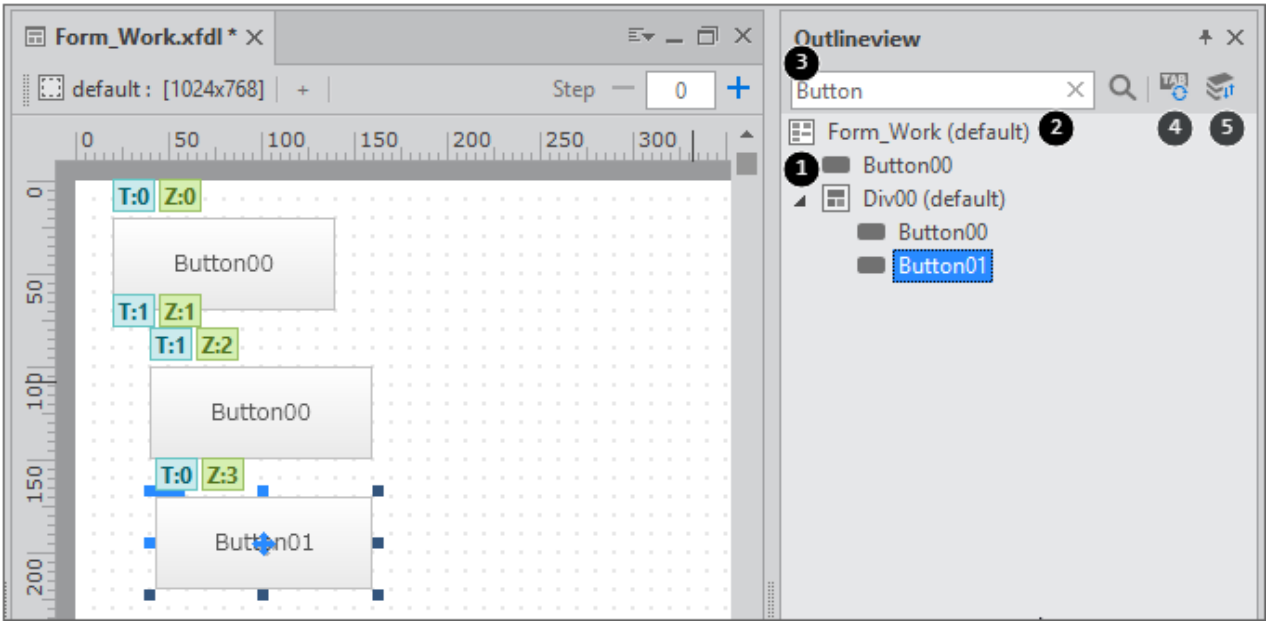


When a property has subordinate properties as shown in the figure below, an icon that can expand the properties will be shown in front of the property. Properties are automatically synchronized with their subordinate properties and vice versa.



12.1.9 Outlineview

Displays the components included in the Form object in a tree structure. The Outlineview panel can be activated by selecting the menu [View > Outlineview].



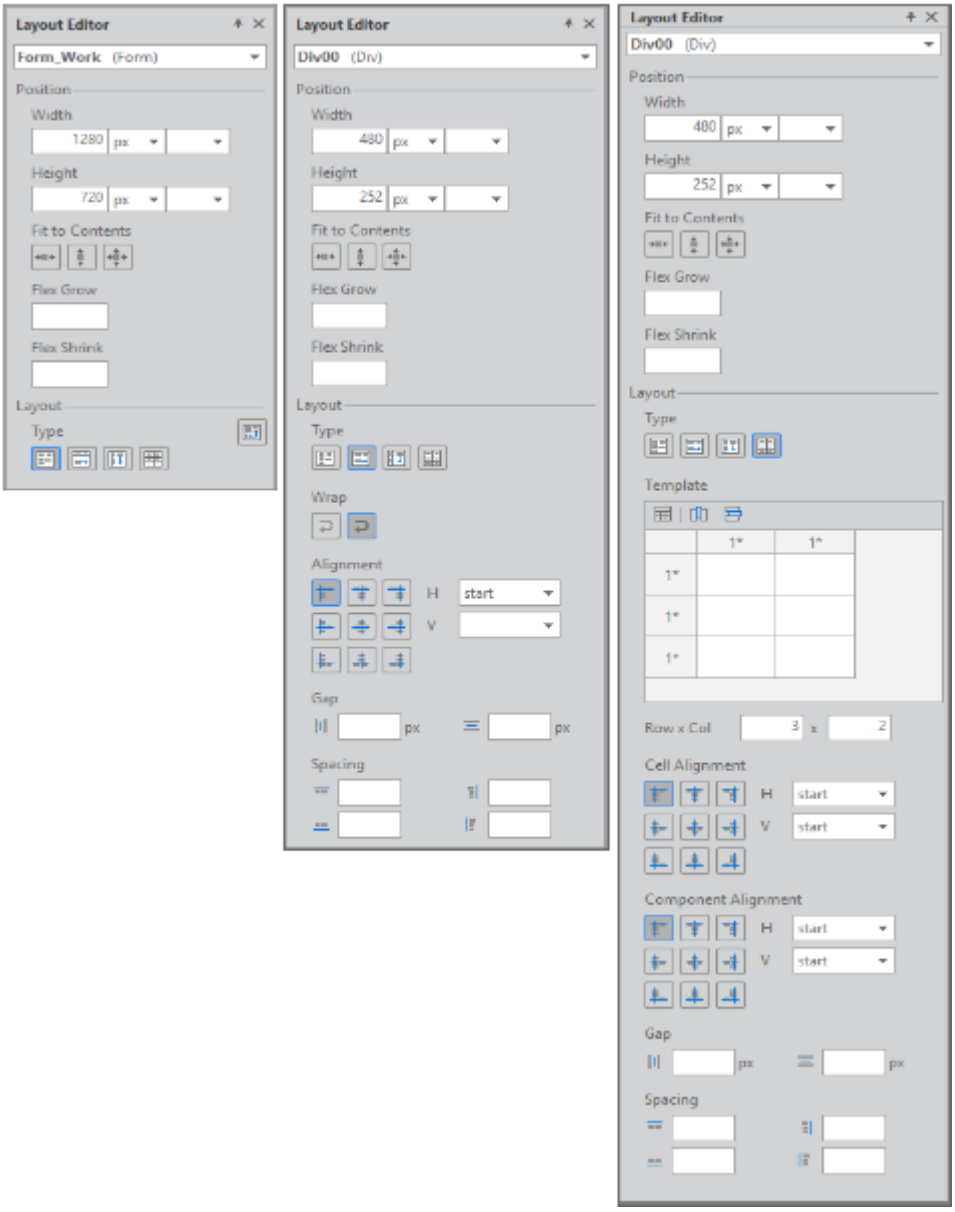
	Description
1	Displaying the list of components placed in the Form object The component list is displayed in order according to the following standards. ● The tablecellarea property of the component if the type property value is “table” ● The layoutorder property of the component if the type property value is “horizontal” or “vertical” ● Z-order of the components
2	Form name (type property) When a different layout is selected, the type property of the selected layout is displayed.
3	Search according to the component id
4	Apply outline to Tab Order Updates the taborder property values of components in the order displayed in the Outlineview panel.
5	Arrange to Tab Order This is the execution icon for the menu [Design > Arrange to Tab Order] function.

You can select a component in the Outlineview panel and use the following features. The available functions vary depending on the selected screen tab. If a layout is added in the Design tab, some functions are not supported in the additional layout.

	Design	
	Default layout	Additional layout
Display component list	O	O
Search by component id	O	O
Expand or collapse the container component tree list	O	O
Display the selected component in the Properties window	O	O
Supports the Copy Component ID menu in the context menu	O	O
Displays the selected component as selected in the Design window	O	O
Change the component order by drag and drop	O	X
Copy the component by Ctrl + drag and drop	O	X
Supports the same functions as the Design window. - Context menu - Default action on double-click	O	O
Apply outline to Tab Order	O	O
Arrange to Tab Order	O	X

12.1.10 Layout Editor

This is an editing window where the Position and Layout properties can be intuitively configured. The items that can be configured depend on the selected object or component.



Configuration Items	property	Form Object	Container Component	Pane Compo
Position				
Width	width	O	O	O
Height	height	O	O	O
Fit to Contents	fittocontents	▲	O	O
Flex Grow	flexgrow	▲	O	O
Flex Shrink	flexshrink	▲	O	O
Layout				
Type	type	O	O	O
type = "default"				
Auto Layout	-	O	O	X

Configuration Items	property	Form Object	Container Component	Pane Compo
type = "horizontal" | "vertical" | "table"				
Gap	horizontalgap verticalgap	O	O	O
Spacing	spacing	O	O	O
type = "horizontal" | "vertical"				
Wrap	flexwrap	O	O	O
Alignment	flexcrossaxisalign flexcrossaxiswrapalign	O	O	O
type = "table"				
Template	tabletemplate	O	O	O
Row x Col		O	O	O
Cell Alignment	tablecellalign	O	O	O
Component Alignment	tablecellincompalign	O	O	O



If you select the Form, the items marked with ▲ will be displayed in the Layout Editor, but cannot have values set since they are properties not supported by the Form object.

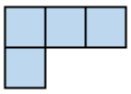
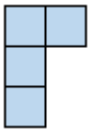
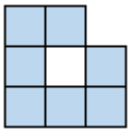
Auto Layout

In cases like the one below, the type attribute value and related attribute values are automatically set through Auto Layout processing.

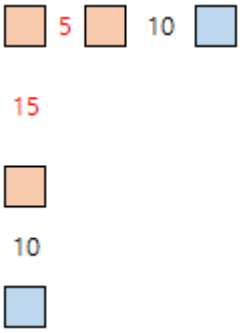
- When selecting the Auto Layout icon in Layout Editor,
  The Auto Layout icon will only be displayed under the following conditions.
  - When the type attribute value of the Form object or container component is "default"
  - When the Form object or container component has one or more child components
- When selecting one or more components and executing the [Group by Panel] menu,

Depending on the component layout type, the values of the type and flexwrap attributes are set according to the priority criteria below.

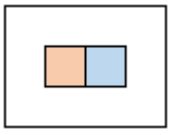
Component Layout Types			type
1	When only one component is placed		"horizontal"
2	When two or more components are arranged vertically		"vertical"
3	When two or more components are arranged horizontally		"horizontal"
4	When two or more components are arranged to be displayed with horizontal line		"horizontal"

Component Layout Types			type
	breaks		
5	When two or more components are arranged to be displayed with vertical line breaks		"vertical"
6	If it cannot be expressed in "horizontal" or "vertical" format		"table"

The values for the horizontalgap and verticalgap properties are determined based on the spacing between the first and second components in each direction.

Component Layout Types		
- Horizontal Direction The space between the first component and the second component is 5. - Vertical directionThe gap between the first component and the second component is 15.		5

The spacing property value is applied only when the type property value of the container component is changed. The value of the spacing property is determined based on the top and left attribute values of the first component.

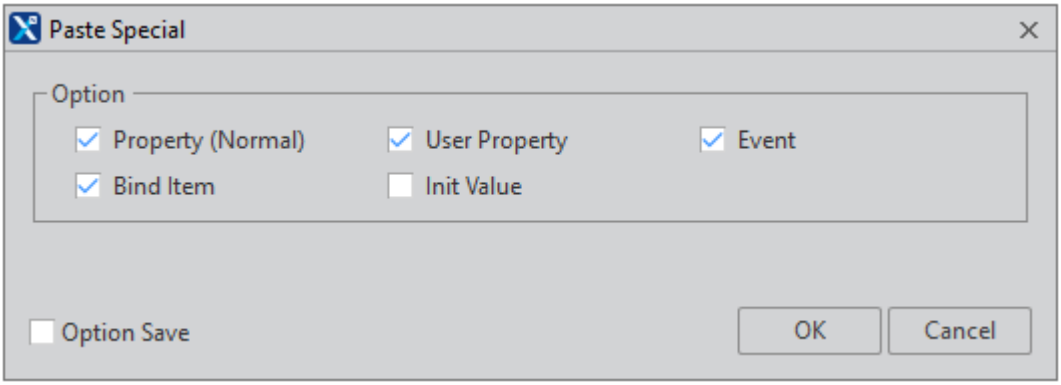
Component Layout Types		spacing
When the child components are arranged within the Div component and the top and left property values of the first component are "5px"		"5px 0px 0px 5px"

12.1.11 Other Features

Paste Special

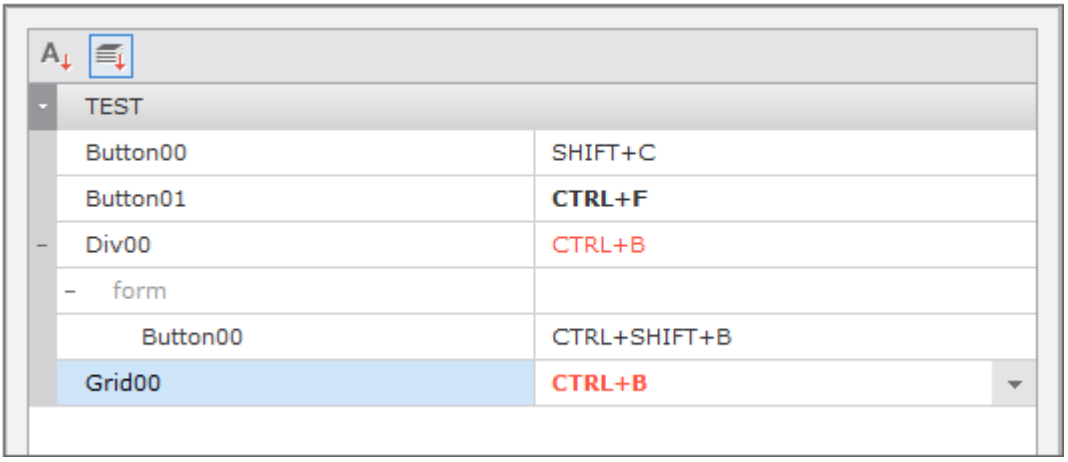
Paste Special is a feature used to copy and paste a component while applying just particular attributes of the component—such as properties, events and bound items—selectively. You can use This function by accessing the menu [Edit > Paste Special].

The component is pasted with only the selected Option information applied. If the "Option Save" item is checked before clicking the [OK] button, the currently selected Option information is reflected in [Option > Form Design > Paste Special].



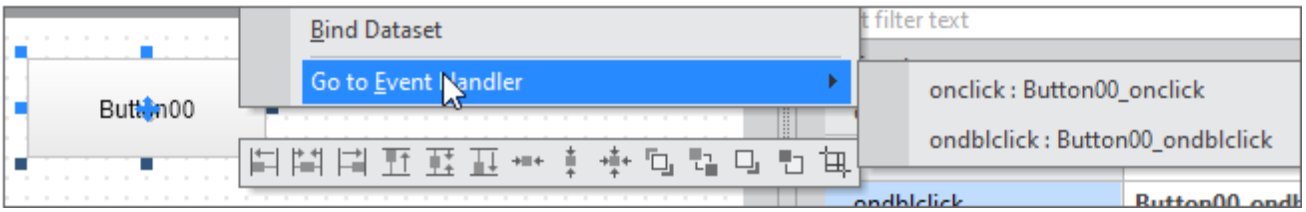
Hotkey editor

This function allows you to set hotkeys to apply to the components in a Form. To this end, you need to access the menu [Design > Hotkey Editor]. Basically, the editor presents hotkeys that have been already set. If certain components do not have hotkeys, their editable fields are empty. If you change an existing hotkey, the changed one will be displayed in bold type. If more than one component have the same key combination, the relevant hotkeys will be displayed in red.



Go to Event Handler

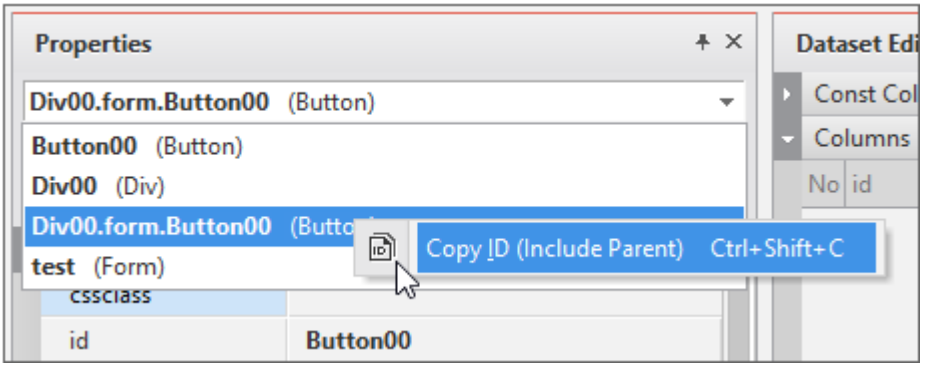
If a selected component holds an event handler, this function will bring you to the codes of the handler in the script. Right-click a component to open the context menu, select [Go to Event Handler] from the menu, and then choose an event handler you want to see.



If a selected component does not hold any event handler, [Go to Event Handler] will not appear on the context menu.

Copy ID

This function copies the ID of a selected component onto the clipboard. For a component that belongs to a Div or Tab component, this function copies the value that includes the IDs of the parent components. You can use this function in three ways: by accessing the menu [Edit > Copy ID (Include Parent)]; by right-clicking a certain component on the Form design pane to open the context menu; and by opening the component list situated at the top of the Properties pane and right-clicking a certain component from the list to open the context menu.



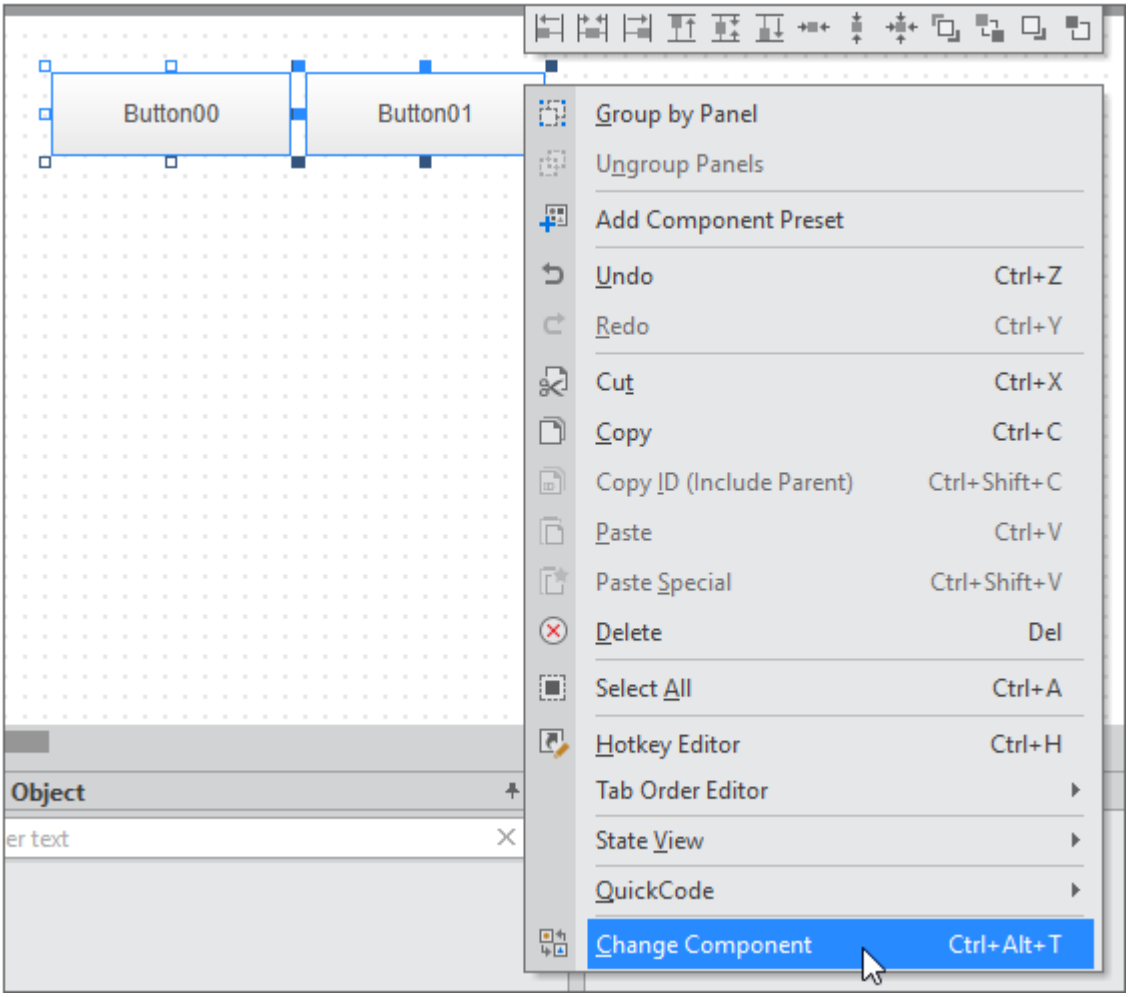
In the case illustrated in the above picture, the value copied onto the clipboard is as below.

```
Div00.form.Button00
```

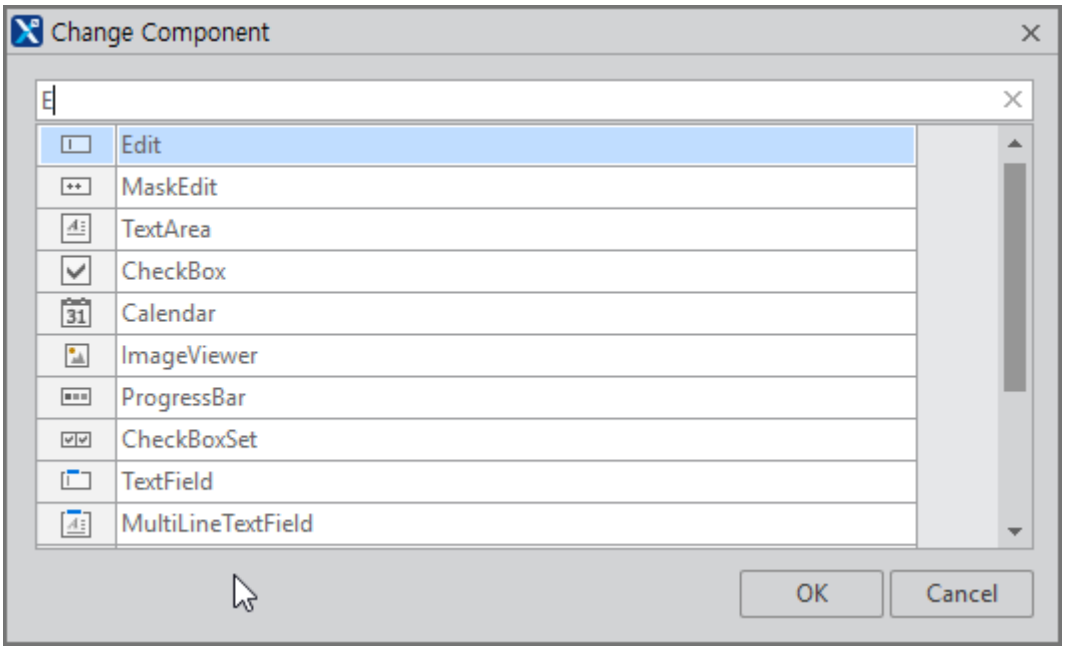
Change Component

The component can be changed to another component while retaining the basic properties, including position properties, displayed on the screen. The component is changed according to its purpose without deleting and re-adding it. Multiple components of the same type can also be selected and changed at once.

Select the component, then choose [Change Component] from the context menu. Select the component to change to in the popup window.



You can select a component from the list or enter the component name in the search field and select it from the results.



Please note the following when changing a component.

- When a component is changed, properties with the same name are retained and all other properties are removed.
- All event settings of the selected component are removed. However, event handler functions created in scripts are not removed.
- All Bind Item settings of the selected component are removed.
- The id property value of the selected component is not changed.

Among UI components, the following components do not support the Change Component feature. These components are excluded because they are not suitable for component-to-component changes.

- GoogleMap
- Grid
- ListView
- Menu
- PopupMenu
- Tab

Container components support changes only between container components. Child components of a Div component are changed to PanelItem when changed to a Panel component.

- Div
- PopupDiv
- View

12.2 Add and Use Component Preset

Component presets allow you to manage and reuse components whose properties and events are set repeatedly.

Panel components cannot be registered as component presets.

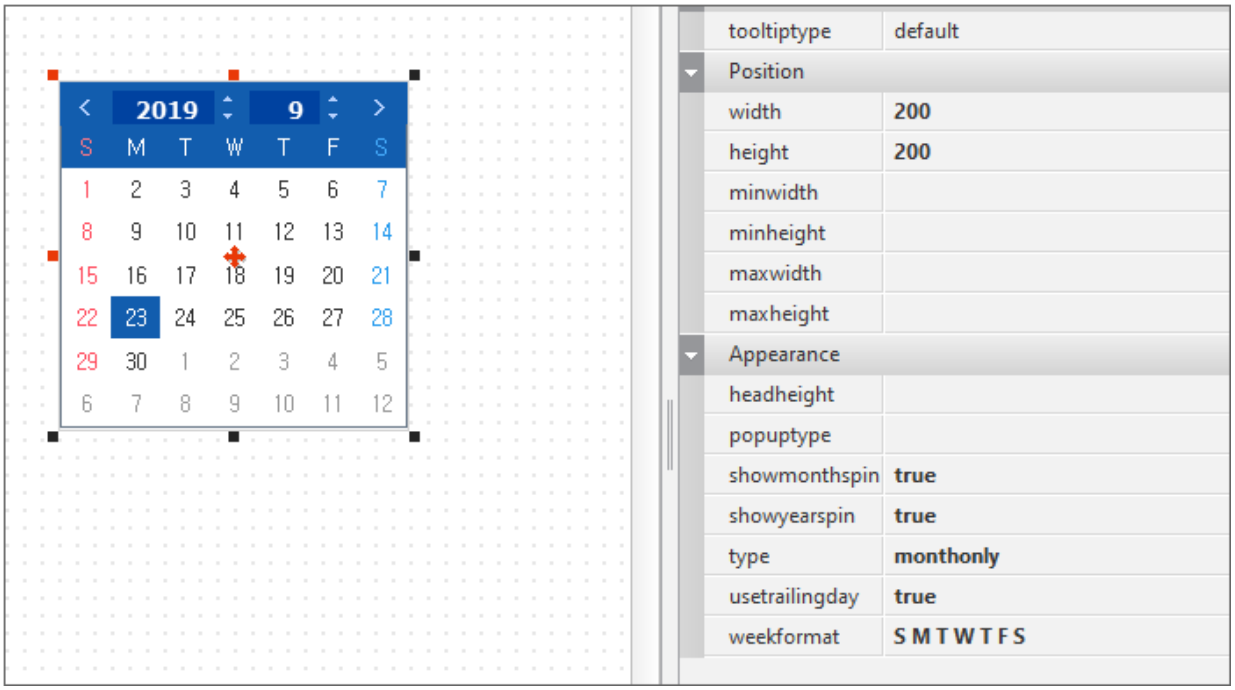
12.2.1 Add Preset

You can add a preset by following the below steps.

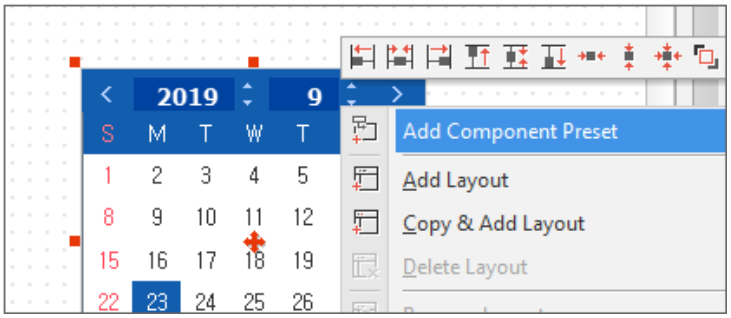
1. Enter any Form and set the properties of a component that you want to add as a preset.

   In this demonstration, a Calendar is placed on the Form with its properties set as below.

```
width: 200
height: 200
showmonthspin: true
showyearspin: true
type: monthonly
usetrailingday: true
weekformat: S M T W T F S
```

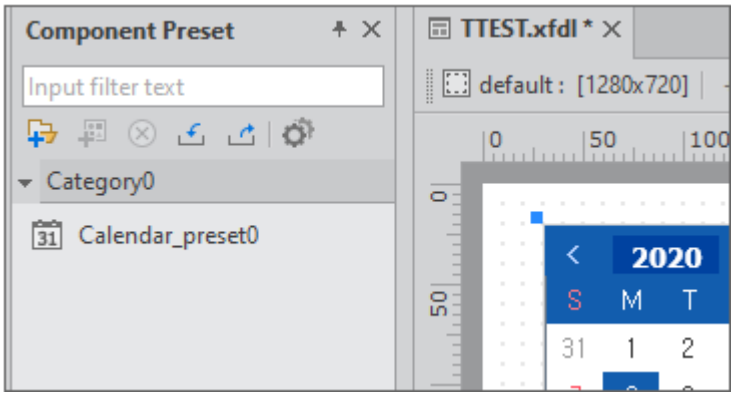


2. Right-click the component and select [Add Component preset] from the context menu.



If the Component Preset pane is already activated, you can also add the component as a preset by clicking the Add Preset button on the toolbar of the pane.

3 The Component Preset pane appears and the component is added as a preset in the default category.

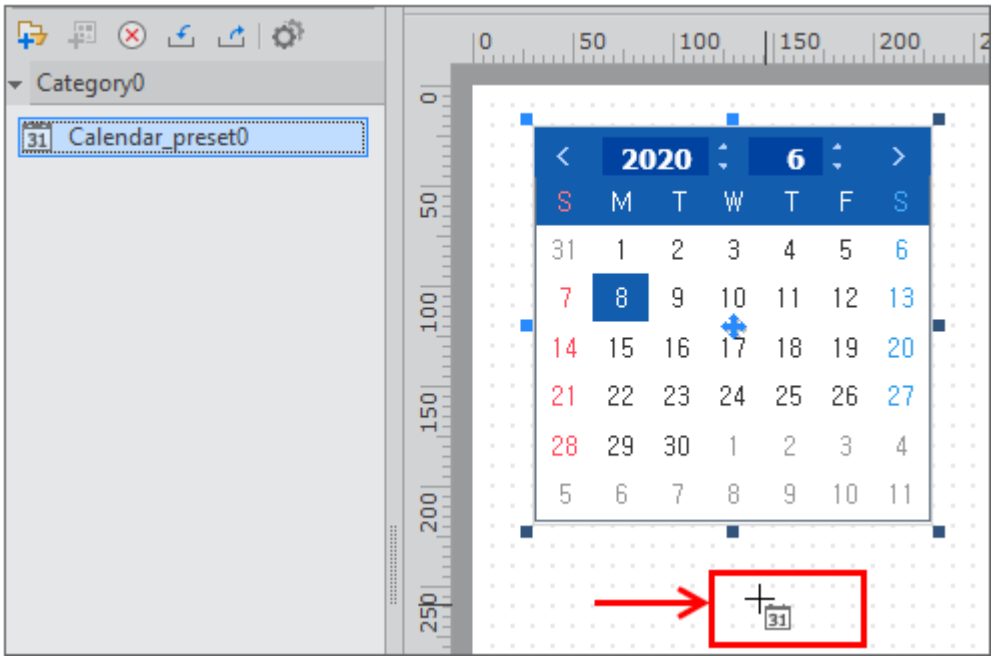


12.2.2 Allocate Preset

An added preset can be placed on the Form by following the below steps.

1 In the Component Preset pane, choose a preset that you want to use.

2 When you move the cursor onto a Form, you can see the cursor change to the icon of the preset you chose.



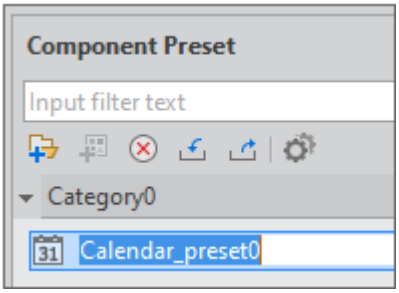
3 Click the position where you want to place the component preset.

You can see the allocated component has the preset properties just as defined.

12.2.3 Manage Preset and Category

Change name

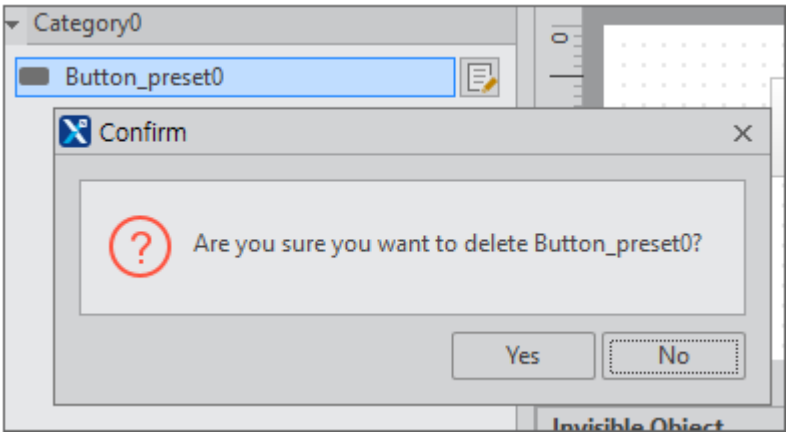
Choose a preset or category in the Component Preset pane. Then, double-click the item or press F2 to enter the editable mode in which you can change the name of the preset or category.



Delete

Choose a preset or category in the Component Preset pane. Then, press the Delete key on the keyboard or click the delete button on the toolbar of the pane to remove the preset or category.

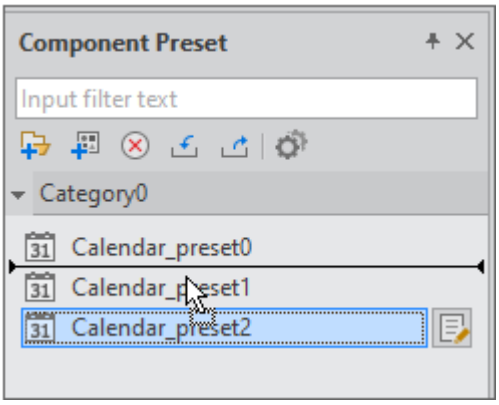
If you delete a category, all the presets under the category will be removed as well.



Change order

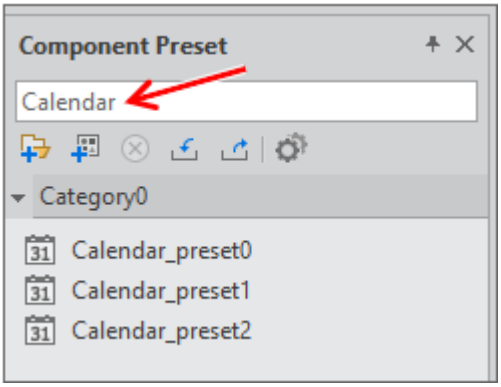
With a preset or category selected in the Component Preset pane, you can change the order by dragging the selected item.

If you change the order of a category, all the presets under the category will be moved as well.



Filtering

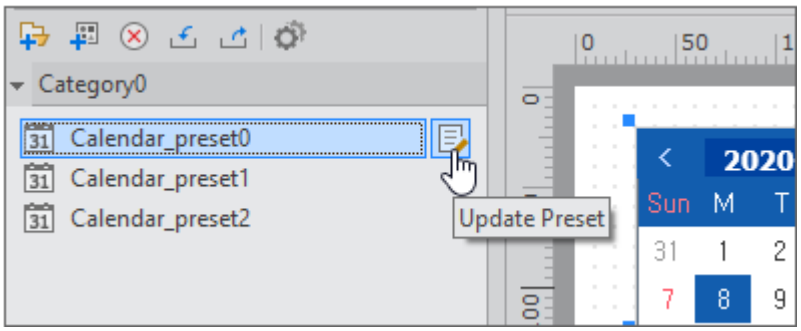
Although you can sort presets by using categories, you can also do it by filtering preset names. This method is particularly useful when you apply certain rules in preset names.



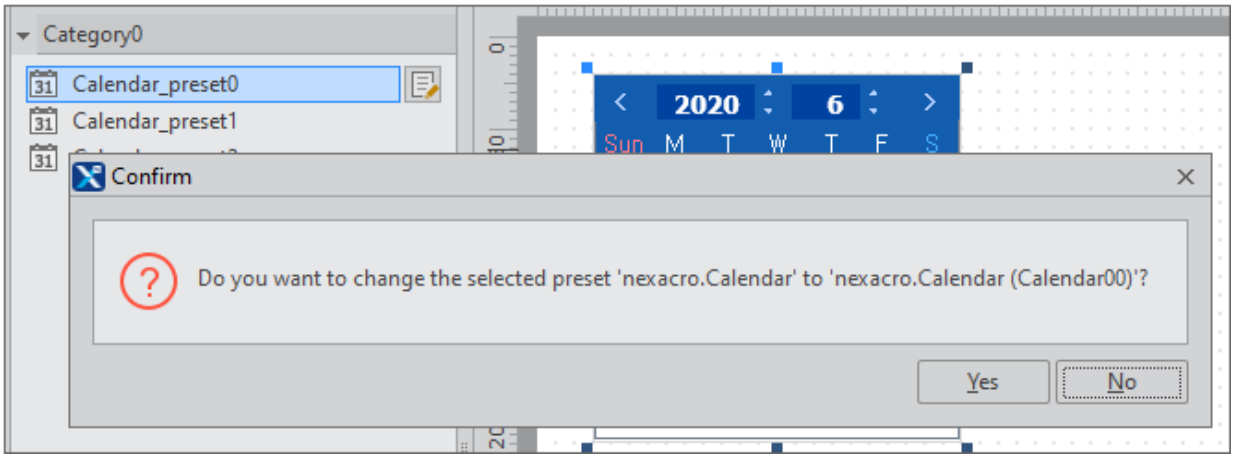
Update preset

You can update an added preset by following the below steps.

1. Place a preset on a Form.
2. Select the preset and change its properties.
3. While the component on the Form is selected, go to the Component Preset pane and choose a preset you want to update. Then, you will find the Update Preset button appearing beside the name of the selected preset.



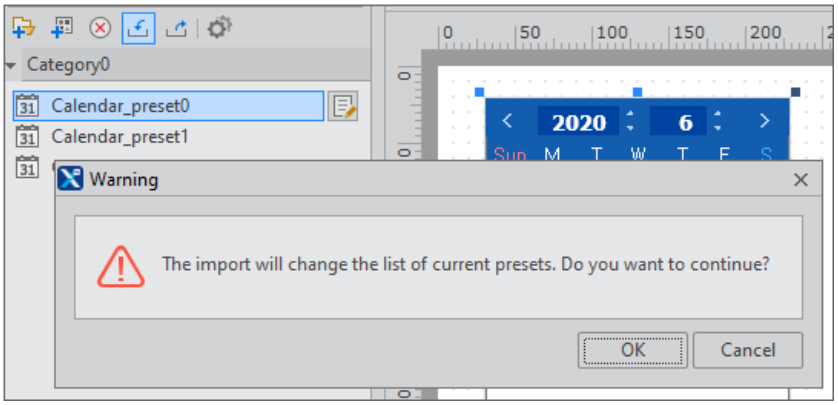
4 Upon clicking the Update Preset button, you are asked whether to update the preset. If yes, the preset will be updated.



12.2.4 Export and Import Component Preset File

The settings of categories and presets can be exported as a file and be used in other projects or by other users. If you click the Export button on the toolbar of the Component Preset pane, your settings will be saved as a file (with the *xpreset* extension).

If you click the Import button, you can import an already-saved file. Once you import a file, the Component Preset pane will display the new settings, which has replaced the existing settings.



12.2.5 Toolbar on Component Preset Pane

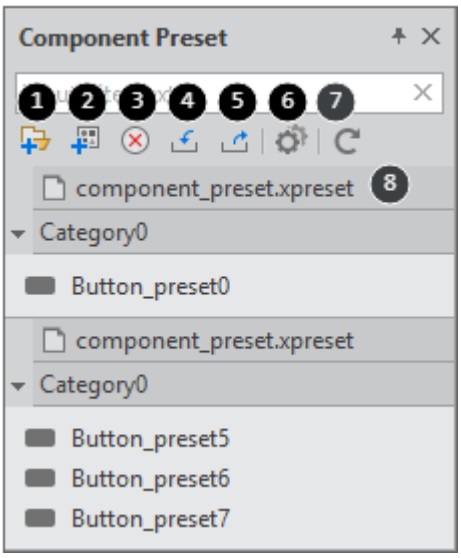


Table 12-1 Standard Bar

Menu		Description
1	Add Category	Adds a category to the pane
2	Add Preset	A component selected in the Form is added as a preset.
3	delete	Deletes a category or preset
4	Import	Imports the component preset file that was set in advance or selects the xpackage file to execute the Import Wizard.
5	Export	Exports component presets as a file
6	Options	Sets the component preset file location.
7	Refresh	Refreshes with the updated content when the component preset file is changed externally.
8	Component Preset List	If two or more component preset files are set in the options, two files are displayed.

13.

Form Designing | Positions and Guides

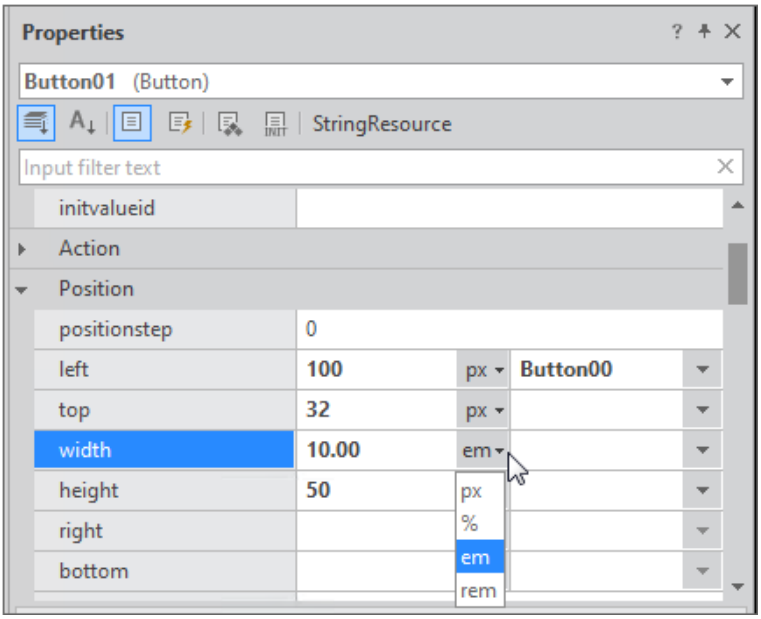
13.1 Position

The location of Forms and components on Nexacro Studio can be designated by setting the property values of left, top, right, bottom, width, and height. For each property value, you can choose one among absolute length unit ("px"), percentage unit ("%"), and relative length units ("em", "rem").

13.1.1 Basic Properties

As seen in the below picture, there are six basic properties related to positions: left, right, width, top, bottom and height. You can specify four values out of the six properties.

You can select the unit for position-related properties in the Properties pane. Absolute length unit ("px") and percentage unit ("%") are set based on the parent component, while relative length units ("em", "rem") are set based on the processing criteria for those units. If PositionBase is set, relative length units will not be supported, and the setting will be relative to the relevant component.



In the case of Form, Mainframe, and Childframe, the width and height actually used can only be specified in pixels.

For a Form, its left and right position values are fixed at 0. Therefore, you can only change the width and height values.

Two of the component's left, right, and width properties, and two of the top, bottom, and height properties must always be set.
If you select a component and select one of the previously set property items in the property window and delete the property value (or select Remove Property Value item from the context menu), it is replaced with another property, and the replaced property value is automatically converted and set.

When deleting the left property value, set the right or width property value
When deleting the top property value, set the bottom or height property value
When deleting the width property value, set the left or right property value
When deleting the height property value, set the top or bottom property value
When deleting the right property value, set the left or width property value
When bottom property value is deleted, set top or height property value

If one of the left, right, width, top, bottom, height property combinations is forcibly cleared and saved in the Source tab, the component may not be displayed in the Design tab.

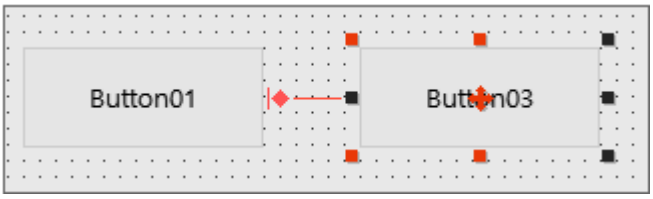
13.1.2 Tracker

When you select a component, the relevant tracker is indicated in red (or blue) according to the position property.

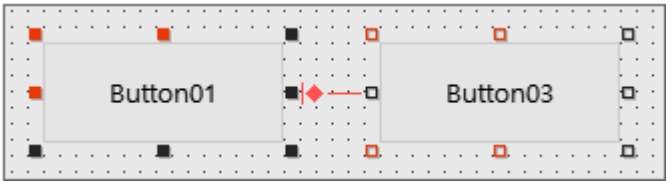
Tracker-applied image	Properties
	left, top, width, height
	left, top, right, bottom

Tracker-applied image	Properties
	left, top, width, bottom
	left, top, right, height

A PositionBase tracker appears at the center of a component unless the value of the component's *positionstep* property



If you select more than one component, only the trackers around the last-selected component will be filled with colors while other trackers are empty. In this case, the last-selected component will serve as the base for applying the Align, Space, Size, Position or Arrange features, which are all located in the menu [Design].



13.1.3 min-, max- and fittocontents Properties

You can set the *minwidth*, *minheight*, *maxwidth* or *maxheight* property to specify the minimum or maximum value of a component's height or width. If you set those properties, their values will override the values of the *width* and *height* properties when *width* and *height* exceed min- or max- properties.

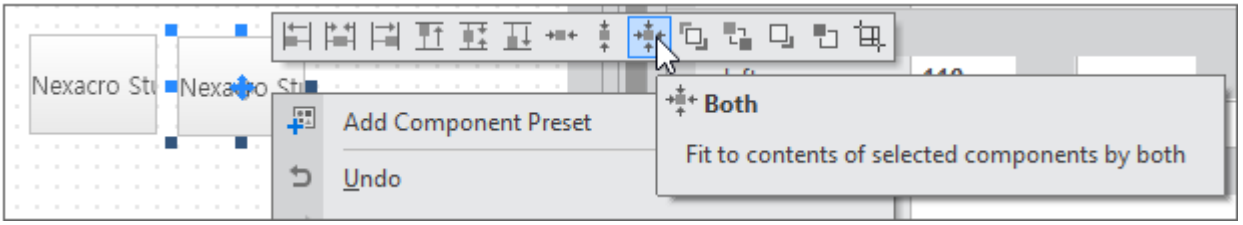
You can set the *fittocontents* property to adjust the size (height and width) of a component so that it fits onto its contents (text, image, etc.). For example, you can resize an ImageViewer component according to the size of the image specified as a value of the *image* property.

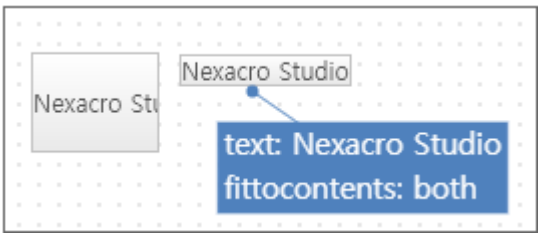


The below order indicates the priority when more than one of those properties are set simultaneously.
minwidth > *minheight* > *maxwidth* > *maxheight* > *fittocontents*
For example, let us assume that you specify a 400x400 image as a value of the *image* property

for a 300x300 ImageViewer with the *maxheight* property set to 350. If you set the *fittocontents* property in this case, the height of the ImageViewer will be 350.

The *minwidth*, *minheight*, *maxwidth*, *maxheight*, and *fittocontents* properties will materialize when the app is executed. If you want to realize those properties in the design mode of Nexacro Studio, you should select the relevant component and then select the menu [**Fit to Contents**], which can be found in the command bar (Design > Fit to Contents), toolbar or context menu.





If you execute an app without having performed the [Fit to Contents] feature in Nexacro Studio, the size of the relevant component may be seen differently between in the design mode of the studio and in the actual screen where the app is running.

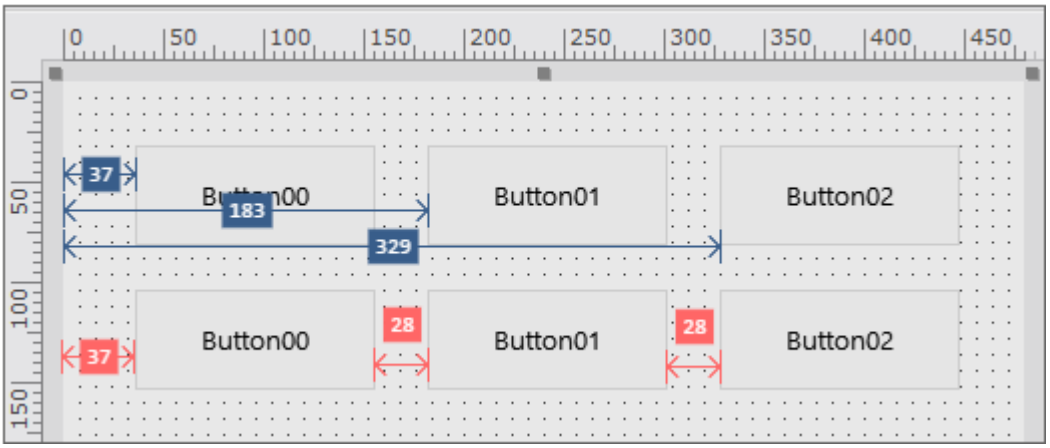
The *minwidth*, *minheight*, *maxwidth*, *maxheight*, and *fittocontents* properties will be applied when the Form is loaded or when you set the values of those properties. When the component's contents are dynamically resized, therefore, the change will not be applied automatically.

13.1.4 PositionBase

You can arrange a component based on another component that has the same parent. If you have a parent component (Form, Div, etc.) as a base, you have to calculate all the distances between the parent and each internal component when you arrange a certain component. If you have a child component as a base, however, you can arrange components easily just by setting the gaps between components.

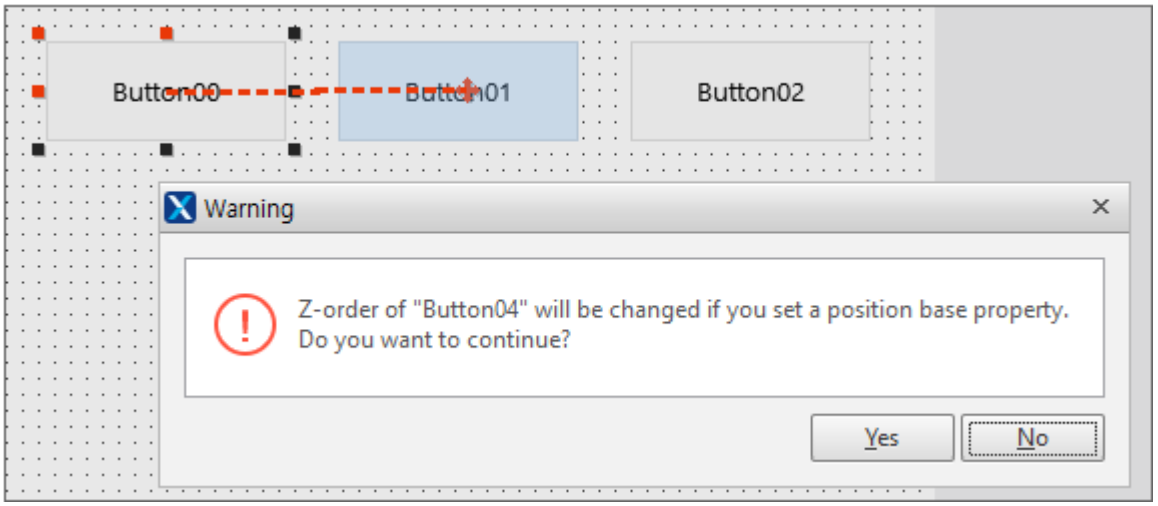
In the below picture, the top three buttons on the first row are arranged based on the parent component. In this case, you need to consider the width of the Button00 and the gap between the Button00 and Button01 to calculate the value of the Button01's *left* property—which is 183. By contrast, the Button01 on the second row

is arranged based on the Button00 on the same row. In this case, you need to know just the gap between the Button00 and Button01 to specify the value of the Button01's *left* property; the value is like "Button00:28".

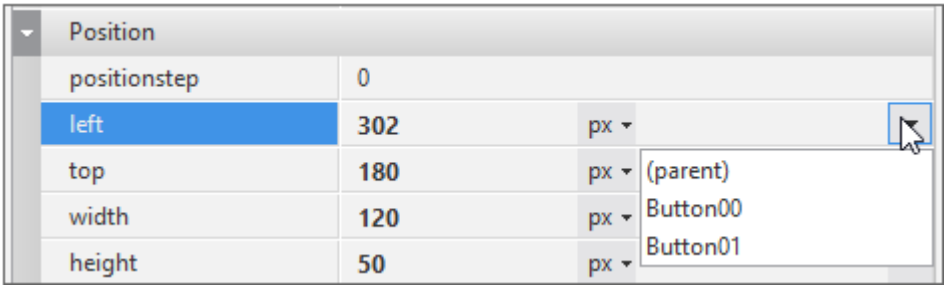


Restrictions

- You cannot set a component as a base if the component's parent is different.
- You cannot set a component as a base if the value of the component's *positionstep* property is different.
- If a component's *positionstep* is set to -1—which means the component is a fixed component—the component cannot be the base of another component. At the same time, it cannot have another component as its base.
- If you set Component A as the base of Component B, the Z-order value of A must be lower than that of B. Otherwise, the alert as shown in the below picture will appear to ask you whether to change the Z-order.



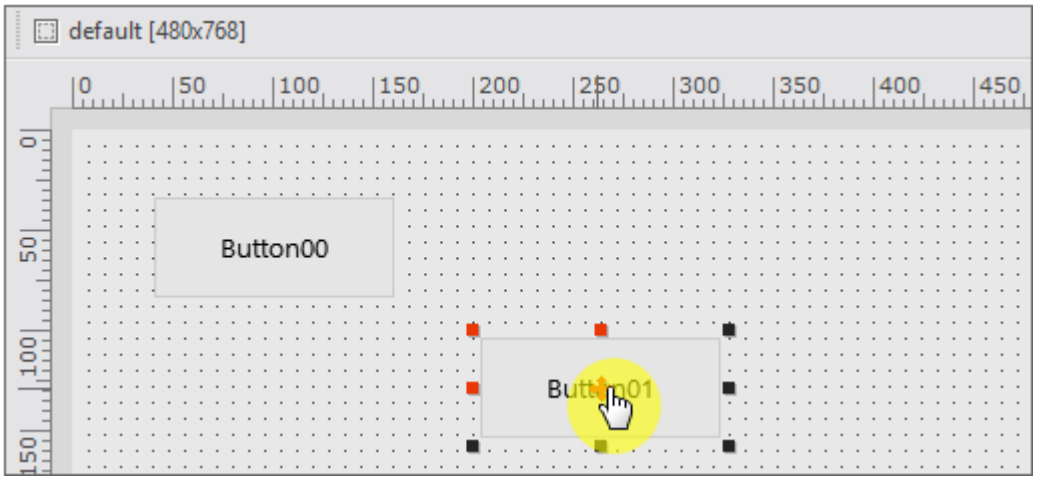
In the cases of the first three restrictions—*positionstep* is set to -1; different parent; and different *positionstep* value—PositionBase properties will not be activated in the Properties pane. Moreover, the PositionBase properties of a component whose Z-order value is higher will not be activated either.



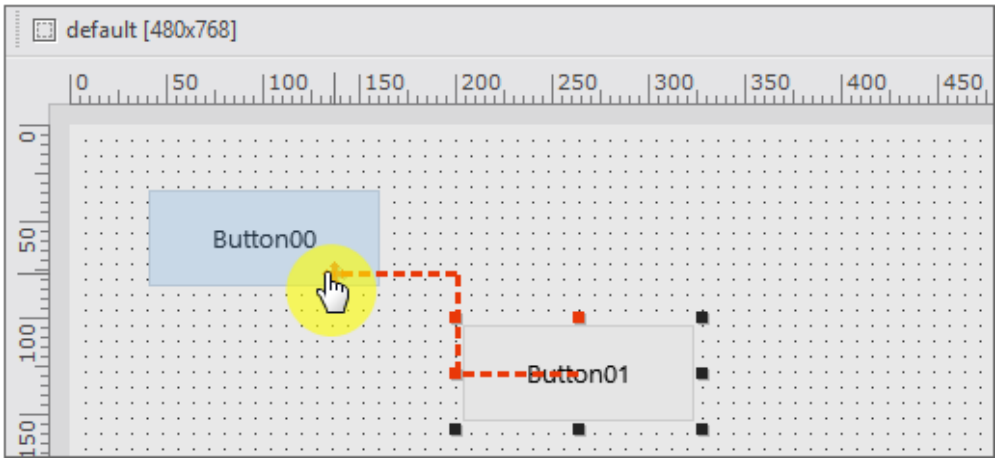
PositionBase tracker

If you select one component, a PositionBase tracker, which takes the form of a four directional arrow, will appear at the center of the component. To set its base component, drag the tracker to a target component. Then, a prompt box will appear to allow you to set exact PositionBase values before you finish setting the base component.

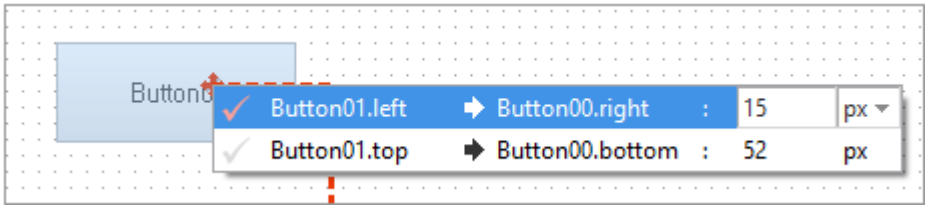
1. Select a component and then press-and-hold its PositionBase tracker with a mouse.



2. Drag the tracker to the target component. If the target component can be designated as a base, its background color will change.



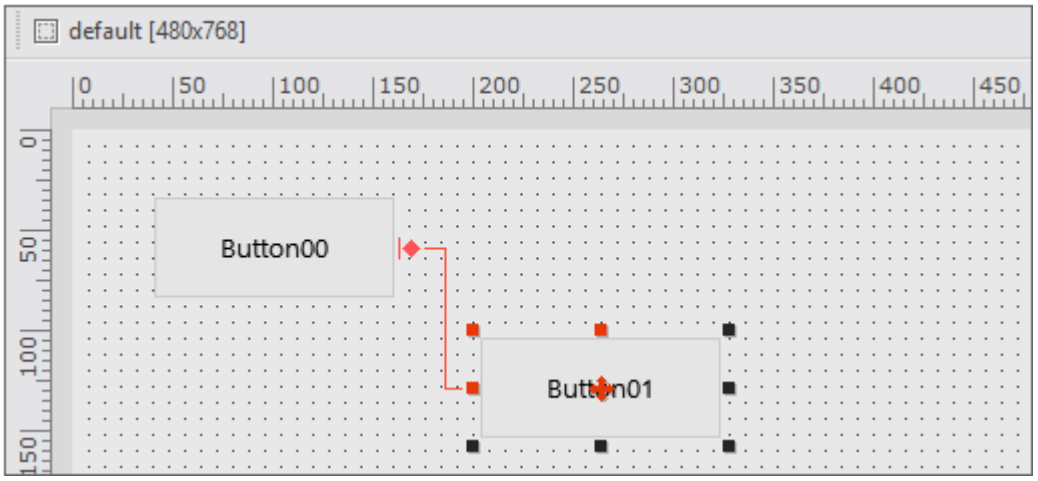
3 Determine which positional properties will be connected to the base component when a prompt box appears. The prompt box presents the properties that can be selected, which vary depending on the direction the tracker is dragged. Select the check box in front of the property that you want to edit.



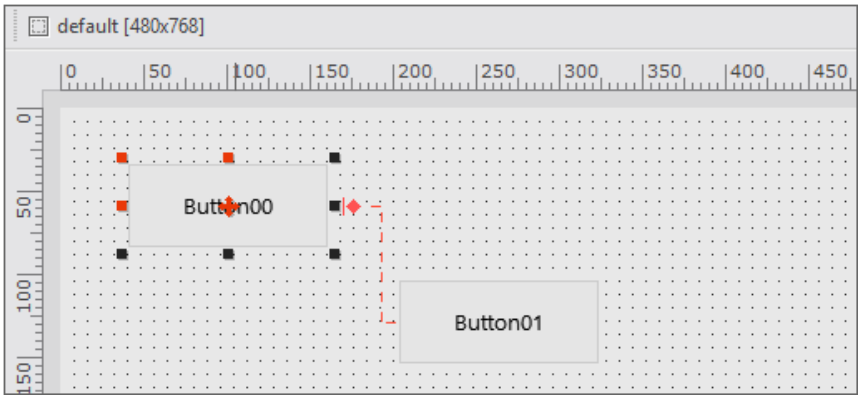
The below table shows the properties that can be selected, which vary according to the directions the tracker is dragged.

top, left	top	top, right
left	**Component**	right
bottom.left	bottom	bottom, right

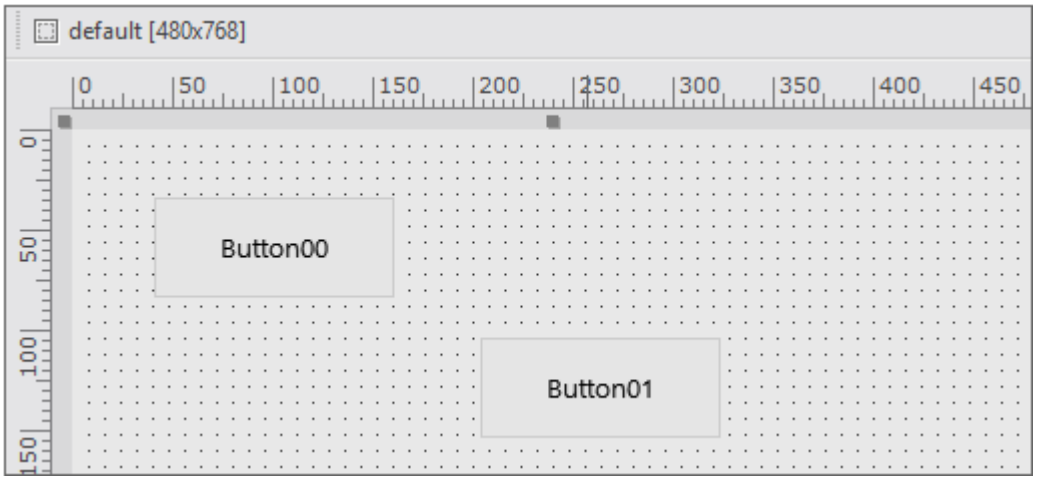
4 After the completion of the setting, a solid line will be used to show the relation between the two components when the connected component is selected.



When the base component is selected, a dashed line will be used.

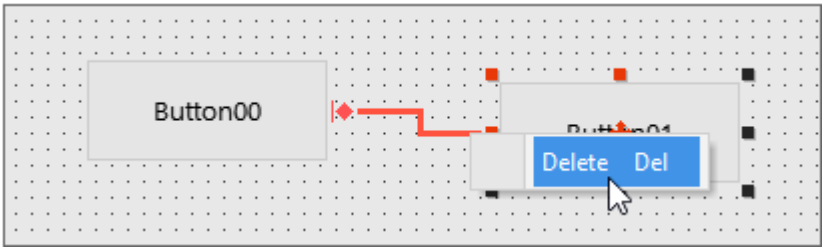


No line will be used when no component is selected.

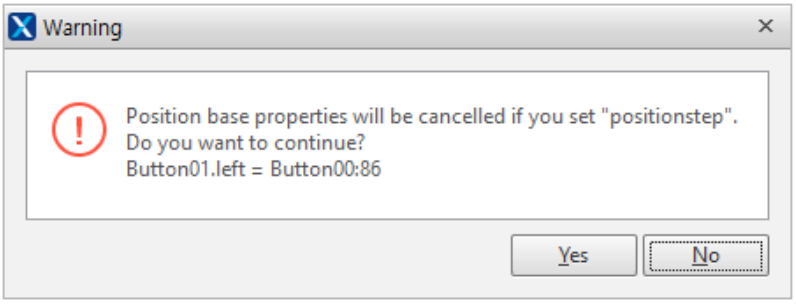


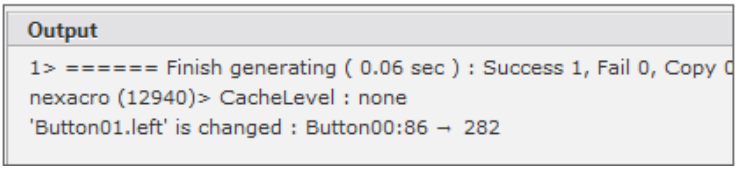
Removing a connection

To remove a connection, select the line of the relation on the Form Design pane and press the Delete key. Otherwise, right-click the line and select [Delete] from the context menu. In another way, select the connected component and delete the values of PositionBase properties.



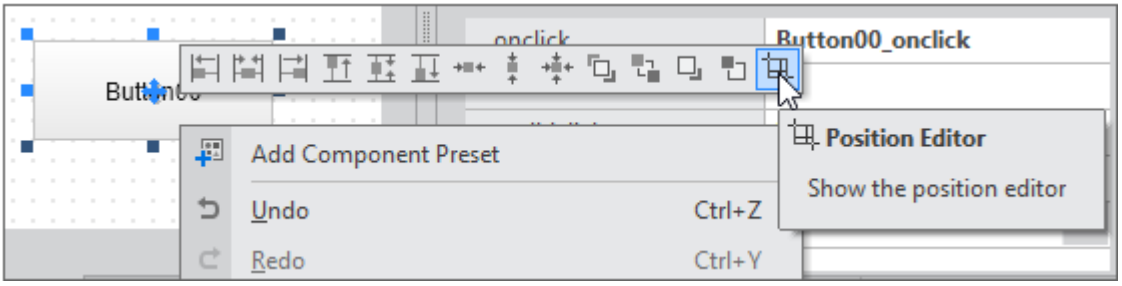
If you change the value of the *positionstep* property or Z-order, the relevant component's relation with its base component will also be removed. In these cases, an alert box will appear to present the PositionBase relation that will be removed. If you proceed with the change, the Output pane of Nexacro Studio will show the result of removing the PositionBase relation.



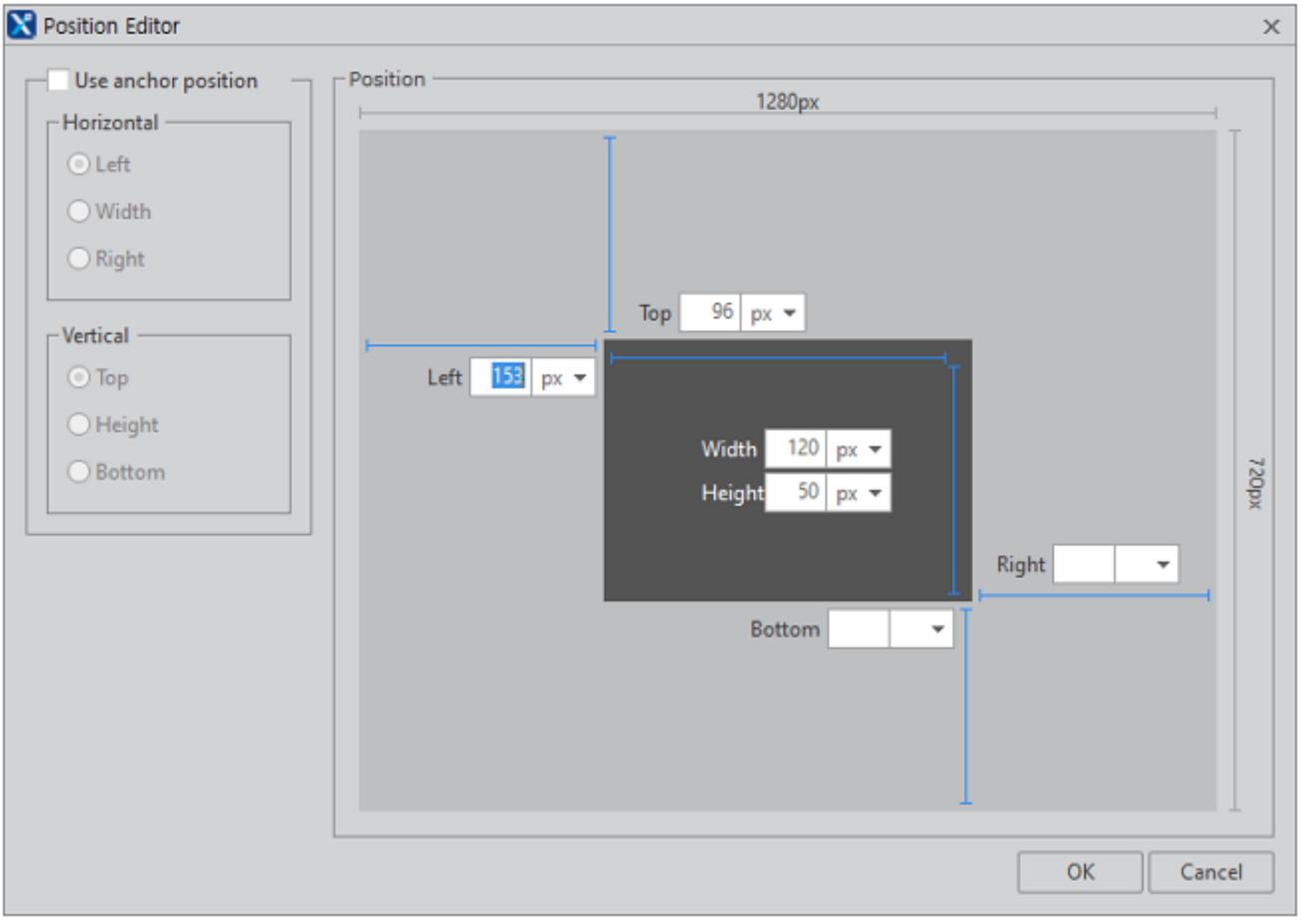


13.1.5 Position Editor

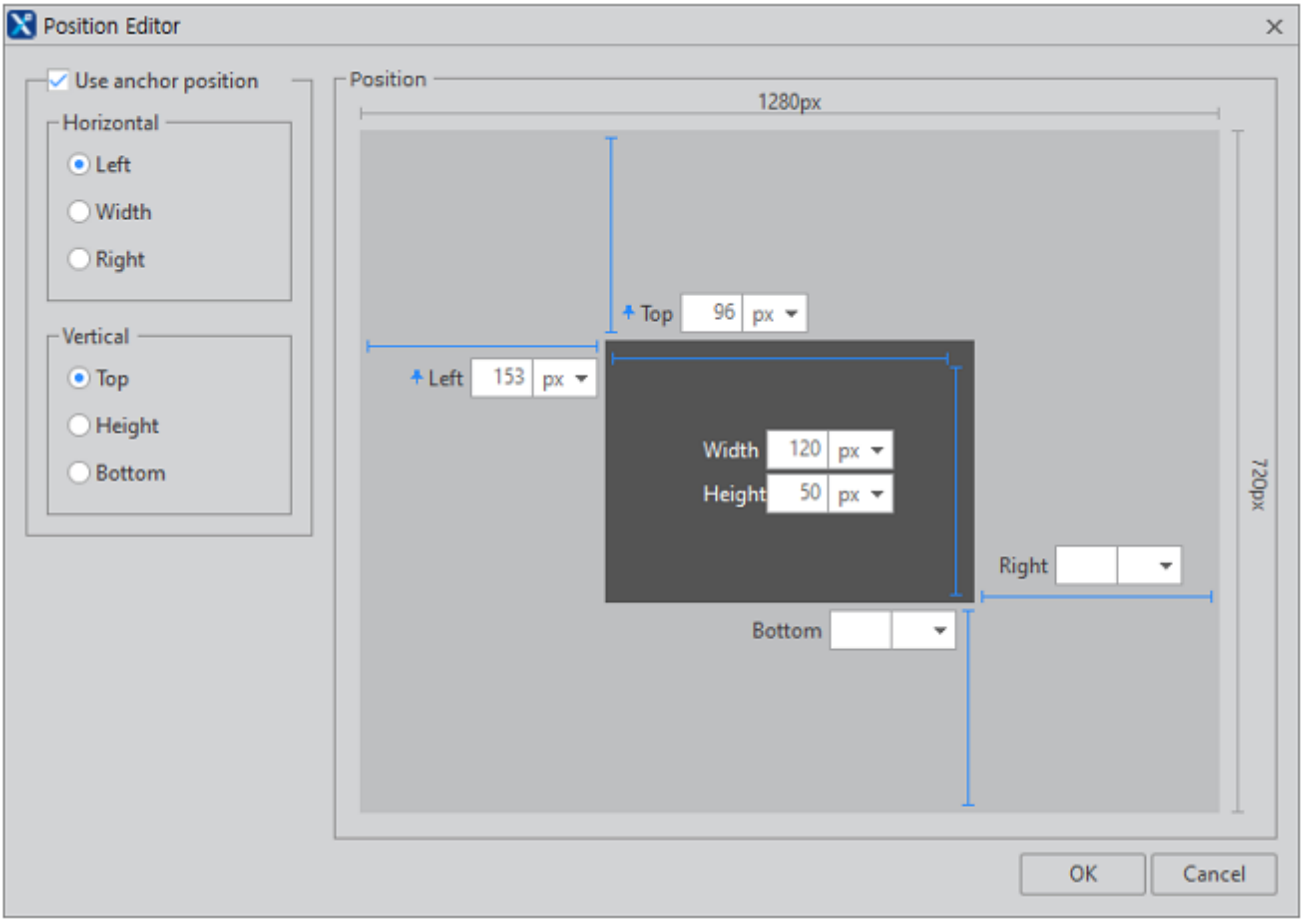
To open Position Editor, right-click a component to open a mini-toolbar and select the [Position Editor] icon illustrated in the below picture. Otherwise, access the menu [Design > Position Editor].



In the editor, you can change position properties of the selected component based on its parent component.



If you select [Use anchor position], you can change position properties with fixing two of them. In the below picture, the *left* and *top* properties are fixed, and you cannot delete those two properties' values.



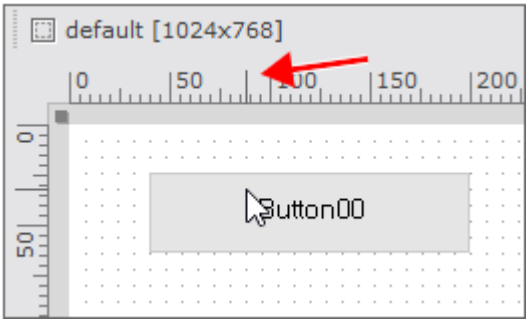
13.2 Guides

You can adjust the options related to guides at [**Tools** > **Options** > **Form Design** > **Guide**]

13.2.1 Ruler

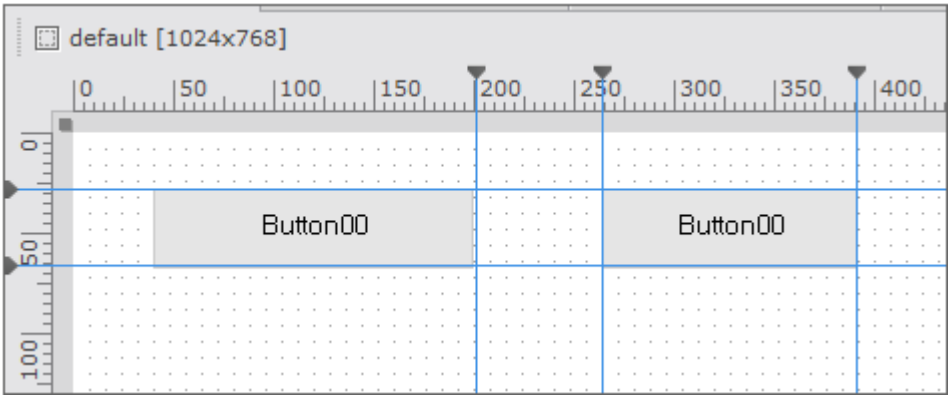
To assign layouts easily when designating locations on the Form design, the ruler unit can be either pixel unit (px) or percentage (%).

When a mouse cursor moves within the Form design panel, the position of the cursor is indicated on both rulers.



13.2.2 Guide Line

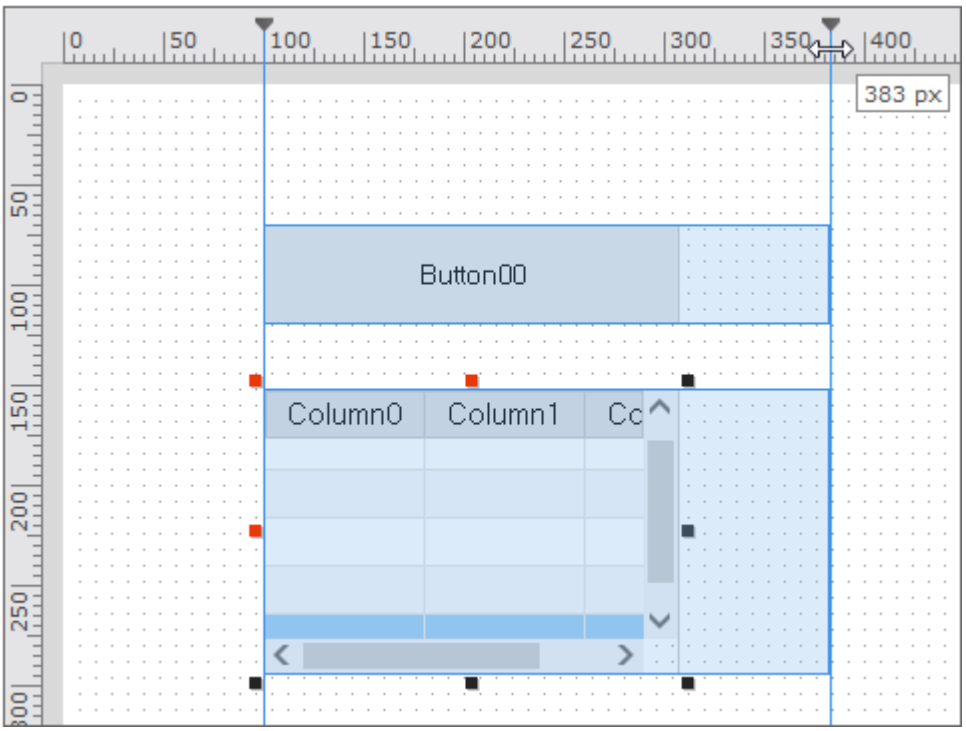
The Guide Line function is included with the Ruler and allows the developer to make vertical/horizontal lines wherever desired to help in aligning components.



Create a guide line by clicking a spot on the ruler. Move a guide line by selecting and dragging it with the mouse. Delete a guide line by selecting it and dragging and dropping it out of the Design Window or double-clicking on the inverted triangle on the guide line.

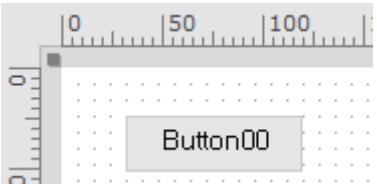
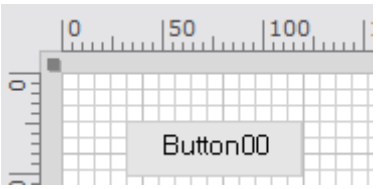
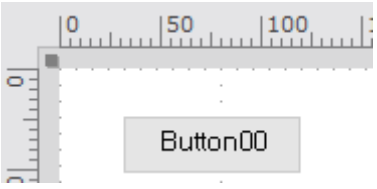
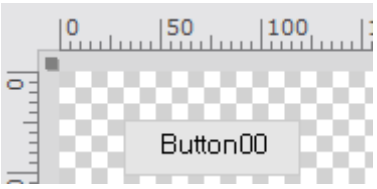
If a Form has two or more layouts, each has its own guide lines. In addition, guide lines will remain unchanged even if you exit Nexacro Studio.

If you fit components in between two guide lines, the component sizes will change according to the direction either guide line moves.



13.2.3 Background Grid

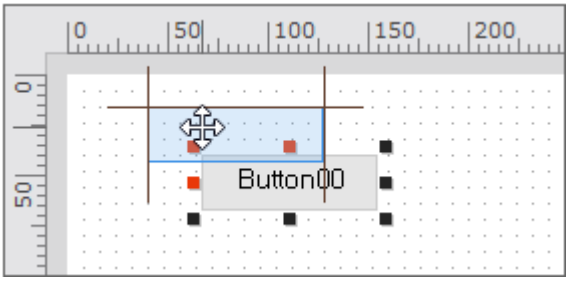
The background of the Form design panel features a Grid to allow users to identify component layouts with the naked eye. The Grid size will differ according to the measurement units of the rulers. The below pictures represent the four Grid patterns.

Dot	
Grid	
DotGrid	
Tile	

The patterns of a background grid will be displayed according to its settings without being influenced by the measurement units of the rulers.

13.2.4 Snap

Use the Magnetic function to make minute adjustments automatically if you put other components or options close to the DotGrid. The Snap function can be activated based on the option value on the **[Tools - Options - Form Design]** menu for DotGrid or other components.



The Snap function can be overridden by pressing ALT while moving components.

14.

Form Designing | MLM, Fluid Layout

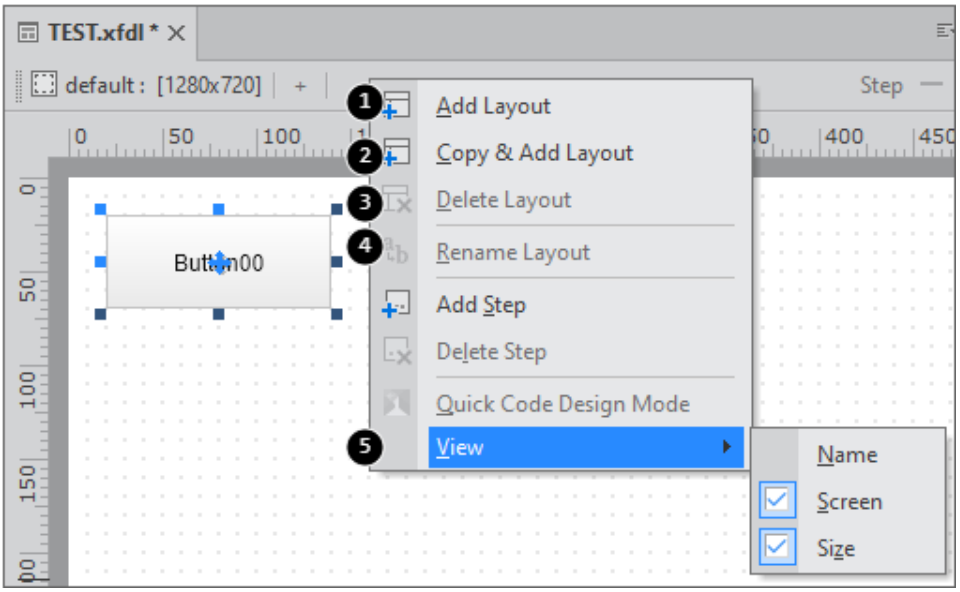
14.1 MLM (Multi Layout Manager)

Nexacro provides the Multi-layout Manager (MLM) feature to allow users to develop multiple layouts for one screen, making it possible to handle various devices with various screen sizes at once. A Form that composes an app can have multiple layouts according to screen sizes while the use of sub-layouts enables customized arrangements of inner components. The Step feature also can provide mobile devices with optimum screens.

14.1.1 Layout

A component that can have editable layouts must have a layout named "default." You can add further layouts in the process of creating a Form. After the creation, you can also add or delete layouts by or assessing the context menu opened from the design pane.

"default" layouts cannot be modified or deleted.



	Icon	Description
1	Add Layout	Adds a layout
2	Copy & Add Layout	Copies the selected layout and adds it as a new layout
3	Delete Layout	Deletes the selected layout
4	Rename Layout	Renames the selected layout
5	View	Displays the information on defined layouts (You can select/deselect items including names, screens and sizes) Even if you choose to hide names, the default layout's name will not be hidden.

Layout Toolbar

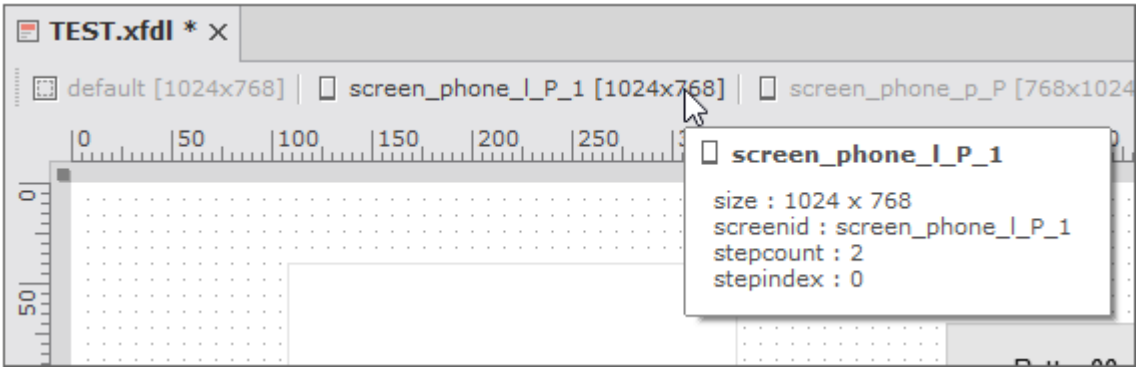
Nexacro Studio provides a toolbar that helps you select and edit layouts under a certain screen.

The toolbar shows three types of information.

- Layout icons indicating screen types
  (none, desktop, tablet, phone, desktop + tablet, desktop + phone, tablet + phone, all)
- Layout name
- Size (width x height)

Tooltips appearing over the layout tabs give additional information.

- screenid
- stepcount
- stepindex
- description



You can edit a certain layout by selecting the relevant layout tab on the toolbar.

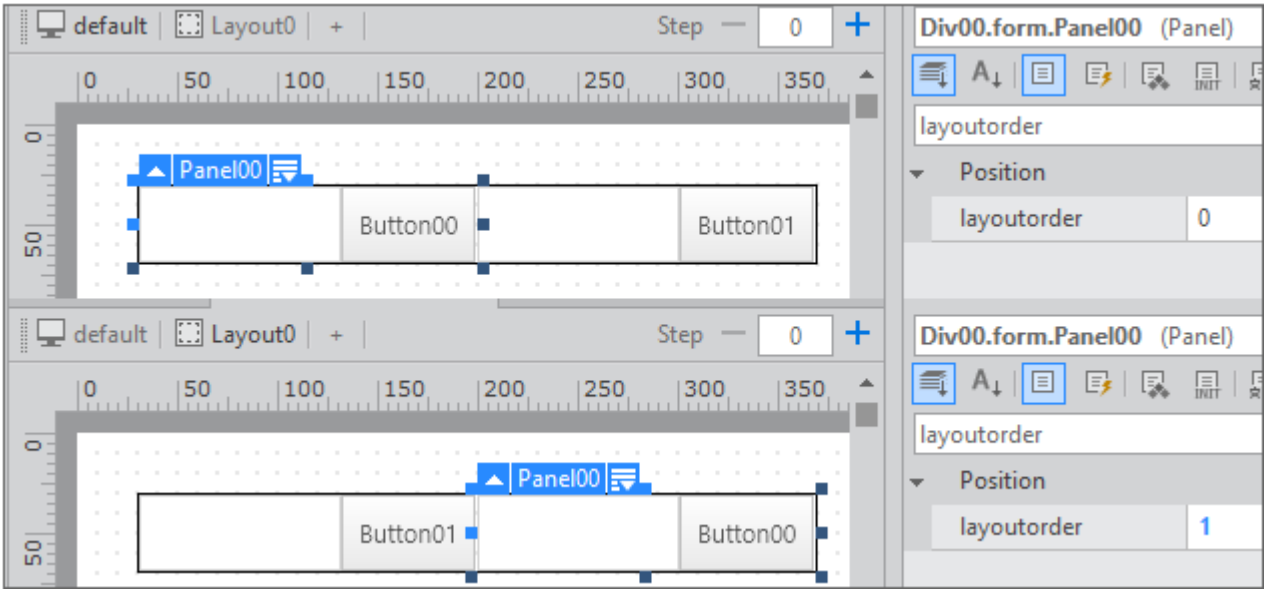
Fluid Layout

type: "horizontal" | "vertical"

In a Fluid Layout environment, when the type property is set to "horizontal" or "vertical," the added Layout will not allow drag operations or changing component positions using the Layout Order Editor.

In a Fluid Layout environment, when the type property is set to "horizontal" or "vertical," the added Layout will not allow drag operations or changing component positions using the Layout Order Editor.

For example, if you want to change the position of a Panel component (Panel00) within a Div component in the added Layout (Layout0) to the right, select the Panel component and change its layoutorder property to 1.

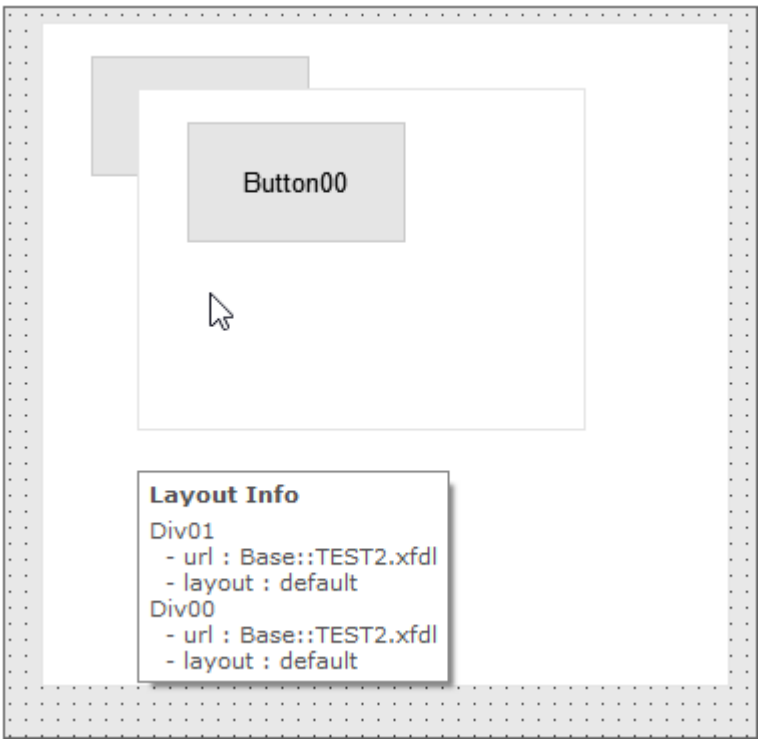


type: "table"

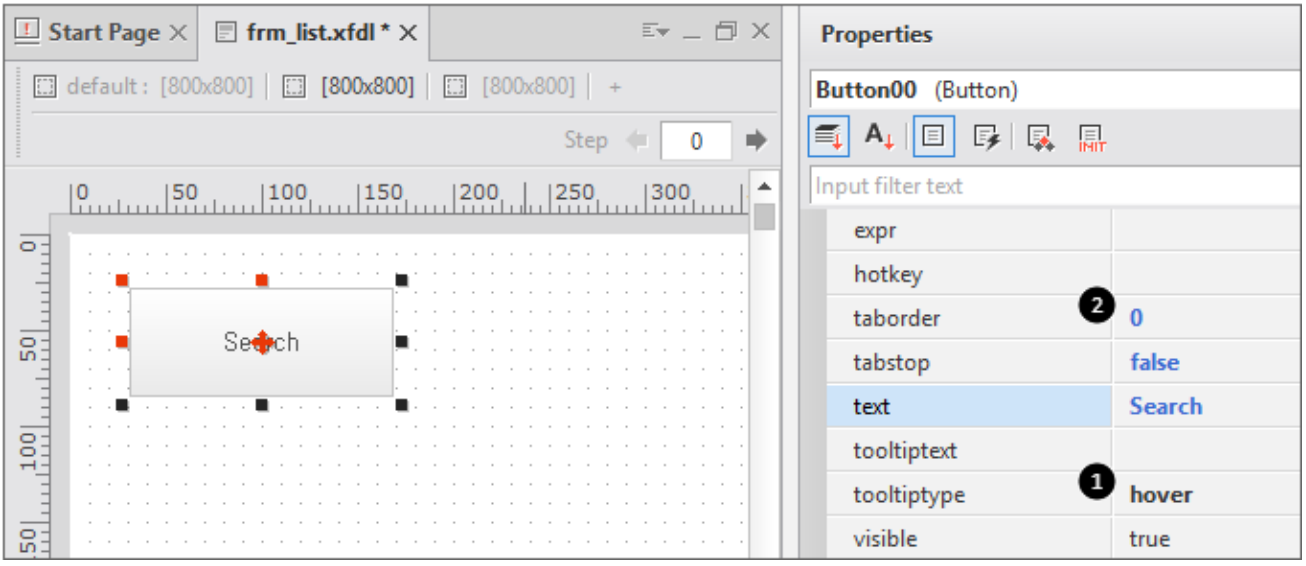
In a Fluid Layout environment, when the type property is set to "table," you can change the position of components by dragging. When changing the position of components, the tablecellarea property value is modified, allowing changes in all layouts.

Layout Tooltip

If a Form is connected to a component through the url property, Nexacro Studio will provide you with the relevant layout information in the form of a tooltip in the design mode. If components overlap, the studio will provide the layout information on the all overlapping components simultaneously.



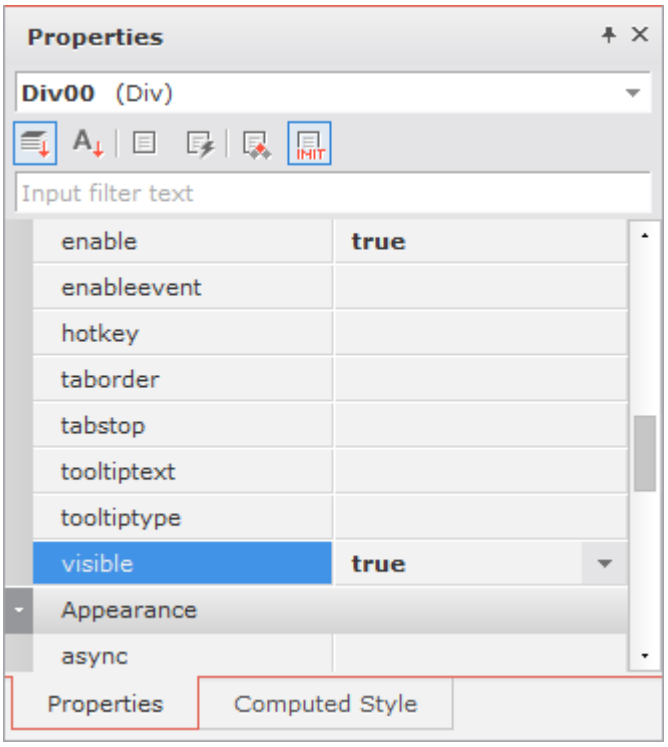
Property Values in Different Font Styles



Color	Indicated information
Bold	Edited in the default layout
Blue	Edited in added layouts (Values are displayed in blue only when you select the relevant layout.)

14.1.2 Initialize

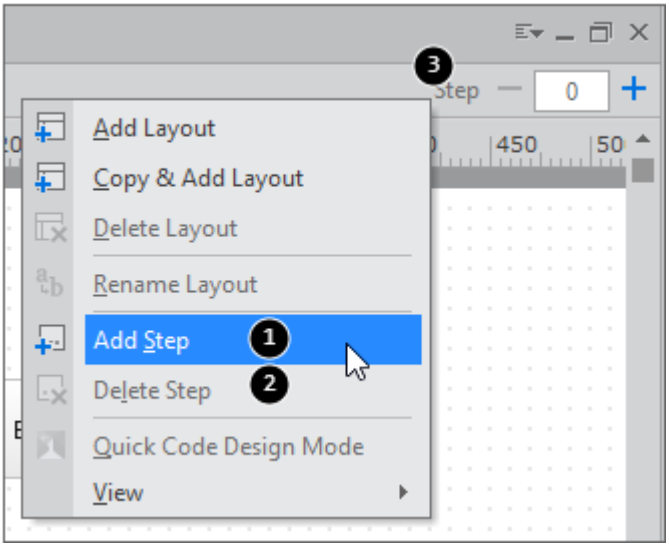
In some cases, initial property values entered by users must be changed according to the switch in layouts. Such values can be managed in **Init Info**. You can apply This function only to few of the whole properties, excluding the ones related to layout information.



You cannot confirm the implementation of the Init Value feature in the editing process. You can do it only when you actually execute the relevant application.

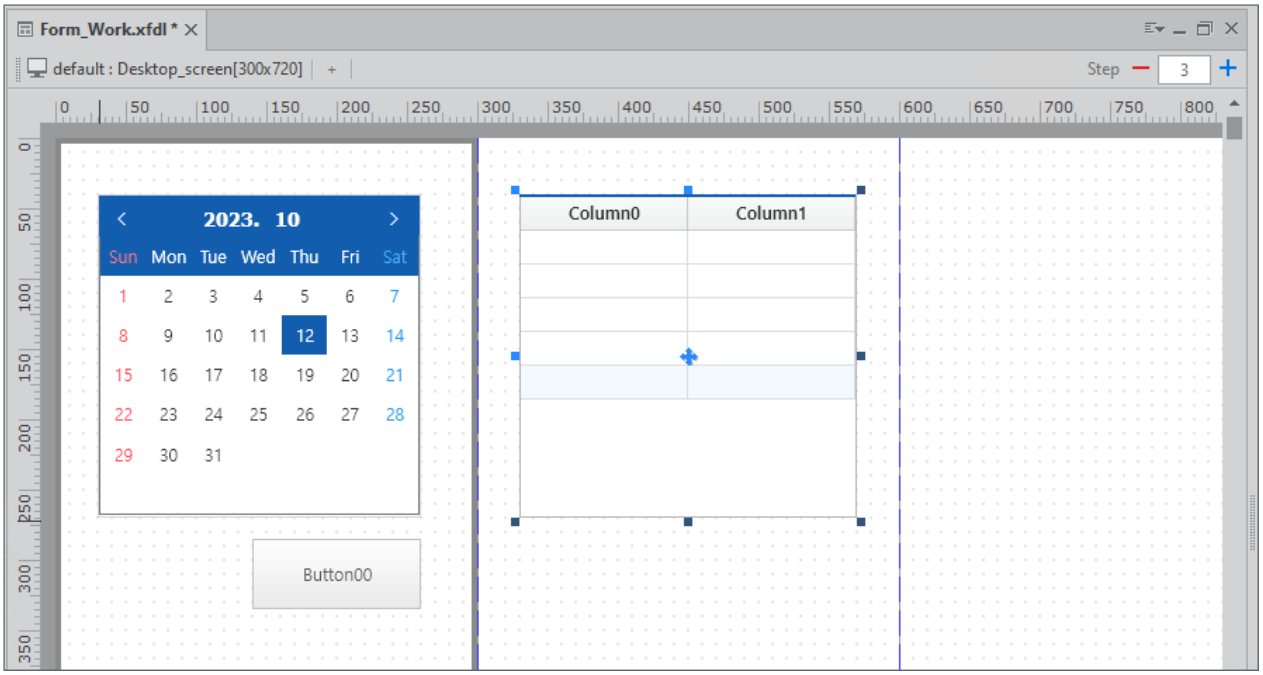
14.1.3 Step

Step is a function which can develop pages that consist of several steps in one Form. When you control the `stepcount` property value, the step will appear as shown in the figure below.



Call-out		Description
1	Add Step	Adds a step page. The value of the *stepcount* property will increase by 1.
2	Delete Step	Deletes a step page. The value of the *stepcount* property will decrease by 1.
3	Add/Delete Step	You can change the number of step pages by clicking the buttons. In addition, you can specify the number of step pages by typing a number in the input box.

stepcount



1. Indicates the step area currently being edited. The edited area may change depending on the location of the mouse pointer.
2. Each step area is indicated by the blue dotted line.



Even if there are several steps, the Form's size does not change. At runtime, only one step is visible with the step changing according to the user interaction.



If you change the value of the *positionstep* property of a component while the component is selected, the component will move to the given step.
If the value of *positionstep* is out of the step's boundary, the value will be treated as 0.

14.2 Fluid Layout

You can set the components to be placed automatically according to the rules without fixing the position of the components placed in the Form Layout. This can be used to implement a feature for dynamically adding the components or automatically changing the component arrangement method according to the screen size.

14.2.1 Setting component placement standards

In Form objects and container components (Div, Tab, View), you can set the default placement criteria in the type property of the Layout object or the type property of the Panel component.

Form object

You can change and create the Layout Type selection value in the Form Wizard or change the type property value in the properties window after creating the form. The type property is a layout object property and can be set for each layout.

Container component

When placing a component directly in a container component without connecting another form file, the type property can be set.

Panel Component

The default value for the type property is "horizontal."

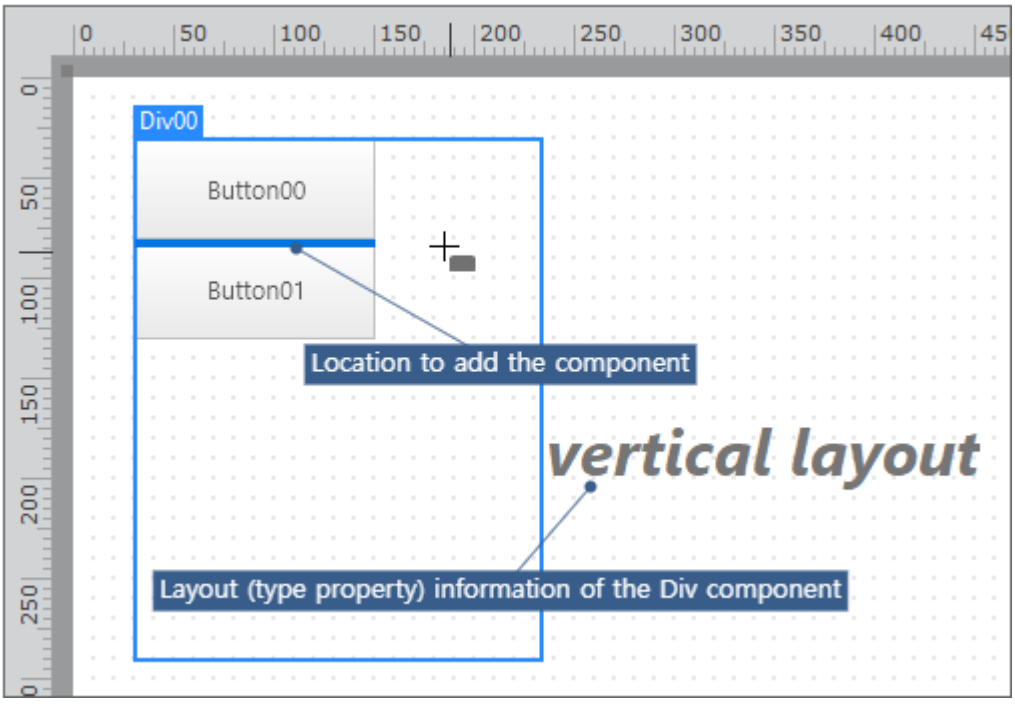
When executing the [Group by Panel] menu after selecting two or more components, the type attribute value is applied according to the Auto Layout standards. For detailed information, please refer to the Auto Layout section.

14.2.2 Placing components (horizontal, vertical)

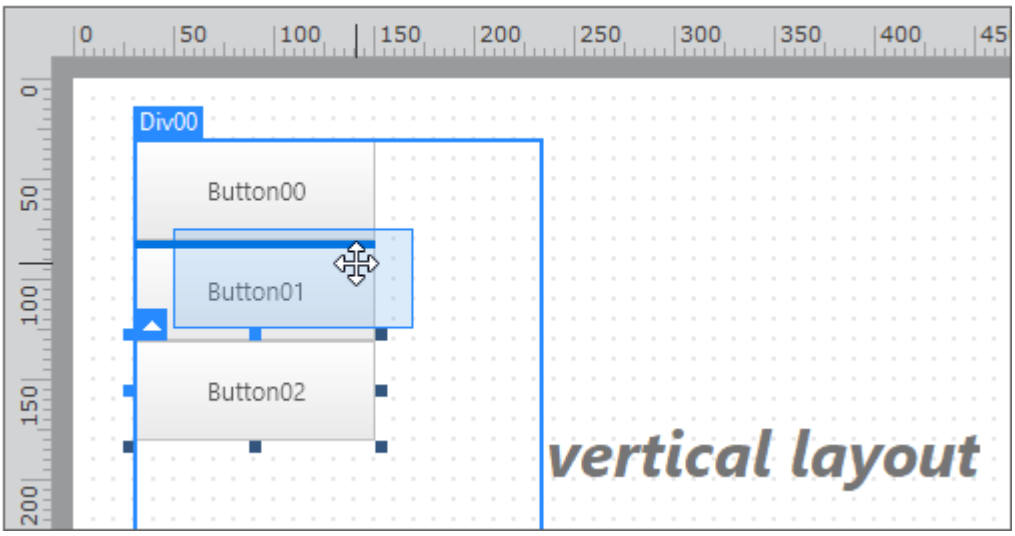
Here is an example of the Div component with the type attribute value set to "vertical".

Select a components and move with the mouse

To place a component, drag it over the Div component area, and a bold blue line will show the location where the component will be placed.



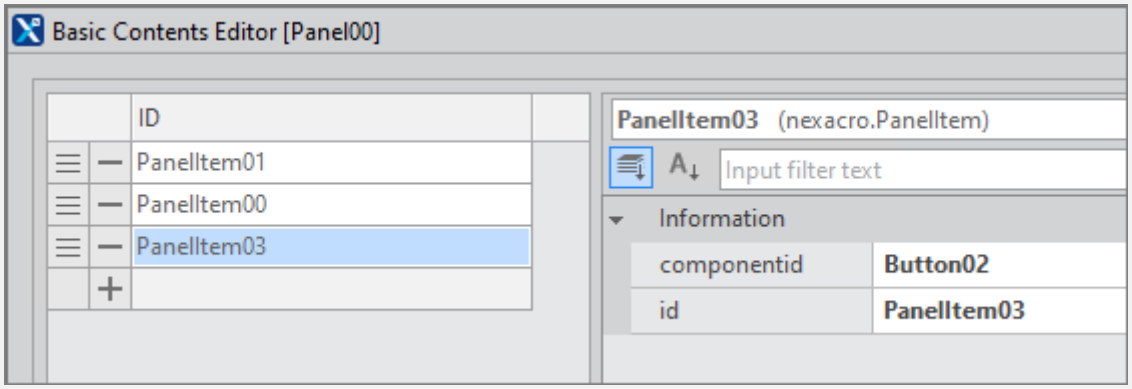
The location of the placed component can be changed.



Grouping and moving the components

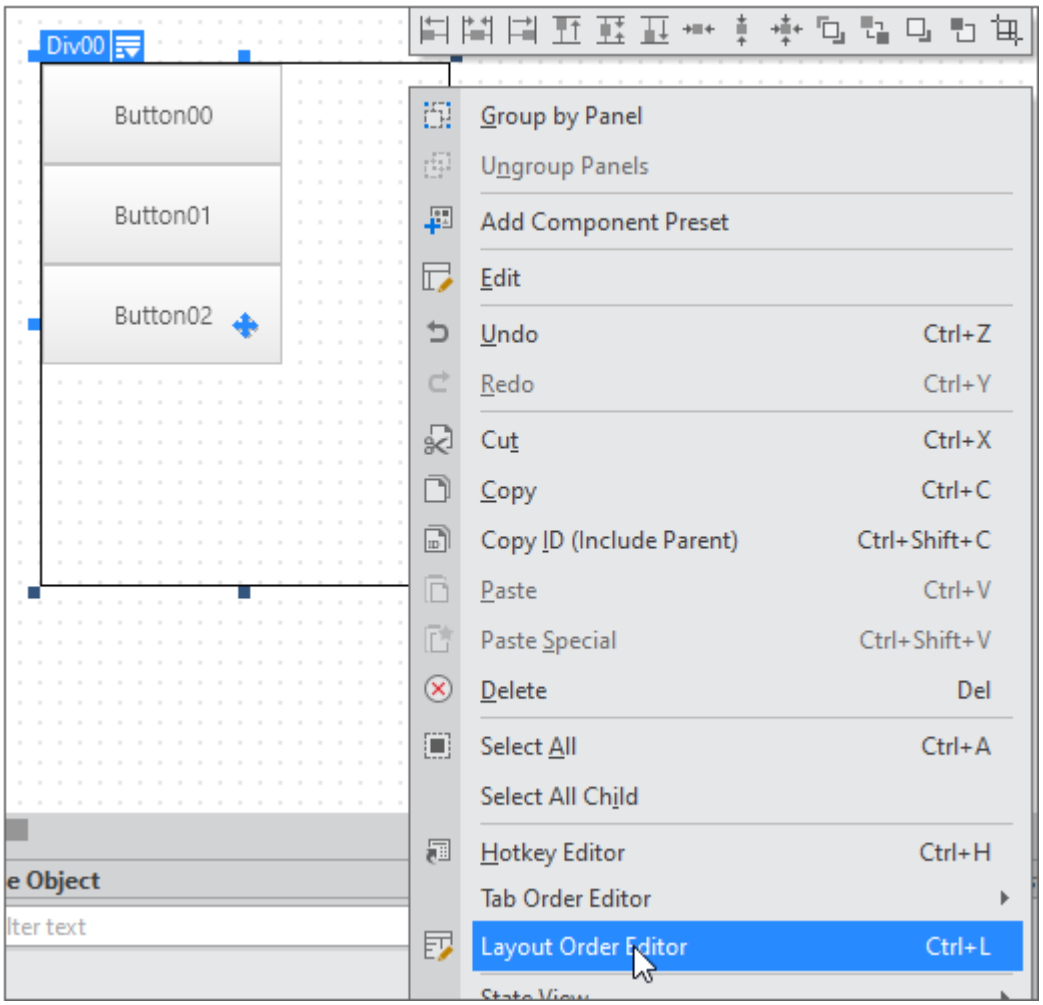
The Layout Order Editor allows you to set the layoutorder property value of components as well as the layout order of components.

The Panel component does not support the Layout Order Editor.
You can change the order of connected components in the items property editor of the component.



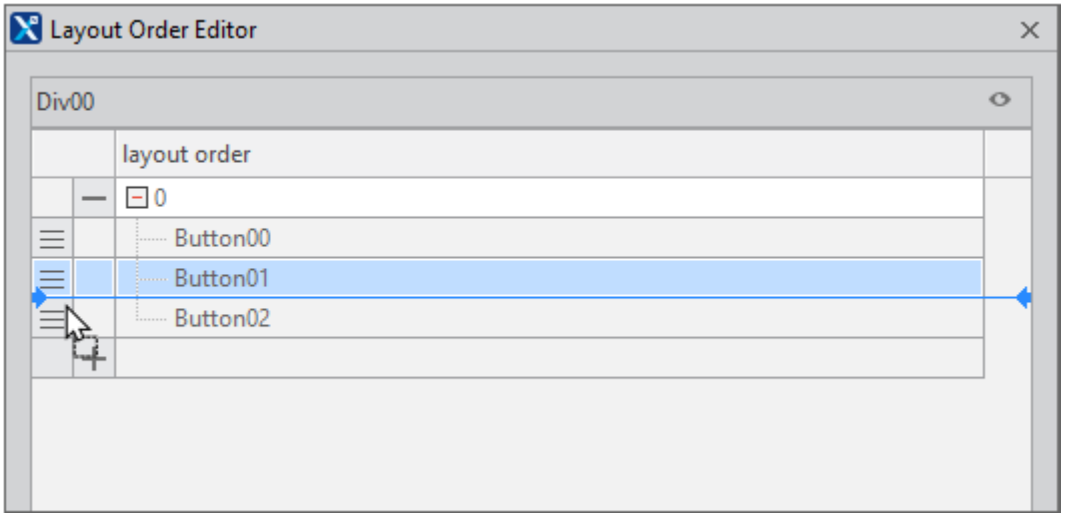
The component's layoutorder property value is used to dynamically create a component and place it in a desired location. In the Layout Order Editor, you can change the position of multiple components at once by setting components to have the same layoutorder property value and changing the layoutorder property value.

1. Select the container component and choose "Layout Order Editor" from the context menu.

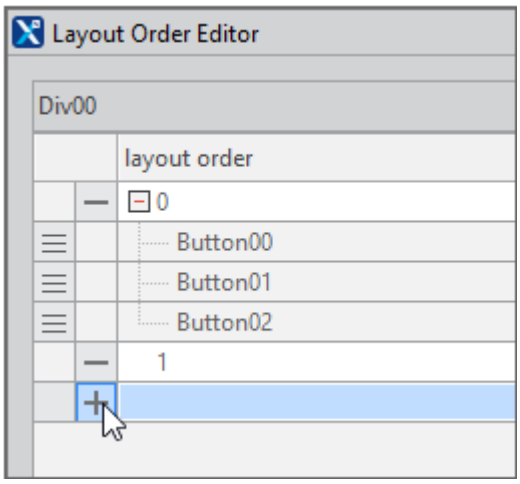


2 In the Layout Order Editor, select and move the Button component with the mouse to change the order.

If the layoutorder property value is not set, it is applied as 0.

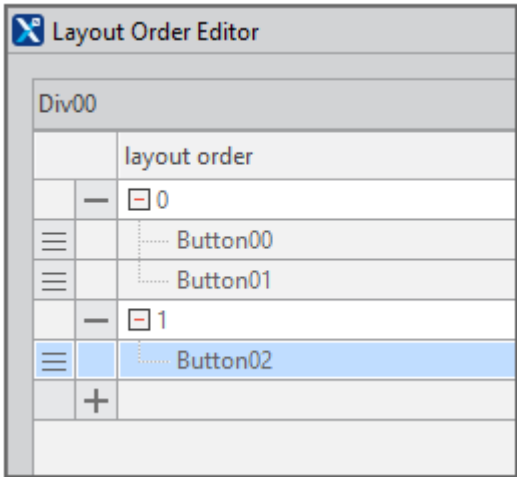


3 Click the [+] button to add a layoutorder item and set the value to 1.

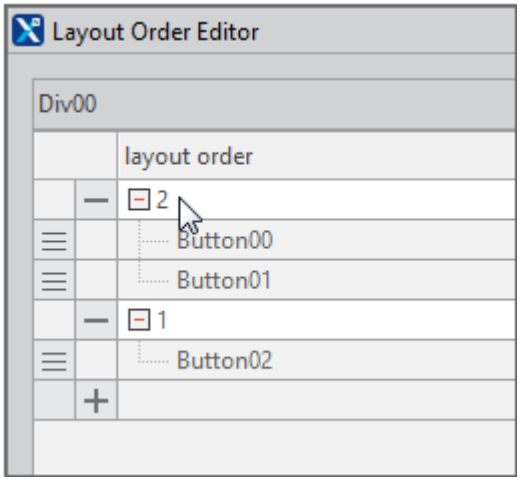


4 Select the Button component with the mouse and move it under the added layoutorder(1).

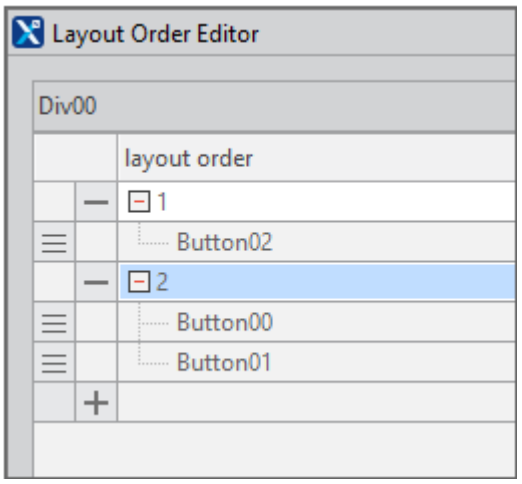
Confirm that the component is placed under the layoutorder(1) group.



5 Click on the layoutorder(0) item and change the value from 0 to 2.



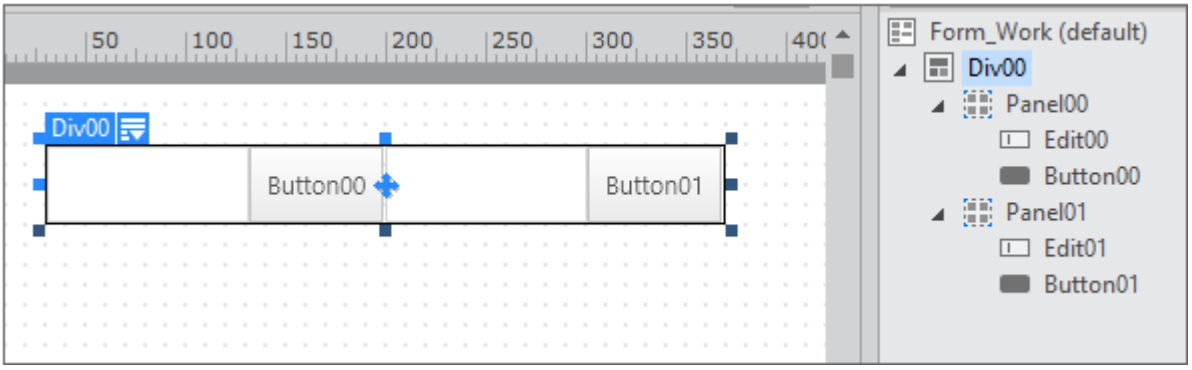
Modifying the value will change the order of the layoutorder items and subcomponents.



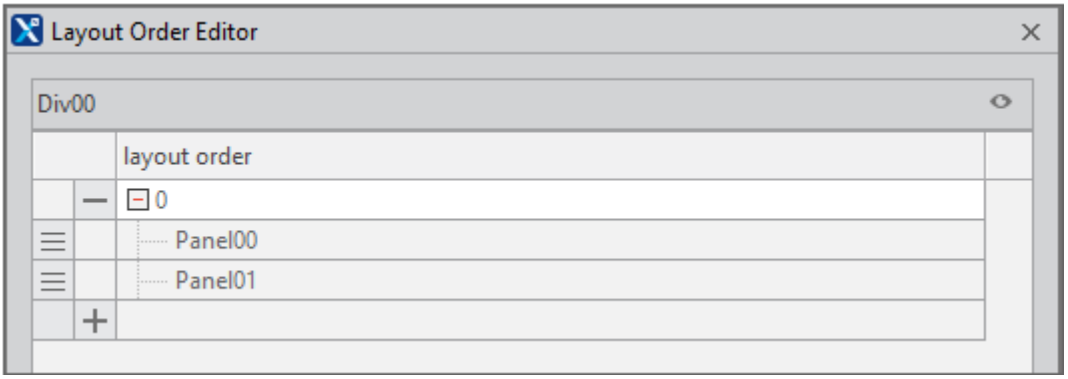
When a container component has a Panel component

When a container component has a Panel component, if you select the "Show panel items" icon in the Layout Order Editor, the components corresponding to each Panel Item will be displayed.

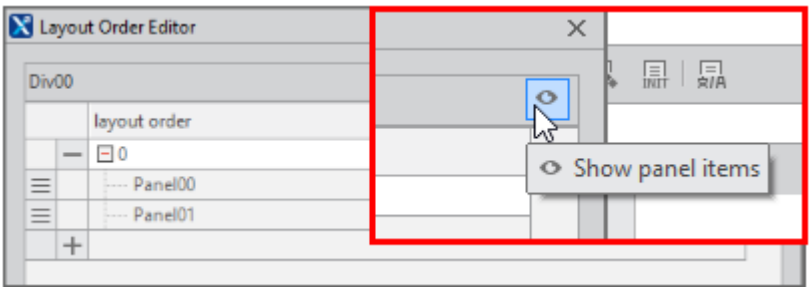
For example, place 2 Panels inside a Div component, and place an Edit and a Button component in each Panel.



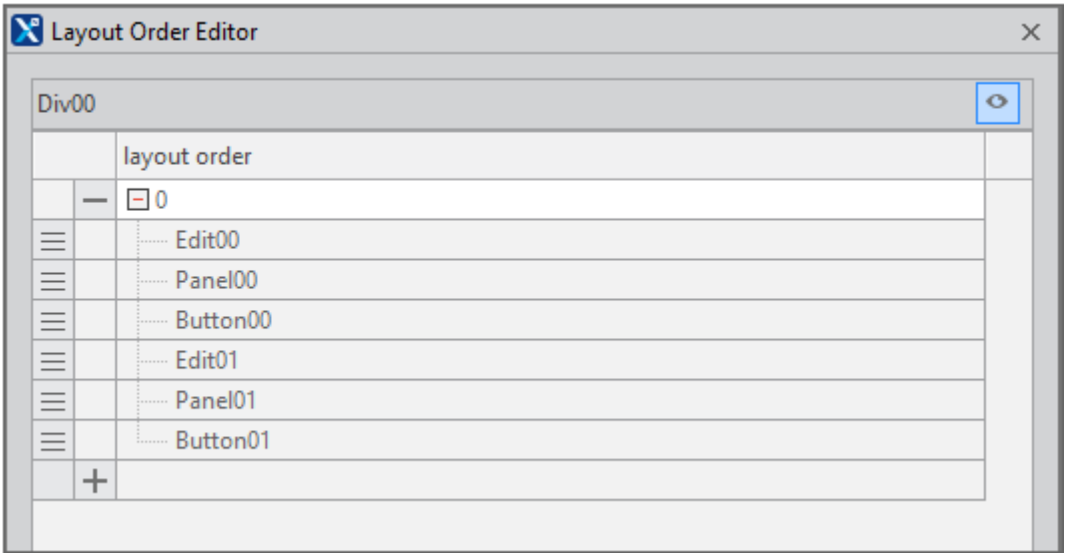
When the Layout Order Editor is run, only the two Panel components are displayed.



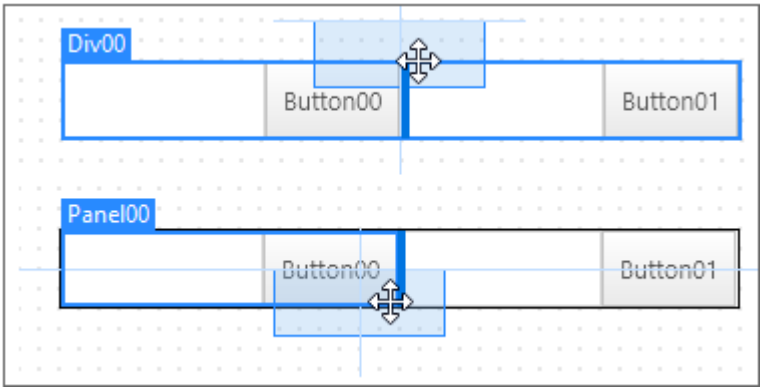
To check and adjust the positions of the components within each Panel, click the "Show panel items" icon.



A list of components corresponding to the Panel Items will also be displayed. The order in which they appear is based on the placement order of the components within the Div component, so changing the order of the Panel Items will not affect the arrangement of components within the Panel.



When using the tracker for placement, the top label can be checked to see which component the new one will be placed under. If placed within the Div component, the Div component label becomes active, and focus is shown across the entire Div component area. If placed within the Panel component, the Panel component label becomes active, and focus is shown only within the Panel component area.

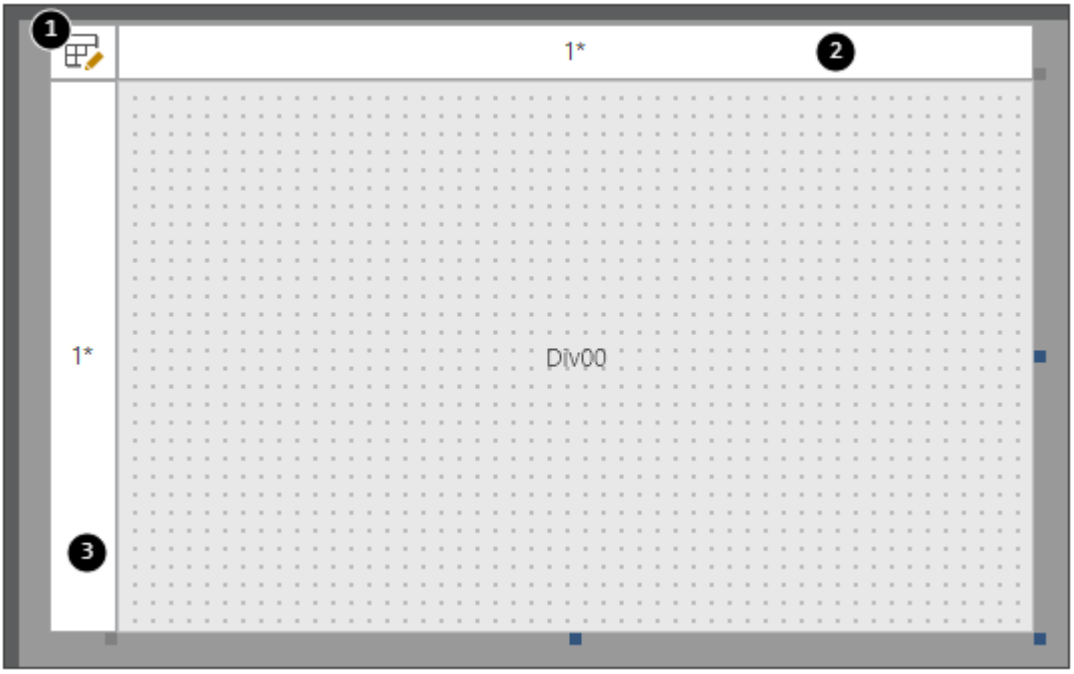


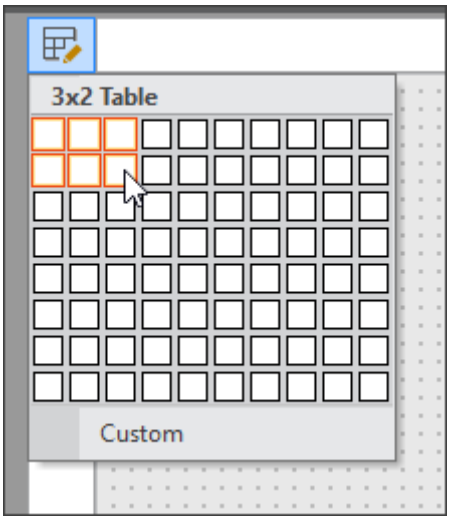
14.2.3 Placing components (table)

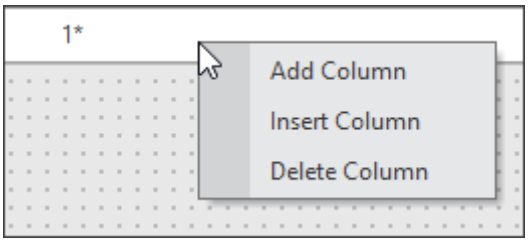
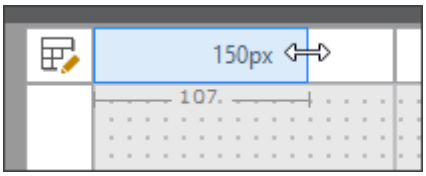
Here is an example of the Div component with the type attribute value set to “table”.

Placing a component at a specific cell position

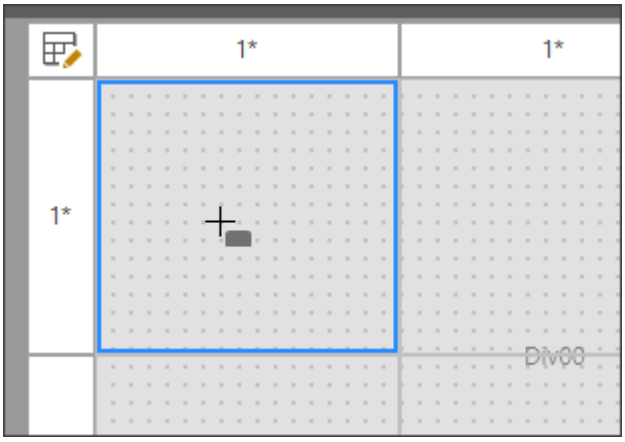
If you set the type property value to "table", the editing window is displayed with the default tabletemplate property default value (1 column, 1 row) applied.



		Description
1	Reset Table	Click the table icon at the top left of the editing window to display the Reset Table window where the number of columns and rows can be changed. 
2	Column editing	Right click to display the context menu for adding, inserting, or deleting the

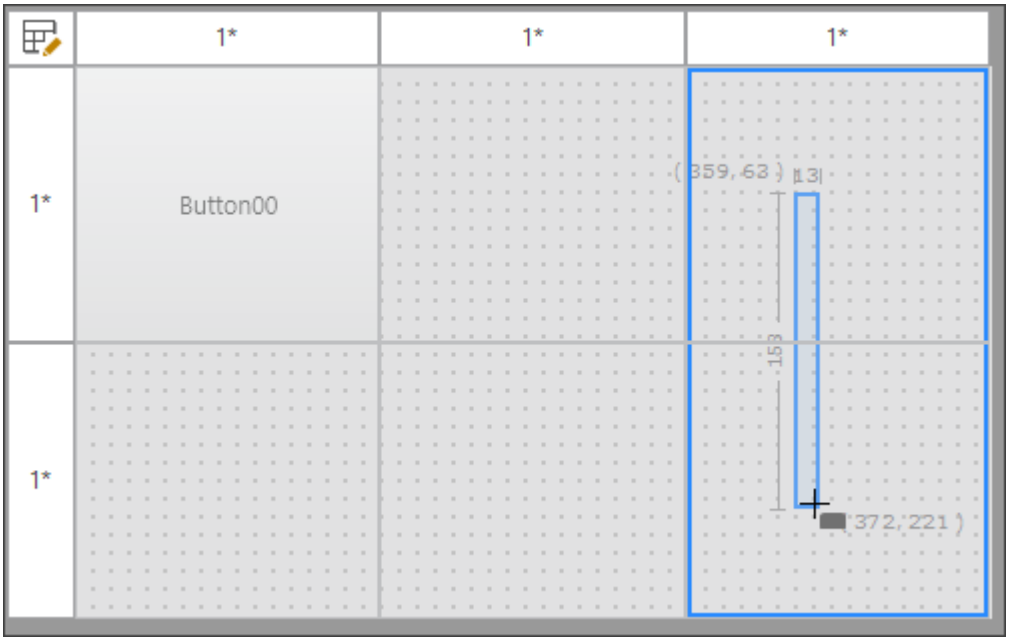
		Description
		columns.  When the column item is clicked, it switches to a editable mode and the value can be changed.  Drag the border with the mouse to change the value. If the item value is in units of px or %, it is changed by 1, and if it is in the unit of *, it is changed by 0.5. 
3	Row editing	Right click to display the context menu for adding, inserting, or deleting rows. When a row item is clicked, it switches to editable mode and the value can be changed. Drag the border with the mouse to change the value. If the item value is in units of px or %, it is changed by 1, and if it is in the unit of *, it is changed by 0.5.

This is an example of the layout set with three columns and two rows. When you drag it to the Div component, a virtual table cell area is displayed and you can place it in the desired cell area.

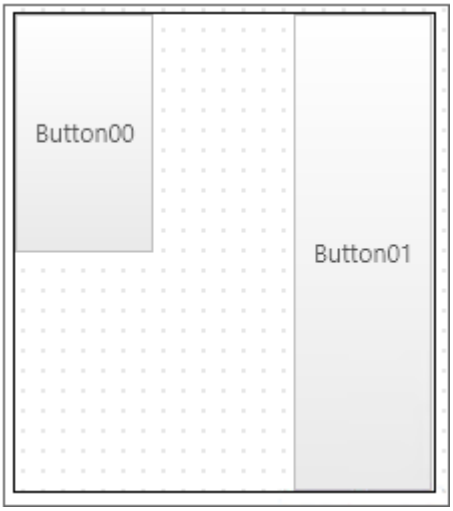


If you check "Display table guidelines" in [Options > Form Design > Layout > Table Layout], the line dividing the table cell area and the value specified as the tabletemplatearea property value are always displayed.

When placing the Button component, if you adjust the size so that it spans over two or more cell areas, you can place the component as if two or more cell areas were merged (Becomes the selected state only when the area to be selected by dragging two or more cells is 50% or more of each cell area).

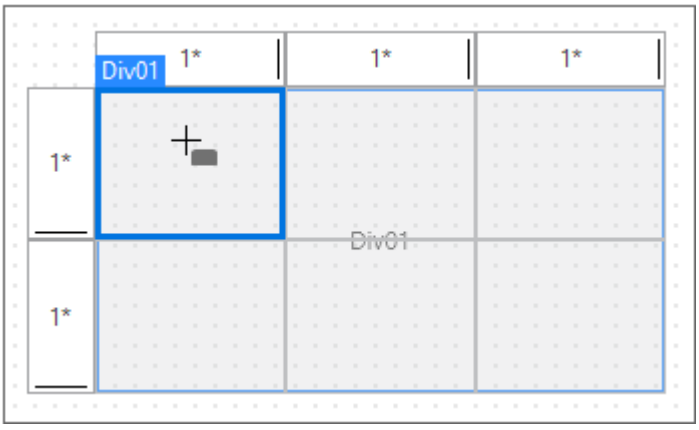


Regardless of the default size of the component, it is resized to fill the cell area (width and height property values are set to 100%). After placing the component, it can be resized.



In the case of container components such as Div, double click to display the editing screen where you can use the Reset Table feature which is available only it the editing mode.

Even when you are not in the editing mode, the guideline information is displayed for reference when the mouse cursor moves over a container component.



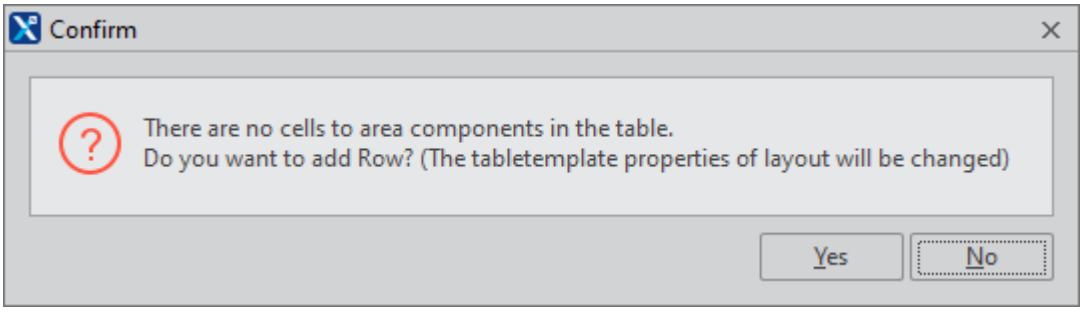
Add, insert, delete columns and rows

Columns or rows can be added, inserted, or deleted in the context menu.

Add

Add columns or rows to the last position.

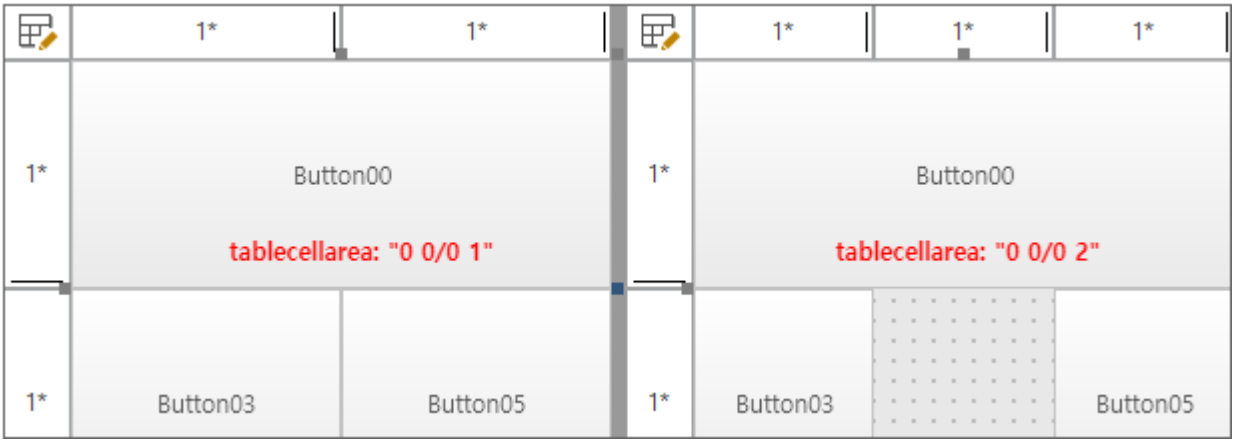
When components are placed in all cells, if you want to add a component to a non-cell area, a window is displayed, asking whether to add the component after adding a row. If no row is added, components are added overlapping the last cell.



Insert

Inserts columns or rows before the selected column or row.

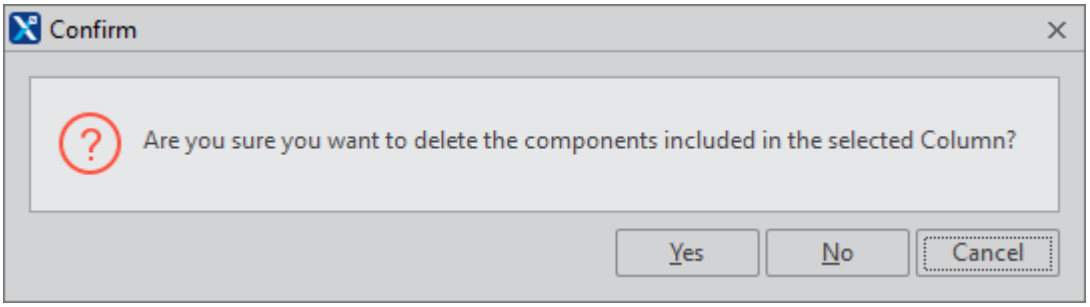
If a column or a row is inserted between the components spanning over two or more cell areas, the tablecellarea property value is changed to include the inserted column or row.



Delete

Delete the selected columns or rows.

When deleting a column or a row, if a component is placed in the corresponding column or row, a window is displayed asking whether or not to delete the component.



If a component is not deleted, the component may overlap or be placed in an invalid cell area depending on the position of the deleted component.

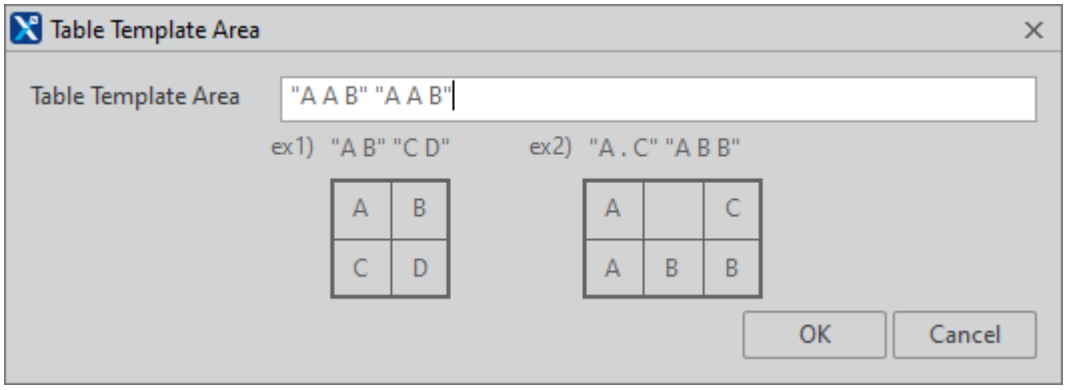
- When deleting the last column or row
  The component's tablecellarea property value becomes invalid and is placed to the right or the bottom of the table area.
  If the size of the component is set in %, it will not be visible on the screen because there is no cell to refer to.
- When deleting a column or row that is not the last
  The tablecellarea property value of the component placed in the deleted column or row is maintained, and the tablecellarea property value of the components placed in the next column or row is moved up by one. Therefore the component in the deleted column or row and the component in the next column or row are overlapped and displayed.
- When deleting a column or row that contains a component that spans over two or more cell areas
  The tablecellarea property value is changed to fit the remaining cell area, not including the deleted columns and rows.

Placing components in the area set with the same cell name

You can set the same cell name for two or more cells and place the component in the area set with the cell name. The cell area with the same cell name must be in the form of a rectangle composed of consecutive cells.

1. In the properties window, click the button in the tabletemplatearea property item.

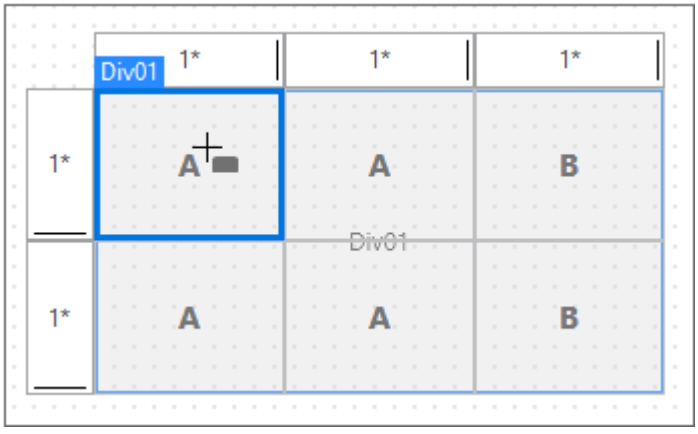
   The Table Template Area window will open and display a simple example. Set the desired tabletemplatearea property value according to the set column or row. You can also copy and use the values in the example.



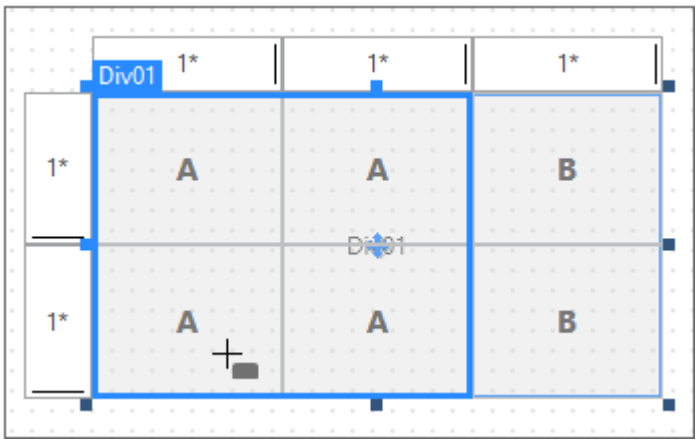
   Enter the tabletemplatearea property value as "A A B" "A A B".

2. Place the button component.

   When you drag it to the Div component, a virtual table cell area is displayed and the cell name is displayed.



3. If you press the Shift key while the mouse pointer is located on a specific cell, the cell with the same name is displayed as selected.



4 When you select a cell with the same name and click the mouse button, the component is placed in that area.

The tablecellarea property value of the component placed as such is set to the cell name. In the example above, the tablecellarea property value is "A"

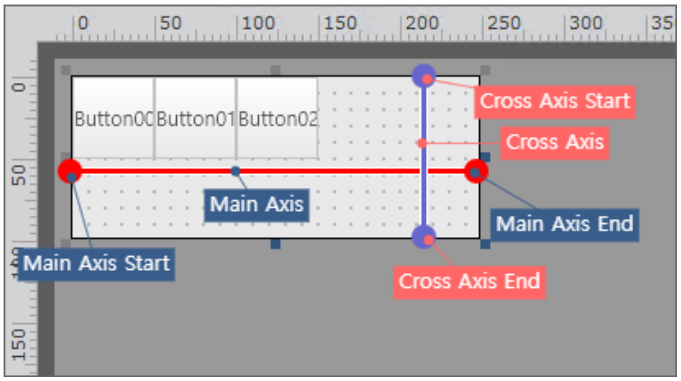
14.2.4 Setting component placement standards and spacing

When placing a component on the screen, it is not placed from the starting position. Function buttons are placed on the right side of the screen at regular intervals to be easily recognizable.

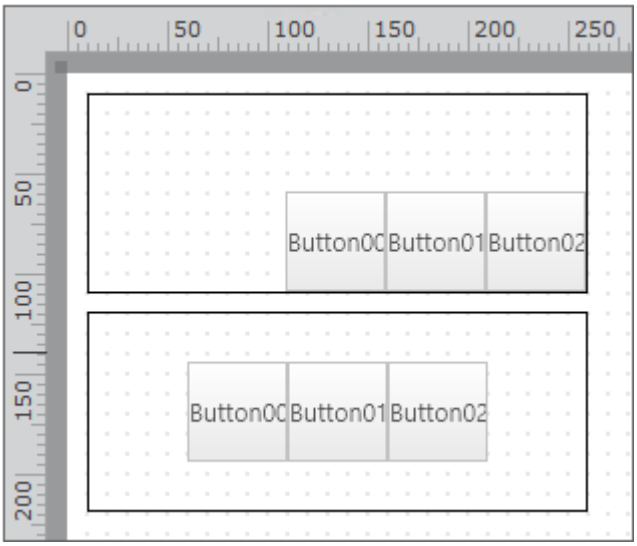
Standards for placing components

If the type property value is “horizontal,” the horizontal axis becomes the main axis, and if it is “vertical,” the vertical axis becomes the main axis. The axis that intersects with the main axis is the cross axis. If the property value is “horizontal,” the vertical axis is the cross axis and if it is “vertical,” the horizontal axis is the cross axis.

If the property values of flexmainaxisalign and flexcrossaxisalign are not set, they are set to "start" as follows.

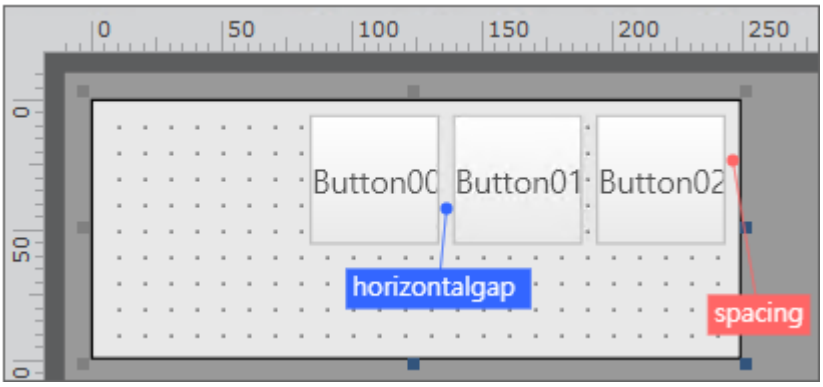


Setting the property values of flexmainaxisalign and flexcrossaxisalign to "end" and "center" will change the placement of the component. The order of placement will not change.



Setting the component spacing

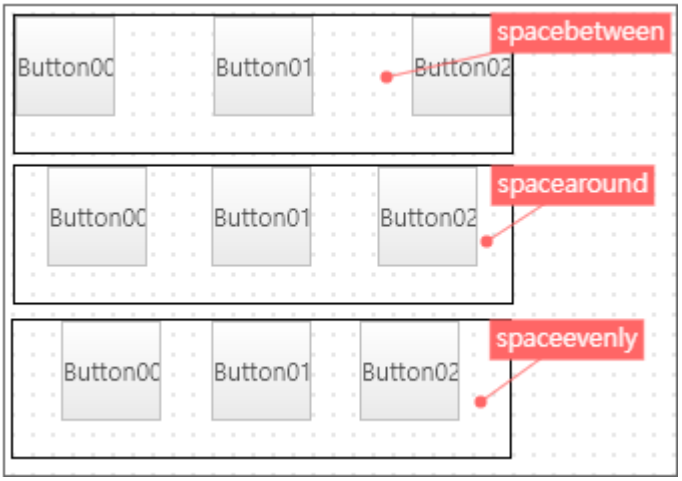
This is what it looks like when the spacing property value is set to "5px" and the horizontalgap property value is set to 5. The spacing property sets the spacing between the layout container and the components. The spacing between each component is set with the horizontalgap property.



The spacing can be set in the flexmainaxisalign property instead of setting a specific value for the spacing.

flexmainaxisalign	Placement method
spacebetween	(1) Place the first and last components at the start and end points of the main axis. (2) Set the remaining margin to the spacing between each component.
spacearound	Divide the total margin by the number of components to set the space in front and behind the components. Spacing between the components is processed as twice the spacing between the layout container and the components.
spaceevenly	Divide the total margin by the number of components + 1 to set the spacing between the components and the spacing between the layout container and the components.

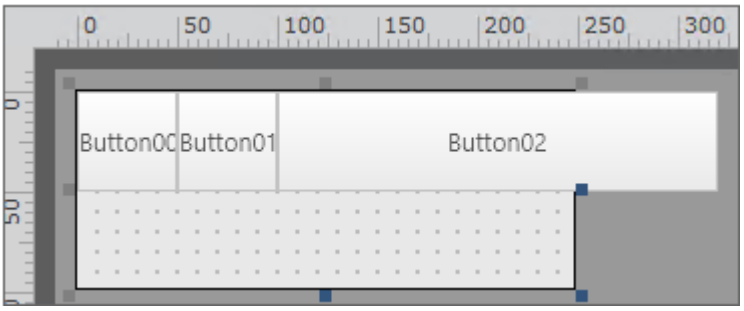
flexmainaxisalign	Placement method
	All intervals are processed as the same value



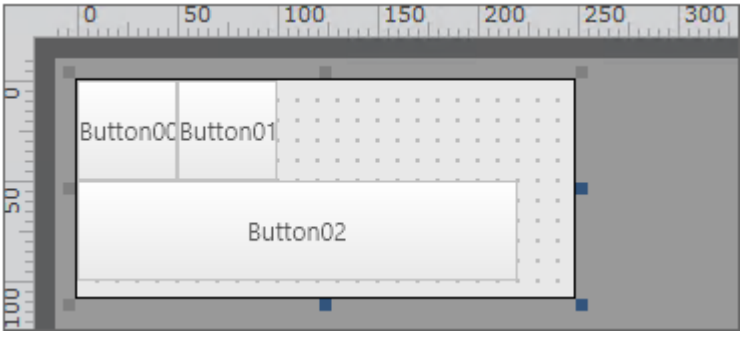
Handling line breaks

If the component's flexshrink property value is not set, it is set to 1 by default. When the placed component exceeds the layout container area, the component is resized to prevent that.

However, if the component's width or height value is set in px and the flexshrink property value is set to 0, the component may exceed the layout container area. The scrollbar is displayed while running



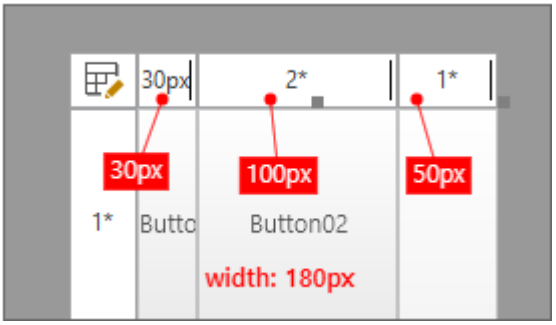
If the flexwrap property value is set to "wrap", components that exceed the area of the layout container are broken and placed on the next line.



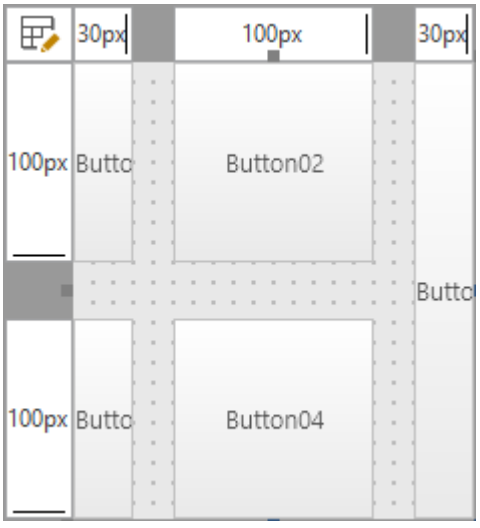
Setting the table cell spacing

When setting the tabletemplate property value, you can select one of three units: "px", "%", and "*". When setting the column and row by clicking one the Create Table button in the Table template Editor, each value is set to "1*".

If there are columns and rows that use "*" as a unit, the remaining margin is divided and set as the size. For example, if the tabletemplate property value is "1* 1* / 30px 2* 1*", the width of the first column is set to 30px, and the widths of the other two columns are set to 2/3 and 1/3 of the remaining margins.



To set the table cell spacing, there must be no column or row that uses "*" as a unit. In the figure below, the tabletemplate property value was set to "100px 100px / 30px 100px 30px" and the tablecellalign property value to "spacebetween spacebetween”



14.2.5 Resizing Based on Placed Components

Div and Panel components support the fittocontents property, which allows automatic adjustment of the width and height of the internal content area based on the component placed.

15.

Binding Data, Creating Events and Editing Contents

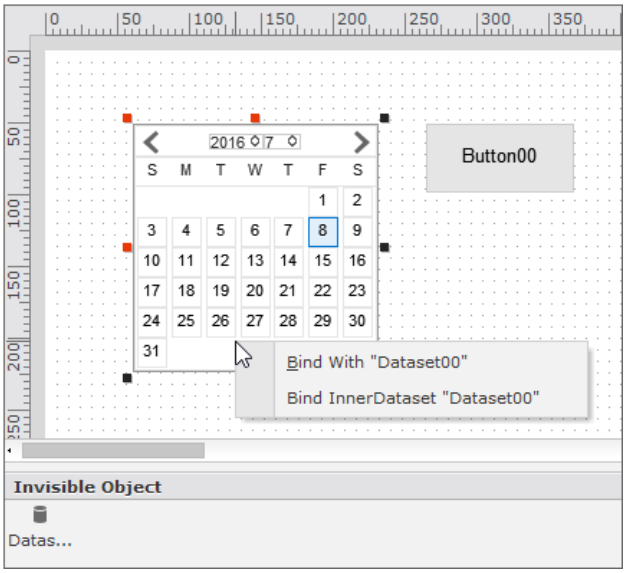
15.1 Data Binding

You can connect the data of the Dataset object to the component to display text or set the property value of the component. Then, the value edited in the component is reflected as the data of the Dataset object. Connecting data to components is called data binding. Like the Grid component, you can connect all data of the Dataset object, or you can connect only the specific column data corresponding to the selected Row as with the Edit component.

15.1.1 Configuring Data Binding

Data can be bound by dragging and dropping the Dataset object created when designing the form onto the component or by selecting the binding-related property in the property window.

When the Dataset object is dragged and dropped onto a component, the format property value setting, Bind InnerDataset, or Bind Item Editor function is called according to the component type. You can specify the properties and column information to be connected in each setting window.



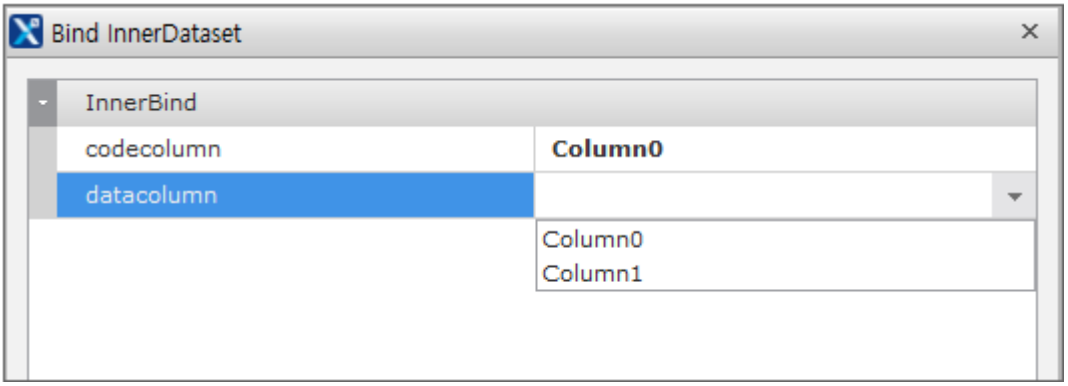
formats Property Value Setting

Components with binddataset, formats properties like Grid, ListView components automatically set the formats property value according to the Dataset object when dragging and dropping the Dataset object or setting the binddataset property value in the property window.

The formats property value can be modified in the desired format in the Grid Contents Editor or the ListView Contents Editor.

Bind InnerDataset

When a Dataset is bound to components that can display data in a list form, such as List Box or Combo, the Bind InnerDataset is accessed because it can designate the Dataset column to use in a list.



Properties	Description
codecolumn	Designates column that is used internally
datacolumn	Designates column that is displayed to the user in the component

The columns that can be set may vary depending on the component. Please refer to the table below.

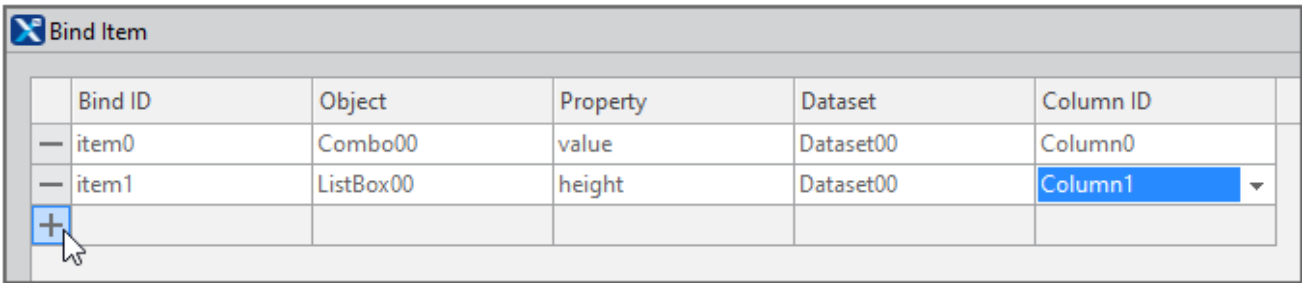
Component	Properties Connected to Columns
Combo, ListBox, Radio	codecolumn, datacolumn
Calendar	backgroundcolumn, bordercolumn, datecolumn, textcolorcolumn
DateField, DateRangePicker, PopupDateRangePicker	backgroundcolumn, bordercolumn, datecolumn, hintingcolumn, readonlycolumn, textcolorcolumn
Menu, PopupMenu, TrayPopupMenu	captioncolumn, checkboxcolumn, enablecolumn, hotkeycolumn, iconcolumn, idcolumn, levelcolumn, userdatacolumn



TrayPopupMenu does not use the Bind InnerDataset window, but rather it sets innerdataset in the Tray setting screen and sets each column connection property.

Bind Item Editor

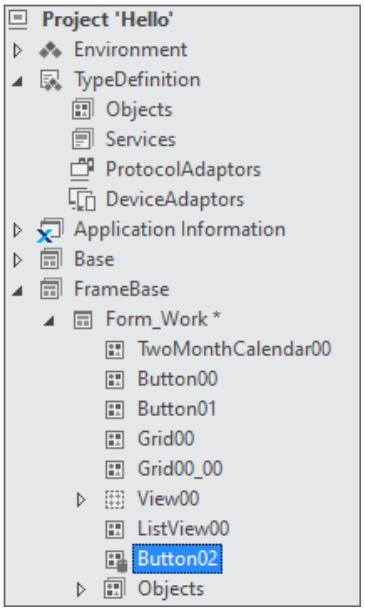
For components that have the innerdataset property when dragging and dropping the Dataset object, you can select Bind InnerDataset or Bind Item. For components without the innerdataset property, Bind Item Editor is executed immediately.



	Properties	Description
❶	Bind ID	User classification ID
❷	Object	ID of component that Dataset is bound to
❸	Property	Component property that Dataset is bound to
❹	Dataset	Dataset that is bound to the component
❺	Column ID	The Dataset Column ID that is bound to the component

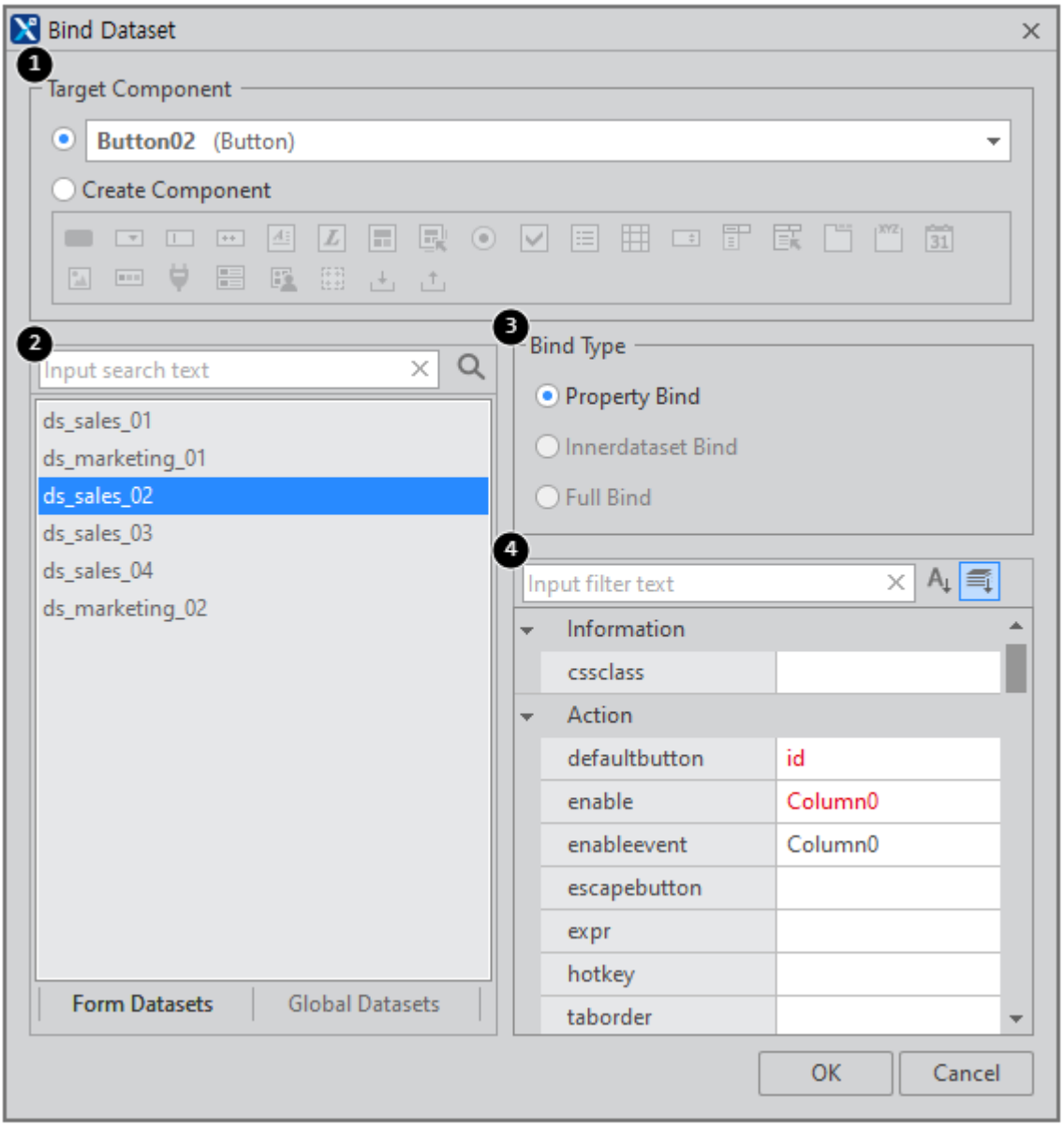
When a Dataset is bound to a Form or component, it is displayed in the Project Explorer window on the item for easy verification, as shown below:

When a Dataset object is bound to the innerdataset property of a component that has an innerdataset property, the icon of that component is not displayed differently in the Project Explorer. Binding status icon display is only applicable when set to Bind Item Editor.



Bind Dataset Editor

If you select a Form object or component on the design screen and select "Bind Dataset" from the context menu, you can configure multiple Dataset objects and binding settings.



	property	Description
1	Target Component	When a component is selected and the Bind Dataset function is executed, the selected component is displayed. You can select another component or select "Create Component" and create a new one.
2	Dataset	A list of available Datset objects. The Dataset object in the Form and the Dataset object set as Global Dataset are displayed.
3	Bind Type	Depending on the target component, selectable Bind Types are activated.

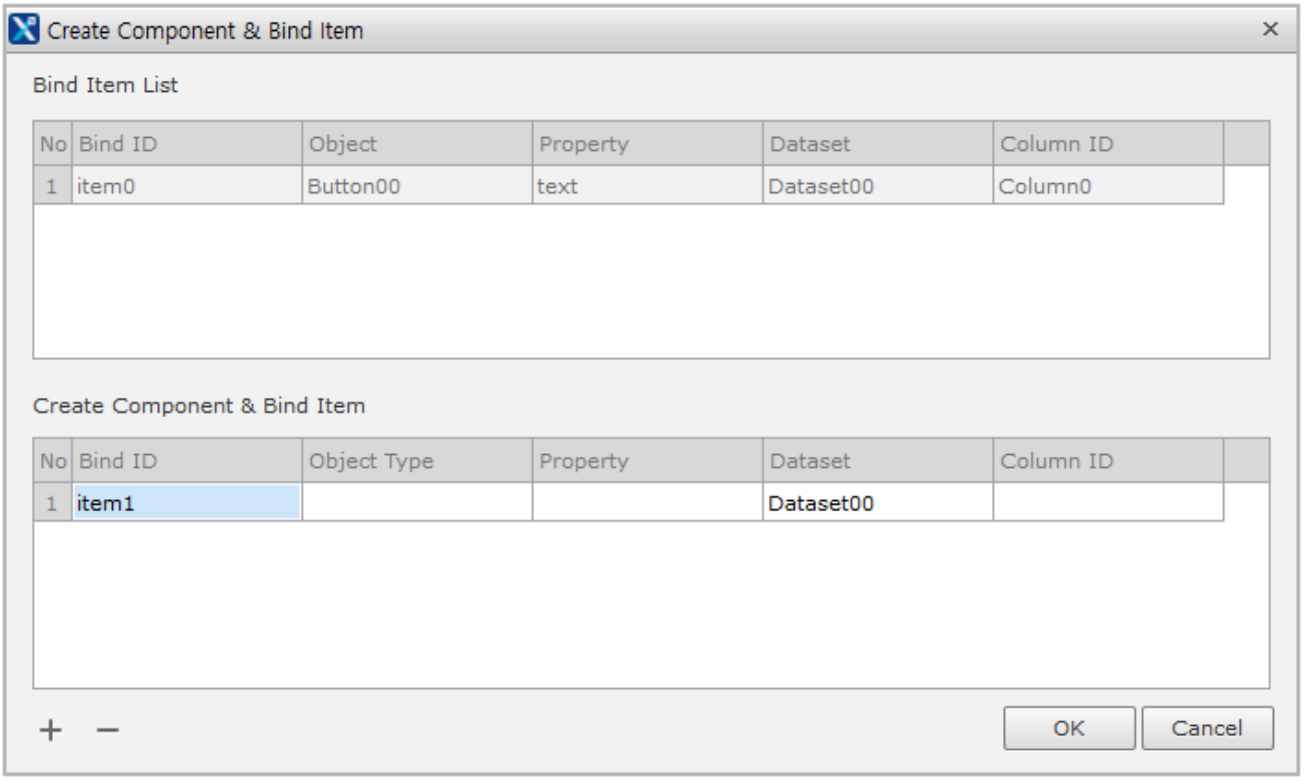
	property	Description
4	Property	A list of properties that can be set according to the Bind Type is displayed. Column ids are displayed in black if they are bound to a Dataset object that is currently selected, and items bound to an unselected Dataset object are displayed in red.

15.1.2 Adding & Editing Bind Item

Create Component & Bind Item Editor

Like the Bind Item editor, this editor is also used to bind a Dataset to a component. However, the difference is that it is used when you want to bind a Dataset while creating the target component. Through the Bind Item editor, in contrast, you can bind a Dataset only to existing components .

If you drag and drop a Dataset object onto the Form area while pressing the Ctrl key, you can open the edit window.

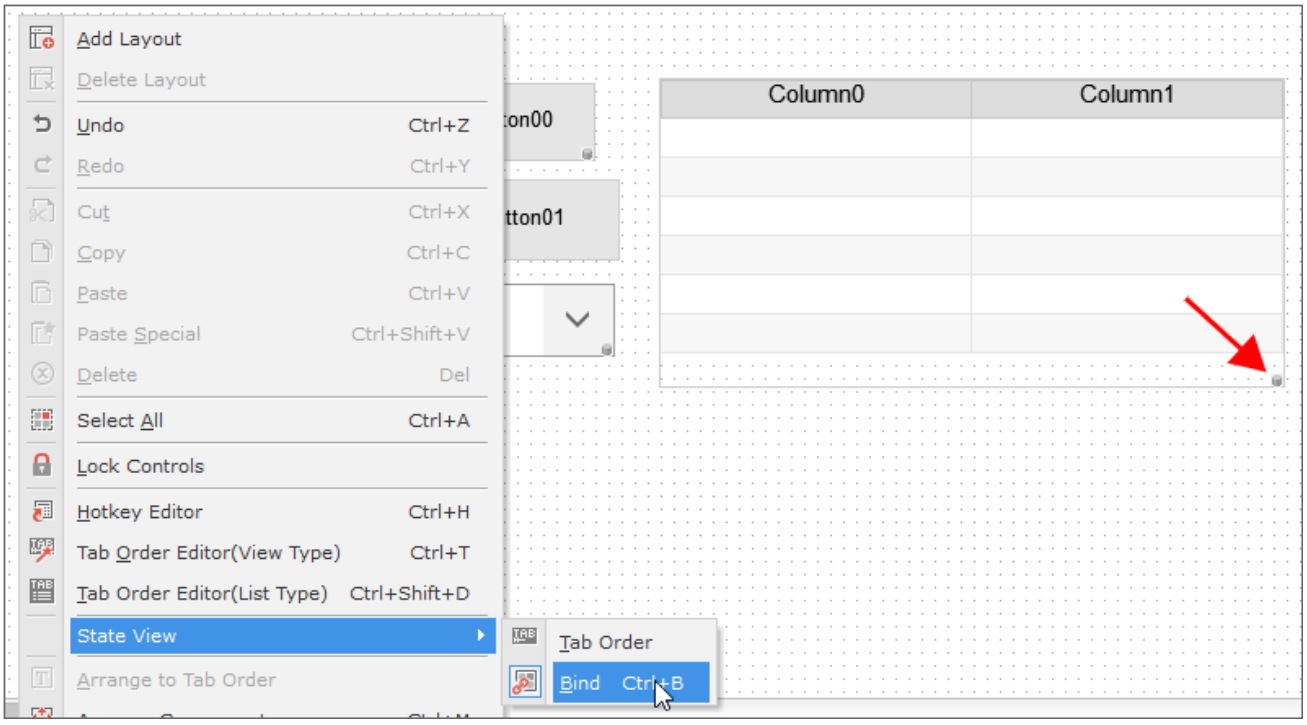


The items in the Bind Item List represent the values you added through the Bind Item Editor. Those items are provided only for information so that you cannot modify those through the Create Component & Bind Item editor.

Displaying & Editing Bind State Icon

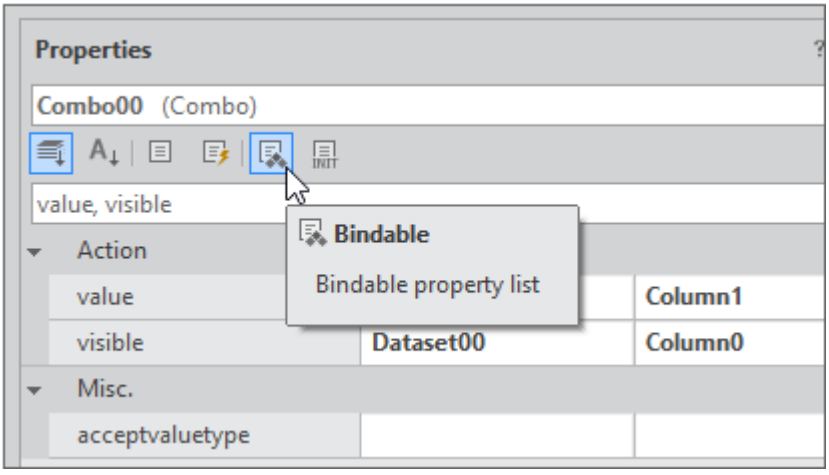
You can see the state of Dataset binding by right-clicking the Form design pane to open the context menu and selecting [State View > Bind]. In this mode, you can identify to which components Datasets are bound.

For the bound component, the Dataset icon is displayed in the lower right corner, and when you click the icon, Bind Item Editor is displayed.



Editing in the Property Window

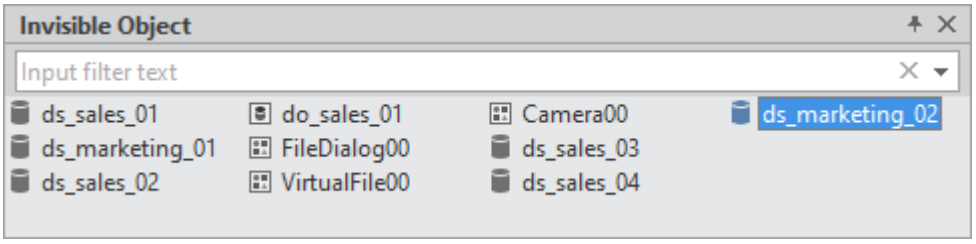
You can add or edit the Bind Item for each property by selecting the component and clicking the [Bindable] icon in the property window.



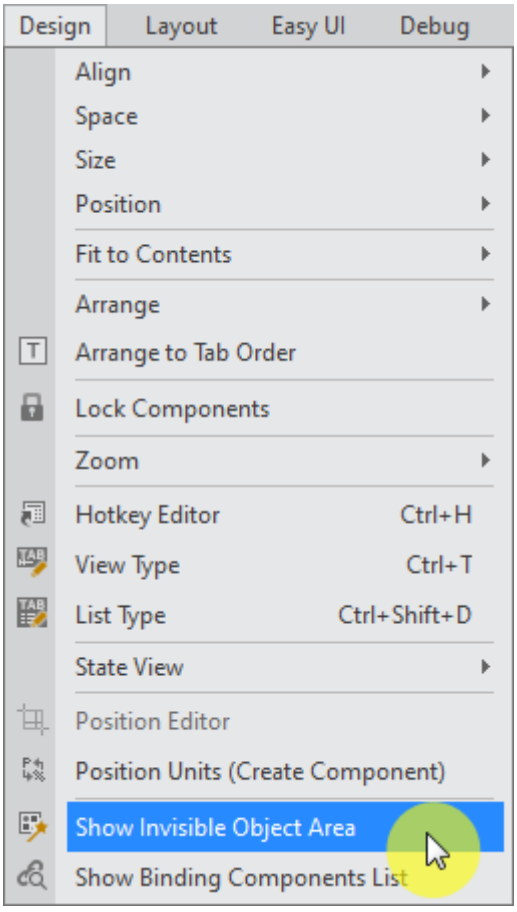
15.1.3 Checking Data Binding Information

Invisible Objects Pane

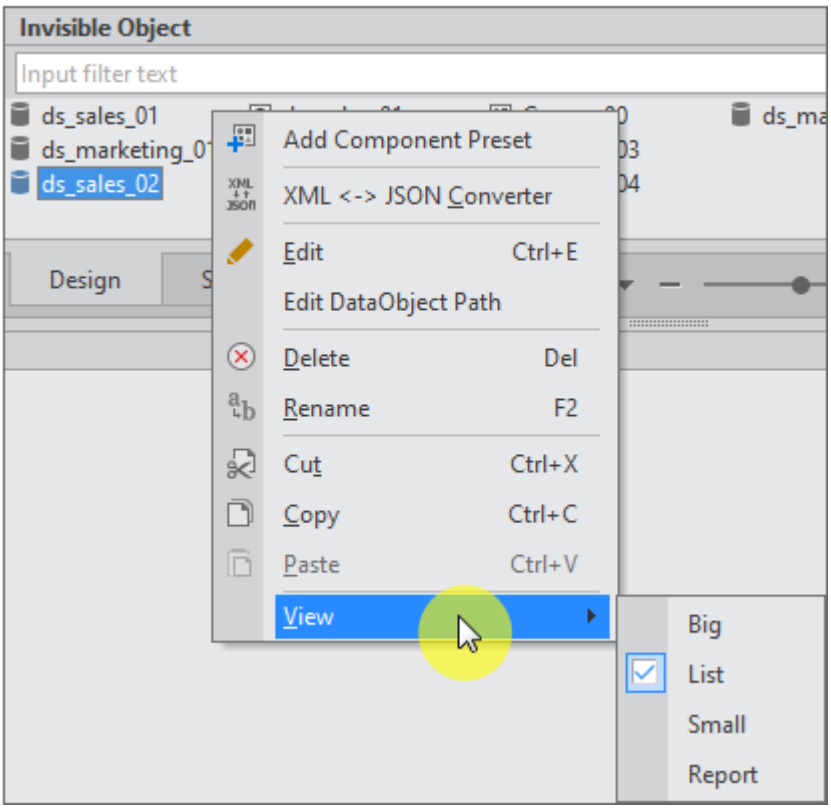
Nexacro Studio provides the Invisible Object area where you can manage invisible objects, which you cannot see physically but use in developing an app. The Invisible Object area is provided in the form of a dockable window.



If the Invisible Object window is not displayed, open the design menu and select [**Show Invisible Object Area**].

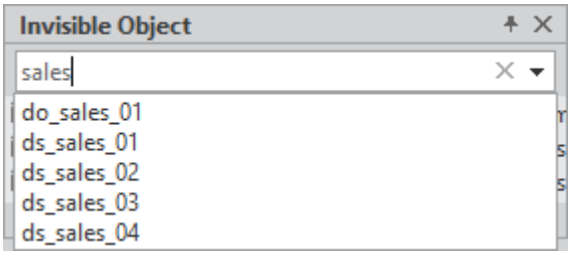


You can change the icon display types by opening the context menu from the Invisible Object window.

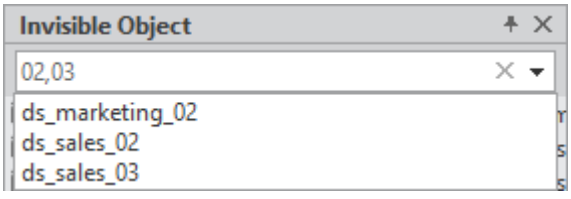


If there are many items in the Invisible Object window, a filtering function is provided to find the desired item immediately. Displays objects whose id matches the input text for filtering in a drop-down list. The list is sorted alphabetically by the id property value.

When you select the desired object from the list, the object becomes selected.

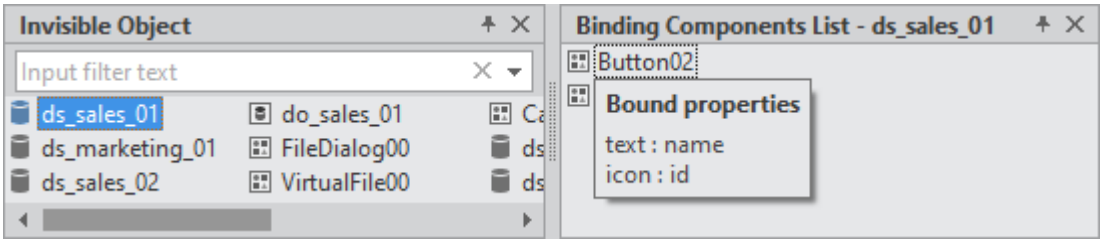


You can use multiple filtering entries using separators (spaces, commas, semicolons).



Binding Components List Pane

This pane shows the components that are bound to the selected Dataset in the Invisible Object pane. If you bring a mouse cursor over a certain component listed in the pane, a tooltip will appears, telling you the information on the relevant binding. Double-click the component item or select "Edit" from the context menu to run the Bind Dataset editor.



15.2 Contents Editor

15.2.1 Grid Contents Editor

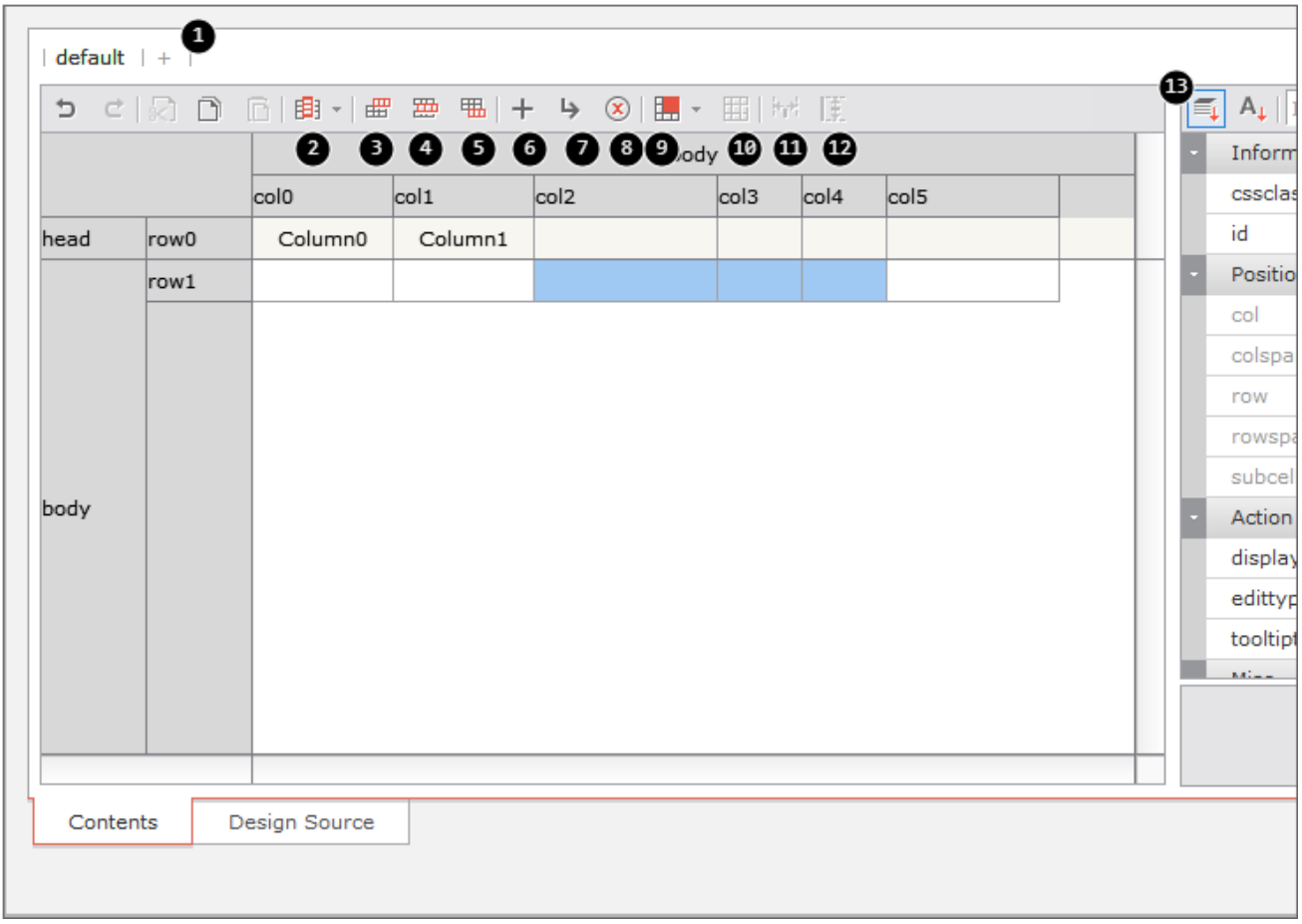
The Grid Contents Editor is used to edit the Grid's contents properties. There are many cell unit properties besides the properties that can be seen in the Properties window.

Access the Grid Contents Editor by double-clicking the **Grid** component on the Design Window.

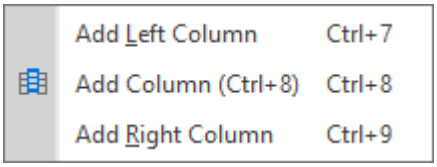
The Grid Contents Editor is composed of the main editor window and the Design Source window.

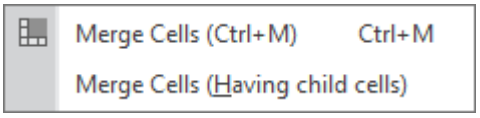
In the main edit window, you can add or delete a cell by using the popup menu and shortcut keys, or can edit properties. The main edit window displays the column index at the top, followed by the head, body, and summary.

Main Editor

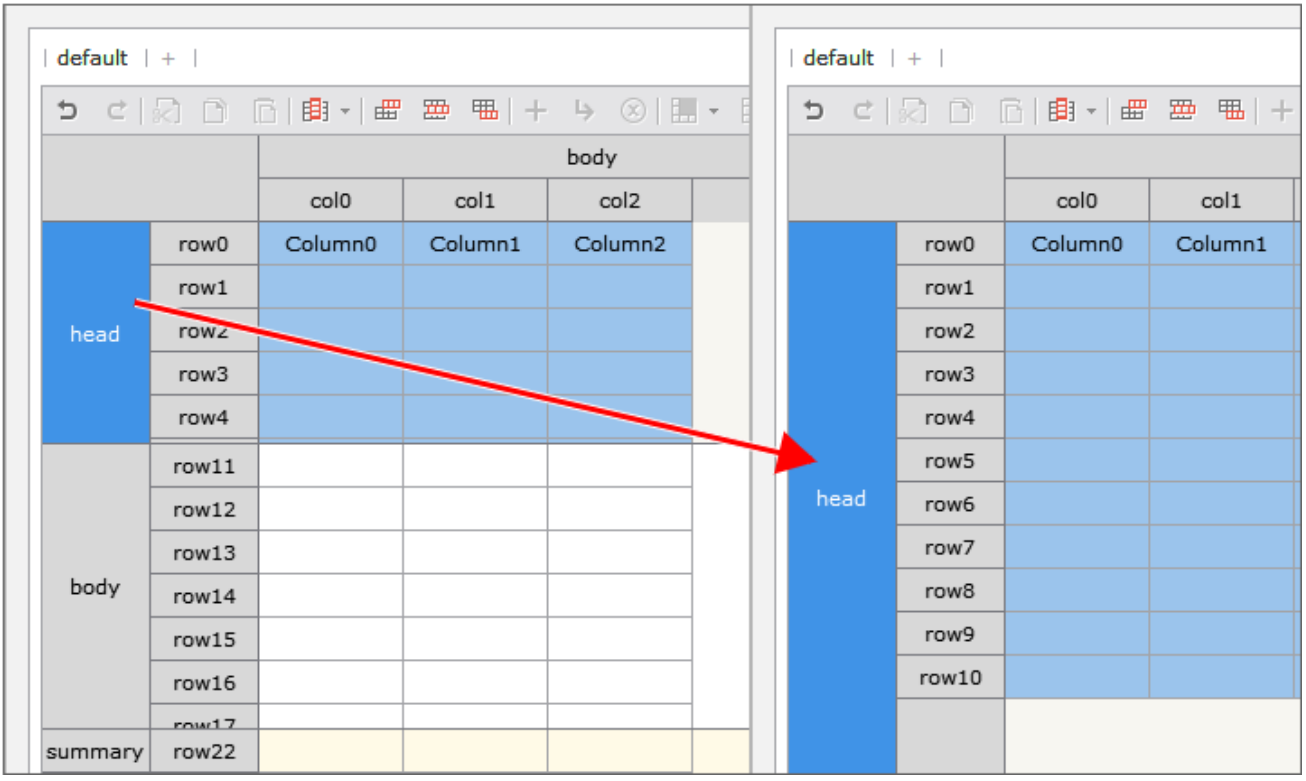


You can also find the same features at the context menu opened from the Grid Contents Editor.

	Icon	Description
1	Format	Adds or deletes a format
2	Add Column	 Adds a column. While the default function is to add a body column, you can also add a left or right column.
3	Add Head Row	Adds a head row
4	Add Body Row	Adds a body row
5	Add Summary Row	Adds a summary row
6	Add	Adds same ones after the selected rows or columns. If cells are selected, you need to choose either columns or rows.
7	Insert	Adds same ones before the selected rows or columns. If cells are selected, you need to choose either columns or rows.
8	Delete	Removes the selected rows or columns.

	Icon	Description
		If cells are selected, you need to choose either columns or rows.
9	Merge Cells	 Combines two or more cells. While the default function is to integrate cells, you can also combine cells with individual properties of the selected remain intact.
10	Split Cell	Separates merged cells
11	Same Width	Adjusts the width of the selected columns. You can specify the value on your own or adjust the value to the width of the last-selected column.
12	Same Height	Adjusts the height of the selected rows. You can specify the value on your own or adjust the value to the height of the last-selected row.
13	Properties pane	Shows the list of properties that a selected object has When selecting a cell in the head, body, or summary row, you can add or delete user properties from the context menu.

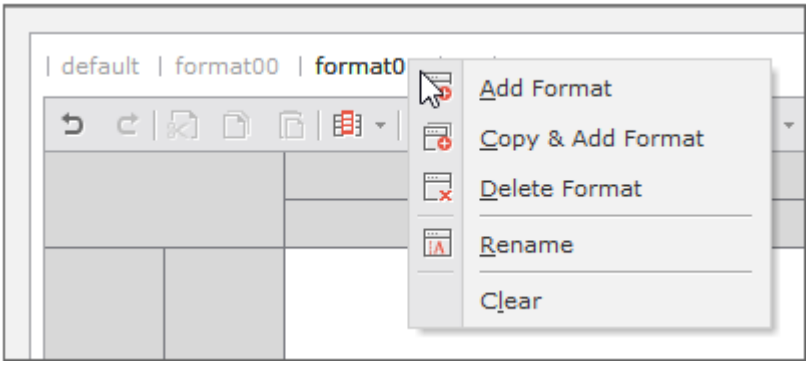
If there are too many rows, double-click one of the row category areas, which are located on the left end of the editor. The first double-clicking will single out the relevant band, and the following double-clicking will return to displaying all the bands.



Format

Sometimes, you need to look at the same data from various perspectives or hide some part of data due to the authorization issue. In these cases, you can set multiple formats for one Grid.

You can either add a new format or modify an existing one after copying it. You are required to specify the *formatid* property of a Grid object in order to devise settings specific to a certain format.



Design Source

The Design Source tab of the Grid Contents Editor shows the structure of a Grid in the form of XML. You can change source codes directly here.



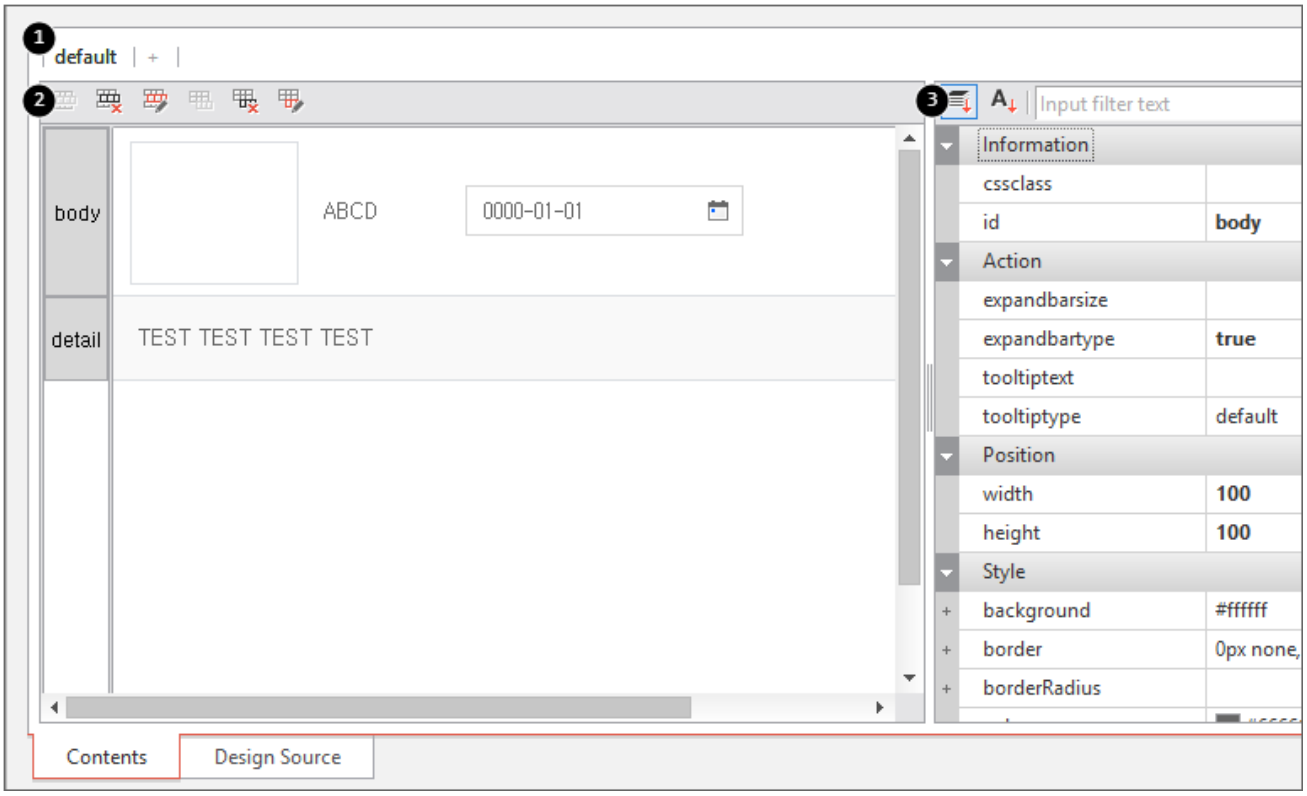
15.2.2 ListView Contents Editor

A ListView component has more properties than what we can see in the Properties pane, and those additional properties are affiliated with each band or cell. The ListView Contents Editor is used to edit such additional properties.

You can open the ListView Contents Editor by double-clicking a ListView on the design pane.

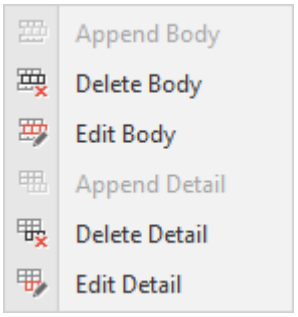
The ListView Contents Editor is composed of the main editor, band editor, properties pane and Design Source pane. With the main editor, you can add/delete the body or detail band or can edit properties by using the context menu or shortcuts. The column on the left side of the main editor can be divided into the body and detail bands. The properties pane on the right side presents property values for each band or cell.

Main Editor



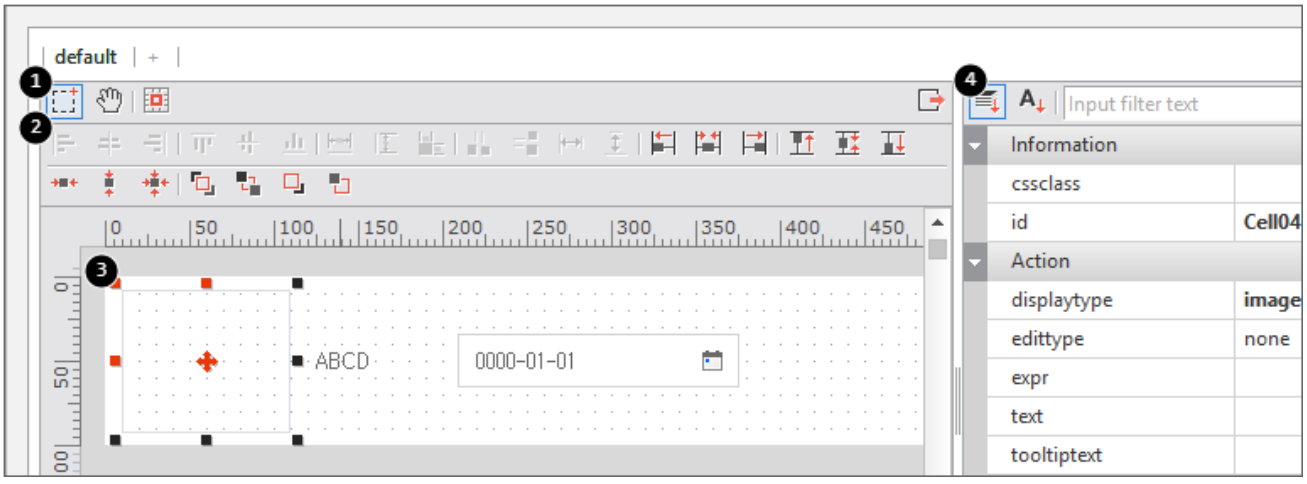
You can also find the same features at the context menu opened from the ListView Contents Editor.

Call-out		Description
1	Format	Adds or deletes a format
2	Toolbar	This toolbar features icons for adding or deleting the body and detail bands. The toolbar changes its menu when the editor switches to the band editing mode.

Call-out		Description
		You can select the same features from the context menu.  Append Body (or Detail): adds the body (or detail) band. These menu items will be inactive if relevant bands have been added already. Delete Body (or Detail): deletes the body (or detail) band Edit Body(Detail): Switches to the mode for editing the body (or detail) band. You can edit cells in each band.
3	Properties pane	Shows the properties of a selected band (body or detail) and allows you to edit them

Band Editing Mode

You can enter the band editing mode from the main editor by selecting the menu item [Edit Body] or [Edit Detail].



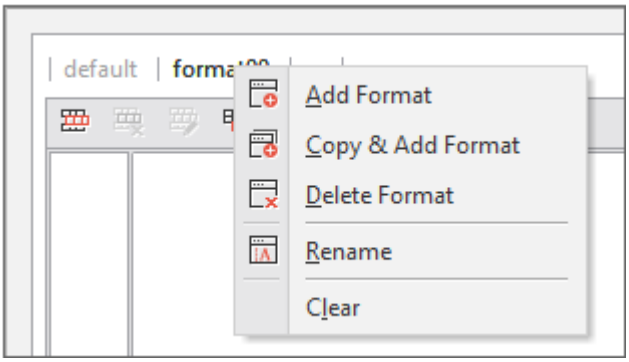
Call-out		Description
1	Toolbar (Cell)	This toolbar features icons for selecting cells, moving the canvas and adding a cell. Select: selects one or more cells Move: moves the position of the background canvas

Call-out		Description
		Cell: adds a cell Exit Editor: switches back to the main editing mode
2	Toolbar (Align)	This toolbar features icons for aligning cells. See Align to learn how to use those features.
3	Editing pane	You can arrange cells on the canvas and change their sizes, just like you do with components on the Form design pane. You can also use the functionality employed to edit a Form: position-related properties, tracker, Fit to Content, PositionBase and guides. See Form Designing | Positions and Guides to learn how to use those features.
4	Properties pane	Shows the properties of a selected cell and allows you to edit them You can add or delete user properties from the context menu.

Format

Sometimes, you need to look at the same data from various perspectives or hide some part of data due to the authorization issue. In these cases, you can set multiple formats for one ListView.

You can either add a new format or modify an existing one after copying it. You are required to specify the `formatid` property of a ListView object in order to devise settings specific to a certain format.



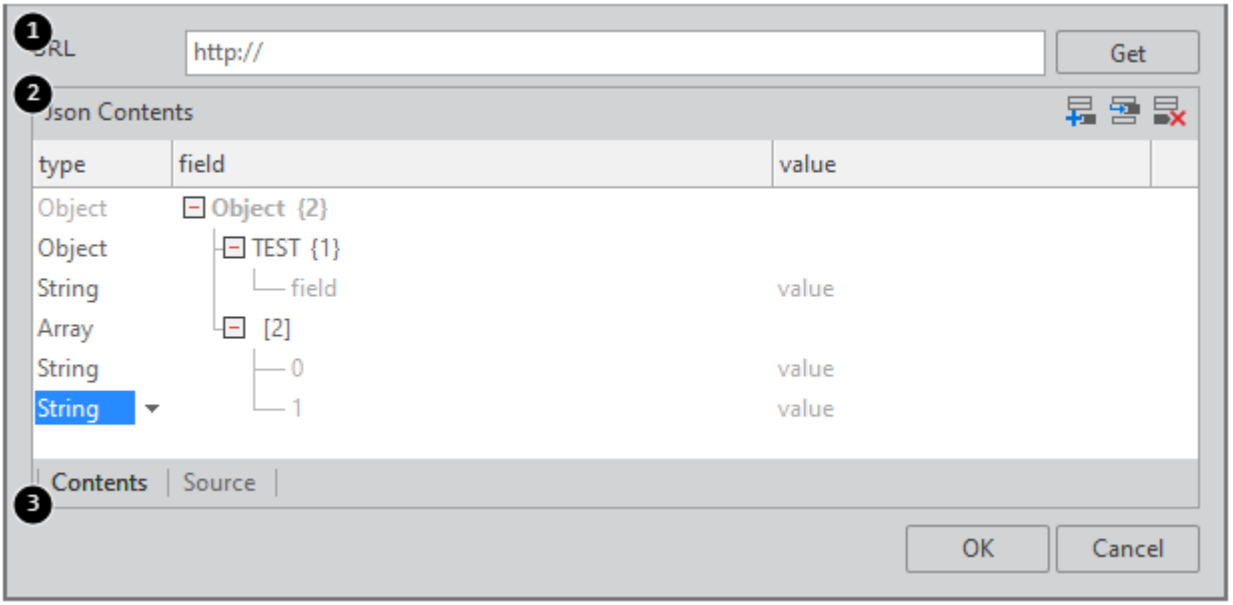
Design Source

The Design Source tab of the ListView Contents Editor shows the structure of a ListView in the form of XML. You can change source codes directly here.



15.2.3 Json Contents Editor

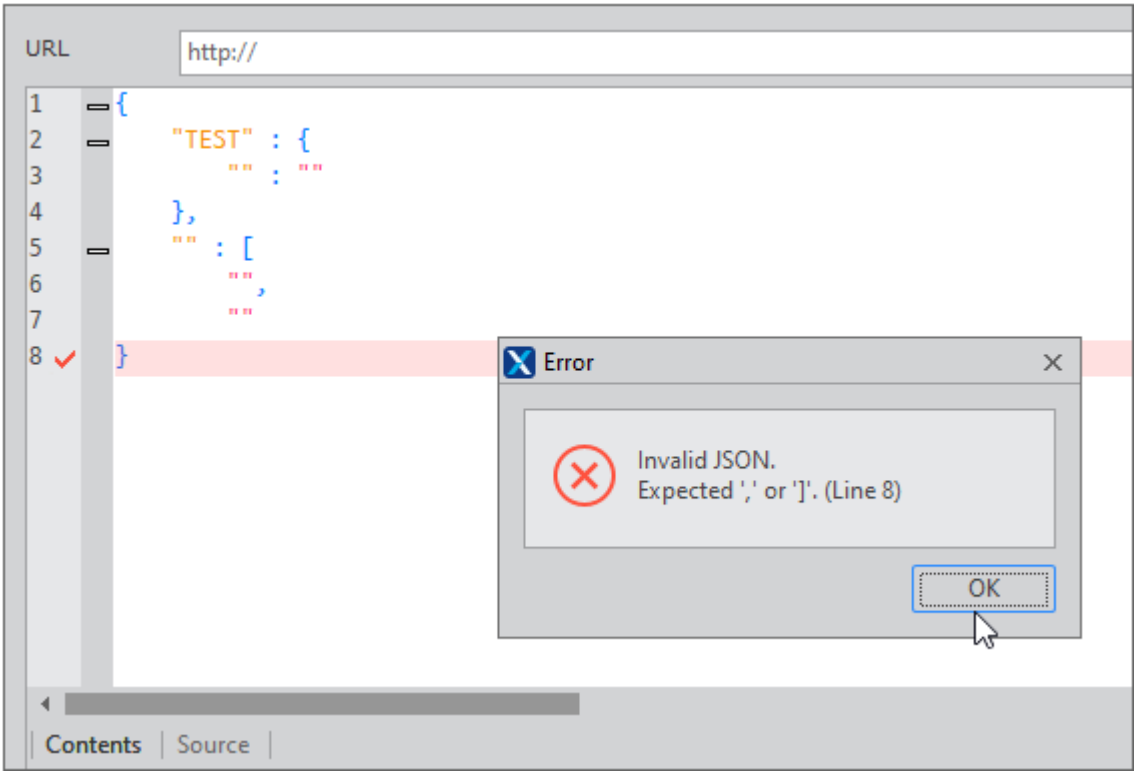
DataObject □□□□□□ contents □□□□ true□□ □□□□□□ ContentsInfo □□ contentsformat □□□□ "json"□ □□□□□ □□ □□ □□ □ □□□ □ □□□□.



	□□	□□
1	URL	URL □□ □ [Get] □□□ □□□□ □□ JSON □□□□ □□□ □□□□□.
2	Json Contents	JSON □□□□ □□□□□. type □□□□ □□ □□ □□□□ □□□ □ □□□□.

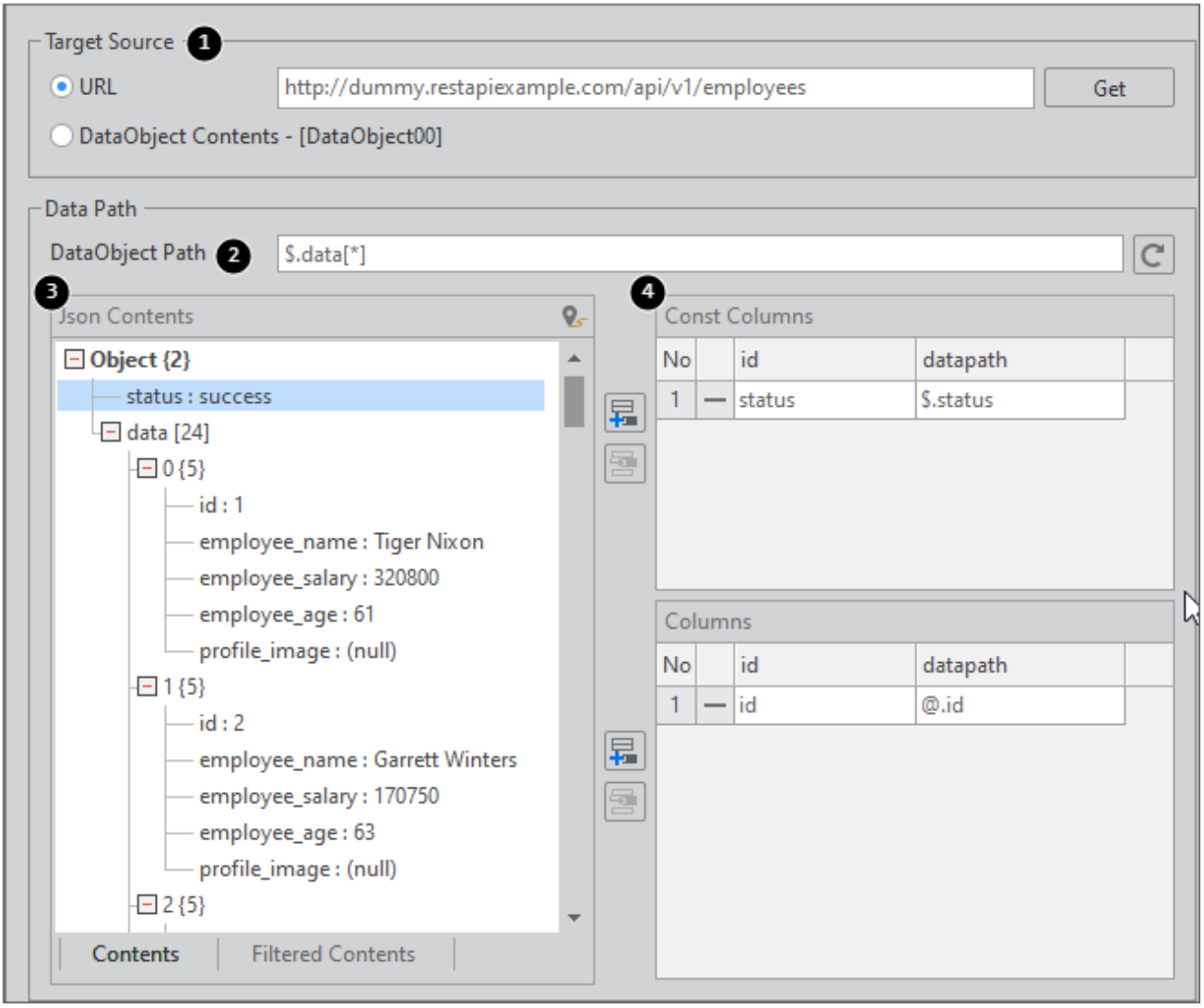
	□□	□□
3	□□ □□	Contents □ □□ □ □□ □□□ □□□□ □□□□□. Source □ □□ □ □□ □□□ □□□□ □□□□□.

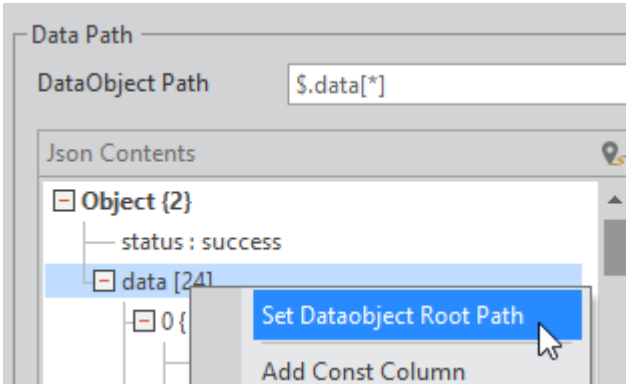
Source □□□ □□ □□ □ □□□ □□□ □□ □□ □□ □□□ □□□□□.



15.2.4 Data Path Editor

Dataset □□□□□□ binddataobject □□□□□ DataObject □□□□□ □□□□ □□□□□ dataobjectpath □□ □□ □ □□□ □ □□□□.

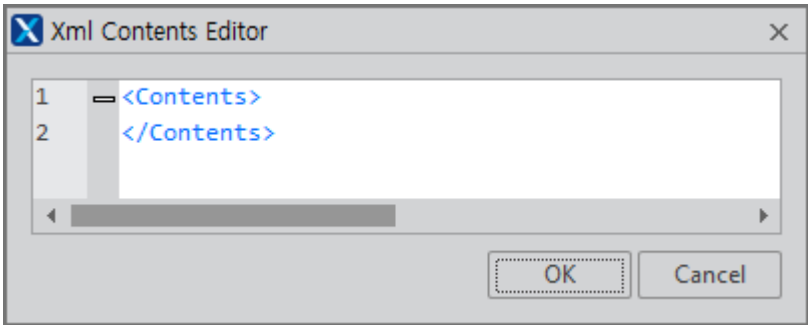


	□□	□□
1	Target Source	binddataobject □□□□□ □□□ DataObject□ □□□□□. DataObject □□□□□ contents □□□□ □□ □□□□ □□ contents□ □□□□ □□□ □□ □□□□ URL □□□□ □□□□ □□□□ □□□ □ □□□□.
2	DataObject Path	Dataobject Root Path□ □□□□□. Json Contents □□□□ □□□ □□□ □□□ □ □□□□. Path □□ □ [Refresh] □□□ □□□□ [Filtered Contents] □□ □□□□ □□□□ □□□□□. 

	□□	□□
3	Json Contents	[Contents] □ □□ □ □□ □□□ □□□□ □□□□□. [Filtered Contents] □ □□ □ □□ □□□ □□□□ □□□□ □□□□□.
4	Columns	Json Contents □□□□□ □□□ □□□ □□ datapath □□□ □□□□□.

15.2.5 XML Contents Editor

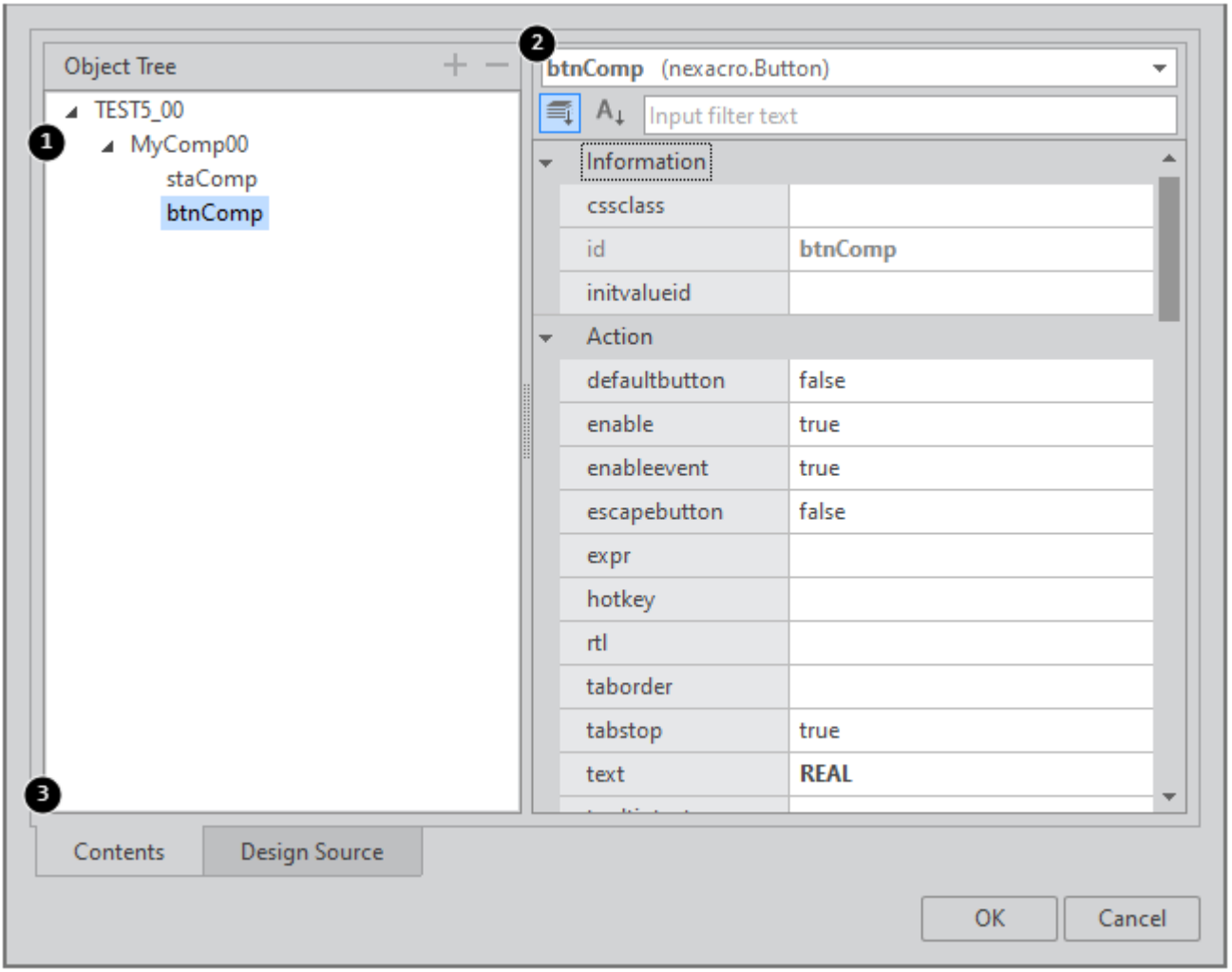
This is a content editor that can add basic XML code. It can be edited when the contents property value is true like the Plugin component, and when the related property of the object whose ContentsInfo tag contentsformat setting value is "xml" in meta-info is selected.



15.2.6 Basic Contents Editor

This is a content editor that can generate the XML code according to the specified format. It can be edited when the contents property value of the object is true and the related property of the object whose ContentsInfo tag contentsformat setting value is "ObjectItem" in meta-info is selected.

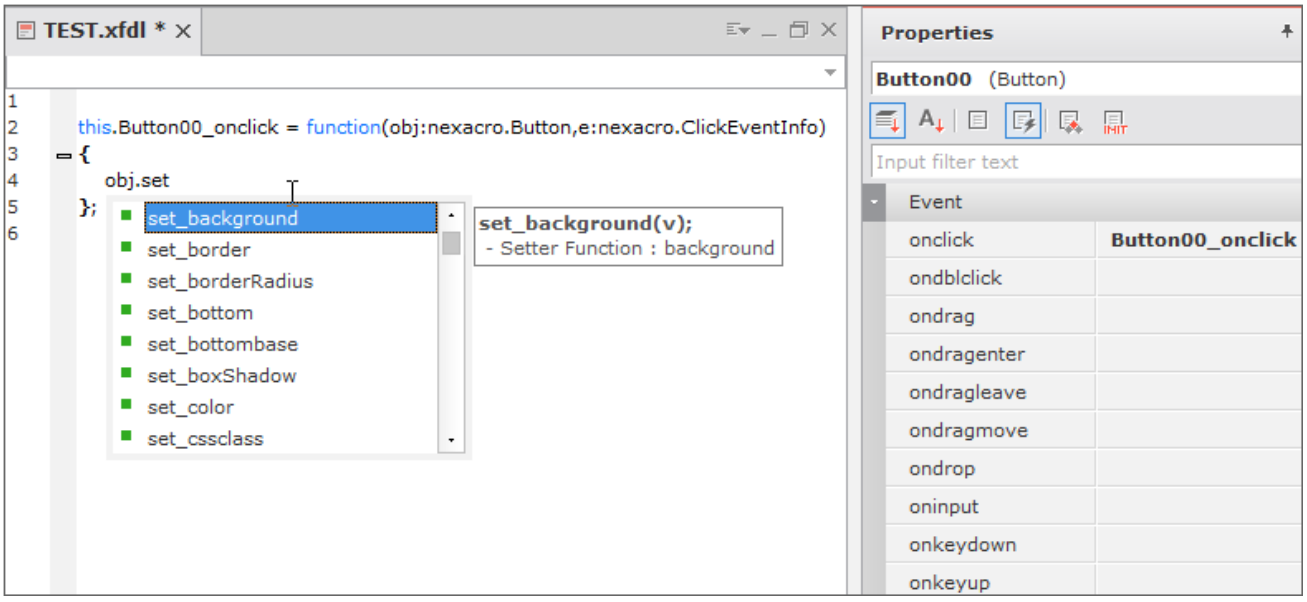
The type and number of items that can be added depend on the typename, maxoccurs, and minoccurs setting values defined in the object.



	Property	Description
1	Object Tree	Can add sub-items according to maxoccurs, minoccurs setting values.
2	Property Window	Can display and edit the property that can be set in the selected item. Displays the property corresponding to the class set as typename.
3	View Mode	Displays the data in the tree format when selecting the Contents tab. Displays the data in the code format when selecting the Source tab.

15.3 Adding Events

You can add an event to a component by selecting the component, and then **Event Edit Mode** from the Properties Window. You can enter the event directly, or create it automatically by double-clicking the relevant event listed in the Properties.

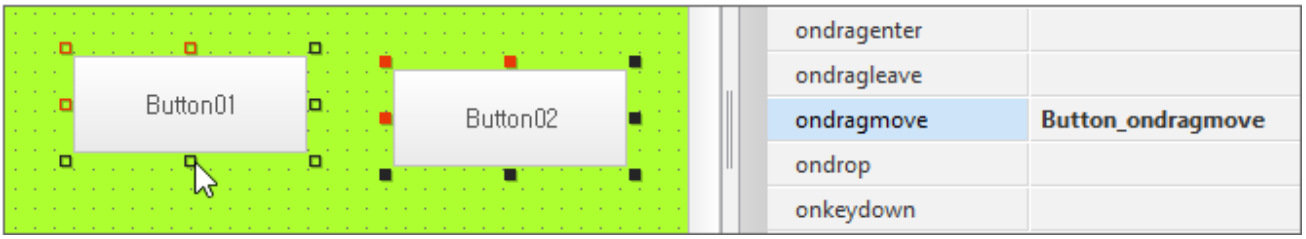


Double-clicking a certain component in a Form will generate a designated event automatically. The component will generate the *onclick* event if it has one. If it does not have *onclick*, it will generate another designated event.

The below-mentioned rules for creating an event handler will be applied if you first select a component or multiple components.

Component selection	Rules for creating handler name
One component is selected.	[Component ID]_[Event name]
Multiple components of the same type are selected.	[Component Type]_[Event name]
Multiple components of different types are selected.	Common_[Event name]

The below picture shows an example of creating an event handler. As seen in the below picture, first select two button components. Then, double-click either component on the Design panel or double-click a certain event in the Properties pane. Eventually, you will see an event handler created.



16.

Generating and Executing Project

To execute a project in the Nexacro, you are required to convert the project into the executable form. To test the converted project, furthermore, you need to choose an execute mode depending on which environment you want to test. Therefore, this chapter touches upon project build and execution.

16.1 Project Generation

16.1.1 Generation

If you have a file that is edited with Nexacro Studio, you need to convert that file into a JavaScript file for execution. In this manual, the action of such converting is called "generate."

Generate Application

This function generates an application.

You can use This function by accessing [**HOME - Generate & Launch - Generate - Application**] in the ribbon interface or [**Generate** > **Application**] in the command bar interface.

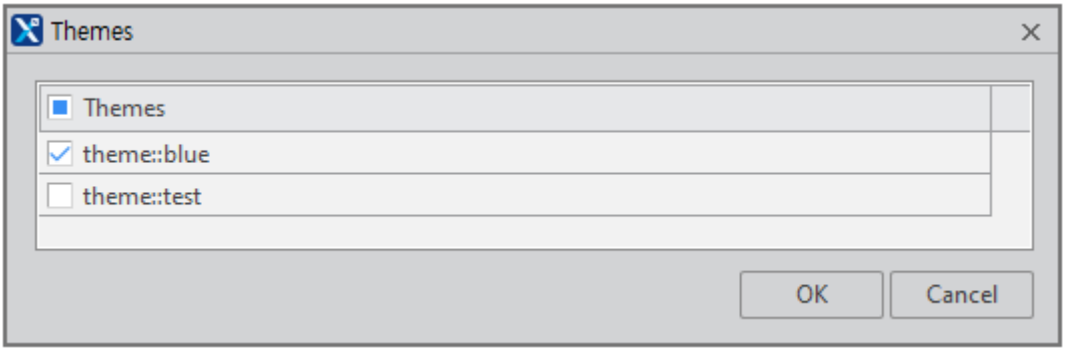


The files registered in **Services** in TypeDefinition do not belong to a certain application. Therefore, such files will always be generated regardless of a selected application.

Generate Theme

This function generates themes.

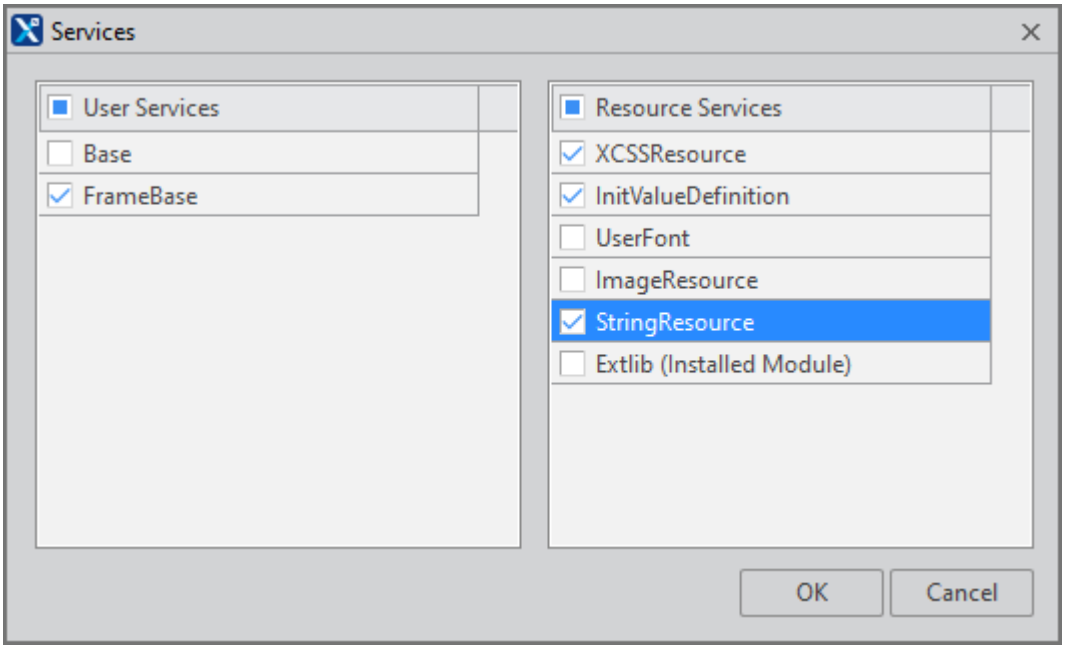
You can use This function by accessing [**HOME - Generate & Launch - Generate - Theme**] in the ribbon interface or [**Generate** > **Theme**] in the command bar interface.



Generate Service

This function generates the files registered in Services in TypeDefinition.

You can use This function by accessing [**HOME - Generate & Launch - Generate - Service**] in the ribbon interface or [**Generate** > **Service**] in the command bar interface.

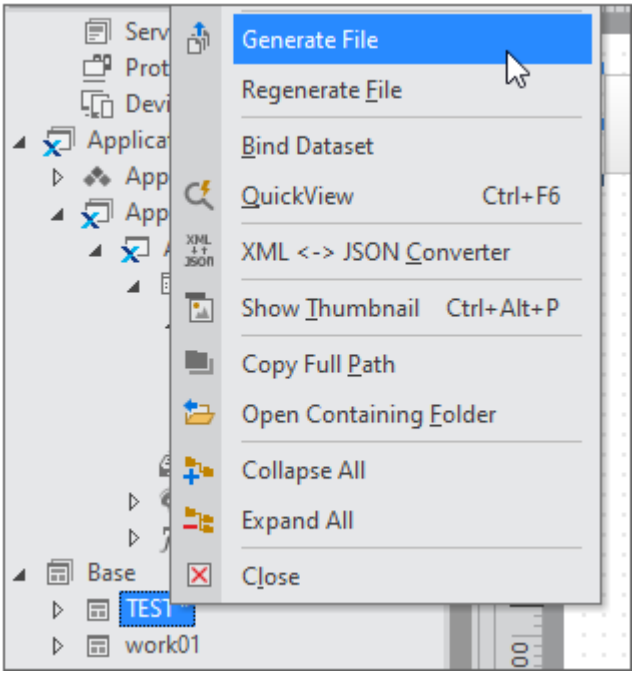


Generate File

This function generates the file relevant to an activated view or selected in the Project Explorer.

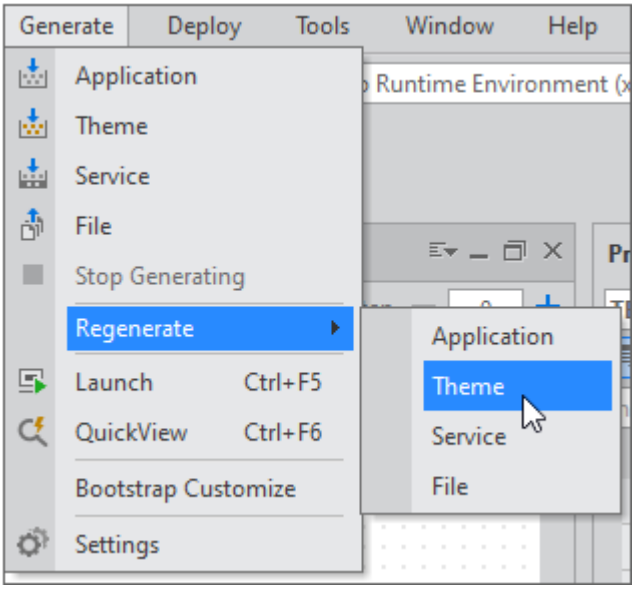
If there is an activated view, you can use This function by accessing [**HOME - Generate & Launch - Generate - File**] in the ribbon interface or [**Generate** > **File**] in the command bar interface. If there is no activated view, the foregoing menus will be inactive.

You can also employ This function by right-clicking a file in the Project Explorer and select [**Generate File**] from the context menu.



Regenerate

As for the "generate" feature, Nexacro Studio does not convert the files that already exist in the designated directory and that have not been changed. However, the "regenerate" feature converts all files regardless. You can employ This function by accessing [**Generate** > **Regenerate**].



16.1.2 Automatic Generation

If you want to open and edit a project in Nexacro Studio, you need a generated project file. Therefore, Nexacro Studio will automatically generate files when it performs the following tasks.

- Loading a project
- Creating a file
- Saving a file after modification
- Before executing Deploy
- Before launching QuickView

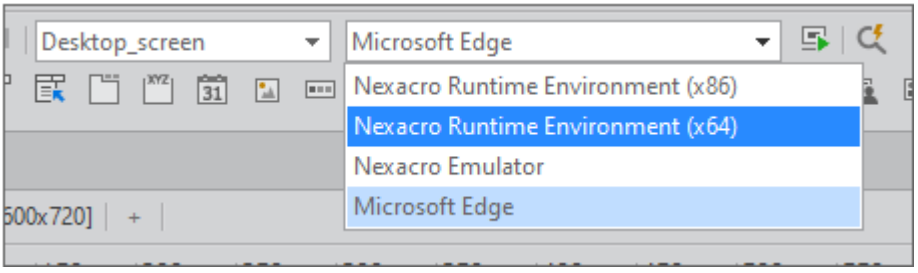
16.2 Project Execution

The outcome developed with Nexacro Studio can be executed on the Nexacro. Nexacro Studio provides two related features: **Launch** for executing an app and **QuickView** for a Form.

16.2.1 Execution Mode

You can set the execution mode by accessing the "browser combo," located in [**HOME - Generate & Launch**] in the ribbon interface or situated on the toolbar in the command bar interface.

> The dropdown list of the "browser combo" only shows the web browsers that are installed in the user's PC and supported by Nexacro Studio.

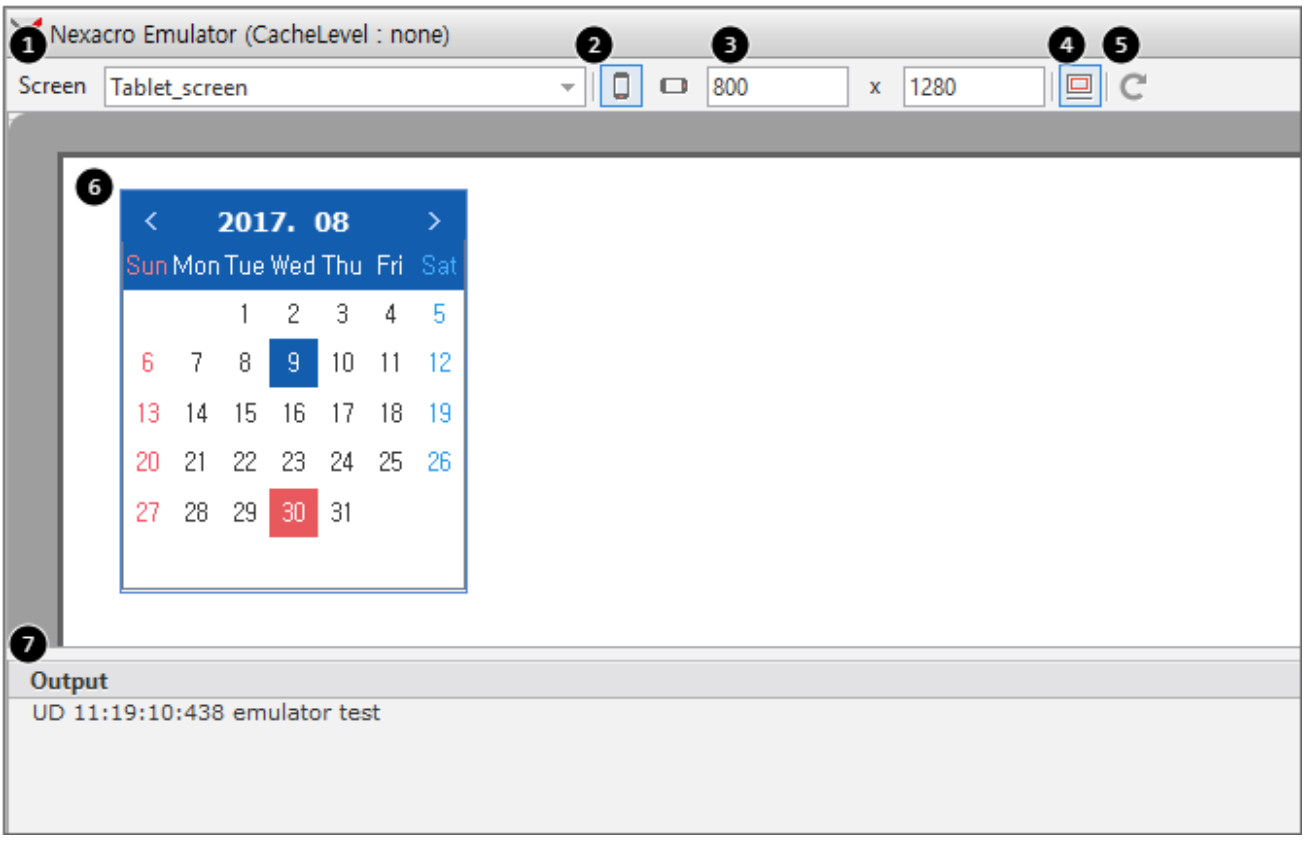


NRE (Nexacro Runtime Environment)

Nexacro can operate in the Nexacro Runtime Environment (NRE), 32-bit or 64-bit, with the *nexacro.exe* file provided as an executable. You can execute an app by directly accessing the created files without a web server.

Emulator

Nexacro Studio provides an emulator that allows you to see how your form is executed according to varying devices and screen settings. You can test various execution conditions conveniently while you still design your forms.



Call-out		Description
1	Screen List	Shows the screen definition list
2	Orientation	Changes the orientation of the screen—horizontal or vertical—when the screen type is a tablet or phone.
3	Size	Screen size
4	Show Output	Toggles the Output pane
5	Reload	Reloads the current screen
6	Preview	Previews the current screen
7	Output	Shows errors and other messages

Web browser

You can try your app on a web browser. You can launch your app on the web browsers installed in your computer. If you use a mobile device, you can enter the URL of your app to a web browser in the mobile device.

16.2.2 Launch

Launch is a feature used to execute a Nexacro-based app. You need to specify a screen and then select an execution mode.

To use This function, you need to access the menu [**HOME - Generate & Launch - Launch**] in the ribbon interface or [**Generate** > **Launch**] in the command bar interface. If you select a web browser as the execution mode, a pop-up will appear to ask you a URL in which your application is located. After you enter the URL, your Nexacro application will run on the selected web browser. If you use Local Web Server, the URL dialog will fetch the value designated as "Server Host Url" in the menu [Tools > Options > Project > Launch].

Assume that you select the Web Server option in the menu [Tools > Options > Project > Launch]. In this case, the below window will appear to allow you to enter a URL if you neither specify a preset URL nor select [Display run configuration on Launch / Quickview].

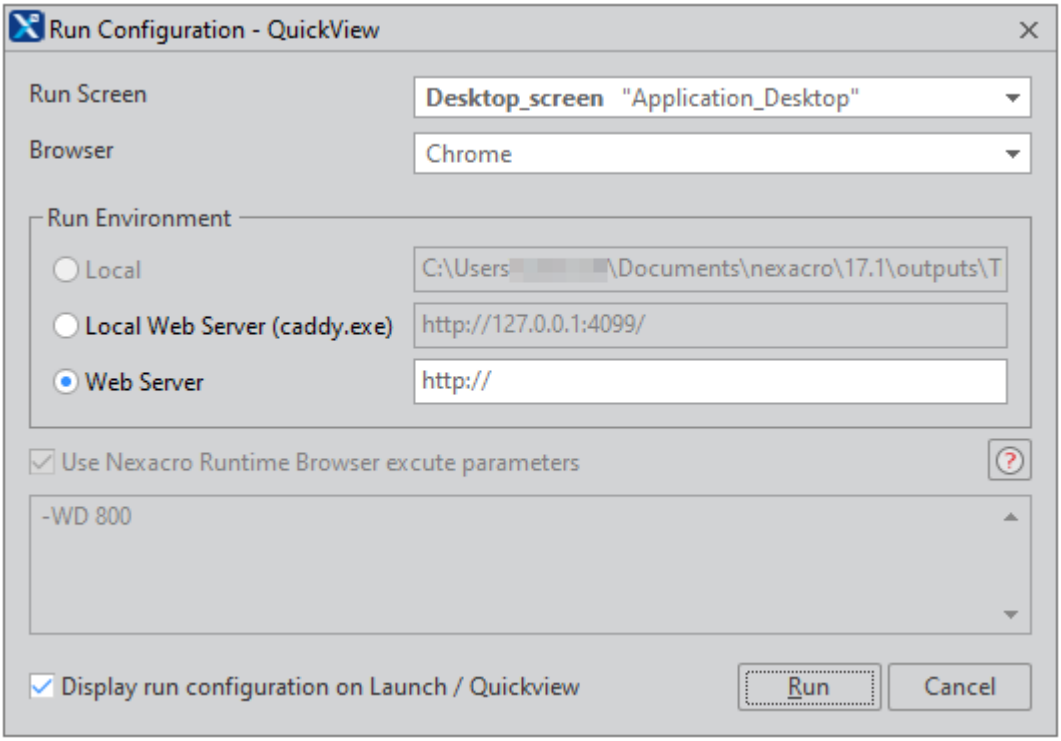




If you want to see the **Run Configuration** window presented below, you need to access the menu [Tools > Options > Project > Launch] and select the option [Display run configuration on Launch/Quickview].



When the [?] button next to the Use Nexacro Runtime Browser execute parameters item is clicked, then the following help item is executed.
Misc. Objects > System > Method > execNexacro



16.2.3 QuickView

If an activated view is the Form editor, you can run QuickView. In addition, you can also operate QuickView by activating the Project Explorer and then selecting a certain form in the tree.

You can use QuickView by accessing [**HOME - Generate & Launch - QuickView**] in the ribbon interface or [**Generate** > **QuickView**] in the command bar interface.

16.3 Managing Bootstrap Template

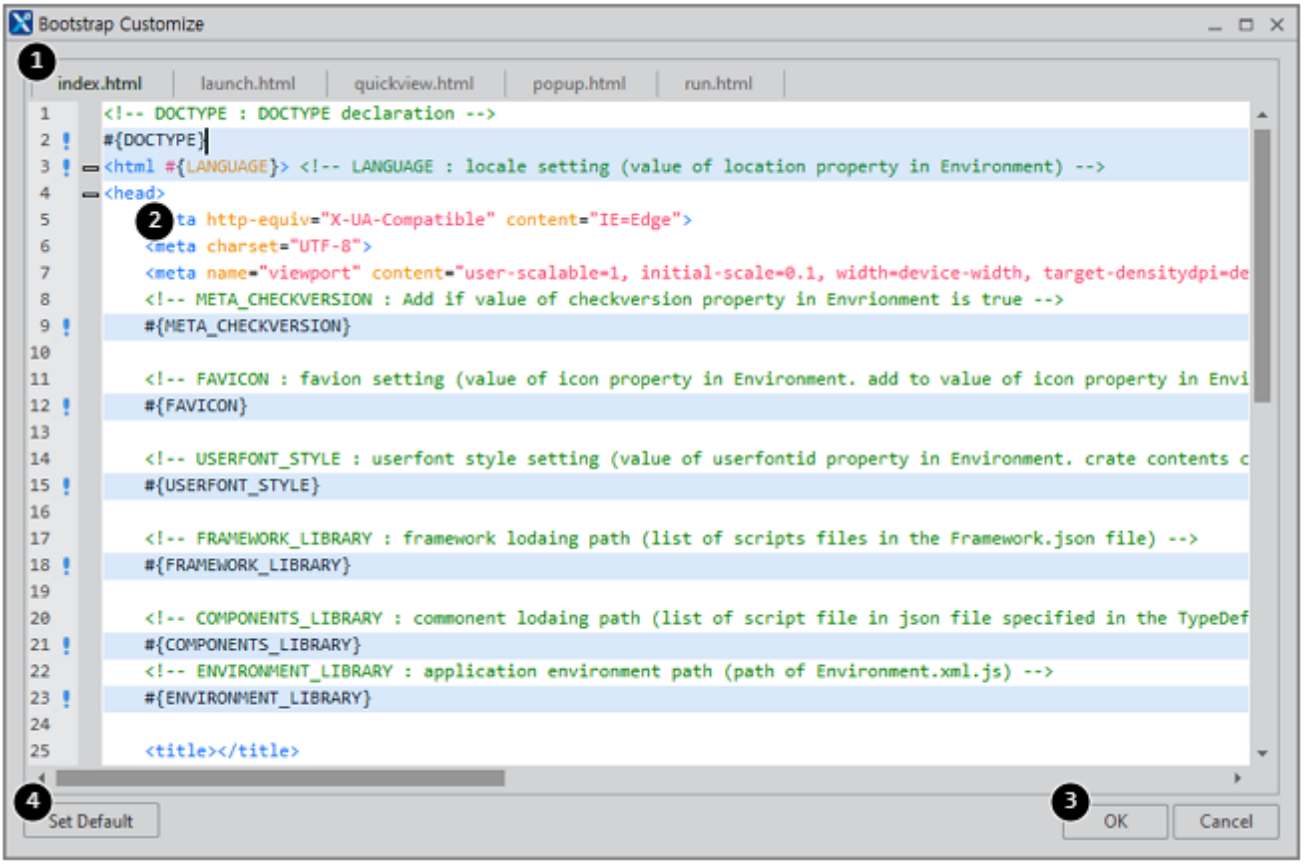
The automatically generated bootstrap HTML file can be created by applying the template with the code required by the user. Select the menu [Generate > Bootstrap Customize] item and edit the template content to reflect the automatically generated bootstrap file.

The bootstrap template allows you to manage the 5 files below.

File	Description
index.html	It is created during Web Browser distribution file Packing. It is deployed on the server without any parameters and executed when the file URL is accessed.

File	Description
launch.html	This file is used when executing [Launch] in Nexacro Studio. It is created during Generate and is not used during deployment. It receives screenid as a parameter and executes it.
quickview.html	This file is used when running [QuickView] in Nexacro Studio. It is created when generating and also when packing the web browser distribution file. It receives screenid and formname as parameters and executes it.
popup.html	This file is used when modeless window is executed with the nexacro.open method. It is created when generating and also when packing the web browser distribution file.
run.html	It is created when packing the Android and iOS operating system distribution file. If the Update Type is "Server", it is created as a Run.html file, and in the case of "Update" or "Local", it is created in a compressed form within the Run.zip file.

The basic functions of Bootstrap Customize are as follows.



	Item	Description
1	Bootstrap Template List	Selects the bootstrap template to edit.
2	Bootstrap Template Edit	Edits the content of the selected bootstrap template. The required keyword is marked with blue "!" next to the line number and a warning message is displayed at the top of the edit window when the required keyword is edited or deleted.
3	Bootstrap Template Reflect	Saves the created template content and regenerates the project

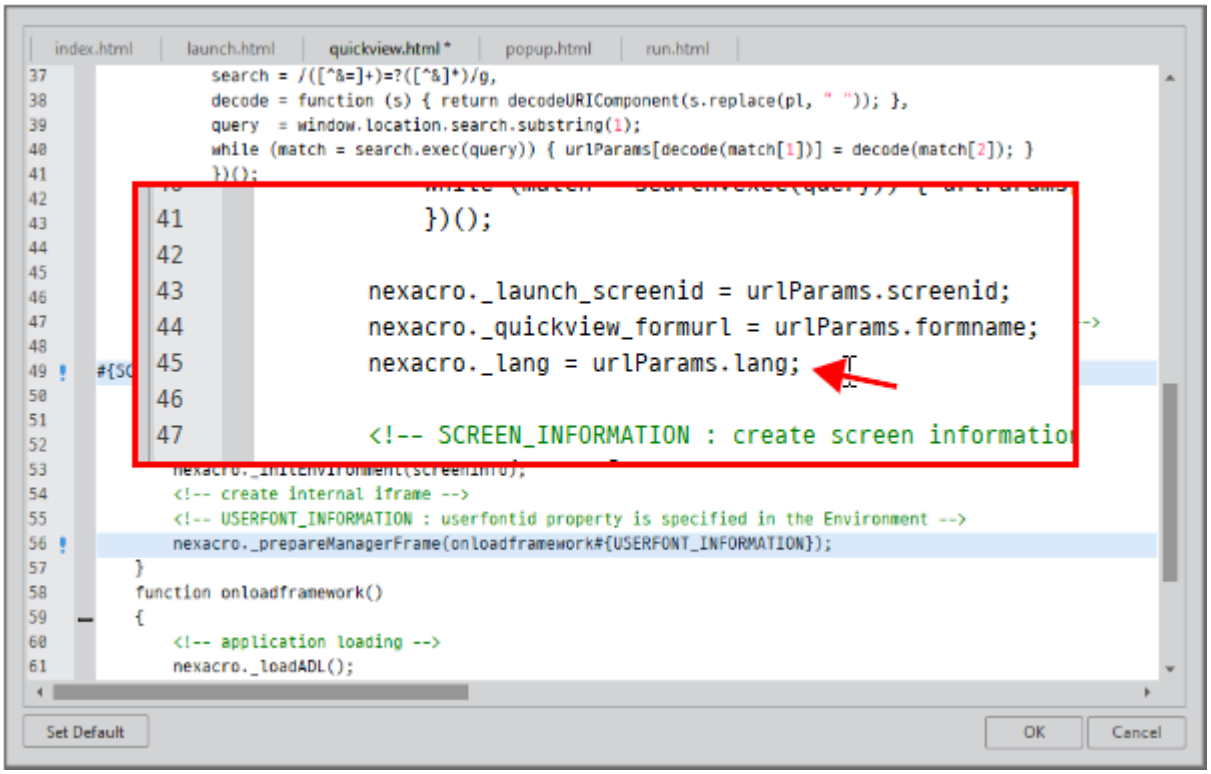
	Item	Description
		bootstrap file.
4	Bootstrap Template Initialize	Provides 2 initialization options. You can revert the currently open bootstrap template to the default or the entire bootstrap template to the default.

16.3.1 Creating a Template with Script Code Added

If you want to implement a function that receives a specific language code using a URL parameter, then you can add the code to the template as shown below.

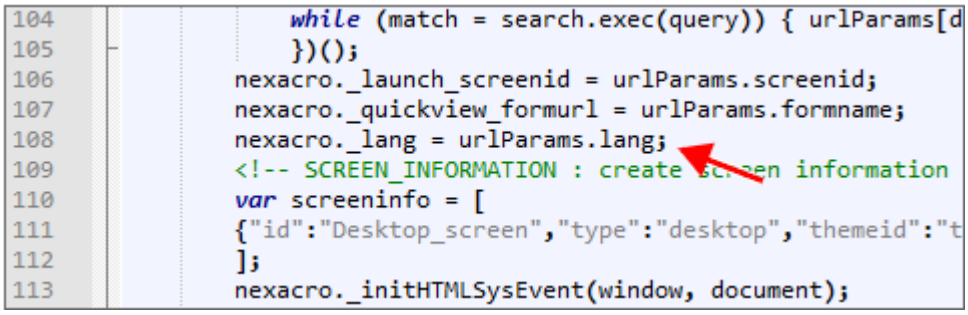
1. Select the menu [Generate > Bootstrap Customize] item and execute Bootstrap Customize.
2. Select the quickview.html file from the bootstrap template list.
3. Add the following code to oninitframework function under the script tag.

```
nexacro._lang = urlParams.lang;
```



4. Click the [OK] button to close the window and create the bootstrap file again.

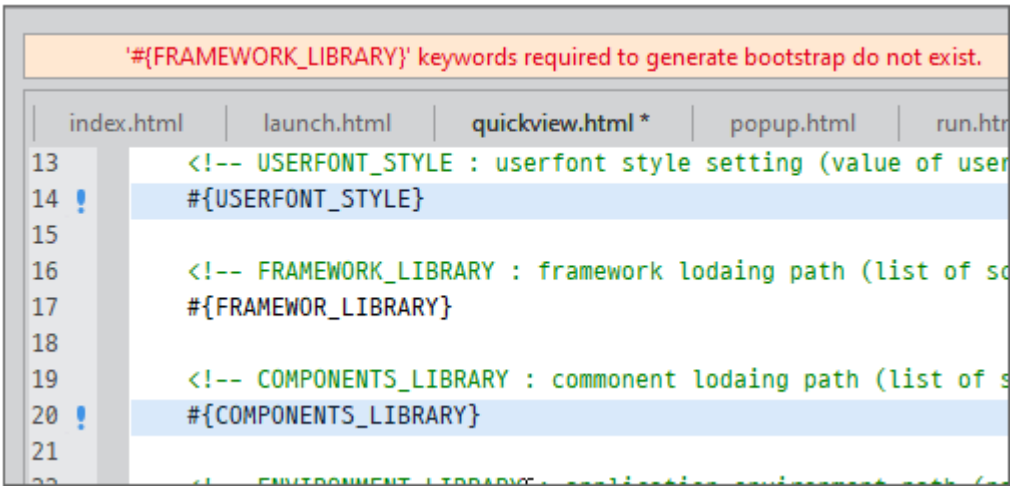
5 Open the quickview.html file generated in the path designated as Generate Path in the text editor to check whether the added code is reflected when creating the file.



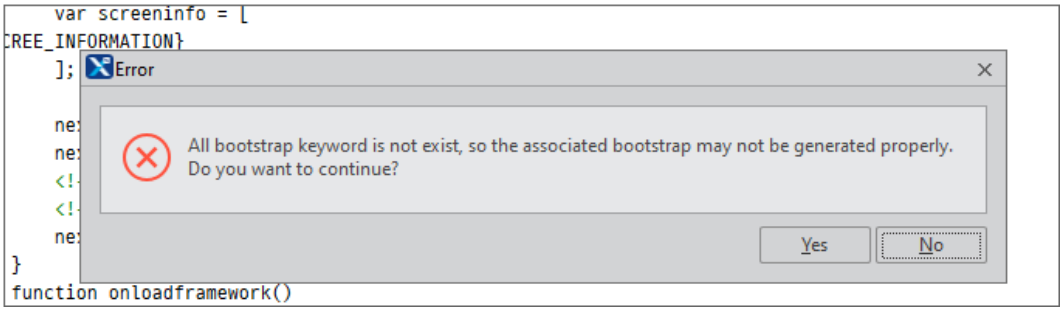
16.3.2 Cautions when Editing Bootstrap Template

Editing or Deleting Keyword

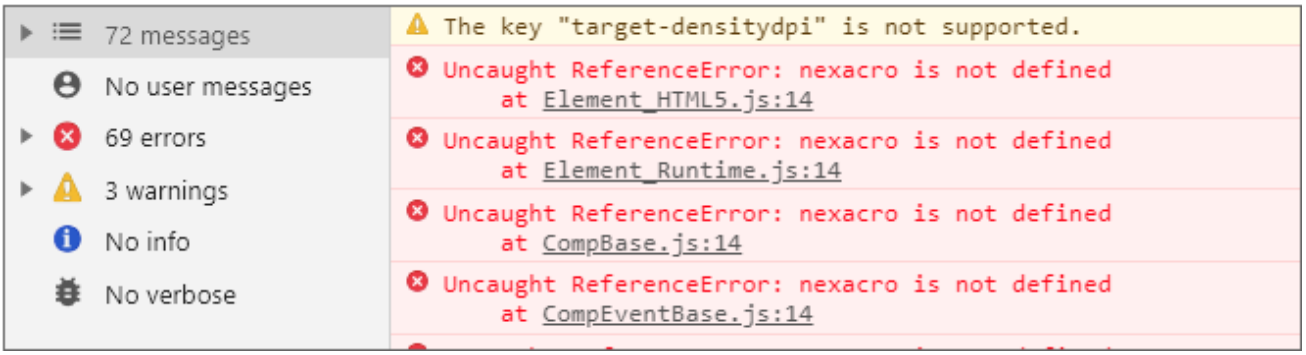
Editing or deleting keyword items is not recommended. If you edit or delete the keyword, then a warning message is displayed in the bootstrap edit window as shown below. In the figure below, the #{FRAMEWORK_LIBRARY} item is edited as #{FRAMEWOR_LIBRARY}.



Even if a warning message is displayed, then it can be reflected after saving. When the [OK] button is clicked, then it will be checked once more.



When the generated code is executed, the framework library cannot be called normally and it is processed as an error.



Editing the Default Script

Functions such as nexacro._initHTMLSysEvent and nexacro._loadADL are parts that call functions specified inside the Nexacro framework, so deleting or relocating functions may affect the app execution. It can be modified to the extent that it does not affect the execution of the app.

16.3.3 Note

Bootstrap Template Configuration File Path

When the bootstrap template is set, a configuration file is created in the project path. To share the bootstrap template file, the configuration file and the project file (XPRJ) must have the tag that specifies the configuration file.

> [project path]\bootstrap.xml

The tag is added to the project file (XPRJ) as shown below.

```
<?xml version="1.0" encoding="utf-8"?>
<Project version="2.1">
...
  <BootstrapCustomize url="bootstrap.xml"/>
</Project>
```

Keyword List

Keyword	Description
#{DOCTYPE}	DOCTYPE tag generation
Environment Information	
#{LANGUAGE}	Creates the HTML property tag according to the local property value
#{META_CHECKVERSION}	Creates the meta tag according to the checkversion property value If the value is not set, then the tag is not created.
#{FAVICON}	Creates the link tag according to the icon property value If the value is not set, then the tag is not created.
#{USERFONT_STYLE}	Creates the tag according to the userfontid property value If the value is not set, then the tag is not created.
Project Information	
#{FRAMEWORK_LIBRARY}	Creates the framework js file script tag
#{COMPONENTS_LIBRARY}	Creates the component js file script tag
#{ENVIRONMNET_LIBRARY}	Creates the environment.xml.js file script tag
#{SCREEN_INFORMATION}	Processes the Screen configuration information
#{USERFONT_INFORMATION}	Processes the xfont configuration information
Run.html	
#{MOBILE_LIBRARY}	Creates the archivefiles.js file script tag
#{PROJECT_URL}	Project URL entered by the user

Splash Loader

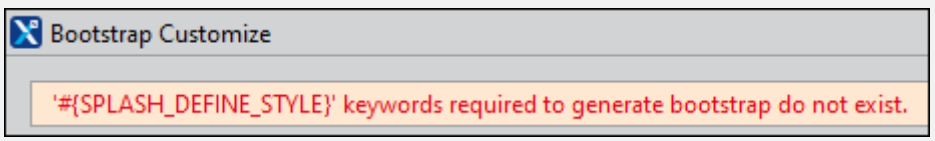
If the Set Splash Loader item is checked in the option [Project > Generate > General] item, the script that displays the Splash Loader is activated.

Keyword	Description
#{SPLASH_DEFINE_STYLE}	Defining the Splash Loader UI
#{SPLASH_SHOW_STYLE}	Splash Loader UI display area settings
#{SPLASH_HIDE_STYLE}	Hiding the Splash Loader UI after loading

If you want to change the Splash Loader UI, change the #{SPLASH_DEFINE_STYLE} keyword to a style tag.



If you delete the #{SPLASH_DEFINE_STYLE} keyword, a warning message will be displayed at the top of the Bootstrap Customize window and a Confirm window will also be displayed upon saving. If you've written the style tag properly, it won't affect the behavior.





The class name "loader" is defined in the #{SPLASH_SHOW_STYLE} keyword. If you want to modify the class name, you must also change the #{SPLASH_SHOW_STYLE} keyword to a tag.

```
<!-- SPLASH_DEFINE_STYLE : Defines the style of the splash screen that is
visible while modules are loading -->
    #{SPLASH_DEFINE_STYLE}
```

```
<!-- SPLASH_DEFINE_STYLE : Defines the style of the splash screen that is
visible while modules are loading -->

<style>
.loader {
  position: absolute;
  left: 50%;
  top: 50%;
  z-index: 1;
  width: 150px;
  height: 150px;
  margin: -75px 0 0 -75px;
  border: 16px solid #f3f3f3;
  border-radius: 50%;
  border-top: 16px solid #111111;
  -webkit-animation: spin 2s linear infinite;
  animation: spin 2s linear infinite;
  }

  @-webkit-keyframes spin{
    0% { -webkit-transform: rotate(0deg); }
    100% { -webkit-transform: rotate(360deg); }
   }

  @keyframes spin {
    0% { transform: rotate(0deg); }
    100% { transform: rotate(360deg); }
   }
</style>
```

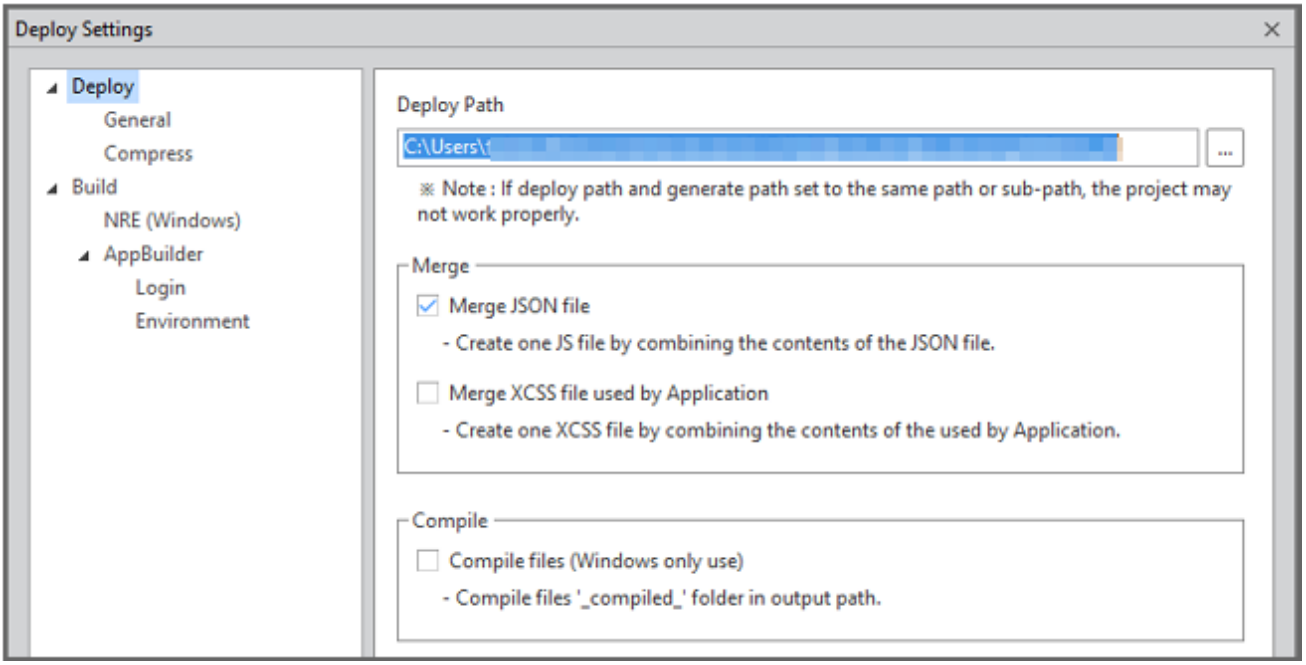
17.

Creating Installation Files (or Web Documents) for Deployment

Once you finish developing an application project, you need to make it possible for users to install the project on their devices or access it through web browsers. This can be done with the Packing and Build App features. In the Packing stage, you can create the files necessary for deployment by selecting an operating environment and update type. Next in the Build App stage, you can compress those created files to make an installation file.

17.1 Configuring Deploy Environment

You can set the necessary values related to Deploy by clicking the menu [Deploy > Settings] item. Only items related to Deploy environment configuration among the menu [Tools > Options] are put together and the values to set are the same.



17.2 Packing- Create Archive Files

In this stage, you can select operating environments and corresponding update types for your project. You can designate the files to be included in archive files, varying according to operating environments.

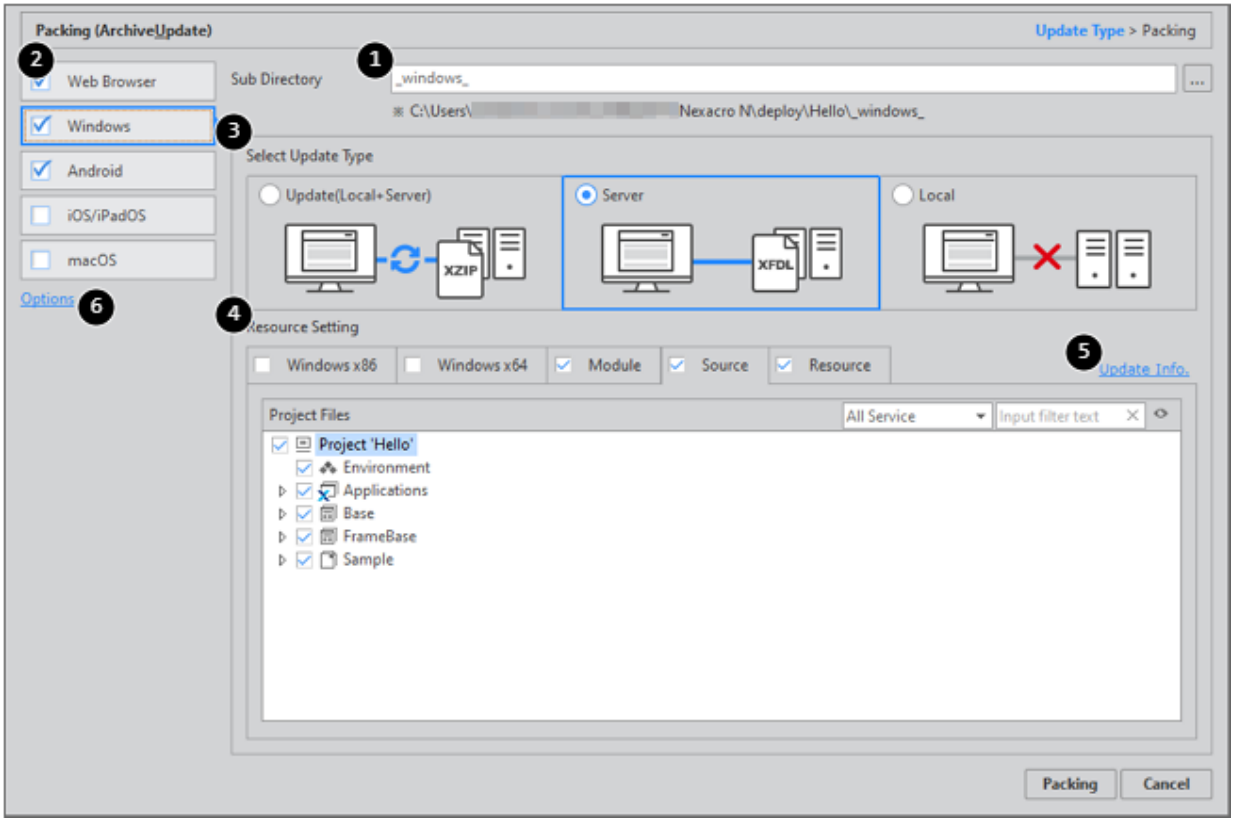
17.2.1 Launch Packing Wizard to Create Archive Files

1. Launch the Packing wizard by accessing the menu [Deploy > Packing(Archive&Update)].

2. In Output Path, specify the location where archive files will be created.

3. Click the Next button to proceed to the next step, which allows you to select packing options for each operating environment.

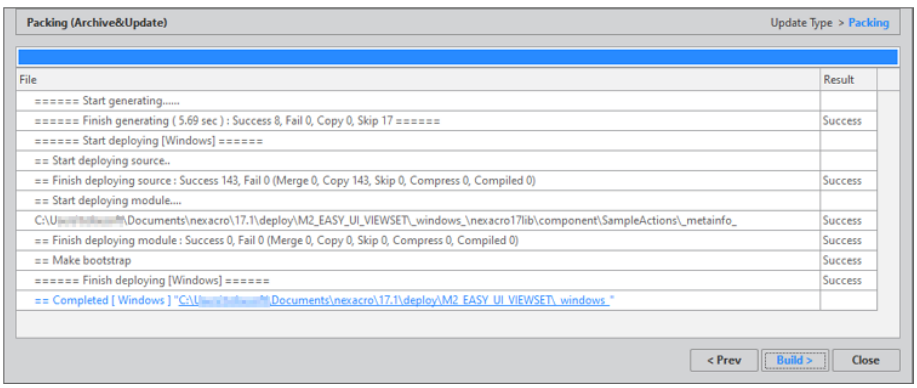
4. Select operating environments first and then choose corresponding update types and other options. Click Options if you want to use the compression and obfuscation functions.



Call-out		Description
1	Sub Directory	Designates a folder where archive files will be created. Sub-folders will be created automatically according to selected operating environments Below is the list of the sub-folders that can be created according to selected operating environments. Web Browser (_web_) Windows (_windows_) Androids (_android_) iOS (_ios_) macOS (_macos_)
2	Operating environments	Selects the operating environments for which archive files will be created. You can set the options of selected environments. You cannot adjust the settings of unselected environments.
3	Update Type	Selects one from three update types according to how you deploy the app. Configurable options vary according to update types. You do not need to select an update type for the web browser environment.
4	Resource Setting	Configures options, which vary according to operating environments.
5	Update Info	Opens the Update Info setting window.

5 Upon clicking the Packing button, Nexacro Studio will begin to create archive files according to selected operating environments and configurations. Results will be displayed after the creation of archive files is complete.

If archive files are successfully created, Nexacro Studio will present blue-colored links, which you click to open the folders that contain relevant files. Clicking the [Build App] button at the bottom left will lead you to Build App, a wizard for creating deployment files. When you select 'Web Browser' only as an operating environment, the [Build] button will not activate.



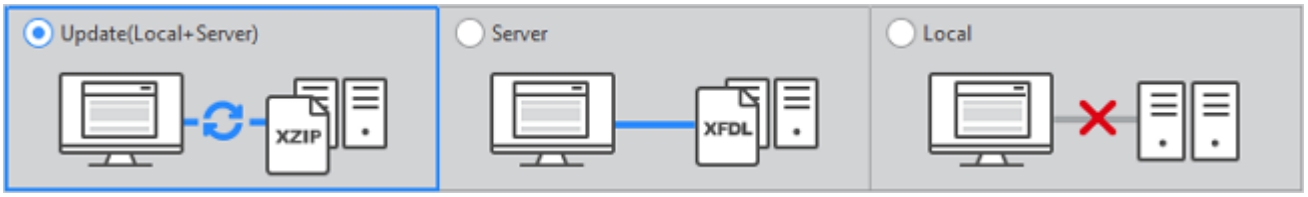
index.html

In the case of the web browser environment, Nexacro Studio does not create an installation file. Instead, it creates an HTML file through which you can access the app by using an web browser. In the *_web_* folder, you will find that necessary files have been created including the *index.html* file. Once you upload all those files to a server, you can access the app through a web browser by entering the below URL.

```
http://[domain name]/index.html
```

17.2.2 Select Update Type (for each operating environment)

For the Windows, Android, iOS and macOS environments, you need to select an update type, which alters a selection of settings options as well as created archive files.



Update Type

- Update (Local + Server)
  To install an app, you need to download necessary archive files and save them to your local storage. If the app detects a change in the version of its contents, relevant updates will be downloaded via the network and saved to your local storage.
- Server
  You can execute an app only with a network connection. You need to download necessary files from the server.
- Local
  To install an app, you need to download necessary archive files and save them to your local storage. You can execute an app without a network connection.



Even when the update type is Local, you need a network connection depending on apps for the purpose of accessing data or connecting to the server.

17.2.3 Configurable Options (according to operating environments and update types)

You can select types of files that will be deployed, according to operating environments and update types. In the configurable option section, you can select files to deploy.



When updating modules, sources or resources, the updating will not be conducted if the version of the incoming update package equals that of the previous package. The reason is that Nexacro will consider that both packages are the same.

Update Info.

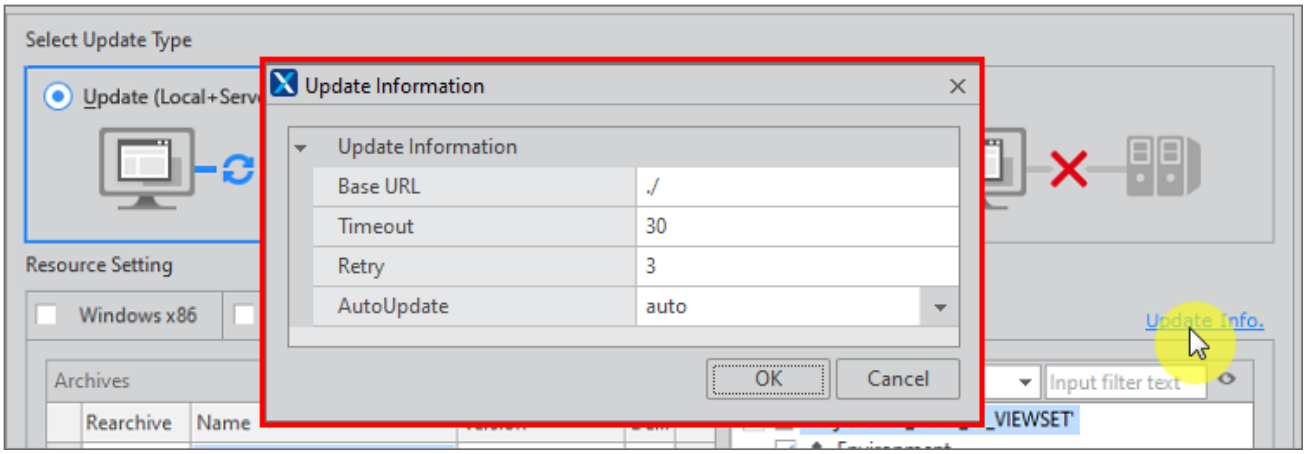
You can set update policies for receiving recent contents from the server. You need to designate the directory from which contents are downloaded. Moreover, you need to set the timeout for updating and determine the number of permitted retries. (An updating process will finish as a failure if you fail to receive contents for a specified number of retries.) Lastly, you need to determine whether the updating is performed automatically or manually (on user demand).

Applicability according to operating environments and update types

	Update (Local + Server)	Server	Local
Web Browser	X		
Windows	O (Required)	O (Required)	X
Android	O (Required)	O (Required)	X
iOS	O (Required)	O (Required)	X
macOS	O (Required)	O (Required)	X

Configurable options

Click the [Update Info] item to the right of the Resource Setting item and then edit the content.



Menu name	Description
Base URL	Base location of the files that require updates such as *SetupResource.zip* and *SetupRuntime.exe*.
Timeout	Timeout for updating
Retry	Number of permitted retries
AutoUpdate	Selects between 'auto' and 'manual'

Update Info. is required so you cannot deselect it.

Setup

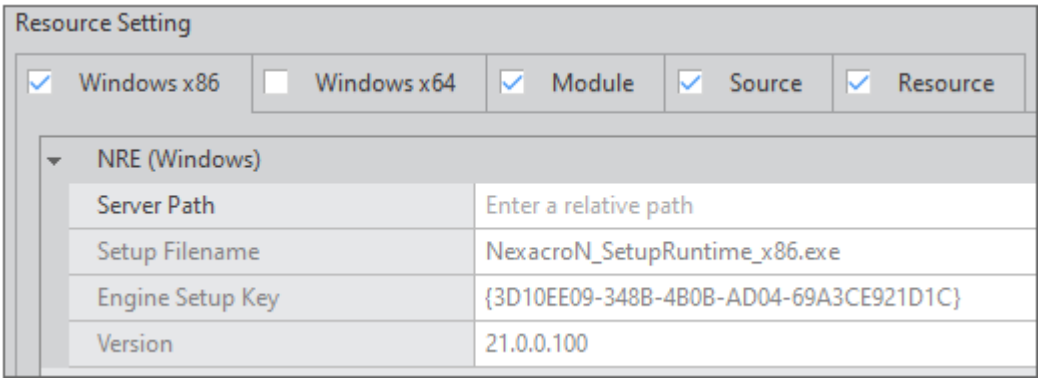
You can deploy the basic setup of Nexacro without using the Build App wizard. You just need to create a bootstrap file and resource files and add setup update information to the bootstrap file.

Applicability according to operating environments and update types

	Update (Local + Server)	Server	Local
Web Browser	X		
Windows	O	O	O
Android	X	X	X
iOS	X	X	X
macOS	X	X	X

Configurable options

You can select the types of operating systems where the deployed runtime will run. The information on selected OSes will be added to the bootstrap file *start.json*.



Menu name	Description
Use	Checks an operating system to which the Nexacro runtime will be deployed.
Type	Type of an operating system
Server Path	This is a relative path whose base is the location defined as Base URL in Update Info. If the location of deployed setup is different from the Base URL, you need to use this option. For example, the below URL will be used if you specify "UPDATE" as the Base URL and "SETUP_FILE" as the Server Path. `http://localhost/UPDATE/SETUP_FILE/[setup file name]`
Setup Filename	Name of the setup file
Engine Setup Key	Setup key
Version	Version of the setup file



You can change some options related to the setup—such as a shortcut icon, splash image and engine setup key—in the Build App stage.

Module

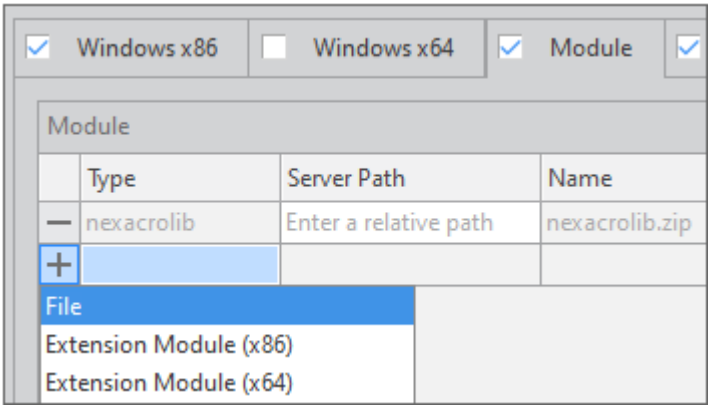
You can determine whether to deploy the framework *nexacrolib*, which is necessary for executing a Nexacro-based app. If the execution environment is Windows, you can add external modules.

Applicability according to operating environments and update types

	Update (Local + Server)	Server	Local
Web Browser	X		
Windows	O	O	O (Required)
Android	O	X	O (Required)
iOS	O	X	O (Required)
macOS	O	X	O (Required)

Configurable options

You can add, change or delete module files to deploy. Provided options are varying according to operating environments.



Menu name	Description
Type	Module type OS versions are specified for extension modules. (Ext [OS version]) The type of a module is specified as there are three module types as follows: framework (*nexacrolib*), file.
Server Path	This is a relative path whose base is the location defined as Base URL in Update Info. If the location of the module is different from Base URL, you need to use this option. For example, the below URL will be used if you specify "UPDATE" as the Base URL and "SETUP_FILE" as the Server Path. `http://localhost/UPDATE/MODULE/[module file name]`
Name	Name of the module file (unchangeable)
TargetPath	This is a path to a local storage to which files are downloaded. If you specify the default path (./), files will be saved to the %USERAPP% directory.

Menu name	Description
	You can also specify another path like %USERAPP%Component/TEST/.
Version	Version of the module
Description	Memo (irrelevant to system operation)



See %USERAPP% Alias paths for further information

Alias Path

Configurable options (for Local update type)

The framework file (*nexacrolib.zip*) is set by default, and you cannot delete or change this file. Only two options are presented: Type and Name.

Configurable options (for Windows operating environment)

You can add extension modules and files. For Android, iOS and macOS, you cannot deploy any module other than the framework file.

Configurable options (for web browser environment or for Server update type)

You need to deploy the framework in the form of files executable on web browsers, rather than the form of a compressed file.

Source

You can specify source files (or folders) to deploy, selected from the files in your project. You should select an entire project for the first deployment while you can pick up only necessary files for an update. Source files are deployed in the form of either individual files or compressed archive files depending on operating environments and update types.

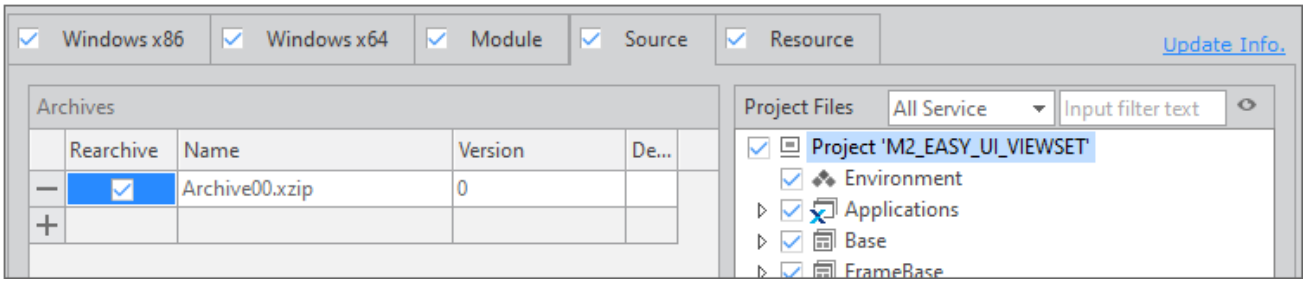
Applicability according to operating environments and update types

	Update (Local + Server)	Server	Local
Web Browser	O		
Windows	O	O	O (Required)
Android	O	O	O (Required)

	Update (Local + Server)	Server	Local
iOS	O	O	O (Required)
macOS	O	O	O (Required)

Configurable options

You can select files for deployment from Project Files. If there are already-made archive files, you can choose one or more archives and then select files for deployment from Project File. In this way, you can selectively add source files to an archive.



Menu name	Description
Rearchive	When checked, you can build the archive file again by including just necessary source files
Name	Name of an archive file.
Version	Version of the archive file
Description	Memo (irrelevant to system operation)

Configurable options (for Web Browser operating environment or Server update type)

While no archive file is created, you just need to select files for deployment from Project Files. You cannot find the Archives option in the configurable options section.

Configurable options (for Local update type)

Only the Rearchive and Name options are presented.

Resource

You can specify resource files to deploy. You should select entire resources for the first deployment while you can pick up only necessary files for an update. Resource files are deployed in the form of either individual files

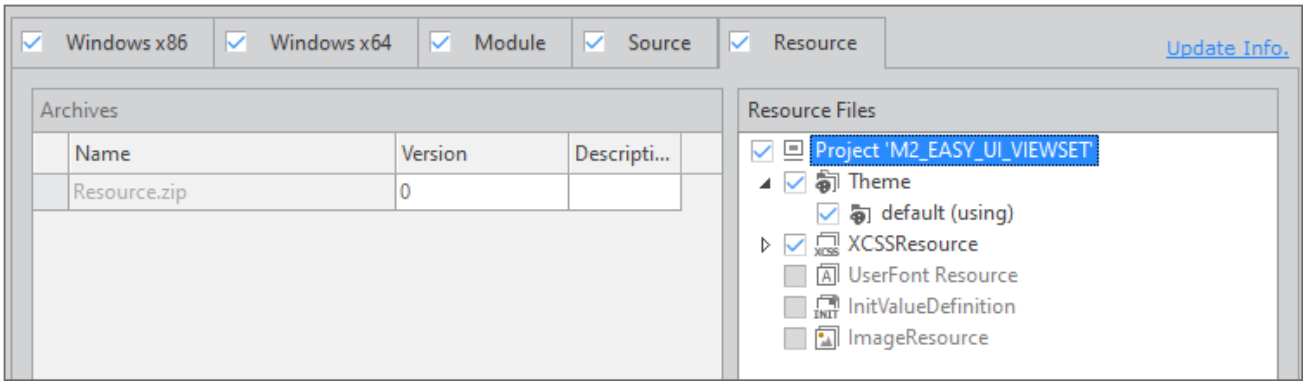
or compressed archive files depending on operating environments and update types.

Applicability according to operating environments and update types

	Update (Local + Server)	Server	Local
Web Browser	O		
Windows	O	O	O (Required)
Android	O	O	O (Required)
iOS	O	O	O (Required)
macOS	O	O	O (Required)

Configurable options

You can select files for deployment from Resource Files. As for the Archives option, you can create only one archive file, *Resource.zip*.



Menu name	Description
Name	Name of the archive file (unchangeable)
Version	Version of the archive file
Description	Memo (irrelevant to system operation)

Configurable options (for Web Browser operating environment or Server update type)

While no archive file is created, you just need to select files for deployment from Resource Files. You cannot find the Archives option in the configurable options section.

Configurable options (for Local update type)

Only the Name option is presented.

17.3 Build App- Create Installation Files

App Builder related articles

- **Server Setup Guide > App Builder**
  App Builder Installation
  Server Configuration
  Deploy Server Setting
  Signature Setting by OS
  User Library Settings
  Nexacro Library Settings
  User Settings
- **Deployment Guide > App Builder**
  Create App
  App List
- **Development Tools Guide > Build App- Create Installation Files**
  Build App- Create Installation Files

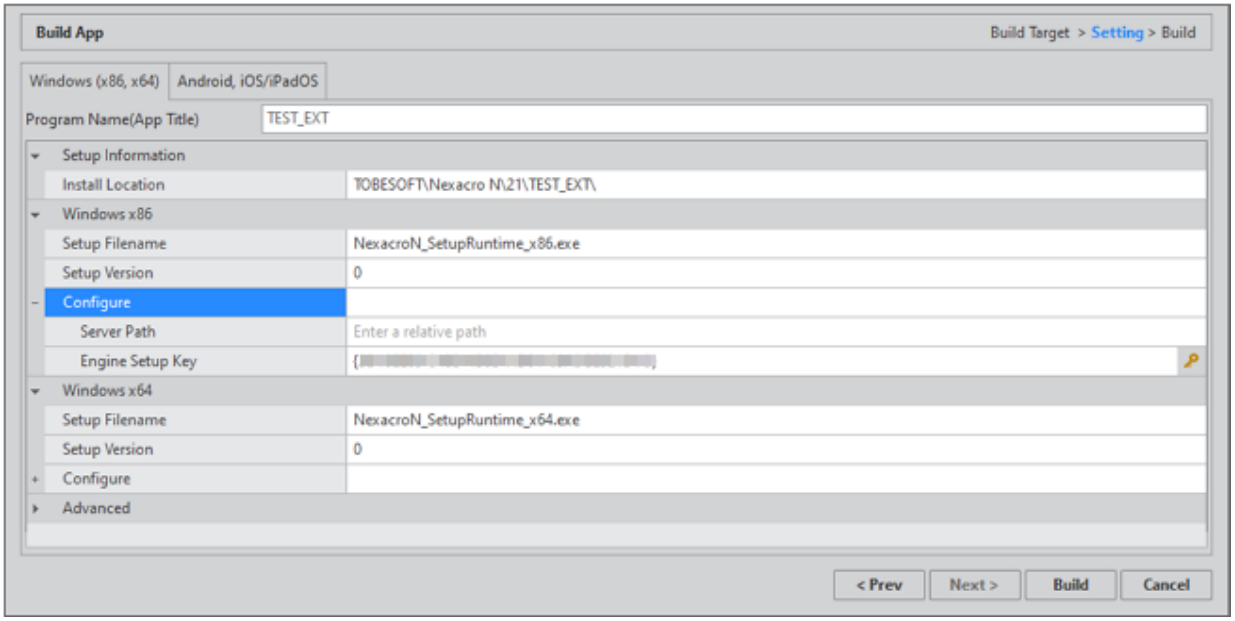
In this stage, you can create installation files by including necessary archive files created in the previous Packing stage. You can choose options according to the operating systems to which your application will be deployed.

17.3.1 Launch Build App Wizard to Create Installation Files

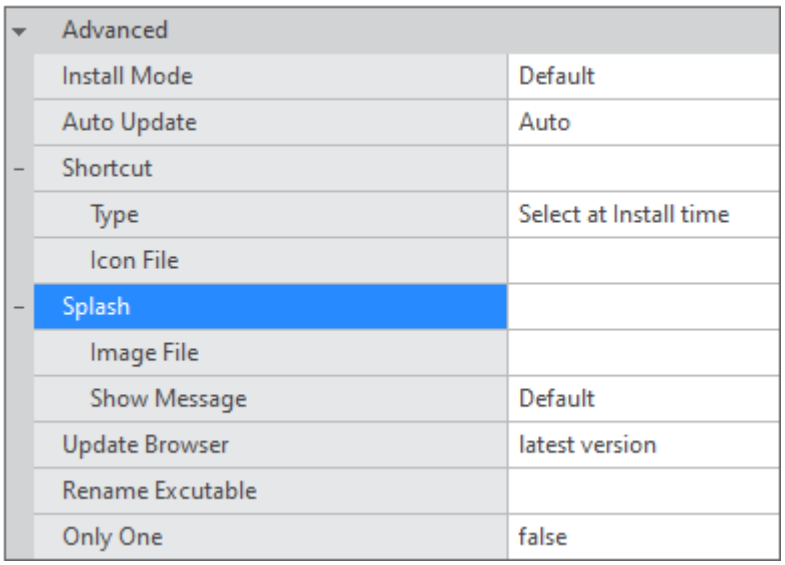
1. To launch the Build App wizard, click the **Build App** button, which appears in the last step of the Packing stage upon the creation of archive files. Otherwise, access the menu [Deploy > Build].

   In the case of clicking the [Build] button in the Packing step, it moves to the property setting screen, and in the case of accessing the Build App menu, it moves to the setting screen after selecting Build Target.

2. In the case of selecting the Windows operating system among the Build Target items, check the Setup-related settings and click the [Build] button.



Menu item	Description
Program Name	This is the name that will be displayed in the menu *Uninstall or change a program*. The default value is the name of the project.
Build Type	
Build Type	Selects the operating system version.
Setup information	
Install Location	Specifies the path where the app will be installed. The path will be located under the "Program Files" folder in Windows. The default value is "TOBESOFT\Nexacro N\21\{program Name}".
Windows [according to versions]	
Settings varying according to selected versions of Windows. You can use different file names for each version.	
Setup Filename	Specifies the name of an installation file to create
Setup Version	Specifies the version of the installation file
Windows [Operating System Version] > Configure **- Configure when changing the Server Path or Engine Setup Key.**	
Server Path	This is a relative path whose base is the location defined as Base URL in Update Info. If the location of the setup is different from Base URL, you need to use this option. For example, the below URL will be used if you specify "UPDATE" as the Base URL and "SETUP_FILE" as the Server Path. `http://localhost/UPDATE/SETUP_FILE/[setup file name]`
Engine Setup Key	Designates an engine setup key (CLSID)

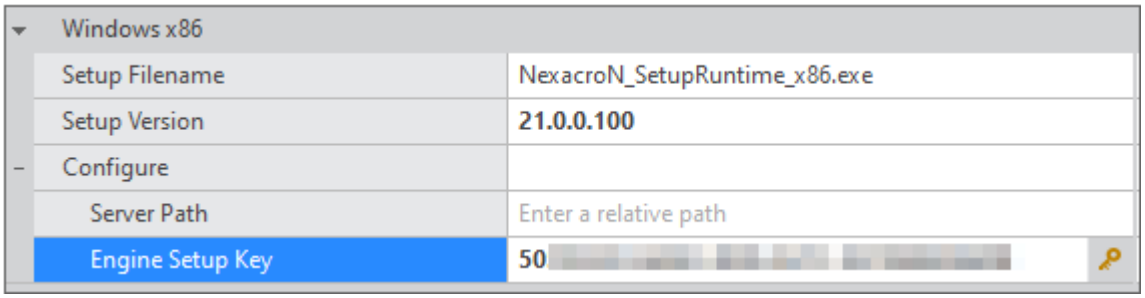


Menu item	Description
Advanced	
Install Mode	Default: default installation mode Silent: the application will be installed with default settings without the appearance of a dialog during installation.
Auto Update	This option is used to determine whether to update the engine of Nexacro automatically. This option overrides the AutoUpdate option, which falls into the Update Info section in the Packing stage. The default value is "Auto". If you select "Manual", the engine will not be updated automatically.
Shortcut Type	Desktop: creates a shortcut on the desktop Start Menu: creates a shortcut on the start menu Desktop + Start Menu: creates shortcuts on the desktop and start menu both Select at Install time: allows users to designate the location of a shortcut when the engine is installed. No use: no shortcut is created. The default value is "Select at Install time". If the Install Mode is set to "Silent", the option "Select at Install time" is not available and the default value is "Desktop".
Shortcut Icon File	Changes an icon that represents the app set-up file, executable program and uninstaller program The default value is the icon of Nexacro.
Splash Image File	Changes the image of the splash screen to a specified file
Splash Show Message	Default: displays the default message Do not show: no message is displayed.
Update Browser	Determines whether to update the browser emulator to the latest version
Rename Executable	Changes the name of the executable program

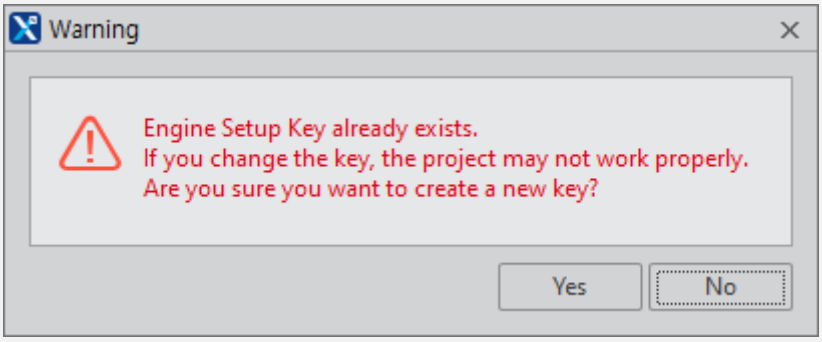
Menu item	Description
	The default value is nexacro.exe.
Only One	Setting to prevent duplicate execution of apps with the same key value

If your app includes a WebBrowser component, the version of the browser emulator will vary according to the value of the Update Browser option. If more than one app are installed in your PC, a conflict may occur over the settings on browser emulators.
In this case, you can prevent such a conflict by renaming the file *nexacro.exe* through the Rename Executable option.

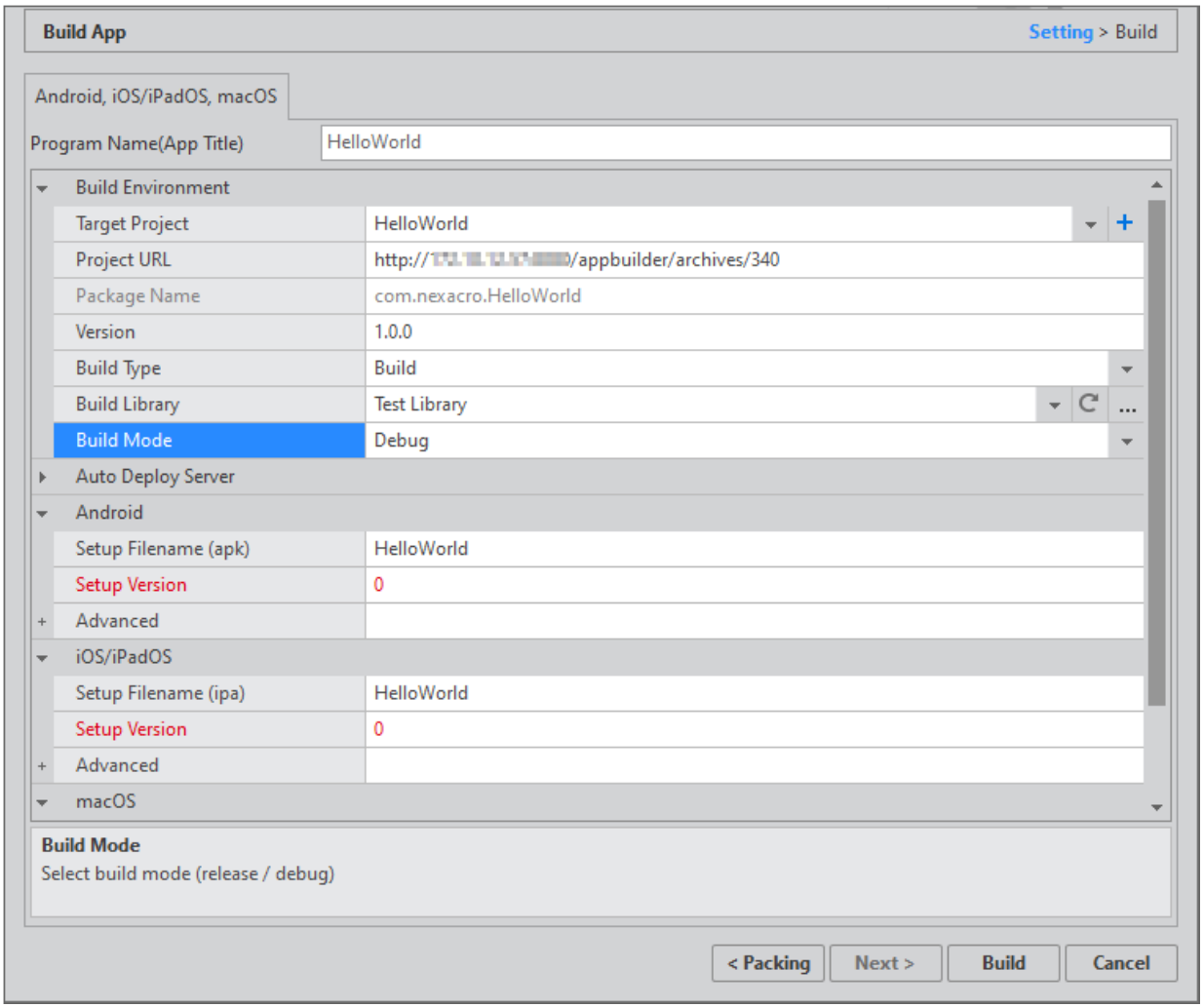
You can either use the default engine setup key as it appears in the input field automatically or use a new setup key by clicking the key-shaped icon on the right side of the input field. You are recommended to create new engine setup keys and manage them separately if you need to install multiple apps in your PC.



An installed app will not be updated if its engine setup key is newly created in the process of an update.



3 If you select Android, iOS or macOS among build targets, you can set options related to AppBuilder.



Menu	Description
Project information	
Program Name	This is a program name that will appear when the app is installed in an operating system.
Build Environment	
TargetProject	You can either create a new App Builder project or select an existing project.
Project URL	This is the location to which your app will be deployed. The default value is an URL automatically created by AppBuilder.
Package Name	Displays the Package name.
Version	Specifies the file version.
Build Type	- Build Builds a selected project. If the project has been built before, it will be built anew. - Upload Contents Uploads only the contents of a selected project.
Build Library	You can select the Nexacro libraries that will be included in the project build.

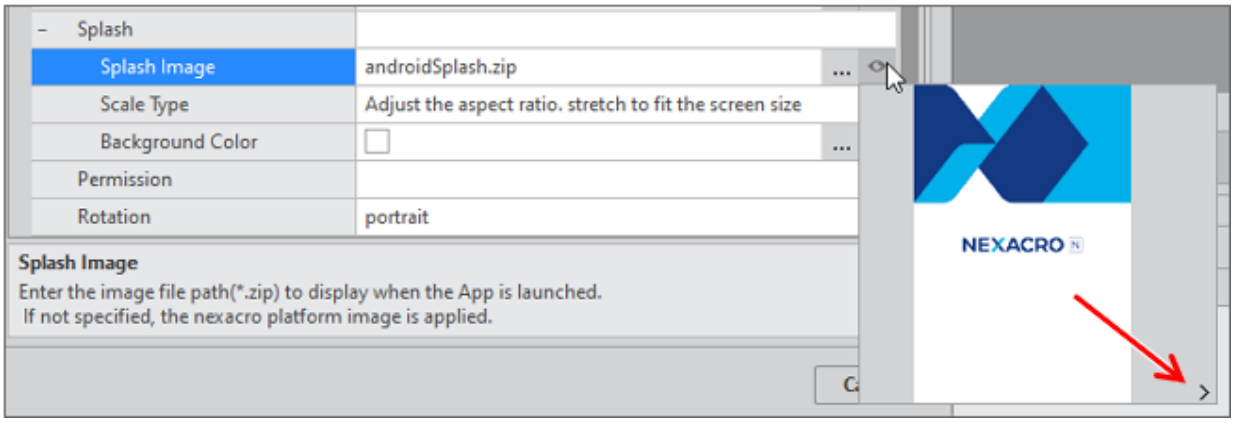
Menu	Description
Project information	
Build Mode	Selects the build mode.
Auto Deploy Server	
Deploy Mode	Configures whether to use the Deploy server. When selecting true, you can select Deploy Server.
Deploy Server	When the Deploy Mode item value is set to true, selects the Deploy server from the Deploy Server list set in App Builder.
Deploy Directory	Displays the path set in Deploy Server. The setting value can be modified.
[Each operating system]	
Setup Filename	Specifies the name of an installation file to create.
Setup Version	Specifies the version of the installation file.
[Each operating system > Advanced]	
Signing	Defines the signing that will apply to the project build.
Plist URL	This is the URL for a plist file for updating in-house. You can change the URL if you run a separate server for deployment. If you use the AppBuilder server, the default value will be used. This option is applicable only to iOS.
Built-in Library	You can select 32-bit and 64-bit shared libraries that will be applied to the build of an installation file.
User Library	You can select the user libraries that will be included in the project build. This option is applicable only to Android and iOS, not to macOS.
Icon	Specifies an image file for the icon of the app.
Splash Image	- Splash Image Specifies an image file for the splash screen of the app. -Scale Type Set the Splash image size. You can choose one of the 4 options below. > Adjust the aspect ratio. stretch to fit the screen size : Adjust aspect ratio. Stretch processing to fit the screen size > Maintain the aspect ratio. expand without blank area : Maintain aspect ratio. Extended processing without blank areas > Maintain the aspect ratio. expand to fit : Maintain aspect ratio. scale to fit > Original size : Original image size - Background Color Specifies the splash area background color.

Menu	Description
Project information	
Dmg Image	Specifies a .dmg file as an Apple disk image. This option is applicable only to macOS.
Permission	Specifies permissions for running the app. This option is applicable only to Android rather than iOS and macOS.
Rotation	Selects the option to pin the screen to Portrait or Landscape or unpin to enable the Sensor.

Icon, Image Files

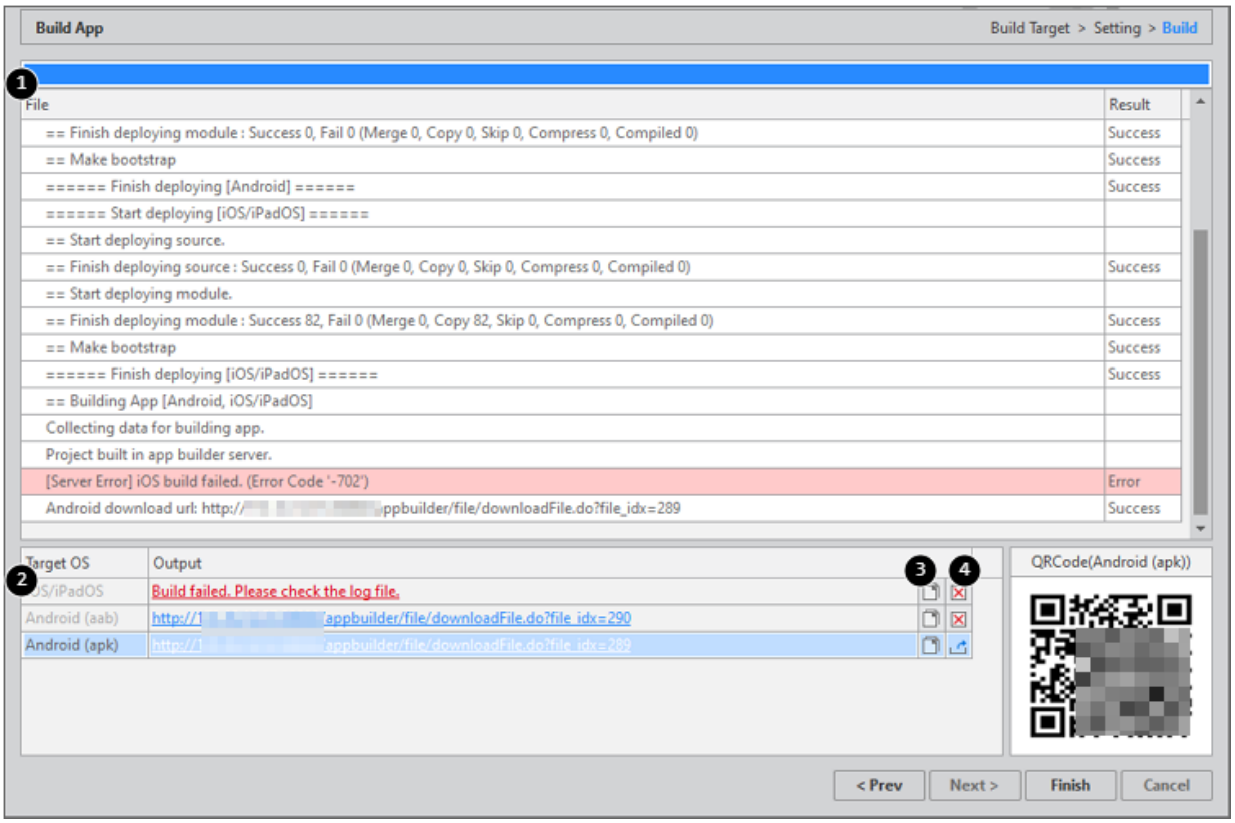
For icon or image files, you should specify appropriate file formats according to operating systems.

When an image file, such as Icon or Splash Image is included, the image preview function is provided when the eye-shaped icon is clicked. If you have bundled multiple image files in a zip file, then you can click the [>] button in the lower right corner to view the next image.



4 Click the Build button to create installation files as configured.

Depending on the operating system or file, if QR codes are supported, the QR code image will be displayed when the deliverable is selected; otherwise, only the deliverable URL is provided. If the build fails, it is displayed in red.



		Description
1	Log	Shows the logs of the build process Error messages will be presented if errors have occurred.
2	Download url	Provides the URLs to download the executable files for each running environment Hovering a mouse pointer over a URL displays the relevant QR code on the right-hand side.
3	Copy url	Copies the URL on the clipboard
4	Save QR Code	Saves the QR code in the form of a PNG file

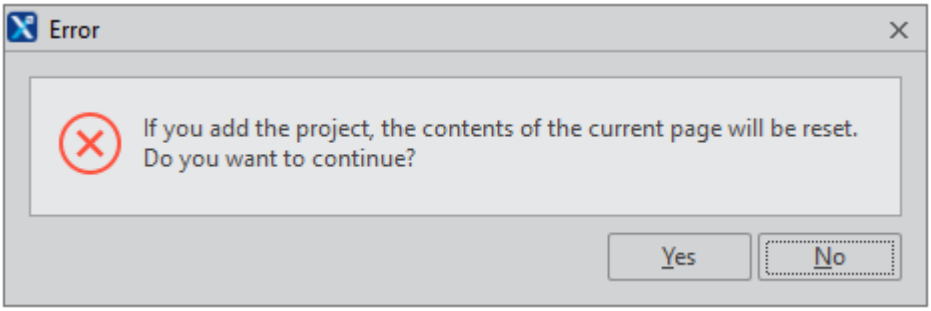
17.3.2 Set Options for AppBuilder

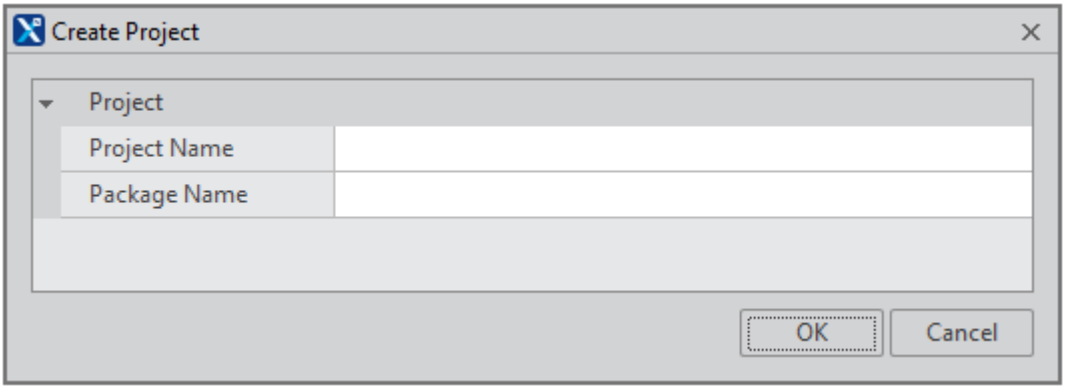
If you build an app through AppBuilder, you need to create an AppBuilder project and set related options, which can be done through the console of AppBuilder. Meanwhile, the Build App wizard can handle the following jobs: creating an AppBuilder project, changing the framework library, registering user libraries and setting permissions.

Create TargetProject (App Builder Project)

`Setting > Build Environment > Target Project`

To use App Builder, you need to create an App Builder project first. You can select an existing project if there is one. If there is no existing project, click the + button to create a new project.



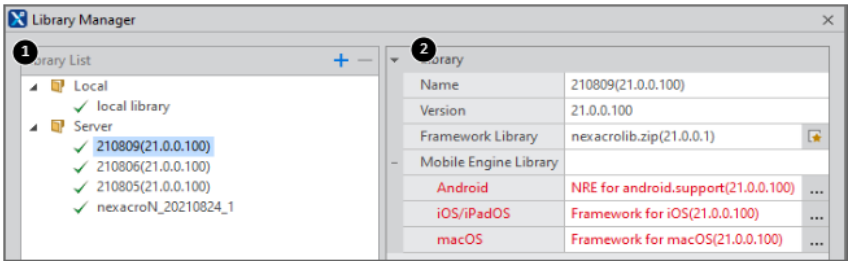


If the package name does not match the value of Signing, App Builder may fail to create an installation file.

Register or Change Build Library

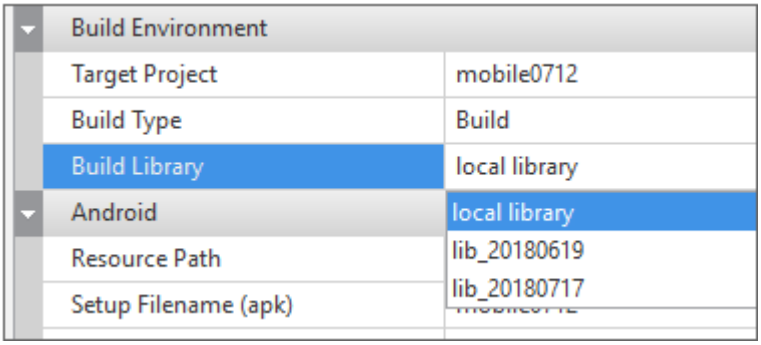
`Setting > Build Environment > Build Library`

You can manage the Nexacro framework library as well as libraries for each operating system. You can edit existing libraries and add new library files.

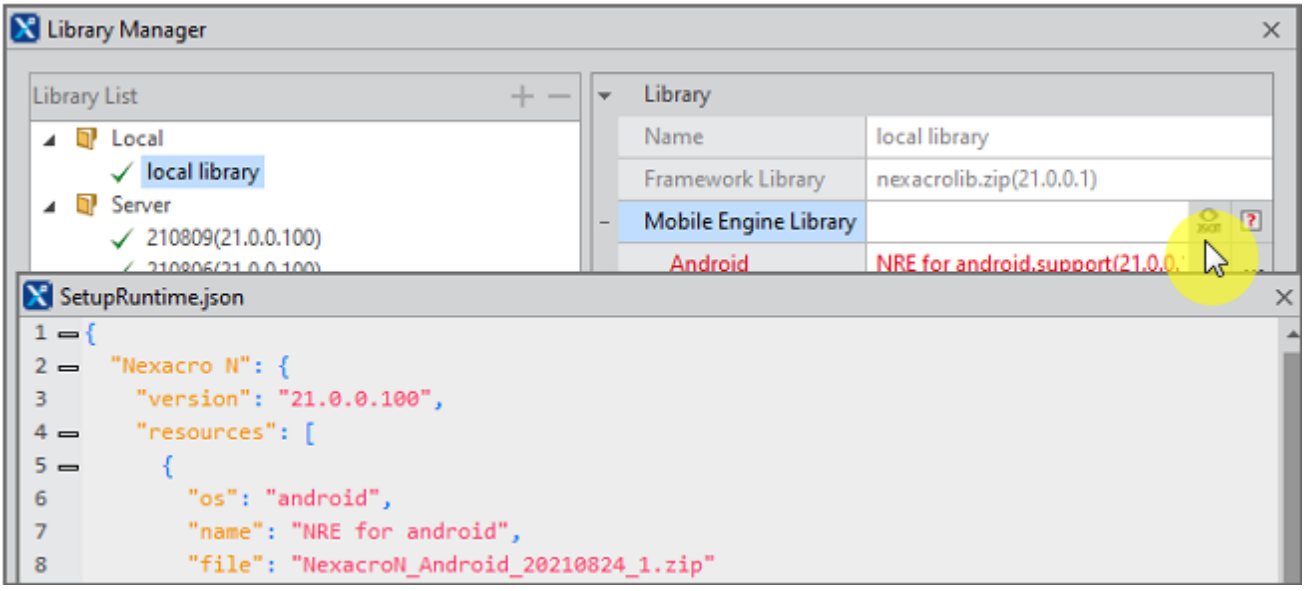


		Description
1	Library list	Displays the list of libraries registered on the App Builder server and libraries to be newly registered. Displays icons in a different form for the items being modified and newly added, depending on the status.
2	Library configuration	Name, Version, Framework Library are required items. At least one of 3 items must be specified for Mobile Engine Library.

If you designate a local library instead of the one registered in the server, App Builder will use the library installed on your PC.

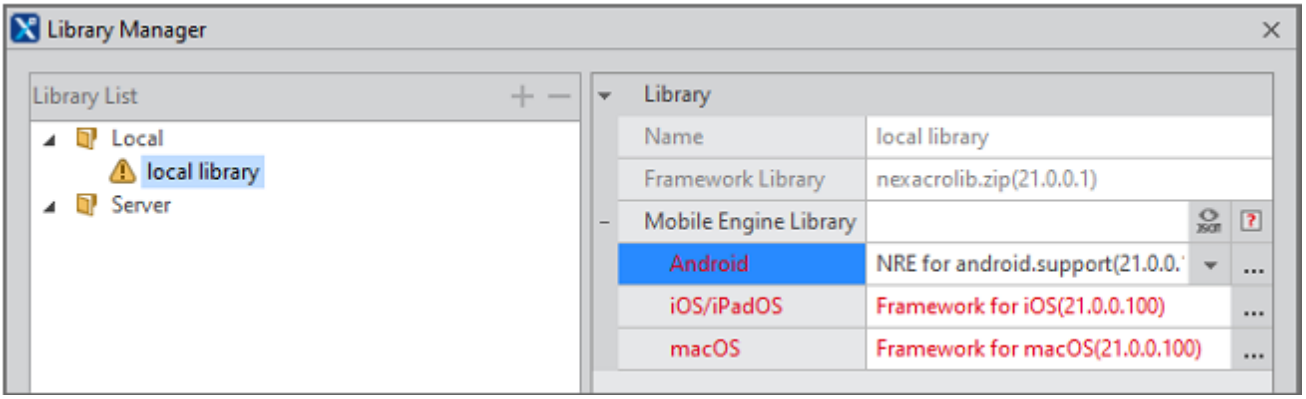


The local library item is set to the product-installed version by default. The list of library files set by default can be found in the SetupRuntime.json file.



The default library file can be changed by the user. Changed contents are maintained until the product updates, and if the SetupRuntime.json file is changed during the product update, then it is initialized according to the corresponding version.

- You can choose a different version of the file from the framework library. If the file version is different, then the item is displayed in red.

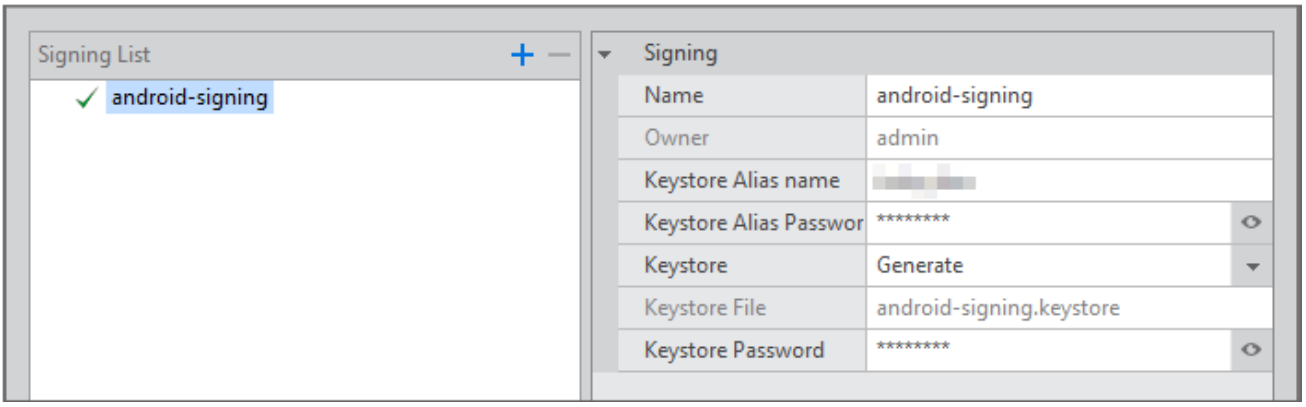


Changing or Registering Signing as New

`Setting > Android, iOS, macOS > Advanced > Signing`

You can register or create the Signing file (for Android) and then check the registered content.

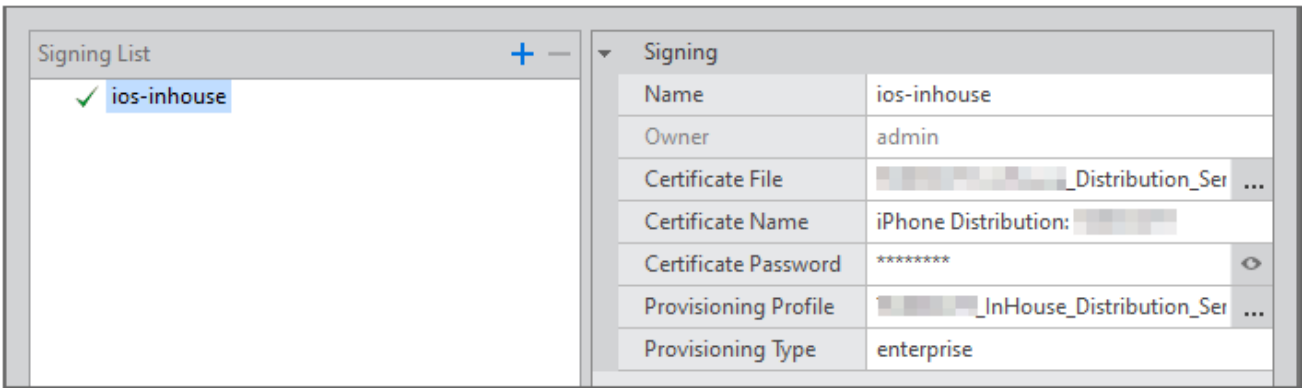
Android



	Item	Description
1	name	Enter the name of the item to register.
2	Owner	Enter the owner or Keystore (or Signing). (It is automatically entered as the login account).
3	Keystore Alias Name	Enter the Keystore Alias name. The Keystore Alias name must be at least 6 characters long.
4	Keystore Alias Password	Enter the Keystore Alias password. The Keystore Alias password must be at least 6 characters long.
5	Keystore File	Create the Keystore file or register the file you have. - Generate: Create the Keystore file in App Builder. - Upload: Register the Keystore file and display the registered file.

	Item	Description
		The registered file is displayed in the form of a link and you can download the file by clicking the link.
6	Keystore Password	Enter the Keystore password.

iOS/macOS

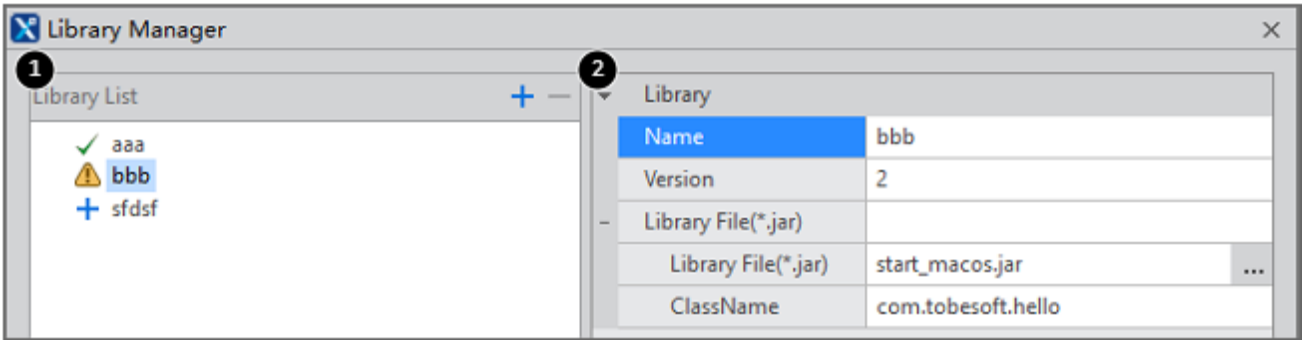


	Item	Description
1	name	Enter the name of the item to register.
2	Owner	Enter the owner or Keystore (or Signing). (It is automatically entered as the login account).
3	Certificate Name	Enter the iOS Certificate name.
4	Certifcate Password	Enter the password for the registered iOS Certificate file.
5	Certificate File	Register the iOS Certificate file.
6	Provisioning Profile (iOS)	Register the Provisioning Profile file.
7	Provisioning Type	Displays the Provisioning Profile file type. This is not an input item and one of enterprise, app-store, ad-hoc, or development is displayed depending on the Provisioning Profile file property.

Register or Change User Library

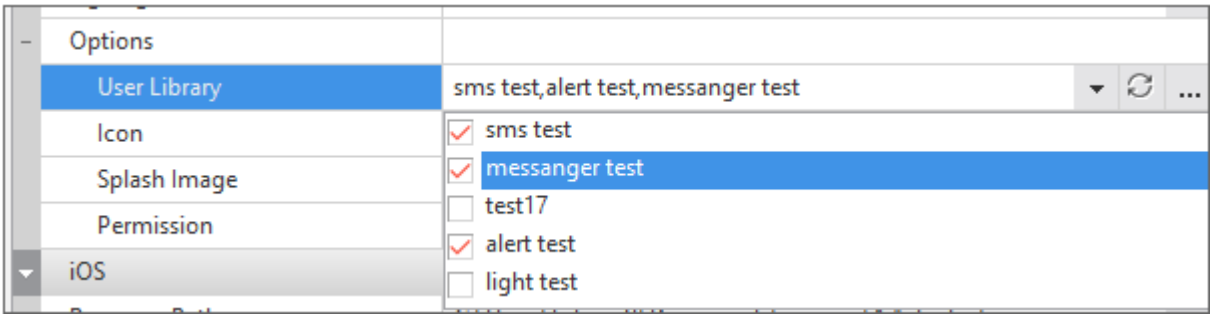
`Setting > Android, iOS > Advanced > User Library`

You can manage user libraries for each operating systems. You can edit existing libraries and add new library files.



		Description
1	User library list	This list shows both the user libraries already registered in the server and the user libraries to be registered.
2	User library configuration	All the items are required. (ClassName is required only for Android.). The below extensions are supported as a library file. Android: *.jar iOS: *.a

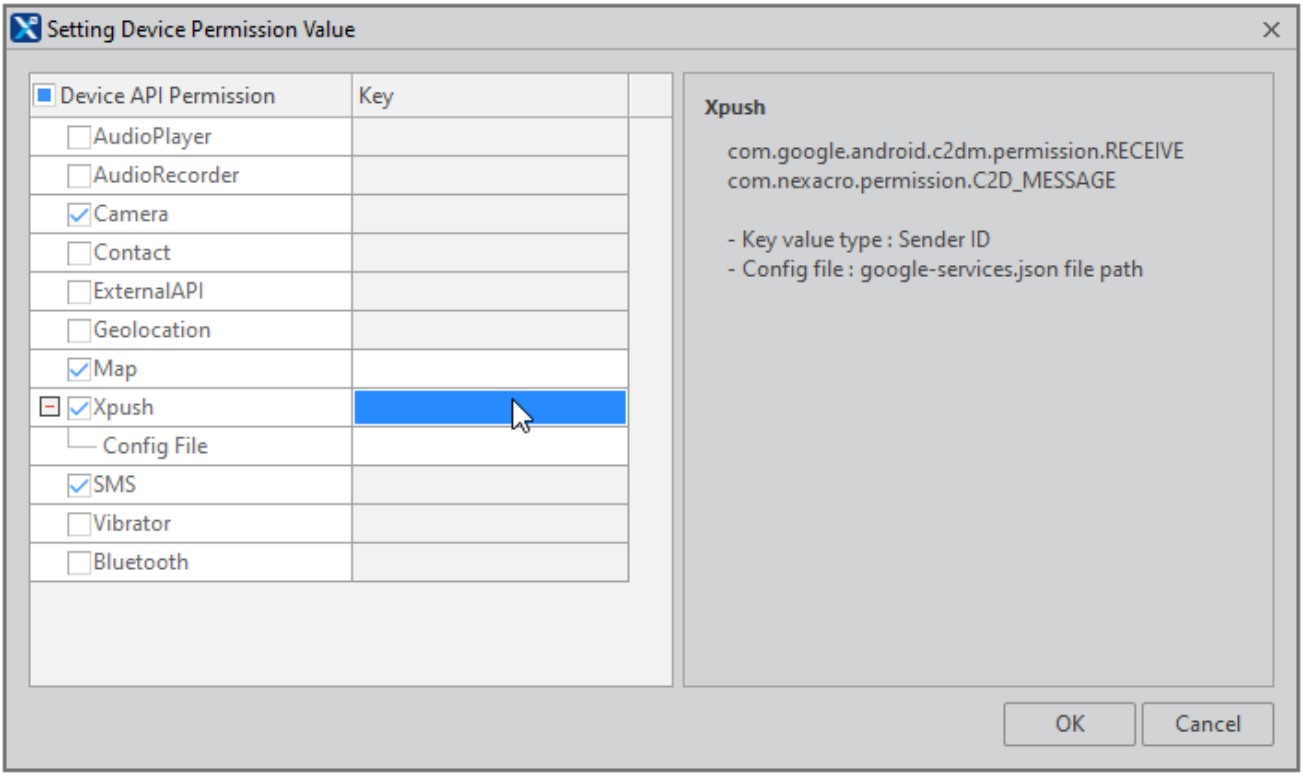
You can select user libraries to apply them to a certain project project.



Define Permissions

`Setting > Android > Advanced > Permission`

You can define permissions required to run an app in Android.

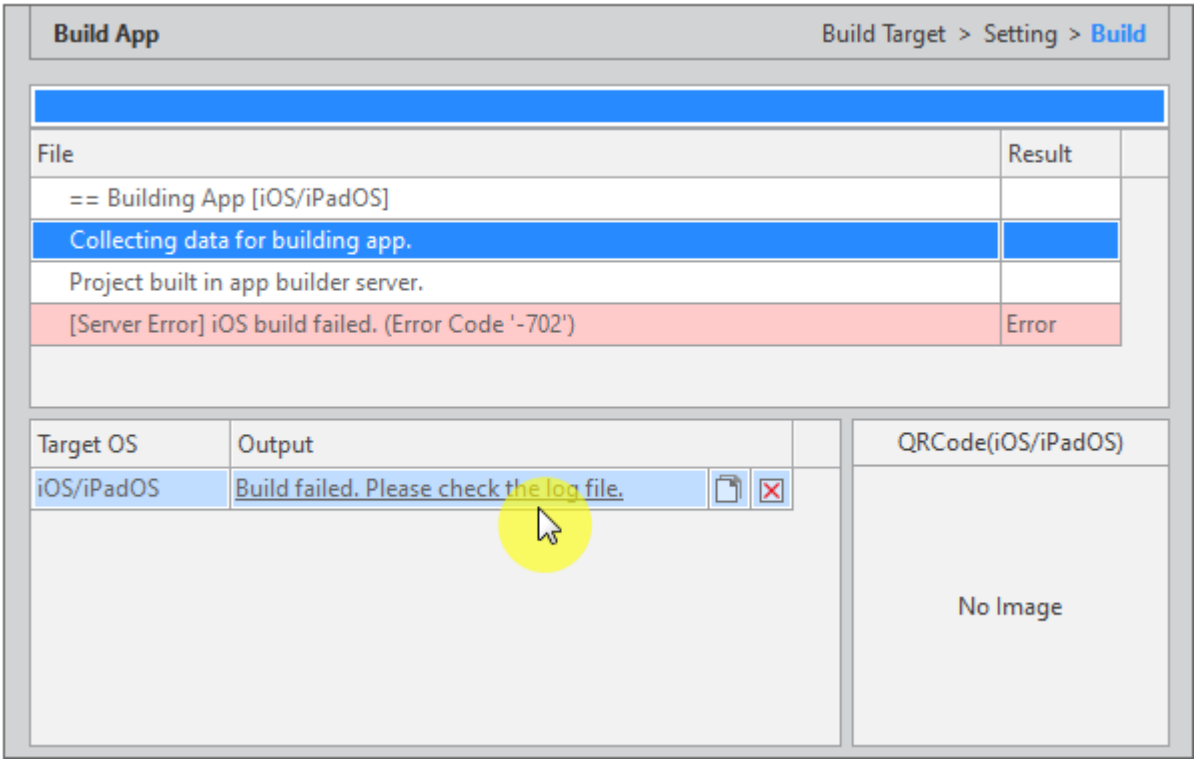


When you designate the permission for Map or Xpush, you are required to enter the key to the device API. Otherwise, the app will not work properly.

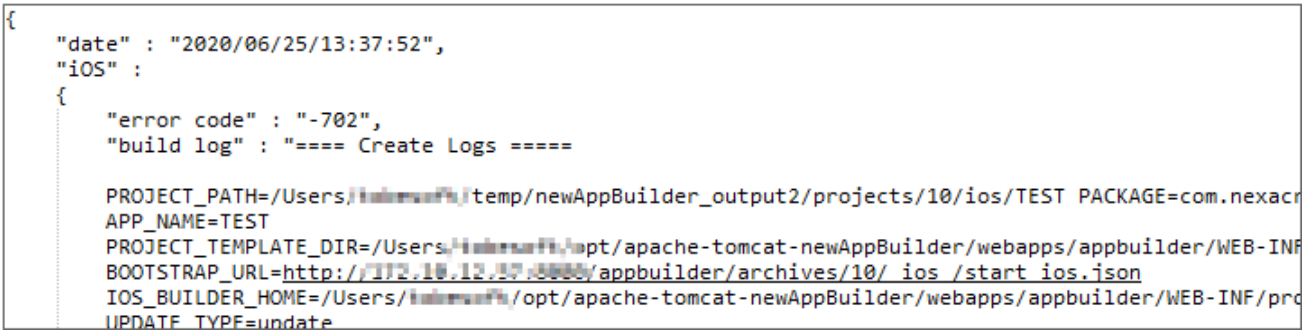
You should attach the Firebase Android config file (*google-services.json*) to designate the permission for Xpush.

17.3.3 Check Logs when AppBuilder Fails to build

If AppBuilder fails to build an app, it will provide a log by presenting a log file link instead of an installation file link.

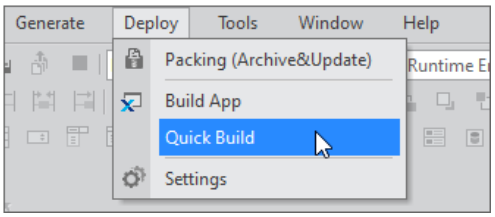


In the log file, you can see an error code as well as the logs created when the build failed.

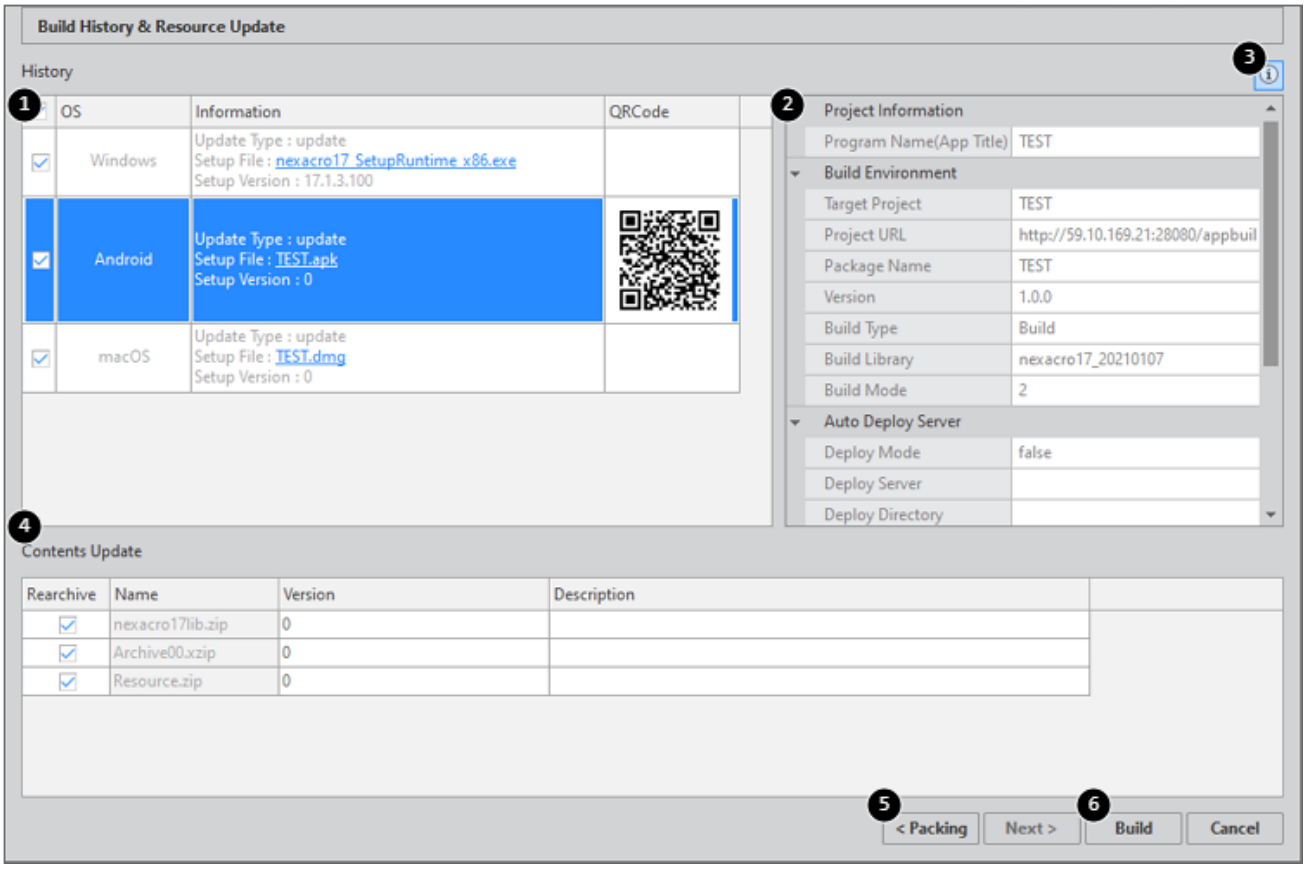


17.4 Quick Build with Previous Settings

If you have the Build history of the project, then you can use the Quick Build function. If there is the Build history, then the menu [Deploy > Quick Build] item is enabled.



In the Quick Build Wizard, you can check the list of each operating system in which the Build has been performed, download the app, or modify the version information of the edited resource file, and then proceed with Build.



	Item	Description
1	History	Displays the Build history for each operating system. Provides the link to download Setup File and the QR Code information. Runs Build only for the selected operating system.
2	Detail	Displays some partial setting values set in Packing, Build Wizard. The content cannot be modified and can only be used for checking.
3		You can close or open the Detail window by clicking the button.
4	Contents Update	Select the Resource to update and modify the Version information. Only the selected Resource is processed.
5	Packing	Runs the Packing wizard.
6	Build	Runs Quick Build. Depending on the Update Type, the internally processed operation varies.

When clicking the [Build] button, it operates as follows according to Update Type.

	Update (Local + Server)	Server	Local
Windows	Packing	Packing	Packing Build

	Update (Local + Server)	Server	Local
Android **iOS** **macOS**	Packing Upload Contents	Packing	Packing Build

18.

Migration

You can convert the version of a Nexacro Platform project from 14 to Nexacro N by using the migration feature of Nexacro Studio.

Nexacro Studio provides the Migration Wizard, which automatically adjusts the configuration of a project such as structure, files and syntax. Moreover, the Studio offers several other tools for comparing and analyzing two different versions so that you can succeed in migration.

In this chapter, you will learn how to use the migration feature provided by Nexacro Studio.



The migration feature provided by Nexacro Studio is just a tool to support migration, meaning that it does not guarantee a perfect migration of a project. If you want to migrate your project perfectly, you need to check the result of the migration performed by Nexacro Studio and then fix the files that were failed to be converted.

18.1 Procedures

The process of migration through Nexacro Studio can be largely divided into four steps: opening a project, Migration Wizard, fixing errors and ending the migration.

- Opening a project is the beginning stage where Nexacro Studio reads your project and determines whether the project needs migration. If the project has been migrated before, you will be prompted to choose a mode for opening the project.
- The Migration Wizard is a migrator that automatically converts the version of your project from 14 to Nexacro N.
- Fixing errors refers to the finishing works where you check the errors resulting from the operation of the Migration Wizard and then fix those problems.
- Ending migration is the last stage where you decide not to open your project in the migration mode anymore after concluding that all the tasks related to the migration have been done.

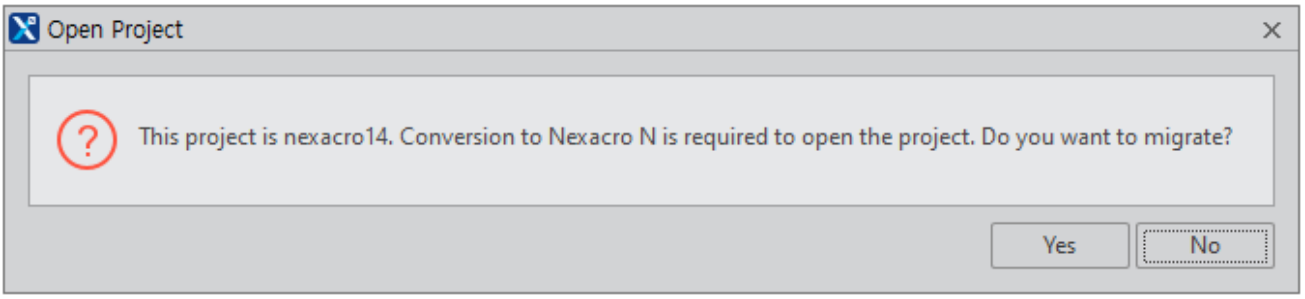
18.1.1 Opening Project

In this step, Nexacro Studio opens your project that is based on Nexacro Platform 14. Afterwards, two possible scenarios will be played out depending on whether you have conducted the migration process for the project before: one scenario is about when you have never performed the migration process before while the other is about the opposite.

When you performs migration for the first time

When Nexacro Studio opens an unmigrated project, you will be asked whether you will conduct migration process as presented in the below picture.

If you want to conduct the process, click Yes.

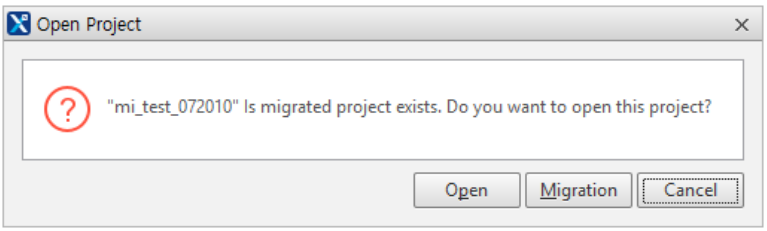


	Description
Yes	Initiates the migration process
No	Closes the dialog box and the project

If this is the case, you can go to Migration Wizard directly without having to read the below section.

When you have performed migration before

When you open a Nexacro Platform 14-based project that has been migrated before, Nexacro Studio will inform you that there is already a migrated version. Then, the Studio will ask you which action you will take next, as presented in the below picture.



Choice	Description
Open	Opens the already-migrated version
Migration	Starts a new migration
Cancel	Cancels the opening of the project

18.1.2 Migration Wizard

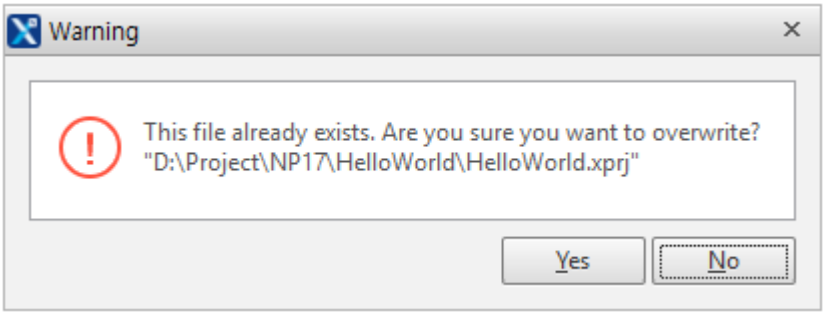
When you first open a project that is based on Nexacro Platform 14, you will be asked whether to migrate the project. If you click "Yes", the Migration Wizard will begin running. The wizard is a tool that performs tasks that you otherwise have to do by yourself: to automatically establish the configuration of a project in a way that fits Nexacro N. For instance, it automatically changes directory structure, environmental data, files, versions and syntax.

To begin with, you need to designate input and output path as illustrated below. Then click the Migrate button to start the process.

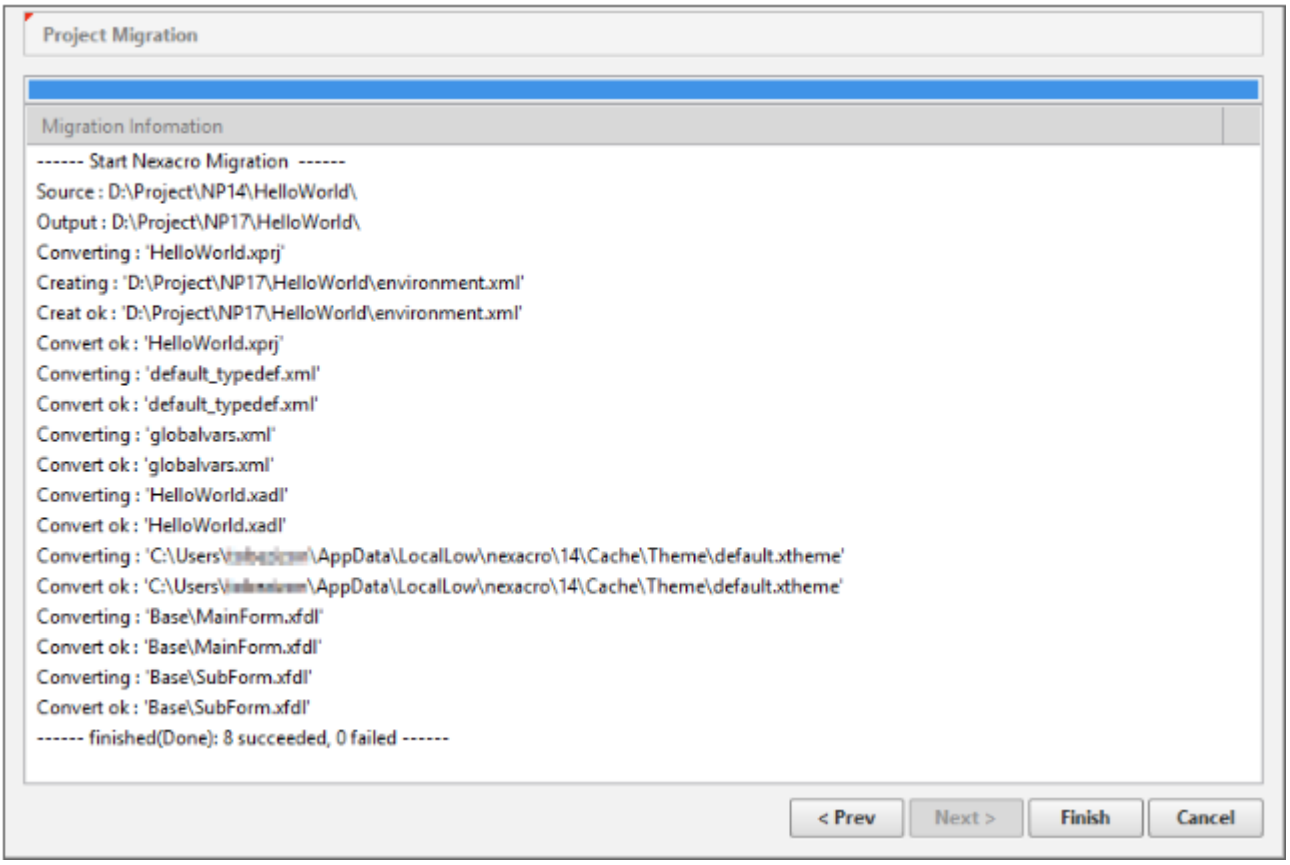


Call-out		Description
1	Target Path	Path to a Nexacro Platform 14-based project that will be migrated
2	Output Path	Path to the resulting migrated project. The default value is a working folder defined in the menu [Tools > Options > Environment > General].
3	Migrate	Starts the migration process
4	Cancel	Cancels the migration

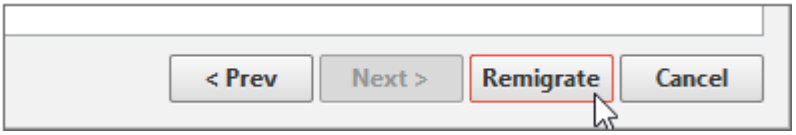
If a project with the same name already exists in the output path, an alert box will appear, asking you whether to overwrite the existing project.



Once an automatic migration begins, you can see the process through log messages, as illustrated below.



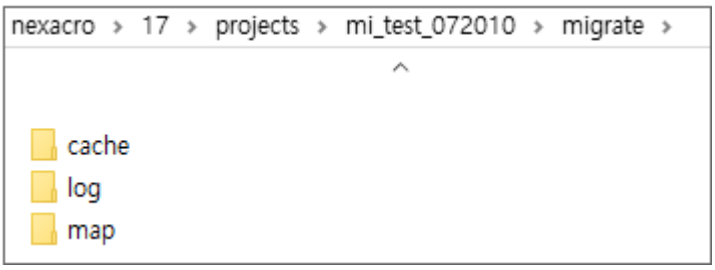
If you want to suspend the process, click the Stop button. If you want to resume, click the Remigrate button.



When the automatic conversion process comes to an end, the message "finished" will be displayed with the numbers of the items that have succeeded or failed in migration. At the same time, the Finish button will be activated.

- If you click the Finish button, the project will be opened in the migration mode after the wizard is closed.
- If you click the Cancel button, Nexacro Studio will do nothing and close the Migration Wizard. The migrated project produced by the wizard will be preserved along with the migration history. When you open the project again later, therefore, you will be asked to continue the migration tasks from where they were left off.

If you have successfully performed the tasks so far, you will find a resulting Nexacro N-based project in the output path you defined above. Among the directories under the project, there is a log file in the *migrate* folder, and it contains the information on the results of migration. You should not modify or delete this file since it is essential for the next step where you fix the files that have been failed to be converted.



18.1.3 Fixing Errors

Once you perform a conversion process by using the Migration Wizard, you can open the resulting project in the migration mode. This mode allows you to review the logs of the conversion process, trace the files that have been failed to be migrated and fix the related errors. This error-fixing process is imperative as it helps you solve the problems related to the functions not provided by Nexacro or to user-written codes.

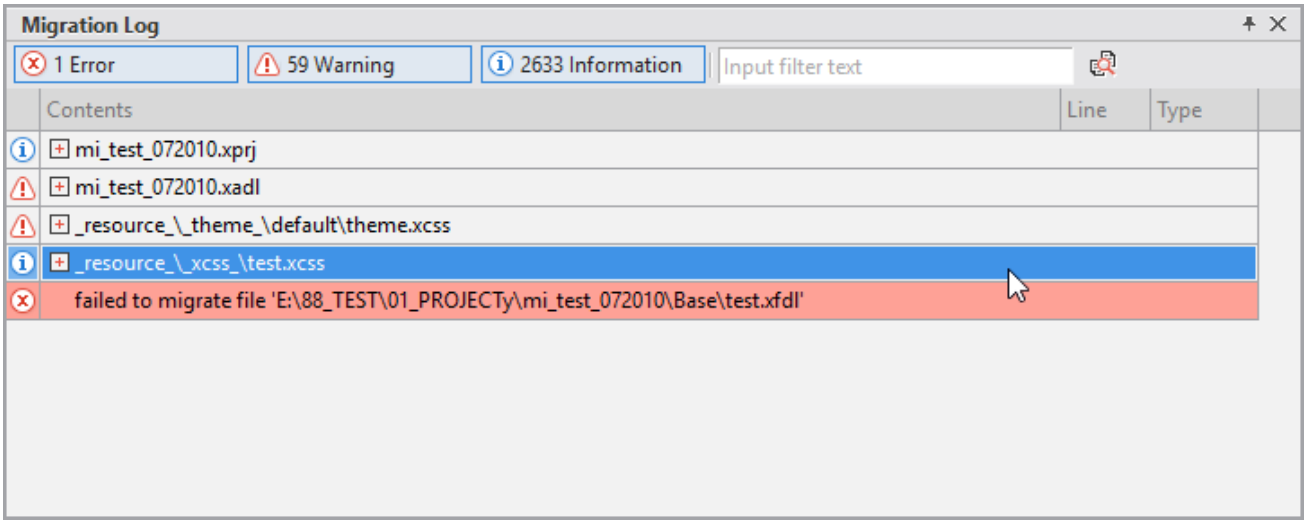
For the details about how to use the migration mode, go to Migration Mode Menu.

You will take the below steps to fix errors.

1. Check migration log: identify a problematic file, determine which part of it causes an error and figure out how the code was changed from its original.
2. Fix error: analyze the error and fix it. To this end, you need to change the existing code slightly or sometimes even to rewrite it completely.

Check migration log

Migration Log allows you to check the items that have failed to be converted or that contain errors.

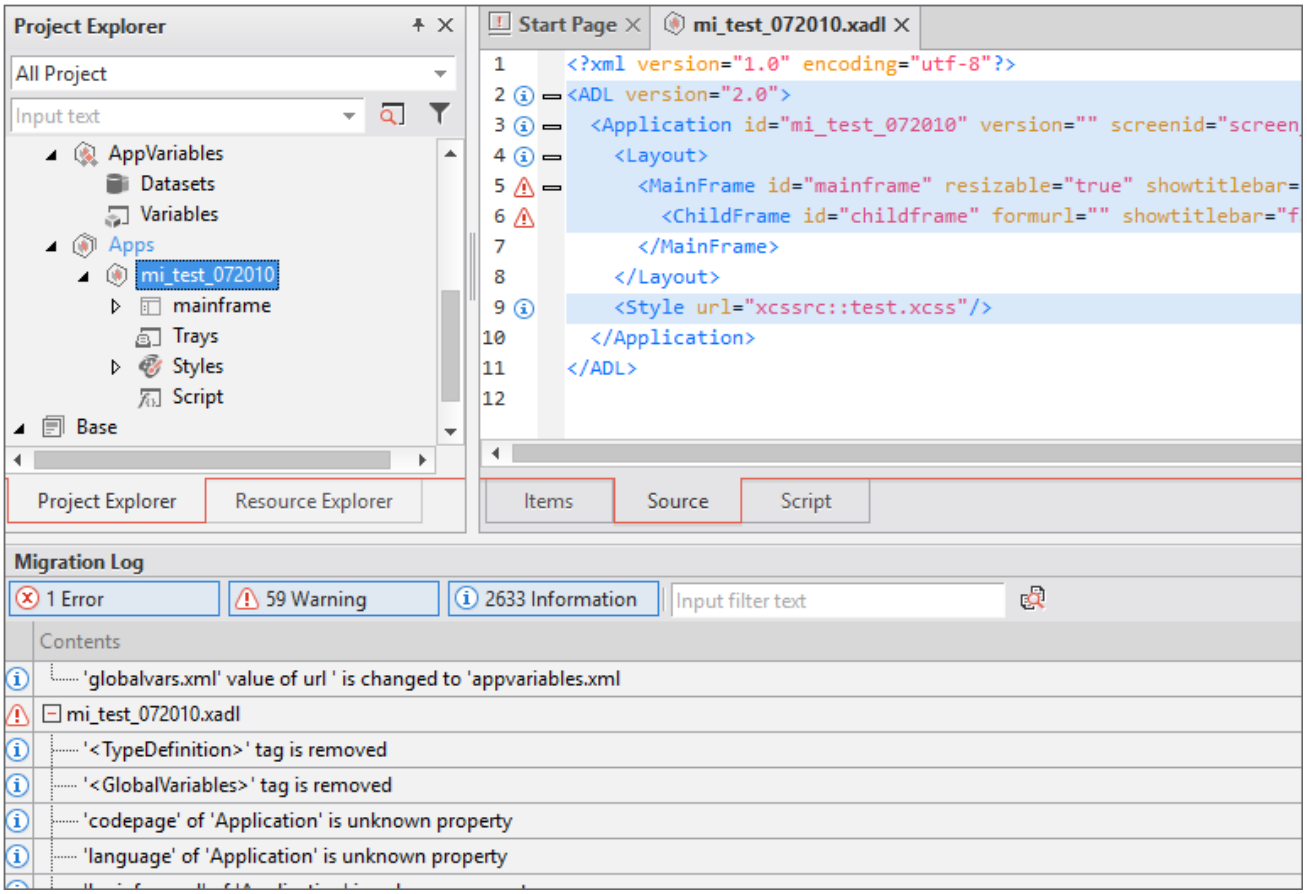


Migration Log is organized in a tree structure with each level containing the following information: **file information** > **changes** > **relevant line**. Each item is featured with an icon that is marked with one of the four symbols indicating log types. The below table presents the descriptions of those symbols.

Symbol	Log type	Description
	Error Log	It indicates a failed conversion or an error. You must resolve this case as it will otherwise prevent the operation of the project.
	Warning Log	It indicates an arbitrary modification or deletion. Although this case does not block the opening or operation of the project itself, you better take a look as it may prevent intended functions.
	Information Log	It indicates that the item has been converted or created successfully. This log provides you with the information on the changes made.

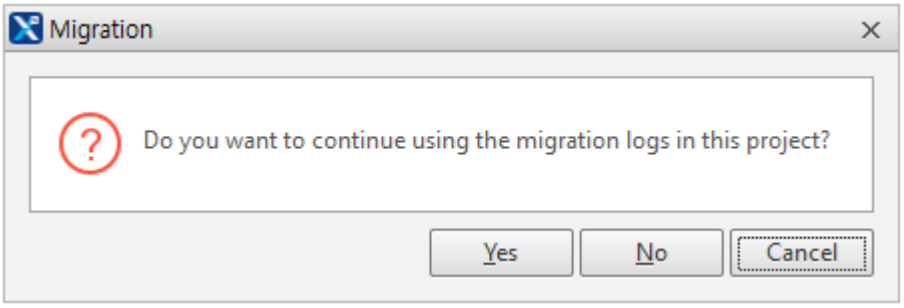
Fix error

You need to think about the solutions for the items that the Migration Wizard has failed to convert. While you can fix some errors with simple changes, other errors may require you to devise different functions as replacements or even to create new files based on Nexacro N.



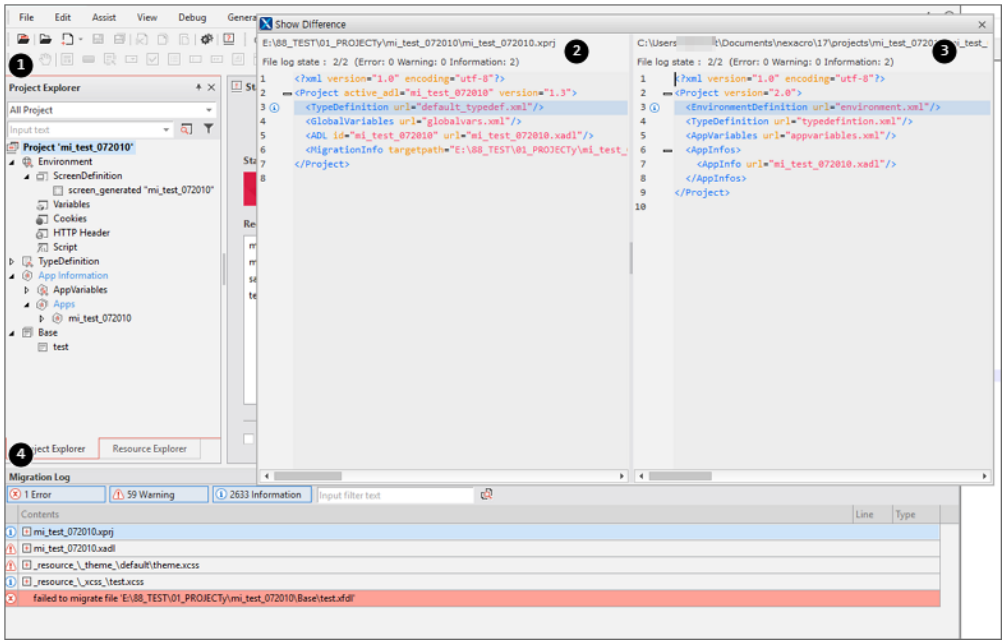
18.1.4 Ending Migration Project

When you close or update a migration project, you are asked whether to maintain the migration logs. If you click "Yes", the migration logs will continue to appear next time when you open the project. If you click "No", the migration logs will be deleted while the project will be opened in the project mode.



18.2 Migration Mode Menu

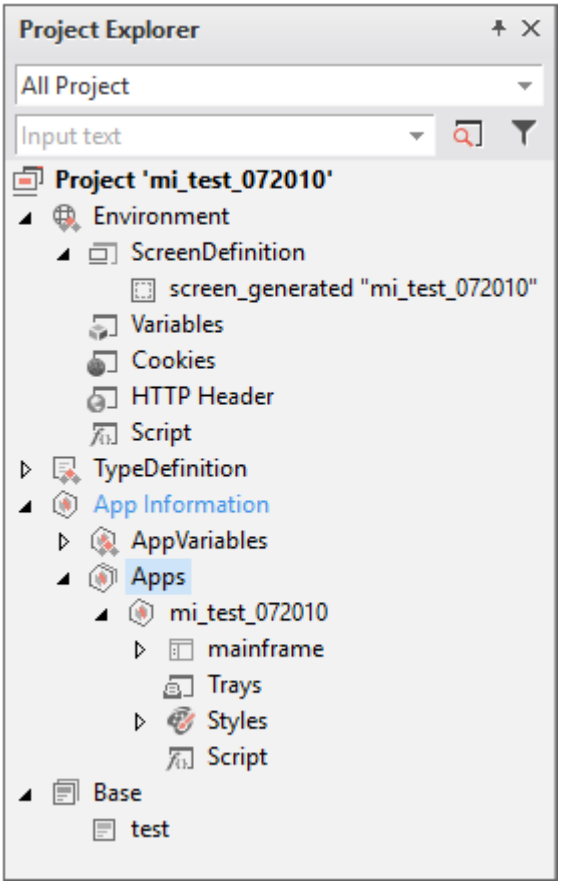
After you finish the automatic conversion conducted by the Migration Wizard, you can open the resulting project in the migration mode. This mode consists of the Project Explorer, Migration Log and DiffView. The Project (Migration) Explorer manages the files that are included in a converted project while the Migration Log pane presents the errors that occurred in the process of the automatic conversion. The DiffView pane allows you to compare two versions of source codes—before and after the automatic conversion. With the foregoing tools, you can see the changes made and modify files if necessary.



Call-out		Description
1	Project Explorer	It explores the file produced as a result of a migration.
2	Target File View	Source codes of the file before the migration
3	Migration File View	Source codes of the file after the migration
4	Migration Log	It shows the logs of the results of a migration. It shows the logs of each file.

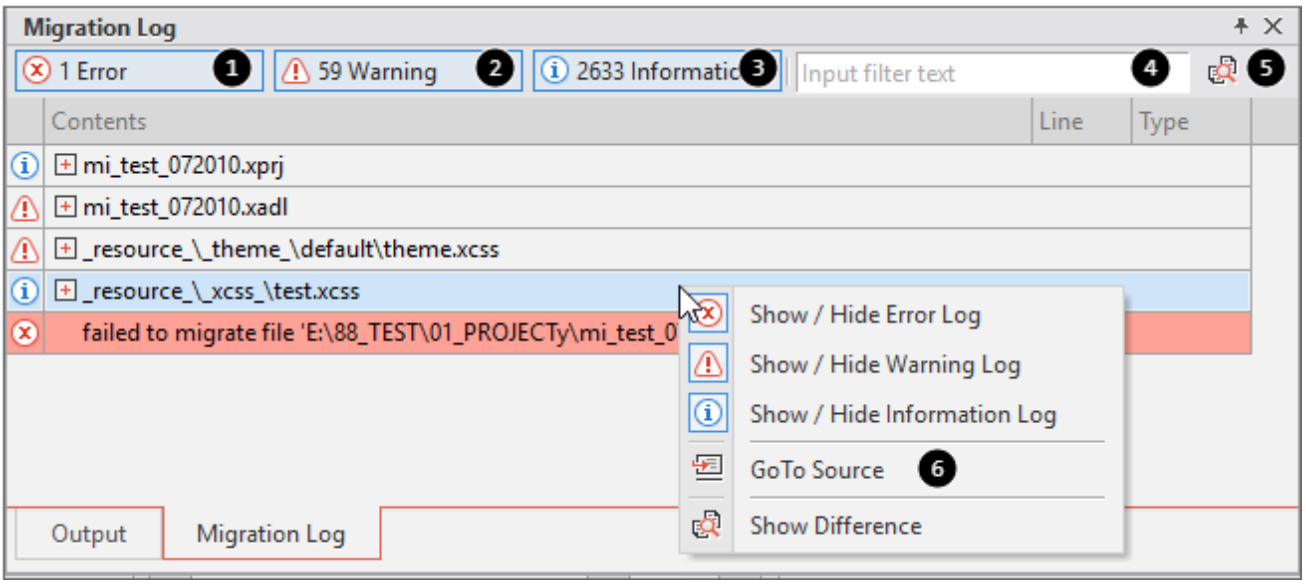
18.2.1 Project Explorer

The Project Explorer in the migration mode basically functions the same as the Project Explorer in the project mode, which lists the files in a project in the form of a tree. The difference is that the explorer in the migration mode presents the information on migration by using colored file names.



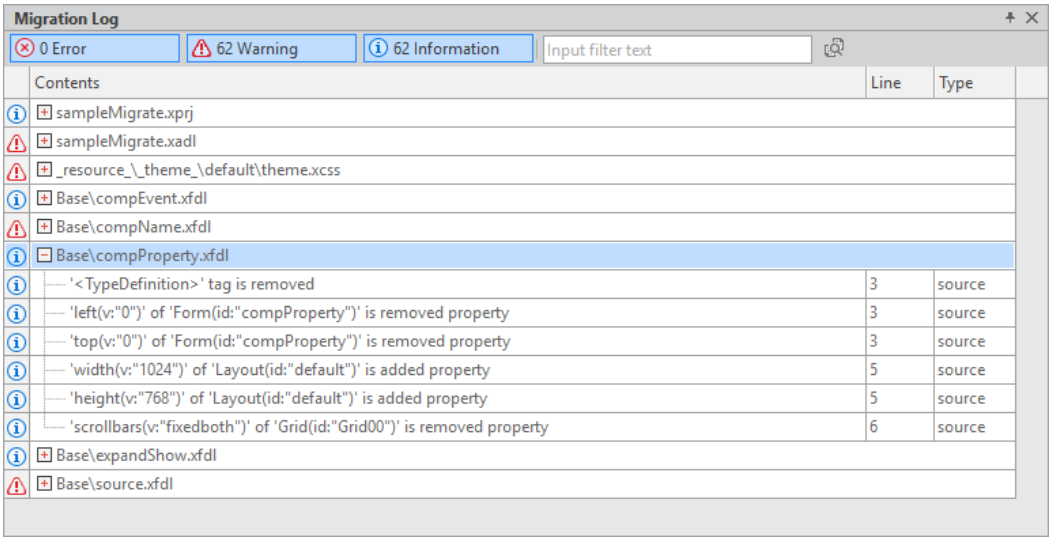
18.2.2 Migration Log

Migration Log presents the results of conducting a migration by giving detains on the changes.



Call-out		Description
1	Show/Hide Error Log	Shows or hides error logs
2	Show/Hide Warn Log	Shows or hides warning logs
3	Show/Hide Info Log	Shows or hides information logs
4	Input Filter Text	Enters filter text
5	Show Difference	Compares a migrated file and its original file
6	GoTo Source	Displays the source codes of the selected file through Nexacro Studio

You can see the details on a warning log or information log by expanding the relevant tree.



18.2.3 Migration DiffView

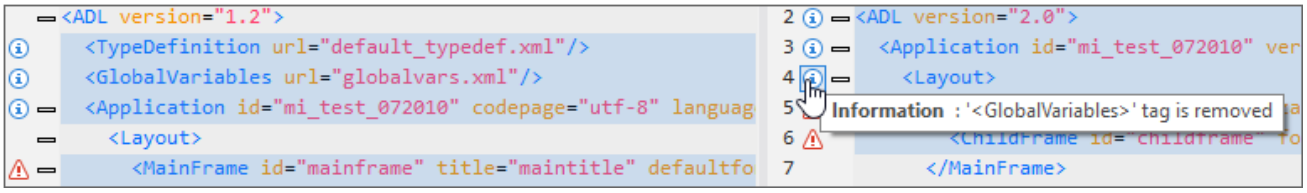
You can open the DiffView editor by right-clicking a certain file from Migration Log and select Show Difference from the context menu.

The below picture shows DiffView opening a file. The original file is on the left pane while the migrated file is on the right pane. Changes are highlighted with blue color.



Call-out		Description
1	Open File	Path to the current file
2	Log Information	Information on the logs of the whole file (log type. number of updated logs and number of remaining logs)
3	Original Source View	Shows the source codes of the original file. Logs are presented for each line. You cannot edit the source codes.
4	Migration Source View	Shows the source codes of the migrated file. Logs are presented for each line. You cannot edit the source codes.
5	Diff Line	When you double-click an item from the Migration Log tree, you will be led to the relevant line displayed on DiffView. The relevant line will be indicated with the 'selected' style.

On the right pane where a migrated file is opened, you can see the information on changes and edit the source codes.


<ADL version="1.2">
<TypeDefinition url="default_typedef.xml"/>
<GlobalVariables url="globalvars.xml"/>
<Application id="mi_test_072010" codepage="utf-8" languag
<Layout>
<MainFrame id="mainframe" title="maintitle" defaultfo
<ADL version="2.0">
<Application id="mi_test_072010" ver
<Layout>
Information : '<GlobalVariables>' tag is removed
</MainFrame>

19.

Development Facilities

This chapter introduces useful features of Nexacro Studio, which you can utilize when editing files.

19.1 Assist

19.1.1 Copy Full Path

This function is designed to copy a current file's path and store it on the clipboard. If a certain file in the tree of the Project Explorer is selected with the explorer activated, the path of the relevant file will be copied.

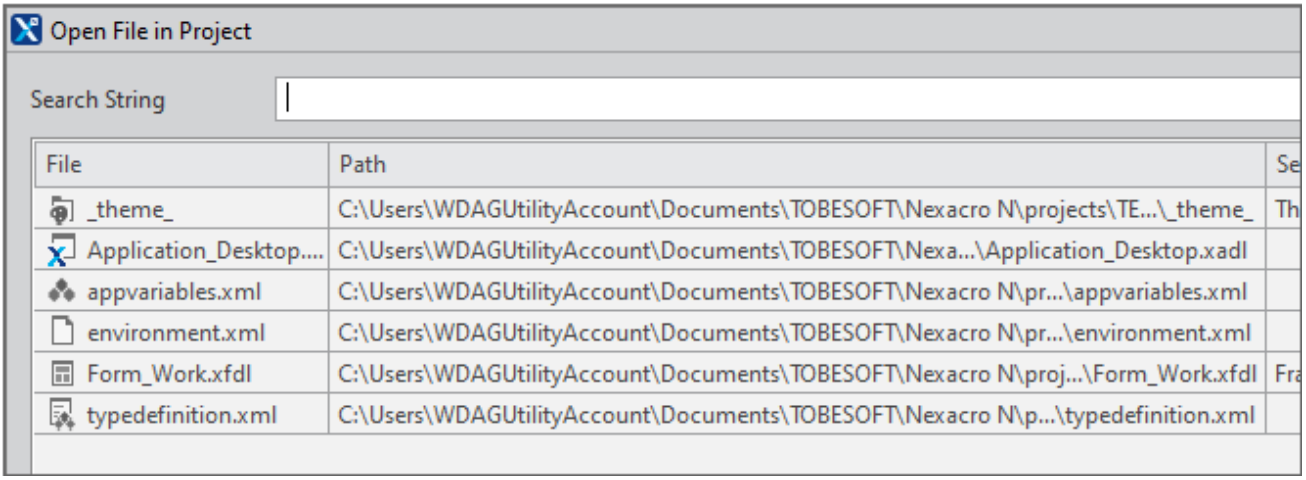
19.1.2 Open Containing Folder

This function is designed to open the directory of a current file through the file explorer set by the user's operating system. If a certain file in the tree of the Project Explorer is selected with the explorer activated, the directory of the relevant file will be opened.

19.1.3 Open File in Project

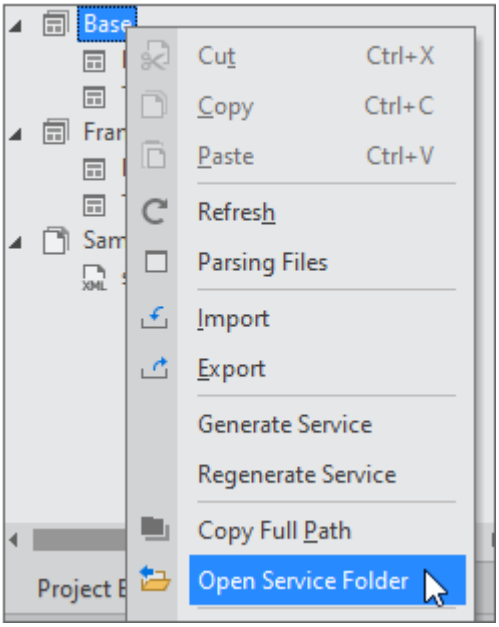
This function is designed to show a list of files that are loaded under a current project.

If you enter characters in the Search String box, the Open File in Project window will show you a list of search results, which are the files whose names include the input characters. The list will be renewed in real time as you type more characters. Double-clicking a certain item in the list will open the editor of the relevant file.



19.1.4 Open Service Folder / Open Folder

This function is used to open the pertinent folder of a service or directory (sub-directory of a file type service). To use This function, open the context menu by right-clicking a relevant item on the Project Explorer.



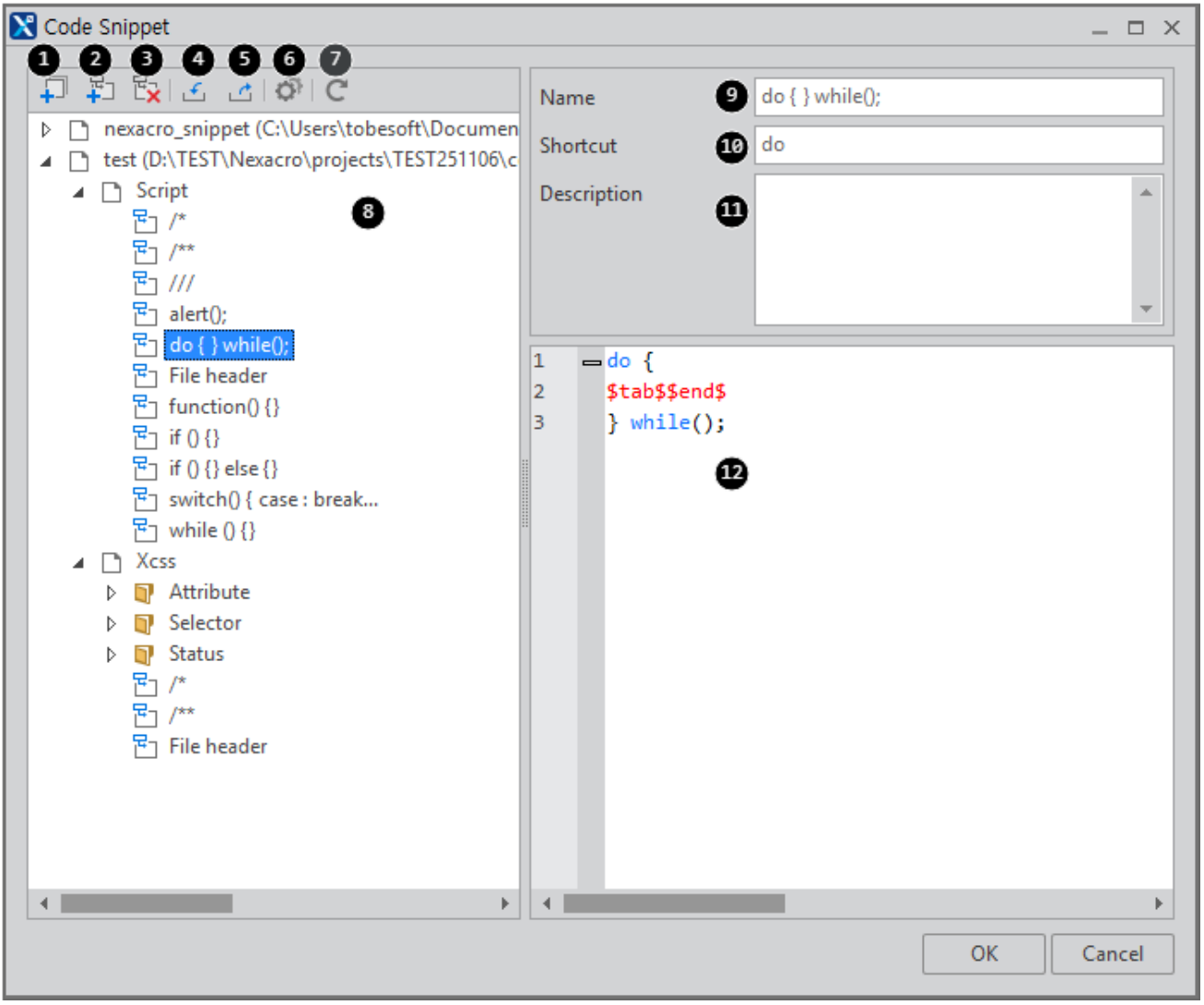
19.1.5 Code Snippet

A code snippet means a piece of codes that is defined by a user in advance. You can define your frequently-used codes as snippets and apply them when you edit files.

Defining a Code Snippet

You can define a snippet by opening the Code Snippet Editor—[**EDIT** > **Code Assist** > **Snippet**] in the ribbon interface or [**Assist** > **Code Snippet**] in the command bar interface.

Basic code snippets are provided by Nexacro Studio while you can add your own snippets.



	Menu	Description
1	Add Group	Adds a code snippet group
2	Add Code Snippet	Adds a new code snippet
3	Delete Code Snippet	Deletes a selected code snippet
4	Import Code Snippet	Imports the code snippet created as the XML file or selects the xpackage file and runs the Import Wizard.
5	Export Code Snippet	Saves a code snippet in the form of an XML file
6	Options	Configures the code snippet file location.
7	Refresh	Refreshes with the updated content when the code snippet file is changed externally.

	Menu	Description
8	Code Snippet List	If two or more code snippet files are set in the options, two files are displayed.
9	Name	Name of the selected snippet
10	Shortcut	Defines a shortcut name used to call the relevant snippet (When you code in the XCSS or Script editor, you can apply a code snippet by entering the name defined as a shortcut of the snippet.)
11	Description	Description about the snippet
12	Code Snippet Source	Defines the codes of the snippet

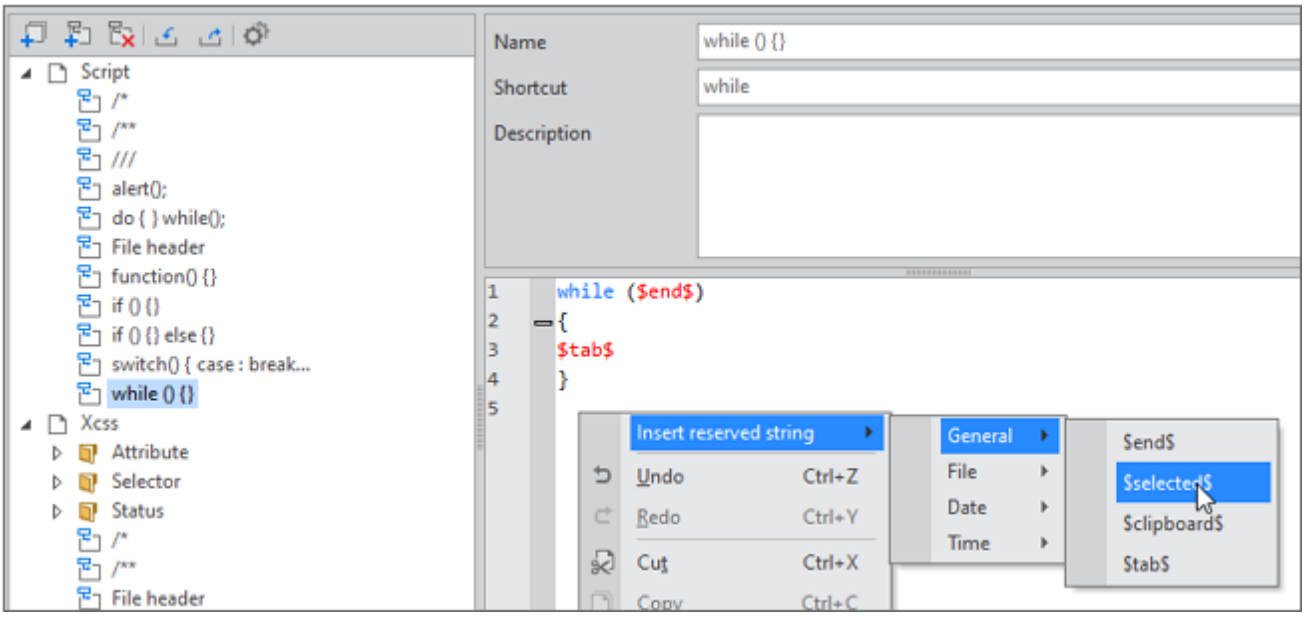
Code Snippet Reserved string

You can use the below reserved words when defining code snippets. For instance, $DATE$ will change a character string into an appropriate format that indicates a today's date.

Reserved string		Description
General	end	Desired position of a caret after applying a code snippet
	$selected$	Currently selected character string; null if there is no selection
	$clipboard$	Character string copied to the clipboard
	tab	Tab character
File	$FILE$	File path that contains the file name
	$FILE_EXT$	File name with its extension
	$FILE_BASE$	File name without its extension
	$FILE_PATH$	File path
	$FILE_EXT_UPPER$	File extension (upper case)
	$FILE_BASE_UPPER$	File name without its extension (upper case)
	$FILE_PATH_UPPER$	File path (upper case)
Date	$DATE$	Year/month/day formatted as "yyyy/MM/dd"
	$DATE_REVERSE$	Current date written according to the locale format
	DAY	Current day of a month written numerically (without leading zero)
	DAY_02	Current day of a month formatted as "dd"
	$DAYNAME$	Current day of the week written in the abbreviated forms (eg. Sun, Mon)
	$DAYLONGNAME$	Current day of the week written in the locale format (eg. Sunday, Monday)
	$MONTH$	Current month written numerically (without leading zero)
	$MONTH_02$	Current month formatted as "MM"
	$MONTHNAME$	Current month written in the abbreviated forms according to the locale format (eg. Jan, Feb)
	$MONTHLONGNAME$	Current month written according to the locale format (eg.

Reserved string		Description
		January, February)
	$YEAR$	Current year written numerically
	$YEAR_02$	Current year formatted as "yy"
Time	$HOUR$	Current hour written numerically (without leading zero)
	$HOUR_02$	Current hour formatted as "HH"
	$MINUTE$	Current minute formatted as "mm"
	$SECOND$	Current second formatted as "ss"

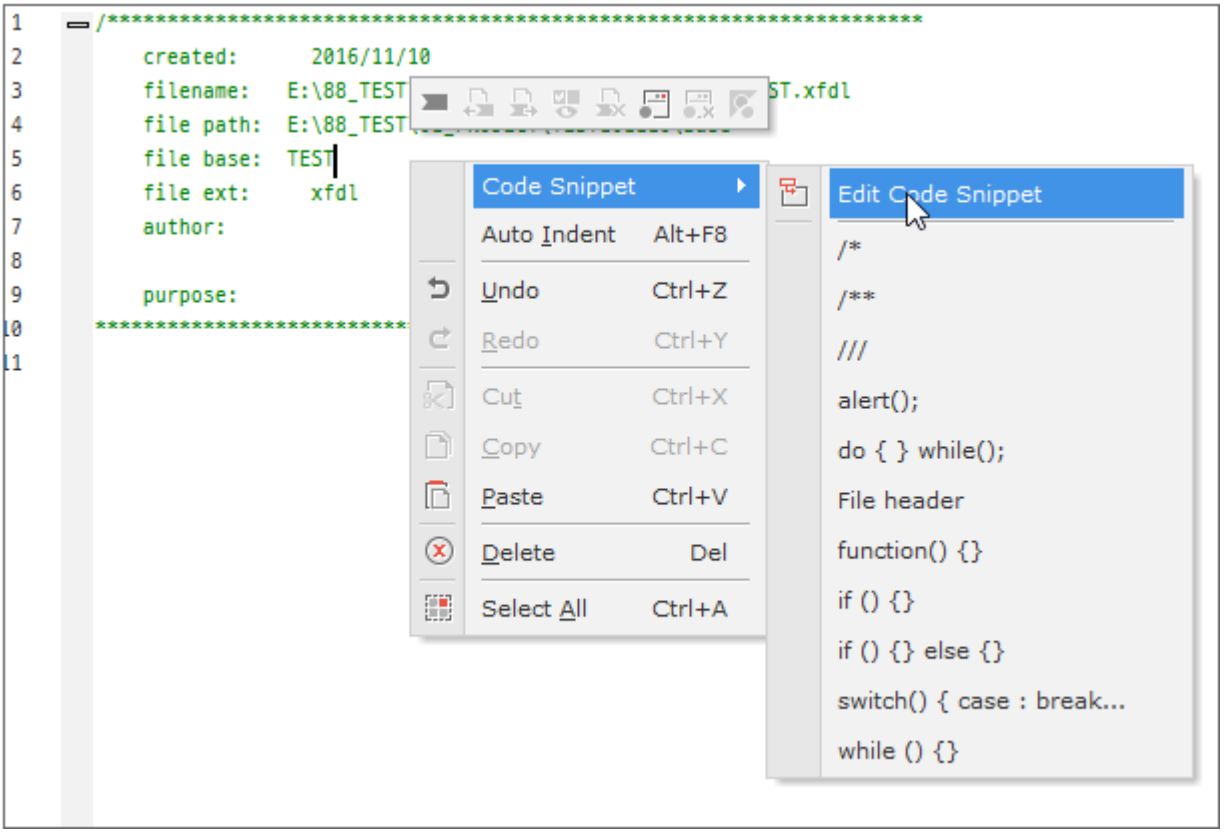
You can use the reserved words by accessing [**Insert reserved string**] in the context menu opened form the code snippet editor pane. Otherwise, type the **$** symbol and then select a reserved word from the drop-down list that appears automatically.



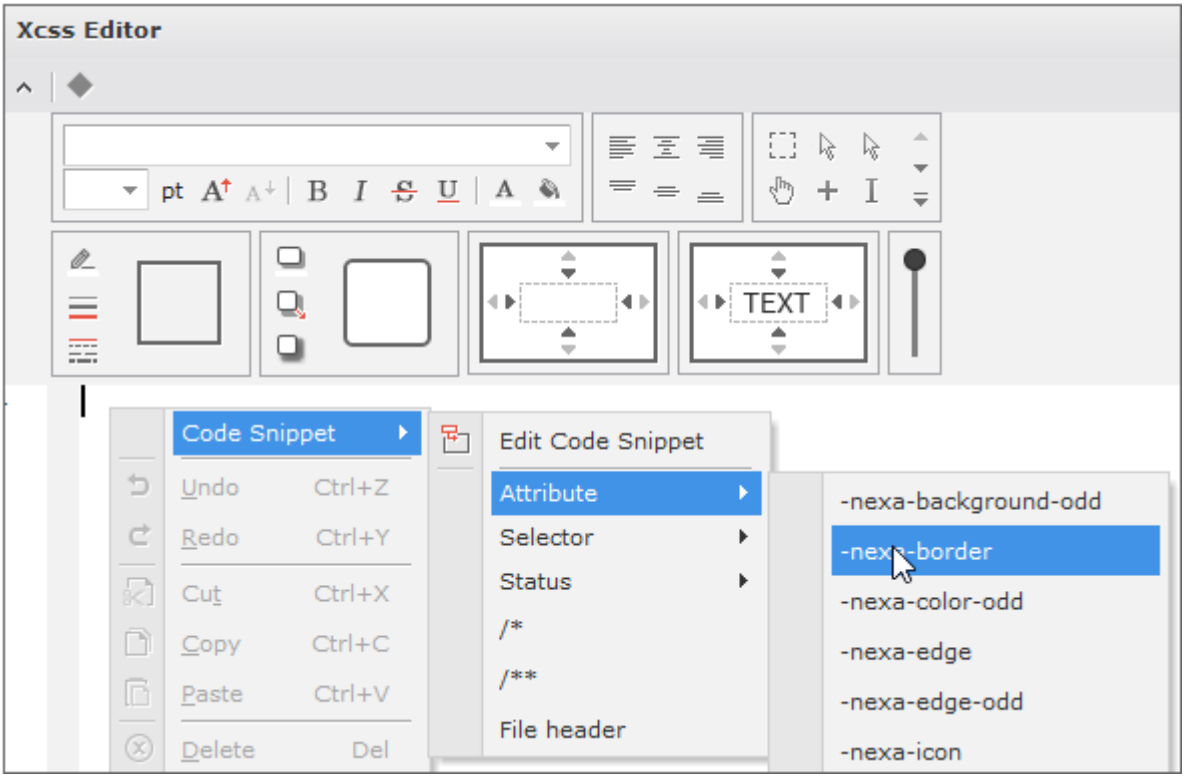
Using Code Snippets

You can call the list of code snippets by accessing [Code Snippet] in the context menu opened from the script editor or XCSS editor. Select a code snippet from the list, and then the relevant code fraction will be inserted to the location of the cursor.

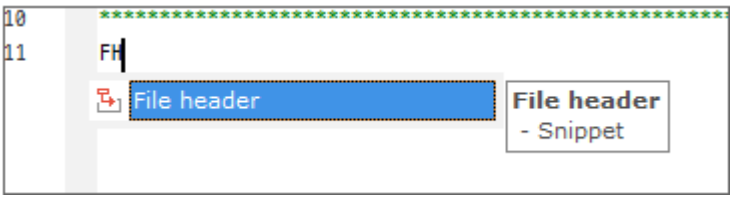
- Code snippet list opened from the script editor



- Code snippet list opened from the XCSS editor



If you enter a shortcut that you have defined in the Code Snippet editor, the relevant code snippet will be suggested. If you select the suggested code snippet, it will be inserted.



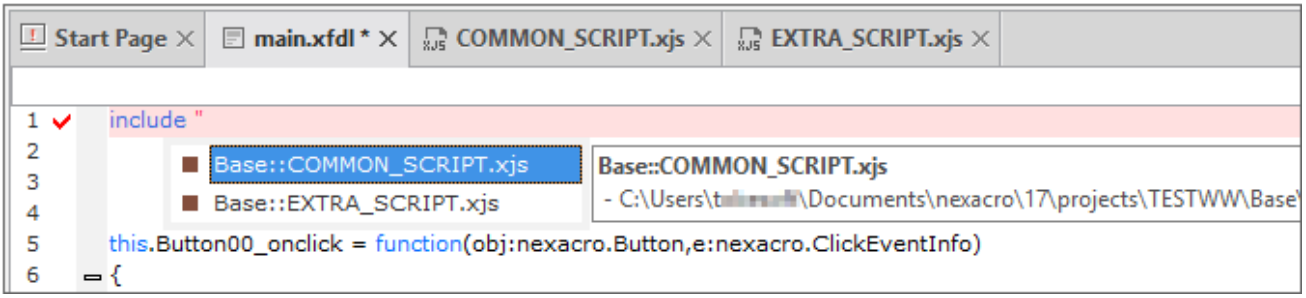
A code fraction is inserted to the current location. For example, if you move the cursor by pressing the tab key, a code fraction will also be inserted to the same position.

19.2 Intellisense

Intellisense is an assistant to help users to edit a file conveniently and quickly. It provides a list of related codes (members, functions, parameters or code snippets) while users type strings in the script or XCSS editor. If you want to open Intellisense without typing a string, enter the `Ctrl`+`Space` key.

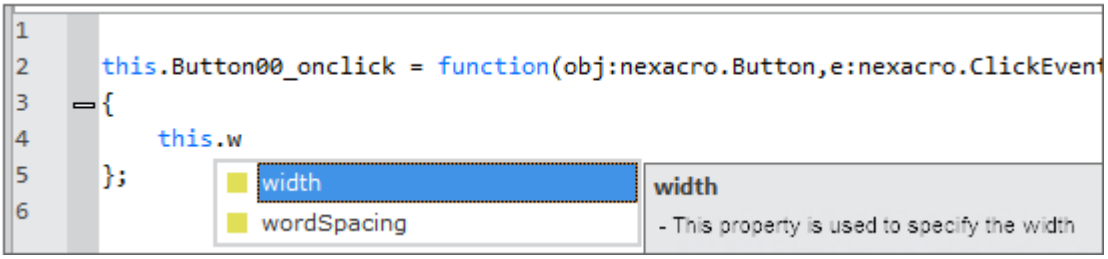
19.2.1 include

When you edit a script, Intellisense can present you with a list of *.xjs* files that can be included in the script. When you write the directive *include* and then enter a quotation mark, Intellisense will automatically appear showing you a file list.



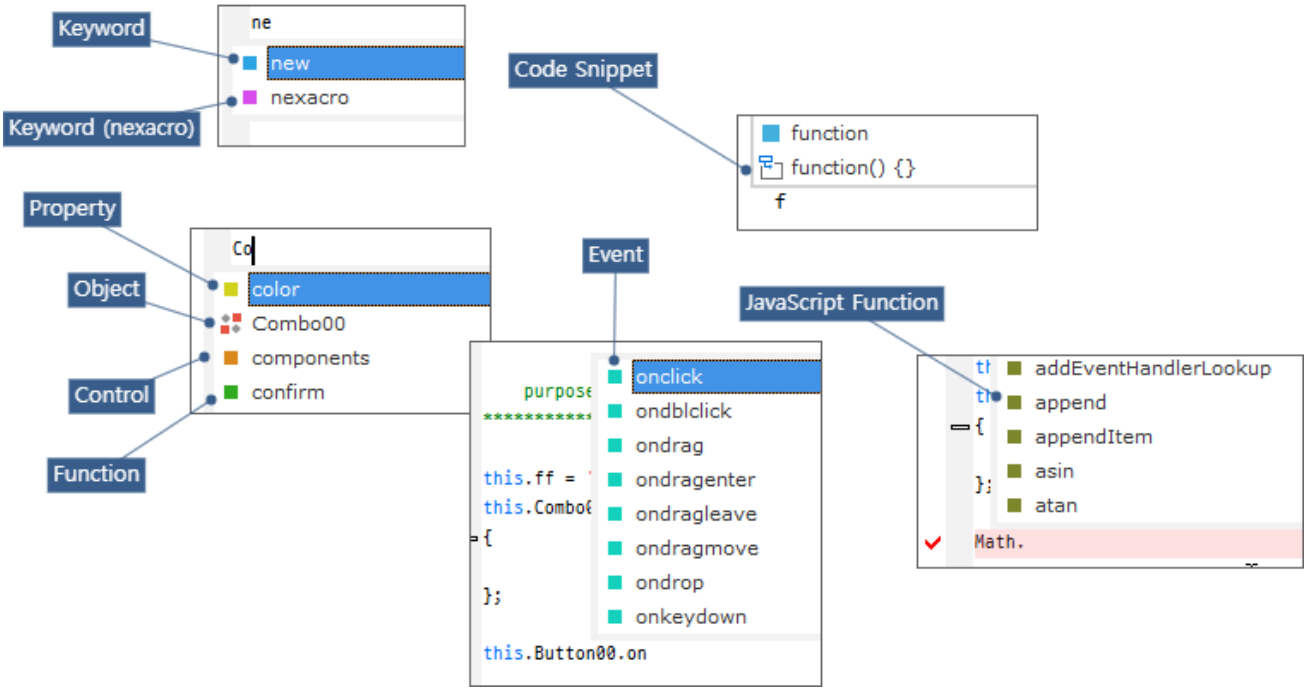
19.2.2 Script

By considering the current editing location in the script editor, Intellisense will display all the members, functions, parameters and code snippets available within the current scope.



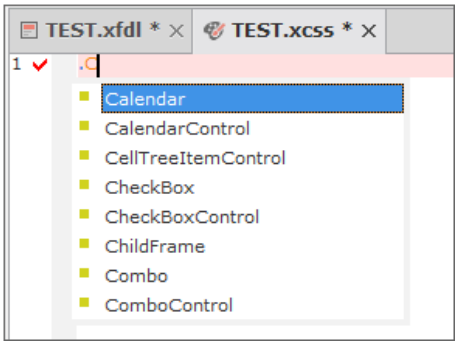
A list of properties and methods starting with the characters entered will be displayed, along with a basic description of each property and method.

The icons in front of each item indicate code types.

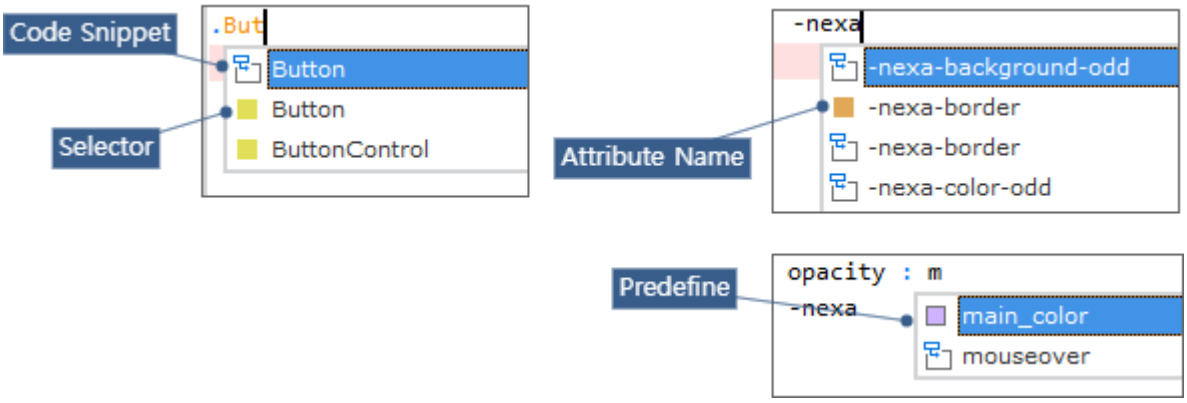


19.2.3 XCSS

In the XCSS editor, Intellisense will display all the available selectors, attributes and Code snippets, varying according to editing locations.

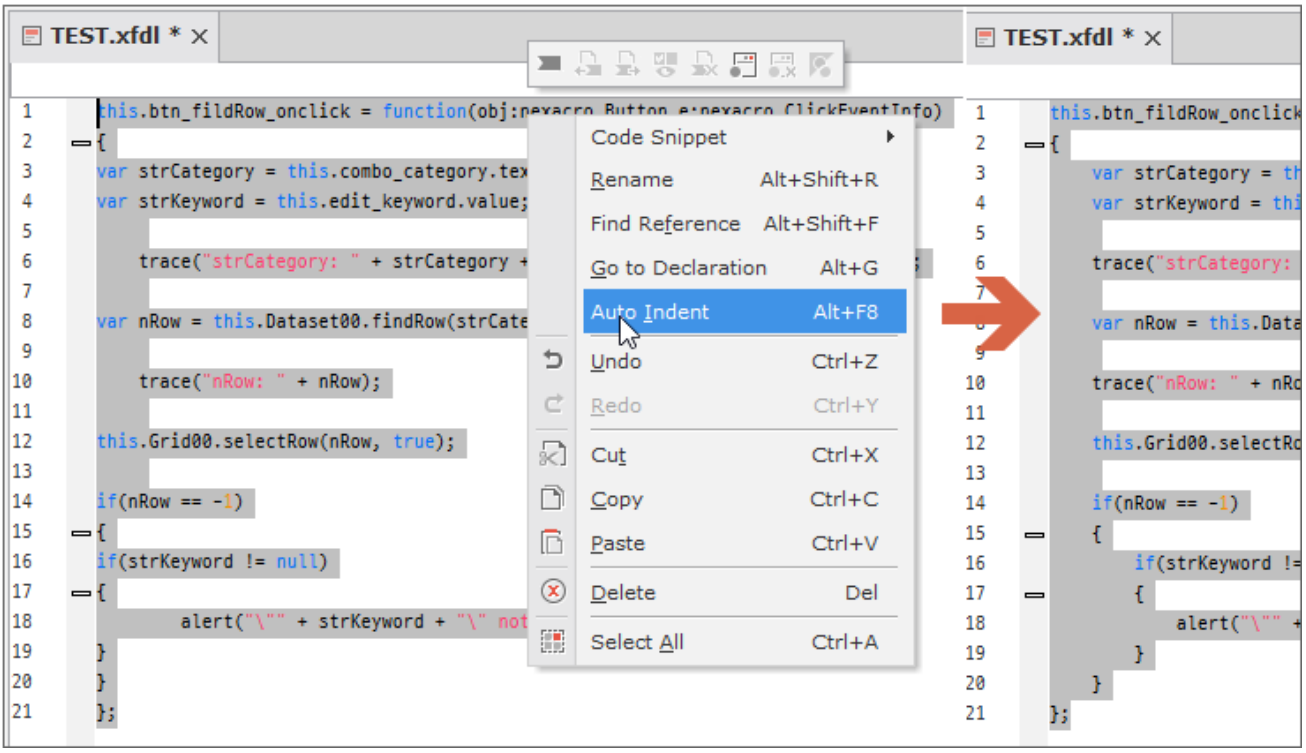


The icons in front of each item indicate code types.



19.3 Auto Indent

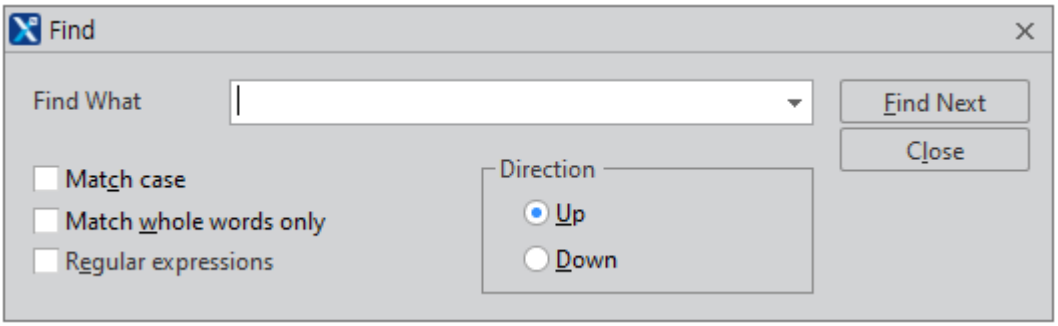
This function indents a selected block automatically in the script editor.



19.4 find and replace strings

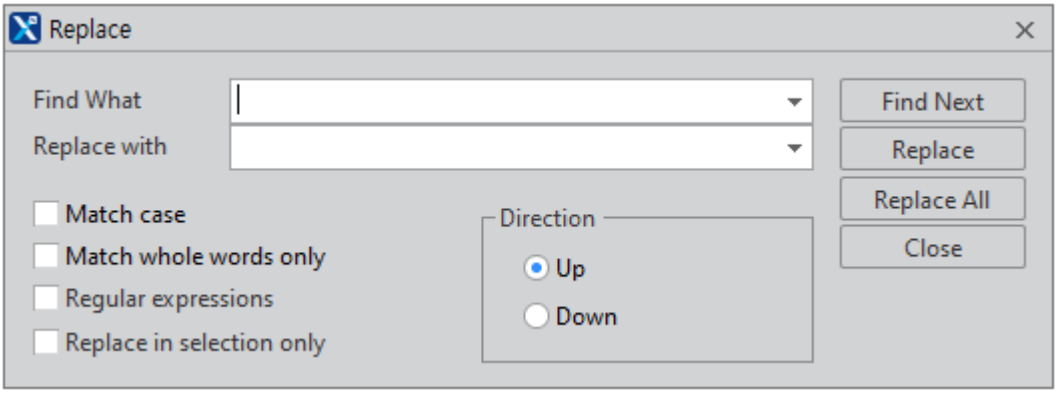
19.4.1 Find

Enter a search string and sequentially find strings that match the criteria.



19.4.2 Replace

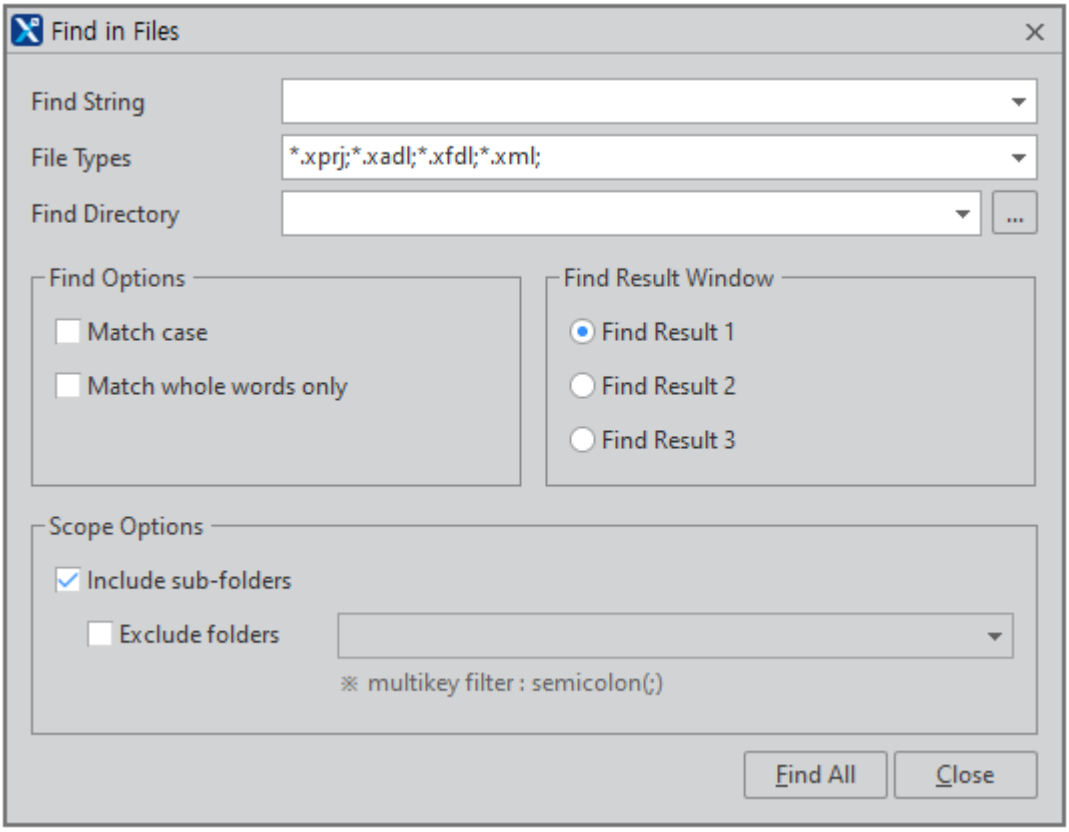
After entering a search string and a string to replace, sequentially find and replace strings that match the criteria, or find and replace multiple strings at the same time.



Options	Description
Replace in selection only	Only the area selected as a block is replaced with the target. You can use only the function to find and replace multiple strings in a block at the same time (Replace All)

19.4.3 Find in File

Set the files in the set directory, find the files containing the string that meets the value entered in the search string and the condition, and display them in the form of a list in the Find Results window.



Options	Description
Include sub-folders	Select whether or not to include subdirectories of the directory to be searched in the search target.
Exclude folders	Set the subdirectory name to be excluded from the search target. You can set more than one name using a semicolon (;).

19.4.4 common options

Options	Description
Direction	Sets the string search direction.
Match Case	Matches the case of the search string.
Match whole words only	Verifies that the search string matches text separated by independent words. If the search string is "Button", <Button> is treated as a matching result, but Button00 is treated as a non-matching result, as it is recognized as a word

Options	Description
	associated with a number.
Regular expressions	Choose whether to use regular expressions in search strings or replacement strings.

Regular expressions

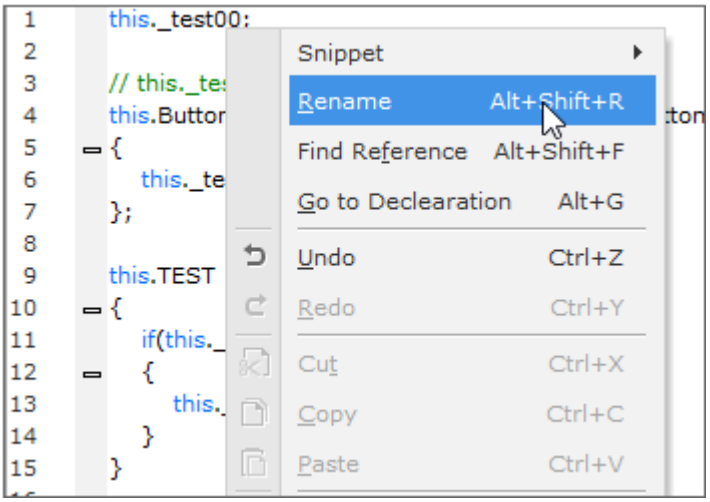


It may not work when characters other than alphabets or numbers are set as a search string.

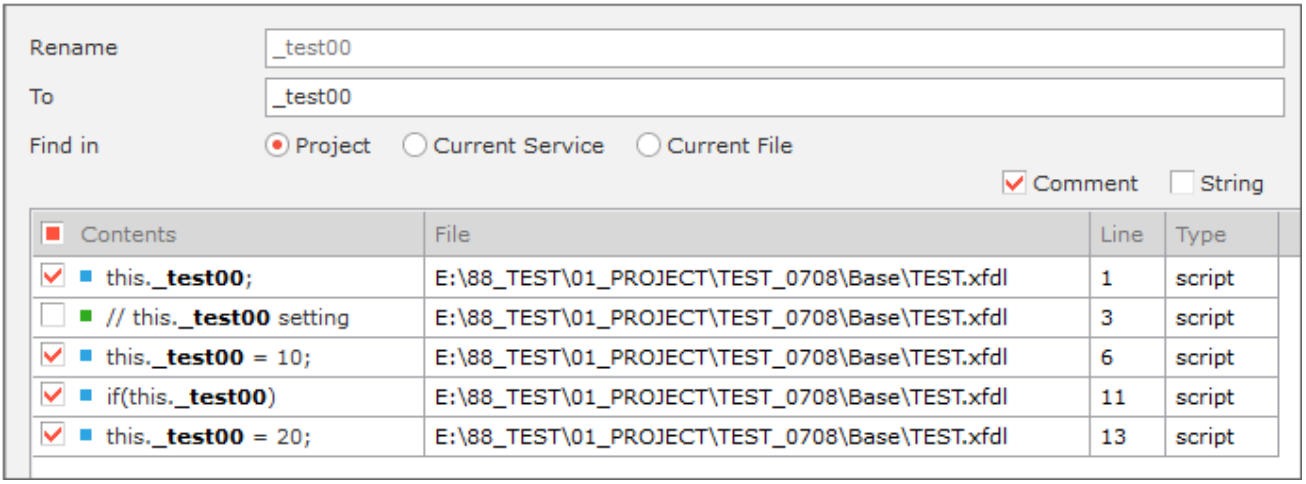
expression	Description
.	Matches any single character, including space and tab characters. If the search string is ".T", "AT", " T", and "1T" are all matched.
\(...\)	Group subexpressions. Can be used to specify the nth group when using \n expressions.
\n	Group numbers from 1 to 9 can be set during swap operation. If the search string is "Button\([0-1]\)\([0-9]\)_TEST", setting the string to be replaced with "TEST\1\2_Button" will change the string "Button00_TEST" to "TEST00_Button". can.
\<	Matches the search string to the starting character of text separated by independent words. Setting the search string to "\<T" will only find the starting characters in "TEST".
\>	Matches the search string as the last character of text separated by independent words. Setting the search string to "T\>" will only find the last character in "TEST".
\x	Special characters used in regular expressions are treated like regular characters. Setting the search string to "\[" will find the regular character "[". However, if the search string is set to "[", it is recognized as [...], which means a character set, and does not operate normally.
[...]	character set. For example, [abc] means any of the letters a, b, or c. [a-z] can also indicate a range.
[^...]	It means the rest of the characters except for the character set. For example, [^A-Za-z] means any character except alphabetic characters.
^	The character set after is the first character of the line.
$	Matches the previous set character to the last character of the line. Setting the search string to ")$" will find the trailing ")" in "function ( )".
*	Matches the previously set character zero or more times. If you set the search string to "Button0*1", you can find "Button1", "Button01", "Button001".
+	Matches the previously set character one or more times. If you set the search string to "Button0*1", you can find "Button01", "Button001", "Button0001".

19.5 Rename

This function is used to replace a certain character string with another within the same project. You can use This function by right-clicking a certain word in the script editor and opening the context menu.

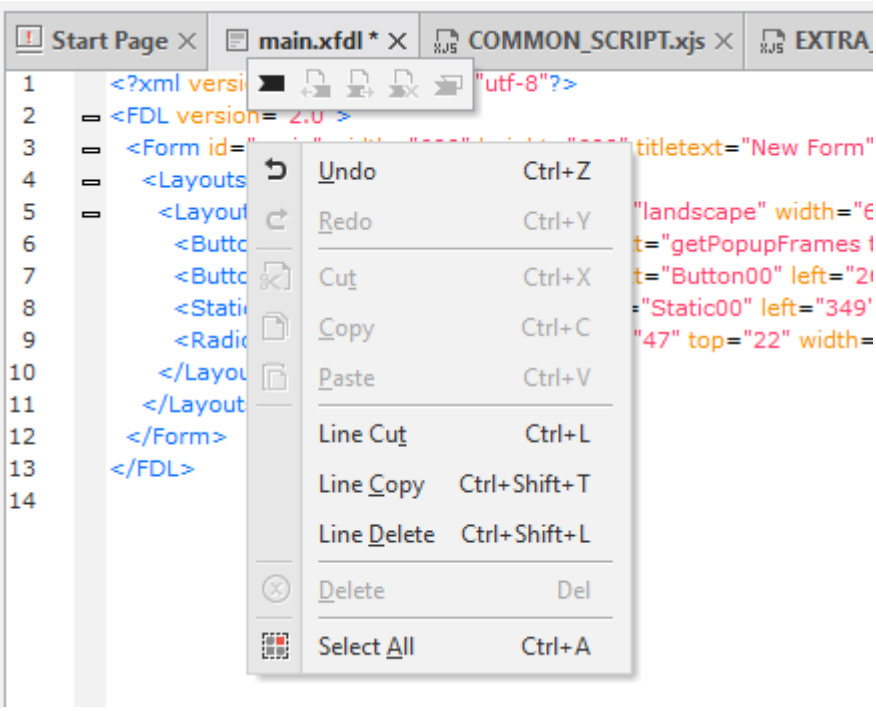


The range for finding replacements can cover a project, current service or current file according to your choice. If you want to search the character strings in comments, check "Comments." Likewise, check "Strings" if you want to search general character strings.



19.6 Code Line Editing

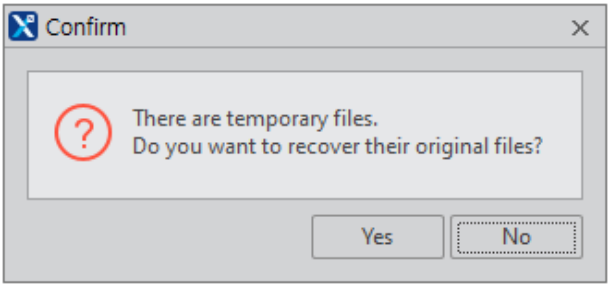
When you edit a script or other types of source codes, you can cut, copy or delete a code line where a caret is placed. You can use the foregoing functions by right-clicking a line from the code editor to open the context menu.



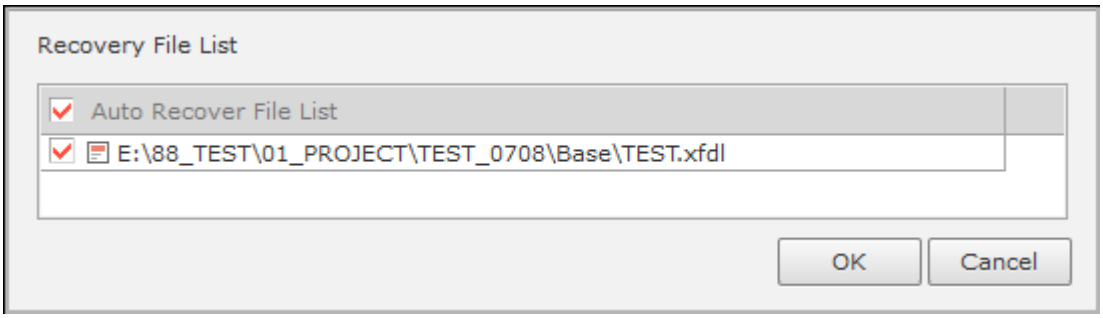
19.7 Auto Recover

When you set up the Auto Recover function from the **Tools > Options** menu, Nexacro Studio will periodically save backups of files being edited.

With Auto Recover configured, when you open a project that was closed abnormally, Nexacro Studio will check the temporary file and prompt you to recover the file from the most recent backup.



Auto Recover displays the target recovery list and recovers the work in process using the saved backup file.



19.8 metadata

The function of applying the data received through the metadata service as the component, object id, and text property value is supported.

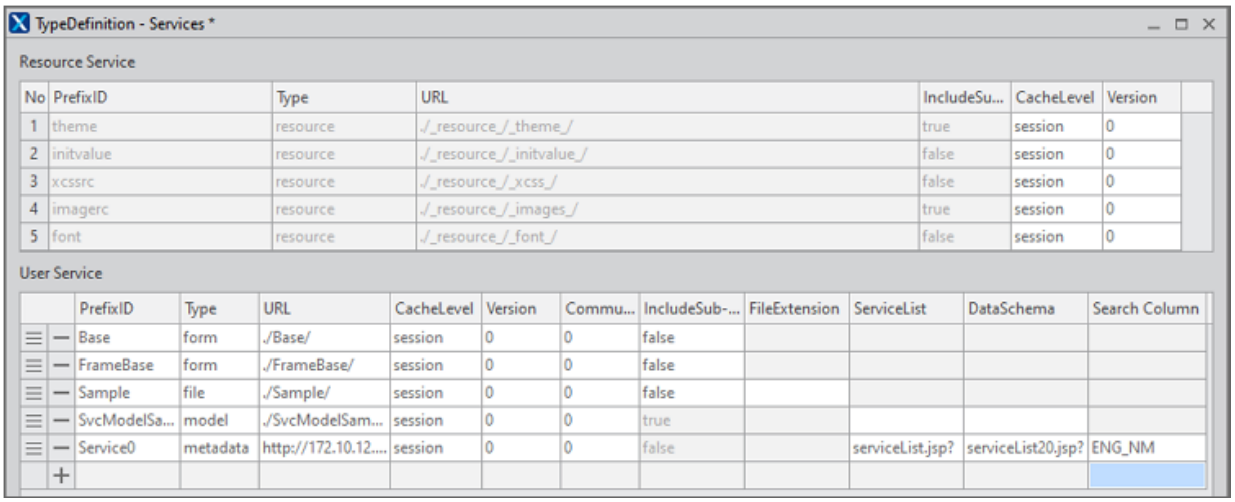
> The metadata service is a function that provides data that can be referenced when setting the id, text property values. It does not automatically reflect changes in data.
> For example, even if the text property value of the Button component is selected from metadata and set to "Submit" for the metadata value to be changed to "OK", then the text property value of the component does not change automatically.

19.8.1 Registering metadata Service

1. Double-click the [TypeDefinition > Services] menu in Project Explorer or right-click and select the [Edit] item to open the edit window as a pop-up window.

2. Register the metadata service.

   Select "metadata" for the type item and enter the service path, ServiceList, DataSchema, and Search Column items. The Service List item has the return value in XML format in which the domain and the model are defined. The DatasetLayout item has the return value in XML format in which the Dataset is defined.



When specifying the ServiceList, DataSchema items, parameters can be transmitted in the following format.

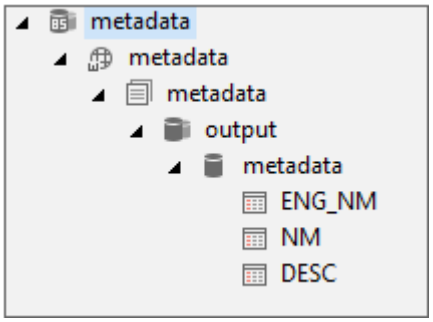
```
http://172.10.12.54:8080/serviceList/
serviceList.jsp
?filter=FILTER value
&domain=DOMAIN value
&model=MODEL value
```



If you do not specify the filter parameter, then there may be restrictions on calling the service.

3. If the service is normally registered, then call the metadata service immediately.

4. When the metadata service is called and data is received, the information is displayed in Project Explorer as shown below.



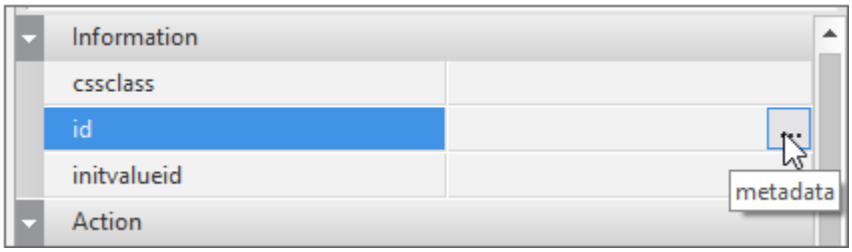
19.8.2 Applying Data Received through metadata Service as id, text Property Values

1. Select the menu [Options] and open the Options window.

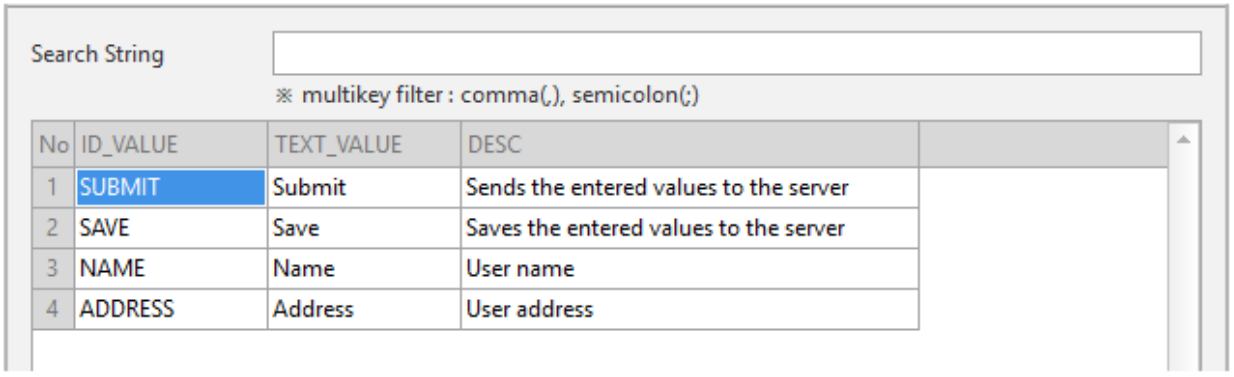
2. In the Options window [Project > Advanced > Metadata] item, check to use "Use user-defined metadata".

> If the "Use metadata with prefixID" item is checked to use, then the id property value is set in the [prefixID+Metadata] format when the id property value is applied.

3. When you select the component or object and select the id or text property item, you can check the expand button. Click the expand button.



4. Check that the window where you can search and select metadata is opened.

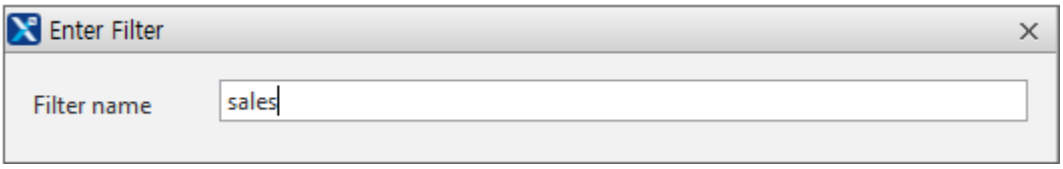


5. Check the item to be reflected as the id or text property value and click the corresponding cell to be reflected as the id or text property value.

19.8.3 Other Functions

Refresh Filtered

You can transmit the filtered data from the service by specifying the filter parameter value. After selecting the service or model in Project Explorer, select the [Refresh Filtered] item from the context menu to display a window where you can enter the text to be filtered.



The entered value is transmitted as the filter parameter value of ServiceList, DatasetLayout.

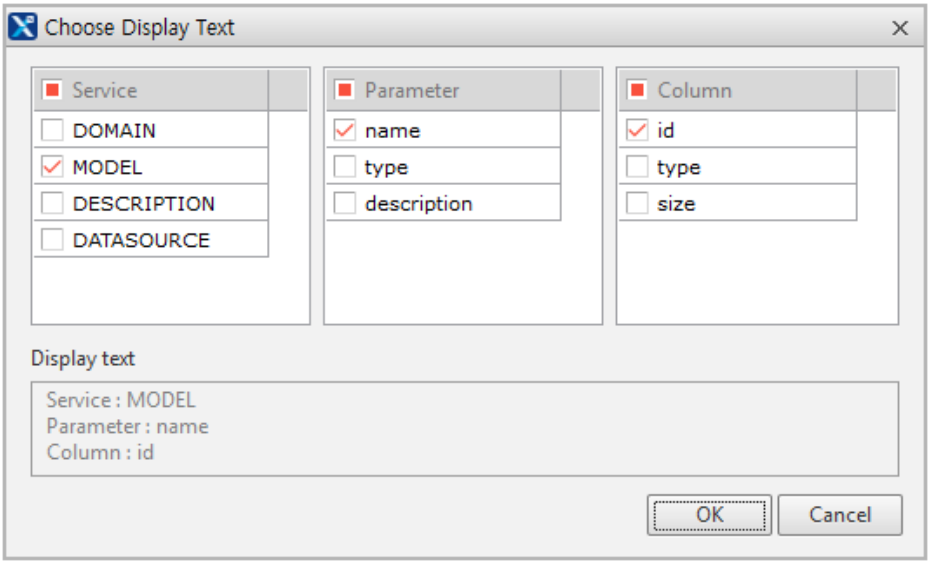
```
[serviceURL + serviceList]?filter=sales
[serviceURL + datasetLayout]?filter=sales
    &domain=[DOMAIN value]&model=[MODEL value]
```

Search Column

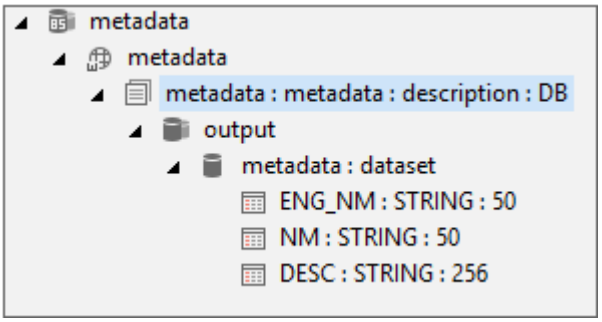
If you specify the Search Column item in TypeDefinition, you can specify the search target column when searching for text in the window where you select metadata. If no value is specified, then the search is processed for all columns.

Display text

You can format the metadata information displayed in Project Explorer. The default appearance is as follows.



Depending on the selected item, you can change the format of the metadata information displayed in Project Explorer.



metadata Loading at Project Start

In the Options window [Project > Startup] item, check to use "Load layout information of a business service/metadata on Startup".

> The metadata loading at the project start uses the Service.xds file created in the project folder. If the data is changed, you must select the [Refresh] or [Refresh Filtered] item from the context menu after selecting the service or model in Project Explorer.

19.8.4 ServiceList, DataSchema Specifications

ServiceList

In ServiceList, the DOMAIN, MODEL items among the ColumnInfo items are required values and the corresponding values should be specified as "metadata".

```
<?xml version="1.0" encoding="utf-8"?>
<Root xmlns="http://www.nexacroplatform.com/platform/dataset" ver="5000">
    <Parameters>
        <Parameter id="ErrorCode" type="int">...</Parameter>
        <Parameter id="ErrorMsg" type="string">...</Parameter>
        <Parameter id="Cache" type="string">...</Parameter>
    </Parameters>
    <Dataset id="output">
        <ColumnInfo>
            <Column id="DOMAIN" type="string" size="255"/>
            <Column id="MODEL" type="string" size="255"/>
            <Column id="DESCRIPTION" type="string" size="255"/>
            <Column id="DATASOURCE" type="string" size="255"/>
        </ColumnInfo>
        <Rows>
            <Row>
                <Col id="DOMAIN">metadata</Col>
                <Col id="MODEL">metadata</Col>
                <Col id="DESCRIPTION">description</Col>
                <Col id="DATASOURCE">DB</Col>
            </Row>
        </Rows>
    </Dataset>
</Root>
```

DataSchema

In DataSchema, the name, type items among the ColumnInfo items are required values, and the corresponding values should be specified as "metadata", "dataset". Also, define Dataset with "metadata" as id and process the data to be actually used.

```
<?xml version="1.0" encoding="utf-8"?>
<Root xmlns="http://www.nexacroplatform.com/platform/dataset" ver="5000">
    <Parameters>
        <Parameter id="ErrorCode" type="int">...</Parameter>
        <Parameter id="ErrorMsg" type="string">...</Parameter>
        <Parameter id="Cache" type="string">...</Parameter>
    </Parameters>
    <Dataset id="output">
        <ColumnInfo>
            <Column id="name" type="STRING" size="50"/>
            <Column id='type' type='STRING' size='50'/>
            <Column id='description' type='STRING' size='256'/>
        </ColumnInfo>
        <Rows>
            <Row>
                <Col id='name'>metadata</Col>
                <Col id='type'>dataset</Col>
                <Col id='description'></Col>
            </Row>
        </Rows>
    </Dataset>

    <Dataset id='metadata'>
        <ColumnInfo>
            <Column id='ID_VALUE' type='STRING' size='50'/>
            <Column id='TEXT_VALUE' type='STRING' size='50'/>
            <Column id='DESC' type='STRING' size='256'/>
        </ColumnInfo>
        <Rows>
            <Row>
                <Col id='ID_VALUE'>SUBMIT</Col>
                <Col id='TEXT_VALUE'>Submit</Col>
                <Col id='DESC'>Sends the entered values to the server</Col>
            </Row>

            <Row>
                <Col id='ID_VALUE'>SAVE</Col>
                <Col id='TEXT_VALUE'>Save</Col>
                <Col id='DESC'>Saves the entered values to the server</Col>
            </Row>
```

19.9 Developing Add on

You can create and use Add on to process, check, or output open project information in Nexacro Studio.

You can check or output basic information using Add on Template. You can also modify the template project to create Add on in the desired format.

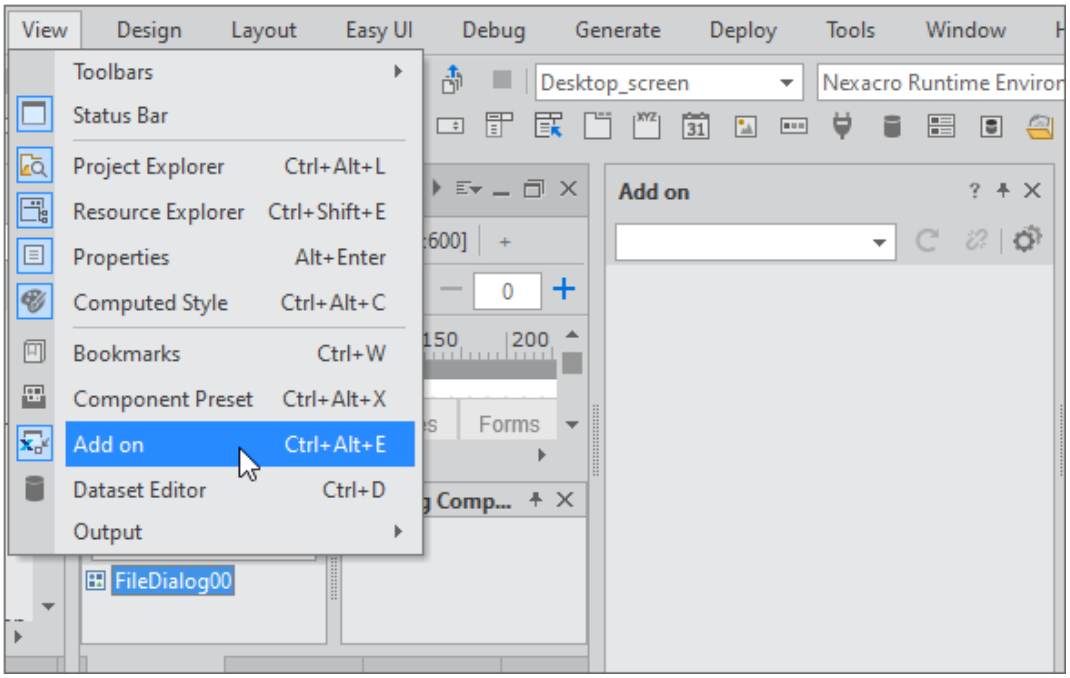
If Add on (including the nexacrostudio method) is run in NRE, WRE other than the Add on window, then a script error occurs. Add on is designed to operate only in the Add on window and the corresponding script error is normal.
`ReferenceError: nexacrostudio is not defined`

Below, we will look at how to create and connect a simple Add on.

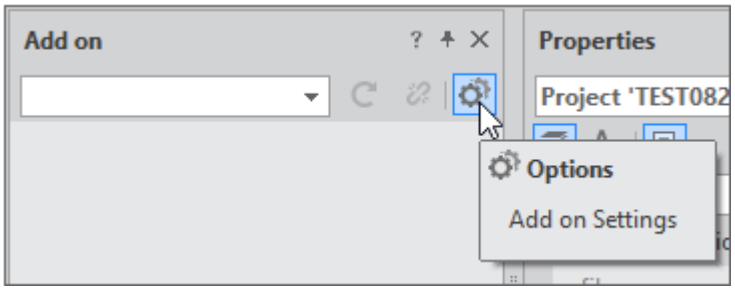
1. Create a new project.
2. Place the Button, TextArea components on the screen.
3. Write the onclick event handler function of the Button component as follows.

```
this.Button00_onclick = function
(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.TextArea00.value = nexacrostudio.request(requesttype.all);
};
```

4. Select the menu [View > Add on] to display the Add on window.

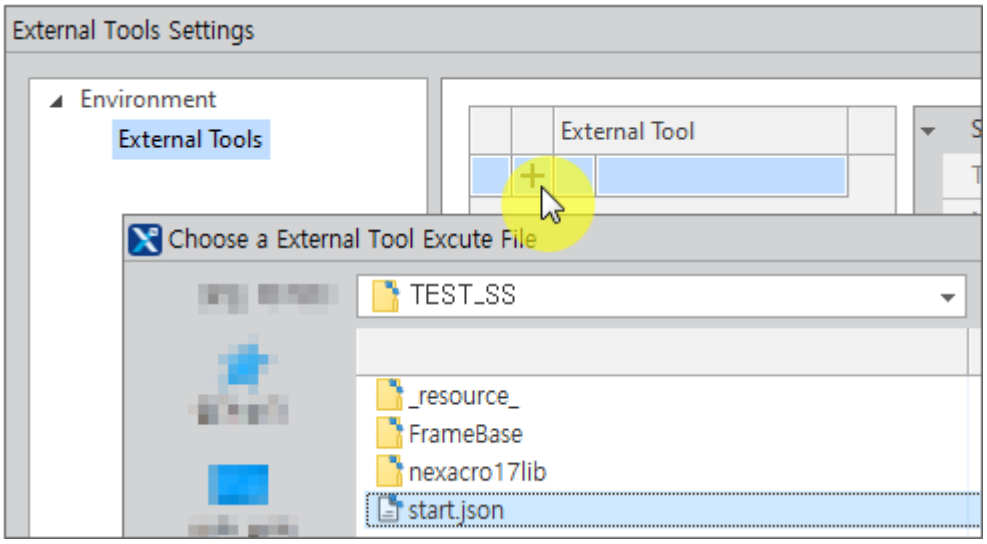


5 Click the Options button in the Add on window to display the option setting window.

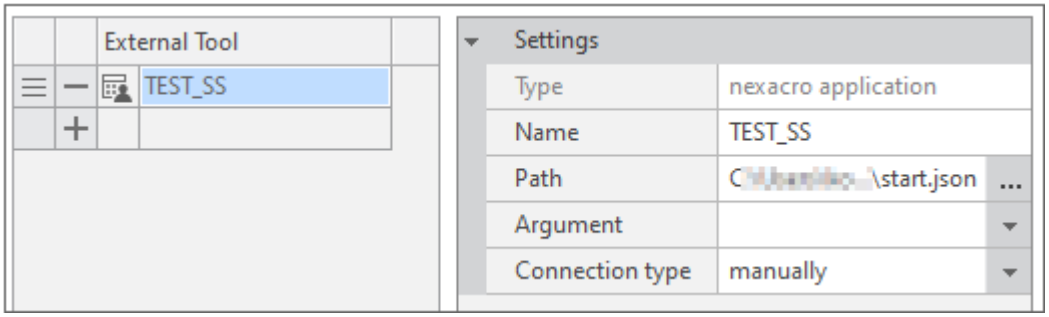


6 Click the [+] button and select the start.json file.

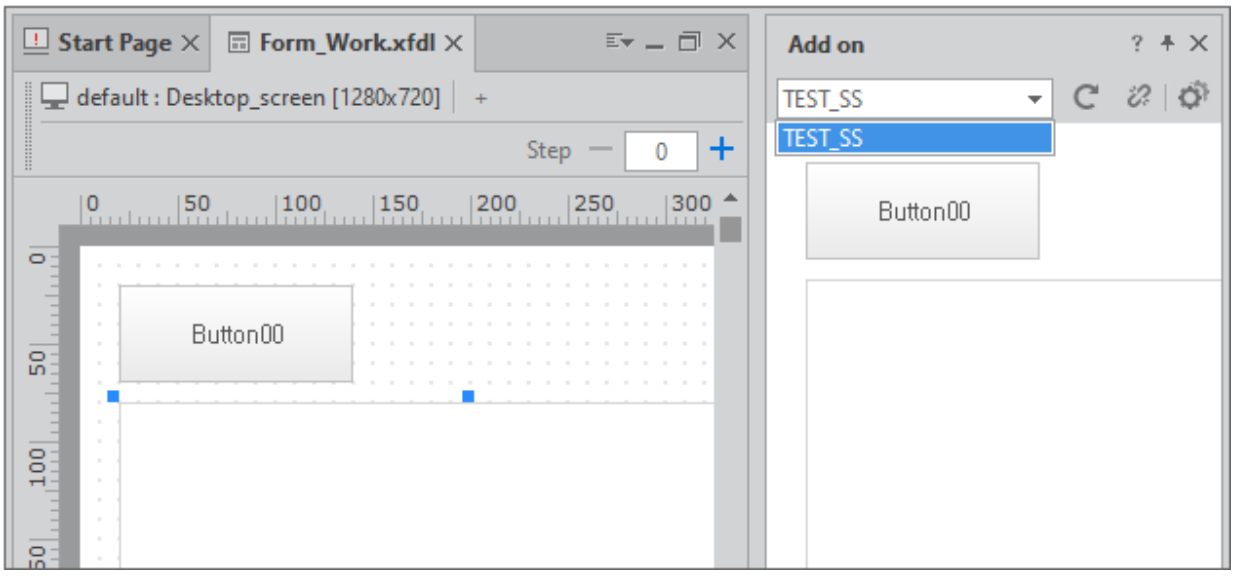
In the file explorer, the output folder location of the currently open project is opened as the default location.



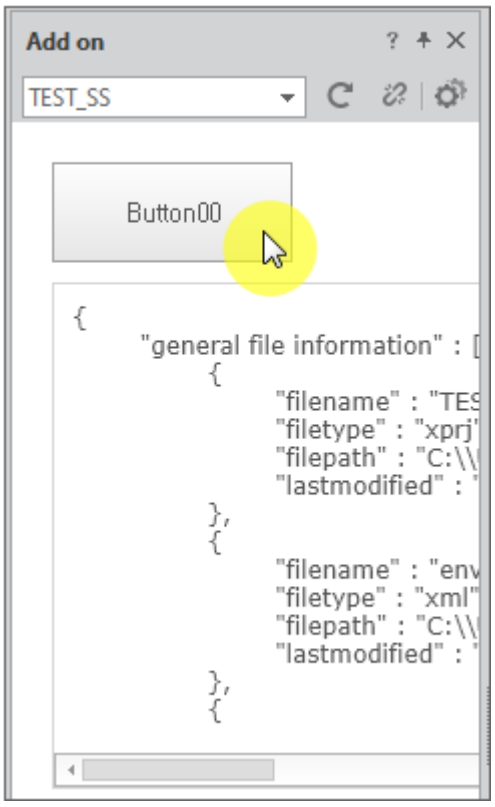
7 Check the content and click the [OK] button to register Add on.



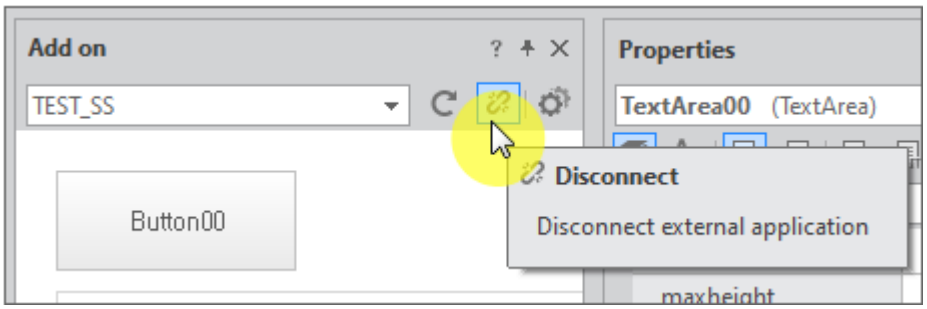
8 Select the application registered in the Add on window from the drop-down list, the screen is then executed and displayed.



9 Click the button in the Add on window to see the information displayed.



10 If you click the Disconnect button, then the connection is terminated and the screen will not be displayed.



When the [?] button is clicked, help related to the nexacrostudio method is displayed.



The corresponding help item may not open normally depending on the execution environment.
Please refer to the Appendix > nexacrostudio Interface > Method item.

19.10 HTTP authentication

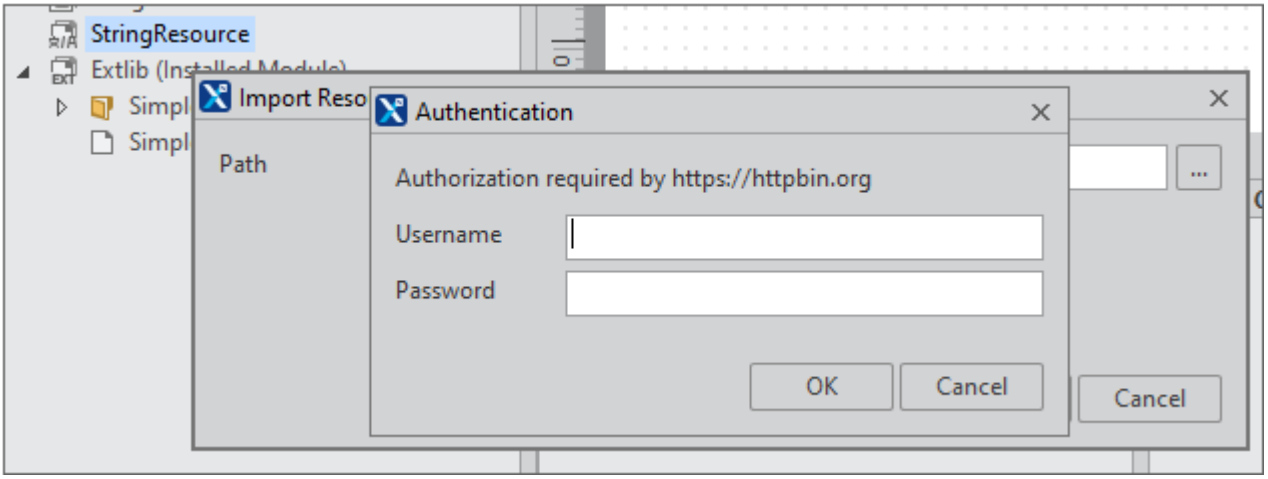
“HTTP authentication” can be used when accessing a specific URL and retrieving data within Nexacro Studio. For example, it can be used in the following functions.

- File > New > StringResource
- Resource Explorer > StringResource > Import
- DataObject > Json Contents Editor
- TypeDefinition > User Service > metadata
- Other URL-related functions

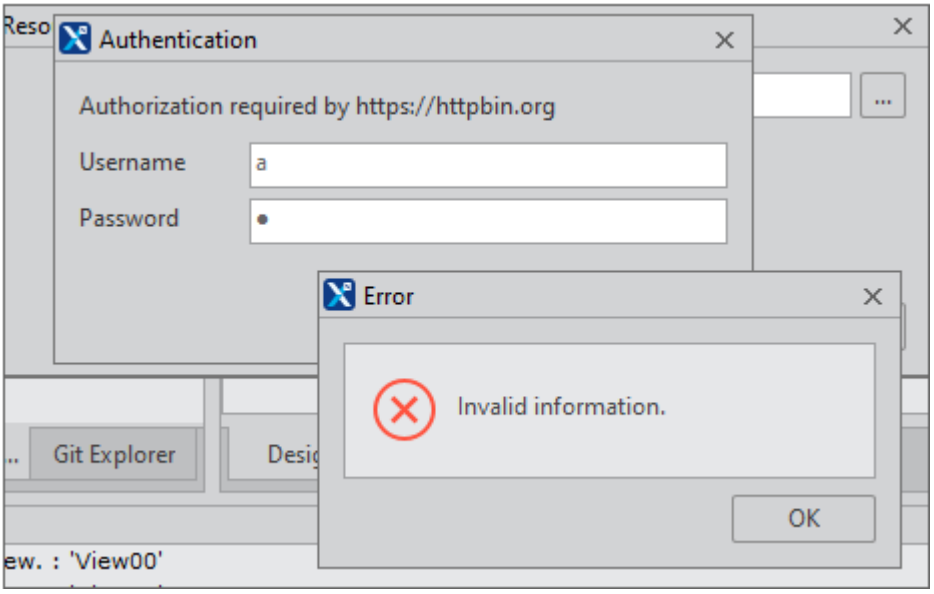
This is not applicable when entering a URL in the property or method of a component or object.
It is applicable only when operating for the purpose of checking data within Nexacro Studio.

19.10.1 Operating method

For example, when using the StringResource > Import function in Resource Explorer, a window will be displayed requesting authentication when you enter a URL that requires authentication and attempt to retrieve data.



Once the authentication is successful, you can proceed to the next step. If the authentication fails, an authentication failure message is displayed and authentication is requested again.



20.

LiteDB: Looking Up Database and Writing Queries

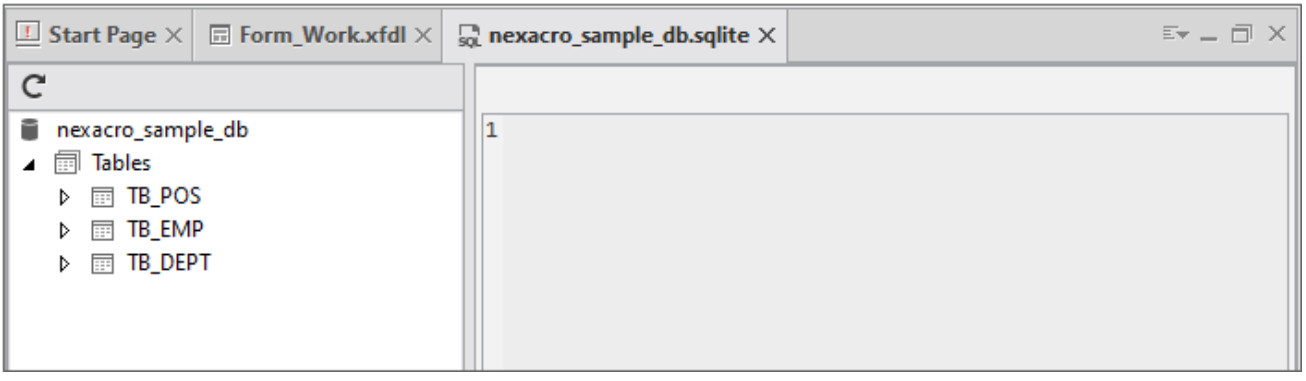
By using a LiteDBConnection and LiteDBStatement objects, you can access and manage a database file stored in a device that operates an app. The foregoing two objects allow you to look up data and write queries when a certain database object is selected.

20.1 Look up database

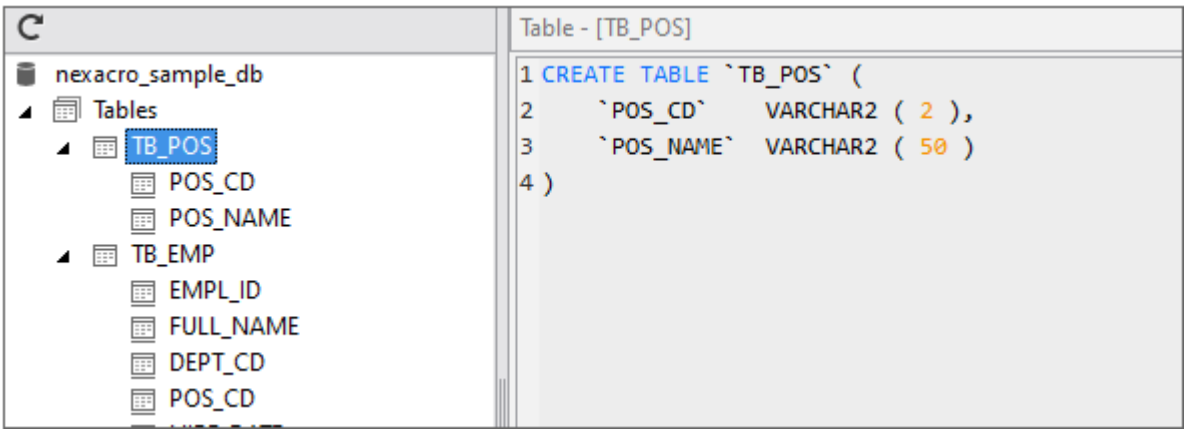
After setting a database file as a value of the *datasource* property of a LiteDBConnection object, double-click the object icon of the database. Then, a new tab page will open to provide you with the information on the database. You can do the same job by opening the context menu from the icon and selecting Edit.



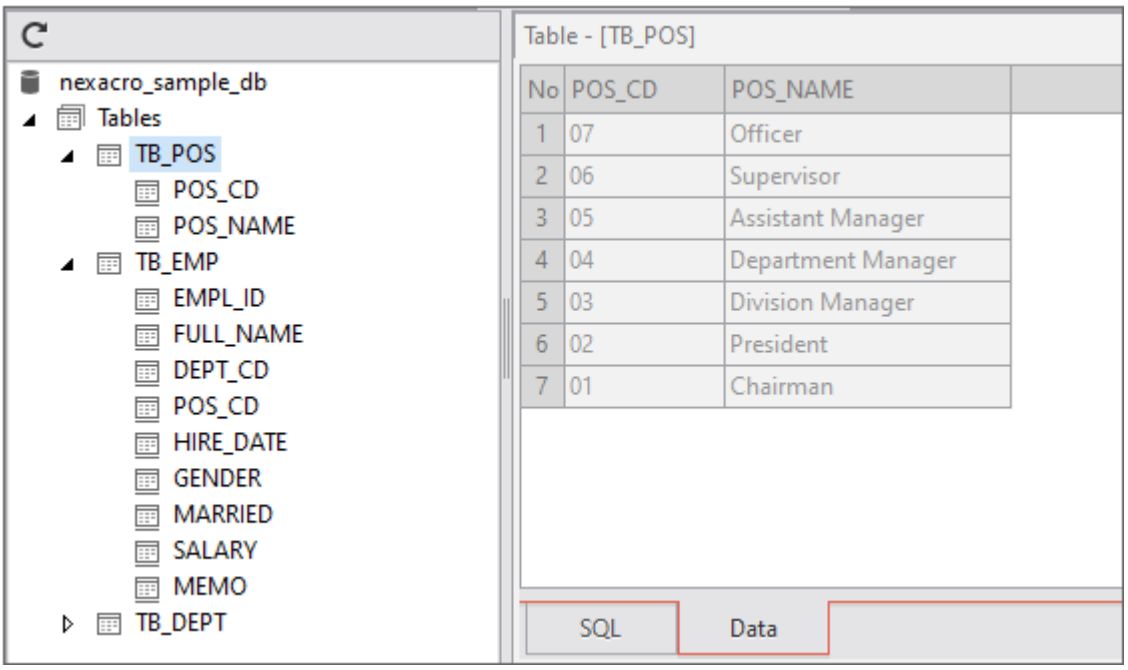
Follow the below link to download the database file used as a sample in this chapter. After saving the file to a project folder or another appropriate directory, set the file as a value of the *datasource* property.
nexacro_sample_db.sqlite



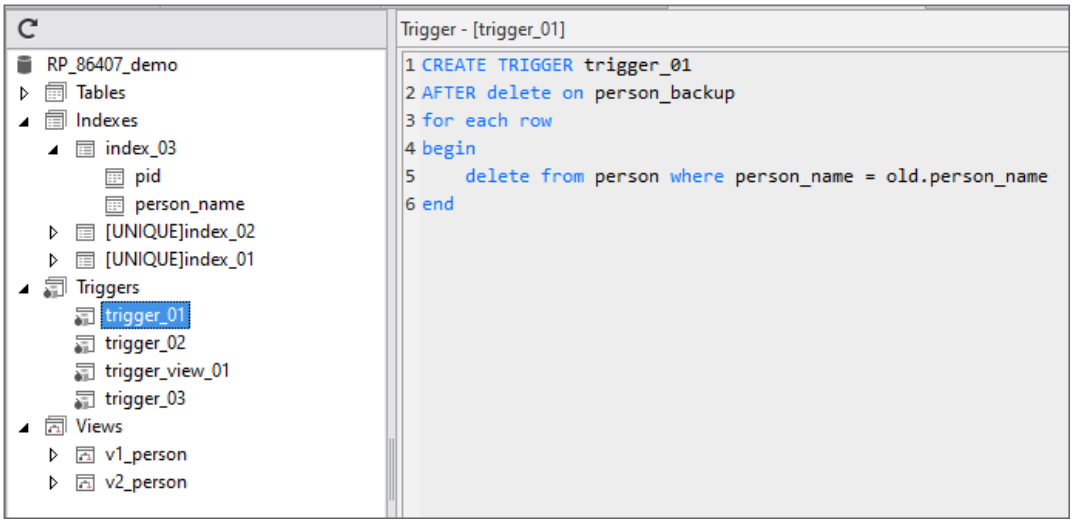
When you select a table, you can see the CREATE TABLE query on the right panel. If you expand a table in the tree view, you can see the columns included by the table. You can select and copy a query but cannot edit it.



If you select the Data tab at the bottom of the right pane, you can see the data of the selected table.



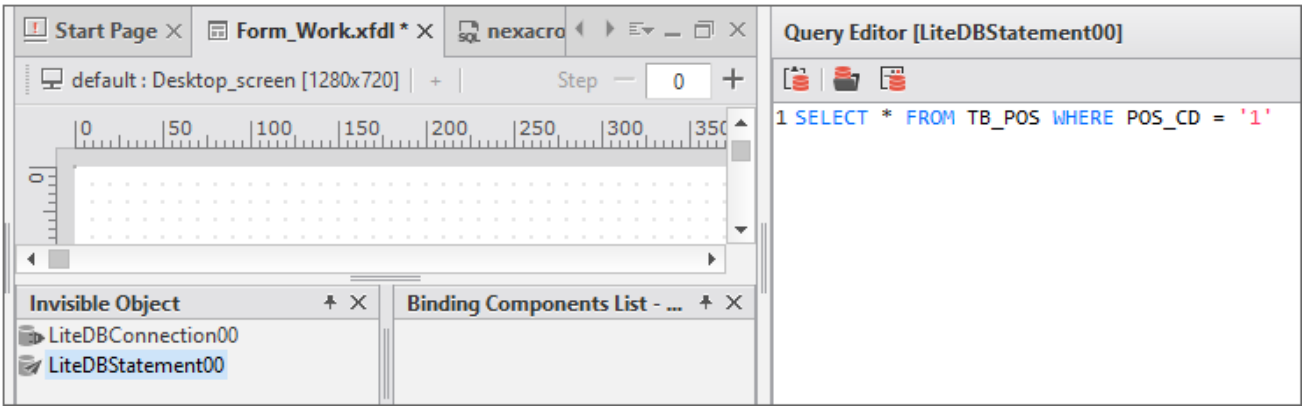
If the database includes indexes, triggers and views, you can also see the relevant information.



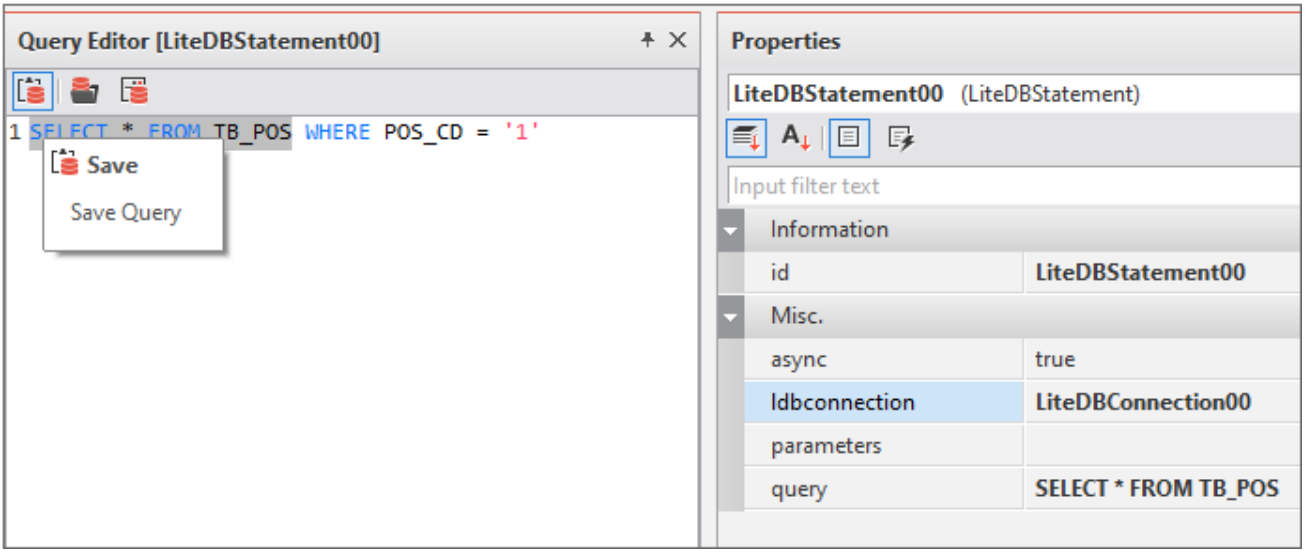
20.2 Write queries

After setting a LiteDBConnection object as a value of the *ldbconnection* property of a LiteDBStatement object, double-click the icon of the LiteDBStatement to open the Query Editor. You can do the same job by opening the context menu from the icon and selecting Edit.

You can write queries on the editor or load a query file that contains existing queries.

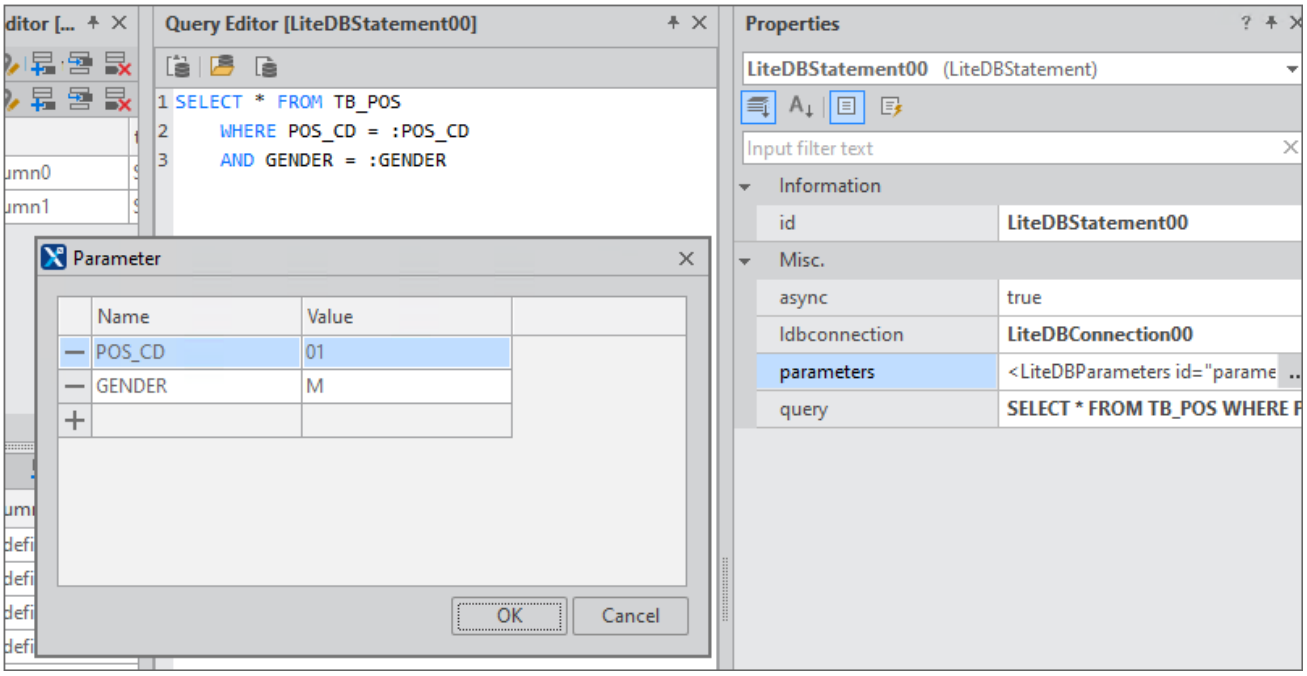


Written queries can be applied as a value of the *query* property of the LiteDBStatement. If you block-select a certain portion, only the block will be a value of *query*.



Menu item		Description
	Save Query	Applies written queries as a value of the *query* property. (If you block-select a certain portion, only the block will be applied.)
	Load Query File	Loads a query file that contains existing queries.
	Save Query File	Save written queries as a file.

You can separately define parameters used for writing queries. Set the value of the *parameters* property as presented below. If you add a parameter in the format like `:POS_CD` to a query statement, the parameters defined by the *parameters* property will be used when the query is executed.



21.

Applying Multiple Languages

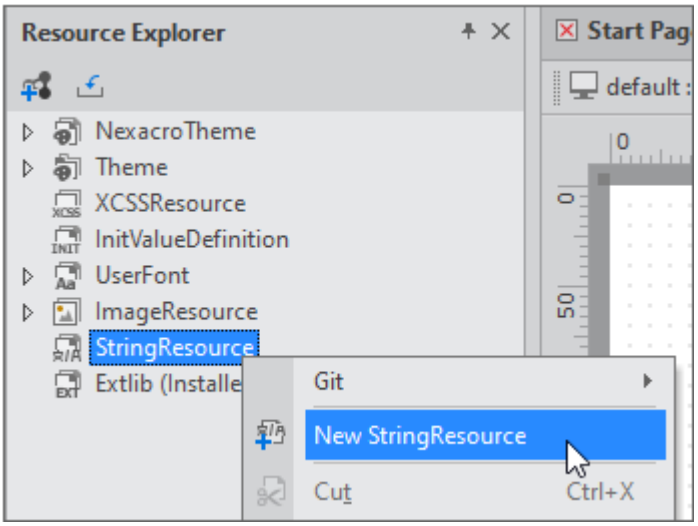
You can load and apply language-specific strings for use within the application by accessing a StringResource you created within your project or data in a remote location.

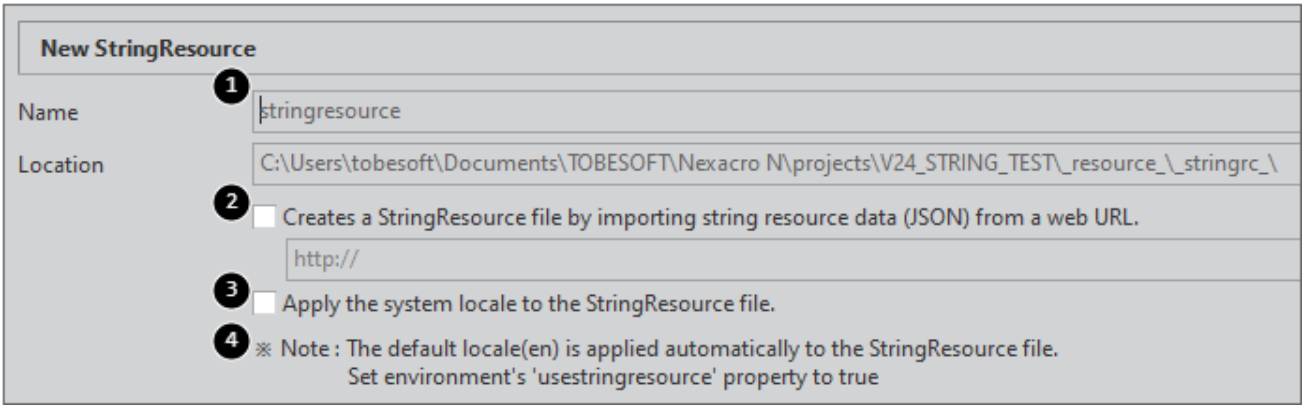
21.1 StringResource

StringResource is data consisting of keys and a language column. When you specify a key within the page, the string corresponding to the key and language can be loaded and applied.

21.1.1 Create a new StringResource

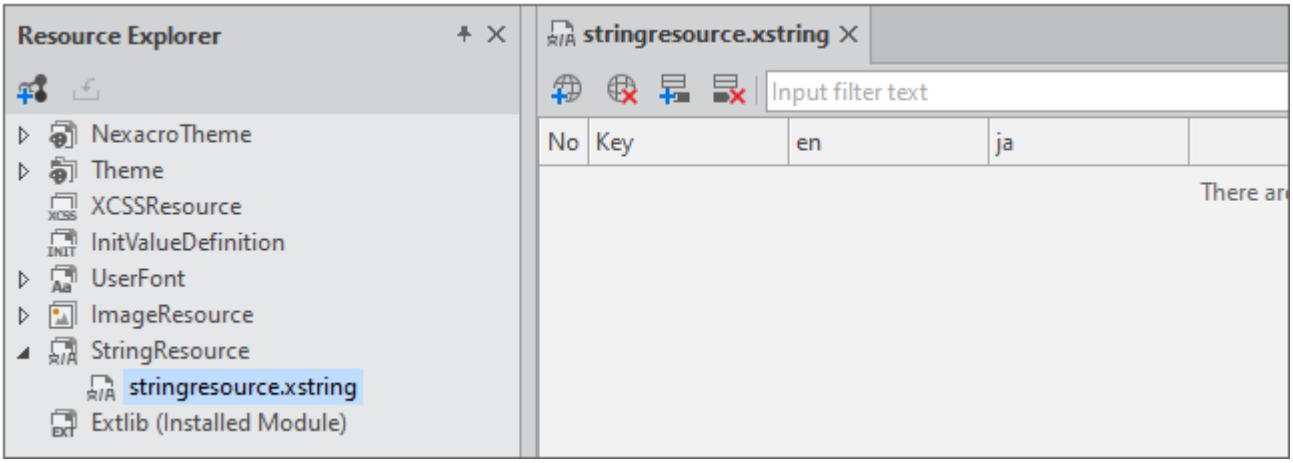
You can select StringResource within the Resource Explorer tab and create a new StringResource item.



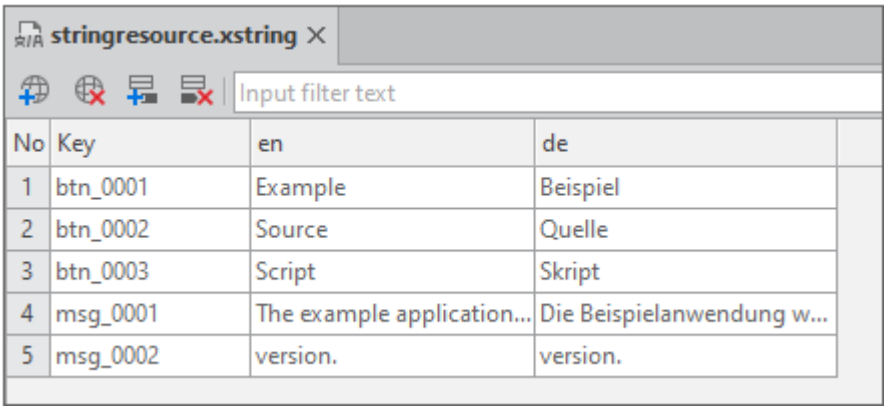


	Item	Description
1	Name, Location	Only one StringResource can be created in a project. The name and path of the file created will be set as default values and cannot be modified.
2	Web URL	Select whether to receive data from a remote resource and create StringResource. When selecting an item, you must set a URL to receive the resource file. For data types, see Remote StringResource Data Types
3	System Locale	The language column set in the system locale is additionally created. For example, if the system locale is set to Japanese, an additional Japanese (ja) language column is created.
4	Note	This is the default setting when StringResource is created. - The English(en) language column is created by default. - Change the value of the environment's useringresource property to true.

The editing window is opened once the StringResource is created. If you have set up additional system locale settings, you can check whether the default (en) column and system locale (ja) column have been added.



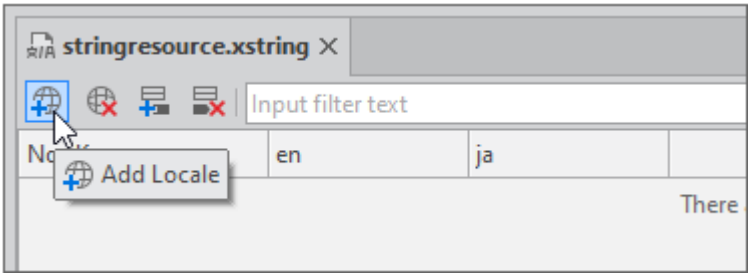
If the StringResource is created by receiving resource from a web URL, you can check the added status of the columns and data set in the resource.



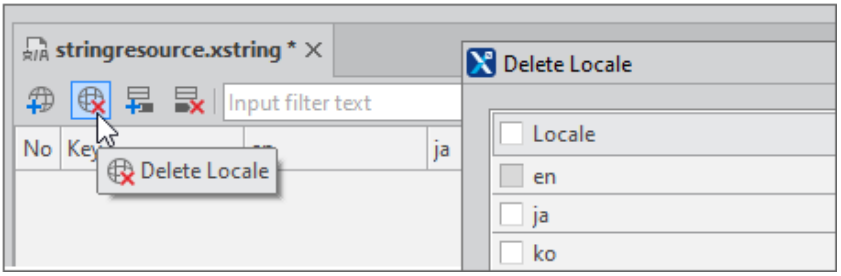
If the SDK version was changed for a project created on a previous version, the TypeDefinition will not have the StringResource information.
In this case, create the StringResource information by selecting [File > New > StringResource].

21.1.2 Adding and Deleting the Locale (Language Column)

Click the [Add Locale] icon on the toolbar to open the window where you can select a language to add. Select a language and click the [OK] button to add the column for the selected language.

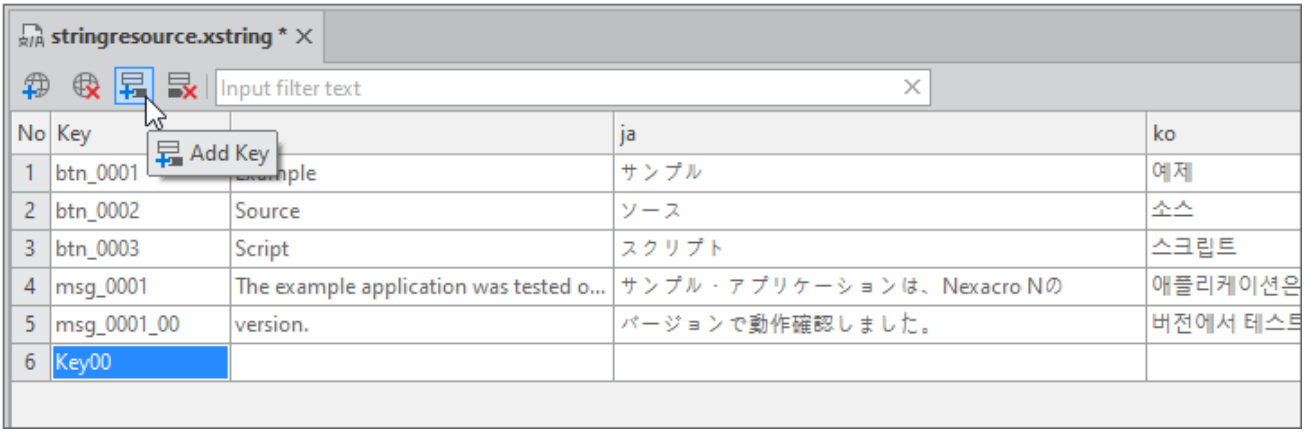


You can delete language columns that are no longer used. Click the [Delete Locale] icon on the toolbar to see a list of currently set language columns and select the language to delete. English (EN) cannot be deleted because it is the default language.



21.1.3 Adding and Deleting Data

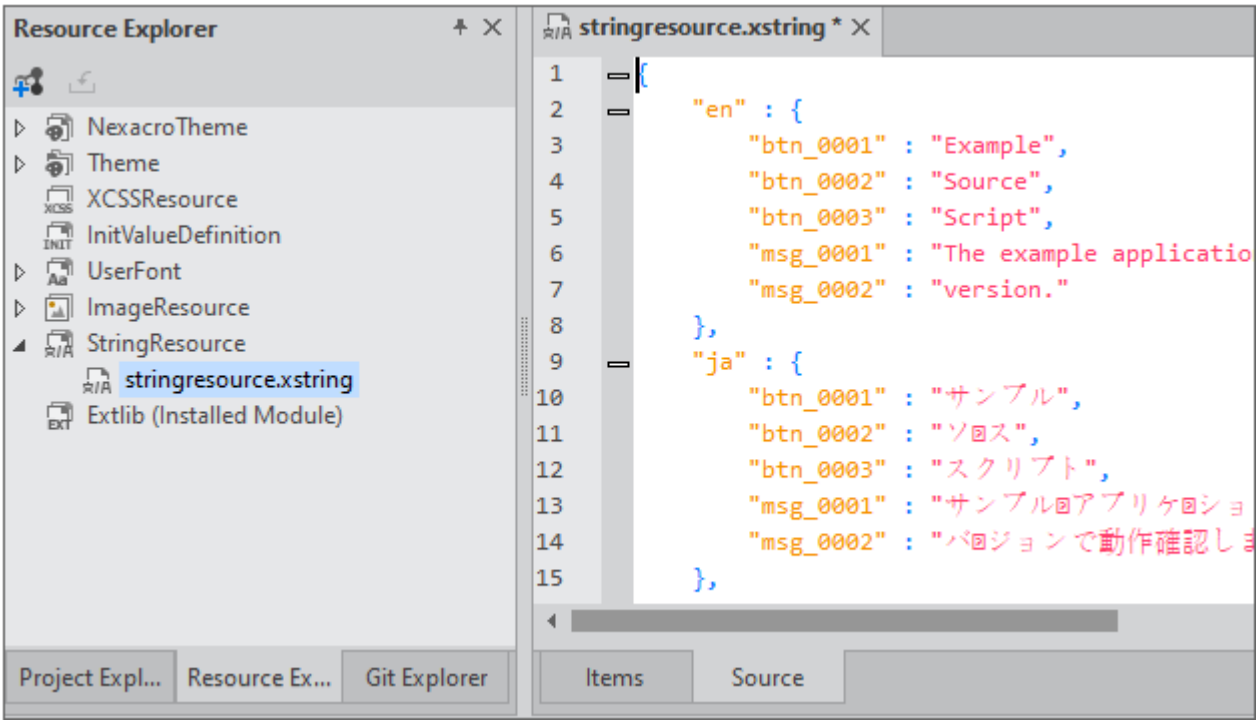
Click the [Add Key] icon on the toolbar to add an empty row. Edit the key value in the added row and enter the appropriate value in each language column.



To delete data, select the key item to delete and click the [Delete Key] icon on the toolbar.

21.1.4 Making Modifications in the Data Source

StringResource data are stored in JSON format. You can check the data you created by selecting the Source tab. You can also modify the value after modifying the data according to the designated format outside.

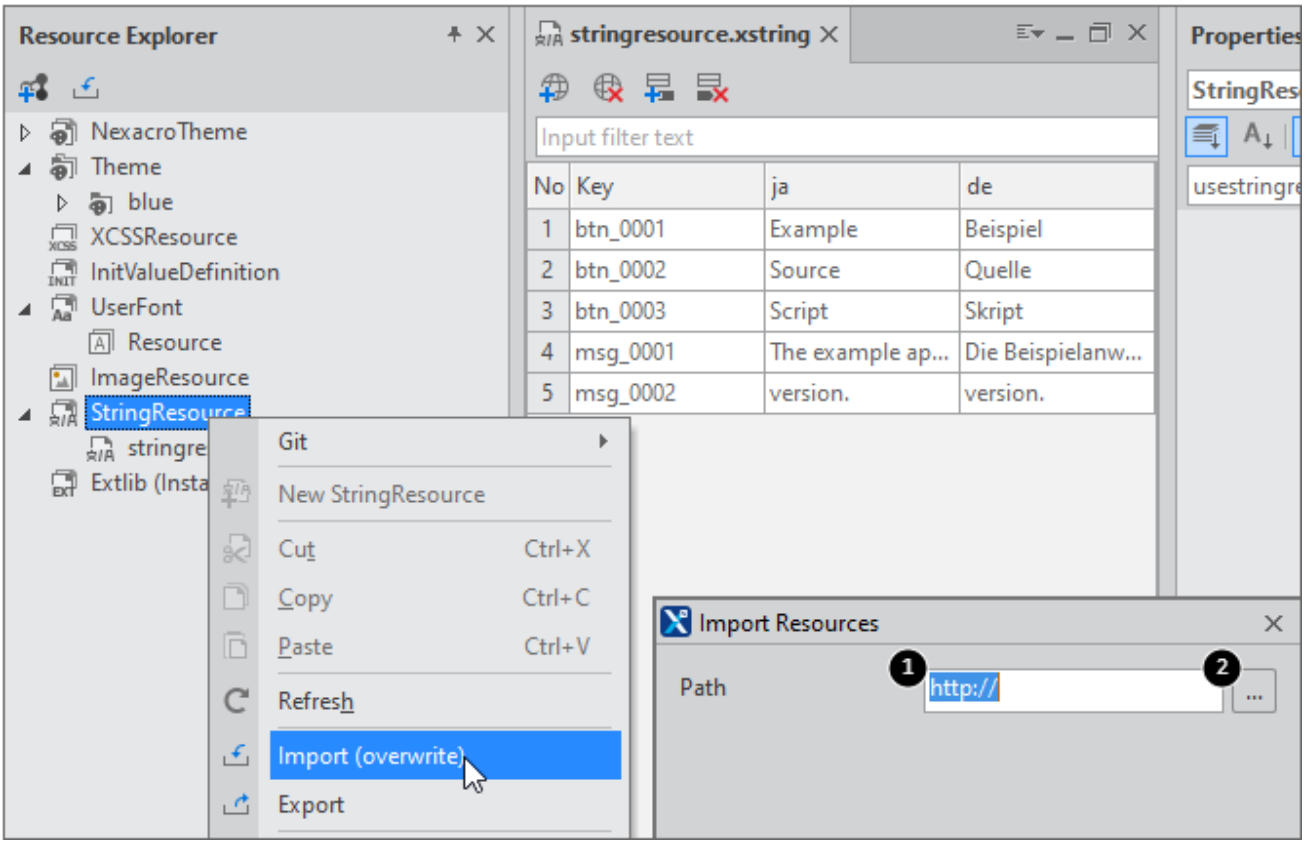


21.1.5 Overwriting with external StringResource data

When Overwrite data is selected, all existing data is deleted and overwritten with new data.

Import (overwrite)

When importing a StringResource from an external source or creating a StringResource and then overwriting it with other data, select the StringResource item in Resource Explorer and select [Import (overwrite)] from the context menu.



Resource files can be selected from remote files or downloaded files.

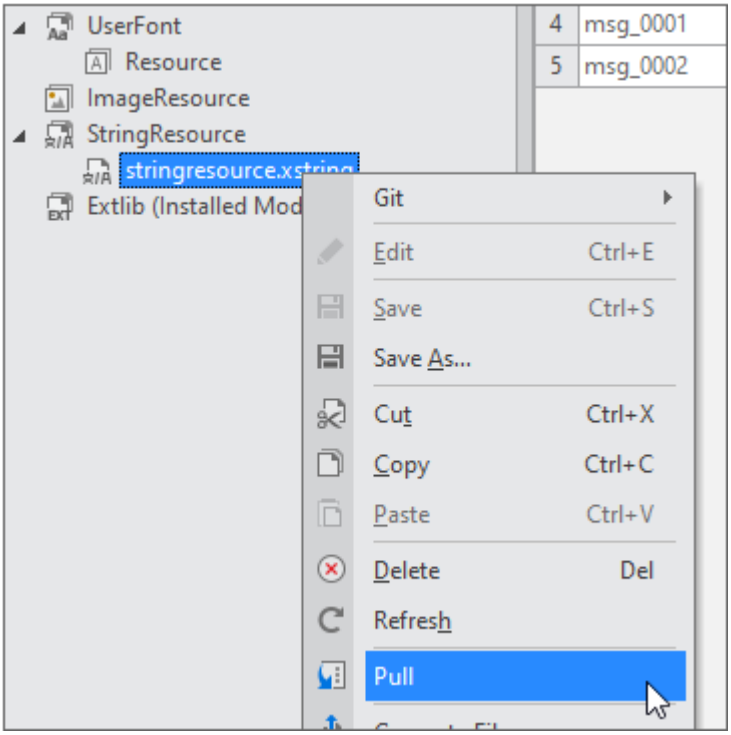
	Item	Description
1	Web URL	Set the URL where the resource file is located.
2	File	Select the resource file.

pull

When creating a StringResource file with a remote resource file or specify a Web URL when importing (overwrite), you can overwrite the data by selecting the [pull] menu.

In Resource Explorer, select the generated stringresource.xstring file entry and select [pull] from the context menu.

There is no need to specify the resource file URL because the previously entered URL information is used.

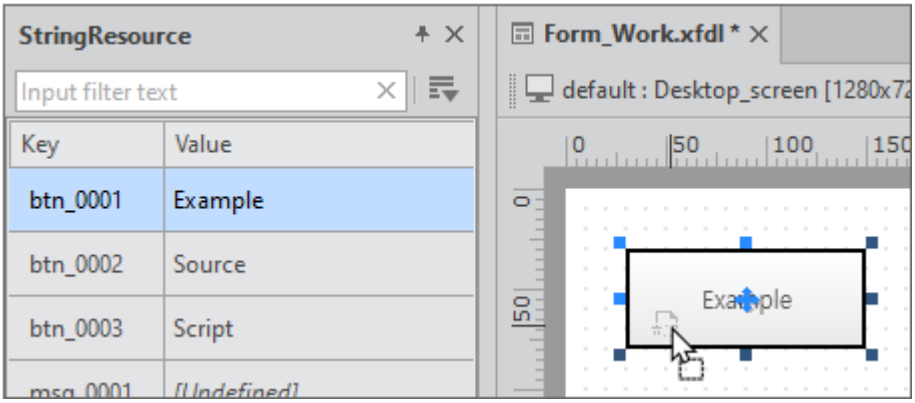


21.2 Applying Multiple Languages

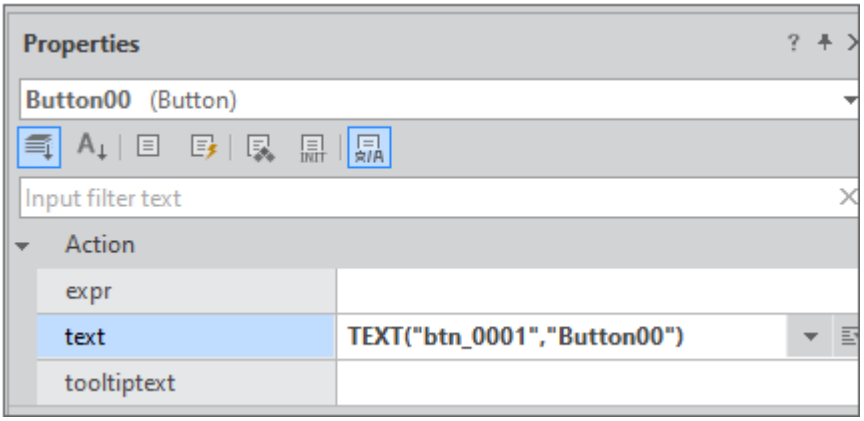
Let's examine how to reflect the StringResource data that have been created as a component property value.

21.2.1 Reflecting Selected Key Value in Components in the StringResource Pane

Select the menu [View > StringResource] item to display the StringResource window. Select the key item you want to apply, drag and drop it onto the component, and the string corresponding to the selected key will be set as the default multilingual property value of the component.

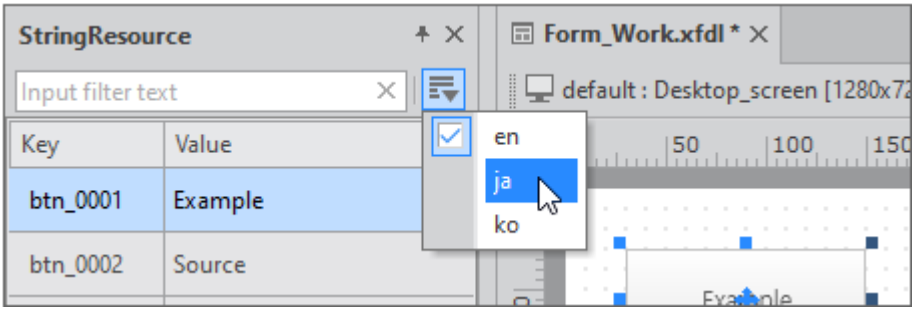


For the Button component, the default multilingual property is text. When you drag and drop a key value in the StringResource pane, it will be applied to the text property value.

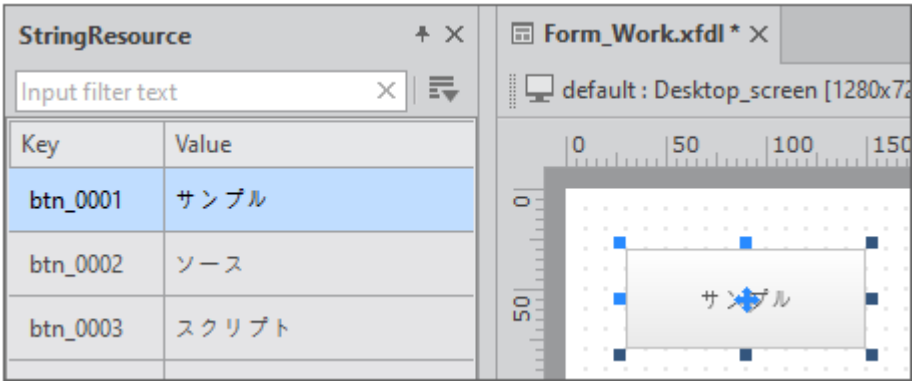


21.2.2 Changing the Language Applied to the Page

Click the button in the top right corner of the StringResource window to display a list of languages you can set.

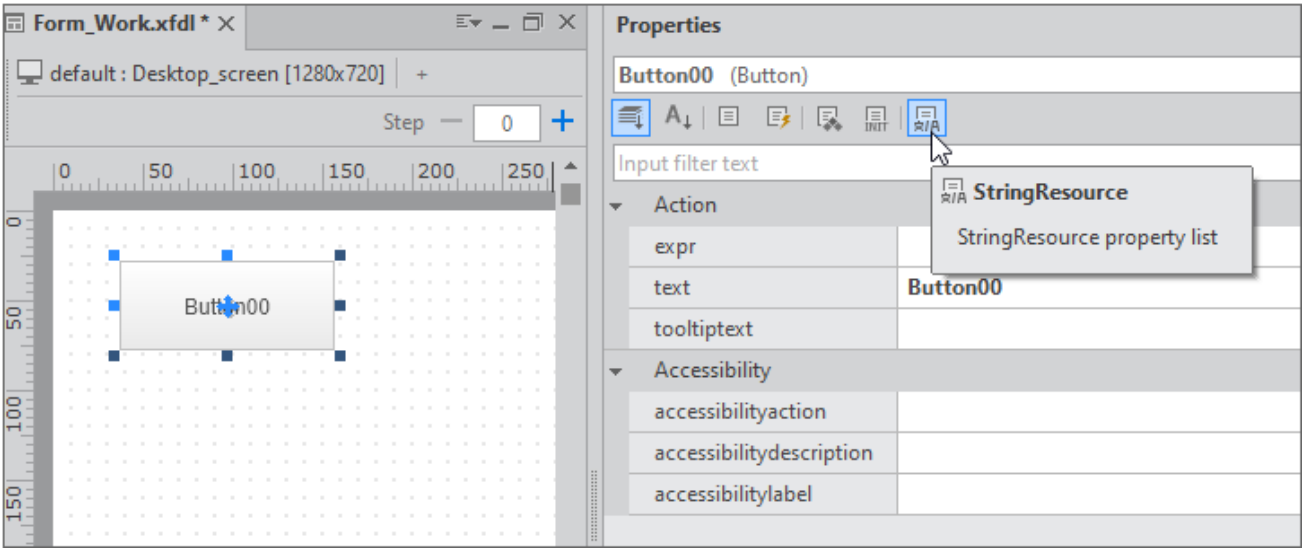


When you select a language to apply to the page, the StringResource window, as well as the property values applied to the page, will display the changes.

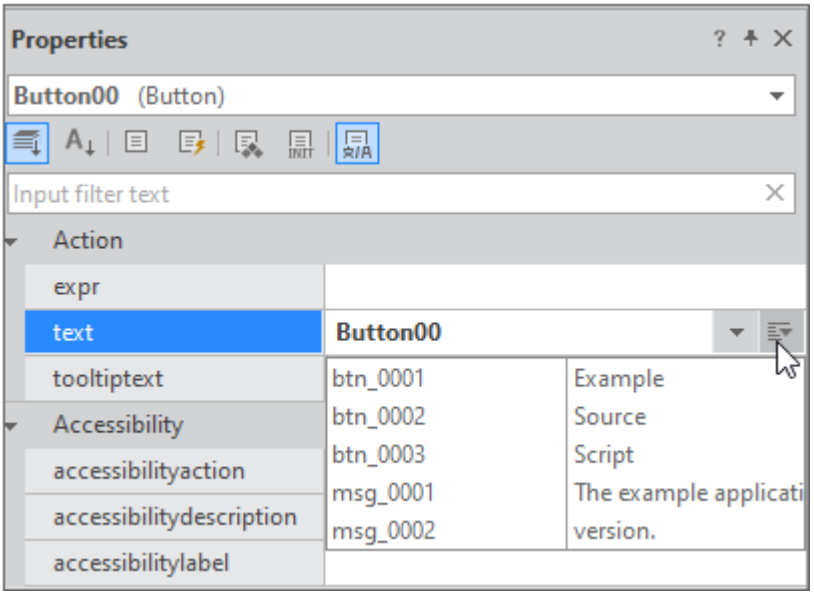


21.2.3 Reflecting the Key Value in Components in the Properties Pane

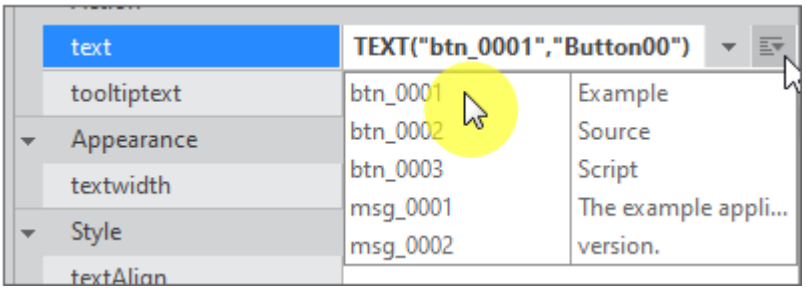
After selecting a component, select [StringResource] on the toolbar of the Properties pane to display a list of properties to which multiple languages an be applied. Among the Component properties, the properties where you can set text will be displayed.



Click the button on the right side of the Properties pane to display a list of keys you have added to the StringResource. When you select a key value, the selected value will be reflected as a property value for the Component.



If there is no property value that has been set, the key value will be loaded, and if there is a property value that has been set, the key value will be checked and if there is no value corresponding to the key value, the existing property value will be set.

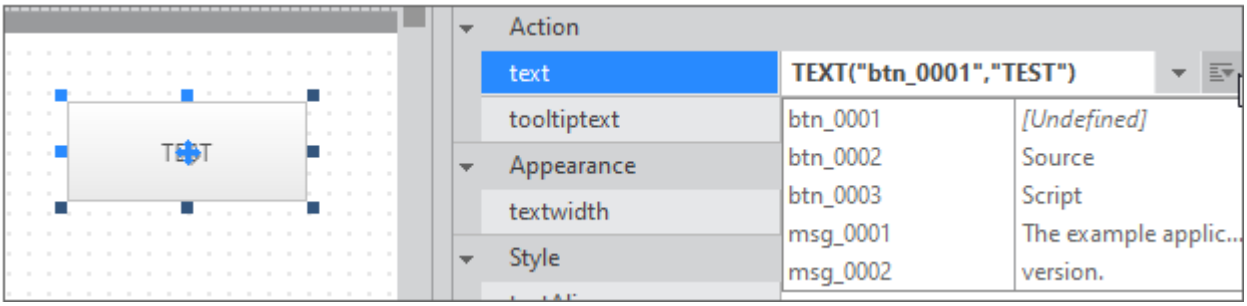


21.2.4 TEXT function

To apply multiple languages to a component property value, you can use the TEXT function to apply StringResource values. The TEXT function is applied in two forms as shown below:

- `TEXT([key value])`
  Displays the data corresponding to the key value.
  Displays the key value if there are no data corresponding to key value.
- `TEXT([key value], [alter value])`
  Displays the data corresponding to the key value.
  If there is no data corresponding to the key value (Undefined), the alter value is displayed.
  If a Key value that is not in the StringResource is set, the alter value is displayed.
  If the key value is applied while the property value is set, the existing property value will be set as the alter value.

In the following case, the string “TEST” is displayed because there is no language column value corresponding to “btn_0001”. If there is no column value, it is displayed as [Undefined].



21.3 Importing the Key Value with a Script

21.3.1 getStringResourceValue method

This is a method of importing the value set in a StringResource.

For example, the value of the text property of a Button component can be set as follows:

```
this.Button00.text = nexacro.getStringResourceValue("btn_0001");
this.Static00.text = nexacro.getStringResourceValue("msg_0001")
    +" 24.0.0.100 "
    +nexacro.getStringResourceValue("msg_0002");
```

21.4 Importing Data from a Remotely Located StringResource

21.4.1 Remote StringResource Data Types

This is the data type needed when creating a new StringResource using a remote resource or calling the nexacro.loadStringResource method.

The returned data should be formatted as shown below:

```
{
    "[Locale Code_1]":    {
        "[Key_1]" : "[Value_1]",
        "[Key_2]" : "[Value_2]"
    },
    "[Locale Code_2]":    {
        "[Key_1]" : "[Value_1]",
        "[Key_2]" : "[Value_2]"
    }
}
```

```
{
    "en":    {
            "btn_0001" : "Example",
            "btn_0002" : "Source",
            "btn_0003" : "Script",
            "msg_0001" : "The example application was tested on the Nexaro N",
            "msg_0002" : "version."
    },
    "de":    {
            "btn_0001" : "Beispiel",
            "btn_0002" : "Quelle",
            "btn_0003" : "Skript",
            "msg_0001" : "Die Beispielanwendung wurde auf dem Nexaro N
getestet",
            "msg_0002" : "version."
    }
}
```

If you do not specify a value for the key, it is set to null.

```
{
    "en":    {
            "btn_0001" : "Example",
            "btn_0002" : "Source",
            "btn_0003" : "Script",
            "msg_0001" : "The example application was tested on the Nexaro N",
            "msg_0002" : "version."
```

```
    },
    "de":    {
            "btn_0001" : "Beispiel",
            "btn_0002" : "Quelle",
            "btn_0003" : "Skript",
            "msg_0001" : "Die Beispielanwendung wurde auf dem Nexaro N
getestet",
            "msg_0002" : "version."
    },
    "ko" : {
            "btn_0001" : null,
            "btn_0002" : null,
            "btn_0003" : null,
            "msg_0001" : null,
            "msg_0002" : null
    }
}
```

21.5 Multiple Language Application Criteria

21.5.1 Language

They are applied in the following order of priority:

1. Environment locale properties
2. User operating system locale information

If the applicable language column is not present in the StringResource, it will be applied based on English (en).

21.5.2 StringResource Data

It will be applied in the following order of priority:

1. loadStringResource method
2. If the usestringresource property value is false: stringresourceserviceurl property

If the usestringresource property value is true: StringResource

22.

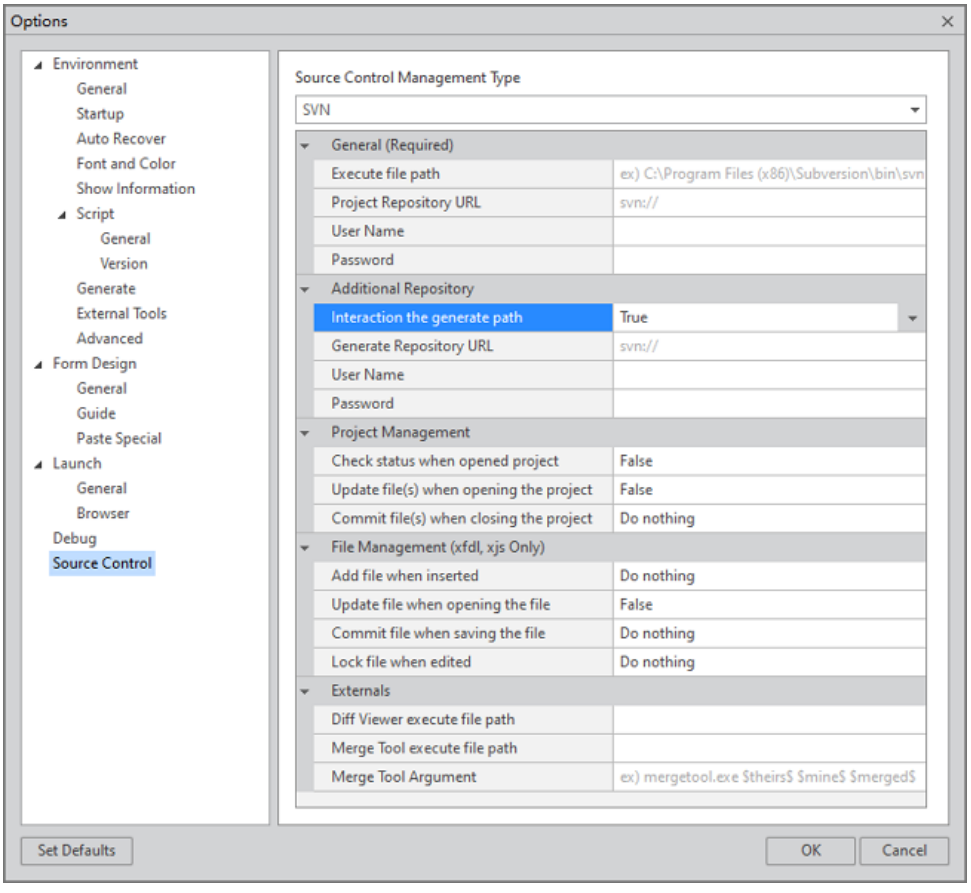
Supporting Source Control Software

Nexacro supports the connectivity with revision control software. Features may differ depending on programs.

22.1 Subversion (SVN)

22.1.1 Settings for Subversion

If you install a Subversion client, you can select **SVN** by accessing [**Tools** > **Options** > **Source Control**].

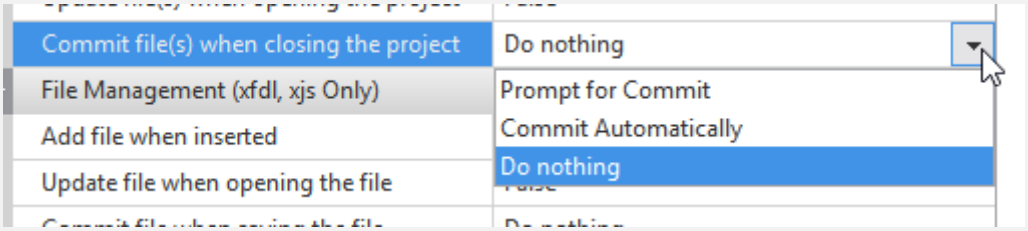


Nexacro Studio supports the following options for the use of Subversion.

Option	Description
General (Required)	
Execute file path	Path to the executable of Subversion Command-Line Client
Project Repository	Information on the URL and account for accessing the repository of project source codes
Additional Repository	
Interaction the generate path	Controls the Generate Path
Generate Repository	Information on the URL and account for accessing the repository of generated source codes
Project Management	
Check SVN status when opened project	Checks file statuses automatically when opening a project
Update file(s) when opening the project	Loads files of latest versions automatically when opening a project
Commit file(s) when closing the project	Commits checked-out files automatically when closing a project
File Management (xfdl, xjs files only)	
Add file when inserted	Performs the **Add** command automatically when a client creates a file
Update file when opening the file	Loads the latest version automatically when opening a file
Commit file when saving the file	Commits a file automatically when saving the file
Lock file when edited	Locks a file automatically while you are editing the file so that other users are not allowed to change the file
Externals	
Diff Viewer execute file path	Registers an external program for comparing files according to versions
Merge Tool execute file path	Registers an external program for merging files
Merge Tool Argument	Enter the Argument value used in Merge Tool.



In the below menu items, you can choose one of the three options.
Commit file(s) when closing the project
Add file when inserted
Commit file when saving the file
Lock file when edited



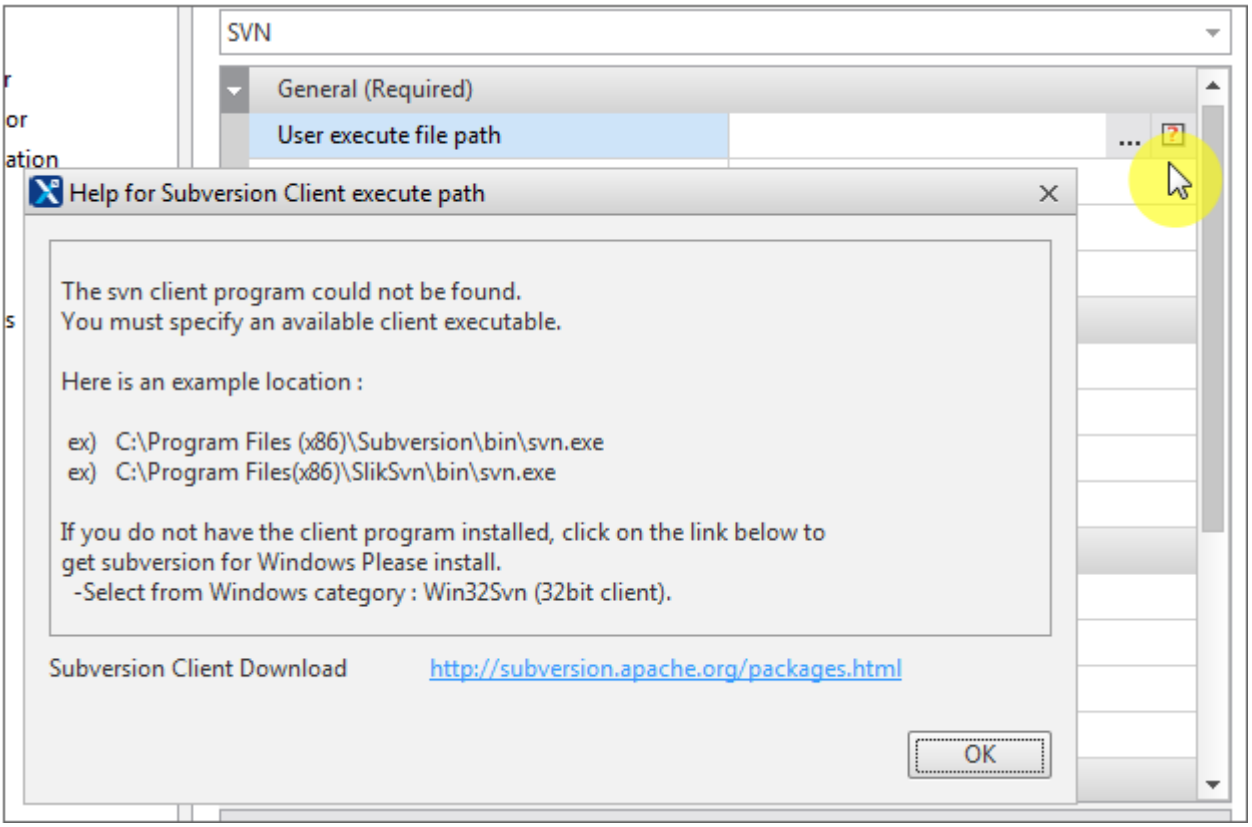
Commit Automatically: conducts the relevant command automatically without asking the user

Prompt for Commit: asks the user whether to conduct the relevant command

Do nothing: does not conduct the relevant command anyway

You must install a Subversion client to make the SVN option activated. Nexacro Studio identifies the installation of a Subversion client by checking the related entries in Windows Registry.

If you click the [?] button at the end of the User execute file path option, you can see the information that helps you install the Subversion client.



If the version of Subversion is earlier than 1.6.0, either the SVN option does not appear in Nexacro Studio or some features do not work.

This documentation does not provide a guide for the installation of the Subversion client. You can get related information by following the below link.
http://subversion.apache.org/packages.html

22.1.2 Toolbar

The toolbar in Nexacro Studio provides the below features for Subversion.

INCLUDE_ERROR(ELEMENT_REMOVED)

	Name	Description
	Update	Load files of latest versions from the repository
	Commit	Apply modifications to the original files in the repository
	Revert	Return to former status of a file by cancelling the previous task
	Lock	Lock a file while you are editing the file to prevent other users from making a modification
	Unlock	Unlock a file
	Import	Register a project in the repository
	Add	Add a new file. You can apply the added file to the repository by performing the Commit command.
	Delete	Change the status of a file to "deleted". You can delete the relevant file in the repository by performing the Commit command. The actual file stored in your PC will not be deleted.
	Rename	Change a file name. SVN does not have a *Rename* feature--it indeed deletes the existing file and adds a new file.
	Edit Conflicts	Edit a file in the conflict state
	Resolve	Resolve conflicts
	Clean up	Browse a file recursively and erase error messages
	Refresh Status	Refresh the file
	Show Log	See the modification history
	Compare with Previous Revision	Compare the current and previous versions and identify differences. This function executes a program registered through *Diff Viewer execute file path* under the Options menu.
	Repository Browser	Launch Repository Browser

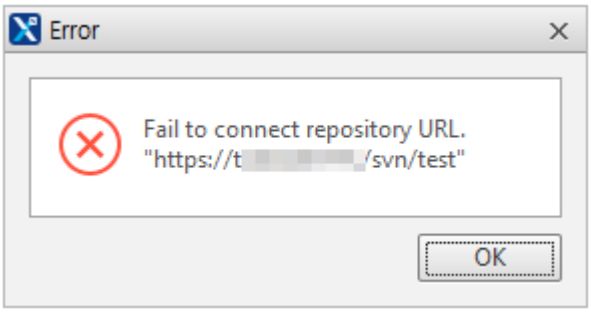
The above functions are also provided by the menu bar of Nexacro Studio. If you want to use Subversion

through the menu bar, you need to access the View menu and activate the SVN Output and Check for Modification (Project) panes.

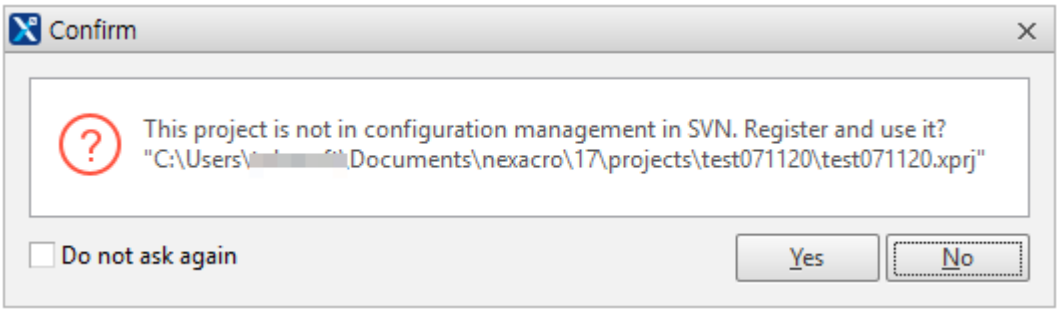
> To activate the menu bar and toolbar related to SVN, you must install a Subversion client first and then set Source Control in the menu [Tools > Options].

22.1.3 Authentication

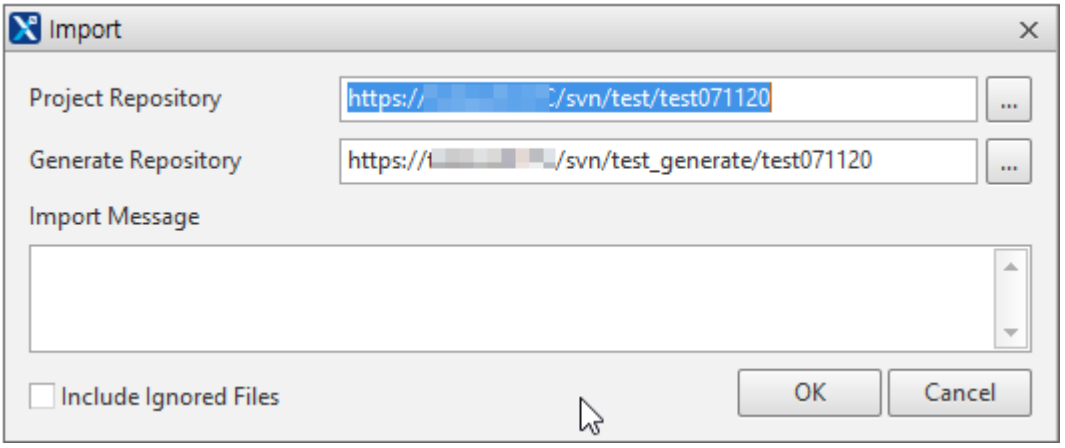
In the previous stage, you learned how to set up the information on the project repository or generate repository including the URL and account. Then, you now need to access the repository to deal with the authentication. If you are successfully authenticated, there will be no message. If you fails, however, the below message will appear.



If you change the option of using SVN while working on a project unconnected to the repository, the below dialog box will appear to ask you whether to register the project in the repository. If you click "Yes", Nexacro Studio will begin the process of importing the project to the repository.



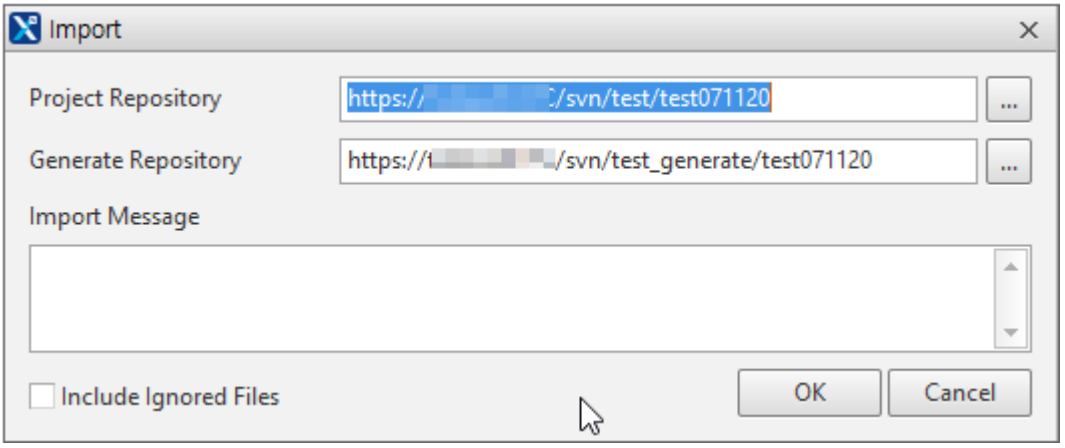
If you set the project repository only, the following pop-up will also present the project repository only. If you set the project repository and generate repository, however, the pop-up will also present both. Folders named after the projects will be automatically created under the directories that were designated as repositories.



22.1.4 Basic Features

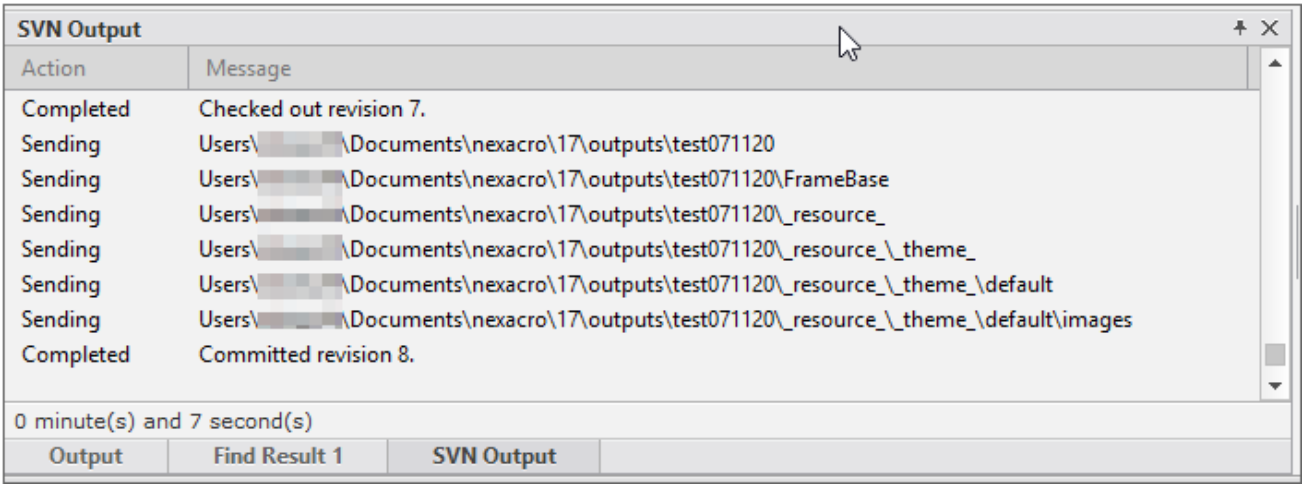
Import

You can create a new project in the repository or import an existing project that is not registered. If Subversion is already in use, a new project will be imported automatically. To import an existing unregistered project, you need to access the menu [SVN > Import].



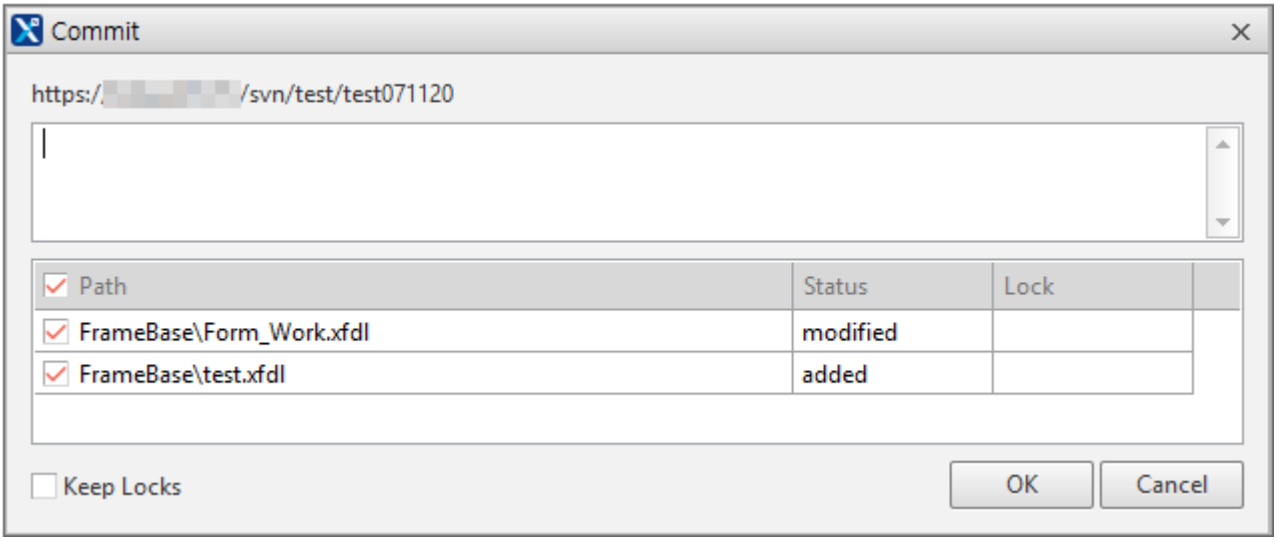
Item	Description
Project Repository	This is the URL of the repository for saving project source codes. A folder named after the project will be created at this address.
Generate Repository	This is the URL of the repository for saving generated source codes.
Import Message	Leave a message concerning the imported project.
Include Ignored Files	Include files that Subversion is set to ignore.

An imported project will be automatically checked out for the connectivity with Subversion.



Commit

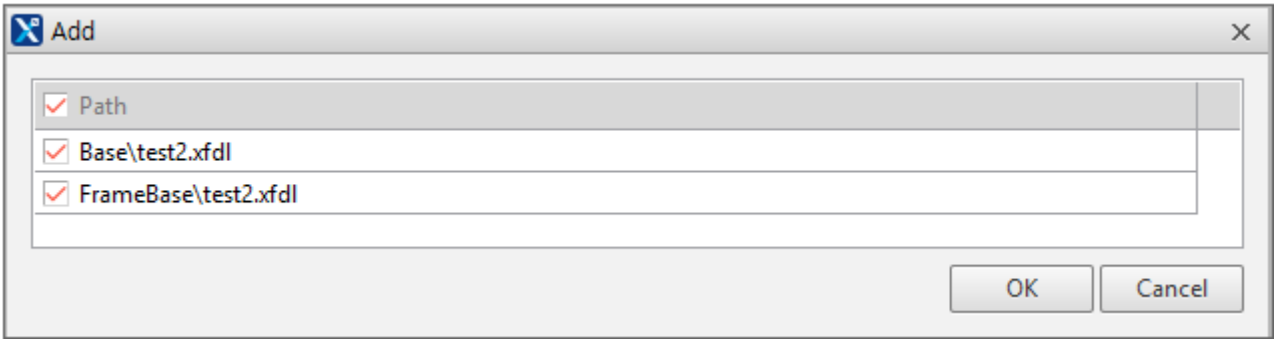
This function is used to apply user-made modifications to the relevant file in the repository.



Name	Description
Message	You can leave a message about your commit.
Path	A list of files that you will commit.
Keep Locks	Determine whether to maintain the locks of the files to be committed. This function will be activated only when relevant files are already locked.

Add

This function is used to add new files to the repository.

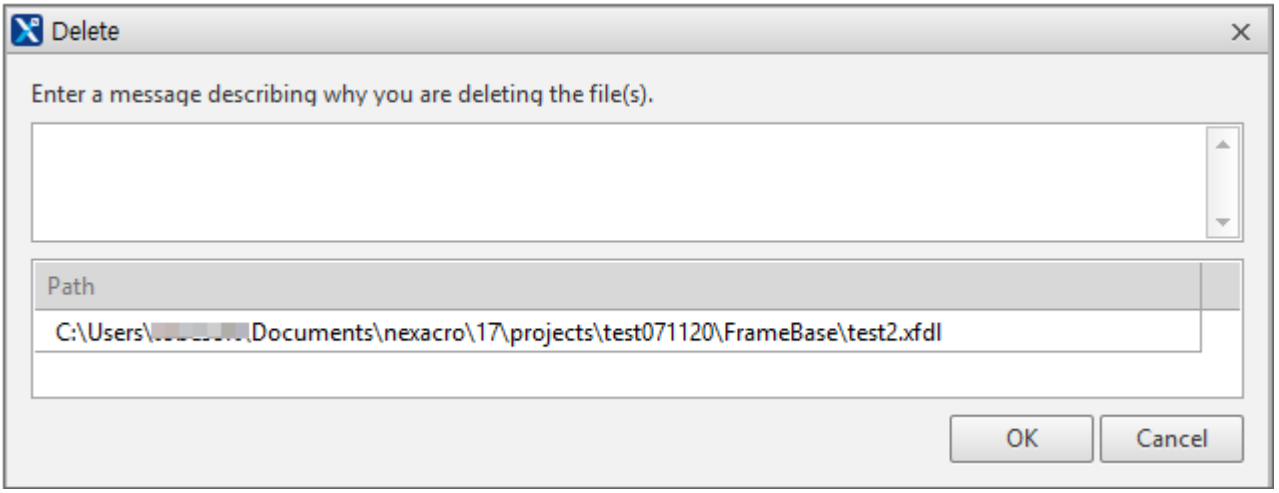


Item	Description
Path	Show a list of files to add.

Performing *Add* merely puts selected files on the waiting list. Those files are actually added to the repository through the *Commit* command.

Delete

This function is used to delete files from the repository.



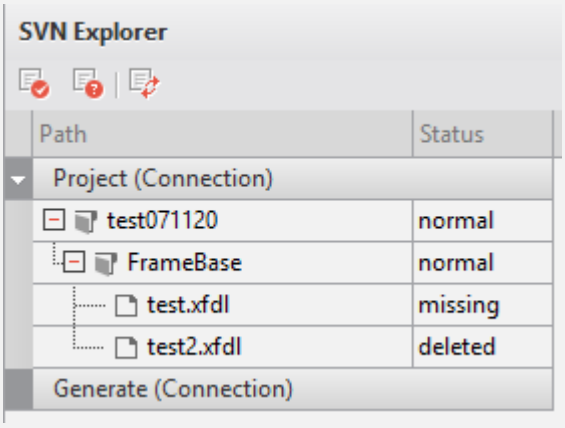
Name	Description
Message	You can leave a message about your deletion.
Path	A list of files that you will delete

Performing *Delete* merely puts selected files on the waiting list. Those files are actually deleted from the repository through the *Commit* command.



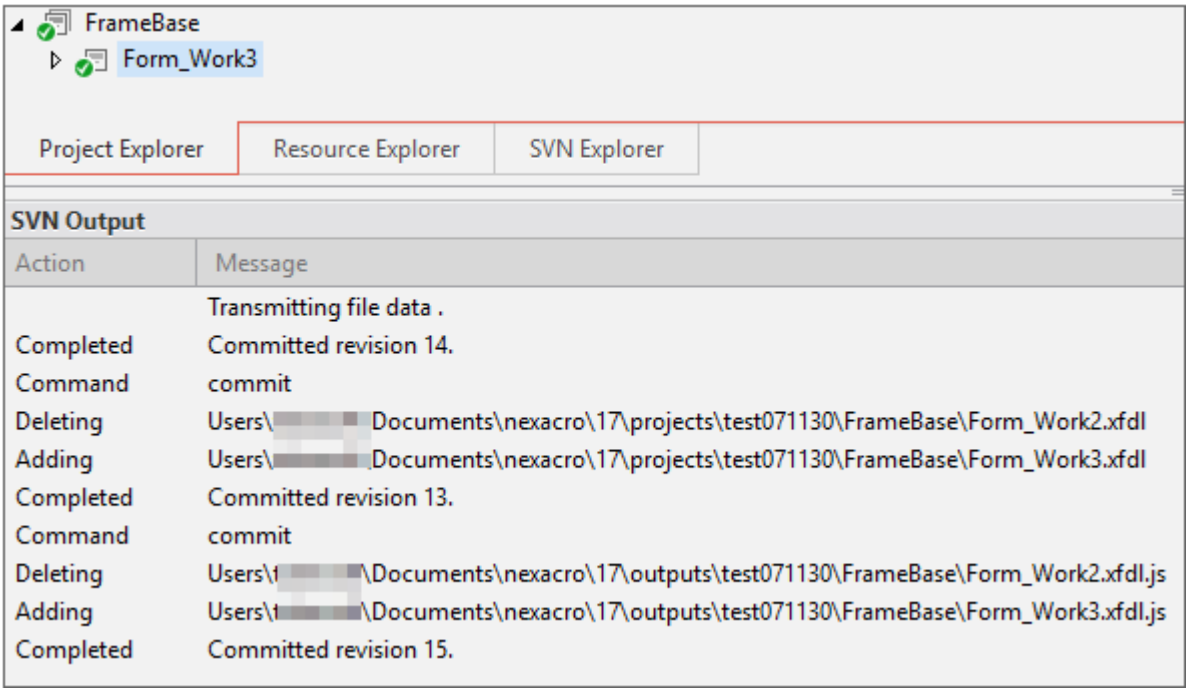
There are two menu items used to delete files: [Edit > Delete] and [Svn > Delete]. The difference is that, if you select [Svn > Delete], you can leave a message about the reason for the deletion.

If you delete registered files in tools other than Nexacro Studio (e.g. Windows Explorer) and the deletion is not done with the SVN features, the statuses of those files will turn to "missing". In this case, those missing files can be restored with the Update function.

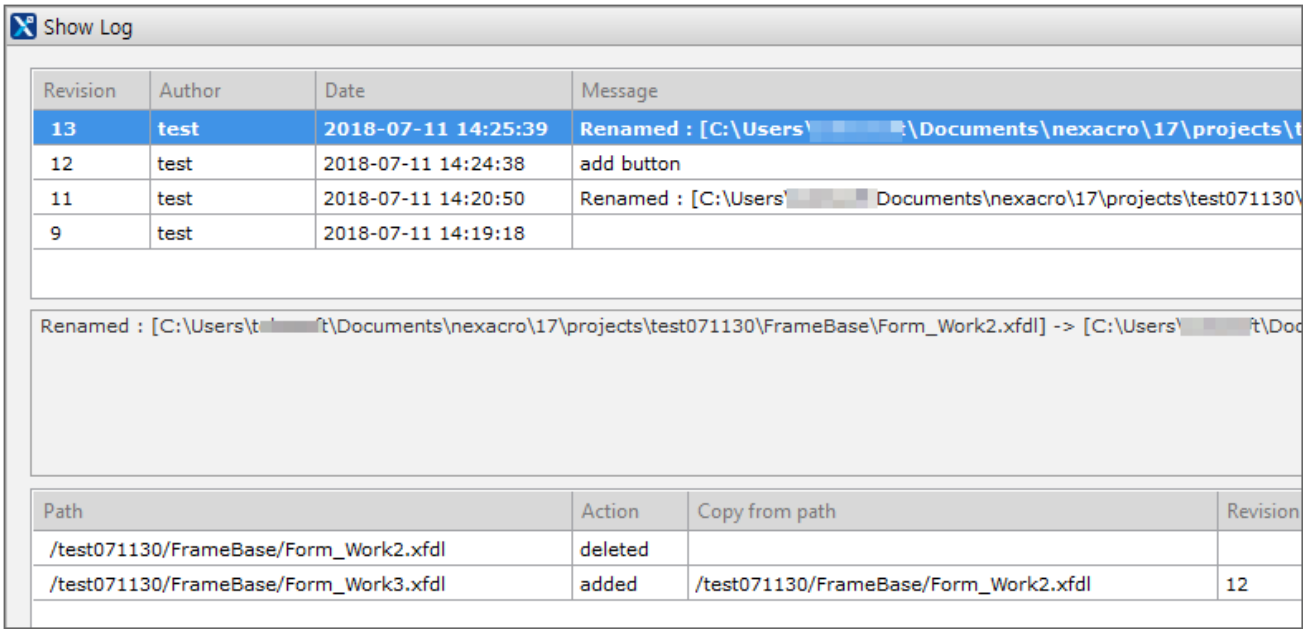


Rename

This function is used to rename a file. The way this function renames a file is atypical: it copies an original file, adds the copy, renames the copy and then deletes the original. The commit command will be conducted as soon as the file is renamed.



Even if a file is renamed, you can check the history of the changes in that file by accessing the menu [SVN > Show Log].



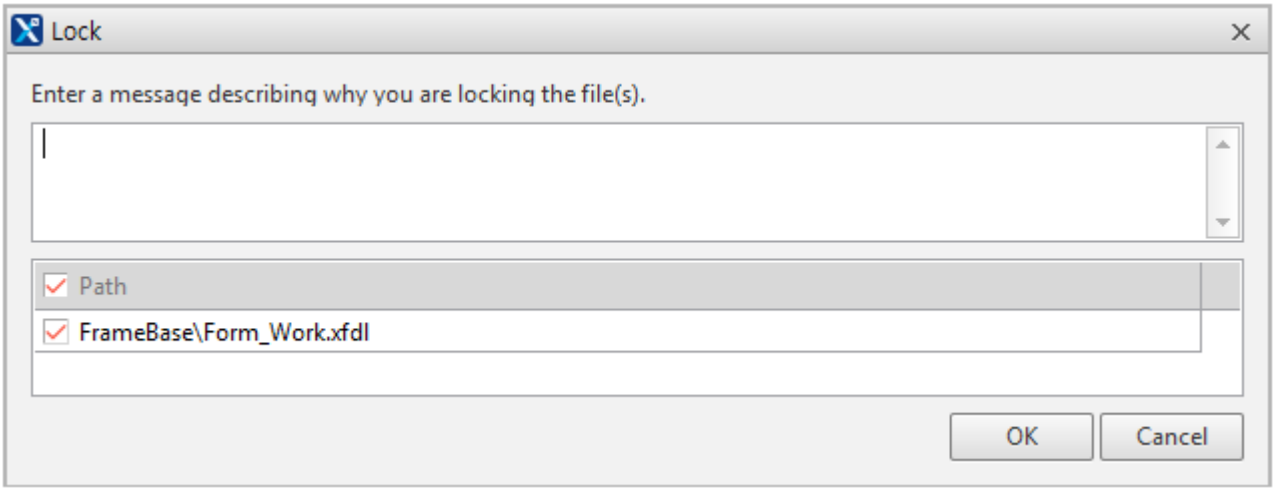
Update

This function is used to update working copies in a user's PC based on the latest versions in the repository. The *Update* command is activated only after SVN statuses of files are verified. This function does not need extra inputs, only presenting the results of the execution.

22.1.5 Lock

Lock

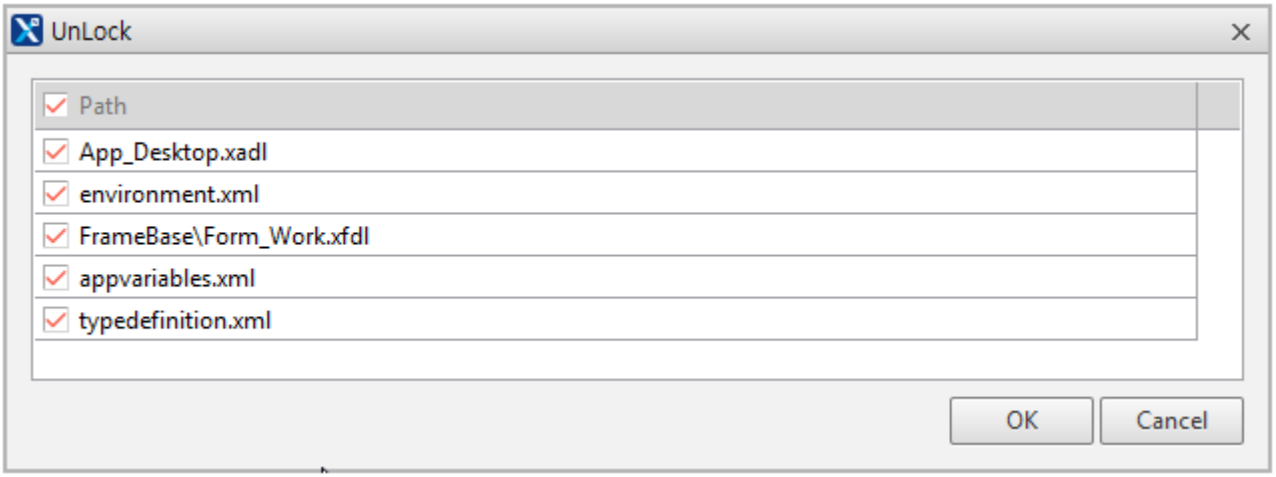
This function is used to lock selected files in order to grant editing permission only to the user who accesses the files.



Name	Description
Message	You can leave a message about the instance of locking files.
Path	A list of files that you will lock

UnLock

This function is used to unlock files.

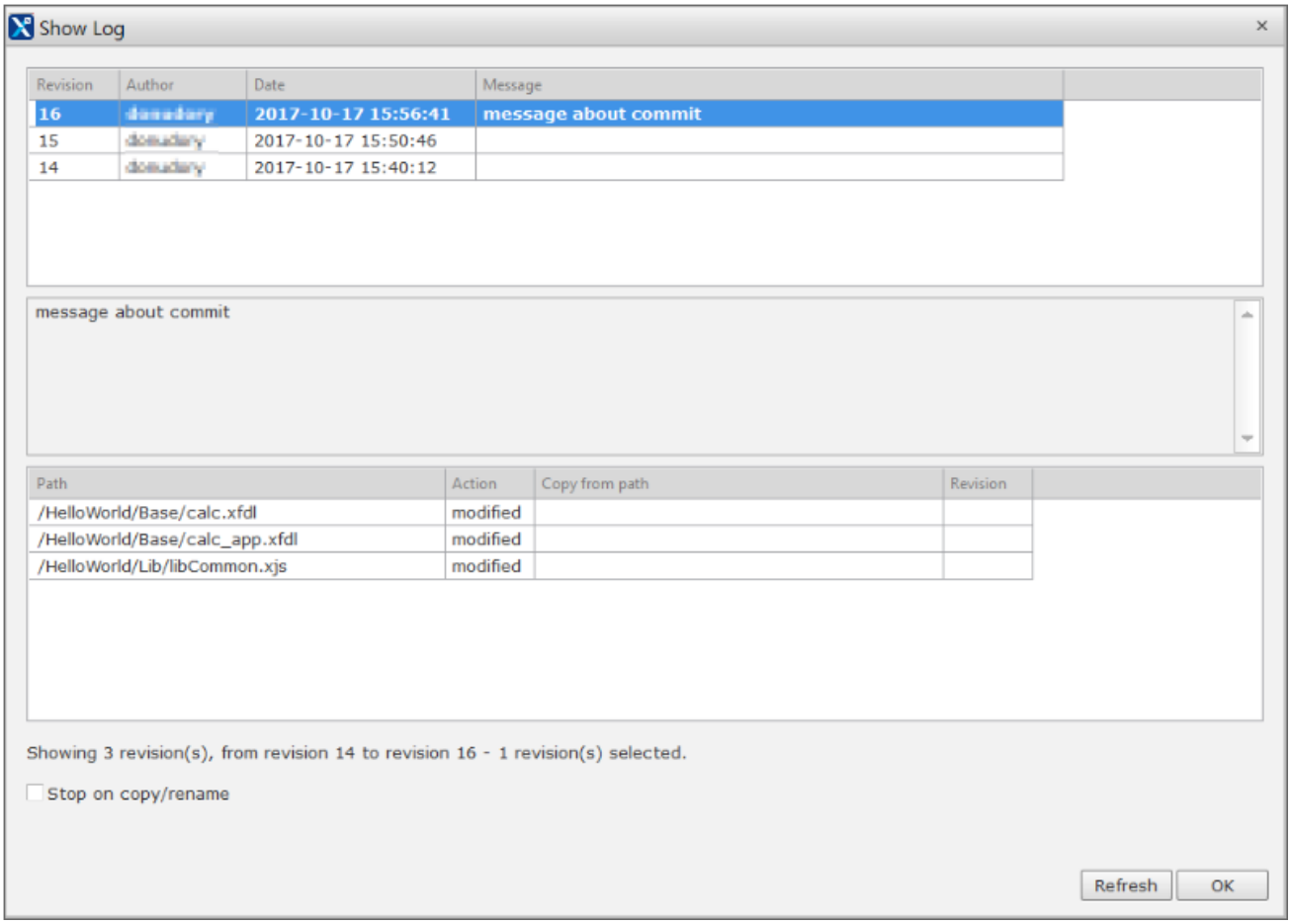


Item	Description
Path	Show a list of files to unlock.

22.1.6 Utilizing Log

Show Log

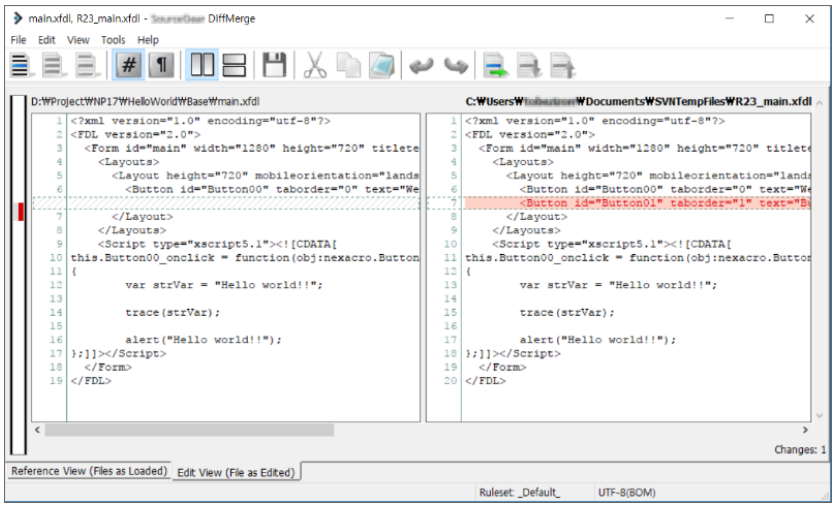
This funtion is used to display a modification log.



Compare with Previous Revision

This funtion compares a working copy with one of the previous revisions stored in the repository and shows the differences. This function will execute a program adopted as a '**Diff Viewer execute file path**' in [**Options > Source Control**].

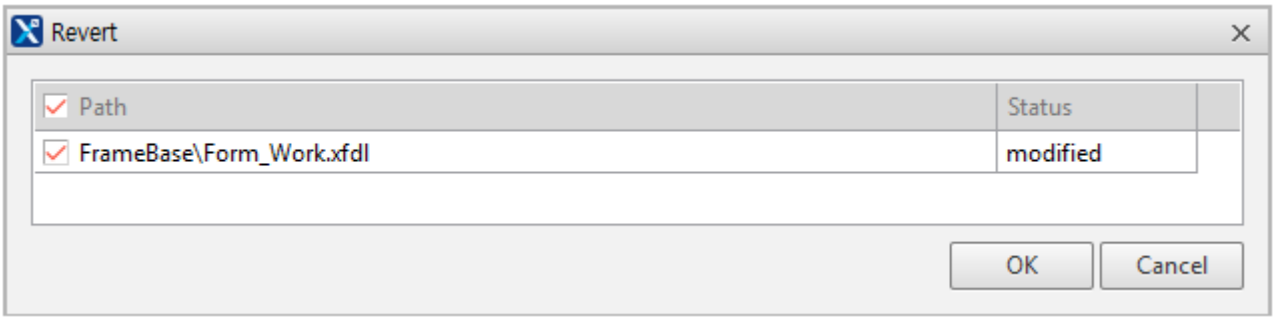
The picture below shows how to run *Compare with Previous Revision* to compare the final revision of the *main.xfdl* file with the previous revision through an external Diff tool.



Basically, Nexacro Studio does not provide Diff and merge tools. You need to separately install commercial tools, open-source programs or freeware programs.
http://winmerge.org/ (open-source)
https://sourcegear.com/diffmerge/ (freeware)

Revert

This function is used to go back to the former state of a working copy by undoing the previous task.



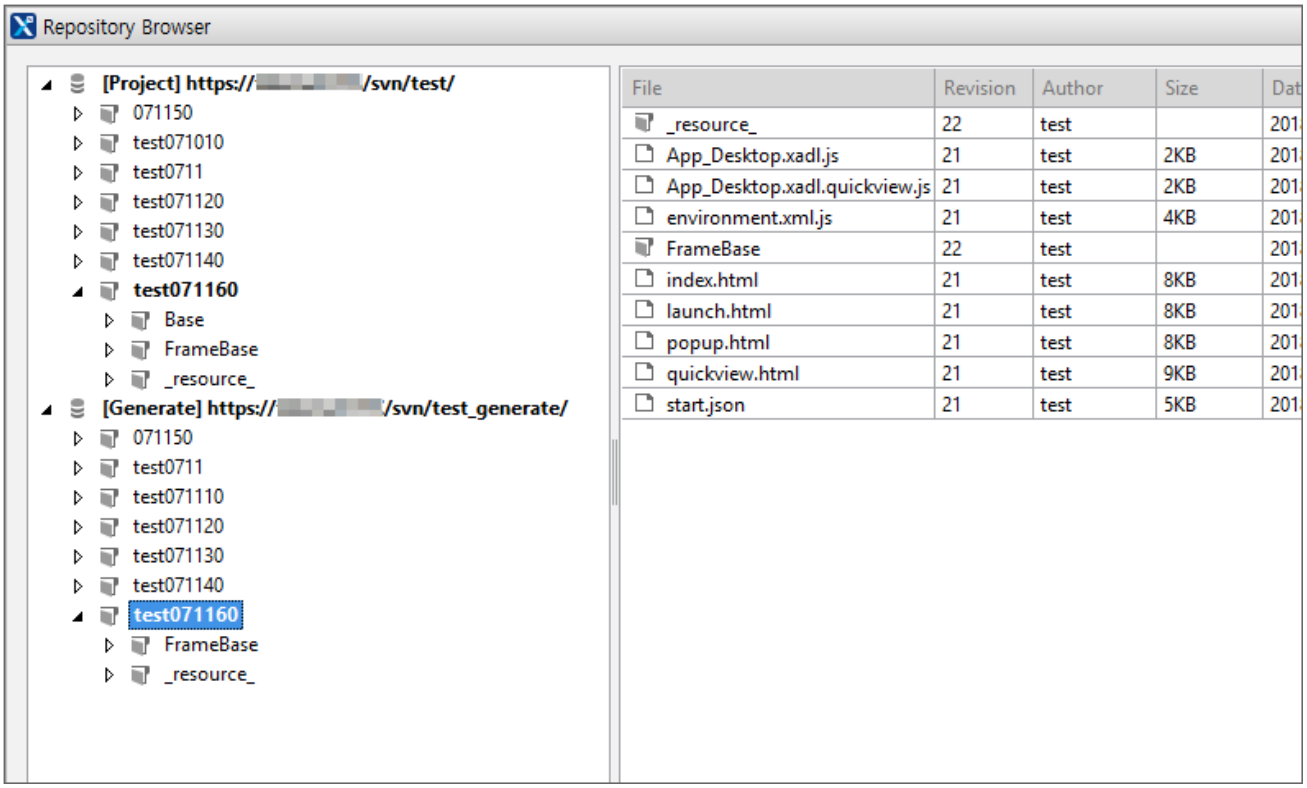
Item	Description
Path	Show a list of files to revert.

Refresh Status

This function is used to refresh the SVN statuses of working copies.

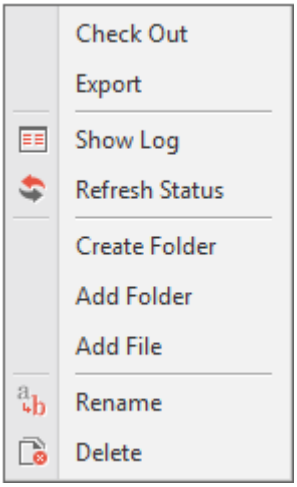
22.1.7 Repository Browser

This is a tool for a SVN client, which is used to display file lists in the repository and support SVN's features.



Item	Description
Folder Tree	Show folders in the repository in the form of a tree
File List	Show a list of files located in the current folder.

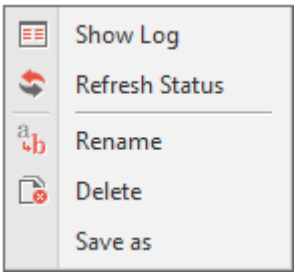
Right-clicking a repository will display the following context menu.



Item	Description
Check Out	Check out selected files and send them to the designated folder in the client PC.
Export	Export selected files to the designated folder in the client PC. The exported files are not checked out yet and thus not connected to the repository.
Show Log	Show a history of the modifications to the repository.

Item	Description
Refresh Status	Refresh the information on the folder.
Create Folder	Create a folder under the current repository.
Add Folder	Add all the files in a selected folder from the client PC to the designated repository.
Add File	Add a file from the client PC to the repository.
Rename	Change the name of a selected repository.
Delete	Delete a selected repository.

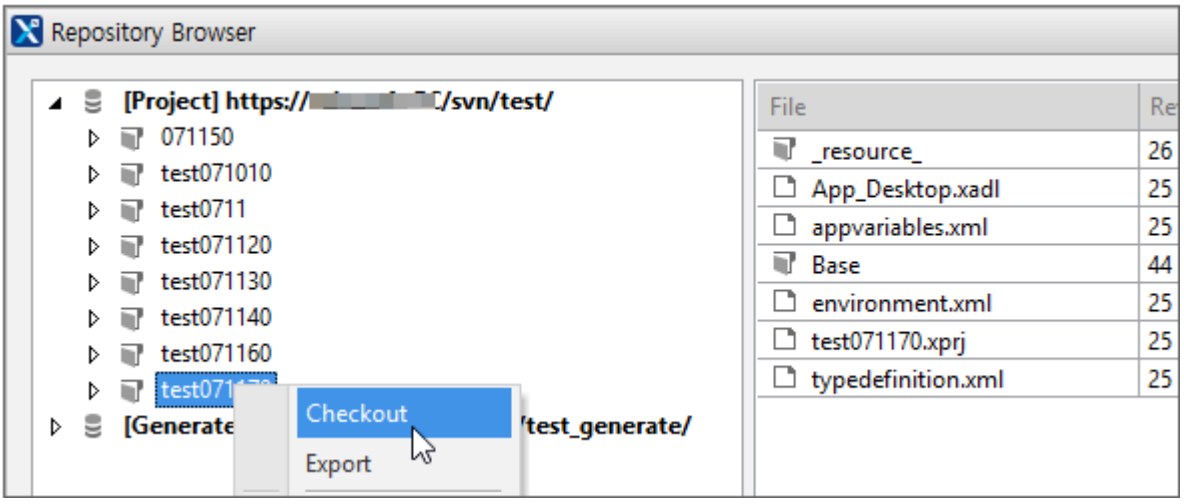
Right-clicking a file will display the following context menu.



Item	Description
Show Log	Show a history of the modifications to the file.
Refresh Status	Refresh the file.
Rename	Change the name of a selected file.
Delete	Delete a selected file.
Save as	Create a copy of a selected file.

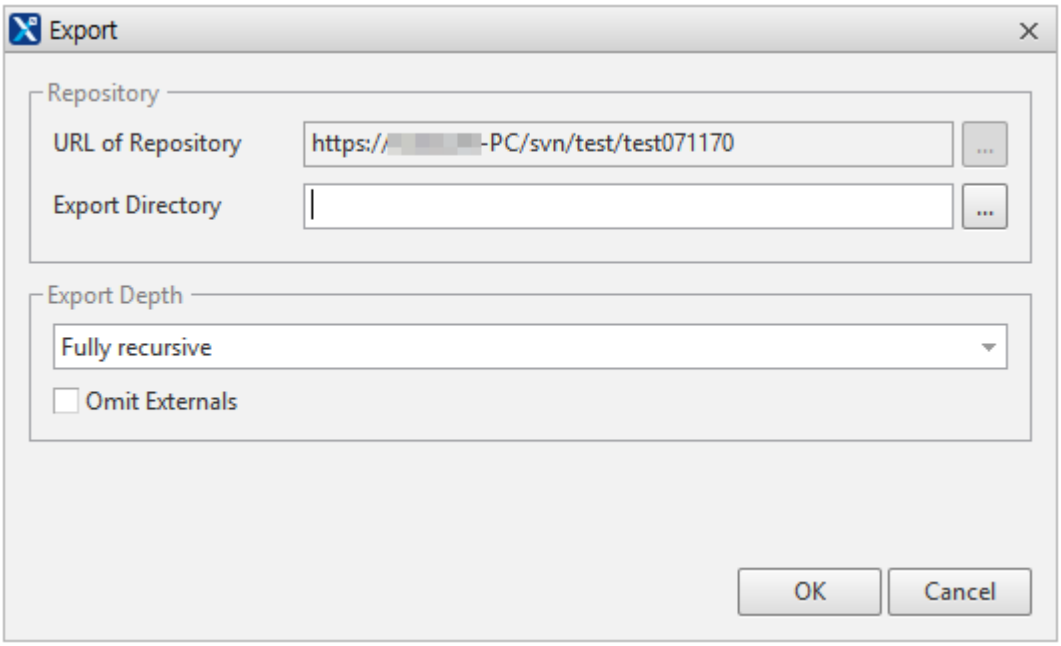
Check Out

You can select a folder from the Repository Browser and check it out to a location you want.



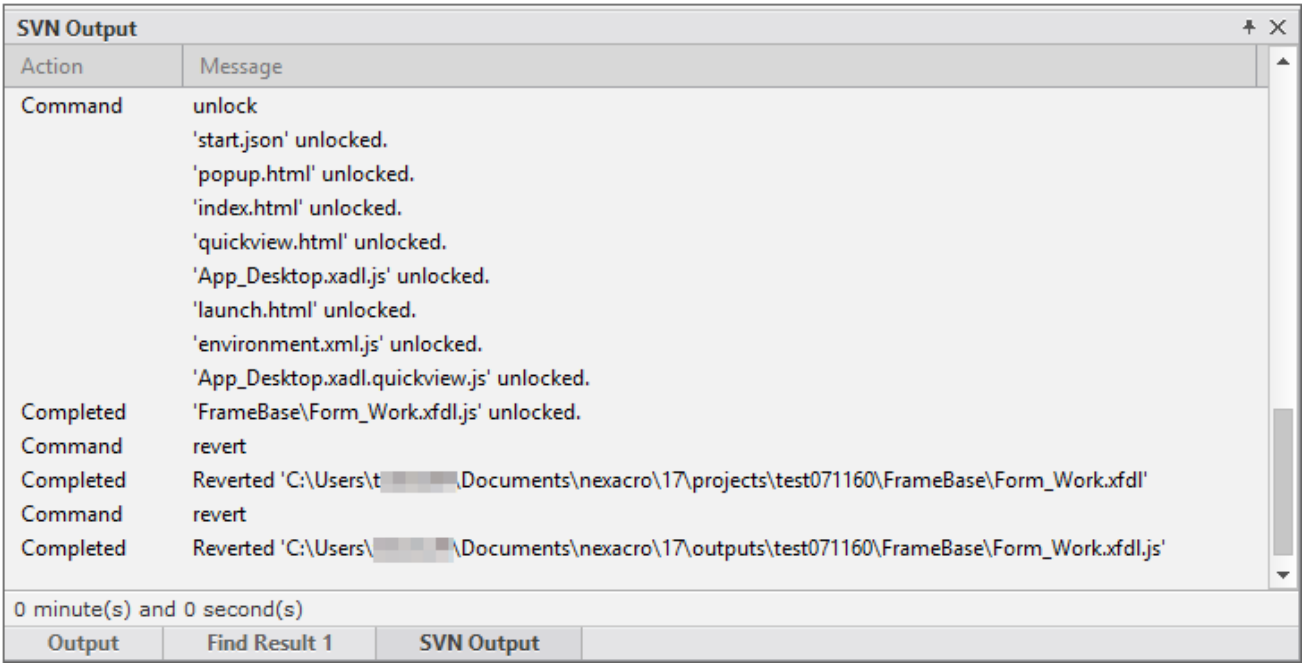
Export

You can select a folder from the Repository Browser and export it to a location you want. The exported files will not be connected to the repository.



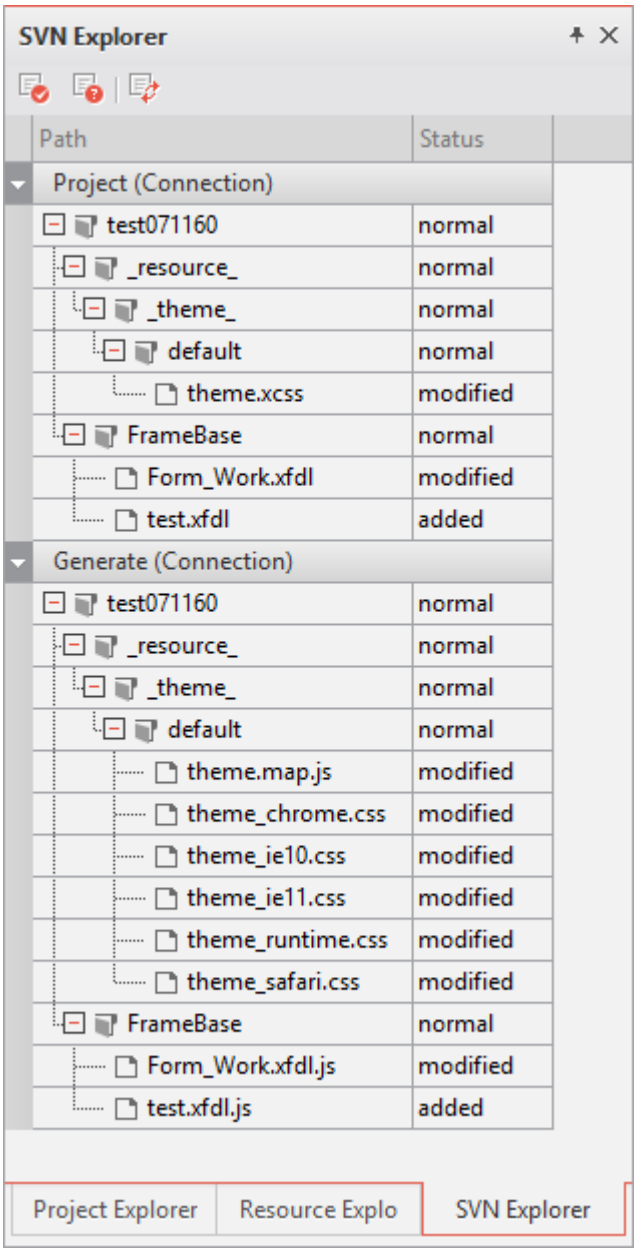
22.1.8 SVN Output

This is a pane to show the results of conducting SVN-related commands.



22.1.9 SVN Explorer

This pane presents the information on the directories and files that belongs projects or repositories managed by SVN.

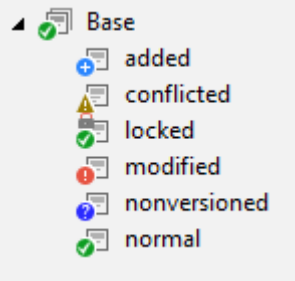


You can filter information by clicking the buttons on the menu bar.

	Name	Description
	Show unmodified	Show status of all files including those are not modified
	Show unversioned	Show status of all files including those are not registered in SVN
	Refresh list	Update the current list to the latest

22.1.10 Subversion Status

Depending on Subversion statuses, Nexacro Studio presents the following overlay icons in the Project Explorer.



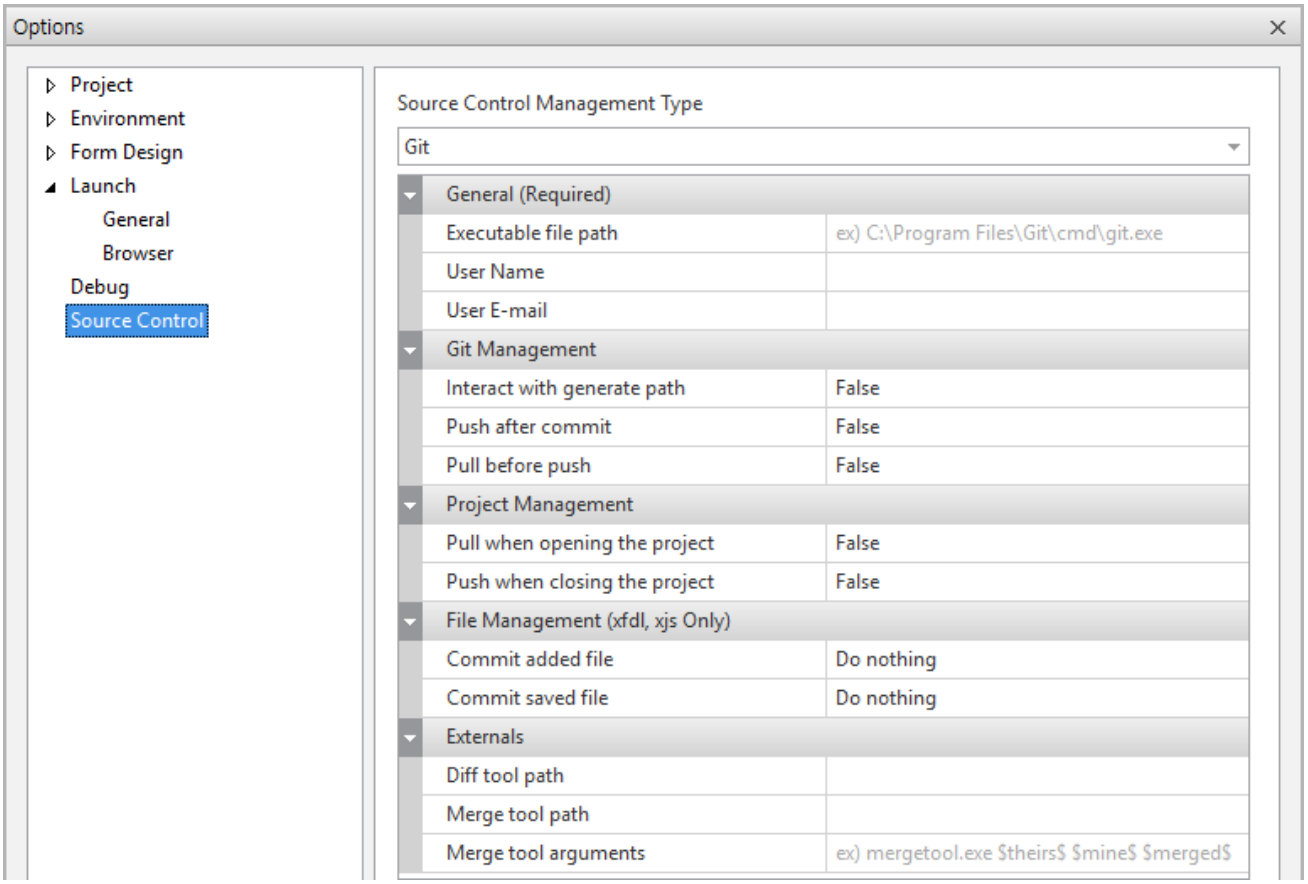
Item	Description
added	Set to be added to the repository
conflicted	A conflict has occurred during the process of “Update.”
locked	The file is locked now.
modified	The file has been modified.
nonversioned	Not registered in the repository
normal	No modification yet to the file

22.2 Git

22.2.1 Configure Git

Set options

After installing a Git client, go to the menu [Tools > Options > Source Control] in Nexacro Studio and select "Git" as the Source Control Management Type.



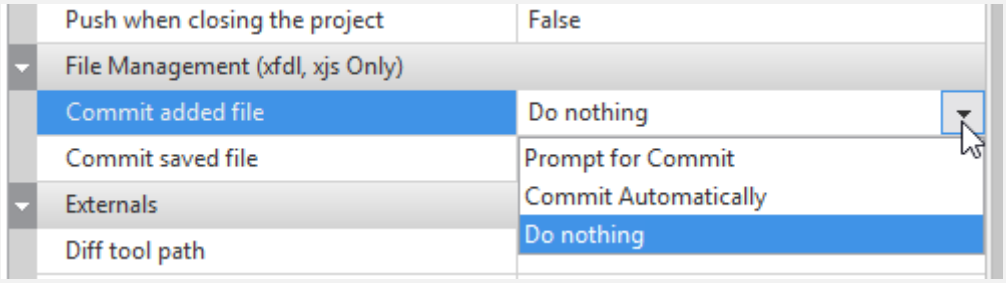
The below table presents an explanation about the options provided by Nexacro Studio for the use of Git.

Option	Description
General (Required)	
Executable file path	Location of the executable of the Git client. If the *git.exe* file is installed in the default path, this option will be set automatically.
User Name	Name of the person who will use the Git client
User E-mail	Email account of the person who will use the Git client
Git Management	
Interact with generate path	Determine whether to manage the source code in the generate path using Git
Push after commit	Determine whether to perform the push operation automatically as soon as a commit is made. This option is available when a remote repository is connected.
Pull before push	Determine whether to perform the pull operation automatically as soon as the push operation is performed. This option is available when a remote repository is connected.
Project Management	
Pull when opening the project	With a remote repository connected, this option

Option	Description
General (Required)	
	determines whether to fetch updates as soon as the project is opened. (Pull)
Push when closing the project	With a remote repository connected, this option determines whether to push your changes to the remote server as soon as the project is closed. (Pull -> Push)
File Management (only for .xfdl and .xjs files)	
Commit added file	Determine whether to perform a commit when files are added
Commit saved file	Determine whether to perform a commit when files are saved
Externals	
Diff tool path	Location of the executable of a tool, which will be executed upon the *Show Diff* command You cannot use *Show Diff* without setting this option.
Merge tool path	Location of the executable of a tool, which will be executed upon the *Edit Conflicted* command You cannot use *Edit Conflicted* without setting this option.
Merge tool arguments	This option allows you to enter the arguments applied to the merge tool.

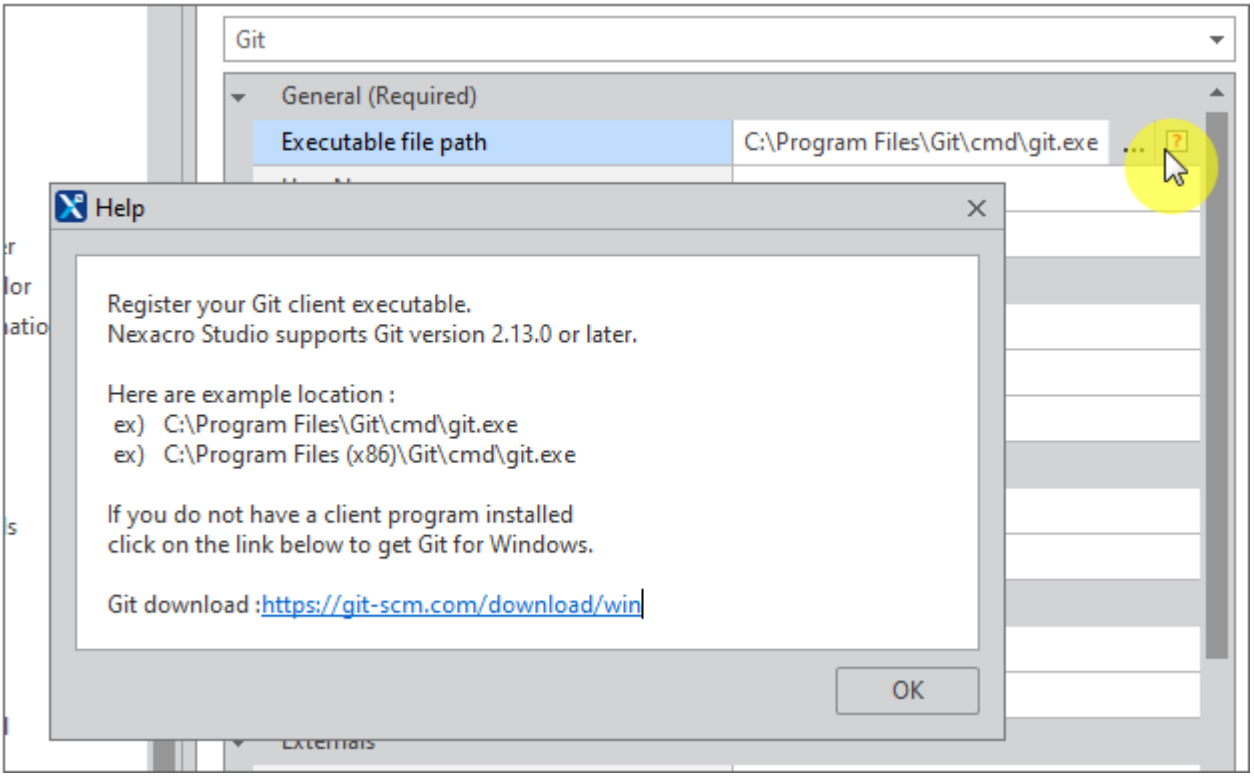


You can select one of the three choices for the below two options.
Commit added file
Commit saved file



Prompt for Commit: a commit can be performed after you check the changes
Commit Automatically: a commit will be performed automatically
Do nothing: the option will not be performed

You can see the necessary information for installing a Git client by clicking the question mark icon ([?]) on the right side of the *Executable file path* option.

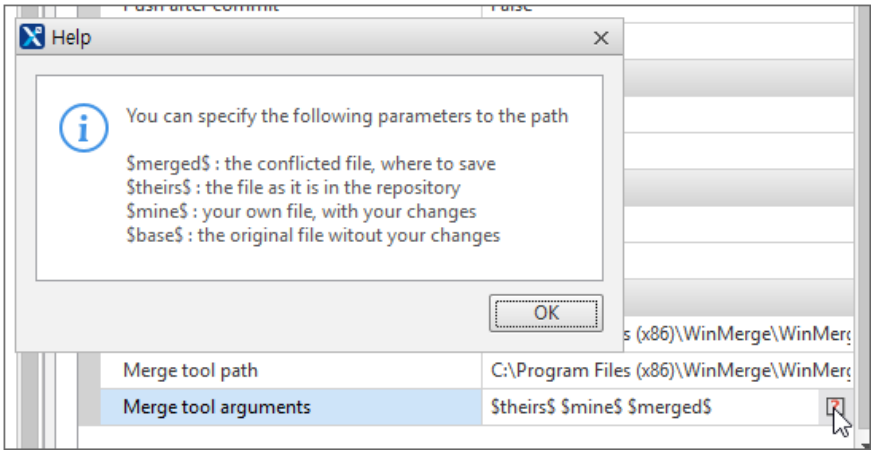


You should install a Git client whose version is 2.13.0 or later.

If you need further information on the installation of a Git client, click the below link.
https://git-scm.com/downloads

Define merge tool arguments

If you designate a merge tool used for the *Edit Conflicted* command, you should enter the arguments appropriate to each tool. You can see the information on the related placeholders by clicking the question mark icon ([?]) on the right side of the *Merge tool arguments* option.



The below table presents sets of arguments used for popular merge tools.

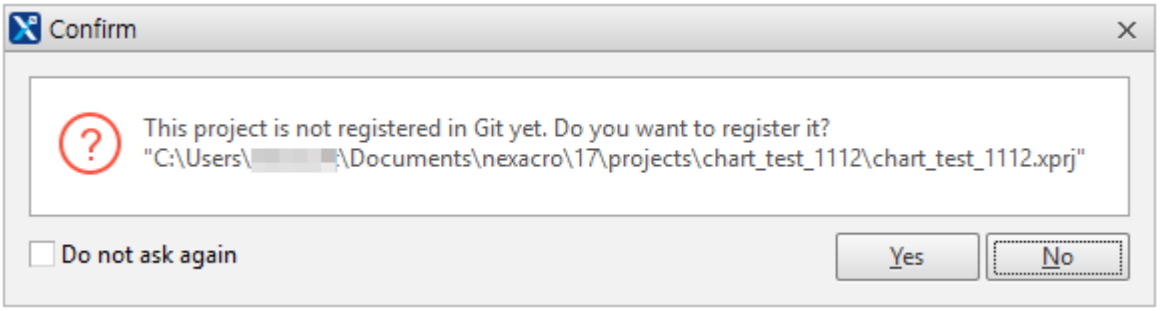
> The patterns presented in the below table are just examples, and they can vary according to the versions of each tool.

Merge tool	Arguments
WinMergeU.exe	$theirs$ $mine$ $merged$
TortoiseMerge.exe	/base:$base$ /their:$their$ /mine:$mine$ merged:$merged$
CodeMerge.exe	/tf=$theirs$ /mf=$mine$ /rf=$merged$ /bf=$base$
Meld.exe	$theirs$ $mine$ $merged$
p4Merge.exe	$base$ $theirs$ $mine$ $merged$
sgdm.exe	/t1="Mine" /t2="Base" /t3="Their" /r=$merged$ $mine$ $base$ $theirs$
kdiff3.exe	$base$ $mine$ $theirs$ -o $merged$ --L1 Base --L2 Mine --L3 Theirs

22.2.2 Start Project

Add a project to local repository

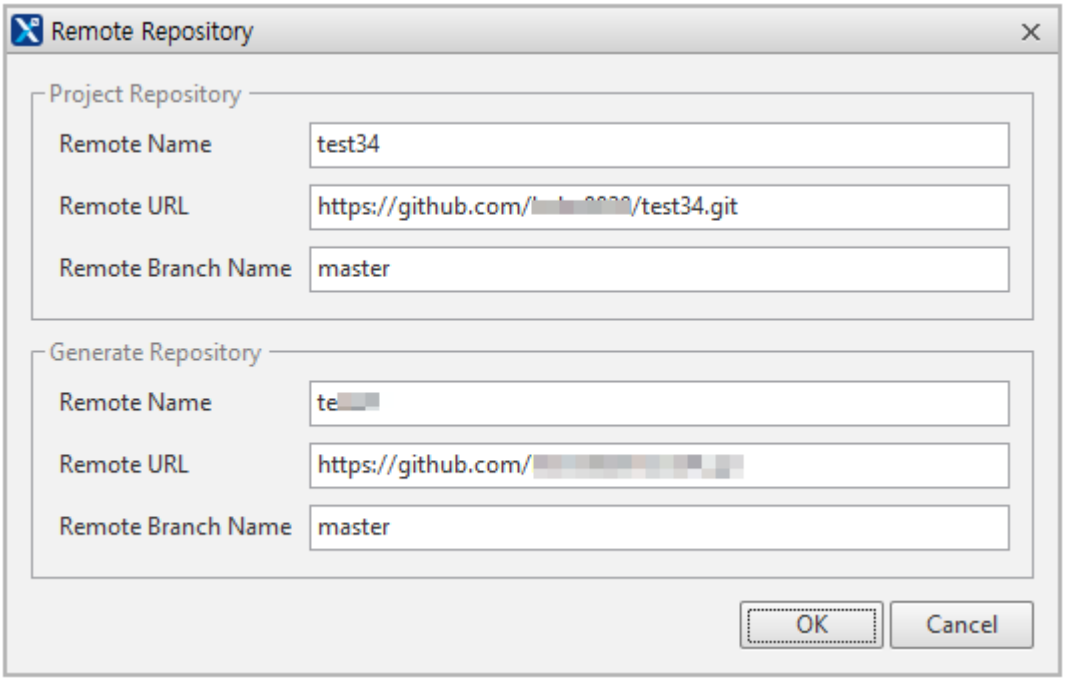
If you open a project that is not registered in your local repository, Nexacro Studio will ask you whether to add the project to the local repository. If you click Yes, the project will be added. Even if you click No, you can add the project later by selecting the menu [Git > Init New Repository].



Once a project is registered, the Git local repository will be initialized and then files will be added and committed.

Add a project to remote repository

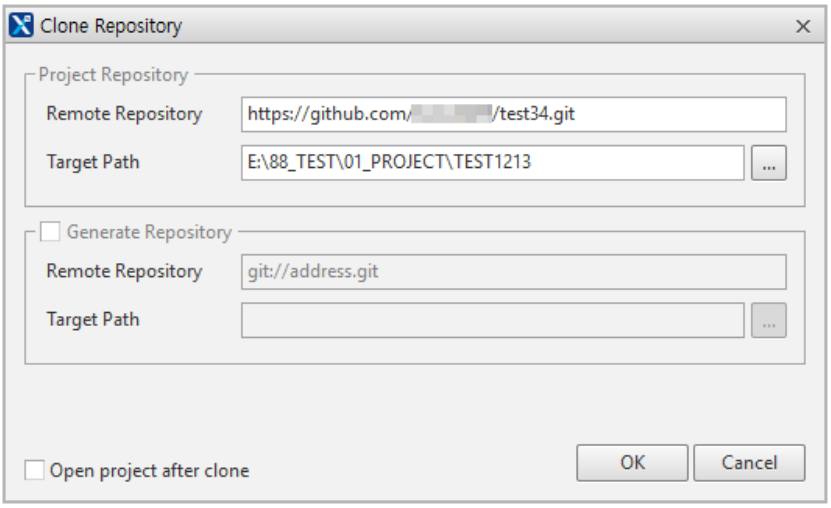
You can define the settings for the connection with a remote repository by accessing the menu [Git > Set Remote Repository]. If you set the *Interact with generate path* option to "true", you can designate a generate repository.



	Description
Remote Name	This is the name of the remote repository. The default value is "origin".
Remote URL	This is the location of the remote repository. You can use four protocol types: Local, HTTP, SSH and Git.
Remote Branch Name	This is the name of a branch in the remote repository. Changes made in the local repository will be pushed to the foregoing branch. The default value is "master".

Clone a project into local repository

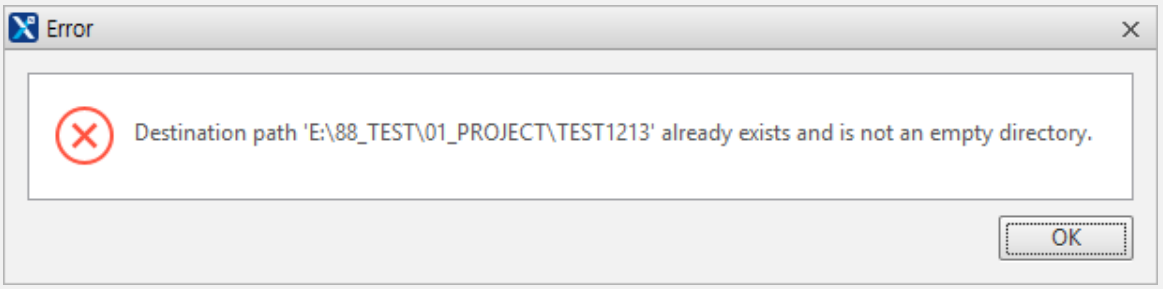
Access the menu [Git > Clone Repository] to start a project by downloading a project registered in the remote repository into your local repository.



	Description
Remote Repository	This is the location of the remote repository.
Target Path	This is the location in the local repository into which a project will be cloned.
Open project after clone	This option determines whether to open a downloaded project automatically as soon as the *Clone* command is performed if the project contains its project file (*.xprj*).

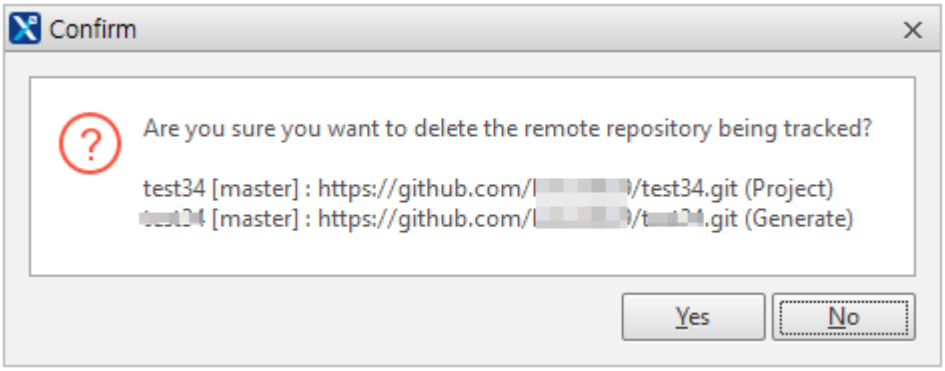
Once *Clone* is performed, the remote repository specified as the value of the Remote Repository option will be set as the remote repository for the downloaded project automatically.

Only an empty directory can be set as the value of the Target Path option. If the designated directory already contains a file or sub-directory, the *Clone* command will not be performed.



Delete data from remote repository

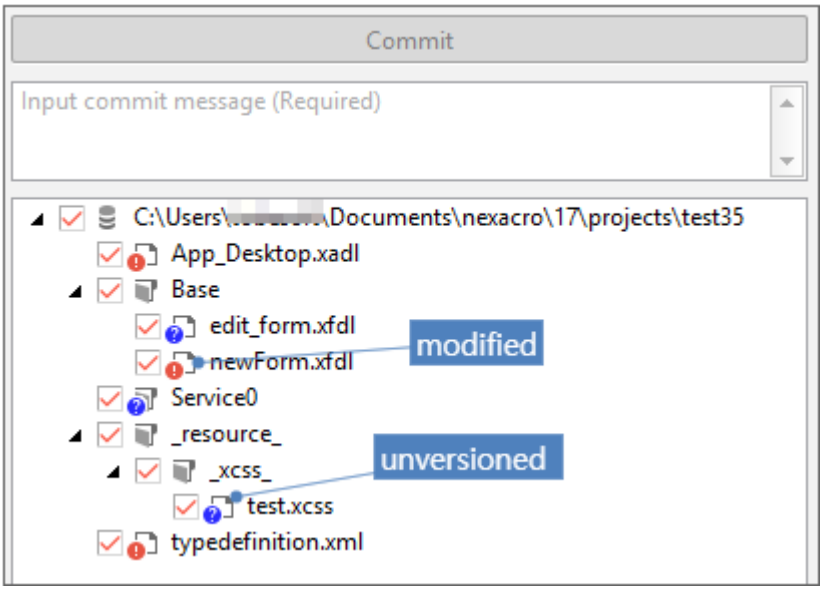
Access the menu [Git > Unset Remote Repository] to delete remote repositories that are tracked by the corresponding local repositories.



22.2.3 Apply Changes

Add files or change existing files

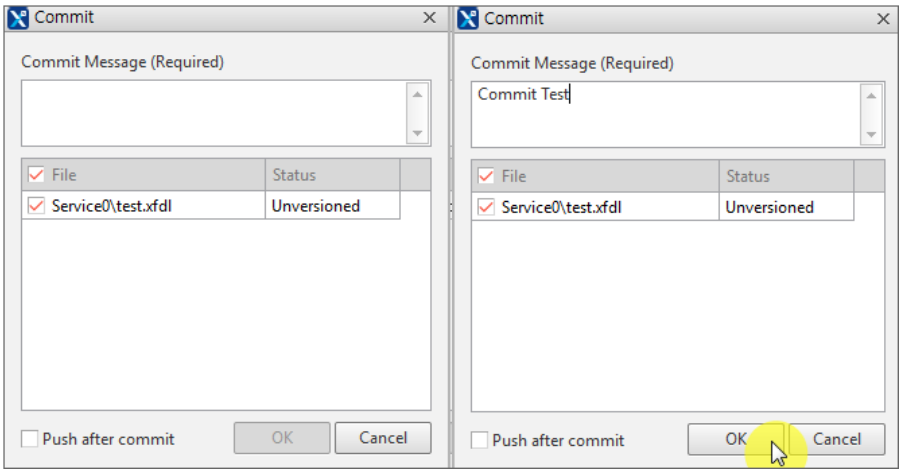
A file will be marked as "unversioned" in the Project Explorer and Resource Explorer if it is added to a project. If you change an existing file, it will be marked as "modified". The Git Explorer shows only added or modified files as well as their parent directories. The Git Explorer also lets you check the statuses of those files.



Commit changes to local repository

If you want to commit changes through the Project Explorer or Resource Explorer, select a relevant file in either explorer and then open the context menu to choose [Git > Commit]. In the Git Explorer, select a relevant file and then click the Commit button.

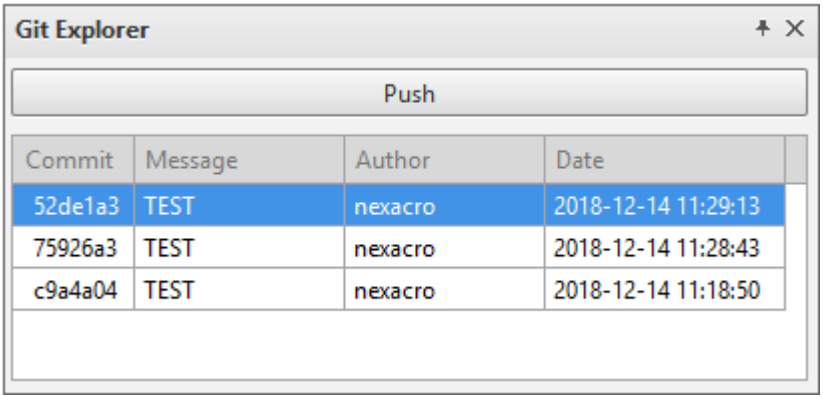
If you do not write a commit message, you cannot proceed with the commit because the OK button will not be activated. Therefore, write a message explaining the changes you made.



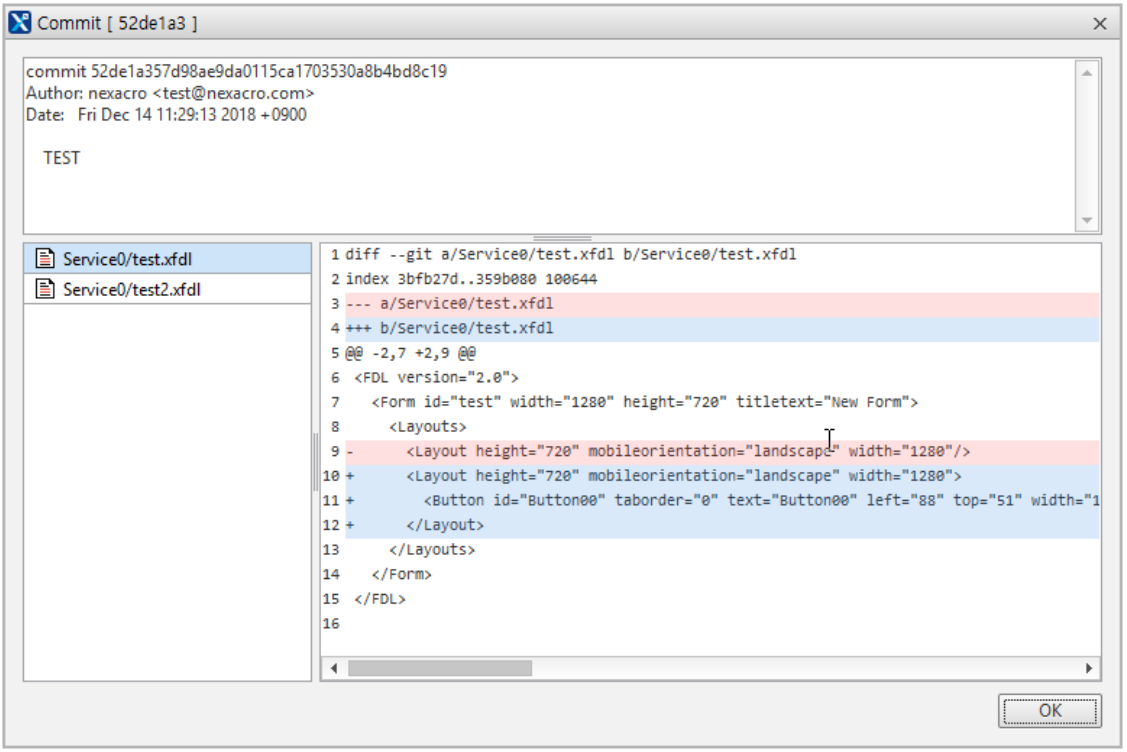
If you click the checkbox next to **Push after commit**, changes will be pushed to the remote repository automatically as soon as you make the relevant commit. This option is available when a remote repository is connected. Once you click the checkbox, the related option located in [Tools > Options > Source Control > Git] will be automatically changed accordingly.

Push changes to remote repository

When a remote repository is connected, you can push the commits saved in your local repository by performing the push operation. To that end, go to the menu [Git > Push] or click the Push button in the Git Explorer.



You can see the details on changes by double-clicking a certain commit in the Git Explorer or selecting the menu [Git > Push].

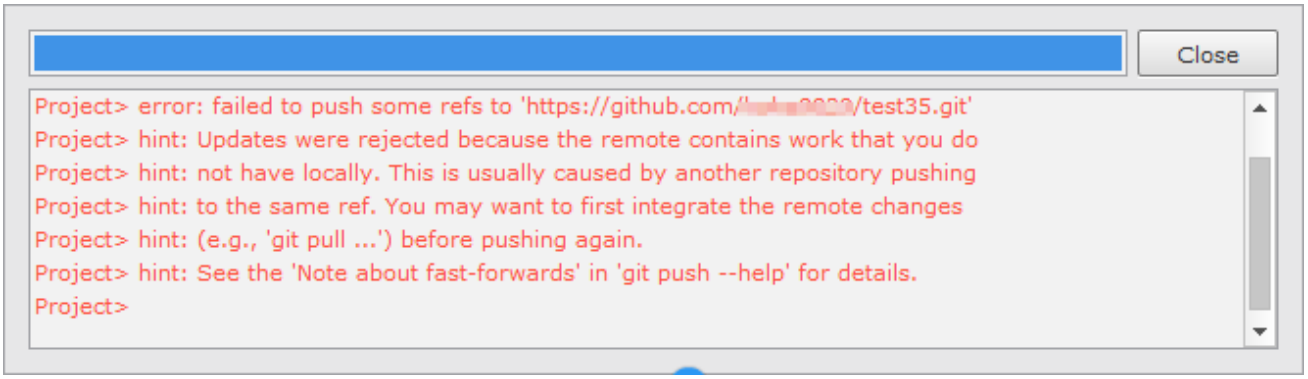


22.2.4 Resolve Conflicts

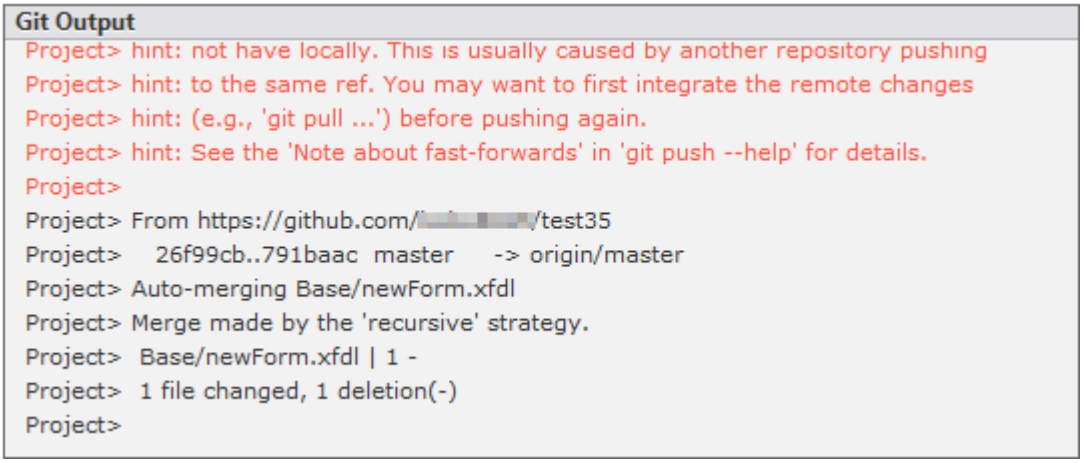
A conflict can occur if you and another user work on the same file. If a conflict occurs in a file, the file will be marked as "unmerged", and you can modify the source code by opening the file. If you set the *Merge tool path* option, you can edit the source code with a merge tool by selecting the menu [Git > Edit Conflicted].

When automatic merge is performed

If a local repository and the connected remote repository have different versions of a file, the below-mentioned messages may be presented after commits are pushed from the local repository. In this case, the pull operation is performed first and then the push operation follows.

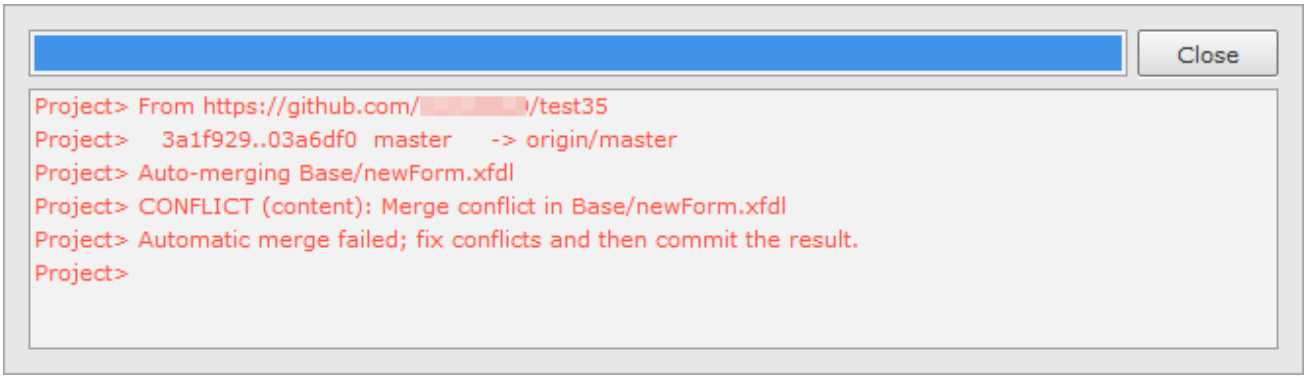


If an automatic merge is successful after a pull operation, the below-mentioned messages will be printed on the Git Output pane. If the merged file is already opened, the file will be reloaded.

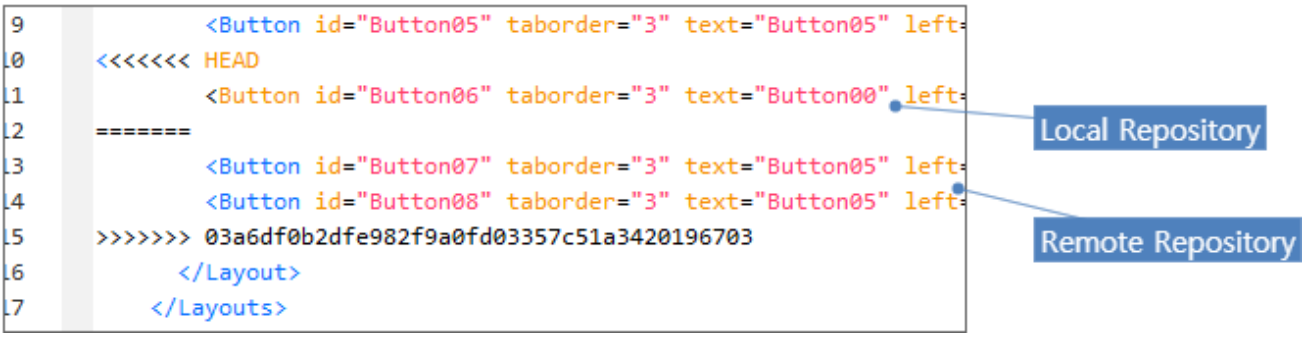


When conflict occurs

If an automatic merge fails after a pull operation, the below-mentioned messages will be printed, providing details on the conflict that occurred in the file.



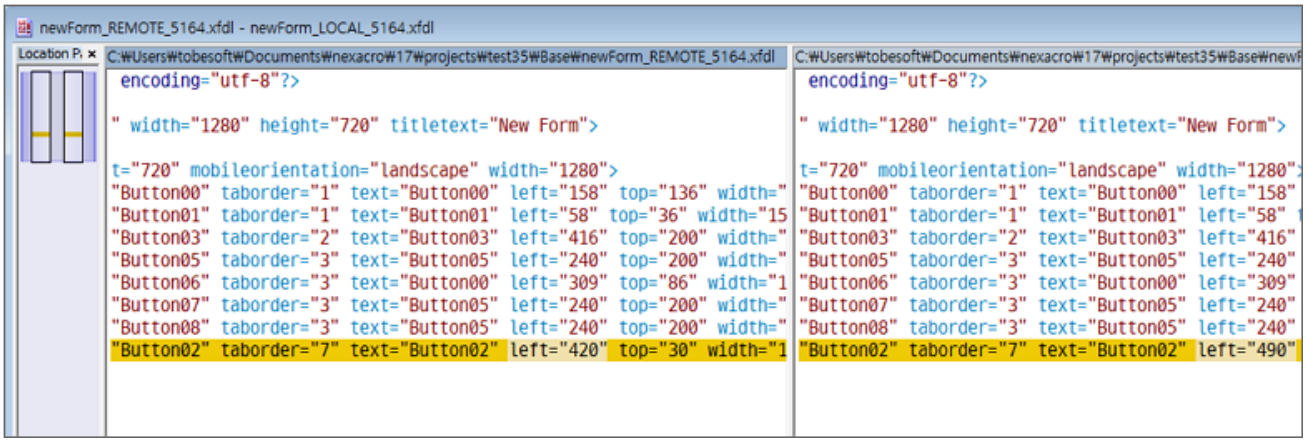
The file with the conflict will be opened with the XML editor of Nexacro Studio. In the below picture, the code written under "HEAD" had been committed in the local repository before the execution of the pull operation. The code written under "=======" is brought from the remote repository. You can resolve the conflict by checking the difference between the two sets of code and making proper changes.



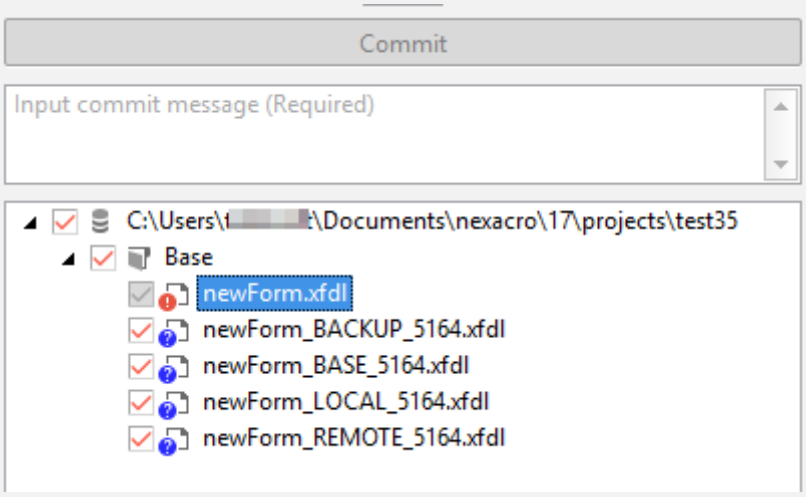
After resolving the conflict, you should perform the commit and push operations again.

Resolve conflicts with merge tool

While the XML editor of Nexacro Studio is good enough to resolve simple conflicts, it will be much better to use an external merge tool for complicated conflicts.



With a merge tool in use, your Git client creates temporary files, which are supposed to be deleted automatically when the relevant conflict is resolved and resulting changes are saved. However, such temporary files may fail to be removed from time to time. Since those temporary files are controlled by a Git client, Nexacro Studio will not delete those file arbitrarily. In this case, therefore, you should remove the temporary files by yourself if required.

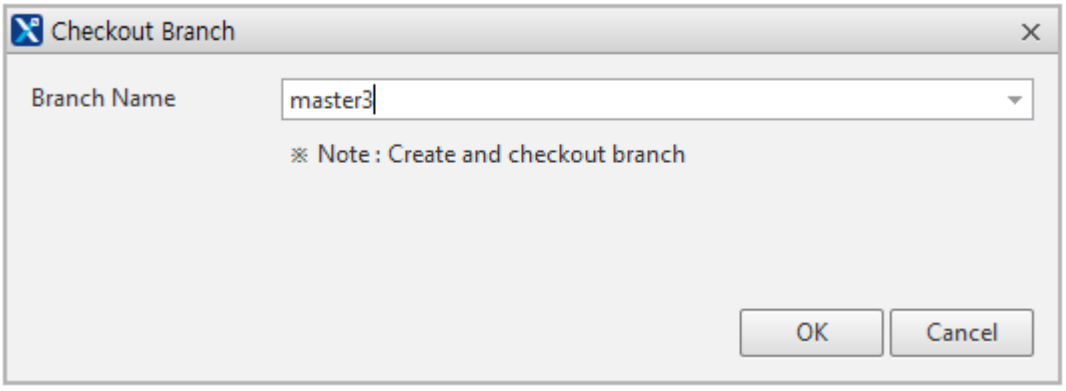


22.2.5 Create or Switch Branch

By accessing the menu [Git > Checkout Branch], you can switch to another branch or create a new branch.

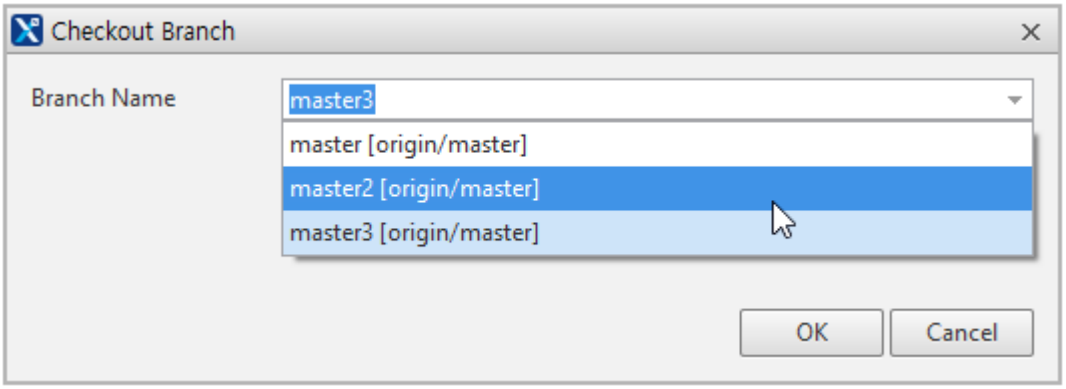
Create branch

If you enter a branch name that has not existed before, a new branch will be created.



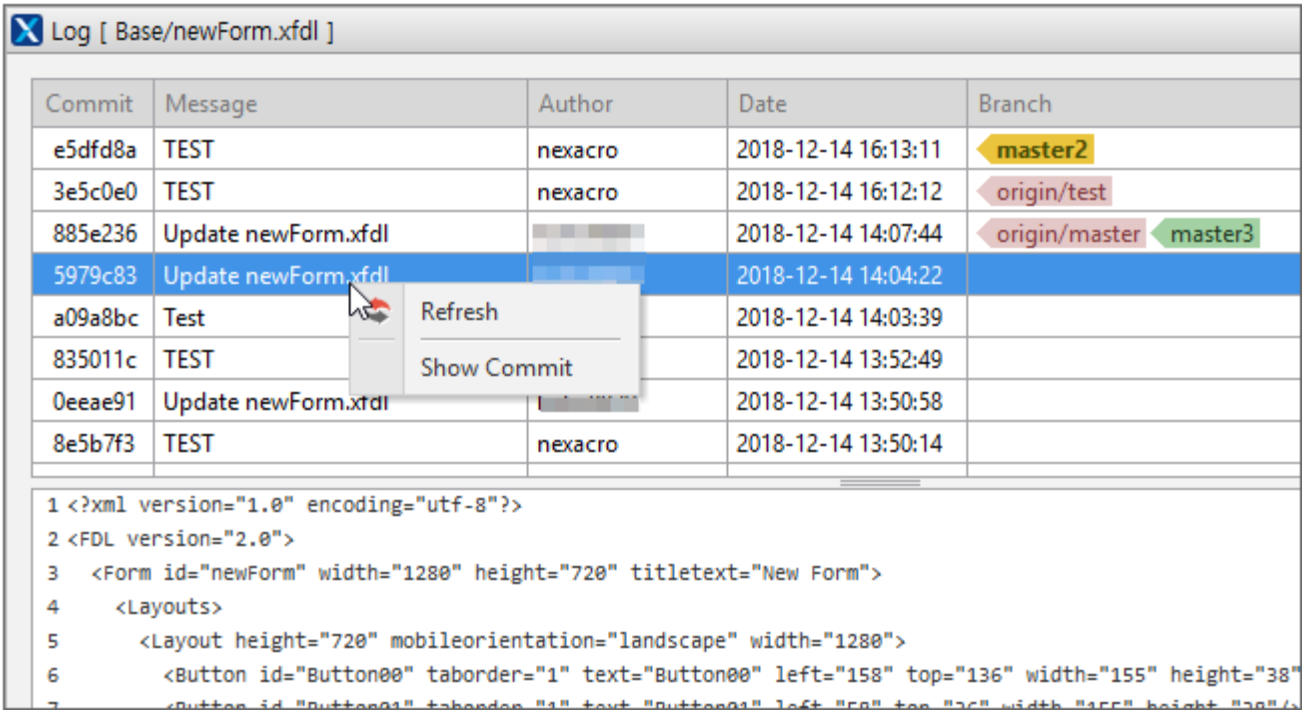
Switch to another branch

You can switch to another branch by selecting the relevant item from the branch list.



Check branches

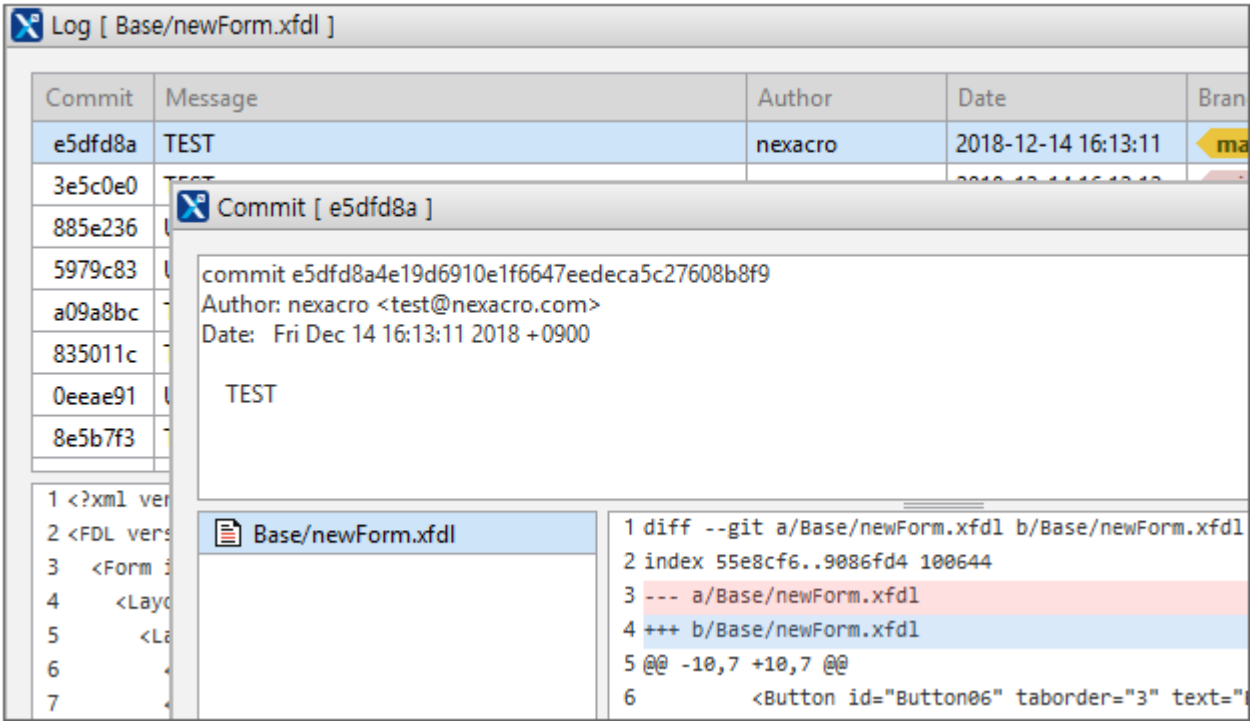
By accessing the menu [Git > Show Project Log] or [Git > Show File Log], you can see the information on branches, classified according to commits or files.



	Description
Commit	Commit SHA-1 checksum
Message	Commit message
Author	Person who worked on the commit
Date	Time when the commit was made
Branch	Information on the branches to which the commit is related.

	Description
	Yellow: branch that you are working on currently Green: branch in a local repository Pink: branch in a remote repository

A window showing the information on a commit will be opened if you double-click the item or select Show Commit from the context menu.



22.2.6 Output Pane / Status

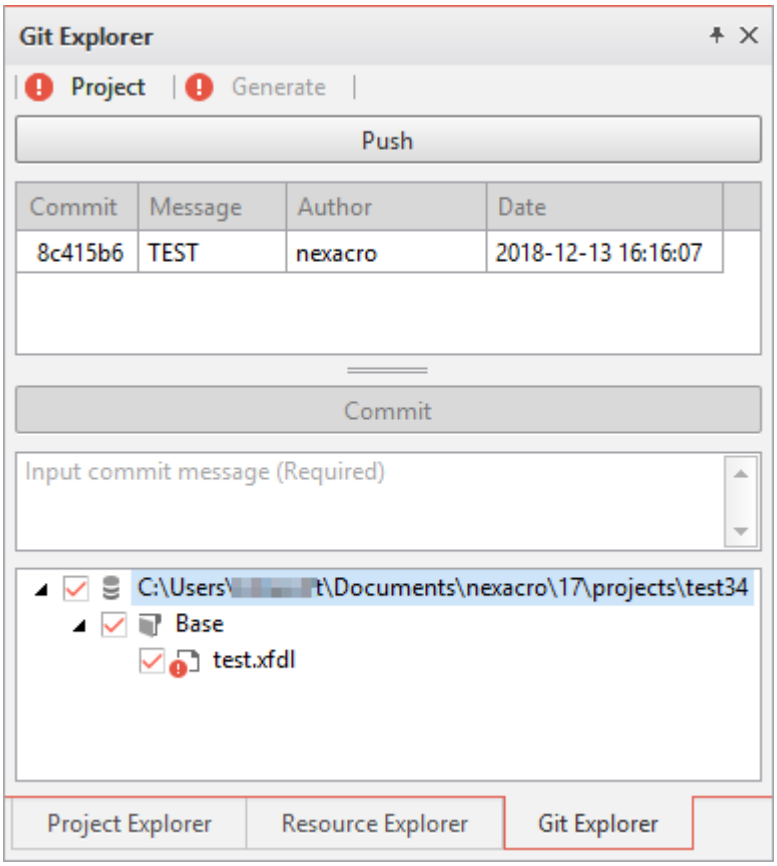
Git Output

This pane shows the results of Git performing tasks. If an error occurs during operation, the relevant message will be displayed in color.



Git Explorer

The Git Explorer allows you to see the changes made for a current project and perform necessary tasks. The explorer is largely divided into the Push and Commit sections. If you set the *Interact with generate path* option to "True", two tabs will appear at the top, dividing the Project and Generate parts.



Push

The Git Explorer shows the commits that have not been pushed yet to a connected remote repository. Click the Push button to conduct the push operation.

Commit

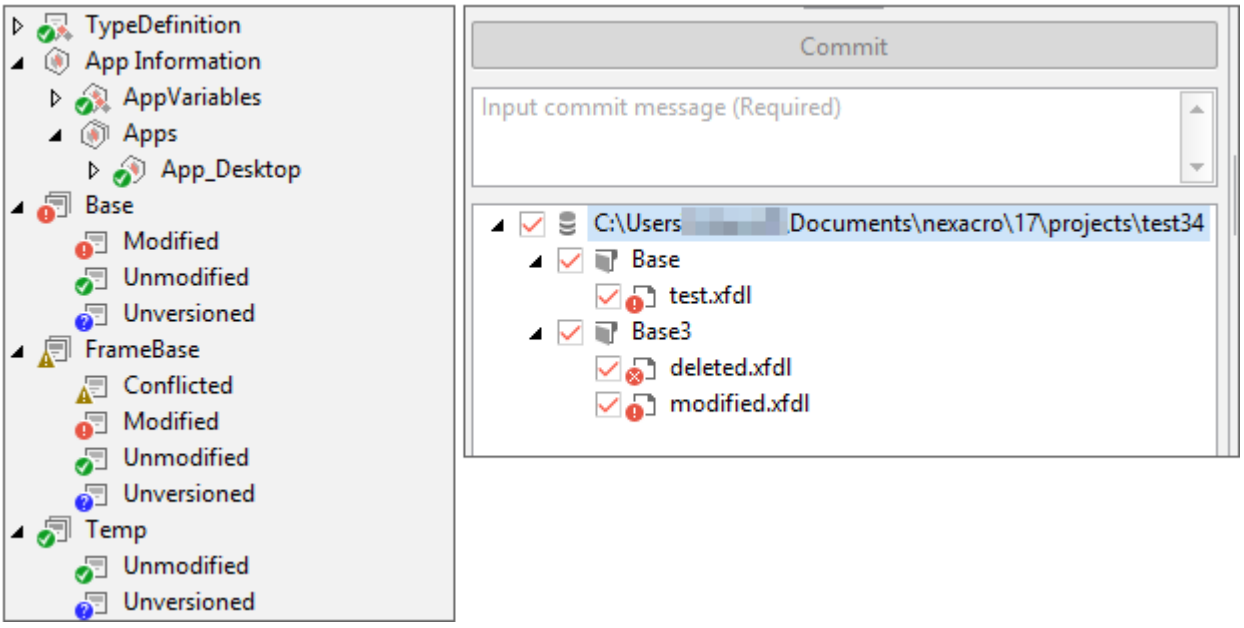
The Git Explorer shows the items that can be committed in a local repository. Click the Commit button after writing a commit message to perform the commit operation.

> An operation done in the Project division of the Git Explorer can be applied equally to the Generate division, just like you do the same task through the main menu. For example, if you make a commit in a project, the same commit will be made in the Generate repository. However,

the vice versa is not true: a task done in the Generate repository does not affect the other. Sometimes, you cannot control the files in the Generate division from the Project Division because the files on both sides are in different states. In this case, you should go to the Generate division of the Git Explorer and control files by yourself.

Git Status

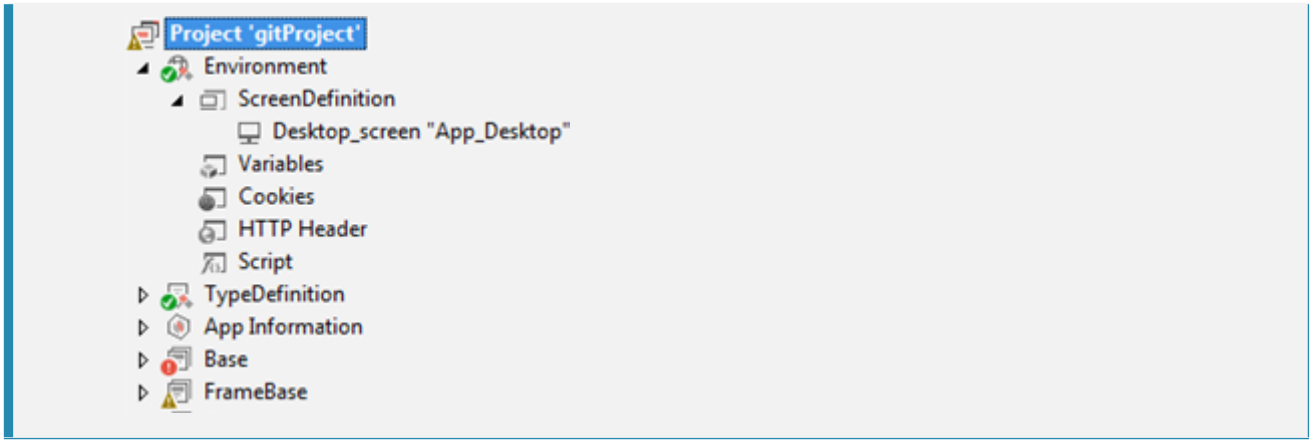
Icons indicating statuses appear in the Project Explorer or Git Explorer.



Since directories do not have their status values in Git, an icon for a directory is determined by the priorities among the statuses of children. In the above picture, the FrameBase Directory presents the Conflicted icon, resulting from the priority rules applied among the child Forms. The order of priority for status icons is presented below.

```
Conflicted > Modified > Added > Unmodified > Unversioned
```

While a project name is managed through the relevant *.xprj* file, the name also serves as the name of the top directory in the Project Explorer. Therefore, the icon beside a project name is influence by the statuses of the children rather than the status of the relevant *.xprj* file.


Project 'gitProject'
Environment
ScreenDefinition
Desktop_screen "App_Desktop"
Variables
Cookies
HTTP Header
Script
TypeDefinition
App Information
Base
FrameBase

23.

Setting Options

This chapter describes the environment options you can change to enhance your experience using Nexacro Studio.

From the **[Tools > Option]** Menu, you can configure the Nexacro Studio development environment.



When you click the 'Set Defaults' button at the bottom of the Options window, you can select one of two options.

- Set options on this page as default: Resets only currently open options to default values.
- Set all options as default: Reset all options to default values.

When the 'Set Defaults' button is clicked, some items added by the user as shown in the list below are not initialized and the added values are maintained.* Project

  - Generate > Event > Command to execute before generate
  - Generate > Event > Command to execute after generate
  - Deploy > Compress > Ignore compress files
  - Macros > User Macros
- Environment
  - External Tools > External Tool
  - Source Control > Source Control Management Type

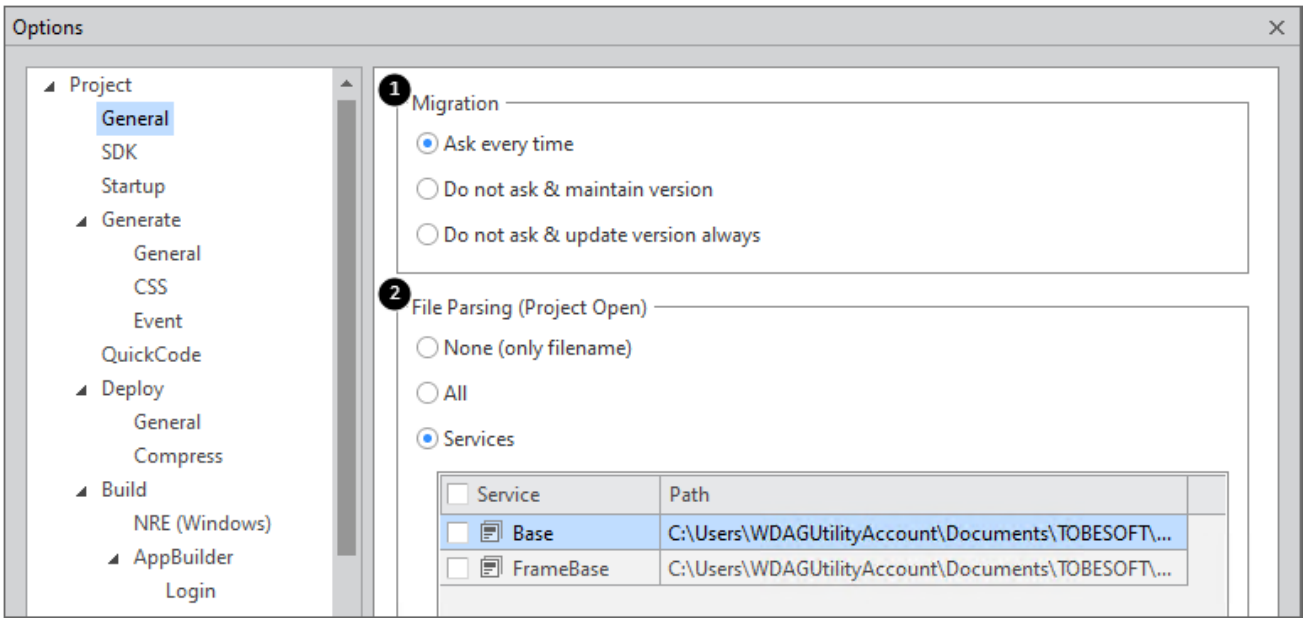
23.1 Options – Project



Project-related options are visible only while a project is opened. The option values are managed separately according to projects.

23.1.1 General

You can set up general options for managing a project.

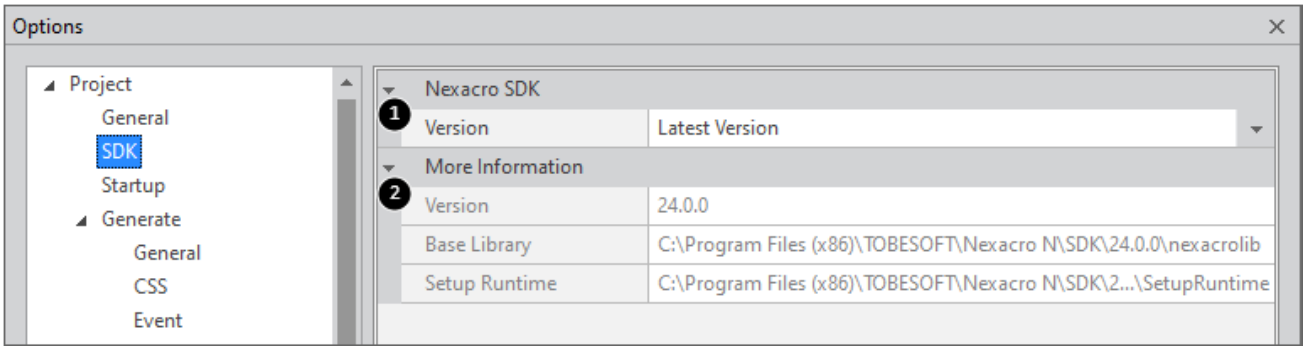


	Option	Description
1	Migration	Sets the migration option for file versions lower than Nexacro Studio can support • Ask everytime: Asks you every time whether to migrate files • Do not ask & maintain version: Not conducts migration without asking you • Do not ask & update version always: Conducts migration without asking you
2	File Parsing	Determines what types of file contents are displayed in the Project Explorer by parsing the components and scripts included in XFDL, XJS and JS files. This option is applied when a project is opened. • None (only filename): no file contents are displayed. • All: all file contents are displayed. • Services: the file contents of designated services are only displayed.

Loading time can be prolonged if you select "All" for the File Parsing option while the opened project includes a number of files.

You can display the file contents later after you select "None" for the File Parsing option. To this end, select a project or service in the Project Explorer and then select [Parsing Files] from the context menu.

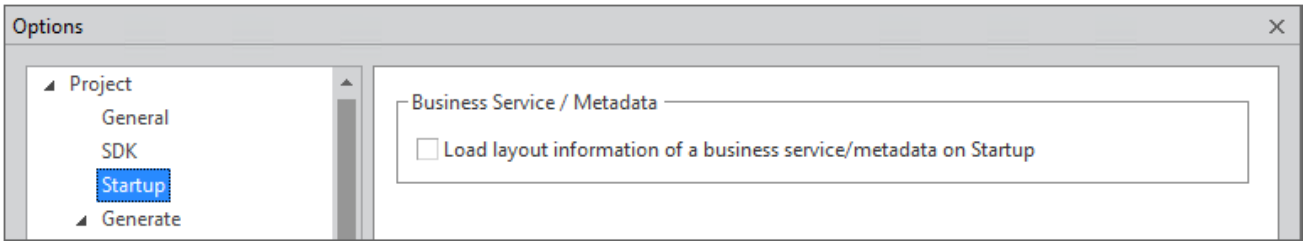
23.1.2 SDK



	Option	Description
1	Version	Select the SDK version to use in your project. - Latest Version: Use the latest version of the SDK in the SDK folder. - [Version]: Use the selected SDK from the installed SDKs. - User Setting: Enter the information yourself.
2	More Information	Select ""User Setting"" under Version to edit the information. - Version: Select from the list of installed SDKs. - Base Library: Set the Base Library folder for the selected SDK. - Setup Runtime: Set the Setup Runtime folder for the selected SDK.

23.1.3 Startup

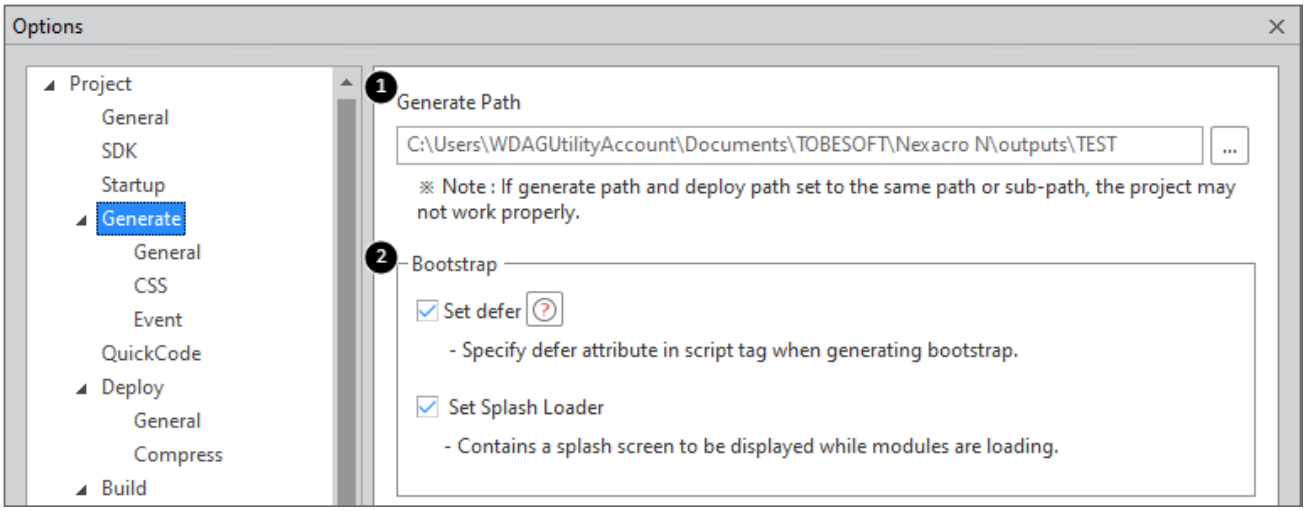
You can set the actions conducted at the time of loading a project.



	Menu	Description
1	Load layout information of a business service/metadata on Startup	Determines whether to load business services and metadata and to show them on the Project Explorer when Nexacro Studio begins.

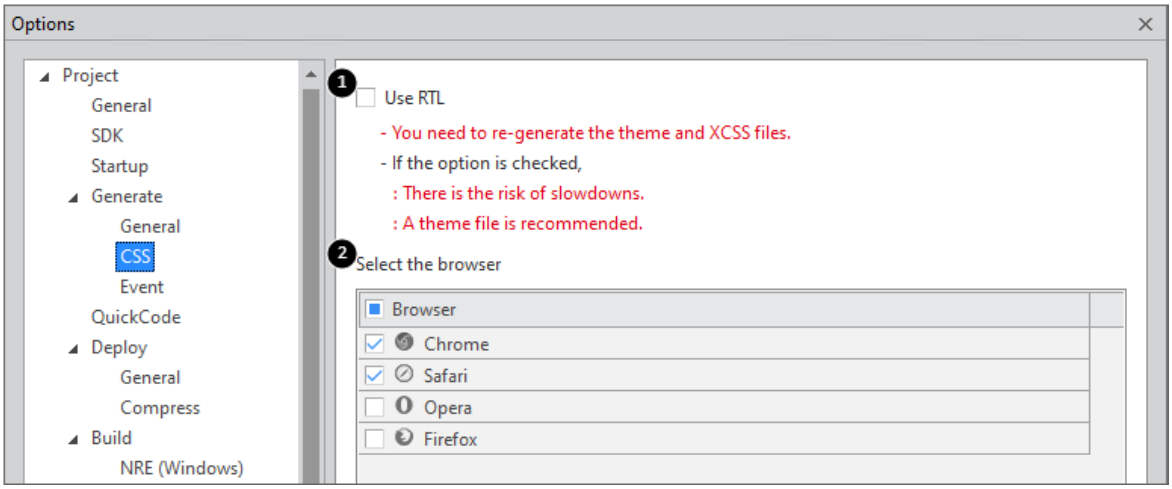
23.1.4 Generate

Gereral



		Description
1	Generate Path	Specifies the path for saving the files that are produced as the result of building a project. If the Generate path is equal to or located under the Deploy path, the project may not work properly.
2	Bootstrap	-Set defer Sets whether to include the defer attribute within the Script tag when creating the bootstrap file. - Set Splash Loader Sets whether to include a script to display the Splash Loader when creating the bootstrap file.

CSS



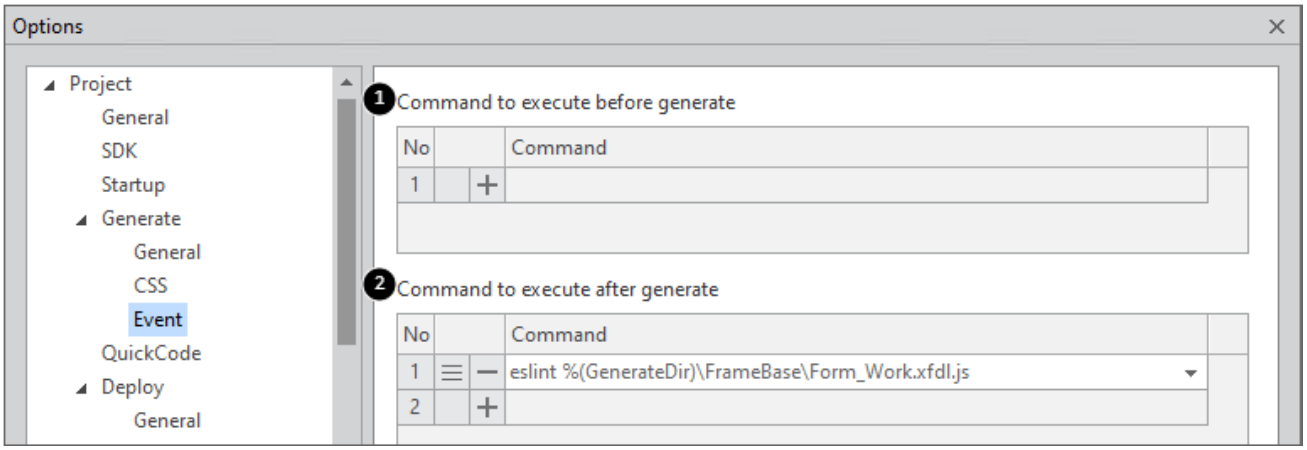
		Description
1	Use RTL	This option determines whether to generate CSS files for the use of the right-to-left (RTL) layout. You must select this option when you use an RTL-related property. If you select this option not from the beginning, you should re-generate theme and XCSS files by yourself.
2	Select the browser	Selects web browsers for which CSS files will be created. The CSS files for unselected browsers will not be created.

CSS files will not be created for the web browsers not selected in the Css Generate option. Moreover, there is no guarantee that the relevant app will work properly in such unselected browsers.
For example, your app may not work properly in Firefox if you do not create a CSS file for Firefox.

When the *Use RTL* option is selected, the sizes of the XCSS files in an app may get bigger because code for the RTL layout is added to all selectors in the process of generation. This increase in size is likely to affect the performance of the app negatively.
If XCSS code is included in a theme file, Nexacro can offer an optimization scheme for processing the repetitions contained in the XCSS code. If you implement RTL, therefore, you are recommended to use a theme file instead of applying XCSS files to the app separately.

Event

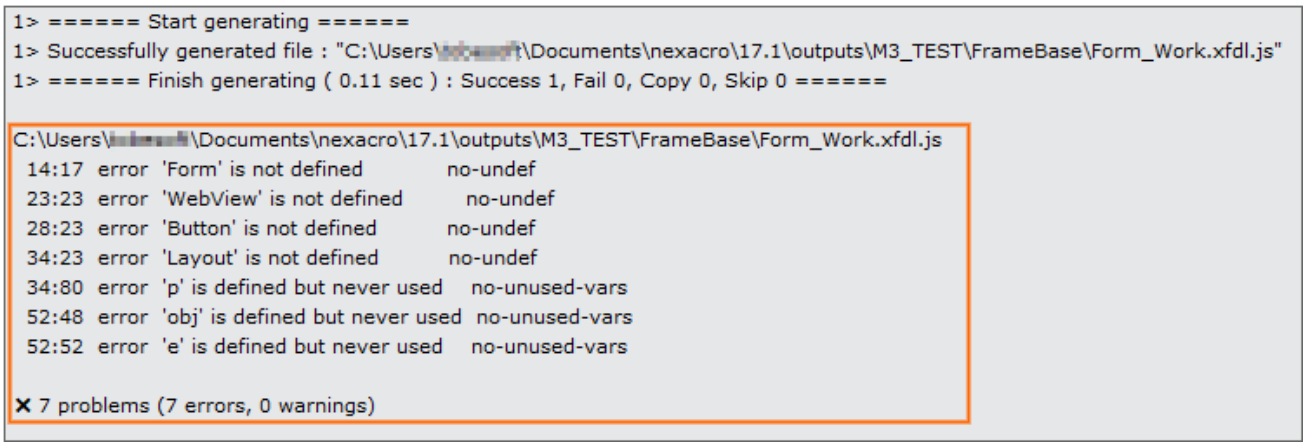
It executes the specified command before or after running Generate and then displays the results in the Output window.



	Property	Description
1	...before generate	Executes the specified command in the Command item before running Generate.

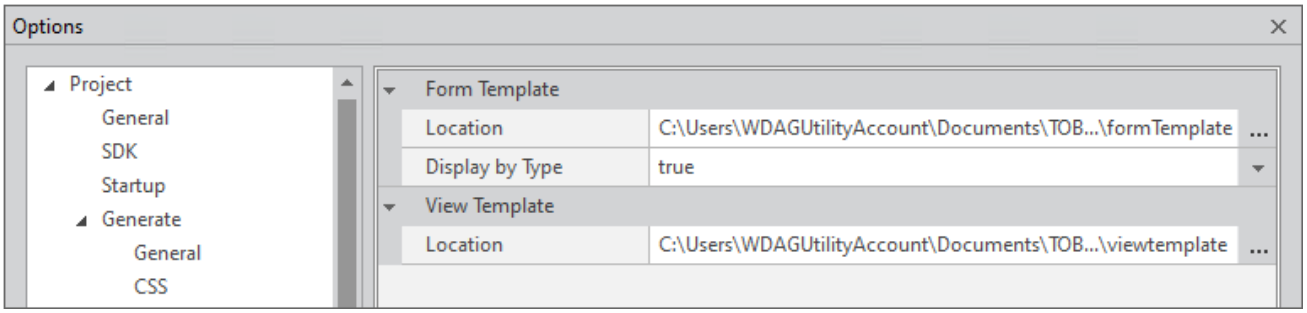
	Property	Description
2	...after generate	Executes the specified command in the Command item after Generate execution is completed.

If there is a value returned after executing the specified command, then it is displayed in the Output window.

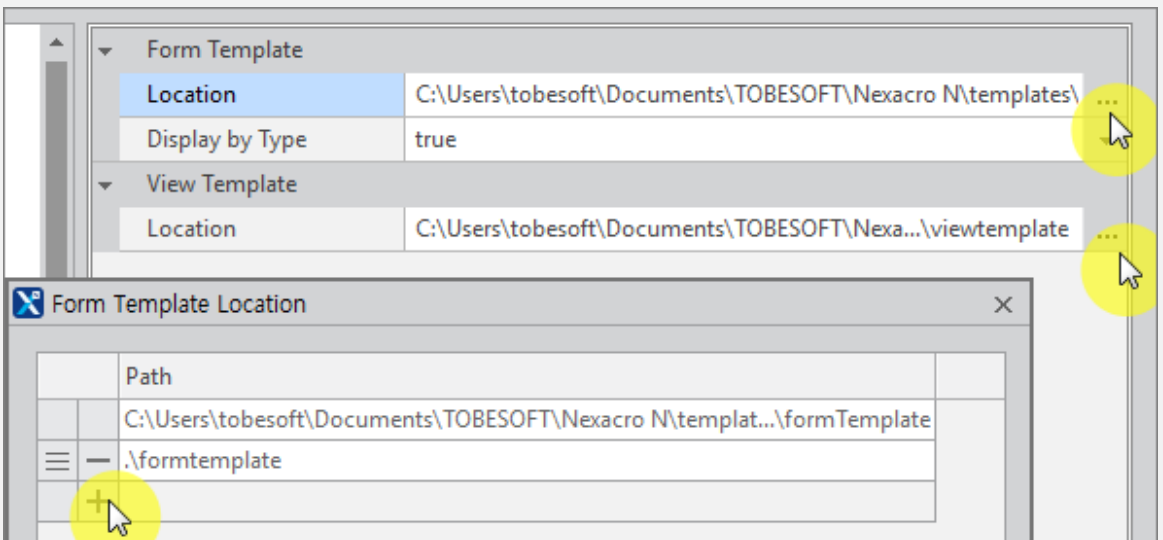


23.1.5 QuickCode

It sets the storage path for the files created when using the QuickCode function.



- You can modify the default path or manage two or more paths.
- When set to a folder within the project path, a relative path is used.

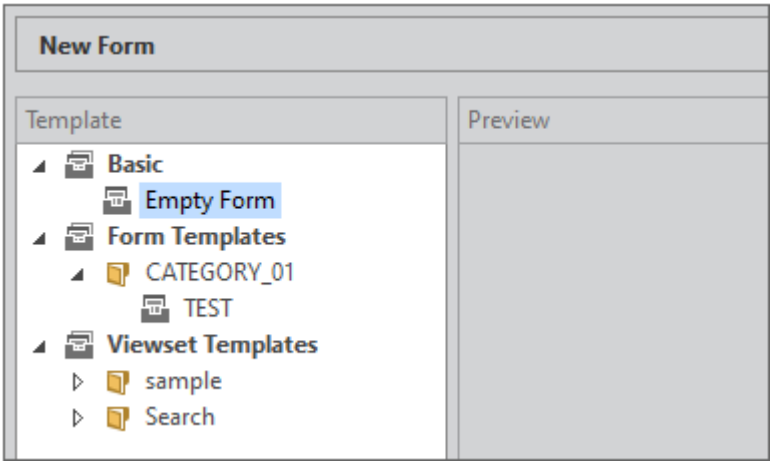


Form Template, Viewset Template

It sets the storage path for the Form template and the ViewSet template file. A folder with the template name is created, and the necessary files are created in the folder. For the ViewSet template, the default path is the viewset folder under the Form template path.

```
C:\Users\[user]\Documents\TOBESOFT\Nexacro N\templates\user\formTemplate
C:\Users\[user]\Documents\TOBESOFT\Nexacro
N\templates\user\formTemplate\viewset
```

The Display by Type item sets the format in which the Template is displayed in the Form Wizard. If it is the default value (true), then Form Templates and Viewset Templates are displayed separately, and if it is set to false, then they are displayed in the order of the folder name.



View Template

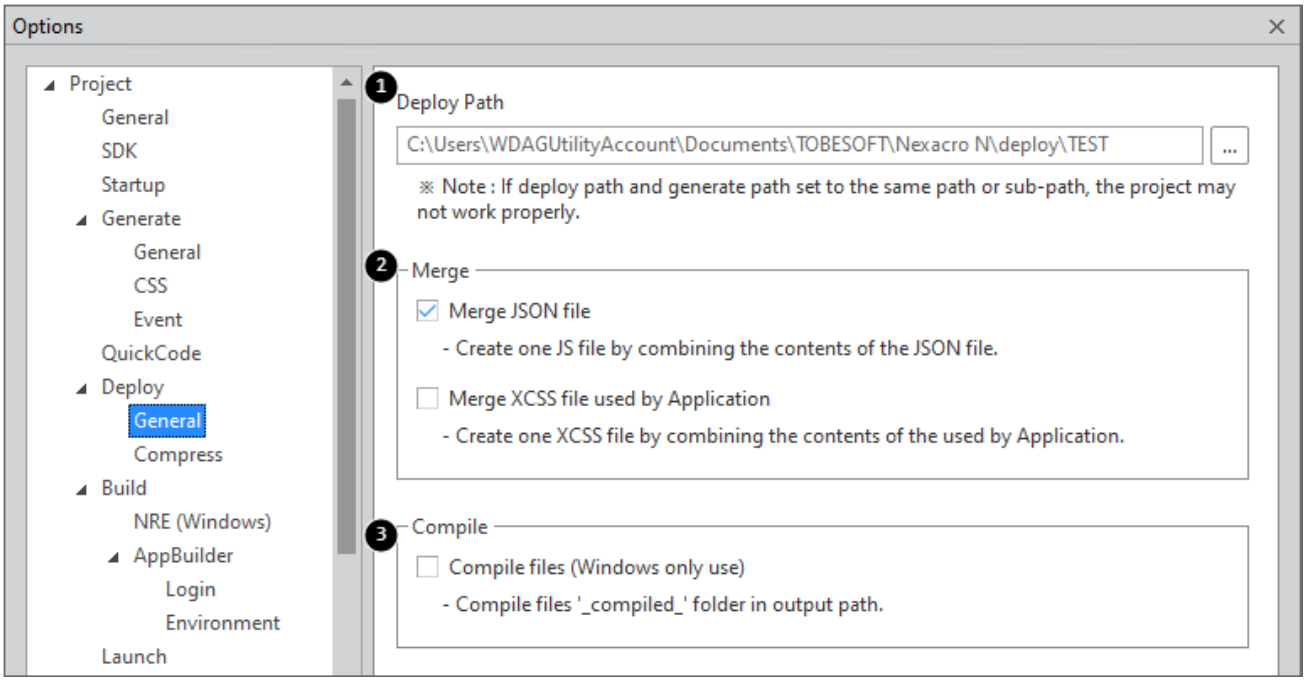
It sets the path where the View Template file (.xviewgen) is saved.

```
C:\Users\[user]\Documents\TOBESOFT\Nexacro N\templates\user\viewtemplate
```

23.1.6 Deploy

You can set options related to project deployment.

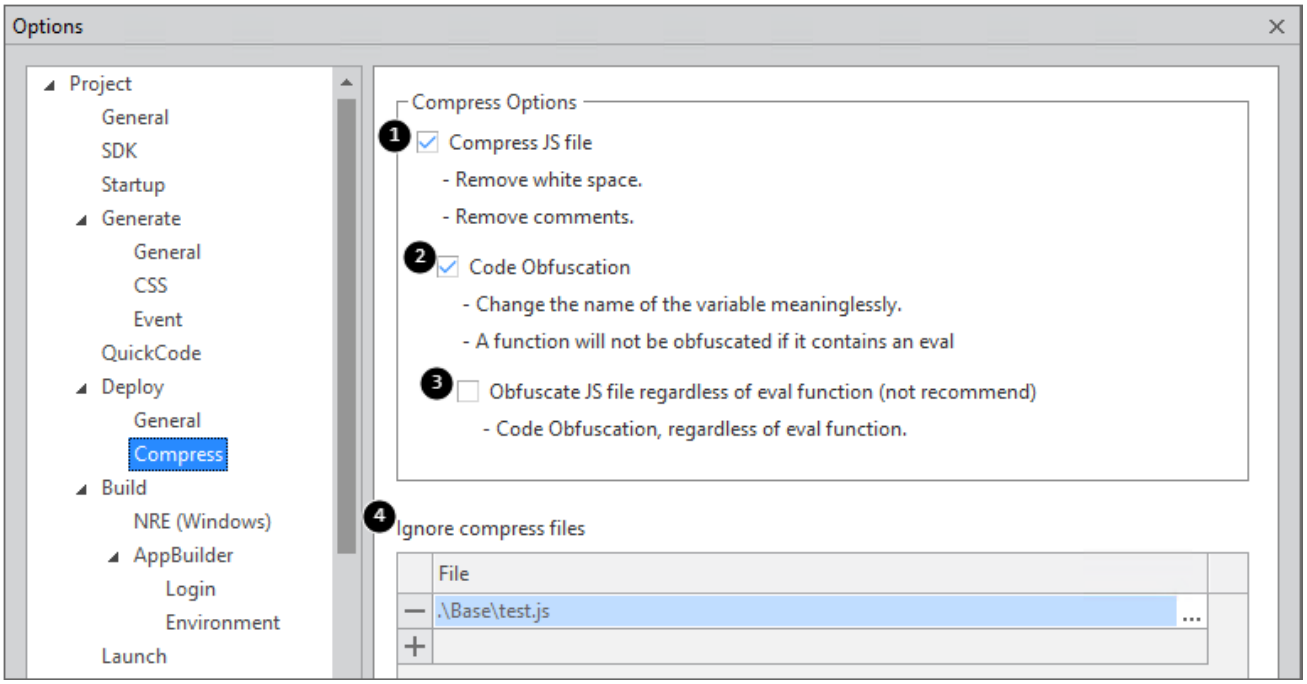
General



	Option	Description
1	Deploy Path	Designates the output path used in the Deploy menu.
2	Merge	- Merge JSON file Determines whether to create a single JavaScript file by combining the JavaScript files specified in the JSON file. - Merge XCSS file used by Application Determines whether to create a single XCSS file by combining the XCSS files used in an Application
3	Compile	Configures the settings for compiling the scripts of a project file to deploy

The selected options will apply to deployment-related wizards as default values and also to the process of the Auto Deploy feature.

Compress



	Option	Description
	Compress Options	Sets the options for compressing and obfuscating the scripts of a project file to deploy. The configured options will represent the default values for the deployment-related wizards and be applied to the Auto Deploy feature as well. ❶ removes whitespaces and comments ❷ obfuscates JavaScript files ❸ obfuscates JavaScript files regardless of eval functions (not recommended)
4	Ignore compress files	Specifies a list of files that will be excluded in the process of compression The list will be saved in the *.ignorecompress* file, which is located under the directory where the relevant XPRJ file is stored.

For the code created when Compress Options is applied, please refer to the process below. It is the script code written in the Form_Work.xfdl file.

Form_Work.xfdl

```
1  // Button01 Test
2  this.Button00_onclick = function
(obj:nexacro.Button,e:nexacro.ClickEventInfo)
3  {
4      var temp_btn = "TEST";
5      eval("trace('test'"+temp_btn+")");
6  };
7
```

```
/************************************************************************
 * Button Test
 ************************************************************************/
this.Button01_onclick = function
(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var temp_btn = "TEST";
    trace('TEST_2'+temp_btn);
};
```

The code below is the created JavaScript code when none of the Compress Options are selected.

Form_Work.xfdl.js

```
// Button01 Test
this.Button00_onclick = function(obj,e)
{
    var temp_btn = "TEST";
    eval("trace('test'"+temp_btn+")");
};


/************************************************************************
 * Button Test
 ************************************************************************/
this.Button01_onclick = function(obj,e)
{
    var temp_btn = "TEST";
    trace('TEST_2'+temp_btn);
};
```

The code below is the code with blank characters and comments removed by applying only the first option.

Form_Work.xfdl.js / Option 1

```
this.Button00_onclick=function(obj,e){var temp_btn="TEST";eval(
"trace('test'"+temp_btn+")");};this.Button01_onclick=function(obj,e){var
temp_btn="TEST";trace('TEST_2'+temp_btn);};
```

The code below is the code that processed obfuscation by applying up to the second option. The part with the eval function was not processed for obfuscation.

Form_Work.xfdl.js / Option 1, 2

```
this.Button00_onclick=function(obj,e){var temp_btn="TEST";eval(
"trace('test'"+temp_btn+")");};this.Button01_onclick=function(_a,_b){var _c=
"TEST";trace('TEST_2'+_c);};
```

The code below is the code with up to the third option applied. Regardless of the eval function, it processes obfuscation throughout the code.

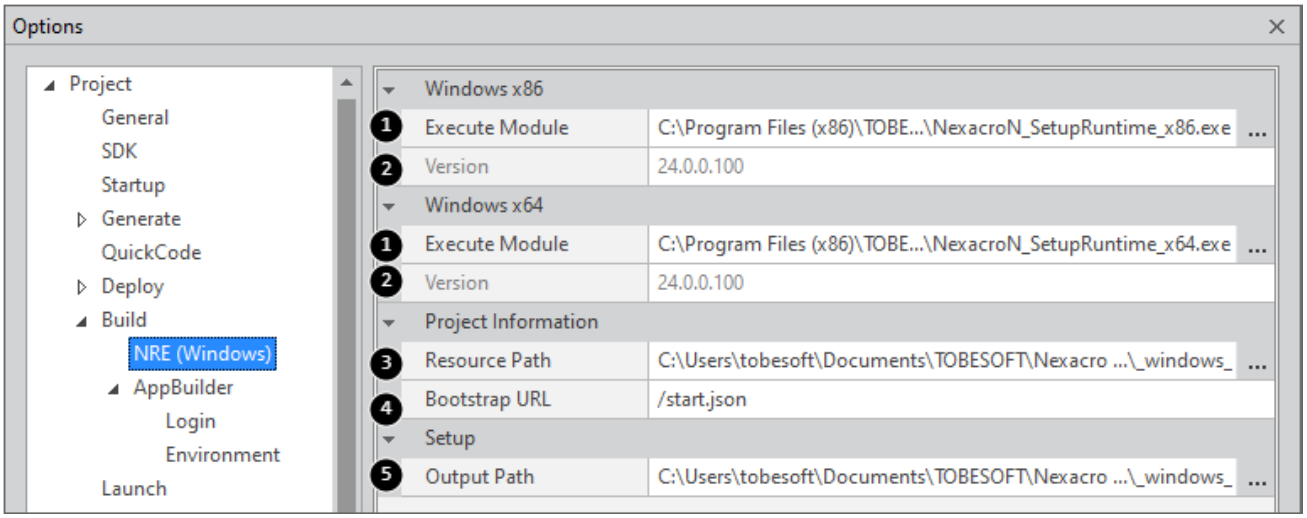
Form_Work.xfdl.js / Option 1, 2, 3

```
this.Button00_onclick=function(_a,_b){var _c="TEST";eval("trace('test'"+_c+
")");};this.Button01_onclick=function(_a,_b){var _c="TEST";trace('TEST_2'
+_c);};
```

23.1.7 Build

It sets the basic options required for the app build.

NRE (Windows)

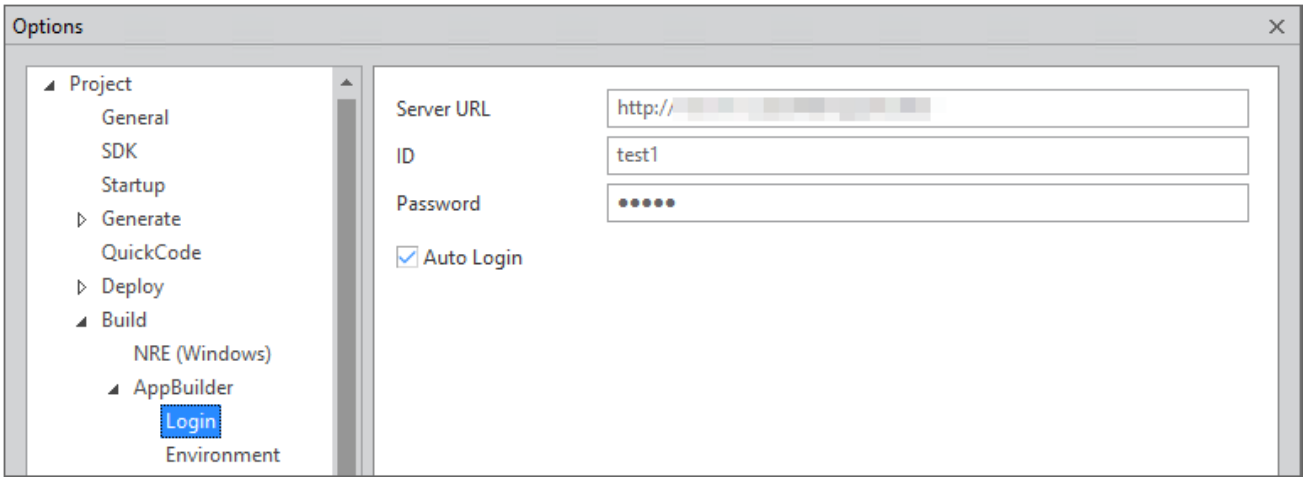


	Property	Description
1	Execute Module	This section is available when you choose the Windows environment. The Build App wizard locates the setup module created under the directory where Nexacro Studio is installed and then specifies the file as a default module. You can change the default module to another file if you want to or if the wizard cannot find the file.
2	Version	Displays the selected module version value.
3	Resource Path	Specifies the path to the archive files that were created in the previous stage.

	Property	Description
		If there is no archive file, the process will not proceed.
4	Bootstrap URL	Specifies the URL of a bootstrap file. If you select Local as the update type, you should specify the name of a bootstrap file. Otherwise, you should specify a URL.
5	Output path	Specifies the directory where an installation file will be created.

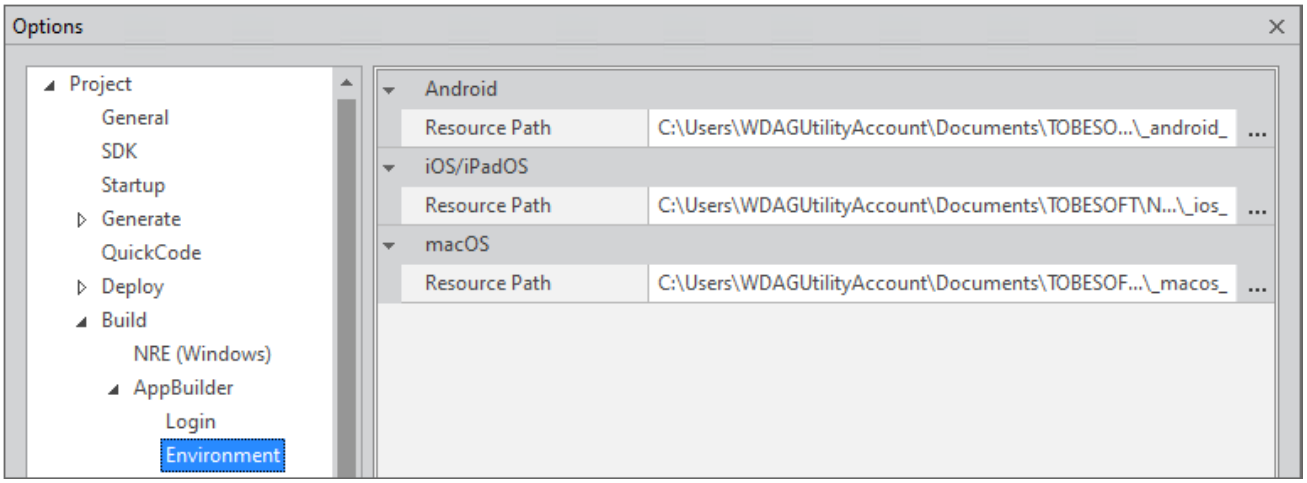
AppBuilder > Login

It can be entered when the Android, iOS, and macOS execution environments are selected. If "Auto Login" is set, then the login window does not appear when the menu [Deploy > BuildApp] is executed.



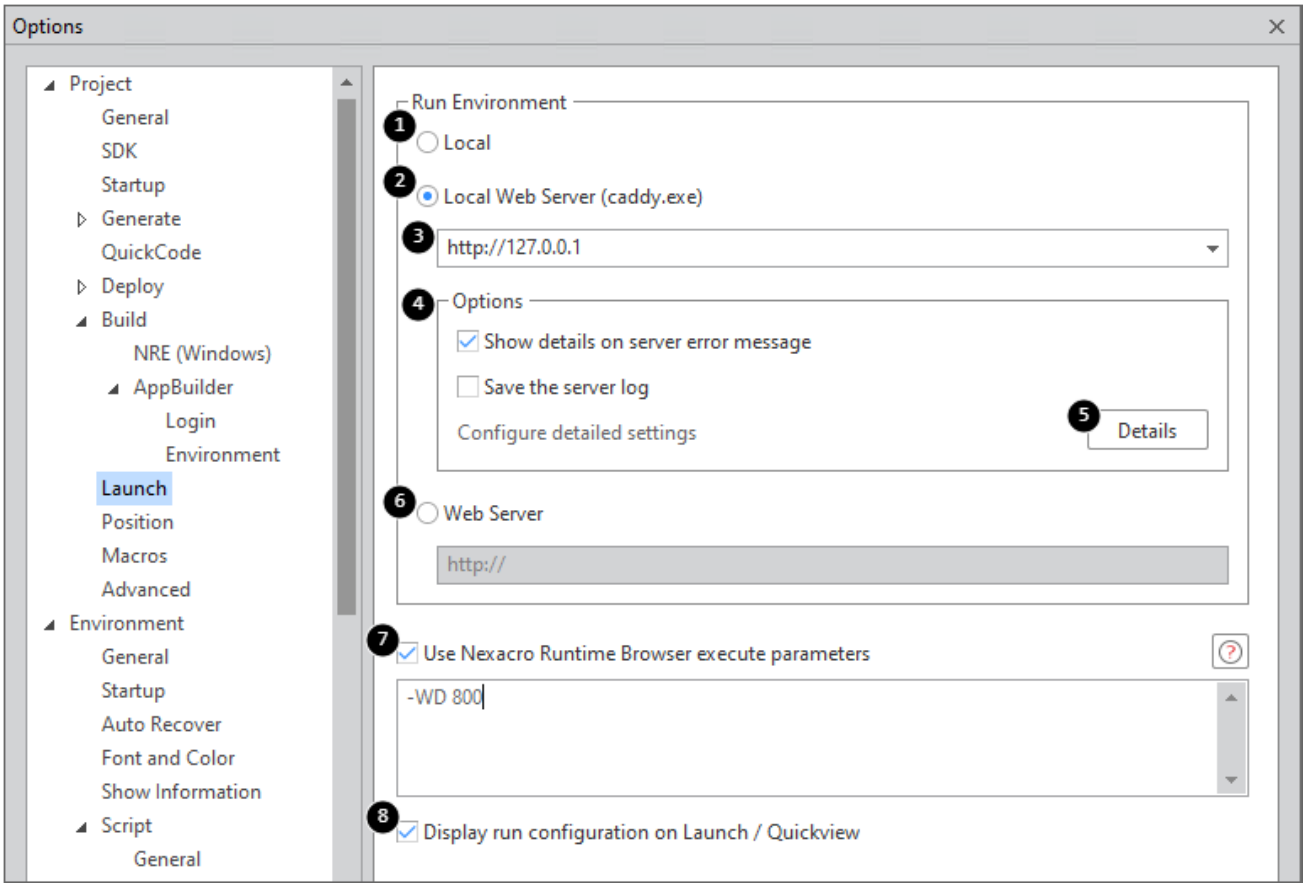
AppBuilder > Environment

It specifies the path to the archive file created in the previous step. If there is no file created, then the task is not processed.



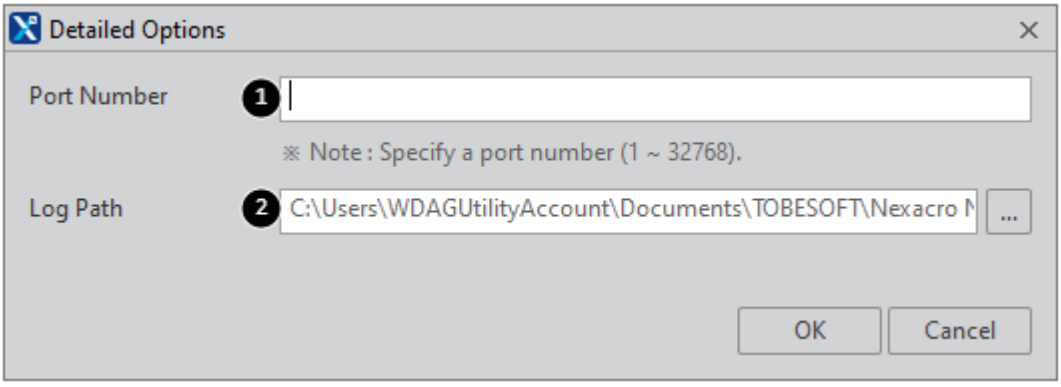
23.1.8 Launch

Nexacro Studio provides a web server feature to allow users to test their apps on web browsers without installing a separate web server. You can select your own test web server in operation. Moreover, you can select a local environment when you use the NRE or emulator.



	Option	Description
1	Local	When the NRE or emulator is used, files are executed directly in a local environment without a web server.
2	Local Web Server	Sets the local web server provided by Nexacro Studio
3	Server Host Url	Determines whether to use a local web server. If you want to use a local web server, choose an option in the form of a host URL (either 127.0.0.1 or localhost).
4	Option	[Show details on server error message]: determines whether to display the errors that occur during an HTTP transaction. The errors are displayed on the Output pane. [Save the server log]: determines whether to save the logs of a web server into a file
5	Details	Sets further options. If you select "Not Used" for the [Server Host Url] option, this option will be deactivated.

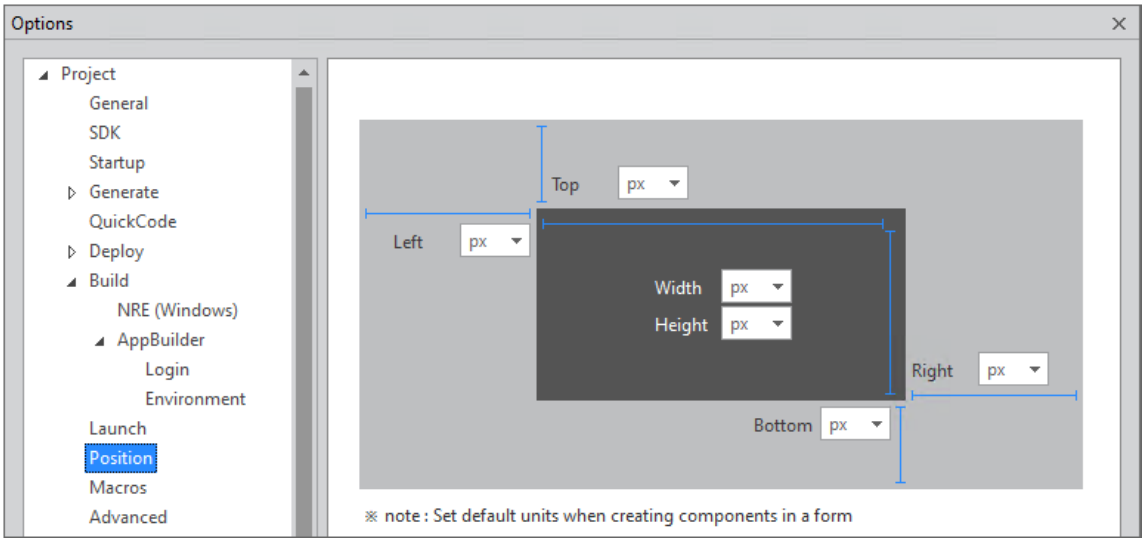
	Option	Description
6	Web Server	Allows you to specify the URL of a web server
7	excute parameters	Sets whether to specify the parameter value when executing NRE. You can specify the parameter value used in the execNexacro method.
8	Display run configuration	Determines whether to display Run Configuration Dialog when executing Launch/QuickView



	Option	Description
1	Port Number	You can specify a port number on your own. If nothing is specified, the port number will be specified automatically.
2	Log Path	If you have chosen to save web server logs into a file, you can specify the path and name of the file.

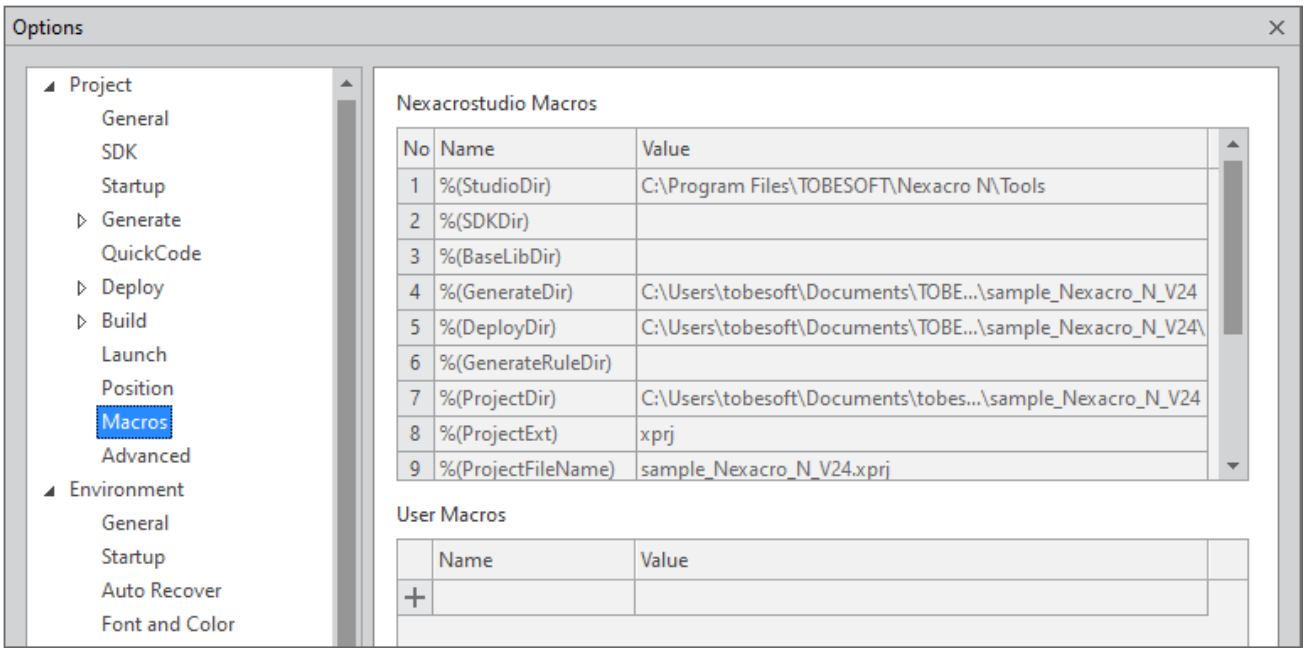
23.1.9 Position

Set up the units for measuring the position when generating a component in a Form. The values designated in Options serve as defaults when generating a Form.



23.1.10 Macros

You can check the macro name and setting value provided by default and additionally set macros.



The Macros provided by default are as follows. You can use other macros, except for the Nexacro Studio execution path, while the project is open.

	Macro Name	Description
1	%(StudioDir)	Nexacro Studio execution path
2	%(SDKDir)	SDK path
3	%(BaseLibDir)	[Project] - [General] - Base Library Path
4	%(GenerateDir)	[Project] - [Generate] - [General] - Generate Path
5	%(GenerateRuleDir)	[SDK Path] - generate
6	%(DeployDir)	[Project] - [Deploy] - [General] - Deploy Path
7	%(ProjectDir)	Project path
8	%(ProjectExt)	Project file extension (xprj)
9	%(ProjectFileName)	Project file name (including extension)
10	%(ProjectName)	Project name
11	%(ProjectPath)	Project path (including project file name)
12	%(ActiveFilePath)	Active file path (including filename extension)
13	%(ActiveGenerateFilePath)	Activated File Generate Path (including filename extension)

The %(ActiveFilePath) and %(ActiveGenerateFilePath) macros support only the files with the following extensions among the files included in the project.
- xfdl, xjs, js, xml

Macros can be used to set paths or to set specific values in Option. The available options are as follows.

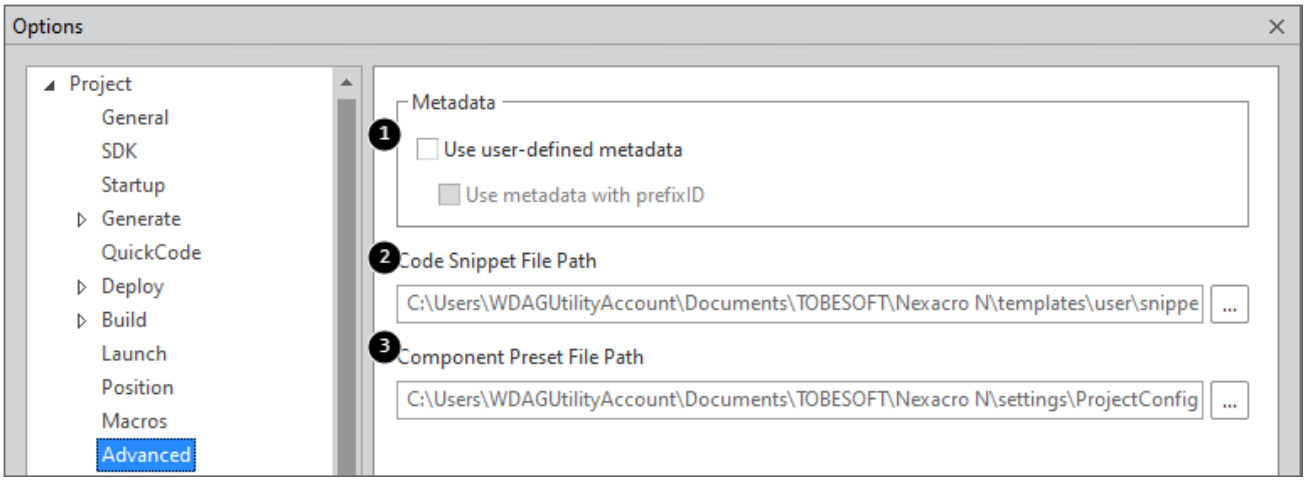
- Project
  - **General** - Base Library Path
  - **Generate** - Generate Path, Before/After Generate Command
  - **QuickCode** - Form Template Location, View Template Location
  - **Deploy** - Deploy Path, Ignore compress files
  - **Build > NRE(Windows)** - Execute Module, Resource Path, Bootstrap URL, Output Path
  - **Build > AppBuilder** - Server URL, ID, Resource Path
  - **Launch** - Local Web Server URL, Web Server URL, Nexacro Runtime Browser excute parameters
  - **Advanced** - Code Snippet File Path, Component Preset File Path
- Environment
  - **External Tools** - Name, Path, Argument

When running Nexacro Deploy, it can also be used in the following format. At this time, macros cannot be used for the option value or project XPRJ file path. As in the example below, you can use macros when specifying values for all options, except for the -P option shown in red.

```
nexacrodeploy.exe  -P "C:\Test\Test.xprj" -O "E:\ResultGenerate" -B
"C:\TestGenerate\nexacrolib" –GENERATERULE "C:\TestGenerate\generate"
nexacrodeploy.exe  -P "C:\Test\Test.xprj" -O "%(GenerateDir)" -B "%(BaseLibDir)
" –GENERATERULE "%(GenerateRuleDir)"
```

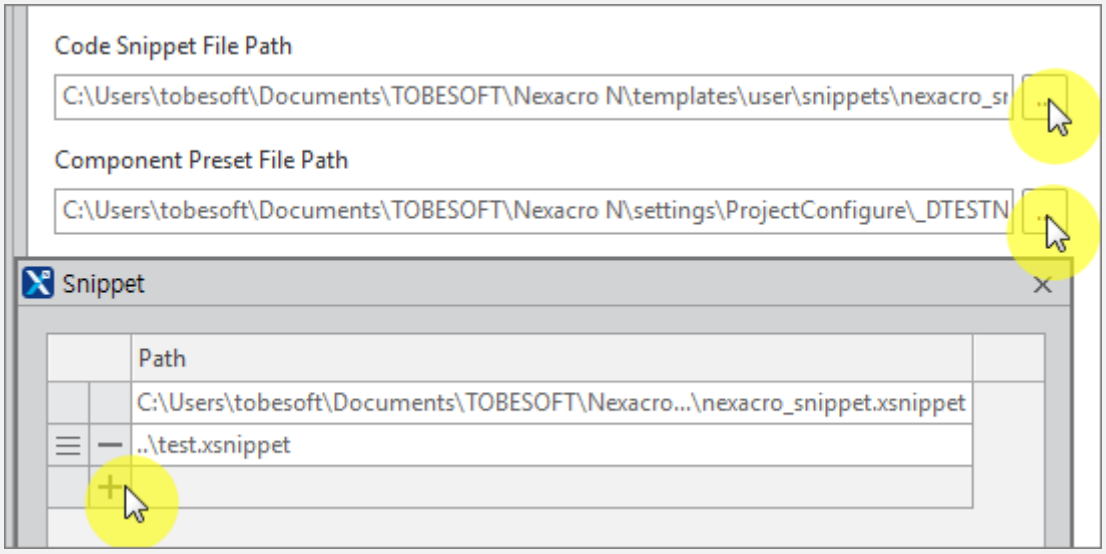
23.1.11 Advanced

It enables the function to use metadata in the project. The component ID is set by combining PrefixID of the component defined in TypeDefinition and the selected metadata.



	Property	Description
1	Metadata	[Use user-defined metadata] Sets whether to use metadata [Use metadata with prefixID] Sets the component ID by combining PrefixID of the component defined in TypeDefinition and the selected metadata
2	Code Snippet File Path	Sets the code snippet file location. If not set otherwise, then the file provided by default when installing Nexacro Studio is used.
3	Component Preset File Path	Sets the component preset file location. If not set otherwise, then the file provided by default when installing Nexacro Studio is used.

- You can modify the default path or manage two or more paths.
- When set to a folder within the project path, a relative path is used.

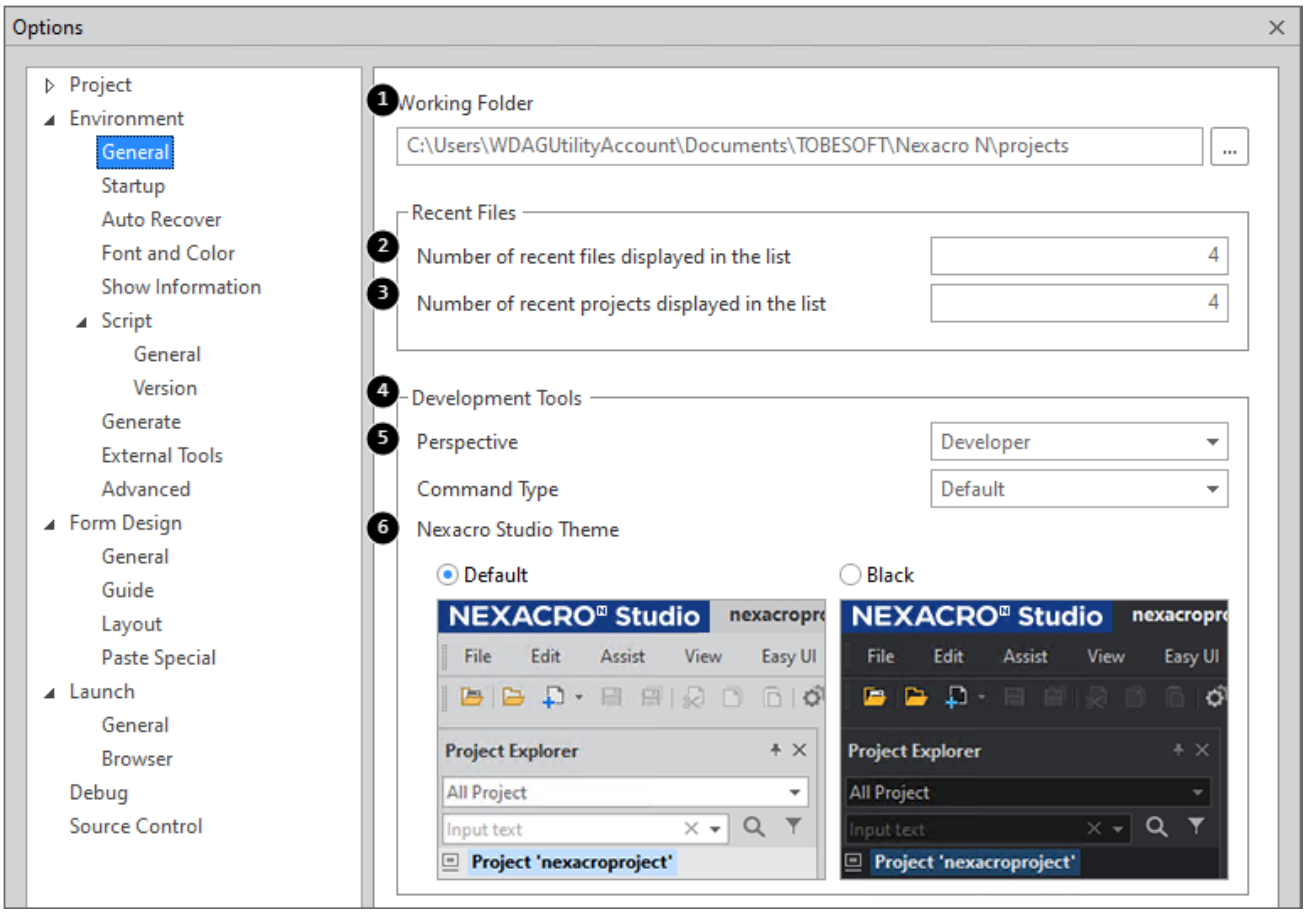


23.2 Options – Environment

You can set up overall options for Nexacro Studio.

23.2.1 General

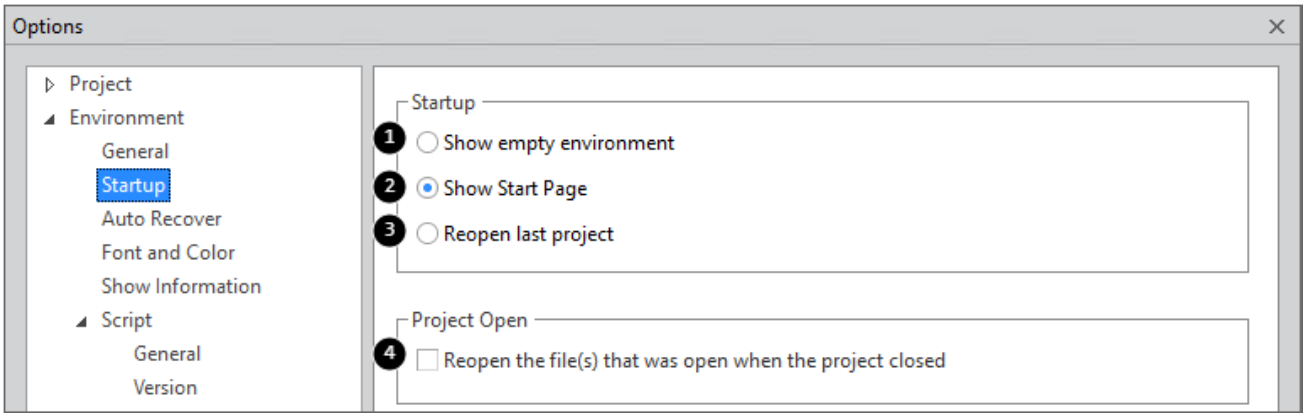
Select **General** to set up the general options for Nexacro Studio.



	Option	Description
1	Working Folder	Designates the folder where new projects will be saved
2	Number of recent files displayed in the list	Controls the number of files displayed in the [File – Recent Files] menu. A maximum of 16 files can be displayed.
3	Number of recent projects displayed in the list	Controls the number of recently used project items displayed in the [File – Recent Projects] menu. A maximum of 16 projects can be displayed.
4	Perspective	Determines the layout of Nexacro Studio (Developer / Designer)
5	Command Type	Determines the types of menu interface (Default / Ribbon)
6	Nexacro Studio Theme	Selects a UI theme of Nexacro Studio (Default / Black)

23.2.2 Startup

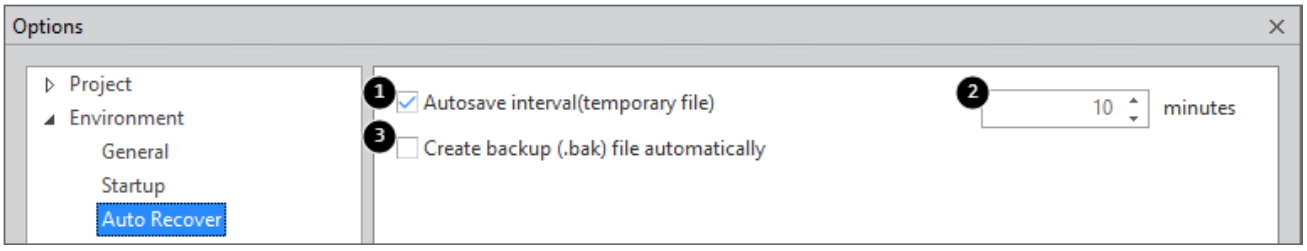
Controls the behavior of Nexacro Studio when it is started.



	Option	Description
1	Show empty environment	Nexacro Studio will begin with nothing opened.
2	Show StartPage on Startup	Nexacro Studio will open the StartPage when it begins.
3	Reopen last project	Nexacro Studio will re-open the last project worked on when it is started
4	Reopen the file(s)...	Nexacro Studio will open last-opened files when it opens a certain project.

23.2.3 Auto Recover

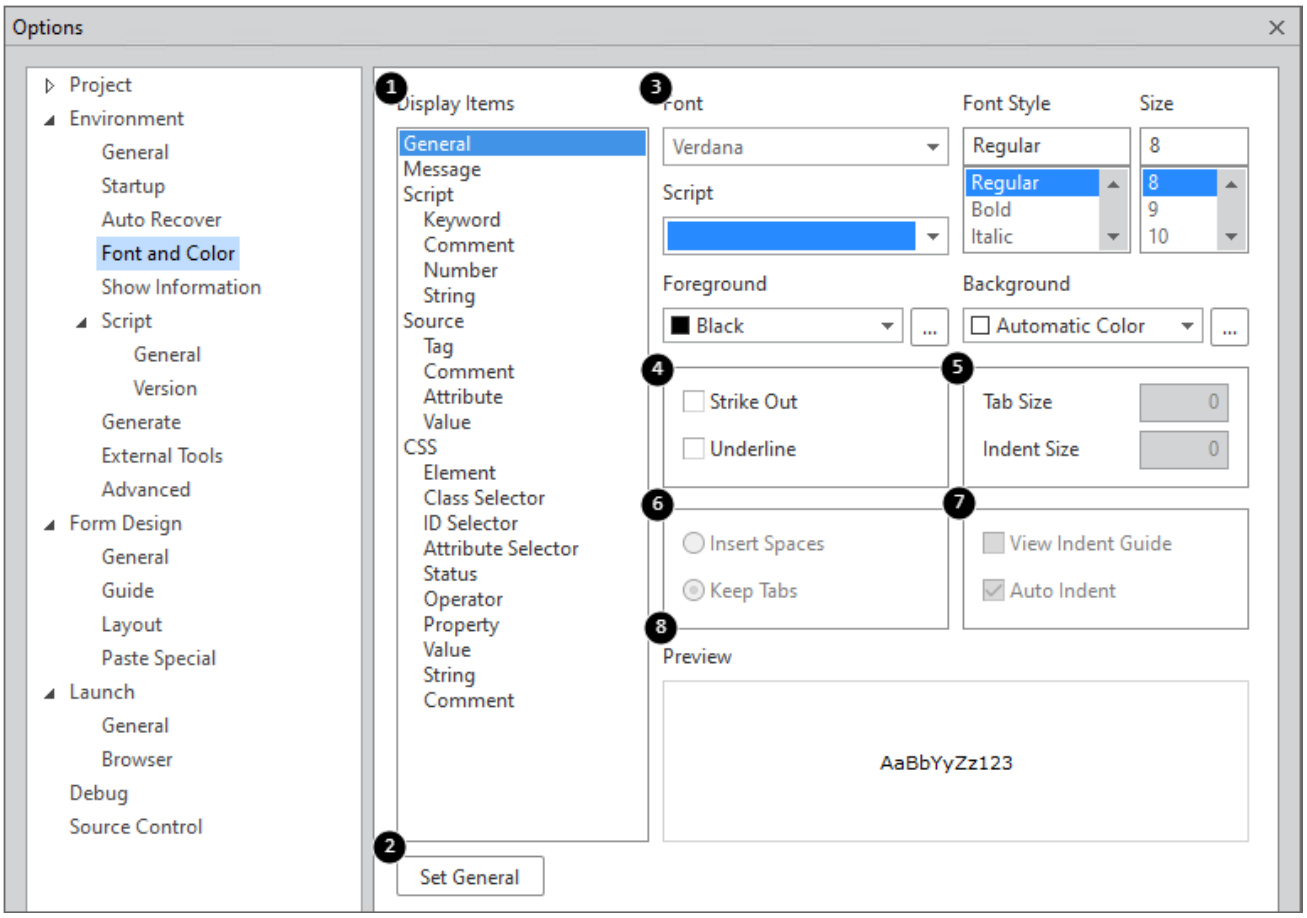
Sets up the auto recovery options.



	Option	Description
❶	Autosave interval (temporary file)	If checked, a temporary file will be created.
❷		Specifies the autosave interval
❸	Create backup (.bak) file automatically	Creates a backup file

23.2.4 Font and Color

Sets up the font and font color that are used in each Nexacro Studio window.

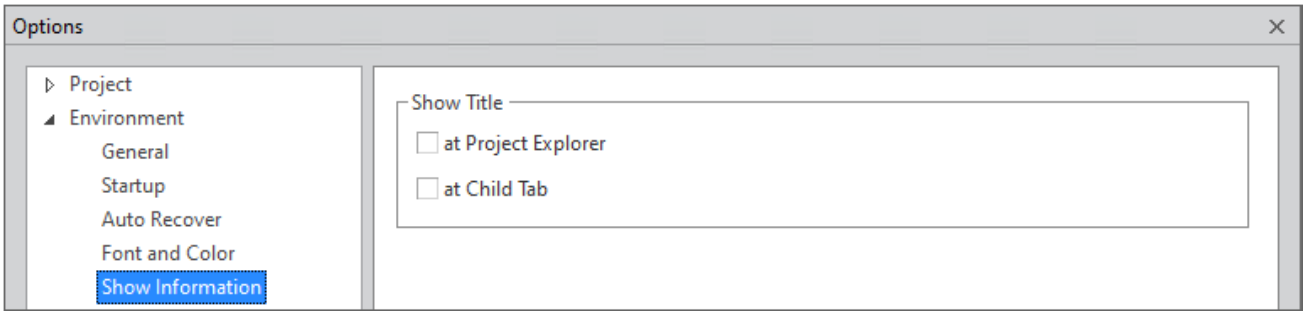


	Option	Description
1	Display items	Selects the items whose display font and color are being changed
2	Set General	The value set as the 'General' item value in Display Items is reflected in the rest of the Display Items. It can be used when you want to set the basic Font, Color style and directly modify the rest of the Display Items. For example, if you change the Size value of the 'General' item to 9 and click the 'Set General' button, the Size values of all other items are changed to 9.
3	Font	Select font
	Font Style	Sets font style
	Size	Sets font size
	Script	Sets the language scripts that can be used in the selected font
	Foreground	Sets the font color
	Background	Sets the font background color

	Option	Description
4	Strike Out	If checked, causes text to be displayed in "strike through"
	Underline	If checked, causes text to be underlined
5	Tab Size	Sets the tab size
	Indent Size	Sets the indent size
6	Insert Spaces	When selected, causes tabs to be converted to spaces
	Keep Tabs	When selected, causes tabs to be maintained as tabs
7	View Indentation Guide	When selected, caused the indentation guide to be displayed
	Auto Indent	When selected, enables automatic indenting
8	Preview	Displays sample text formatted as specified by the options selected

23.2.5 Show Information

You can set up the option of displaying the titles of Forms.

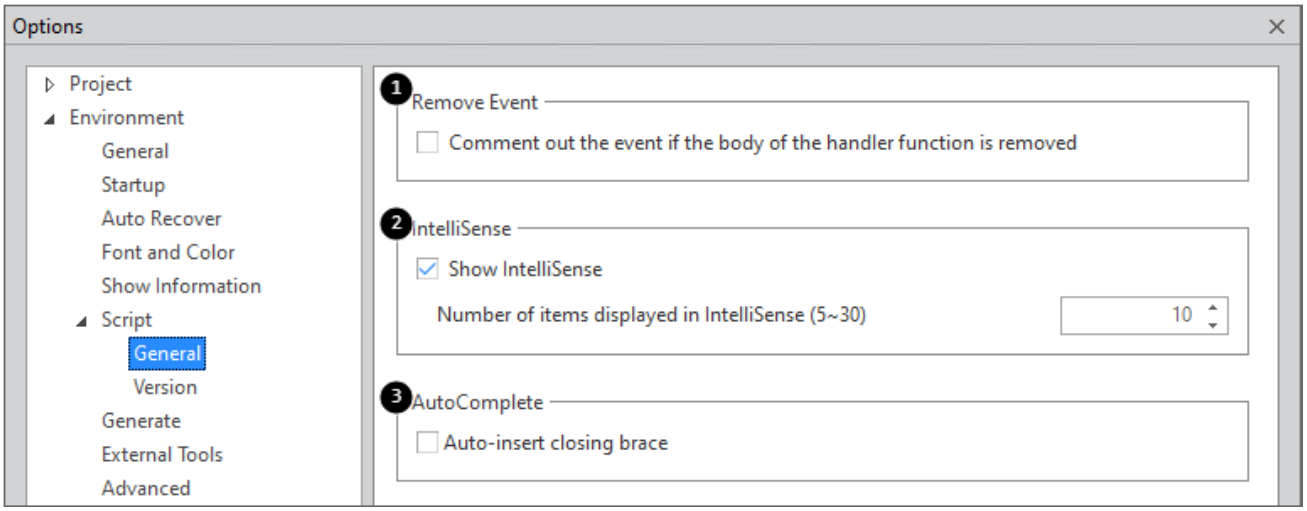


	Option	Description
❶	Show Title	Controls whether to display title text: • at Project Explorer: Displays the title text information in FDL on Project Explorer • at Child Tab : Displays the title text information in FDL on the tab of the editing screen

23.2.6 Script

Sets up the options used in script editing screen.

General

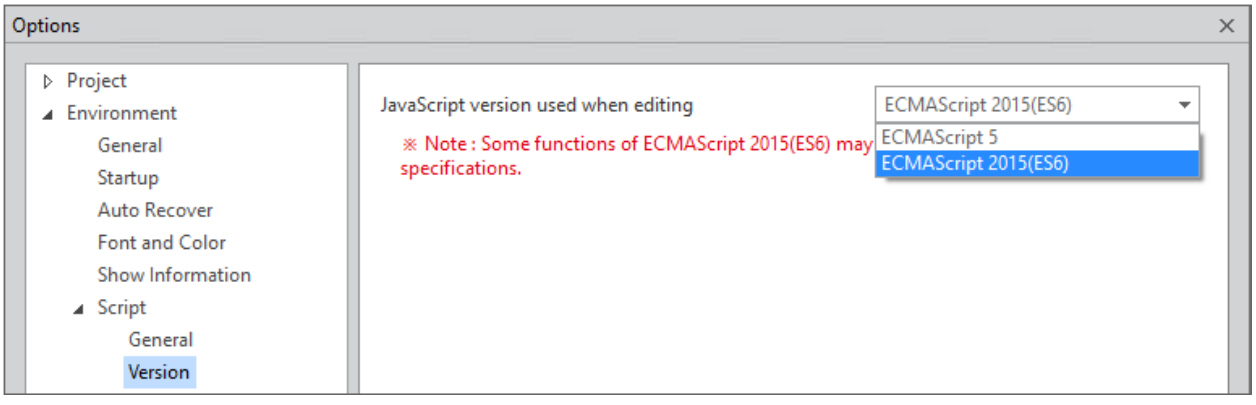


	Option	Description
1	Remove Event	Determines whether to comment out an event function when the relevant event is removed through the Properties pane
2	IntelliSense	Determines whether to display IntelliSense Sets up the number of items displayed in the IntelliSense list
3	Auto Complate	Sets up whether to insert a closing brace automatically

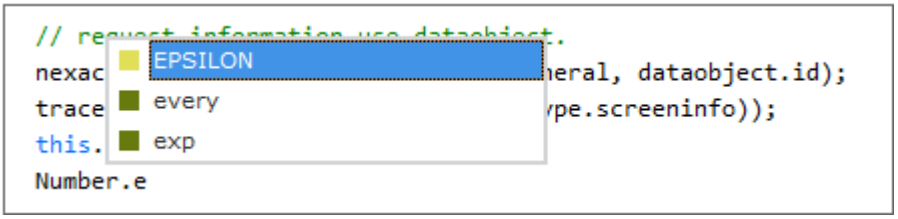
Version

Select the JavaScript version to use when writing the script. Depending on the version selected, the items supported by Script IntelliSense will change, and the script verification rules are applied differently for Generate.

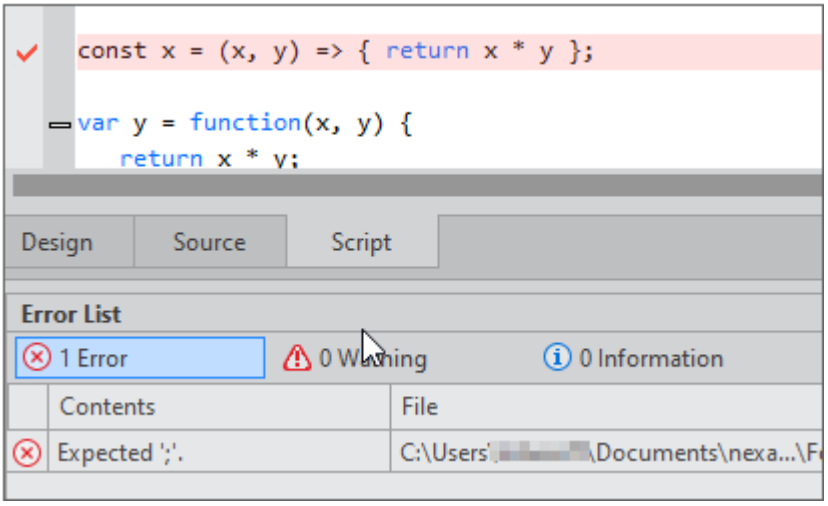
If you select "ECMAScript 2015(ES6)" as the Version option, then a script error may occur depending on the execution environment. You need to write the code in consideration of the user environment.



This is the IntelliSense screen displayed when "ECMAScript 2015(ES6)" is selected as the Version option and "Number.e" is entered in the script edit window. The "Number.EPSILON" property, which is supported from ES6, is additionally displayed.

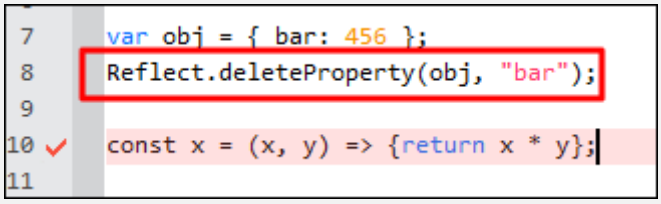


If you select "ECMAScript 5" as the Version option and use ES6 syntax in the script edit window, then it will be processed as a script error, and Generate will not be processed normally.



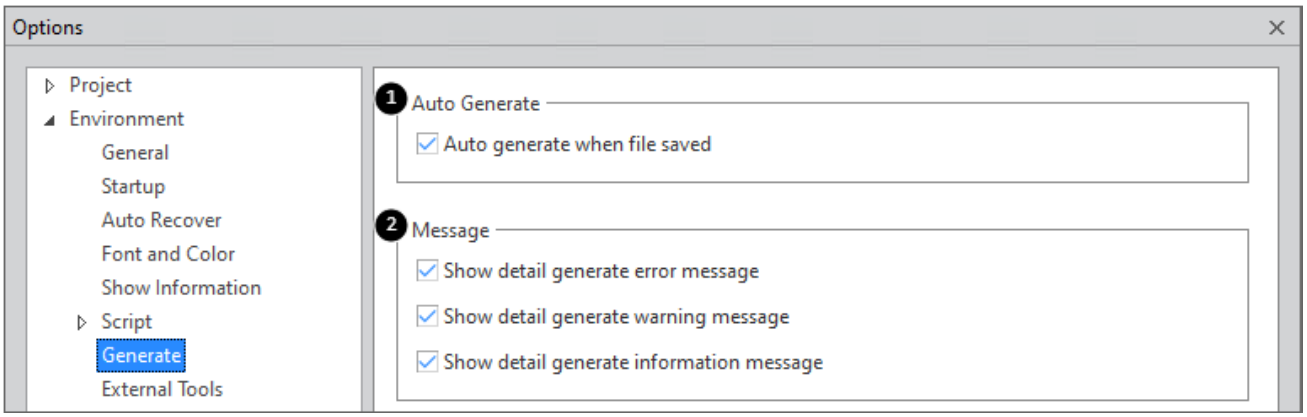
Some of the ES6 grammars are recognized as user-defined classes, so editing scripts may not generate errors.
For example, when using built-in objects such as Proxy, Reflect, and Promise added in ES6 within a script, even if the Version option is set to "ECMAScript 5", it is not treated as a script error and Generate file is created normally.

In the code below, const declarations are checked for script errors, but Reflect built-in objects are not treated as errors.



23.2.7 Generate

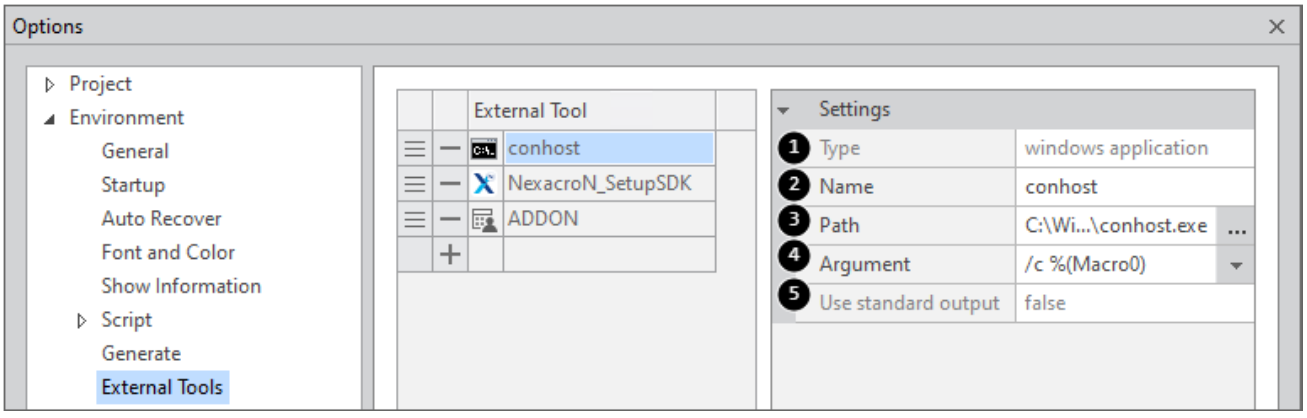
You can set the options for generation process.



		Description
1	Auto Generate	Determines whether to conduct generation automatically when you save a file
2	Message	Selects the types of messages that will be printed in the generation process

23.2.8 External Tools

You can register Add on programs or Add on that can be executed. The registered external program can specify Argument and then it can be executed directly from the toolbar or menu. Add on can be executed in the Add on window and control project information.



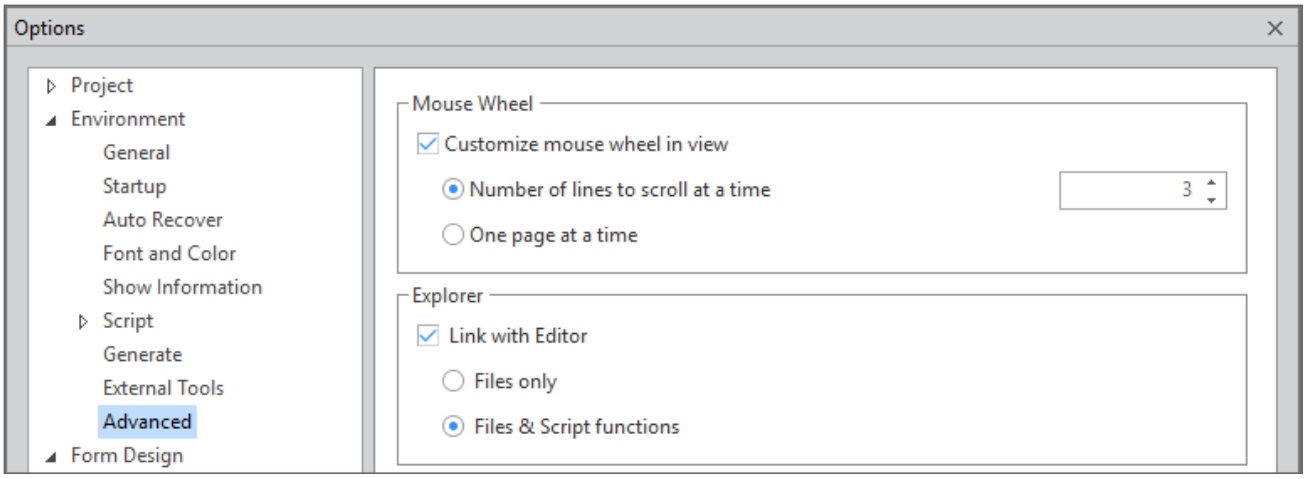
	Setting Property	Description
1	Type	Displays the registered External Tools type. ● windows application: Windows application program ● nexacro application: Add on

	Setting Property	Description
2	Name	Sets the name to be displayed in the list.
3	Path	Sets the execution path. ● Windows application program: The execution file path ● Add on: start.json file path
4	Argument	Sets Argument to be added for the execution.
5	Use standard output	Sets when the Type property is "windows application" and is the console program that supports the standard output. Selects whether to display the result message in the Output window after execution. If the standard output is not supported, then the corresponding property item is disabled.
	Connection type	Sets when the Type property is "nexacro application". ● open project: When the project is open, the first registered Add on is automatically connected. Only one of the registered nexacro applications can be set. The rest are automatically changed to manually. ● manually: Connects when Add on is selected directly.

When adding a new item, select whether to register the Windows application program or Add on. Select the application program with the exe extension or start.json.

23.2.9 Advanced

Configure scrolling for the mouse wheel and how it interacts with Project Explorer and Resource Explorer.



Property	Description
Mouse Wheel	

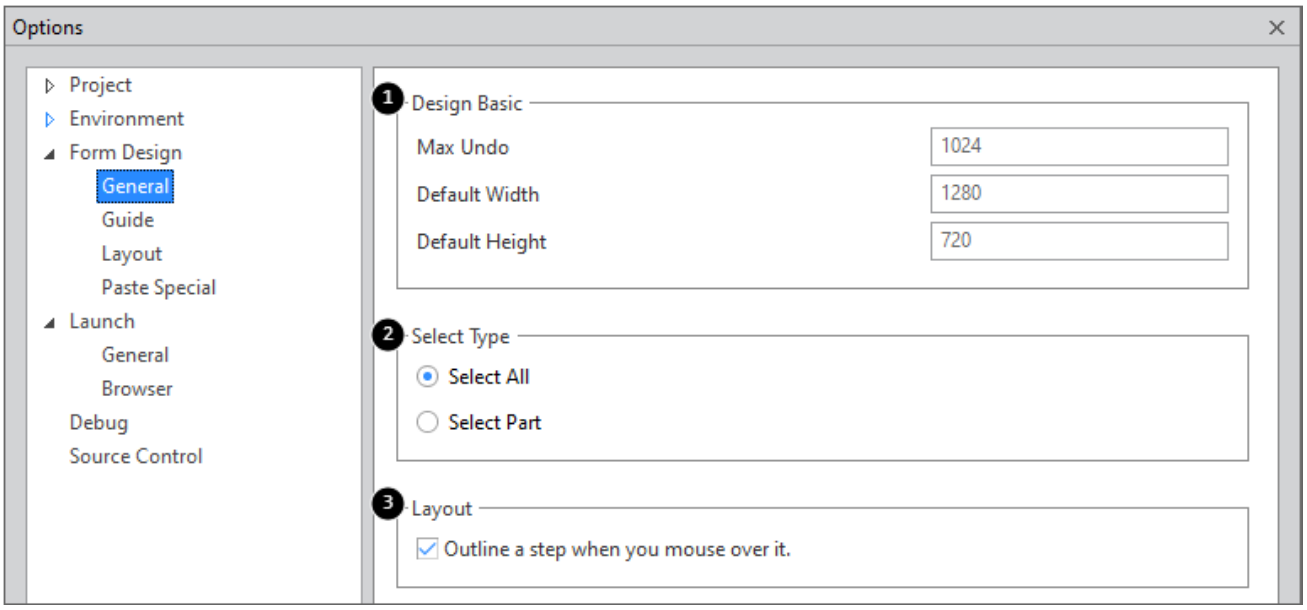
Property	Description
Customize mouse wheel in view	Checks whether to apply the mouse wheel operation option in the text editor. Checks whether to apply the mouse wheel motion option in the text editor. This option is applied only within Nexacro Studio apart from the Windows setting.
Number of lines to scroll at a time	Sets the number of lines to be scrolled at once for the mouse wheel scroll operation.
One page at a time	Moves as much as the number of currently displayed screen lines for the mouse wheel scroll operation. It may vary depending on the font size or window size.
Explorer	
Link with Editor	Choose whether to enable the Link with Editor function in Project Explorer and Resource Explorer. When selected, the setting is applied to both views and the icon state is updated accordingly.
Files only	When a file is open in the editor, the following behavior applies: ● Selecting the file in Project Explorer or Resource Explorer navigates to its tab in the editor. ● Switching tabs in the editor scrolls to and selects the corresponding file in Project Explorer or Resource Explorer. Note: This does not switch between Project Explorer and Resource Explorer tabs. For example, selecting an xcss file tab while the Project Explorer is active will not switch to the Resource Explorer.
Files & Script functions	Adds a script navigation feature to the 'Files only' option. ● Selecting a script item in the Project Explorer will move to the corresponding script position in the editor. This behaves the same as double-clicking the script without selecting the option. Note: The reverse navigation (from editor to script item in the Project Explorer) is not supported.

23.3 Options – Form Design

Sets up options related to the Design Window.

23.3.1 General

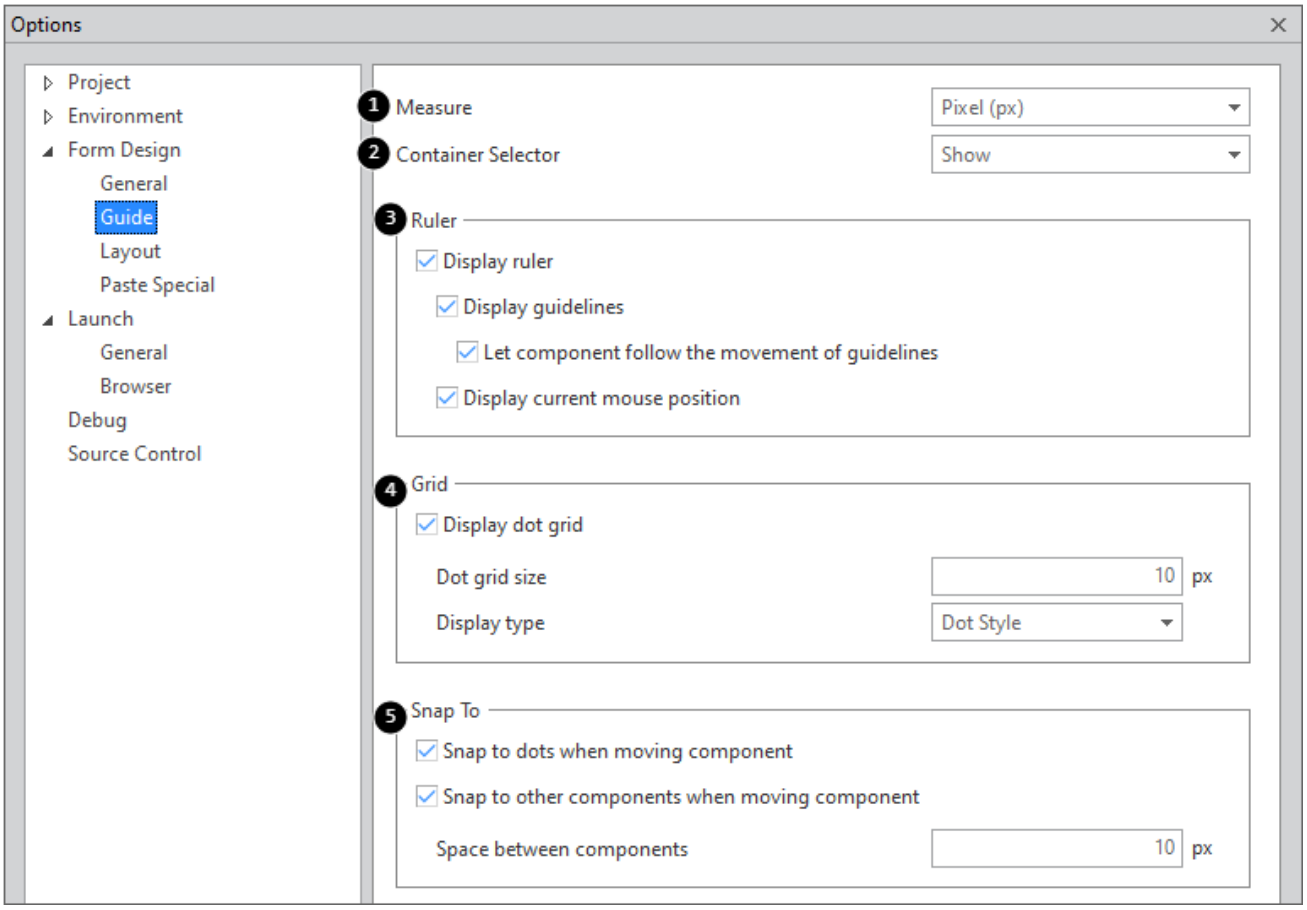
Configures general options for Form Design.



	Option	Description
1	Max Undo	The maximum number of changes than can be undone
	Default Width	The default width when creating a new Form (0~12000)
	Default Height	The default height when creating a new Form (0~12000)
2	Select Type	The way of dragging a rectangle on the Form design panel to select components. Go to Selection Change for the details.
3	Layout	Displays the currently-edited step

23.3.2 Guide

You can set up the options for the Form design editor, including Ruler, Guide Line, Grid, and Snap



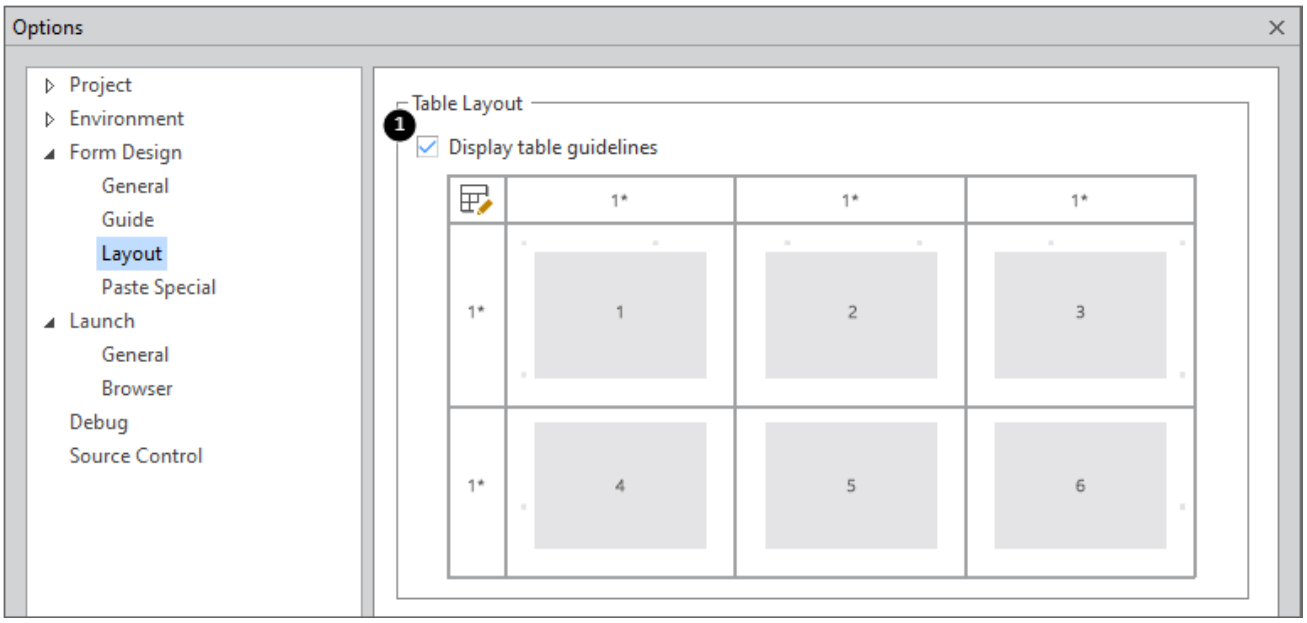
	Option	Description
1	Measure	Determines the types of unit for the position on the design window
2	Container Selector	Set whether to display additional UI for convenience of component selection when there are components placed in a container component or Panel component.
3	Display ruler	When checked, the ruler will be displayed on the Design window canvas
	Display guidelines	When checked, guidelines will be displayed on the Design window canvas
	Let component follow the movement of guidelines	When checked, components will follow the movement of a guideline if they are set to align to the guideline
	Display current mouse position	When checked, indicators will appear on the horizontal and vertical rulers, showing the current coordinate of a mouse pointer
4	Display dot grid	When checked, the dot grid will be displayed on the canvas of the Design window
	Dot grid size	The dot grid size to be displayed on the canvas of the Design window (2~12000)
	Display type	Determines the types of dot displayed on the canvas of the design window

	Option	Description
5	Snap to dots when moving Component	When checked, the magnetic function will be enabled and will align controls with dots when moving them on the canvas
	Snap to other components when moving component	When checked, the magnetic function will be enabled and will align controls with other components when moving controls on the canvas
	Space between components	Determines the gap width between a control and components, applied when the magnetic function is implemented for the moving control

When you select a component and move the component using an arrow key, the movement will be made as far as the pixel number specified in the *Dot Grid Size* property. For example, if you set *Dot Grid Size* at 8, you will move a component 8 pixels every time you press an arrow button. You can make an 1-pixel movement if you hit an arrow key with the Ctrl key pressed.

23.3.3 Layout

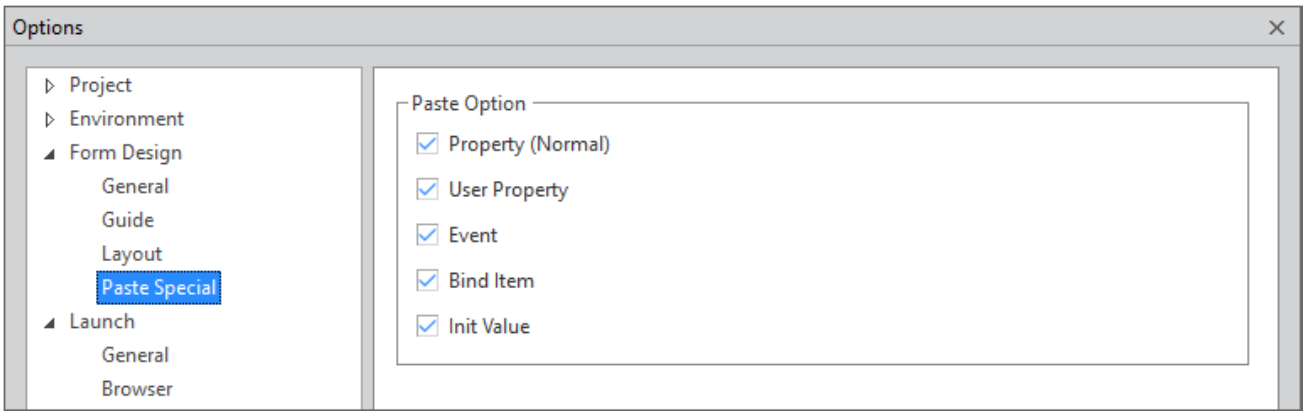
Set options according to the type property value of the layout object.



	Option	Description
1	Display table guidelines	If the type property value of the layout object is "table", set whether to display the line dividing the table cell area and the value specified as the tabletemplatearea property value.

23.3.4 Paste Special

Configures the availability of options under "Paste Special" in Form Design.

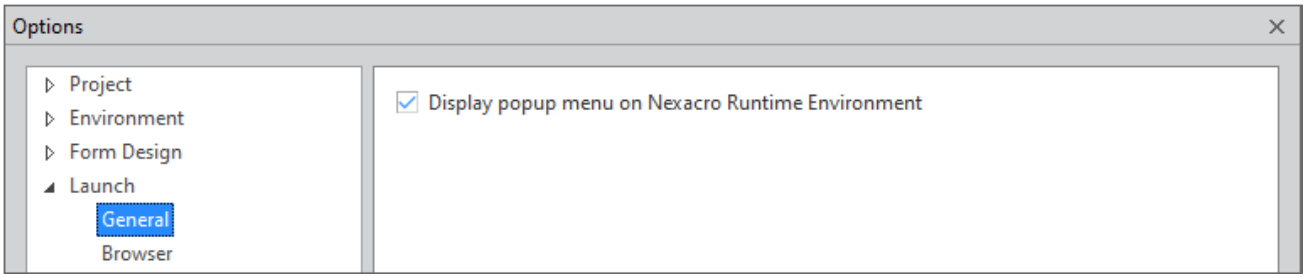


Option	Description
Property (Normal)	Paste general properties
User Property	Paste user properties
Event	Paste event(s)
Bind Item	Paste bind item(s)
Init Value	Paste InitValue information

23.4 Options - Launch

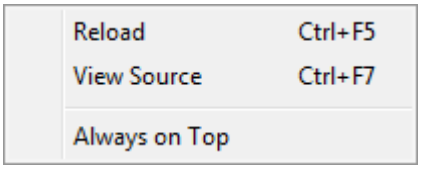
23.4.1 General

Option used when launching a Nexacro Studio project.



	Option	Description
	Display popup menu	Sets up whether to display the popup menu related to source in the launched Nexacro (NRE)

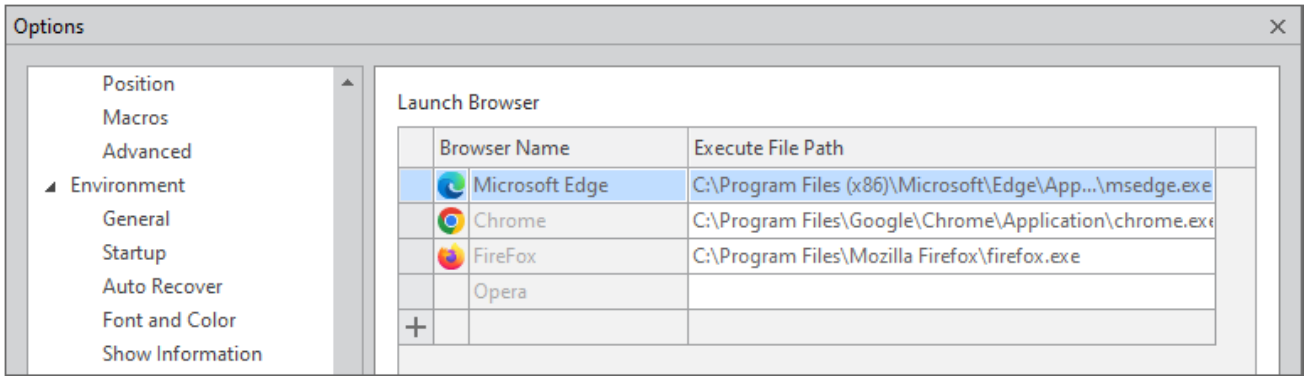
The below picture is the context menu opened when Nexacro is launched. It is not supported when a project is executed through a web browser.



Menu	Feature
Reload	Refresh the current screen. You can view revised content on Nexacro Studio on a screen where Quick View is not turned on.
View Source	Source code of the executed Nexacro-based app is shown in Nexacro Studio. You can use it when manipulating several Forms with a menu.
Always on Top	The app being executed is fixed at the highest-level area of Windows

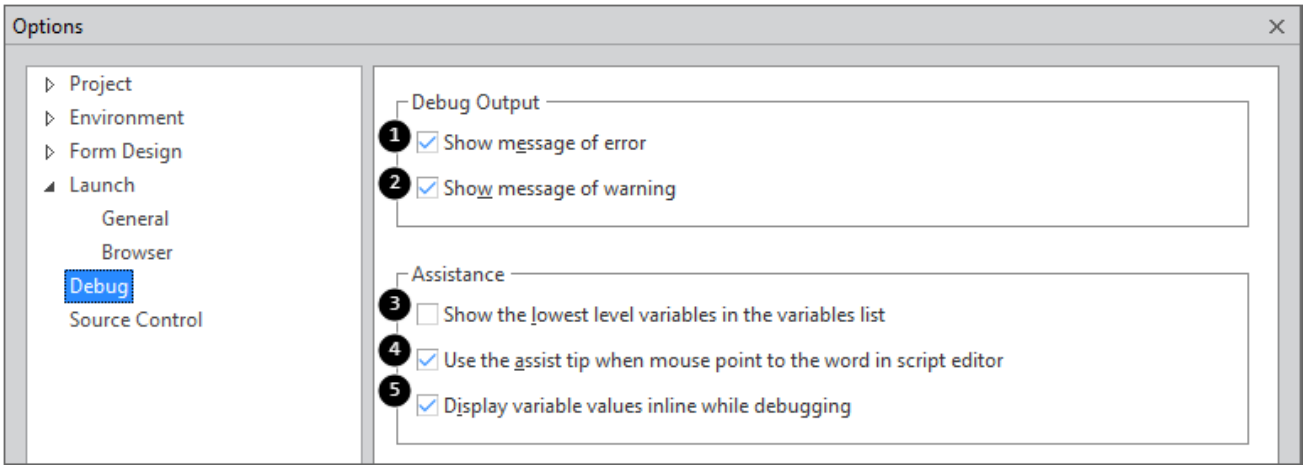
23.4.2 Browser

You can designate web browsers to which you connect your applications when initiating Launch or Quick View.



23.5 Options – Debug

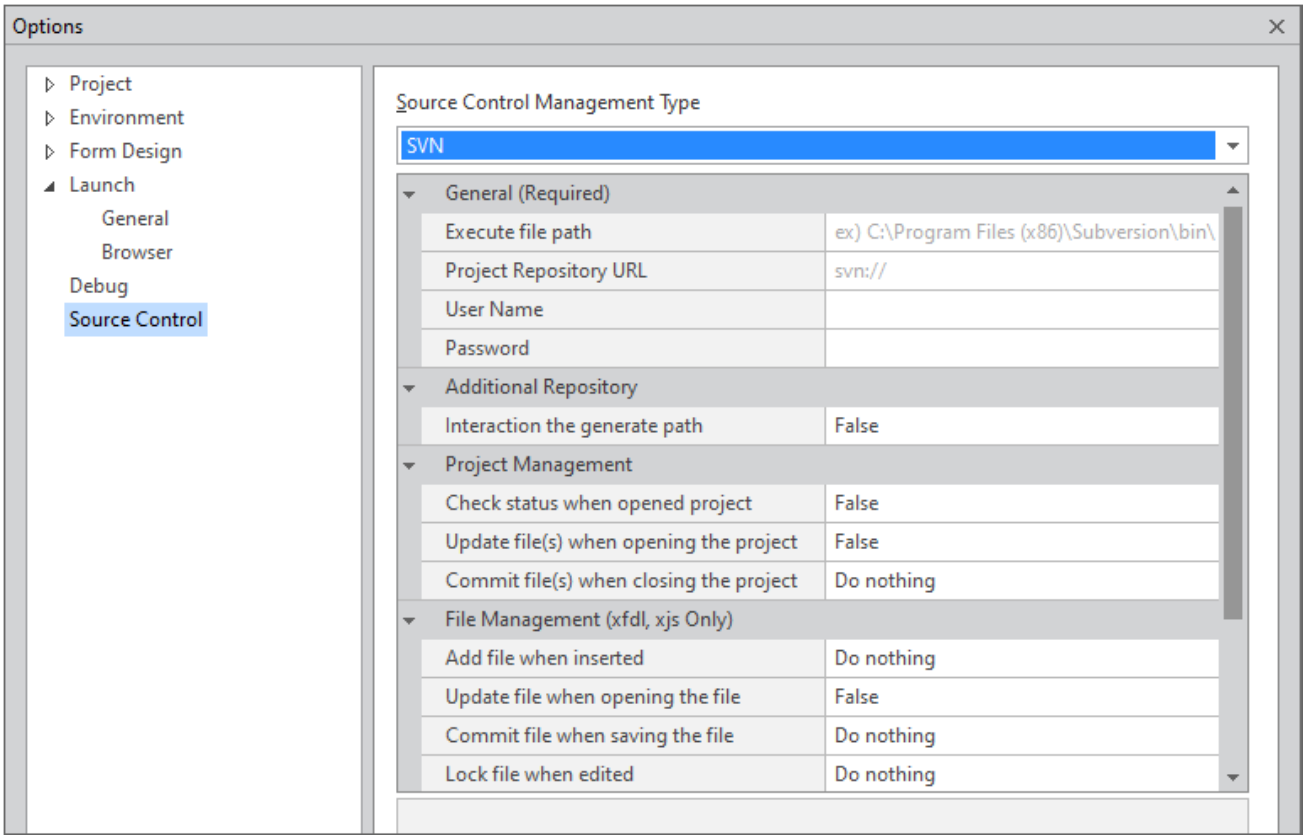
Configure general options related to debugging



	Option	Description
1	Show message of error	Displays the error messages on the Output pane
2	Show message of warning	Displays the warning messages on the Output pane
3	Show the lowest level variables in the variables list	In the debugging mode, presents up to the variables at the lowest level of the Watch or Variables list
4	Use the assist tip when mouse point to the word in script editor	In the debugging mode, displays the relevant values or information when a mouse pointer hovers over a certain variable or object on the script pane
5	Display variable values inline while debugging	In the debugging mode, displays the relevant values or information on the right-hand side of the same line where the relevant variable or object is written on the script pane

23.6 Options – Source Control

Set up source control options. This option is displayed only when available source control software is installed on user's computer.



Option	Description
Source Control Management Type	Selects which of the source control applications installed on the user's computer will be applied to projects
Options	Sets which source control options will be applied (based on the selected source control application)

24.

Toolbar, Menu bar, Keyboard shortcut

24.1 Toolbar Functions

Nexacro Studio provides access to various functions through toolbars.

24.1.1 Standard Bar

The Standard Bar provides quick access icons for the main functions of Nexacro Studio.



Menu Option		Description
	Open Project	Open a project
	Open	Open files that can be edited in Nexacro Studio
	New	Create a project, Application, Form, user component, CSS file, XML file, XJS file, etc.
	Save	Save open form document
	Save All	Save all open form documents
	Cut	Cut selected area and save to clipboard
	Copy	Copy selected area and save to clipboard
	Paste	Paste clipboard contents
	Options	Show the Options window of Nexacro Studio
	About	Show the information on the current version of Nexacro Studio and allow you to enter a license

24.1.2 Align

Align Bar allows access to a number of alignment, distribution, and control-layering functions commonly used when developing form screens.



Table 24-1 Align Bar

Menu Option		Description
	Align Lefts	Align to the left value of the component that was selected last
	Align Centers	Align to the horizontal center value of the component that was selected last
	Align Rights	Align to the right value of the component that was selected last
	Align Tops	Align to the top value of the component that was selected last
	Align Middles	Align to the vertical center value of the component that was selected last
	Align Bottoms	Align to the bottom value of the component that was selected last
	Same Width	Align to the width value of the component that was selected last
	Same Height	Align to the height value of the component that was selected last
	Same Size	Align to the size of the component that was selected last
	Distribute Horizontally	Distribute components with same horizontal space between them. Rearrange components with equal space.
	Distribute Vertically	Distribute components with same vertical space between them. Rearrange components with equal space.
	Distribute Horizontally by Specified Value	Distribute components with same horizontal space between them.
	Distribute Vertically by Specified Value	Distribute components with same vertical spaces in between.
	Position Center	Move selected components to the horizontal center of Form Canvas
	Position Middle	Move selected components to the vertical center of Form Canvas
	Position Left	Move selected components to the left side of Form Canvas
	Position Right	Move selected components to the right side of Form Canvas
	Position Top	Move selected components to the top of Form Canvas
	Position Bottom	Move selected components to the bottom of Form Canvas
	Fit to Contents	Adjust the selected components to fit their content and the specified min and max property values. Resize selected components so that they fit into contents, min- properties or max- properties
	Bring to Front	Bring selected component to the front Fluid Layout: Move item to the first position

Menu Option		Description
	Send to Back	Send selected component to the back Fluid Layout: Move item to the last position
	Bring Forward	Bring selected component forward Fluid Layout: Move item to one step earlier
	Send Backward	Send selected component back Fluid Layout: Move item to one step later
	Arrange to Tab Order	The order that components appear in the source codes is rearranged according to tab order

24.1.3 Bookmark

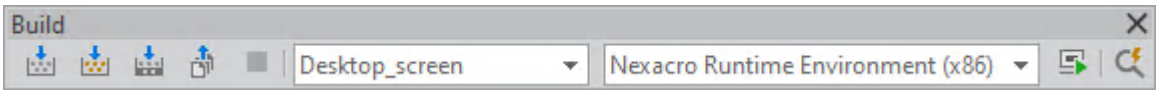
Bookmark Bar is a collection of Bookmark functions that can move to specific lines during source editing.



Menu Option		Description
	Toggle Bookmark	Sets or clears the bookmark at the current cursor location
	Previous Bookmark	Moves cursor to the previous bookmark location
	Next Bookmark	Moves cursor to the next bookmark location
	Delete All Bookmarks	Clears all bookmarks
	Previous Bookmark in Document	Moves cursor to the previous bookmark location in the current the current document
	Next Bookmark in Document	Moves cursor to the next bookmark location in the current the current document
	Delete All Bookmarks in Document	Clears all bookmarks in the current document
	Go to Bookmark	Moves the cursor to the line of a selected bookmark
	Enable/Disable Bookmark	Includes/excludes all the selected bookmarks when you are searching
	Enable/Disable All Bookmarks	Includes/excludes all the bookmarks when you are searching

24.1.4 Build

This is a toolbar related to a project/Form build. It provides the features associated with *nexacrodeploy.exe* and the execution of a Nexacro-based app.



Menu Option		Description
	Generate Application	Generates a JavaScript file for the project. The generated file will be saved in a the folder designated in Options or in the folder selected in the dialog box.
	Generate Theme	Generates a JavaScript file for the selected theme
	Generate Service	Generates a JavaScript file for the selected service group
	Generate File	Generates a JavaScript file for the Form file being edited
	Stop Generate	Stops the generation process in progress
	Screen	Selects screen information to be used when executing
	Browser	Selects an operating environment to be used when executing an app. You can select between the NRE, the emulator and a web browser installed on your computer.
	Launch	Executes an app
	QuickView	Executes activated Forms

24.1.5 Debug

Debug Bar includes a collection of functions used during debugging.



Menu Option		Description
	Start Debugging	Starts the project in the debugging mode Conducts the Continue function while debugging
	Start Form Debugging	Starts the debugging mode in the currently selected form
	Stop Debugging	Terminates the debugging mode
	Restart	Restarts the debugging mode
	Step Into	Performs one task step while in the debugging mode. If another function is called, debugging continues into the

Menu Option		Description
		function.
	Step Over	Performs one task step while in the debugging mode. If another function is called, executes the entire function call, stopping at the statement after the function call..
	Step Out	Performs task steps until the current function returns to its caller. Stops at the step after the call to the current function.
	Run to Cursor	Goes to the line where the cursor is currently located and then stops at that point
	Toggle Breakpoint	Sets/removes breakpoints
	Set Conditional Breakpoint	Sets a breakpoint
	Edit Breakpoint	Edits the condition for activating the selected breakpoint
	Delete All Breakpoints	Deletes all set breakpoints
	Go to Source Code	Activates the breakpoint list window and moves to the point where the selected breakpoint source code is located
	Enable/Disable Selected Breakpoint	Enables/Disables selected breakpoints
	Enable/Disable All Breakpoints	Activates/deactivates all set breakpoints

24.1.6 Component

The Objects Bar is a collection of components registered in TypeDefinition. Screens may be designed by dragging desired components when editing screen.



The items on the Component toolbar differ according to the components registered in the **Objects** list in TypeDefinition. The below table presents the default items set upon installation.

Selecting and moving components

Icon	Name	Description
	Select	Selects components by clicking and dragging with a mouse
	Move	Moves the position of the canvas area, which is present on the Form design tab

Basic components

This is the component that appears on the Component toolbar by default when you create a new project.

Open the TypeDefinition > Objects window, expand Modules > ComComp.json or MobileComp.json, then select unused components to hide them from the toolbar.

Icon	Component Name	Description
Modules > ComComp.json		
	Button	Used to place a button control on a Form
	Combo	Used to place a combo (dropdown) control on a Form
	Edit	Used to place an Edit (editable text box) control on a Form
	MaskEdit	Used to place a MaskEdit control on a Form. MaskEdit allows control of data formatting for items such as dates, times, phone numbers, and long numeric values
	TextArea	Used to place TextArea (multi-line text box) control on a Form
	Static	Used to place a static (fixed text) control on a Form
	Div	Used to include one or more Forms within another or when handling related components in one group
	PopupDiv	Used when constructing several sections in one screen
	Radio	Used to place a radio button (pick one item of many) control on a Form
	CheckBox	Used to place a checkbox (true/false) control on a Form
	ListBox	Used to place a listbox (list of items) control on a Form
	Grid	Used to display the contents of a Dataset in tabular form
	Spin	Used to allow users to enter data using a "spinner" control - a control that allows the user to increment or decrement the value in the fiedl
	Menu	Used to construct menu items
	PopupMenu	Used to construct a popup menu
	Tab	Used to construct a page with multiple, tabbed sections
	GroupBox	Groups related components
	Calendar	Used to allow a user to enter a date by selecting from a calendar
	ImageViewer	Used to display images on a Form
	ProgressBar	Used to place a graphic representing the status of a process on a Form
	Plugin	Used when using a plugin
	Dataset	Saves data in tabular form

Icon	Component Name	Description
	ListView	Components that represent Datset object data as items
	DataObject	Objects for calling, creating, and managing data
	CheckBoxSet	Component for selecting one or multiple values from a list of values
	MultiCombo	Component for selecting multiple items from a list of items
	Panel	Component for grouping two or more components together
Modules > MobileComp.json		
	TextField	Component for receiving single-line string input from users
	MultiLineTextField	Component for receiving multi-line string input from users
	DateField	Component for recceiving date and time input
	DateRangePicker	Component for receiving start and end date input or selecting start and end date
	PopupDateRangePicker	Component for displaying a popup window for entering or selecting a start and end date.
	SpinField	A component used to easily enter numeric values according to predefined rules
	SelectField	A component used to select a single item from a predefined item list

Default Component - Add

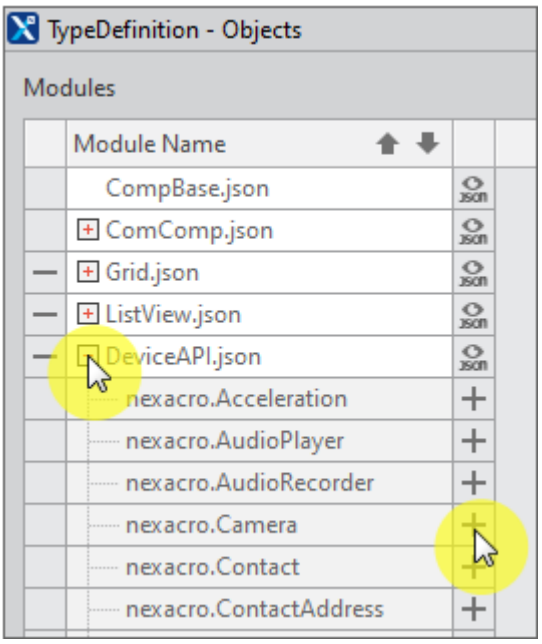
Open TypeDefinition > Objects pane and then expand Modules > ComComp.json. Then, select the necessary component to add it.

Icon	Name	Description
	FileDIalog	Launches a file dialog for users to open a file or select a directory to save a file
	FileDownload	Downloads files
	FileDownTransfer	Object for processing a file download operation
	FileUpload	Uploads files
	FileUpTransfer	Object for processing a file upload operation
	Sketch	Canvas where users can draw a simple figure, signature, etc.
	WebBrowser	Wrapper object of a web browser
	WebView	Load the web page and display it on the screen
	GoogleMap	Prints a map where designated locations are marked or allows users to mark certain locations on a map
	ExcelExportObject	Exports the contents of a Grid in the form of a spreadsheet file

Icon	Name	Description
	ExcelImportObject	Imports a spreadsheet file and converts it into a Dataset object
	VideoPlayer	Wrapper object for playing and streaming video files
	VirtualFile	Virtual file object to read or use a file in a local system

DeviceAPI

Open TypeDefinition > Objects pane and then expand Modules > DeviceAPI.json. Then, select the necessary component to add it.



Icon	Name	Description
	Acceleration	Provides the information on the acceleration recorded by a mobile device.
	AudioPlayer	Plays an audio file using a mobile device
	AudioRecorder	Records audio data and save it as a file using a mobile device
	Camera	Takes a picture using the camera of a mobile device and then processes the image or image path
	Contact	Processes the contact data of a mobile device
	ContactAddress	Object of the address field of the contact data
	ContactField	Object of the note field of the contact data
	ContactOrganization	Object of the organization field of the contact data
	ContactPhoto	Object of the photo field of the contact data

Icon	Name	Description
		
	ContactSet	Combines all the fields of the contact data
	ExteralAPI	Operates an external device (otherwise, external module or app)
	Geolocation	Provides the information on the location of a mobile device
	ImagePicker	Connects to the photo album of a mobile device and processes an image or image path.
	LiteDBConnection	Processes the connection to a database supported by a mobile device
	LiteDBStatement	Object to pass a query
	Network	Provides the information on the network connection of a mobile device.
	Sms	Object for SMS service, thus sending and receiving SMSes
	Phone	Enabling the feature to make calls from a smartphone
	Vibrator	Uses the vibration function of a mobile device
	TCPClientSocket	Providing the ability to connect to a server using TCP communication
	BluetoothLE	Providing BluetoothLE-type connectivity with external devices

XPush

Open TypeDefinition > Objects pane, click the [+] button for Modules, select and add "Push.json," expand Modules > Push.json, and select and add the XPush component.

Icon	Name	Description
	XPush	Object to process the messages from an X-PUSH server

XAgent

Open the TypeDefinition > Objects window, click the [+] button in the Modules section, select and add "XAgent.json", expand Modules > XAgent.json, and select the XAgent component to add it.

Icon	Name	Description
	XAgent	X-AGENT server communication object

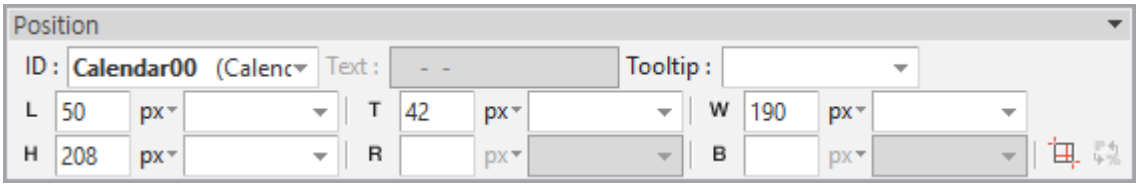
Graphics

Open TypeDefinition > Objects pane, click the [+] button for Modules, select and add "Graphics.json," expand Modules > Graphics.json, and select and add the Graphics component.

Icon	Name	Description
	Graphics	Component for using Graphics objects such as shapes, lines, images, paths, text, etc.

24.1.7 Position

The Position toolbar contains the features to edit texts, tooltips, and positions of components.



Menu	Description
ID	Select an ID of a component
Text	Enter a text of a component
Tooltip	Enter a tooltip text of a component
Left (L)	Enter a left-hand position value of a component
Top (T)	Enter a top position value of a component
Width (W)	Enter a width value of a component
Height (H)	Enter a height value of a component
Right (R)	Enter a right-hand position value of a component
Bottom (B)	Enter a bottom position value of a component
Position Editor	Launch Position Editor for editing position properties of the selected component
Position Units	Change the default position unit used when no component is selected



If a certain property is read-only, such as *text* of an Edit component, the relevant item on the toolbar will be disabled.

24.1.8 QuickCode

This is a collection of features when using QuickCode Core.



	Name	Description
	QuickCode Design Mode	Select whether or not to activate QuickCode Design Mode when editing a form including View components
	Create View Template	Runs the View Template Wizard based on the selected View component.
	View Regeneration	Recreate the selected View with QuickCode Design Mode enabled.
	Controller Chain Editor	Bring up the Controller Chain Editor window.
	Show View Information	With QuickCode Design Mode activated, the View Information window showing the information of the selected View is displayed.
	XML<->JSON Converter	Run the XML<->JSON Converter.

24.1.9 SVN

The SVN toolbar is a collection of the functions related to Subversion.

To activate the SVN toolbar, you must install a Subversion client first and then set SVN as a source control in the menu [Tools > Options].



	Name	Description
	Update	Load files of latest versions from the repository
	Commit	Apply modifications to the original files in the repository
	Revert	Return to former status of a file by cancelling the previous task
	Lock	Lock a file while you are editing the file to prevent other users from making a modification
	Unlock	Unlock a file
	Import	Register a project in the repository
	Add	Add a new file. You can apply the added file to the repository by

	Name	Description
		performing the Commit command.
	Delete	Change the status of a file to "deleted". You can delete the relevant file in the repository by performing the Commit command. The actual file stored in your PC will not be deleted.
	Rename	Change a file name. SVN does not have a *Rename* feature--it indeed deletes the existing file and adds a new file.
	Edit Conflicts	Edit a file in the conflict state
	Resolve	Resolve conflicts
	Clean up	Browse a file recursively and erase error messages
	Refresh Status	Refresh the file
	Show Log	See the modification history
	Compare with Previous Revision	Compare the current and previous versions and identify differences. This function executes a program registered through *Diff Viewer execute file path* under the Options menu.
	Repository Browser	Launch Repository Browser

24.1.10 Git

The Git toolbar is a collection of the functionalities related to Git.

> To activate the SVN toolbar, you must install a Git client first and then set Git as a source control in the menu [Tools > Options].



	Name	Description
	Init	Initialize the path of a current project in a Git repository
	Clone	Clone a project from the remote repository
	Branch	Switch from a current branch to another in the local repository If you enter a branch name that has not existed before, a new branch will be created and then the current branch will switch to the new one.
	Commit	Apply changes to the local repository
	Reset	Cancel changes and revert to the state of a previous commit
	Edit Conflicted	Edit a file that contains conflicts

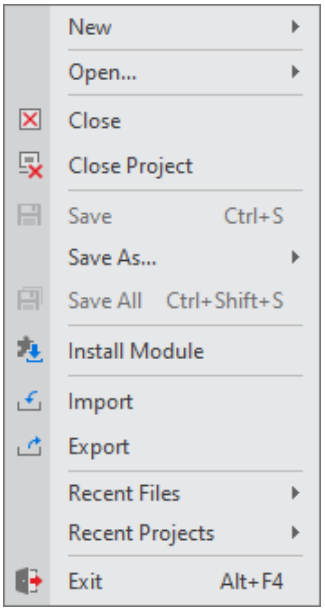
	Name	Description
	Refresh	Refresh a selected item
	Project Log	See the entire logs of a current project.
	File Log	See the logs of a select file
	Show Diff	Compare the latest commit with changes
	Set Remote	Configure the setting of the remote repository and connect it to the local repository
	Unset Remote	Delete remote repositories that are tracked by the corresponding local repositories
	Pull	Pull updates from the remote repository to the local repository
	Push	Push changes from the local repository to the remote repository

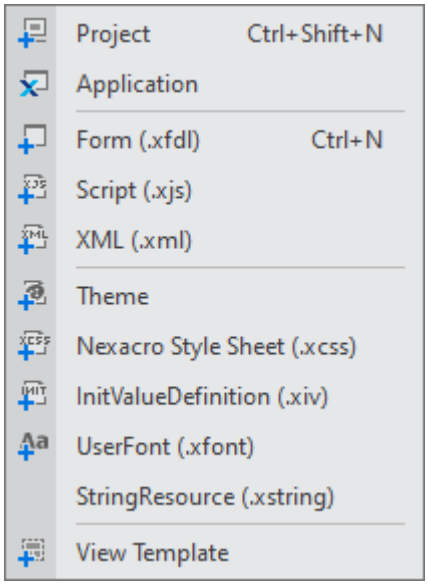
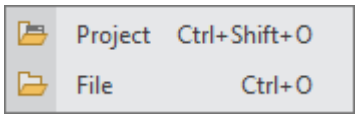
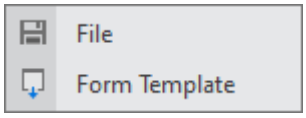
24.2 Menu Bar

The Menu Bar is a collection of functions provided by Nexacro Studio



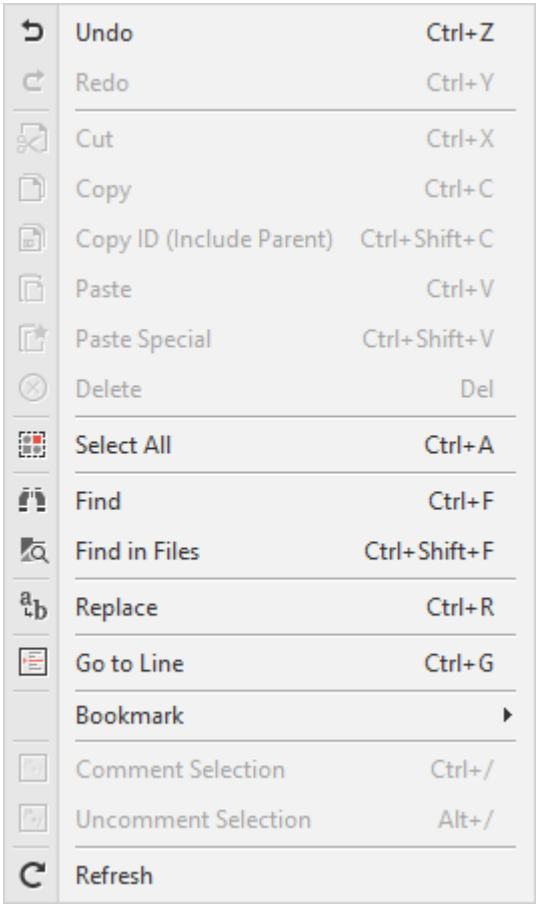
24.2.1 File



Menu Option	Description
New	 • Project- Create a project • Application- Create an Application Information • Form- Create a Form • Script (.xjs)- Create an XJS file in the current project • XML (.xml)- Create an XML file in the current project • Theme- Create a theme file in the current project • Nexacro Style Sheet (.css)- Create a style sheet file in the current project • InitValueDefinition (.xiv)- Create an InitValue file • UserFont (.xfont)- Create a UserFont file • StringResource(.xstring) - Create a StringResource file • View Template – Creates a new View Template file
Open	 • Project - Open new project • File… - Open files that can be edited in Nexacro Studio (js, xcss, xds, xtheme, etc.)
Close	Close form that is open
Close Project	Close project that is open
Save	Save currently active edit screen
Save As	 Save currently active edit screen under another name When the enabled edit screen is Form, it can be saved as Form Template or Viewset Template.

Menu Option	Description
Save All	Save all edit screens that are open and all the changed contents from Project Explorer
Install Module	Install a module file created as a result of deploying a module project
Export	Runs Export Wizard to group the files and template files in the project and exports it as the xpackage file.
Import	Runs the Import Wizard to import the exported xpackage file after executing Export.
Recent Files	Display recently opened files
Recent Projects	Display recently opened projects
Exit	Exit Nexacro Studio

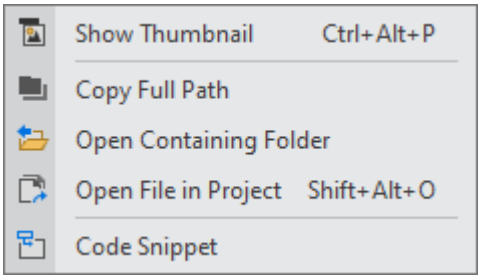
24.2.2 Edit



Menu Option	Description
Undo	Undo to the previous action
Redo	Re-perform the last action that was "undone"
Cut	Cut selected area and put it into the clipboard
Copy	Copy selected area to the clipboard
Copy ID	Copy the ID of the selected component to the clipboard

Menu Option	Description
	(This function is activated only when one component is selected in the Form design pane.)
Paste	Paste contents of clipboard
Paste Special	Paste the properties and bind information of the cut/copied component
Delete	Delete selected components or text of selected area
Select All	Select all text from source or script window
Find…	Find character string in source or script window
Find in File…	Find files that include designated word among files in designated path
Replace	In the source or script window, replace a specific character strings with another
Goto Line…	Move cursor to a specific line in source or script window
Bookmark	Same as the Bookmark toolbar
Comment Selection	Comment out the selected area in the source or script window
UnComment Selection	Remove the commenting from the selected area in source or script window (making the lines executable)
Refresh	Reload file

24.2.3 Assist



Menu Option	Description
Show Thumbnail	Activated when a Form or image is selected, this provides the preview of the selected item.
Copy Full Path	Copies to clipboard the full path where selected files are saved
Open Containing Folder	Opens in Windows Explorer the folder containing the selected file
Open File in Project	Can search for files included in the project and execute them
Code Snippet	Opens the Code Snippet Editor

24.2.4 View

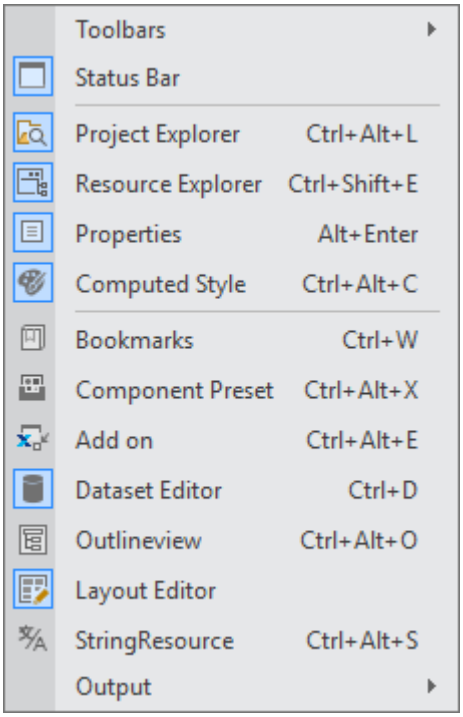
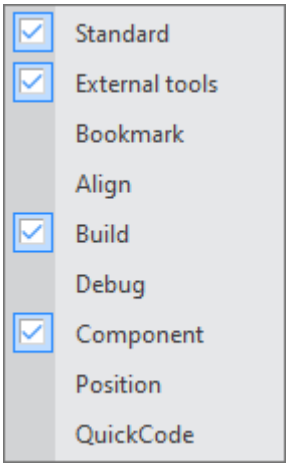
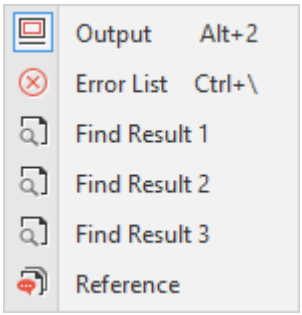


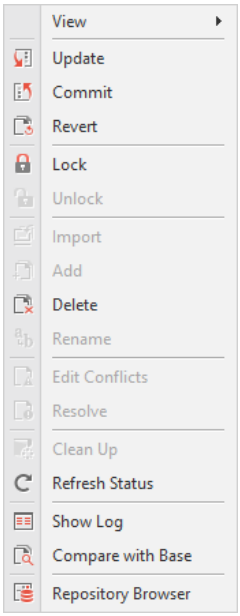
Table 24-2 View Menu

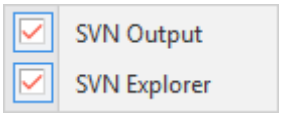
Menu Option	Description
Toolbars	 Display/Hide Seleted Toolbar
Statusbar	Display/Hide Statusbar
Project Explorer	Display the Project Explorer window. When it is open, move focus
Resource Explorer	Display the resources held by the current project
Properties	Display the Properties window. When it is open, move focus
Computed Style	Display the Computed Style tab on the Properties pane
Bookmarks	Display the Bookmarks window

Menu Option	Description
Component Preset	Display the Component Preset pane
Add on	Display the Add on window
Dataset Editor	Display the Dataset Editor
Layout Editor	Displays the Layout Editor window.
Outlineview	The outlineview window is displayed.
StringResource	The StringResource window is displayed.
Output	 • Output - Display the Output window • Error List - Display the Error List window • Find Result1,2,3 - Display the Find Result 1,2,3 window • Reference - Display the Reference window

24.2.5 SVN

To activate the SVN menu bar, you must install a Subversion client first and then set Source Control in the menu [Tools > Options].

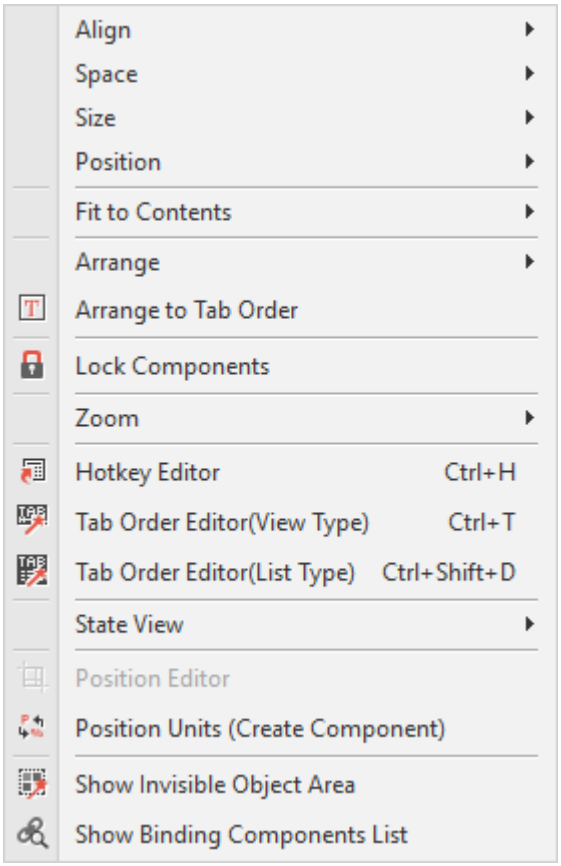


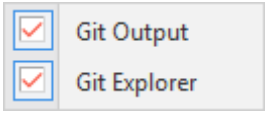
Menu option	Description
View	 • SVN Output - Display the SVN Output pane • SVN Explorer - Display the SVN Explorer
Update	Load files of latest versions from the repository
Commit	Apply modifications to the original files in the repository
Revert	Return to former status of a file by cancelling the previous task
Lock	Lock a file while you are editing the file to prevent other users from making a modification
Unlock	Unlock a file
Import	Register a project in the repository
Add	Add a new file. You can apply the added file to the repository by performing the Commit command.
Delete	Change the status of a file to "deleted". You can delete the relevant file in the repository by performing the Commit command. The actual file stored in your PC will not be deleted.
Rename	Change a file name. SVN does not have a *Rename* feature--it indeed deletes the existing file and adds a new file.
Edit Conflicts	Edit a file in the conflict state
Resolve	Resolve conflicts
Clean Up	Browse a file recursively and erase error messages
Refresh Status	Refresh the file
Show Log	See the modification history
Compare with Previous Revision	Compare the current and previous versions and identify differences. This function executes a program registered through *Diff Viewer execute file path* under the Options menu.
Authentication Data	Manage users authorized to access the Subversion server
Repository Browser	Launch Repository Browser

24.2.6 Git



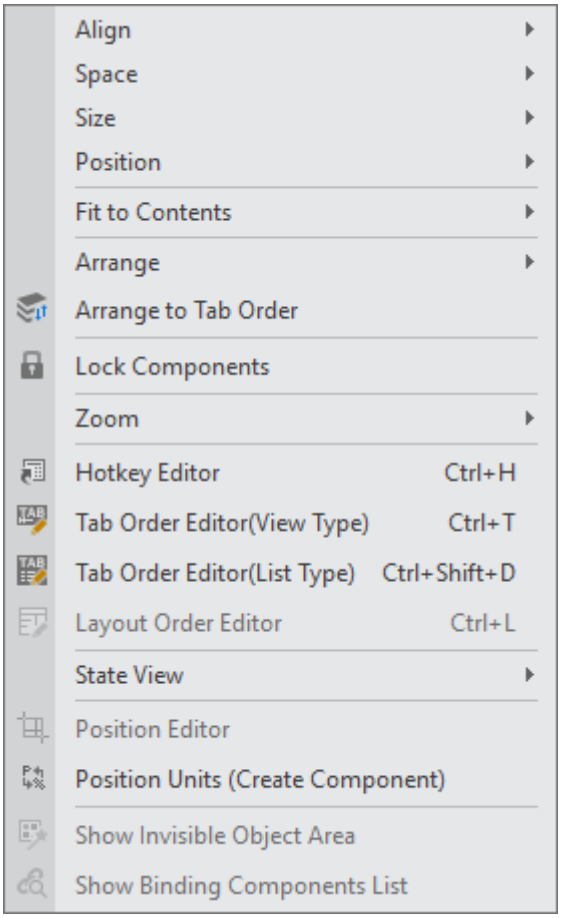
To activate the SVN menu bar, you must install a Git client first and then set Git as a source control in the menu [Tools > Options].

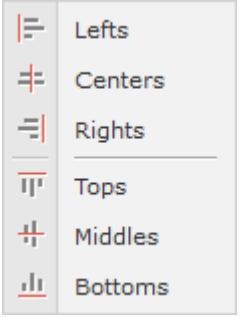


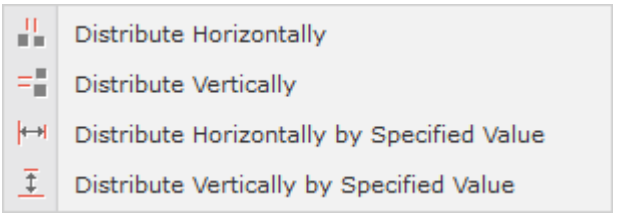
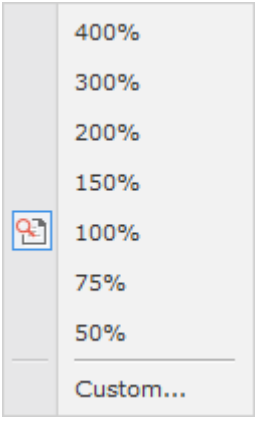
Menu option	Description
View	 • Git Output - Display the SVN Output pane • Git Explorer - Display the SVN Explorer
Init New Repository	Initialize the path of a current project in a Git repository
Clone Repository	Clone a project from the remote repository
Checkout Branch	Switch from a current branch to another in the local repository If you enter a branch name that has not existed before, a new branch will be created and then the current branch will switch to the new one.
Commit	Apply changes to the local repository
Reset Changed	Cancel changes and revert to the state of a previous commit
Edit Conflicted	Edit a file that contains conflicts
Refresh Status	Refresh a selected item
Show Project Log	See the entire logs of a current project.
Show File Log	See the logs of a select file
Show Diff	Compare the latest commit with changes
Set Remote Repository	Configure the setting of the remote repository and connect it to the local repository

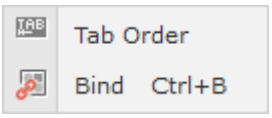
Menu option	Description
Unset Remote Repository	Delete remote repositories that are tracked by the corresponding local repositories
Pull	Pull updates from the remote repository to the local repository
Push	Push changes from the local repository to the remote repository

24.2.7 Design



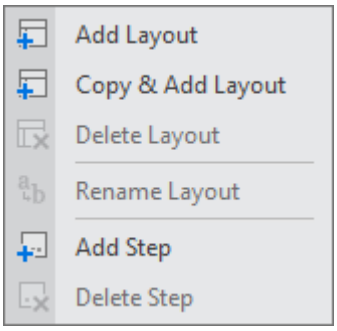
Menu Option	Function
Align	
Space	

Menu Option	Function
	
Size	
Position	
Fit to Contents	Resizes selected components so that they fit into contents, min- properties or max-properties
Arrange	
Arrange to Tab Order	The order that components appear in the source codes is rearranged according to tab order
Lock Components	Locks (or unlocks) a component to prevent it from being moved by mouse
Zoom	 Enlarges or shrinks the current view
Hotkey Editor	Edits a list of components for which hotkeys are specified
Tab Order Editor (View Type)	Begins the tab order editing mode

Menu Option	Function
Tab Order Editor (List Type)	Opens the Tab Order list window so they you can edit the order through it
Layout Order Editor	This will open a window where you can set the layout order of the components and the value of the layoutorder property. In Form object and container components (Div, Tab, View), the menu item will be activated if the value of the type property of the Layout object is not ""default.
State View	 Tab Order: Displays Tab order of component continuously on the Form design pane Bind: Displays bind status of component
Position Editor	Launches Position Editor for editing position properties of the selected component
Position Units	Changes the default position unit used when no component is selected
Show Invisible Object Area	Displays the Invisible Object pane when it is hidden
Show Binding Components List	Displays the Binding Components List pane when it is hidden

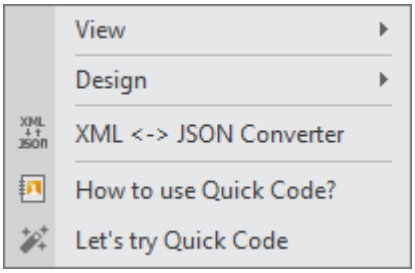
The features related to Align, Space, Size, Position and Arrange correspond to the features provided by their respective toolbars.
Please, refer to Align.

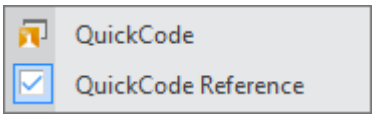
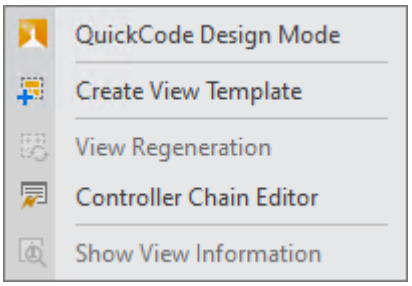
24.2.8 Layout



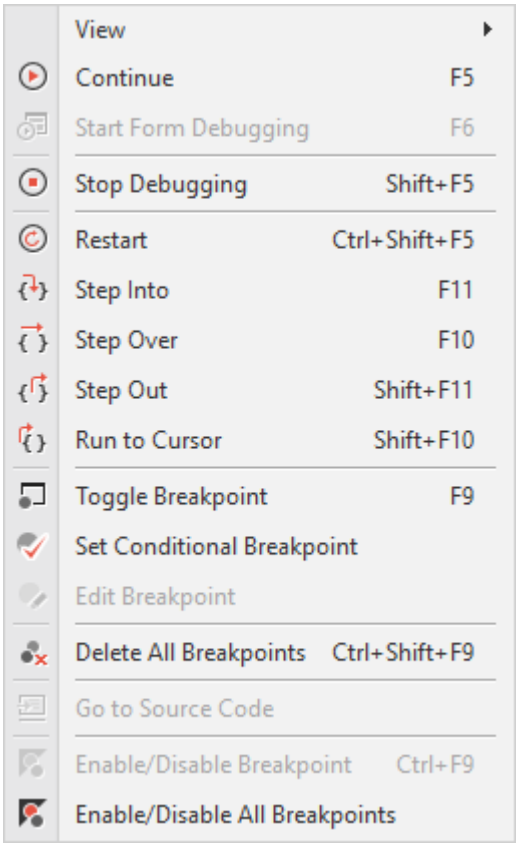
Name	Description
Add Layout	Adds a layout
Copy & Add Layout	Copies the selected layout and adds it as a new layout
Delete Layout	Deletes the selected layout
Rename Layout	Renames the selected layout
Add Step	Adds a step
Delete Step	Deletes a step

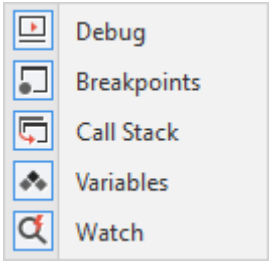
24.2.9 QuickCode



Menu	Function
View	 ● QuickCode: Displays the QuickCode window. ● QuickCode Reference: Displays the QuickCode Reference window.
Design	 ● QuickCode Design Mode: Select whether or not to activate QuickCode Design Mode when editing a form including View components ● Create View Template: Runs the View Template Wizard based on the selected View component. ● View Regeneration: Regenerates the selected View with QuickCode Design Mode enabled. ● Controller Chain Editor: Opens the Controller Chain Editor window. ● Show View Information: Opens the View Information window showing the selected View information while QuickCode Design Mode is activated.
XML<->JSON Converter	Run the XML<->JSON Converter.
How to use QuickCode?	Connects to the online manual. (Development Tool Guide > QuickCode Core)
Let's try QuickCode	Runs QuickCode Tutorial Wizard.

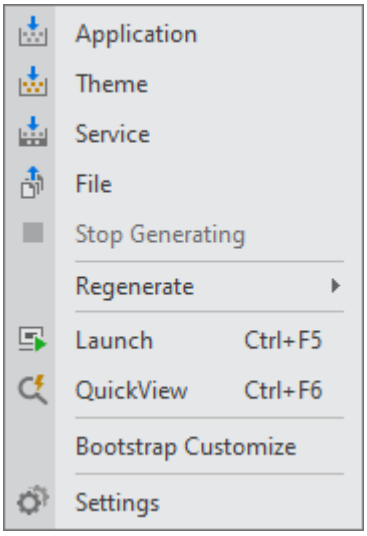
24.2.10 Debug



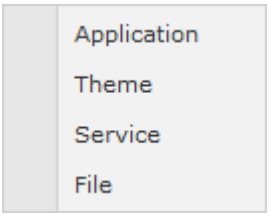
Menu Option	Description
View	 Activates the selected Debug window
Start Debugging	Starts the debugging mode in the project. The menu name will change to [Continue] in the process of debugging.
Start Form Debugging	Starts the debugging mode in the selected form
Stop Debugging	Terminates the debugging mode
Restart	Restarts debugging
Step Into	Performs one task step while in debug mode. If another function is called, debugging continues into the function.
Step Over	Performs one task step while in debug mode. If another function is called, executes the entire function call, stopping at the

Menu Option	Description
	statement after the function call.
Step Out	Performs task steps until the current function returns to its caller. Stops at the step after the call to the current function.
Run to Cursor	Performs task steps through to the line where the cursor is currently located and then stops at that point
Toggle Breakpoint	Sets/removes breakpoints
Set Conditional Breakpoint	Sets or removes a conditional breakpoint
Edit Breakpoint	Edits the condition for activating the selected breakpoint
Delete All Breakpoints	Deletes all set breakpoints
Go To Source Code	Activates the breakpoint list window and moves to the point where the selected breakpoint source code is located
Enable/Disable Breakpoint	Activates the breakpoint list window and activates/deactivates all selected breakpoints
Enable/Disable All Breakpoints	Activates/deactivates all set breakpoints

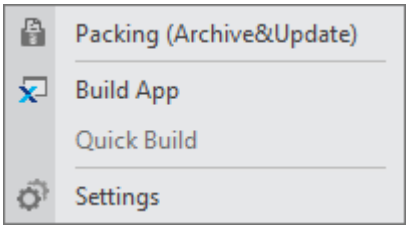
24.2.11 Generate



Menu Option	Description
Application	Generates JavaScript for all open projects. Generated files will be saved in the folder designated in option or selected in the dialog box
Themes	Generates JavaScript for the selected theme
Service	Generates JavaScript for the selected service group
File	Generates JavaScript for the Form file being edited
Stop Generate	Stops the current generation in progress
Regenerate	

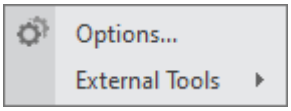
Menu Option	Description
	 Generates entire designated files regardless of whether there are any already-generated files
Launch	Executes activated projects
QuickView	Executes activated Forms
Bootstrap Customize	Opens the window where you can edit the bootstrap template file.
Settings	Sets the default option values related to Generate, Launch.

24.2.12 Deploy



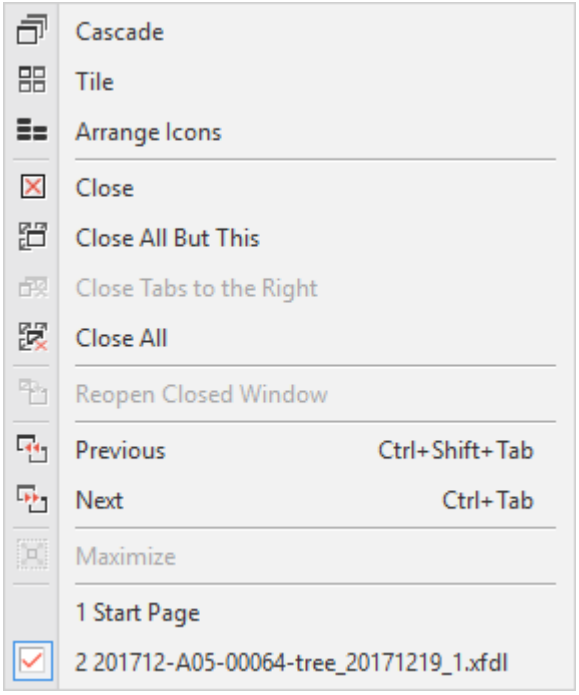
Name	Description
Packing	Creates archive files for each operating environment - Web Browser - Windows - Android - iOS - macOS
Build App	Creates app installation files for each operating system - Windows (x86 and x64) - Android - iOS - macOS
Quick Build	Runs the Quick Build window.
Settings	Sets the default option values related to Deploy, Build.

24.2.13 Tools



Menu Option	Description
Options…	Opens Options Setup Window of Nexacro Studio
External Tools	Displayed only when External Tools of the windows application type are registered in the option.

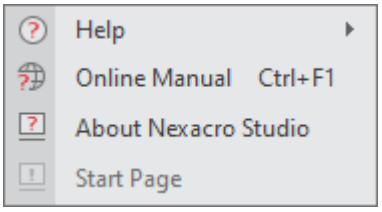
24.2.14 Window

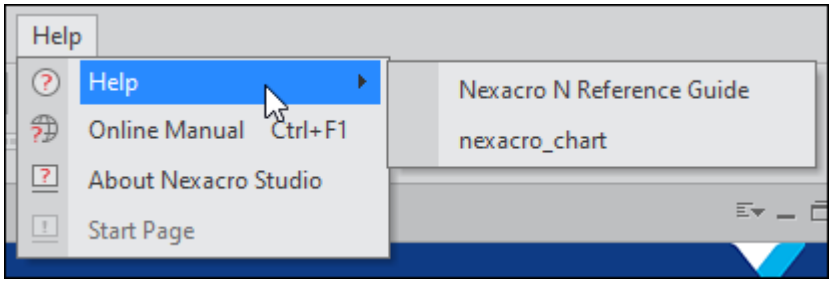


Menu Option	Description
Cascade	Arrange editor windows in a terraced (cascading) form
Tile	Arrange editor windows in a tile form
Arrange Icons	Minimized icon arrangement
Close	Close current editor window
Close All But This	Close all screens except the currently window
Close Tabs to the Right	Close all the tabbed windows on the right of the current window
Close All	Close all editor windows
Reopen Closed Window	Open a recently-closed window in the current project

Menu Option	Description
	Performing this option opens windows one by one. If no window is left to open, this option will be deactivated.
Next	Move to next editor window
Previous	Move to previous editor window
Maximize	Maximize all editor windows

24.2.15 Help



Menu Option	Description
Help	Display the Help Window If you install an additional module like chart components, you can select the help of the added module. 
Online Manual	Open online manuals on a web browser
About Nexacro Studio…	Display the Nexacro Studio information window
Start Page	Display the start page, which shows you recent project list and updates on Nexacro Studio

24.3 Shortcut keys

When developing Nexacro-based apps, you can improve productivity by using the shortcut keys. This sub-chapter explains useful shortcut keys.

24.3.1 Common

Feature name	Shortcut key	Description
New Project	Ctrl + Shift + N	Creates a new project
New	Ctrl + N	Creates a new form
Open Project	Ctrl + Shift + O	Opens an existing project
Open	Ctrl + O	Opens an existing file
Save	Ctrl + S	Saves a revised file
Save All	Ctrl + Shift + S	Saves changes made in all files
Copy	Ctrl + C, Ctrl + Ins	Copies selected items to the clipboard
Cut	Ctrl + X, Shift + Del	Cuts selected items and places them on the clipboard
Paste	Ctrl + V, Shift + Ins	Pastes cut/copied items from the clipboard
Delete	Del	Deletes selected items
Select All	Ctrl + A	Selects all items
Undo	Ctrl + Z, Alt + Back	Undo to the previous action
Redo	Ctrl + Y	Re-perform the last action that was "undone"
Find in Files	Ctrl + Shift + F	Opens the "Find in Files" window
Open File in Project	Alt + Shift + O	Opens the "Open File in Project" window
Project Explorer	Ctrl + Alt + L	Opens the "Project Explorer" pane
Resource Explorer	Ctrl + Shift + E	Opens the "Resource Explorer" pane
Properties	Alt + Enter	Opens the "Properties" pane
Computed Style	Ctrl + Alt + C	Opens the "Computed Style" pane
Bookmark	Ctrl + W	Opens the "Bookmark" pane
Dataset Editor	Ctrl + D	Opens the "Dataset Editor" window
Output	Alt + 2	Opens the "Output" pane
Error List	Ctrl + \	Opens the "Error List" pane
Help	F1	Opens the Nexacro Help (Reference Guide)
Online Manual	Ctrl + F1	Opens the online manuals of Nexacro through a web browser
Launch (Screen)	Ctrl + F5	Executes the current project based on a designated screen
Exit	Alt + F4	Exits Nexacro Studio

24.3.2 Window

Feature name	Shortcut key	Description
Next Window	Ctrl + Tab	Activates the next open file
Previous Window	Ctrl + Shift + Tab	Activates the previous open file
Expand Max View	Ctrl + Shift + Enter	Maximize the current window while minimizing all the dockable windows.
Change Tab Next	Ctrl + Page Down	Moves to the tab immediately after the current tab (Design/Source/Script)
Change Tab Previous	Ctrl + Page Up	Moves to a tab immediately before the current tab (Design/Source/Script)

24.3.3 Project Explorer

Feature name	Shortcut key	Description
Rename	F2	Renames selected item on Project Explorer
Search	F3	Selects items that include filter keywords

24.3.4 Properties Window

Feature name	Shortcut key	Description
Show Items	Ctrl + F4	If the selected property value is shown in Combo, the content of Combo is expanded.
Help	F1	Displays indexed help (reference guide) for the selected property or event name.

24.3.5 Form Design

Feature name	Shortcut key	Description
Escape	Esc	Cancels the current task (if there is no current task, selects the superior component of the component currently selected)
Next	Tab	Selects next component indicated by the Tab Order associated with the currently selected component
Previous	Shift + Tab	Selects the previous component indicated by the Tab Order associated with the currently selected component

Feature name	Shortcut key	Description
Move	Arrow keys ↑,↓,←,→	Moves selected components up, down, left, and right
Nudge	Ctrl + Arrow keys ↑,↓,←,→	Minutely moves the selected components up, down, left, and right
Resize	Shift + Arrow keys ↑,↓,←,→	Resizes the selected components
Resize (minutely)	Ctrl + Shift + Arrow keys ↑,↓,←,→	Resizes the selected components minutely
InplaceEdit	F2	Enables changing a selected component's text
Select All	Ctrl + A	Selects all components
Hotkey Editor	Ctrl + H	Opens the Hotkey editor, which allows you to edit hotkeys of all components
Tab Order Editor (View Type)	Ctrl + T	Activates the tab order editing mode (View Type)
Tab Order Editor (List Type)	Ctrl + Shift + D	Opens the Tab Order editor
Bind State View	Ctrl + B	Shows the bind information of components
Quick View	Ctrl + F6	Executes current Forms
Copy ID (Include Parent)	Ctrl + Shift + C	Copies the ID of a selected component
Paste Special	Ctrl + Shift + V	Executes the Paste Special feature
Change Component	Ctrl + Alt + T	Executes Change Component.

24.3.6 Debug

Feature name	Shortcut key	Description
Start Debugging	F5	Starts debugging
Stop Debugging	Shift + F5	Stops debugging
Restart	Ctrl + Shift + F5	Restarts debugging
Step Into	F11	Executes a single instruction on the target (If the instruction is a function call, the debugger steps into the function)
Step Over	F10	Executes the rest of the current function and breaks when the function return is completed.
Step Out	Shift + F11	Executes a single instruction on the target (The current instruction is wholly executed and then the debugger moves to the next instruction)
Run to Cursor	Ctrl + F10	Executes all the instructions up to the current location of a cursor

Feature name	Shortcut key	Description
Start Form Debugging	F6	Debugs a Form

24.3.7 Migration

Feature name	Shortcut key	Description
Migration Log	Ctrl + Shift + M	Opens the Migration Log pane

24.3.8 Source/Script

Feature name	Shortcut key	Description
Find	Ctrl + F	Opens the Find dialog functionality to search for specific text
Find Next	F3	Searches for the next item matching the current search string, relative to the current cursor location
Find Previous	Shift + F3	Searches for the previous item matching the current search string, relative to the current cursor location
Find Word	Ctrl + F3	Searches for the next occurrence of the string selected at the current cursor location
Go to Line	Ctrl + G	Goes to the line whose line number is entered
Match Brace	Ctrl +]	Cursor will move to a brace matching the currently selected brace.
Match Brace (select)	Ctrl + Shift +]	Selects the text enclosed by the currently selected brace and its matching brace
Replace	Ctrl + R	Activates the Find and Replace Dialog to replace a specified text string with another
Select All	Ctrl + A	Selects all in the current file
Comment Selection	Ctrl+/	A selected area is converted into comments
Uncomment Selection	Alt + /	Comment in a selected area will be removed (making code executable).
Lowercase Conversion	Ctrl + U	Converts selected text into lower case
Uppercase Conversion	Ctrl + Shift + U	Converts selected text into upper case
Toggle Bookmark	Ctrl + F2	Creates or removes a bookmark at the cursor location within the current file
Clear Bookmarks	Ctrl + Shift + F2	Clears all bookmarks within the current file
Next Bookmark	F2	Moves the cursor to the next bookmark within the current file
Previous Bookmark	Shift + F2	Moves the cursor to the previous bookmark within the current file
Delete Line	Ctrl + Shift + L	Deletes the line on which the cursor is located
Delete Line (left of the cursor)	Ctrl + Shift + Back	Deletes the contents on the left side of the cursor on the current line

Feature name	Shortcut key	Description
Delete Line (right of the cursor)	Ctrl + Shift + Del	Deletes the contents on the right side of the cursor on the current line
Delete Word (left of the cursor)	Ctrl + Back	Deletes a word or part of a word to the left of the cursor.
Delete Word (right of the cursor)	Ctrl + Del	Deletes a word or part of a word to the right of the cursor.
Tab	Ctrl + Shift + I, Ctrl + I	Identical functionality as Tab
Next Source/Script Window	Ctrl + Shift + F6	Moves to Source/Script window
Move Start Position of Line	Home	Moves the cursor to the beginning of the current line, excluding white space
Select Start Position of Line	Shift + Home	Extends the selection to the beginning of the line
Move End Position of Line	End	Moves the cursor to the end of the current line
Select End Position of Line	Shift + End	Extends the selection to the end of the line
Move Start Position of File	Ctrl + Home	Moves the cursor to the beginning of the current file
Select Start Position of File	Ctrl + Shift + Home	Extends the selection to the beginning of the current file
Move End Position of File	Ctrl + End	Moves the cursor to the end of the current file
Select End Position of File	Ctrl + Shift + End	Extends the selection to the end of the current file
Zoom	Ctrl + -(num), +(num)	Zooms out (-) or in (+), shrinking or enlarging the current view
Zoom Default	Ctrl + /(num)	Apply a scale factor of 100%.
Intellisense	Ctrl + Space	Activates Intellisense
Toggle Breakpoint	F9	Toggles on/off a breakpoint
Delete All Breakpoints	Ctrl + Shift + F9	Removes all breakpoints
E n a b l e / D i s a b l e Breakpoint	Ctrl + F9	Activate/deactivate selected breakpoints
Help	F1	Displays indexed help (reference guide) for properties, methods, and event names selected as a block.

Part II.

QuickCode Core

25.

Create a project with QuickCode samples

QuickCode Tutorial Wizard gives you a quick look at how to configure the screen using QuickCode, configures the necessary development environment, and installs the sample file.

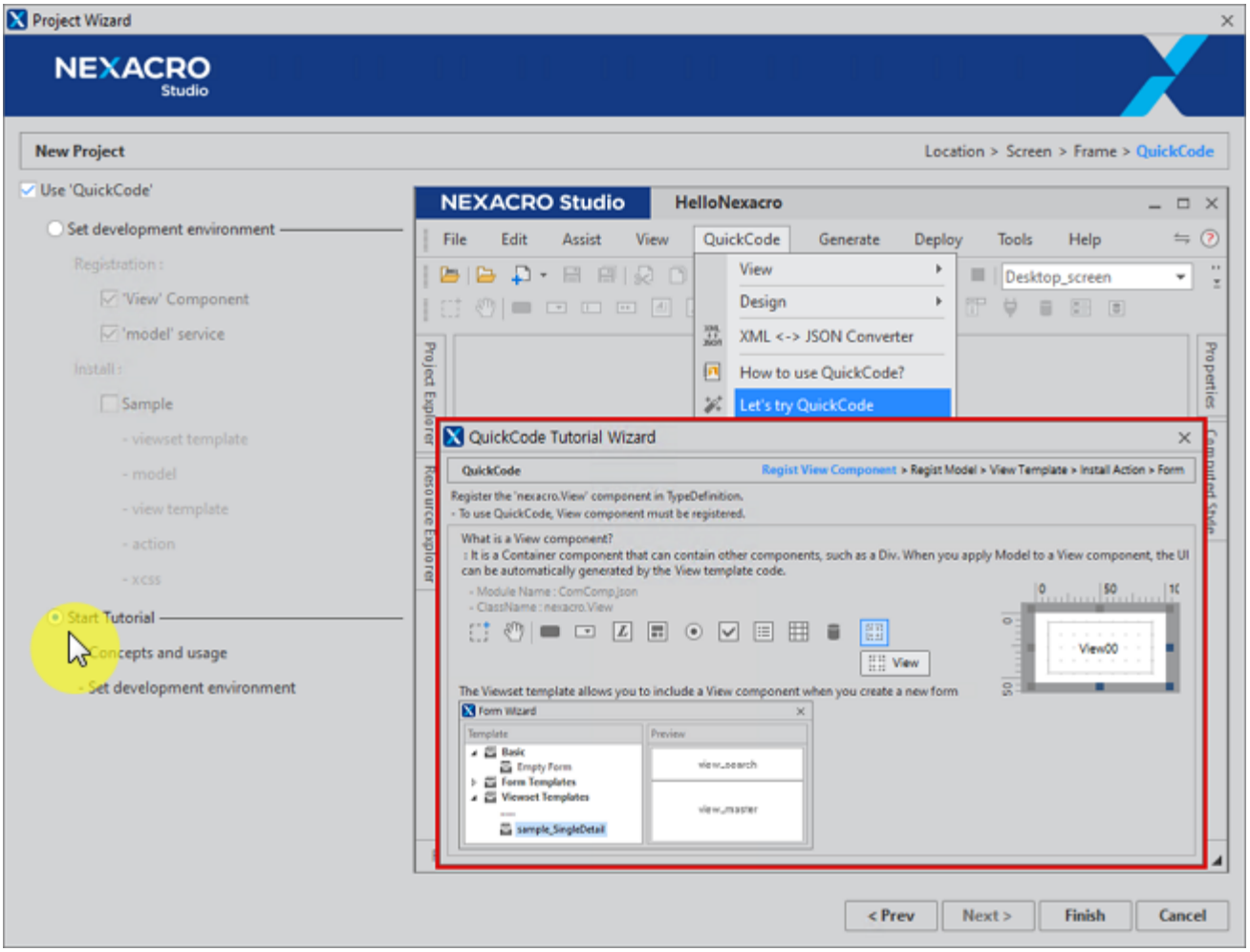
25.1 Running QuickCode Tutorial Wizard

When creating a project, select [Use 'QuickCode' > Start Tutorial] or select the menu [QuickCode > Let's Try QuickCode] with the project opened to run the QuickCode Tutorial Wizard.

25.1.1 Select Use 'QuickCode' in the project wizard

Select the [Use 'QuickCode' > Start Tutorial] item in the QuickCode step and then click the [Finish] button to run the QuickCode Tutorial Wizard.

If you click the [Cancel] button while the QuickCode Tutorial Wizard is running, then you can create a project without setting the QuickCode.

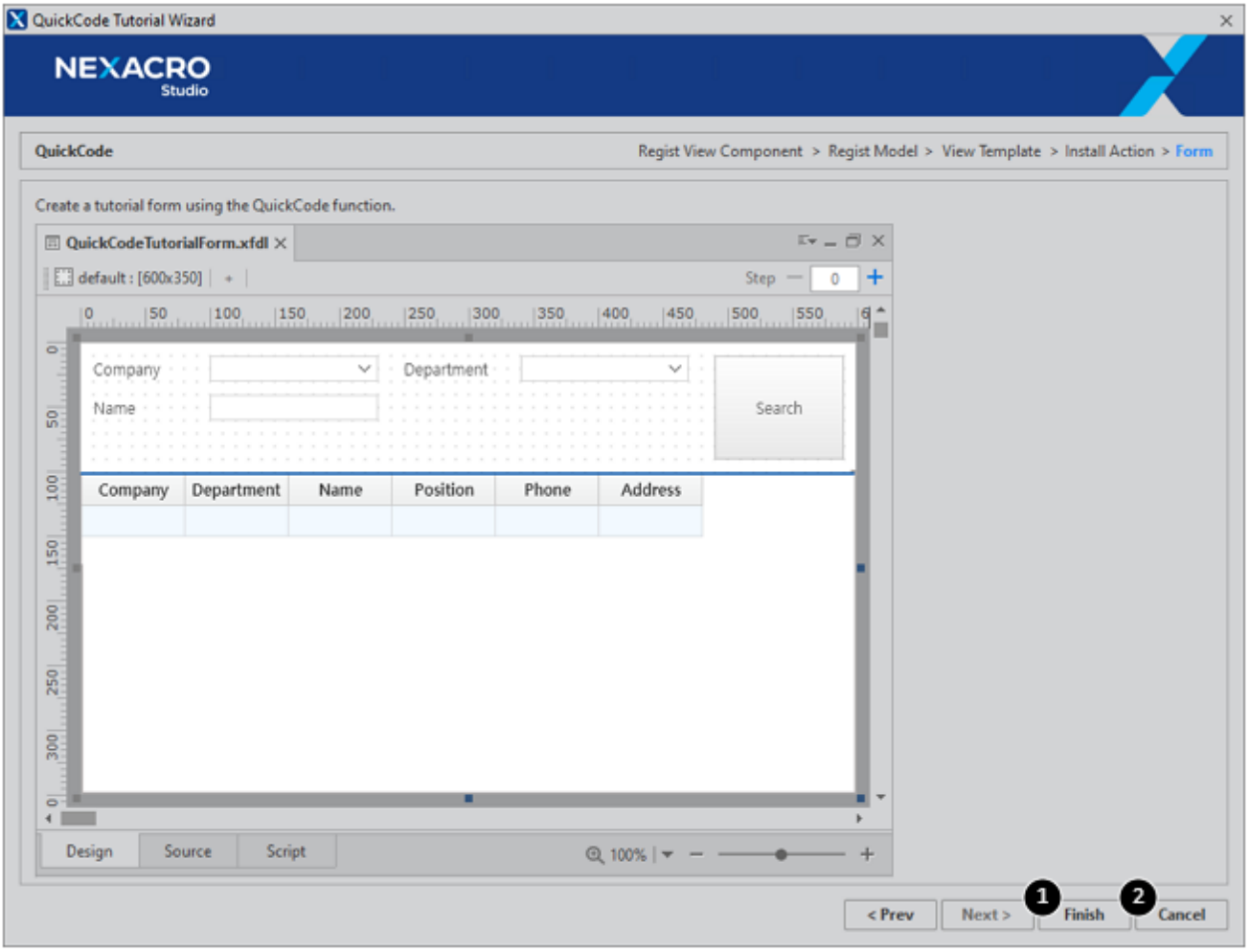


25.1.2 Selecting the Let's Try QuickCode Item from Menu

Select the menu [QuickCode > Let's Try QuickCode] to run the QuickCode Tutorial Wizard. Configure the development environment in the currently open project and then install the sample file.

25.2 QuickCode Tutorial Wizard Execution Screen

The QuickCode Tutorial Wizard does not have a screen to enter the setting values and only consists of a simple function introduction screen. You can browse the contents by clicking the [Next] or [Prev] buttons.



	Description
1	After checking the function introduction content, the [Finish] button is enabled in the final introduction step. When you click the [Finish] button, the development environment is configured and the sample file is installed.
2	If you click the [Cancel] button, then the Wizard execution is ended.

25.3 QuickCode Tutorial Wizard Execution Results

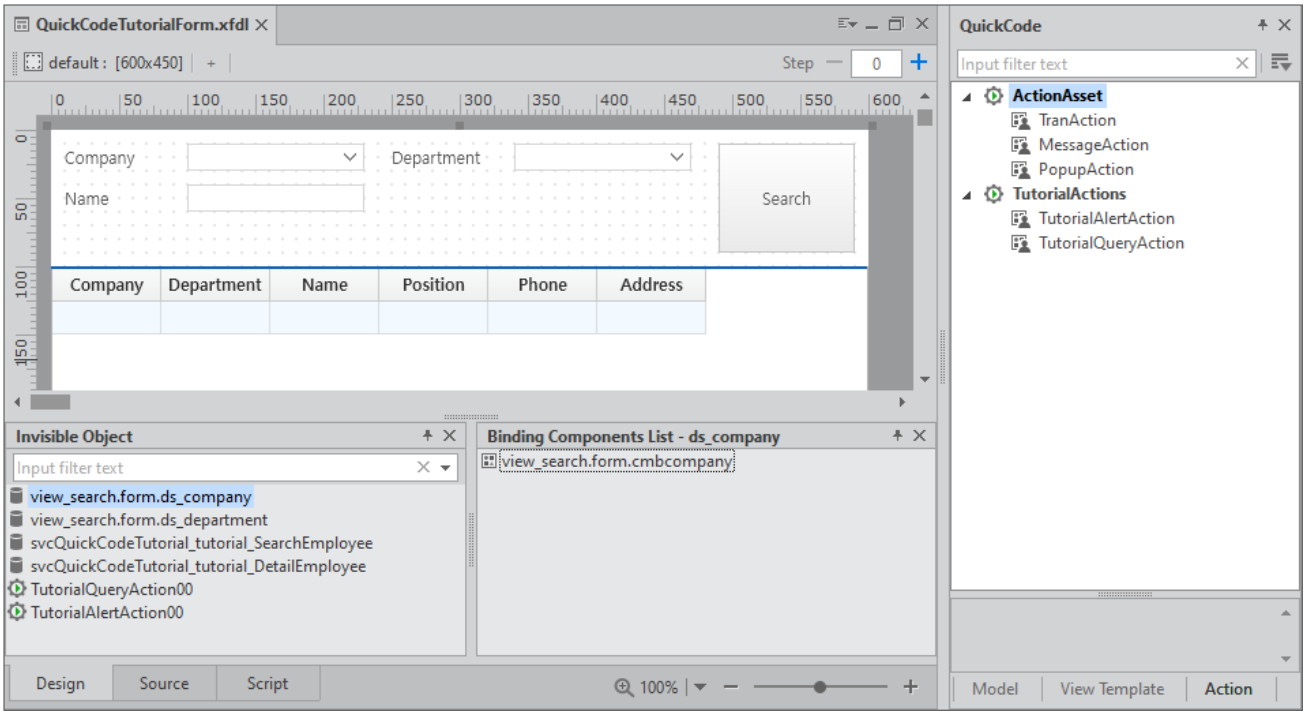
After running the QuickCode Tutorial Wizard and clicking the [Finish] button, the following results are reflected in the project.

- Configures the View component to be available for use and displays it on the toolbar.

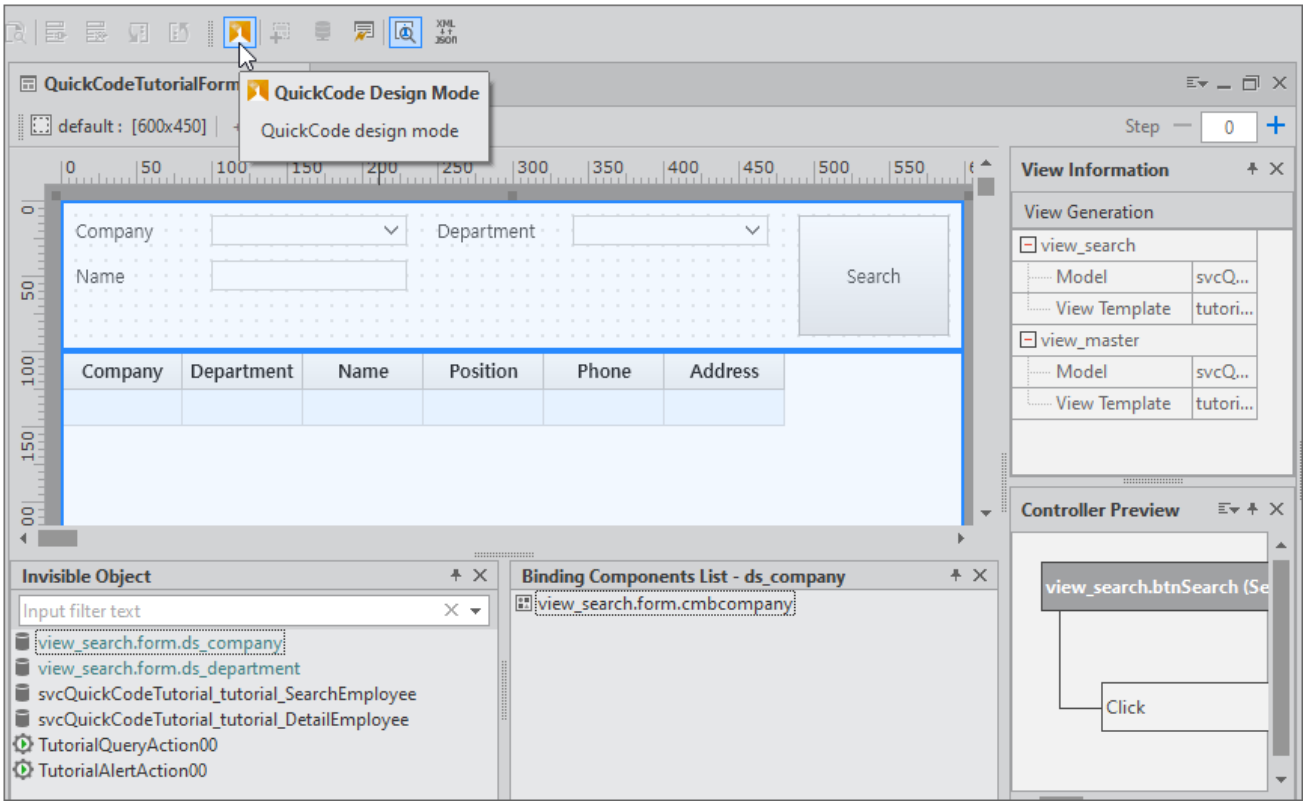
- Displays the QuickView toolbar.
- Registers the model-type service and model samples for running the tutorial.
- Registers View Templates for running the tutorial.
- Installs and registers Action objects for running the tutorial.
- Registers Viewset Templates for running the tutorial.
- Add XCSS file for tutorial execution.
- Also creates the QuickCodeTutorialForm.xfdl file with sample_SingleDetail Viewset Template.

25.3.1 QuickCodeTutorialForm.xfdl

This file is created as a result of running the QuickCode Tutorial Wizard and you can search the data by clicking the [Search] button after running QuickView.



The entire screen consists of two views. View information can be checked by activating QuickCode Design Mode in the toolbar.


QuickCode Design Mode
QuickCode design mode
default : [600x450]
Step
0
Company
Department
Name
Search
Company
Department
Name
Position
Phone
Address
View Information
View Generation
view_search
Model
svcQ...
View Template
tutori...
view_master
Model
svcQ...
View Template
tutori...
Controller Preview
view_search.btnSearch (Se
Click
Invisible Object
Input filter text
view_search.form.ds_company
view_search.form.ds_department
svcQuickCodeTutorial_tutorial_SearchEmployee
svcQuickCodeTutorial_tutorial_DetailEmployee
TutorialQueryAction00
TutorialAlertAction00
Binding Components List - ds_company
view_search.form.cmbcompany

26.

Configure the QuickCode development environment for an existing project and create screens

Let's see how to configure a screen using a simple ViewSet template, View template, Model, and Action.

26.1 Term Definition

This is the term used by QuickCode.

Term	Necessity	Definition
ViewSet Template	X	This is a type of Form template A Form template that includes a View component is called a ViewSet template. Templates are not used to create just one screen, but to quickly create multiple screens with the same layout. Individual screens where it is difficult to apply the ViewSet template can be created by placing the View component directly on the Form object instead of using the ViewSet template.
View Template	O	This template automatically creates other components within the View component. The View template cannot be used independently. Select the View template after selecting the Model, and the appropriate component is automatically placed within the View component according to the Model information. Although the term ‘template’ is used, the screen is created dynamically differently depending on the Model information and the code within the View template.
Model	O	Sets the data information to be handled on the screen. The View template places appropriate components within the View component according to Model information such as label and fieldtype.
Action	X	Events such as screen loading, user input, and data communication are handled. After placing the screen within the View component, an event is set by connecting Action to the Form object or the created component or object.

26.2 Configuring Development Environment

Let’s take a look at how to configure the screen with the previously created ViewSet template, View template, Model, and Action. If you are using the created product, then skip the Creating and starting a new project step and proceed from the Importing ViewSet Template File step.

> In versions 24.0.0.300 and later, the way QuickCode handles the viewdataset property has been changed.
> The following document description and attached files only work properly in versions 24.0.0.300 and later.

26.2.1 Creating and starting a new project

Start by importing all the necessary items in an xpackage file

The xpackage file is created to share necessary resources between projects. By using the xpackage file for QuickCode labs, you can handle the following tasks in one step.

- Adding service and creating the folder (SvcModelSample, Sample)
- Importing model (ModelService.xmodel)
- Importing the View template (viewSampleForm.xviewgen, viewSampleGrid.xviewgen)
- Importing the ViewSet template (viewTemplateSample.xtemplate)
- Creating the sample XML file (sample.xml)

The following steps must be configured.

- Adding the nexacro.View component in TypeDefinition
- Installing the Action module
- Displaying the QuickCode panel

1. Download the xpackage file from the link below.

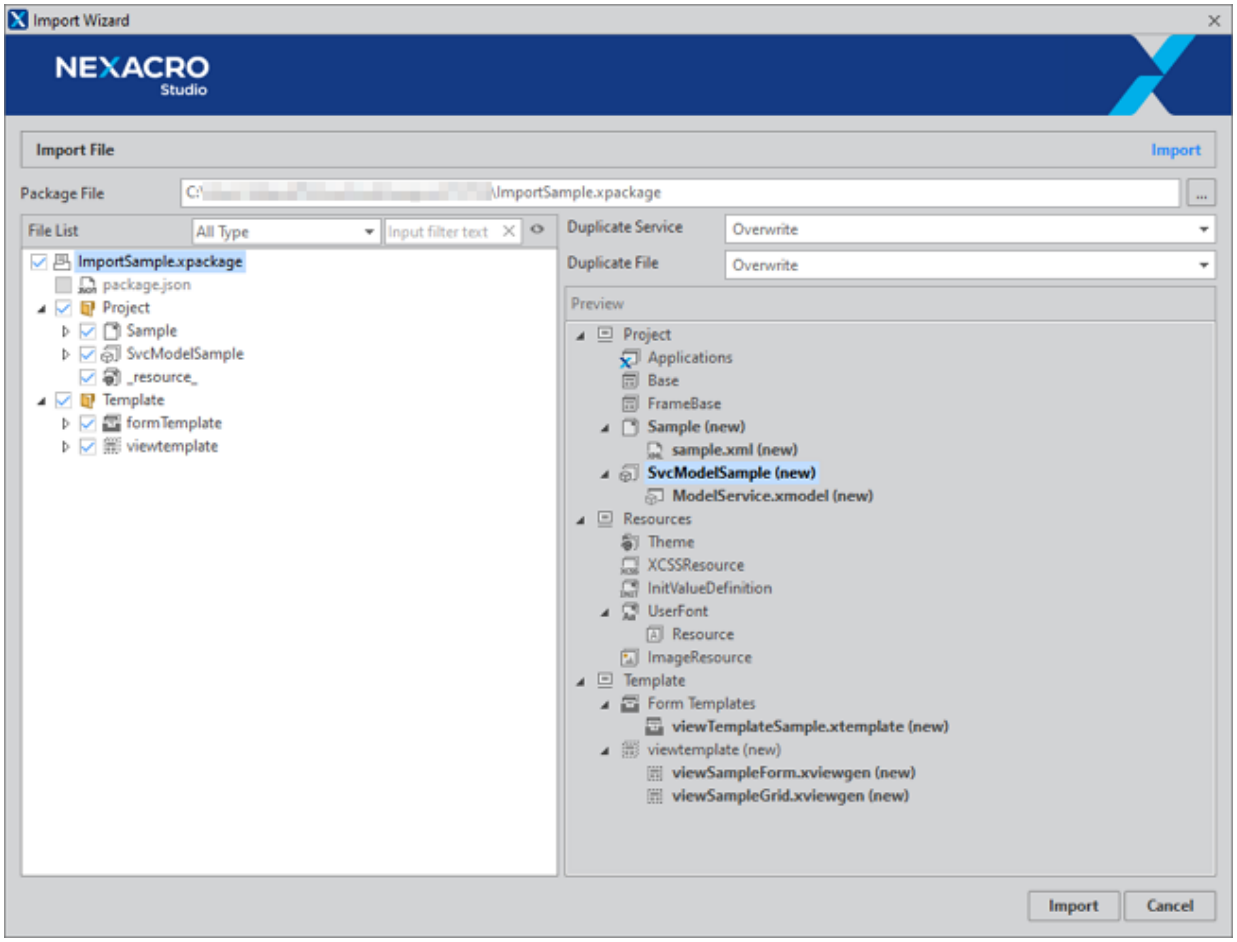
> The xpackage file can be downloaded from the link below
> ImportSample.xpackage
> Please refer to the link below for creating xpackage.
> Export

2. Create a project.

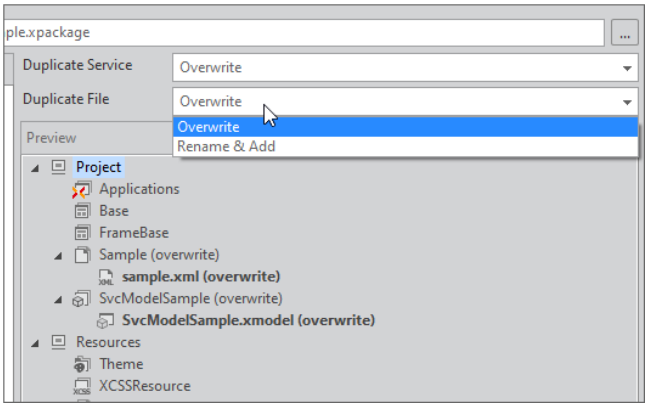
3. Select the menu [File > Import] and run the Import Wizard.

4. Select and enter the downloaded xpackage file in Package File.

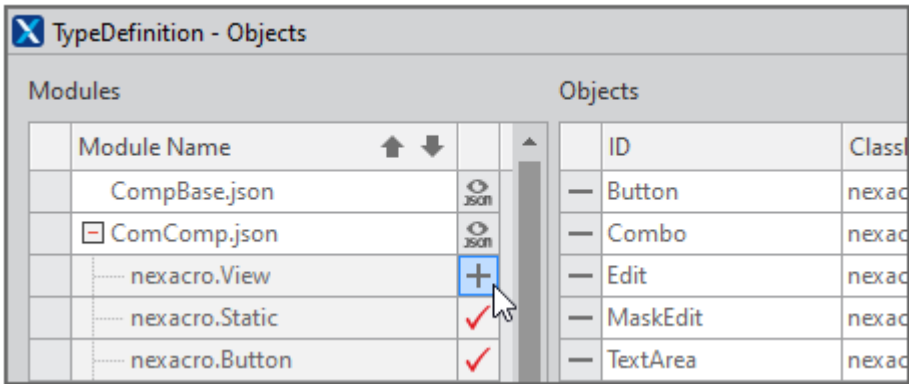
5. Select all the items that can be selected in the FileList and then click the [Import] button.



If the file is the same as the existing item, then it is displayed as overwriting. You can select the option above the Preview screen to change and add the to a different name.



6 Select the TypeDefinition > Objects item in Project Explorer and then add the nexacro.View component.



7 If the QuickCode panel is not displayed, then select the menu [QuickCode > View > QuickCode] and display the QuickCode panel.

INCLUDE_ERROR(ELEMENT_REMOVED)

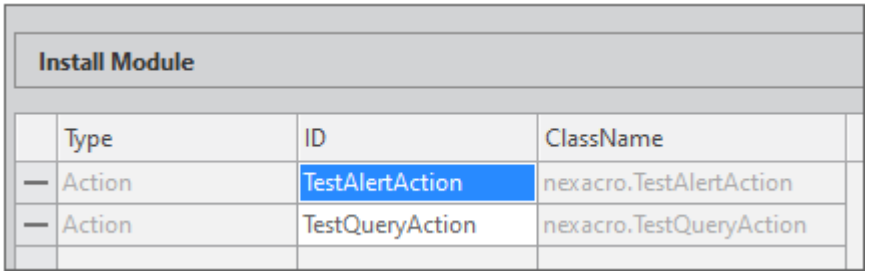
8 Prepare the Action module installation file (SampleActions.xmodule).

You can download the xmodule file from the link below.
SampleActions.xmodule
Please refer to the link below for creating Action.
Creating Action

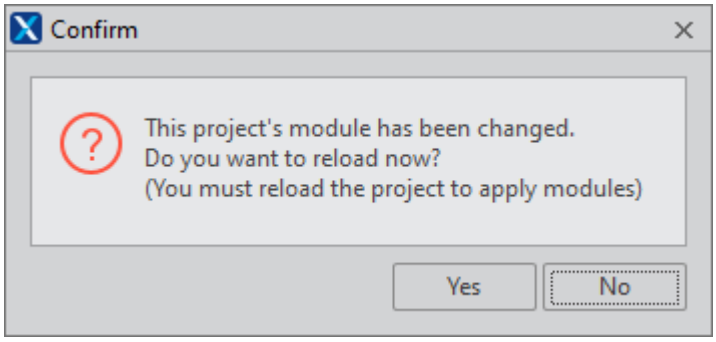
9 Select the menu [File > Install Module] and then run Install Module Wizard.

10 Check "Module Package" for Install Type and then select the xmodule file to import.

11 Check the Action information included in the module and then click the [Install] button.



Refresh the project when installing the module.



12 Select the Action tab in the QuickCode panel and then check if the installed Action module is registered.

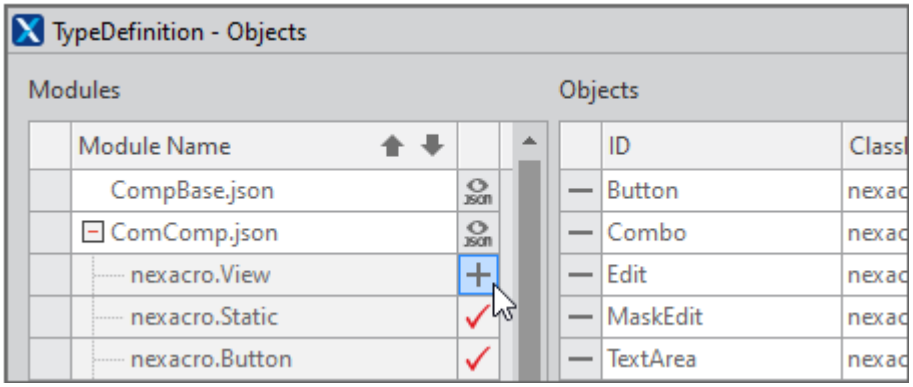
INCLUDE_ERROR(ELEMENT_REMOVED)

13 Continue proceeding with the task of Creating New Form.

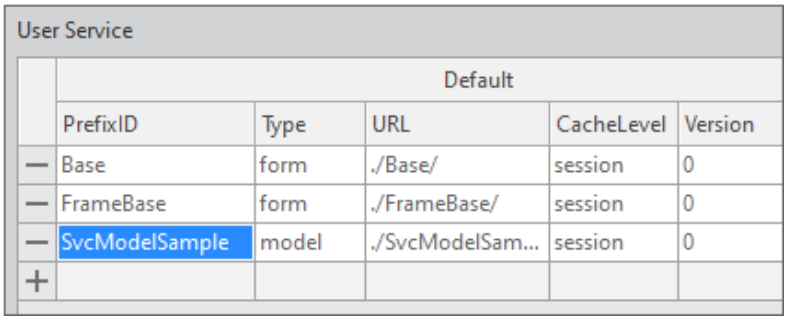
Start by importing each of the necessary items separately

Imports each of the necessary items separately instead of using a xpackage file.

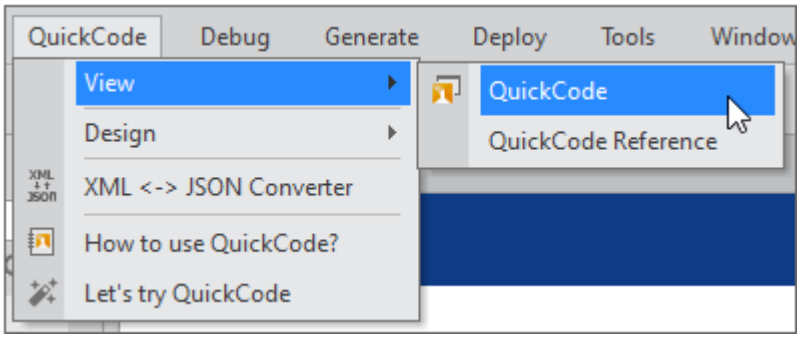
1 Select the TypeDefinition > Objects item in Project Explorer and then add the nexacro.View component.



2 Select the TypeDefinition > Services item in Project Explorer and then add the Model service with the name "SvcModelSample".



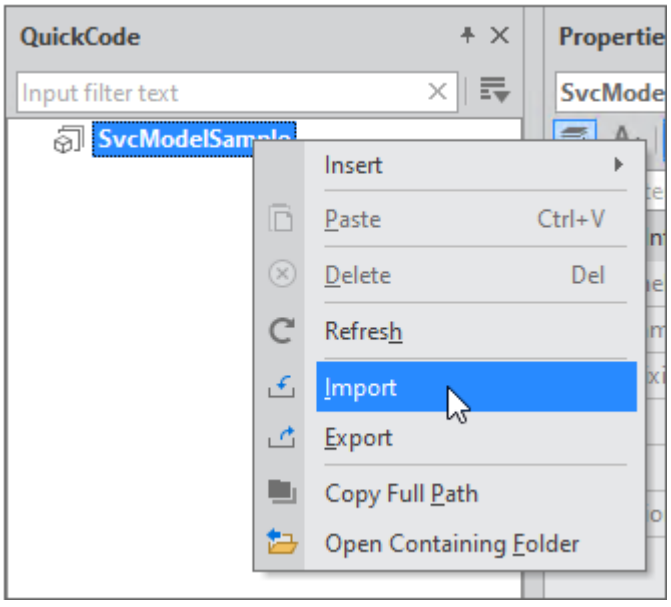
3 If the QuickCode panel is not displayed, then select the menu [QuickCode > View > QuickCode] and display the QuickCode panel.



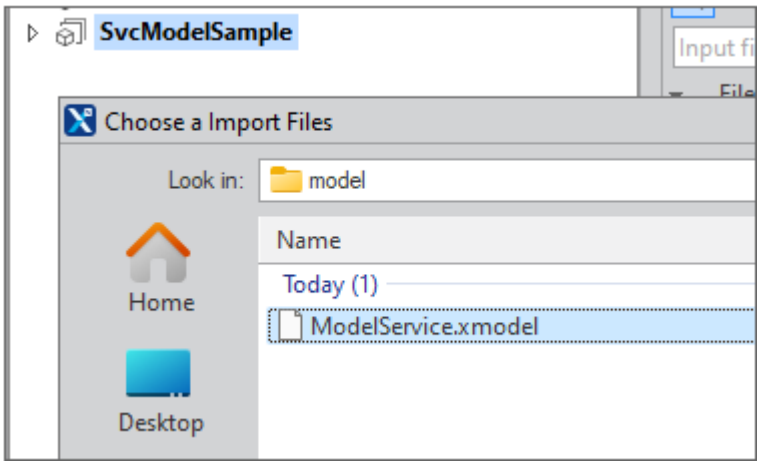
4 Prepare the Model file (ModelService.xmodel).

> You can download the xmodel file from the link below.
> ModelService.xmodel
> Please refer to the link below for creating Model.
> Creating Model

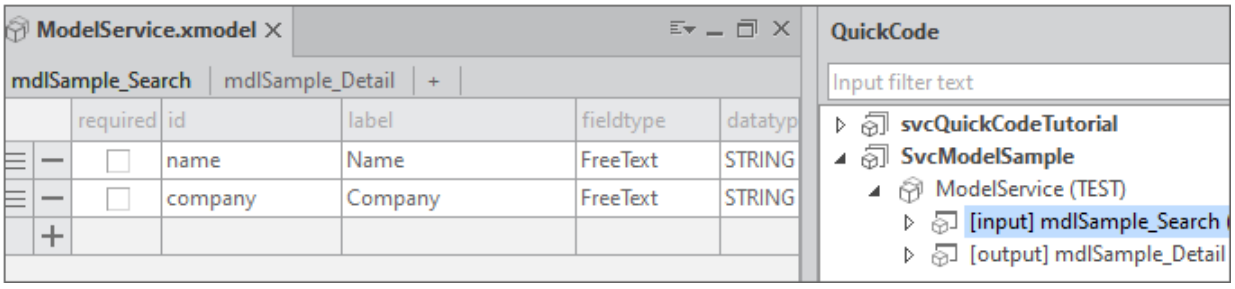
5 Select the Model tab in the QuickCode panel and then select the "Import" item from the context menu.



6 Select the xmodel file to import from the file explorer.



7 Check that Model has been added. You can check the information by double-clicking the Model item.



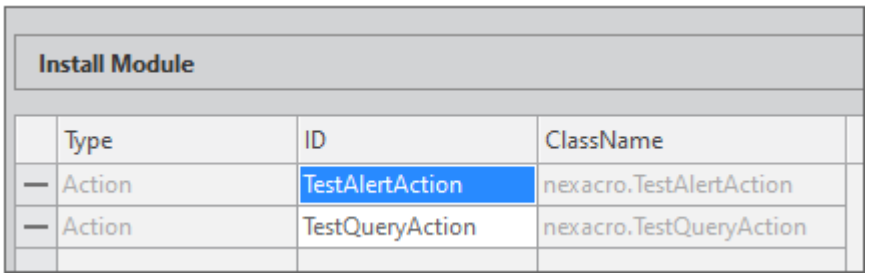
8 Prepare the Action module installation file (SampleActions.xmodule).

You can download the xmodule file from the link below.
SampleActions.xmodule
Please refer to the link below for creating Action.
Creating Action

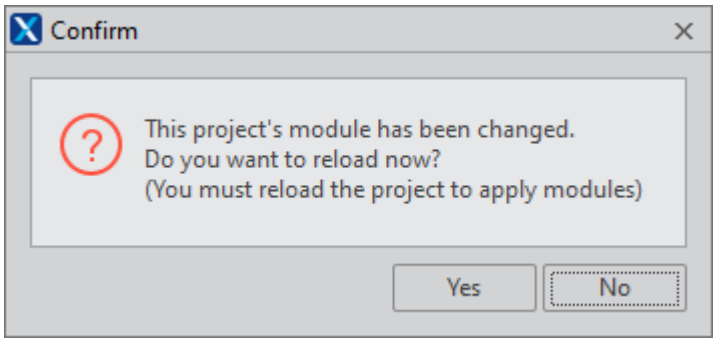
9 Select the menu [File > Install Module] and then run Install Module Wizard.

10 Check "Module Package" for Install Type and then select the xmodule file to import.

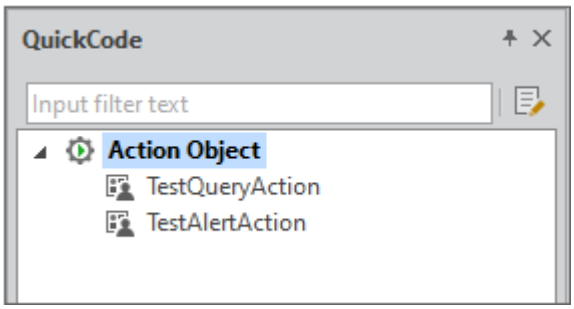
11 Check the Action information included in the module and then click the [Install] button.



Refresh the project when installing the module.



12 Select the Action tab in the QuickCode panel and then check if the installed Action module is registered.



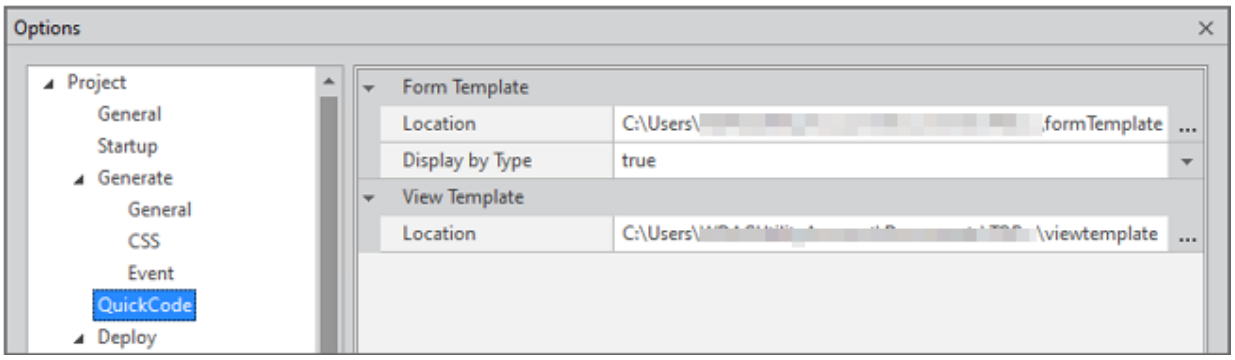
26.2.2 Importing ViewSet Template File

1 Prepare the ViewSet template file.

> You can download the xtemplate file from the link below.
> After extracting, copy the entire folder and then move it under the project Viewset path.
> sample.zip
> Please refer to the link below for creating the Viewset template.
> Registering & Using ViewSet Template

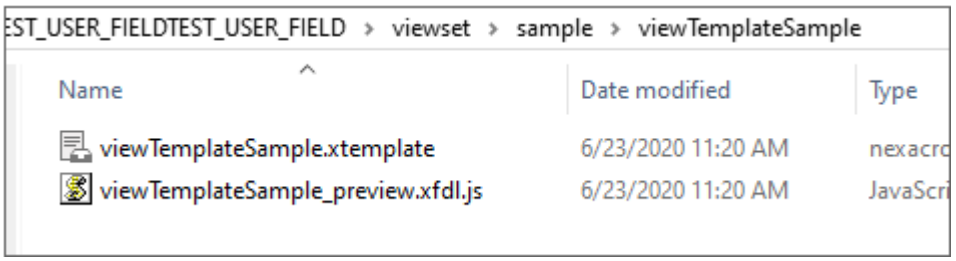
2 Copy the ViewSet template file to the folder.

The default folder where the ViewSet template file is saved is in the viewset folder under the Form Template path in the menu [Tools > Options > QuickCode].



You need to create a category folder (sample) and the ViewSet template folder (viewTemplateSample) under the viewset folder and copy the xtemplate file.

> The _preview.xfdl.js file is the preview file that is automatically created when you select the ViewSet template file for creating a new Form. It is okay to import them together when copying the ViewSet template file.

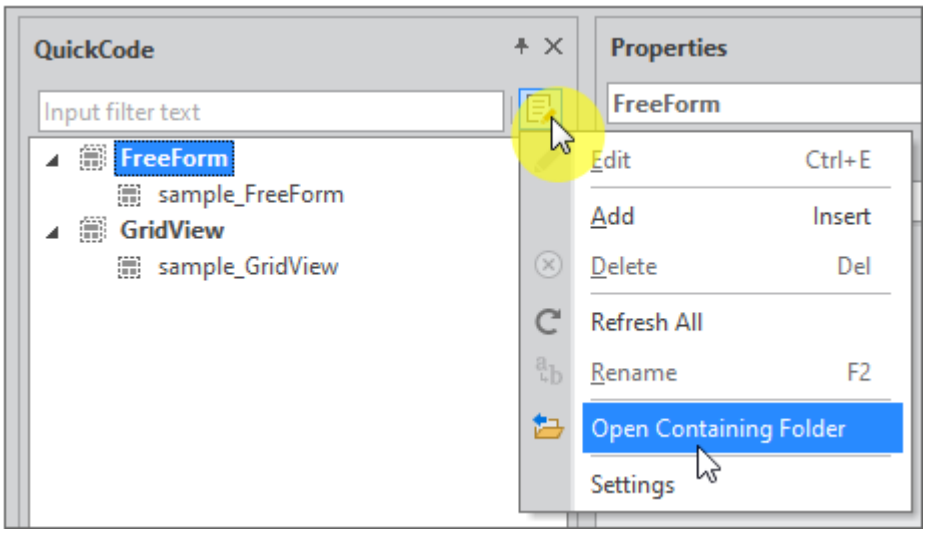


26.2.3 Importing View Template File

1. Prepare the View template file.

> You can download the xviewgen file from the link below.
> viewSampleForm.xviewgen
> viewSampleGrid.xviewgen
> Please refer to the link below for creating the View template.
> Creating View Template

2. Open the View Template tab in the QuickCode panel, and then click the top menu button and select the [Open Containing Folder] item to open the folder.

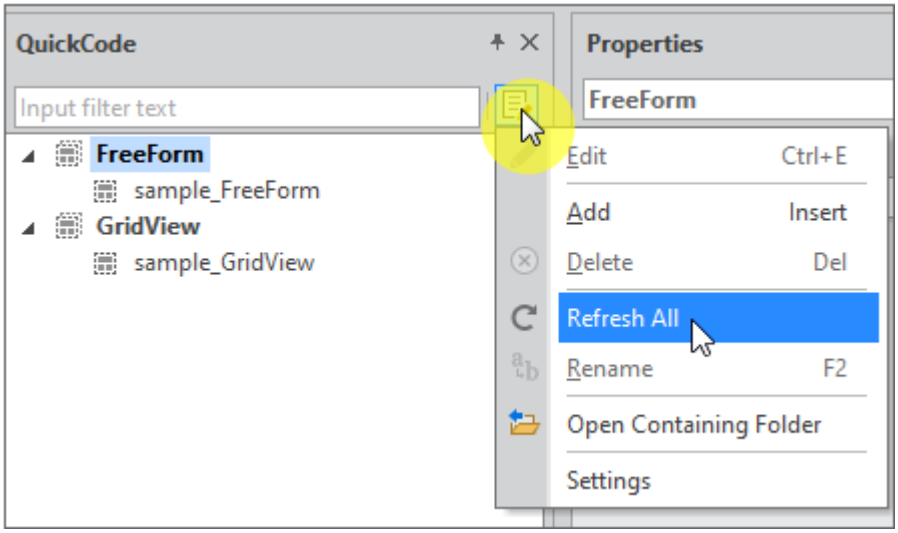




As with the ViewSet template file, the saved default folder can be found in the menu [Tools > Options > QuickCode > View Template]. If you click the Setting button above the Location link in the View Template tab, then it will direct you to the corresponding option.

3. Copy the View template file to the folder.

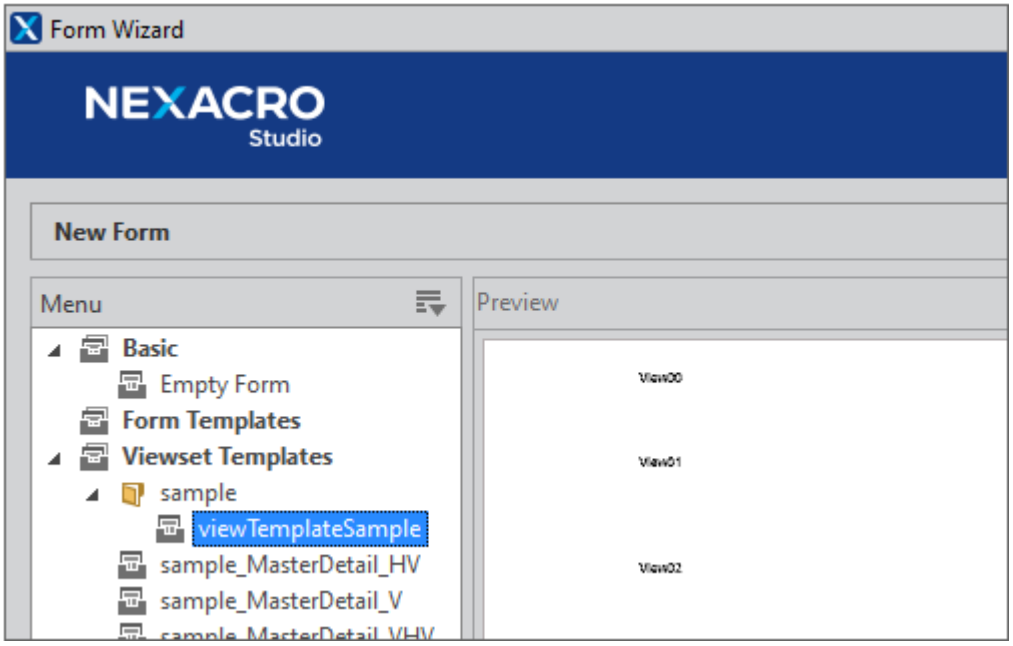
4. After copying the file, click the top menu button in the View Template tab and then select the [Refresh All] item to display the copied View template list.



26.2.4 Creating New Form

1. Select the menu [File > New > Form] and run Form Wizard.

2. Select the Viewset Templates item from the Template list on the left and then select the ViewSet template added earlier.



> In the example, the design of the View component is not included. If properties such as border or background are set on the View component according to the screen design, then they can be added to the theme and reflected.

3. Enter the Form name and then click the [Finish] button.

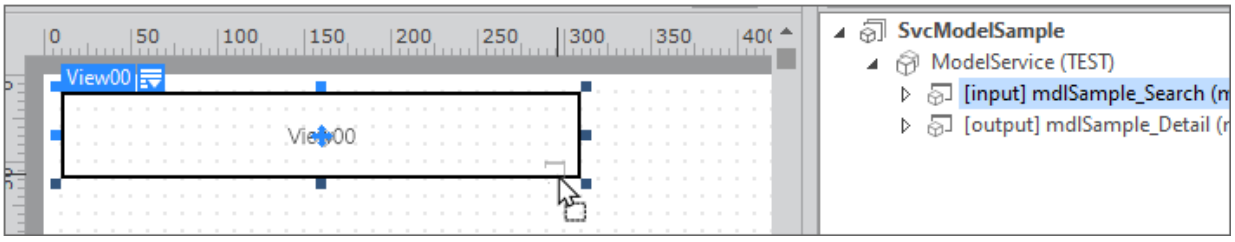
26.3 Creating Screen UI with Model & View Template

After selecting Model and dragging it onto the View component, select the View template and additional properties. The screen UI is created according to the selected item.

26.3.1 Creating Search UI

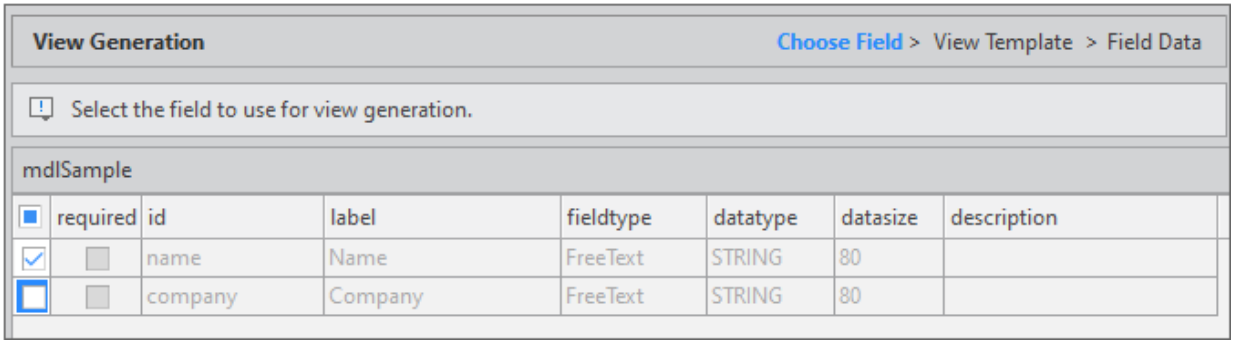
1. Select Model from the QuickCode panel and then drag and drop it on the first View component.

   Drag the Model (mdlSample_Search) over the View component and the area of the View component is displayed in black.



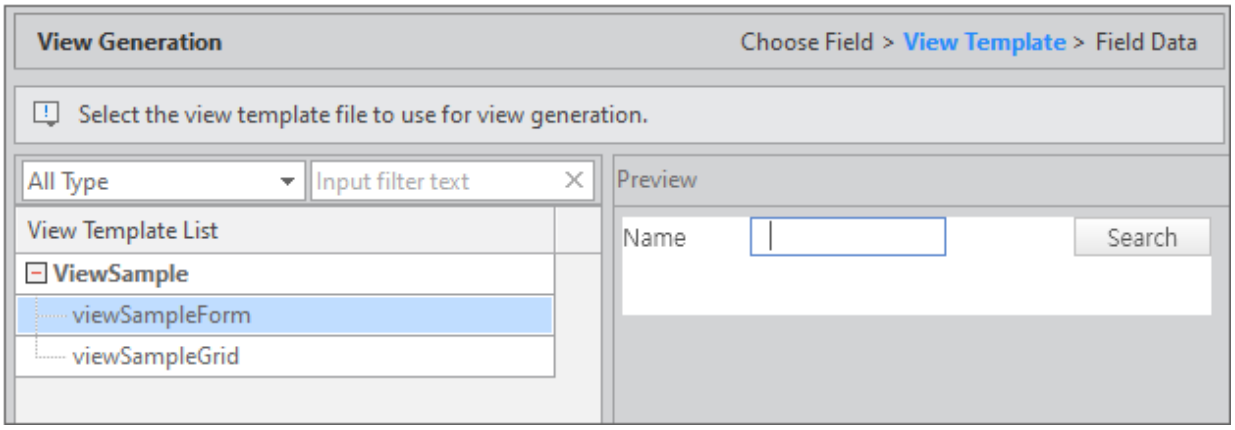
2. Select the field to use.

   Use only the name field. Uncheck the company field.



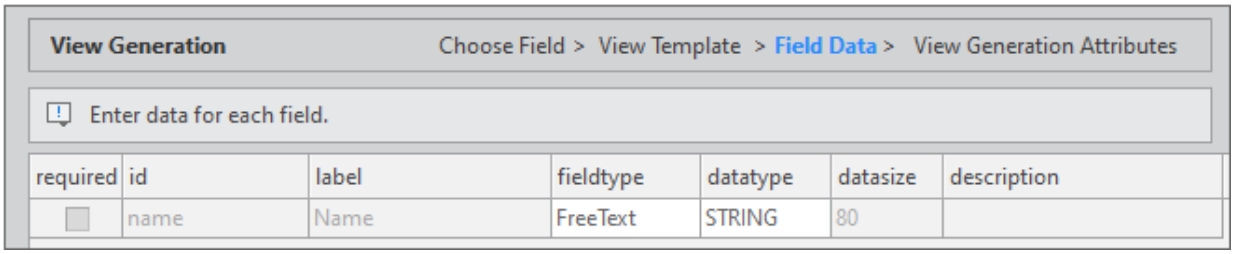
3. Click the [Next] button and select viewSampleForm from the View Template List.

   If you select View Template, you can check the UI image to be created in the Preview area on the right.

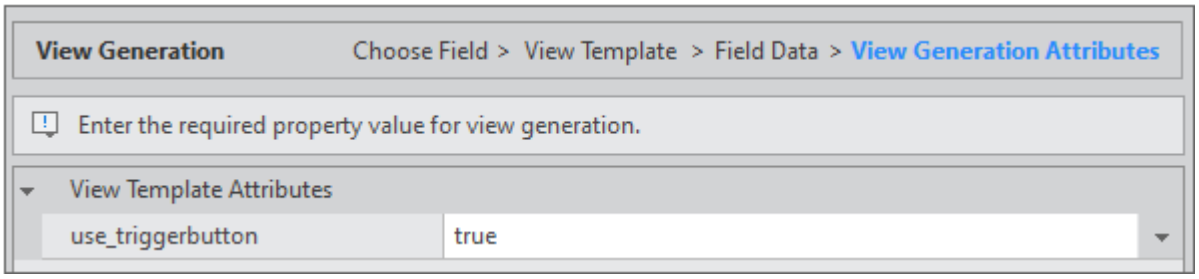


4. Click the [Next] button and check the field data.

You can modify fieldtype and datatype values, or set User Attribute provided by fn_GetFieldUserAttributeList in View Template.



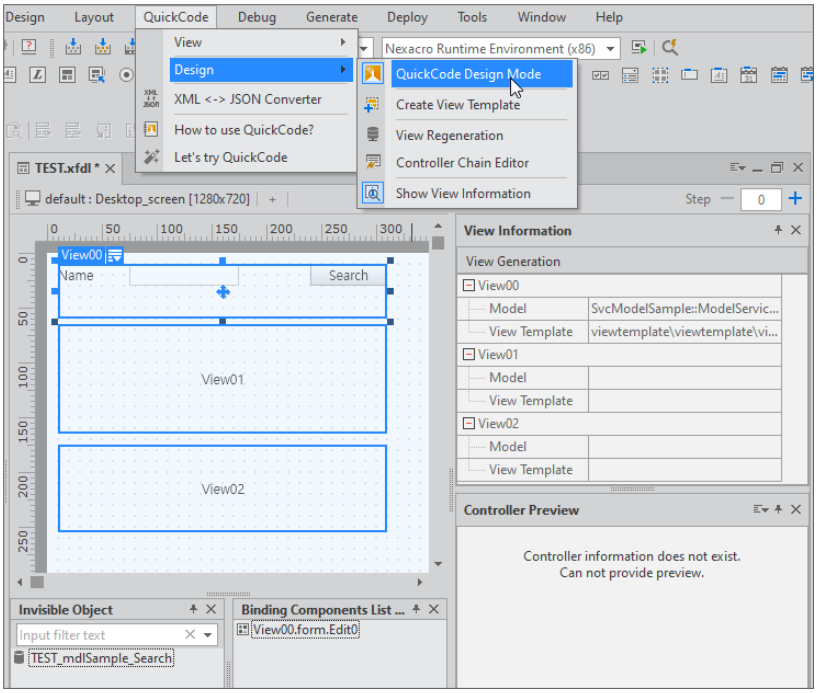
5 Click the [Next] button and select true for the use_triggerbutton item.



6 Click the [Finish] button to create the screen.

7 Check the created View component information in the QuickCode Design Mode.

QuickCode Design Mode is activated by selecting [QuickCode > Design > QuickCode Design Mode] from the menu or selecting QuickCode Design Mode from the toolbar. The View Information panel is displayed on the right side of the Form screen and displays information about each View object.

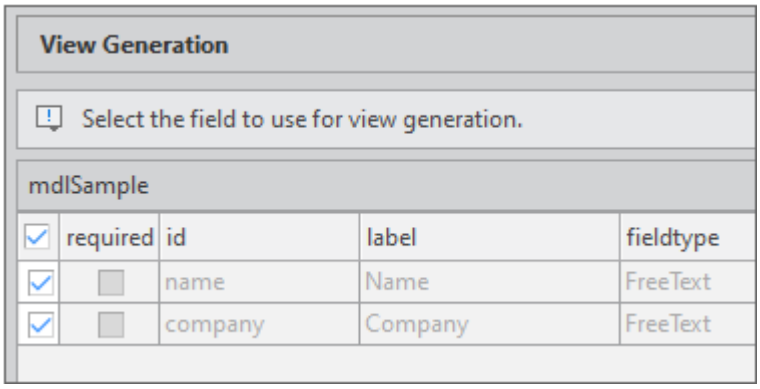


26.3.2 Creating Grid UI

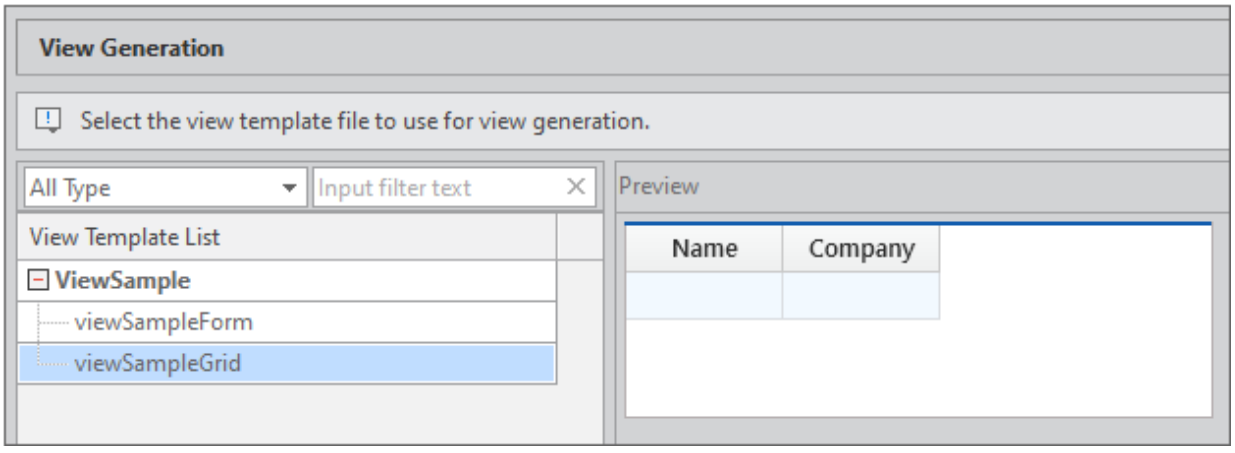
1. Select Model (mdlSample_Detail) from the QuickCode panel and drag it onto the second View component.

2. Select the field to use.

   Keep both fields checked.

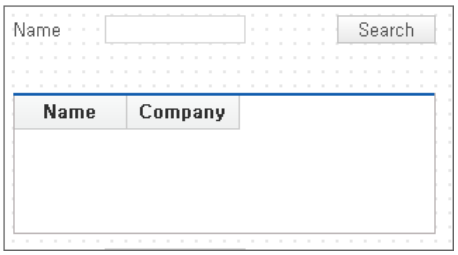


3. Click the [Next] button and select viewSampleGrid from the View Template List.



4. Click the [Next] button and check the field data.

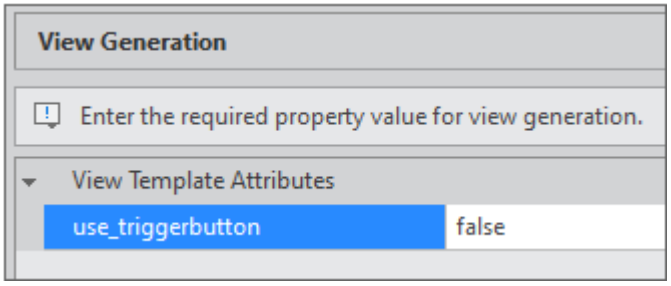
5. Click the [Finish] button to create the screen.



26.3.3 Creating Detailed Information UI

1. Select Model (mdlSample_Detail) from the QuickCode panel and drag it onto the third View component.
2. Select the field to use.

   Keep both fields checked.
3. Click the [Next] button and select viewSampleForm from the View Template List.
4. Click the [Next] button and check the field data.
5. Click the [Next] button and select the value false for the use_triggerbutton item.



6. Click the [Finish] button to create the screen.

   Since the Grid UI and detailed information UI were created by selecting the same Model, they use the same Dataset object and an additional Dataset object is not created.

26.4 Adding Data Search Function Using Action

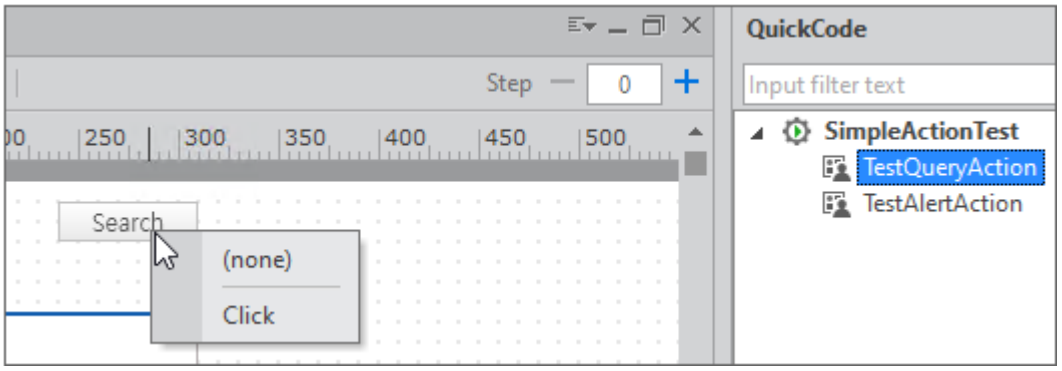
Add a function so that Action operates when the Button component is clicked. In order to implement the data search function without the server connection, create the XML file to search data.

26.4.1 Connecting Action

1. Select the Action tab in the QuickCode panel and select the TestQueryAction item.
2. Drag and drop the selected TestQueryAction item onto the 'Search' Button component placed in the first

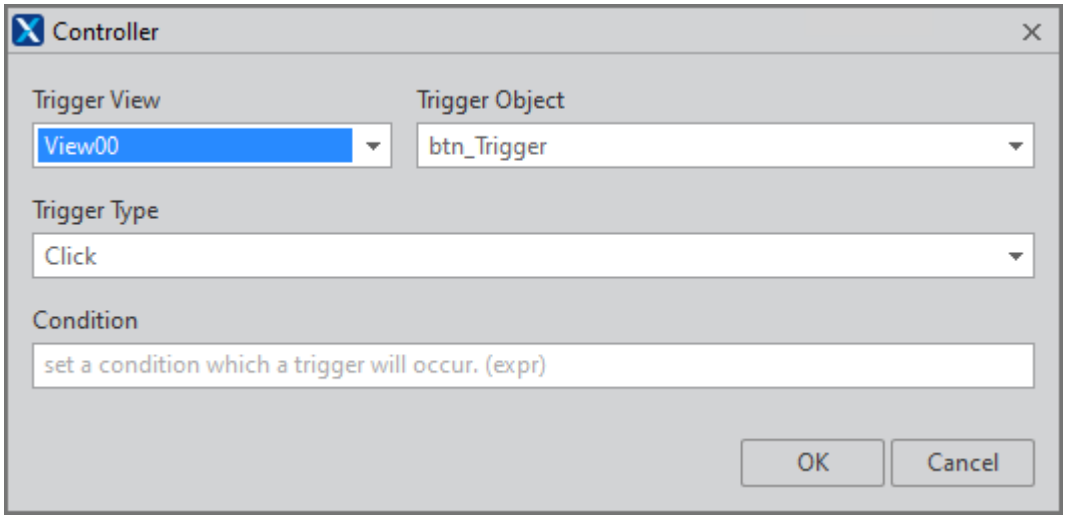
View component.

3 When the context menu appears, select the 'Click' item.



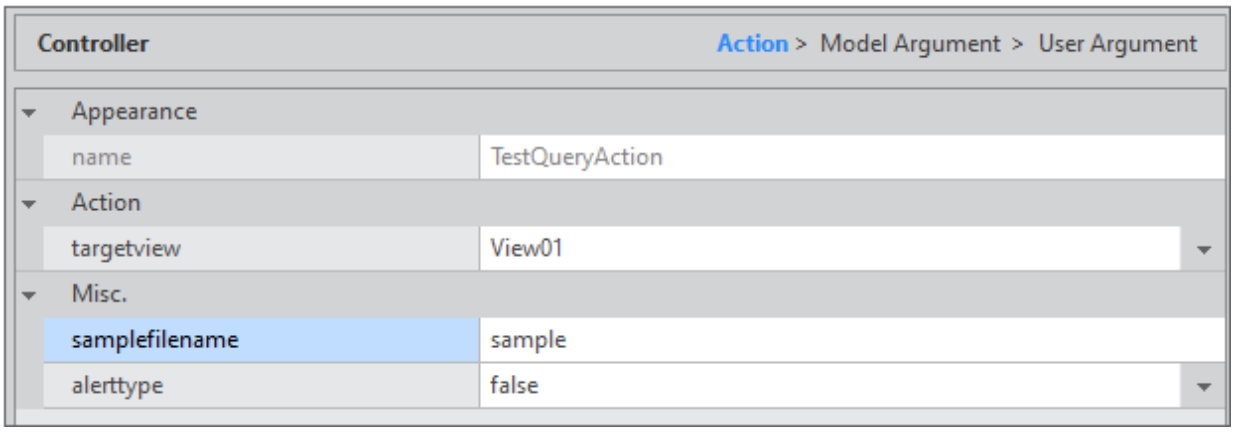
4 Check the trigger type and target and click the [OK] button.

Clicking the Button component with the btn_Trigger id placed in View00 causes a trigger event to occur.



5 Enter action information and other related information to be processed when a trigger event occurs in the Controller Wizard.

Property	value	Description
targetview	View01	View component with Grid component placed on it
samplefilename	sample	XML file name to query information
alerttype	false	Properties to be passed to the action to be processed after data retrieval



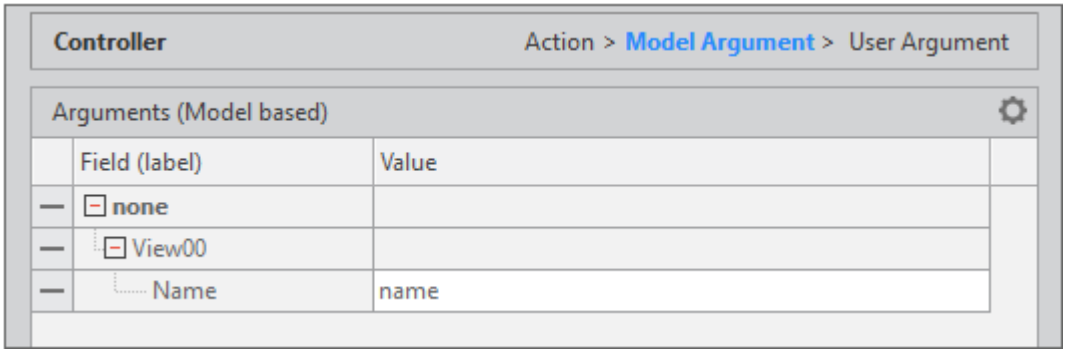
Please refer to the link below for the description of the items that can be selected for the Trigger type.

Item Information Available for the Trigger Type

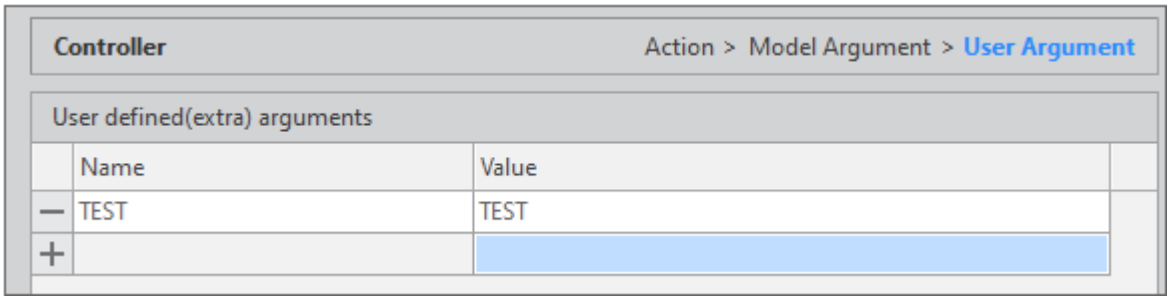
Please refer to the link below for the description of the items that can be added as the Trigger condition.

Trigger Condition Reserved Words & Samples

6 Click the [Next] button and check the field information to be passed when a trigger event occurs.



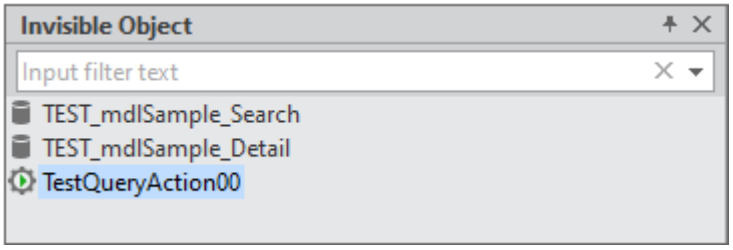
7 Click the [Next] button and add User defined (extra) arguments.



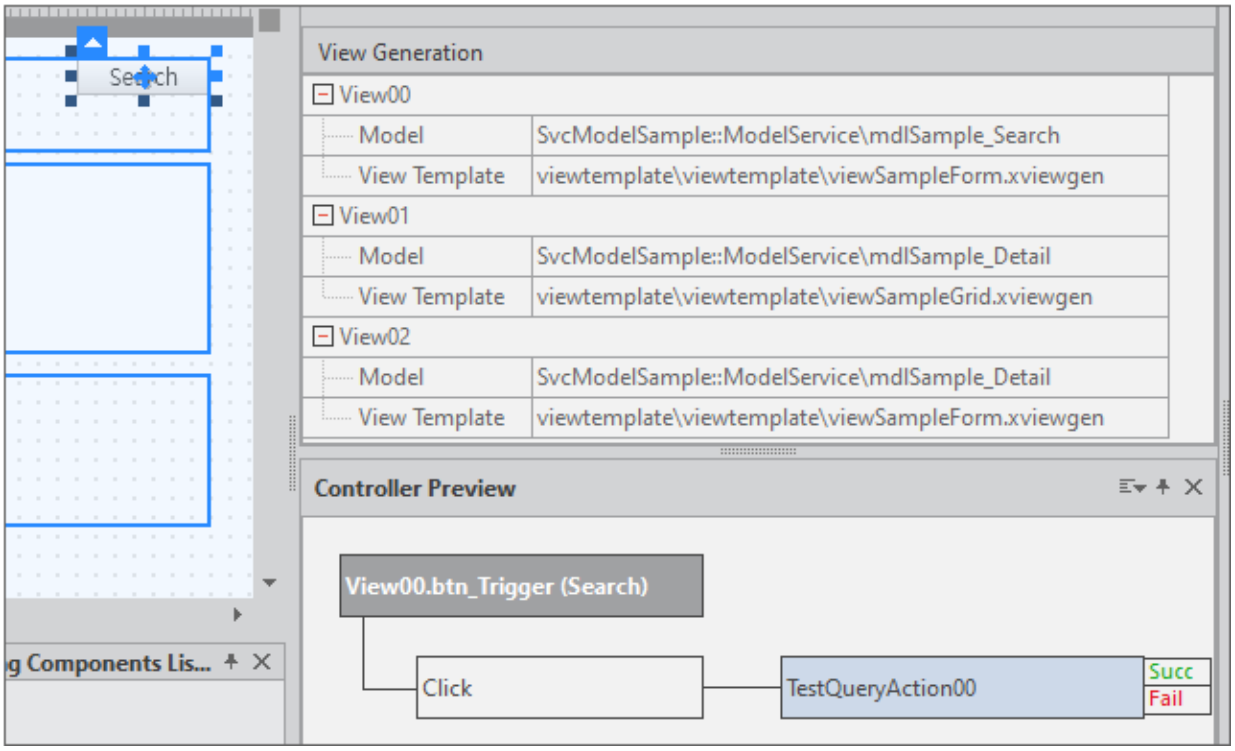
In the example, the transmitted value is only displayed with the trace method.

8 Click the [Finish] button.

You can see that the Action has been added to the Invisible Object area.



If you select the button of the first View component in QuickCode Design mode, you can check the controller information added to the Controller Preview panel.



26.4.2 Creating sample XML File

Place the XML file for simple testing in the local environment.

1 Select the TypeDefinition > Services item in Project Explorer and then add the File service with the name "Sample".

2. Select the menu [File > New > XML] and then enter the file name as "sample".

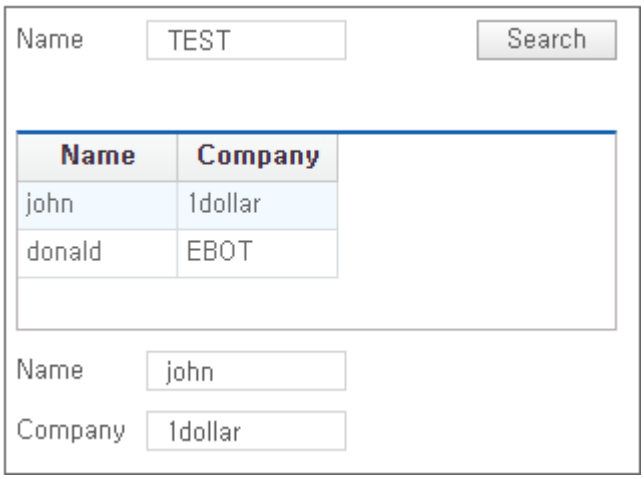
3. Write the XML code with the following contents in the folder.

```
<?xml version="1.0" encoding="utf-8"?>
<Root xmlns="http://www.nexacroplatform.com/platform/dataset" ver="5000" >
     <Parameters>
          <Parameter id="ErrorCode" type="int">0</Parameter>
          <Parameter id="ErrorMsg" type="string">SUCC</Parameter>
     </Parameters>
     <Dataset id="ds_TEST_mdlSample_Detail">
          <ColumnInfo>
               <Column id="name" type="STRING" size="80"/>
               <Column id="company" type="STRING" size="80"/>
          </ColumnInfo>
          <Rows>
               <Row>
                    <Col id="name">john</Col>
                    <Col id="company">1dollar</Col>
               </Row>
               <Row>
                    <Col id="name">donald</Col>
                    <Col id="company">EBOT</Col>
               </Row>
          </Rows>
     </Dataset>
</Root>
```

26.4.3 Executing Screen

1. Click the Quick View button and then execute the screen in the browser.

2. Enter any value in the Name field and click the [Search] button.



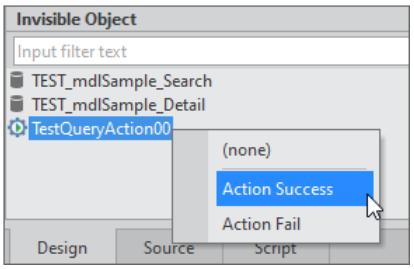
When changing the Row selected in the Grid component, check whether the information displayed in the third View component also changes.

26.5 Adding Action Processed when Data Search Action is Successful

You can specify the Action that should be connected and processed when the required function in the specific Action succeeds or fails. When data search is successful in the previous step, Add Action that displays the Alert message.

26.5.1 Connecting Action

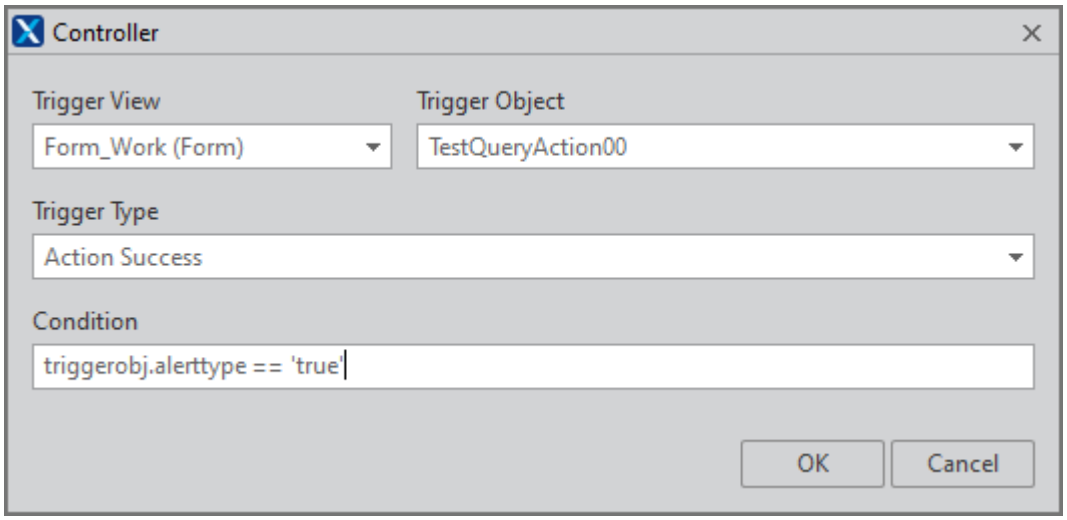
1. Select the Action tab in the QuickCode panel and then select the TestAlertAction item.
2. Drag the selected TestAlertAction item onto the TestQueryAction00 item in the Invisible Object.
3. When the context menu appears, select the 'Action Success' item.



4 Check the trigger type and target, enter condition property values, and click the [OK] button.

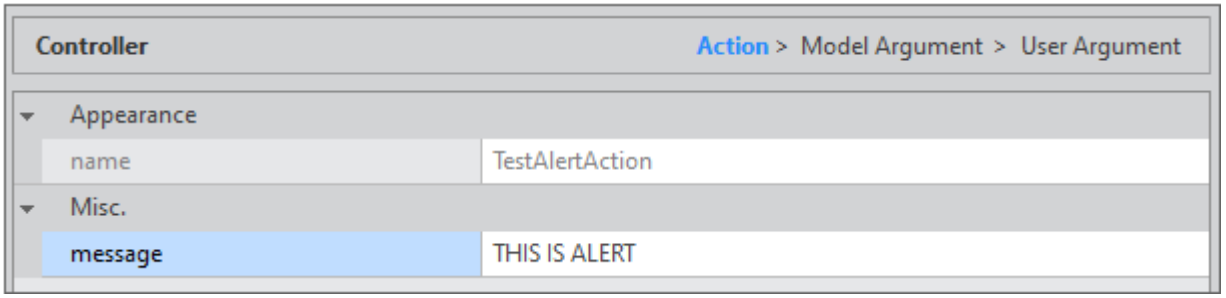
The condition property is the condition under which the action operates. In the data search action, the condition is set to operate only when the alerttype property value is true.

```
triggerobj.alerttype == 'true'
```



5 Enter the message value in Controller Wizard.

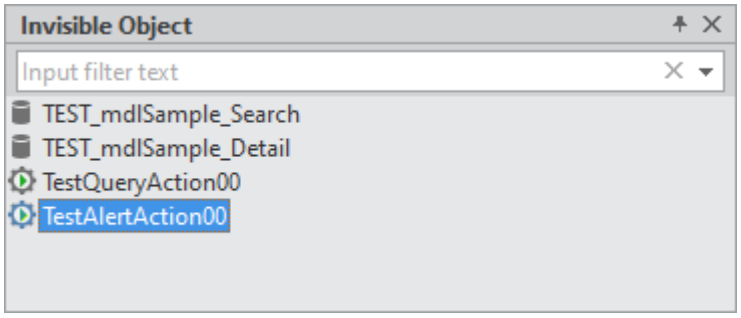
The message is the parameter value to be transmitted when the alert is executed.



Please refer to the link below for the description of the items that can be selected for Trigger type.
Item Information Available for the Trigger Type
Please refer to the link below for the description of the items that can be added as the Trigger condition.
Trigger Condition Reserved Words & Samples

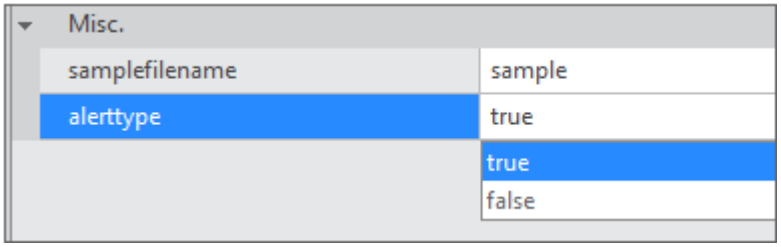
6 Field information passed when an event occurs is not used. After clicking the [Next] button, without moving to the next step, click the [Finish] button to add an Action.

You can see that Action has been added to the Invisible Object area.



26.5.2 Changing Data Search Action Property Value

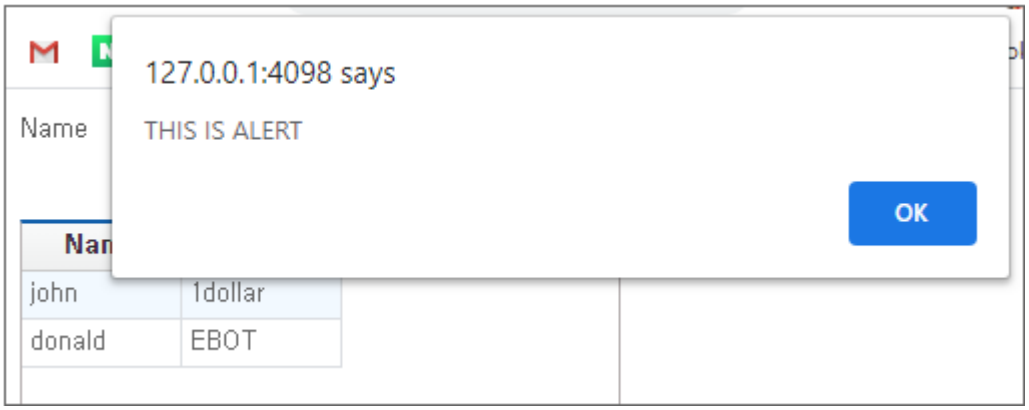
1. Double-click the TestQueryAction00 item in the Invisible Object area.
2. Change the alerttype property value to true.



26.5.3 Executing Screen

1. Click the Quick View button and execute the screen in the browser.
2. Enter any value in the Name field and then click the [Search] button.

   Check that the data is searched and the Alert dialog box is displayed.

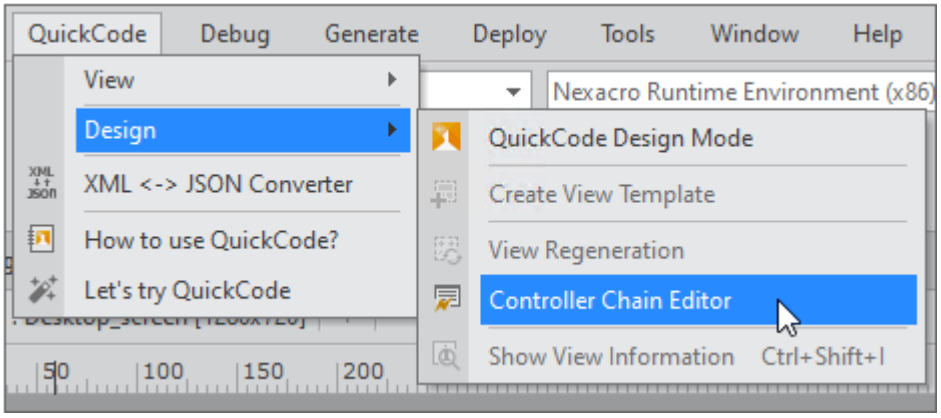


26.6 Adding Only Trigger to Action

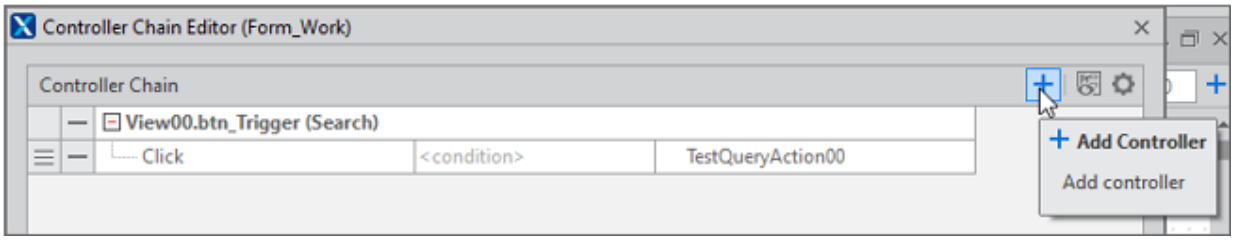
Previously, we added the function to search data when the button is clicked. Add the function to search data when Form is loaded as well as when the button is clicked.

26.6.1 Connecting Action

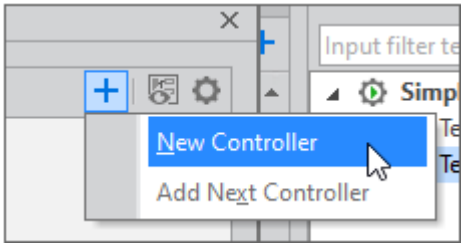
1. Select [QuickCode > Design > Controller Chain Editor] from the menu.



2. Click the "Add Controller" button among the function buttons on the right side of the Controller Chain Editor.



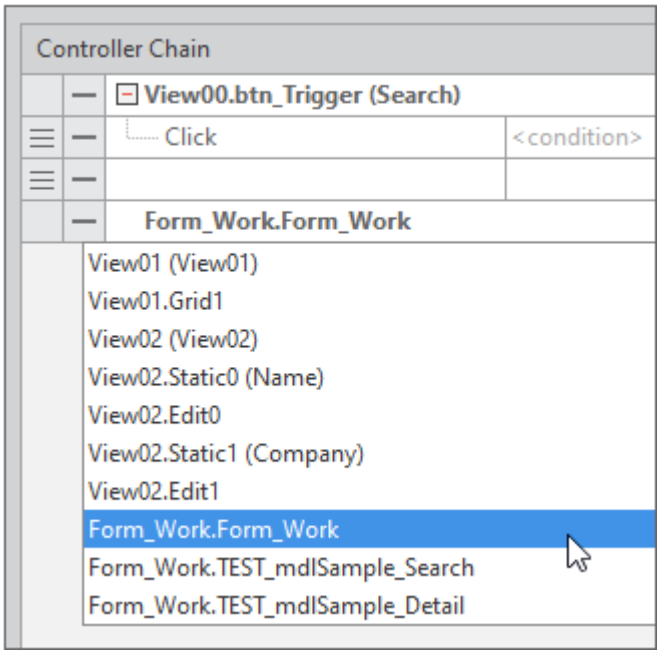
3. Among the sub-items, select the "New Controller" item.



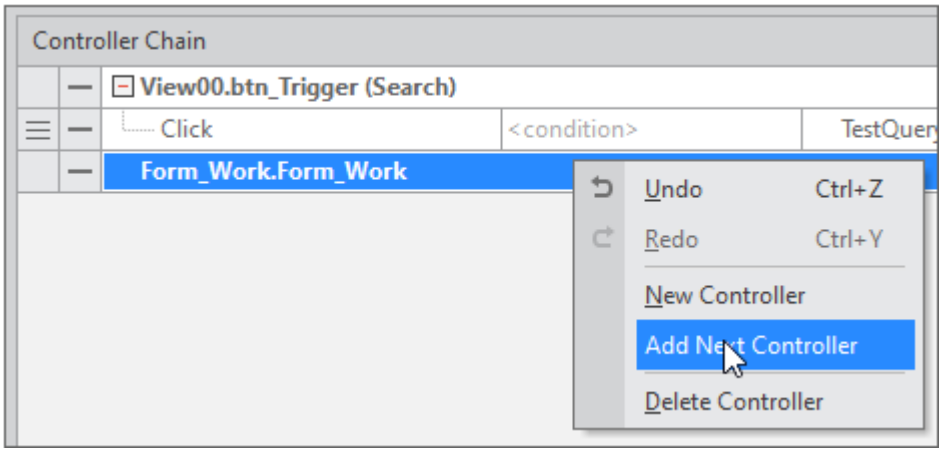
The "Add Next Controller" item becomes active when a controller is selected in the Controller Chain

Editor.

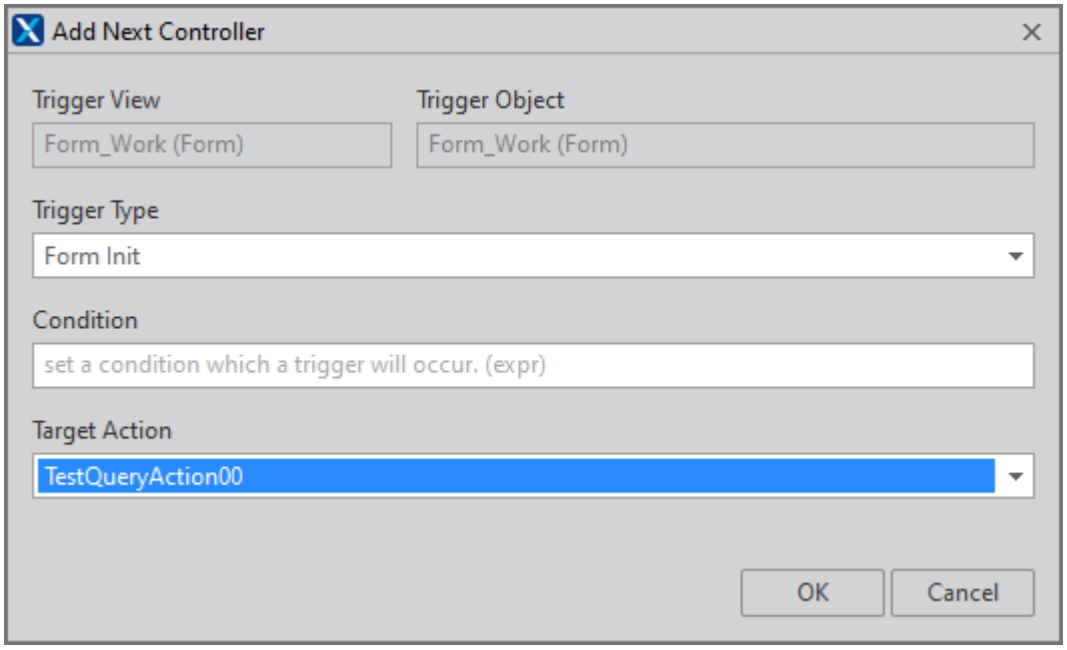
4 Select Form from Trigger View, Trigger Object list.



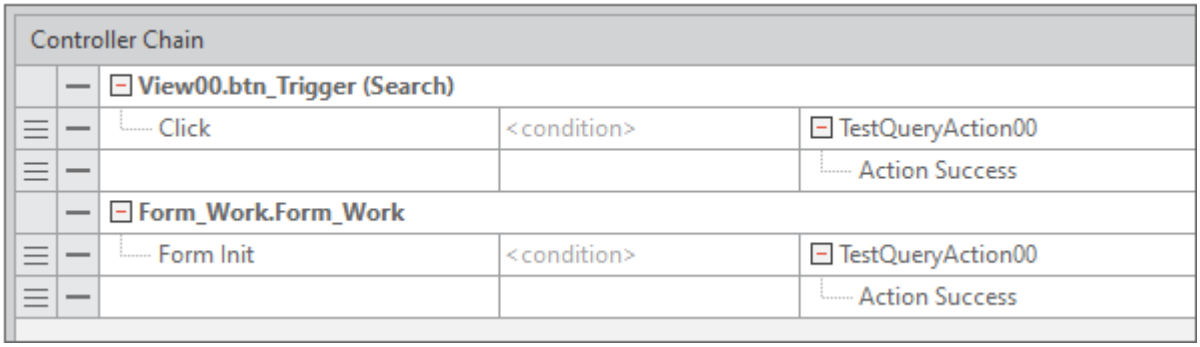
5 With the controller selected, launch the context menu and select the "Add Next Controller" item.



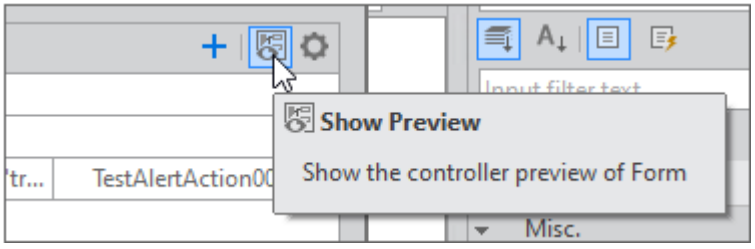
6 Select "Form Init" for Trigger Type and "TestQueryAction00" for Target Action, and click the [OK] button.



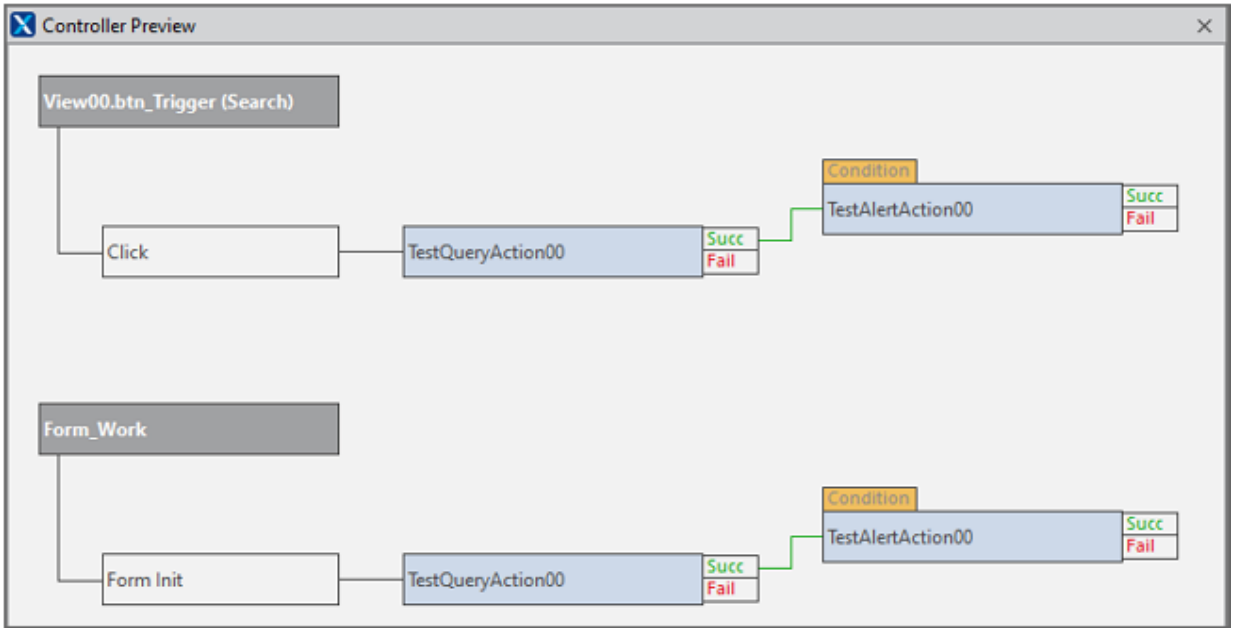
Confirm that a new Controller has been added.



7 Click the "Show Perview" button among the function buttons on the right side of the Controller Chain Editor.



You can check the operation of all registered controllers in advance.



26.6.2 Executing Screen

1. Click the Quick View button and execute the screen in the browser.

   When the Form onload event occurs, the trigger operates and data is searched. You can see that the data is searched without clicking the button.

27.

Develop Model, View Template, and Action

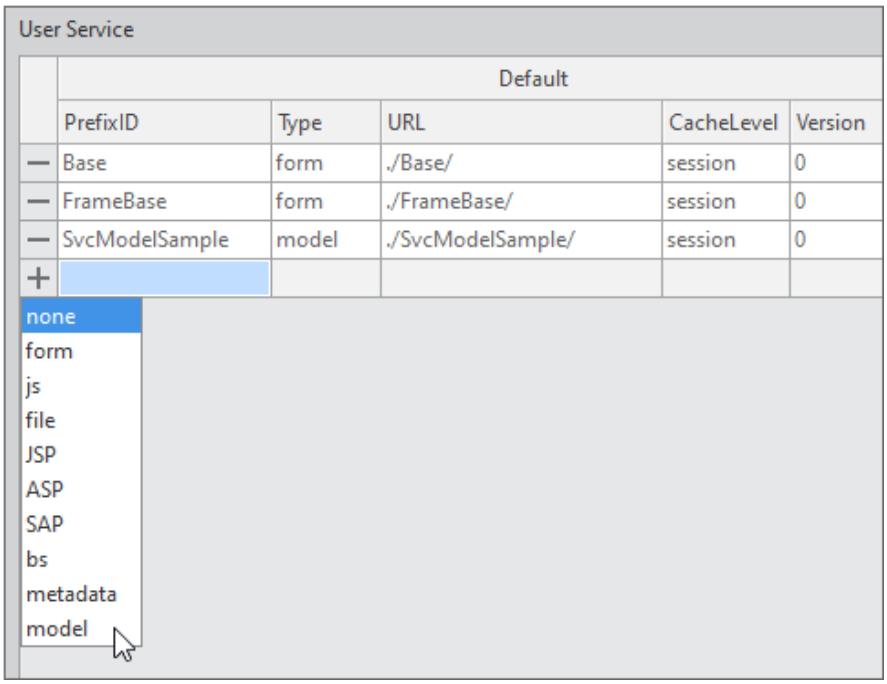
27.1 Creating Model

Model sets the data that must be processed on the screen. Components can be placed according to the Model field value and data can be represented by binding with the Grid component by registering it as the column of the Dataset object. The Model is not used alone and it is used in connection with the View template. When connecting the Model to the View component, the View template is selected and the content is created according to the logic written in the View template.

27.1.1 Adding a service in TypeDefinition

1. Select the [TypeDefinition> Services] item in Project Explorer.

2. Add the service to the User Service item. Select "model" for the Type item and set the PrefixID.





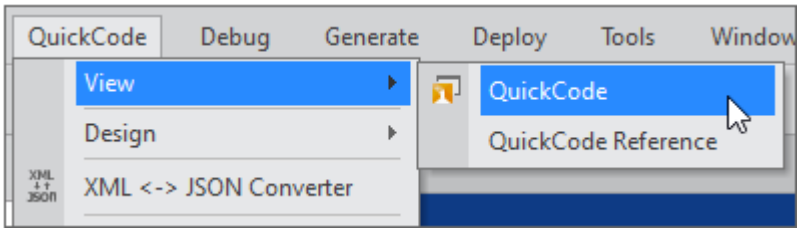
You can also import Model Group, Model Service, and Model data from services set as URL, ServiceList, and DataSchema when registering a model type service. See link below.

Querying the Model service

27.1.2 Adding Model Service items

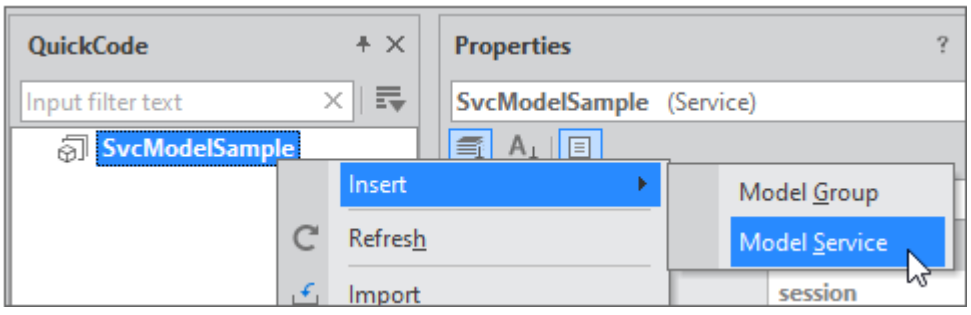
To create a new Model, the QuickCode panel must be enabled.

1. Select the menu [QuickCode > View > QuickCode] item.

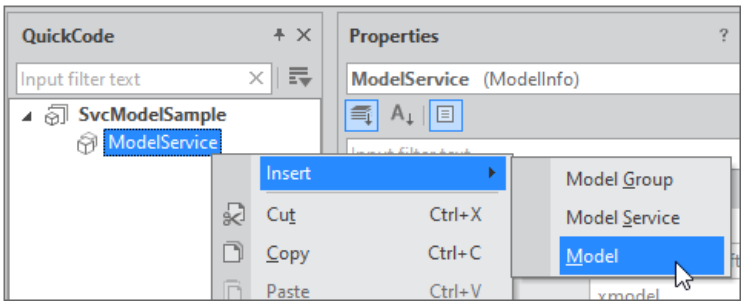


2. Select the Model tab when the QuickCode panel is displayed.

3. Right-click the model type service item added in TypeDefinition to launch the context menu and select [Insert > Model Service] to add Model Service.



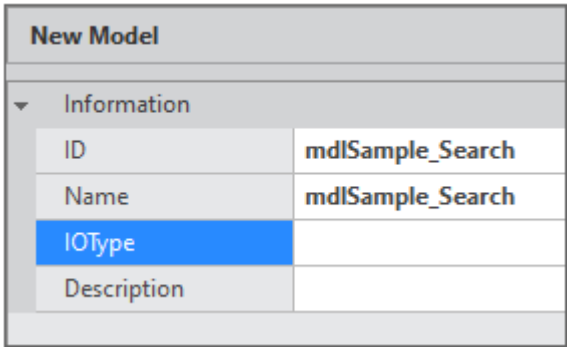
4. Right-click the Model Service item to launch the context menu and select [Insert > Model] to launch the Model Wizard.



27.1.3 Creating Model in Model Wizard

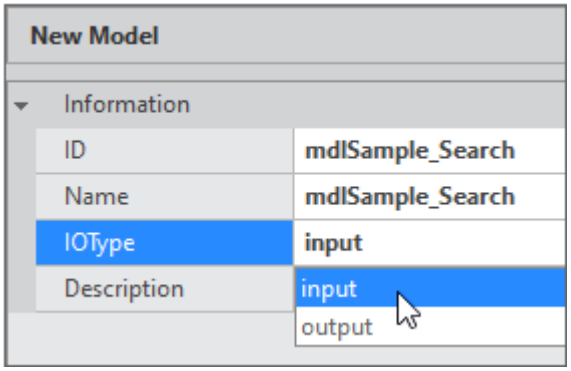
1. Enter Model ID and Name.

   If you enter the ID item value, the Name item value is entered as the same value. The value of the Name field can be changed to any other value.



2. Set the IOType item value according to the data type.

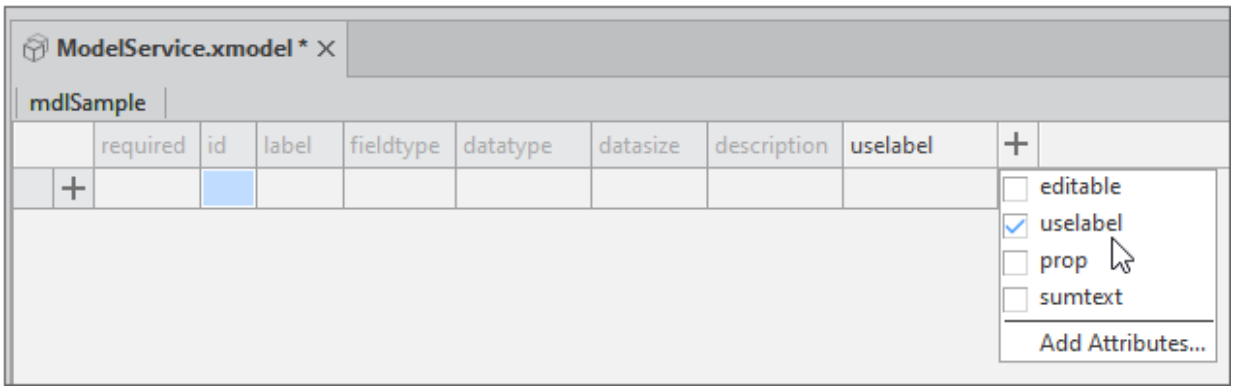
   Select "input" for sending data entered or selected by the user, such as search conditions, to the server, and select "output" for displaying search results.



3. When you click the Finish button, the Wizard is closed and the Model and Model Service settings screen is displayed.

4. Add field attribute information to be added in the model setting screen.

   Click the [+] button at the right end of the Model setting screen and check the 'uselabel' item among the expanded list (Predefined Attribute).

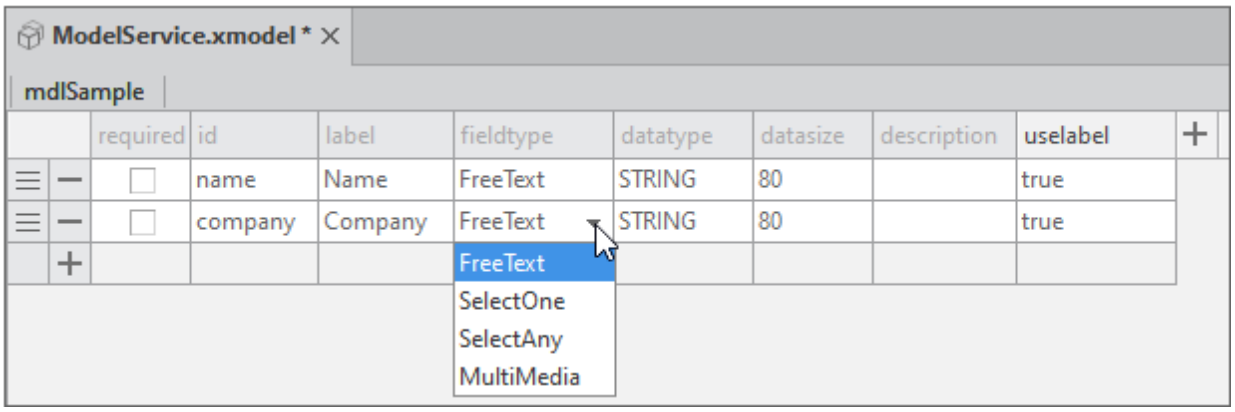


The built-in Field Attribute cannot be deleted. Labels are dimmed to distinguish them from user-added attributes.



5 Click the [+] button on the left of the Model setting screen to add a Field List item.

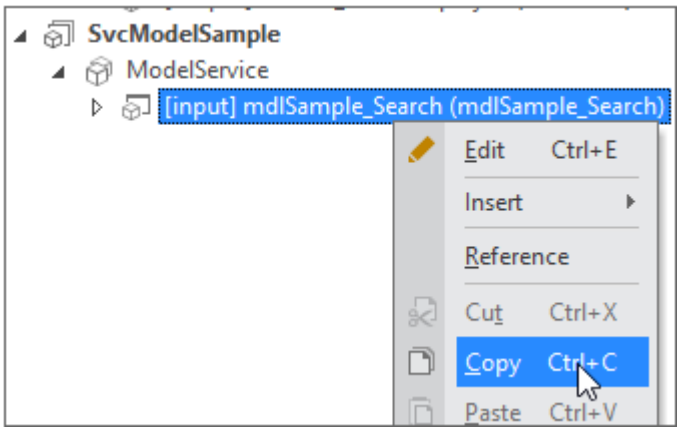
The Field Type value can be modified after adding the item. The Field Type value does not have a function by itself, but rather refers to which component or function to apply to the corresponding field value when configuring the screen in the View template.



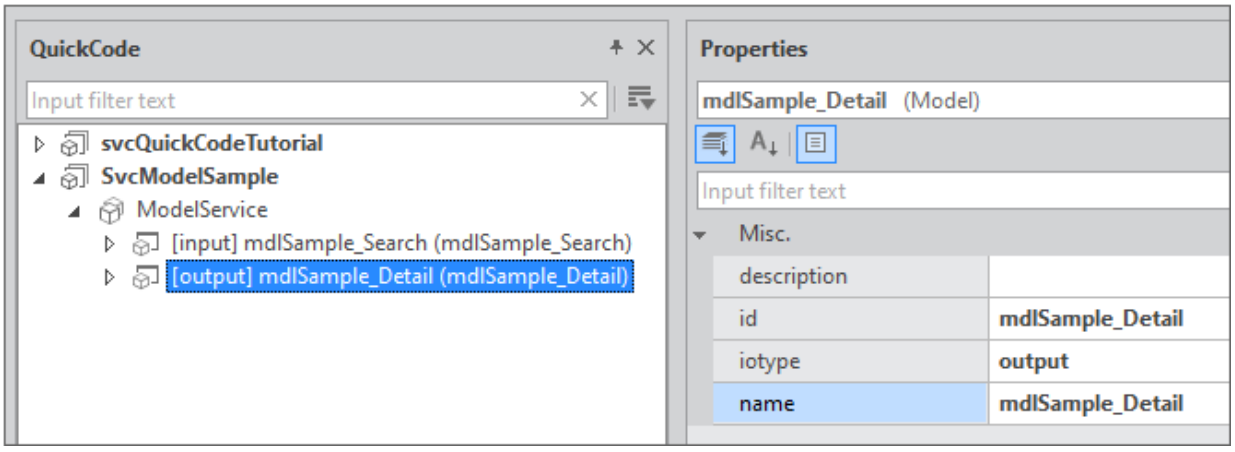
6 Click the Save icon in the menu to save the settings. Double-click the Model item in the QuickCode panel Model tab to display the Model and Model Service setting screen again.

7 Copy the Model and create a Model to process the search results.

Select the Model and select the [Copy] item from the context menu.



Select [Paste] from the context menu to create and save a new Model item. Then, edit the id, iotype, and name in the properties window and save.

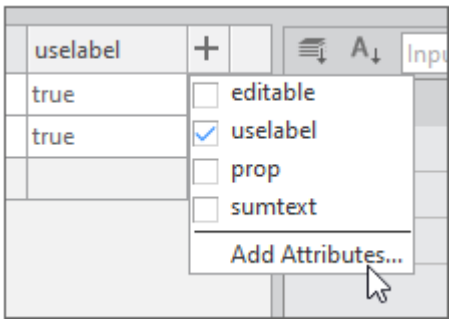


Adding User Attribute

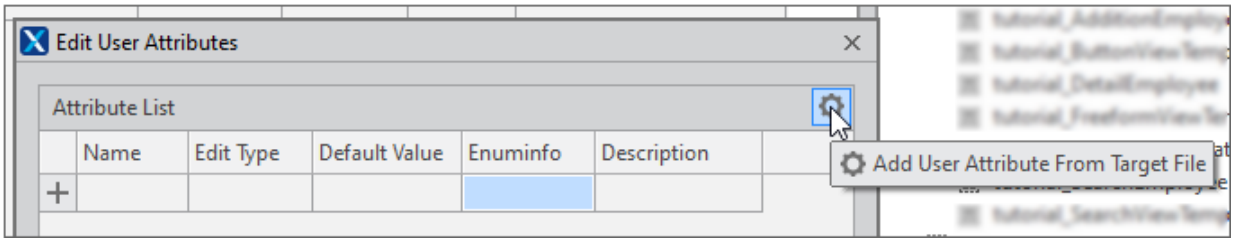
If there is no desired item in the Predefined Attribute, you can add a User Attribute.

If you write the fn_GetFieldUserAttributeList function within the View Template code, you can get the set User Attribute list.

1. Click the [+] button at the right end of the Model setting screen and check the 'Add Attributes' item from the expanded list.



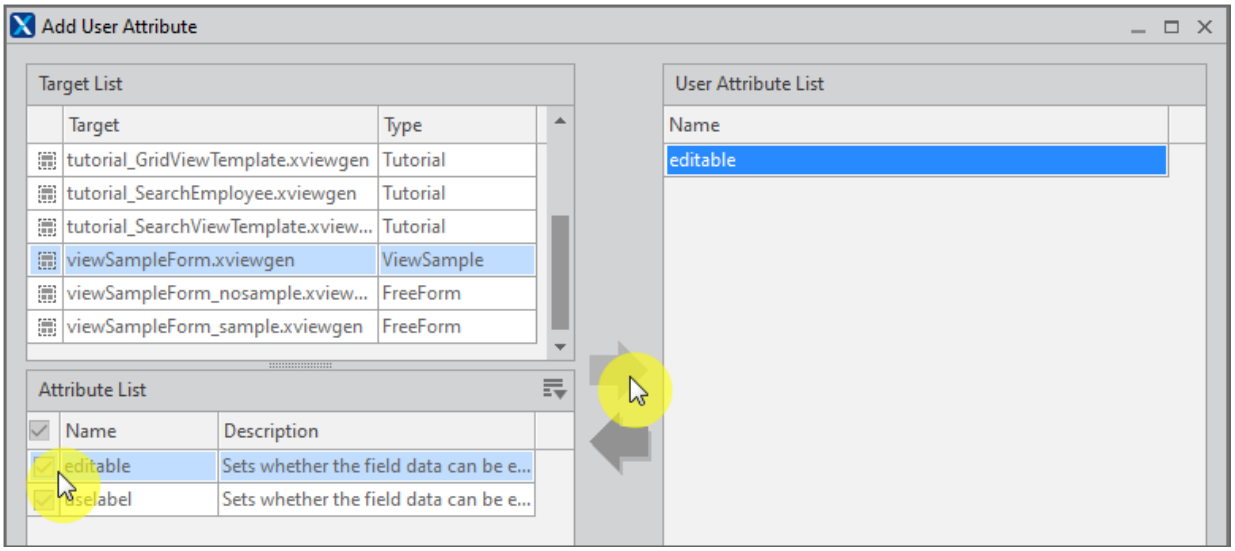
2 Click the settings icon at the far right of the Edit User Attributes window and launch the Add User Attribute window.



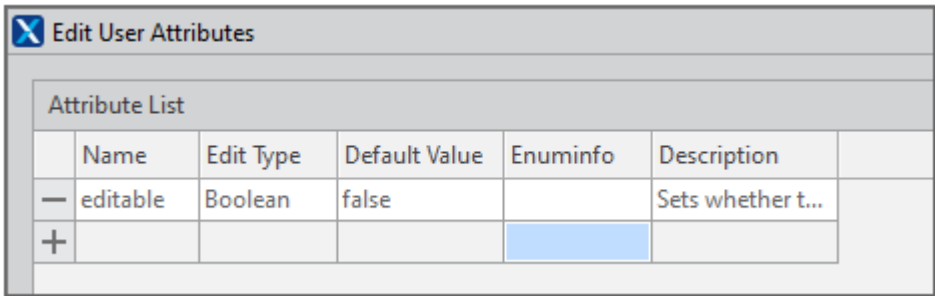
3 From the Target List list, select a View Template to import User Attributes from.

4 Check only the User Attribute items to be added to the Attribute List.

5 Click the right arrow button to add the selected User Attribute item to the User Attribute List area.



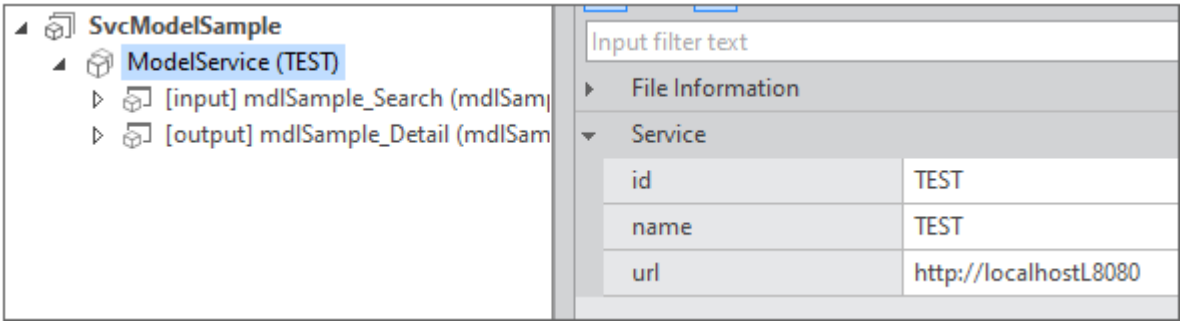
6 If you click the [OK] button in the Add User Attribute window, you can check the items to be added in the Edit User Attributes window.



7 If you click the [OK] button in the Edit User Attributes window, you can see that the User Attribute has been added to the Model setting screen.

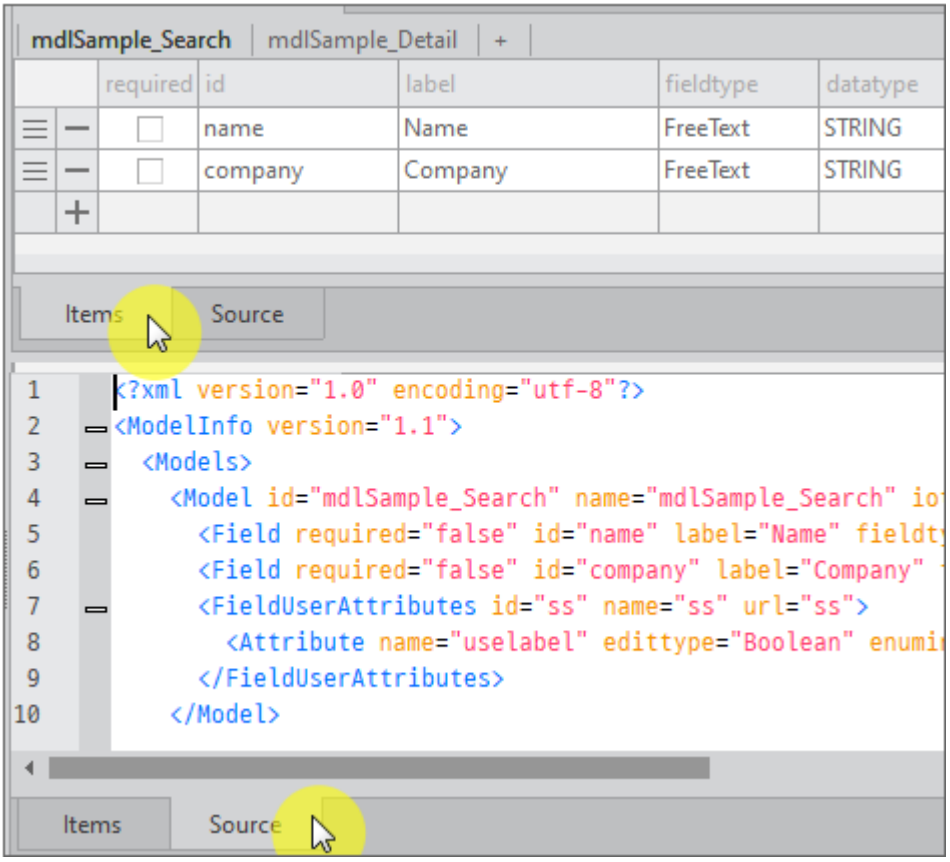
27.1.4 Setting up the Model Service

In the model service editing state, you can set service-related properties in the properties window. Set ID, Name, Url property values.



27.1.5 Check the model information source code

If you change the tab at the bottom of the Model setting screen to 'Source', the generated source code is displayed. You can edit the source code directly and change the bottom tab to "Items" to see the modifications.

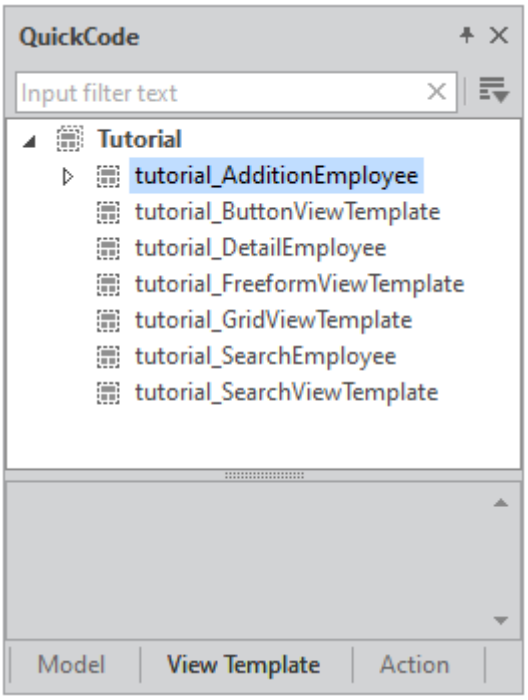


27.2 Creating View Template

27.2.1 Two methods of creating a View template

Creating View Template in View Template Wizard

When creating a project, if you check the 'Use QuickCode' item and select 'Install > Sample', the view template used for the tutorial is copied to the viewtemplate folder. You can check the script by double-clicking the View template item. The example does not use the sample View template, but creates a new View template.

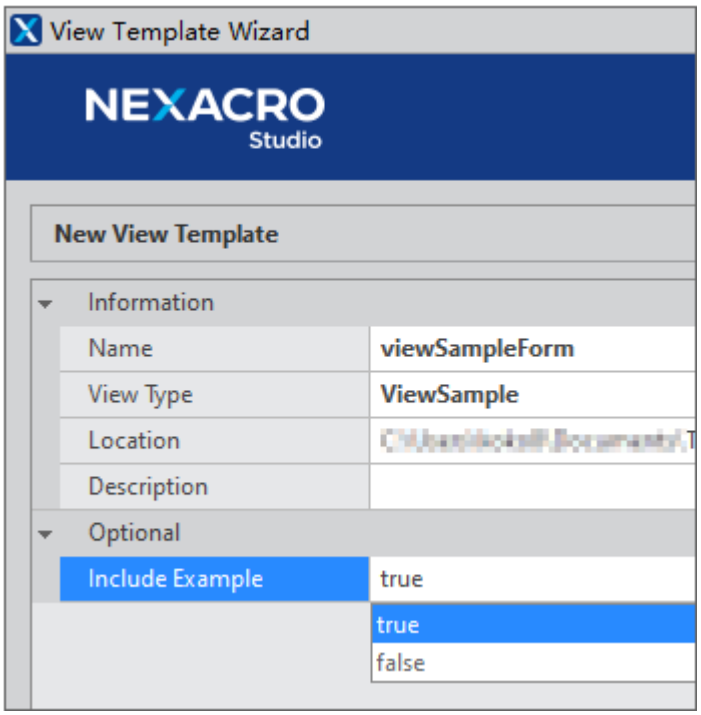


1. Select the [Add] item in the QuickCode menu to run View Template Wizard. You can also run View Template Wizard by selecting the menu [File > New > View Template] item without opening the QuickCode panel.

2. Enter Name.

3. Select or enter View Type.

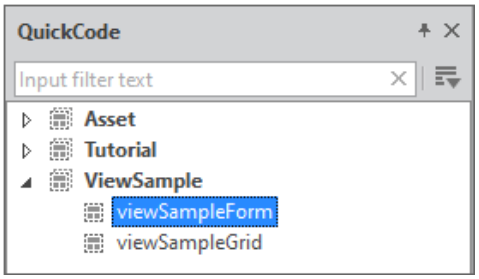
   There are four types of built-in types: "FreeForm", "GridView", "ListView", and "Tutorial", and you can add them by directly entering other values. View Type values added directly are displayed as a list when View Template Wizard is executed. Enter "ViewSample" as the View Type item value to distinguish it from the sample.



If you do not want to add an example code in the form of a comment that can be referenced when creating a View Template, change the 'Include Example' item value to false. If true is selected, a View Template is created with example code as shown below.

```
284  *                              !!! Do not delete it. !!!
285  *---------------------------------------------------------
286  fn_GetViewGenerationResult = function(fieldarray, content
287
288      var vResult = "";
289
290      // TODO : Based on model information, generate the vi
291      /*  // Example code
292      var json_contents = JSON.parse(contents);
293      var json_fieldarray = JSON.parse(fieldarray);
294
295      var nViewWidth = json_contents.View.referenceinfo.rea
296      var nViewHeight = json_contents.View.referenceinfo.re
```

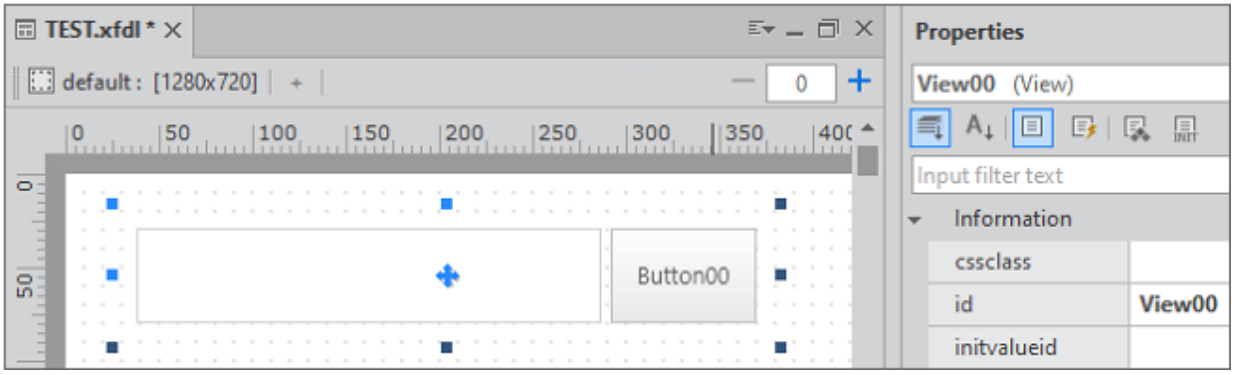
4 Click the [Finish] button and then create the View template. The created View template can be checked in the QuickCode panel View Template tab.



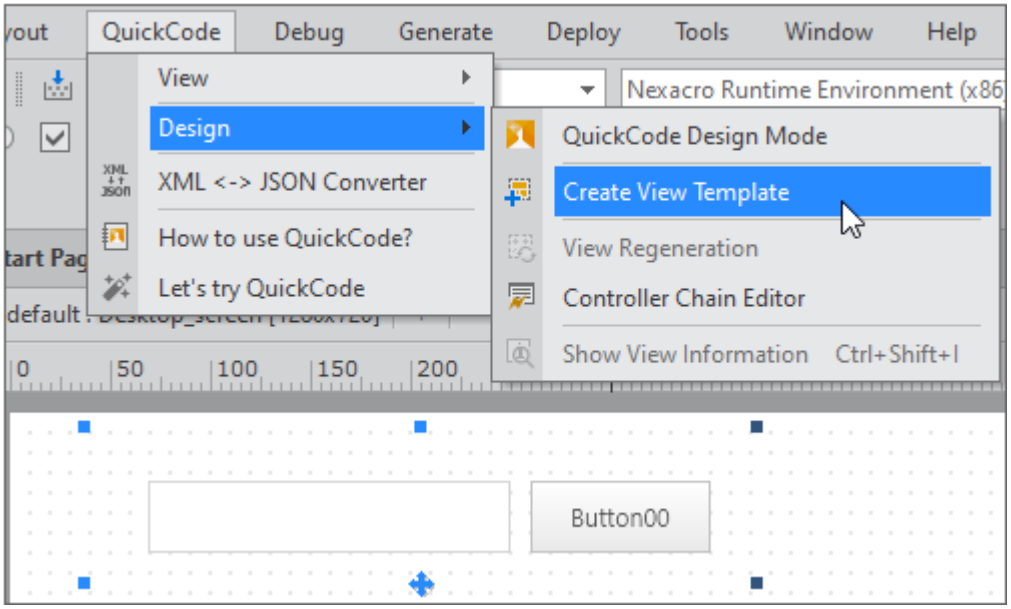
Creating a View Template with a Designed View Component

View templates can be created with a pre-designed View component screen. The code within the fn_GetViewGenerationResult function is automatically generated based on the View component screen.

1. Design the View component screen in the desired form.



2. With the View component selected, select the menu [QuickCode > Design > Create View Template] item.



3. The View Template Wizard will run. Enter items such as Name and ViewType.

4. Click the [Finish] button and create a View template. In the created View template code, you can see that the code within the fn_GetViewGenerationResult function is automatically generated.

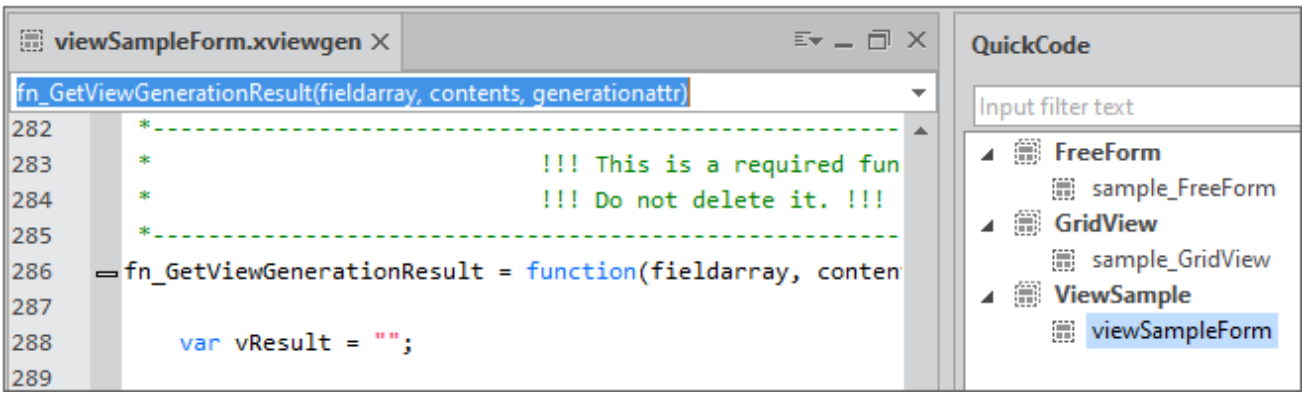
```
fn_GetViewGenerationResult = function(fieldarray, contents, generationattr) {

    var vResult =
    {
        "View" : {
            "tag" : "View",
            "referenceinfo" : {
                "realwidth" : "352",
                "realheight" : "71"
            },
            "attribute" : {
                "id" : "View00",
                "taborder" : "0",
                "text" : "View00",
                "viewdataset" : "viewdataset",
                "left" : "28",
                "top" : "19",
                "width" : "352",
                "height" : "71"
            },
            "Model" : [
                {
                    "fieldid" : "",
                    "Components" : [
                        {
```

27.2.2 Editing View Template Script

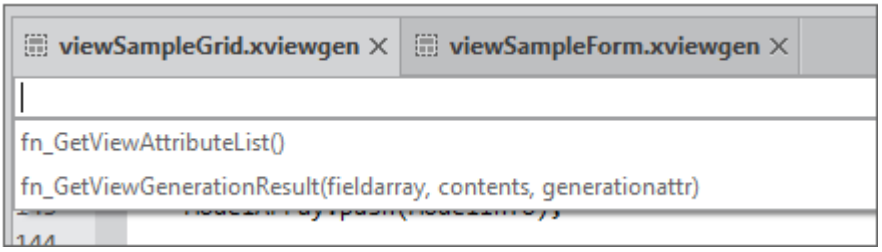
When you create the View template in View Template Wizard or double-click the View template item in the QuickCode panel, a file called [File Name].xviewgen is opened in the edit window. You can edit the script to implement the desired function.

Basic interface functions are written and the necessary information is provided in comments.



The View template must implement the following functions.

Function name	Necessity	Definition
fn_GetViewGenerationResult	O	According to model information, information for configuring screen UI is returned.
fn_GetViewAttributeList	O	Additional property information required for creating View is returned. If the function is not used, null is returned.
fn_GetFieldUserAttributeList	X	Information for processing User Attributes is returned. Only written when using User Attributes.



Next, let's look at the types of parameters received by each function and the types of values that must be returned.

fn_GetViewGenerationResult

This is the function that returns the View contents object created based on the values of parameters (fieldarray, contents, generationattr). The parameters used and values returned by the fn_GetViewGenerationResult function use the JSON-formatted object.

Syntax

```
fn_GetViewGenerationResult(fieldarray, contents, generationattr)
```

Parameters

Parameter	Type	Description
fieldarray	Object	Model Field information
contents	Object	View information
generationattr	Object	View Generation property information

Return

Type	Description
String	Returns the contents-type object excluding referenceinfo as the character string.



Please refer to fn_GetViewGenerationResult Return Value JSON Format for a detailed description of the return value JSON format.

fieldarray Parameter JSON Format

This is the selected Model information. The default value is the field value when creating the model, and it is transmitted along when Attribute such as "uselabel" is added.

- `modelinfo`
  - `serviceid` - Model name (Service ID)
  - `serviceurl` - Service URL
- `fieldcount` - fields count
- `fields`
  - `id` - Model Field ID
  - `labeltext` - Model Field Label
  - `fieldtype` - Model Field Field Type
  - `datatype` - Model Field Data Type
  - `datasize` - Model Field Data Size
  - `uselabel` - Added Attribute

```
{
    "modelinfo": {
        "serviceid": "mdlSample",
        "serviceurl": ""
    },
    "fieldcount": "2",
    "fields": [
        {
            "id": "name",
            "label": "Name",
            "fieldtype": "FreeText",
            "datatype": "STRING",
            "datasize": "80",
            "uselabel" : "true"
        },
        {
```

```
            "id": "company",
            "label": "Company",
            "fieldtype": "FreeText",
            "datatype": "STRING",
            "datasize": "80",
            "uselabel" : "true"
        }
    ]
}
```

contents Parameter JSON Format

This is the View component information to connect Model. You can place the component based on the actual width and height (referenceinfo) at which the component can be placed, excluding the Border area of the View component. The component is placed in consideration of the actual size and spacing of each component and font size.

If the View component already has the model information connected or if there is another component, then the Model information is added, and if the Dataset object is included, then the Objects information is added. Other information related to the View component is transmitted together.

- `View`
  - `tag` - Tag Name (View)
  - `referenceinfo`
    - `realwidth` - Width of the View component, excluding the Border area
    - `realheight` - Height of the View component, excluding the Border area
  - `attribute` - Properties set in the View component
    - `id`
    - `taborder`
    - `text`
    - `viewdataset`
    - `left`
    - `top`
    - `width`
    - `height`
  - `Model`
  - `Objects`
  - `Bind`
  - `InitValue`
  - `Script`

```
{
    "View": {
        "tag": "View",
        "referenceinfo": {
            "realwidth": "490",
            "realheight": "90"
        },
        "attribute": {
            "id": "View00",
            "taborder": "0",
            "text": "View00",
            "viewdataset": "viewdataset",
            "left": "0",
            "top": "0",
            "width": "500",
            "height": "100",
            "border":"5px solid darkkhaki"
        },
        "Model": [],
        "Objects": [
            {}
        ],
        "Bind": [
            {}
        ],
        "InitValue": [
            {}
        ],
        "Script": {}
    }
}
```

generationattr Parameter JSON Format

This is the ViewAttribute setting information. Each item is the value returned from the fn_GetViewAttributeList function, and the item value is the set value in the View Template Attributes step when connecting Model.

- `attributes`
  - `[attribute name]` - ViewAttribute Setting Value

```
{
    "attributes": {
        "use_triggerbutton": "true"
    }
}
```

Return Value JSON Format

This is the View contents value created based on the values of parameters (fieldarray, contents, generationattr) in the View template.

- `View`
  - `tag` - Tag Name (View)
  - `attribute` - Properties set in the View component (the Contents parameter value if not modified)
  - `Model` - Sets the component list for each field as an array
    - `fieldid` - When referencing multiple fields, like the Grid component, enter them separated by commas (,), and then enter the empty value for components without field information.
    - `Components`
  - `InitValue`
  - `Objects`
  - `Bind`
  - `Script`

```
{
    "View": {
        "tag": "View",
        "attribute": {
            "id": "View00",
            "taborder": "0",
            "text": "View00",
            "viewdataset": "viewdataset",
            "left": "85",
            "top": "43",
            "width": "437",
            "height": "110"
        },
        "Model": [
            {
                "fieldid": "name",
                "Components": [
                    {
                        "tag": "Static",
```

```
                    "attribute": {
                        "id": "Static0",
                        "text": "Name",
                        "left": 0,
                        "top": 0,
                        "width": 60,
                        "height": 20,
                        "font": "normal bold 15pt/normal"
                    }
                },
                {
                    "tag": "Edit",
                    "attribute": {
                        "id": "Edit0",
                        "left": "Static0:5",
                        "top": 0,
                        "width": 100,
                        "height": 20,
                        "font": "normal bold 15pt/normal"
                    }
                }
            ]
        },
        {
            "fieldid": "company",
            "Components": [
                {
                    "tag": "Static",
                    "attribute": {
                        "id": "Static1",
                        "text": "Company",
                        "left": 0,
                        "top": 30,
                        "width": 60,
                        "height": 20,
                        "font": "normal bold 15pt/normal"
                    }
                },
                {
                    "tag": "Edit",
                    "attribute": {
```

```
                        "id": "Edit1",
                        "left": "Static1:5",
                        "top": 30,
                        "width": 100,
                        "height": 20,
                        "font": "normal bold 15pt/normal"
                    }
                }
            ]
        },
        {
            "fieldid": "",
            "Components": [
                {
                    "tag": "Button",
                    "attribute": {
                        "id": "btn_Trigger",
                        "right": 0,
                        "top": 0,
                        "width": 70,
                        "height": 20,
                        "text": "Search"
                    }
                }
            ]
        }
    ],
    "Objects": [
        {
            "tag": "Objects",
            "Objects": [
                {
                    "tag": "Dataset",
                    "attribute": {
                        "id": "ds_company"
                    },
                    "Dataset": [
                        {
                            "tag": "ColumnInfo",
                            "ColumnInfo": [
                                {
```

```
                                    "tag": "Column",
                                    "attribute": {
                                        "id": "code",
                                        "type": "STRING",
                                        "size": "80"
                                    }
                                },
                                {
                                    "tag": "Column",
                                    "attribute": {
                                        "id": "data",
                                        "type": "STRING",
                                        "size": "256"
                                    }
                                }
                            ]
                        },
                        {
                            "tag": "Rows",
                            "Rows": [
                                {
                                    "tag": "Row"
                                }
                            ]
                        }
                    ]
                }
            ]
        }
    ],
    "Bind": [
        {
            "tag": "Bind",
            "Bind": [
                {
                    "tag": "BindItem",
                    "attribute": {
                        "id": "item0",
                        "compid": "Edit0",
                        "propid": "value",
                        "datasetid": "viewdataset",
```

```
                        "columnid": "name"
                    }
                },
                {
                    "tag": "BindItem",
                    "attribute": {
                        "id": "item1",
                        "compid": "Edit1",
                        "propid": "value",
                        "datasetid": "viewdataset",
                        "columnid": "company"
                    }
                }
            ]
        }
    ]
  }
}
```

fn_GetViewAttributeList

This is the function that returns the property information required to create View.

Syntax

```
fn_GetViewAttributeList()
```

Return

Type	Description
String	Returns the View Template Attributes list object as the character string. When returning the null value, the additional property setting is not processed in View Generation Wizard.

Return Value JSON Format

- `attributecount` - Number of attributes
- `attributes`
  - `id` - Value displayed as the View Template Attributes item in View Generation Wizard

- `edittype` - Boolean/String/Number/Enum/ViewObjList/Layout
- `defaultvalue` - Default value when edittype is Enum
- `enumlist` - Array of values when edittype is Enum
- `userproperties` - Set custom property when edittype is Layout
- `description` - Description displayed when the item is selected in View Generation Wizard

```
{
    "attributecount": 1,
    "attributes": [
        {
            "id": "use_triggerbutton",
            "edittype": "Boolean",
            "defaultvalue": "true",
            "description": "Create a button for data inquiry (true/false)"
        }
    ]
}
```

When the edittype value is "Enum", it can be processed as follows.

```
{
    "attributecount": "1",
    "attributes": [
        {
            "id": "gridtype",
            "edittype": "Enum",
            "defaultvalue": "Single Line",
            "enumlist": [
                "Single Line",
                "Multi Line"
            ],
            "description": "Specifies the generation type of grid."
        }
    ]
```

If the edittype value is "Layout", it can be processed as follows.

```
{
    "attributecount": "1",
    "attributes": [
        {
```

```
            "id": "view_layout",
            "edittype": "Layout",
            "userproperties": {
                "propertycount": "1",
                "properties": [
                    {
                        "id": "gap",
                        "edittype": "Number",
                        "defaultvalue": 5
                    }
                ]
            }
        }
    ]
```

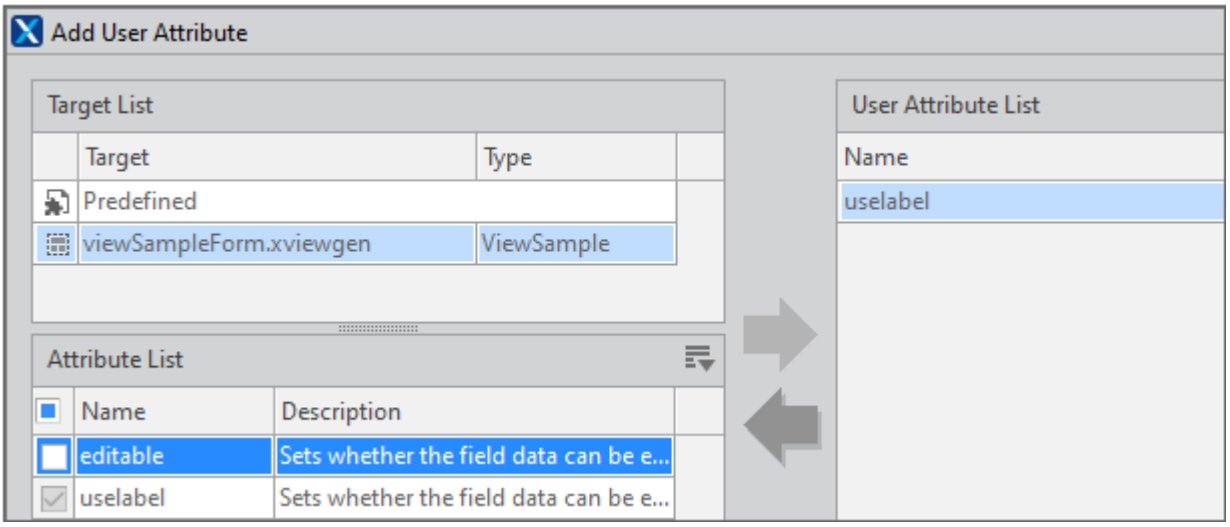
fn_GetFieldUserAttributeList

The fn_GetFieldUserAttributeList function supports two features.

- Returns User Attributes information to be added to a Model field when creating a model.
- When applying a Model, User Attributes information is added to a Model field.

Handling User Attributes when creating a Model

When creating a model, the [Add User Attribute] setting displays the list of View Templates with the fn_GetFieldUserAttributeList function in the Target List and displays User Attributes information that can be added when a View Template is selected.



Handling User Attributes when applying a Model

When applying a Model to a View component, if the selected View Template contains the fn_GetFieldUserAttributeList function, User Attributes information is added to the Model field.



If the User Attribute defined in the Model and the id are the same, the User Attribute defined in the Model is applied.

Syntax

```
fn_GetFieldUserAttributeList()
```

Return

Type	Definition
String	The User Attributes list object to be added as a Model field is returned as a string. The Attribute List is not displayed if a null value is returned.

Return Value JSON Format

- `attributecount` - Number of attributes
- `attributes`
  - `id` - Value displayed as the User Attributes item
  - `edittype` - Boolean/String/Number/Enum/ViewObjList
  - `description` - Description displayed when selecting the Model field item

```
{
    "attributecount": 1,
    "attributes": [
        {
            "id": "editable",
            "edittype": "Boolean",
            "defaultvalue": "true",
            "description": "Sets whether the field data can be edited."
        }
    ]
}
```

27.2.3 View Template Sample

In the example, we create two View templates. viewSampleForm creates the search condition UI according to the Model information or UI that represents detailed information of the item selected in the Grid component. viewSampleGrid represents the Grid component according to the Model information.

In the sample below, Object, Array are used to create the JSON return value format, and you can write the code with only the strings as shown below.

```
...
var strDataset1 = "{";
strDataset1 += "\"tag\": \"Dataset\",";
strDataset1 += "\"attribute\": {\"id\": \"viewdataset\"},";
strDataset1 += "\"Dataset\": [";
strDataset1 += "{\"tag\": \"ColumnInfo\",";
strDataset1 += "\"ColumnInfo\": [";
...
```



In the sample code, all the codes were added inside the function, but the method of creating a separate function by dividing each object included in the return value and combining the result values may be slightly cleaner.

viewSampleForm

Create the Edit component according to the field information, and if the uselabel additional property is "true", then place the Static component in front of the Edit component. The fn_GetViewAttributeList function returns the item called use_triggerbutton and determines whether to create Button.

A Dataset object is created according to the Model, and the created Dataset object is set as the viewdataset property value. Then it is binded to the Edit component value property.

```
fn_GetViewGenerationResult = function (fieldarray, contents, generationattr) {
    var vResult = "";
    var json_contents = JSON.parse(contents);
    var json_fieldarray = JSON.parse(fieldarray);
    var json_generationattr = JSON.parse(generationattr);

    var View = new Object(); // JSON Object
    var tempInfo;
    var tempPostion;
```

```
var ModelArray = new Array();
var ModelInfo;

var ComponentArray;
var ObjectsArray = new Array();
var ObjectArray = new Array();
var DatasetArray = new Array();
var ColumnArray = new Array();
var BindsArray = new Array();
var BindArray = new Array();

for (var i in json_fieldarray.fields) {
    ModelInfo = new Object();
    ComponentArray = new Array();
    ModelInfo.fieldid = json_fieldarray.fields[i].id;

    tempInfo = new Object();
    tempInfo.tag = "Column";
    tempInfo.attribute = new Object();
    tempInfo.attribute.id = json_fieldarray.fields[i].id;
    tempInfo.attribute.type = json_fieldarray.fields[i].datatype;
    tempInfo.attribute.size = json_fieldarray.fields[i].datasize;
    ColumnArray.push(tempInfo);

    if(json_fieldarray.fields[i].uselabel == "true") {
        tempInfo = new Object();
        tempInfo.tag = "Static";
        tempInfo.attribute = new Object();
        tempInfo.attribute.id = "Static"+i;
        tempInfo.attribute.text = json_fieldarray.fields[i].label;
        tempInfo.attribute.left = 0;
        tempInfo.attribute.top = i*30;
        tempInfo.attribute.width = 60;
        tempInfo.attribute.height = 20;
        tempInfo.attribute.font = "normal bold 15pt/normal";
        ComponentArray.push(tempInfo);
    }

    tempInfo = new Object();
    tempInfo.tag = "Edit";
```

```
        tempInfo.attribute = new Object();
        tempInfo.attribute.id = "Edit"+i;
        tempInfo.attribute.left = "Static"+i+":5";
        tempInfo.attribute.top = i*30;
        tempInfo.attribute.width = 100;
        tempInfo.attribute.height = 20;
        tempInfo.attribute.font = "normal bold 15pt/normal";
        ComponentArray.push(tempInfo);

        tempInfo = new Object();
        tempInfo.tag = "BindItem";
        tempInfo.attribute = new Object();
        tempInfo.attribute.id = "item"+i;
        tempInfo.attribute.compid = "Edit"+i;
        tempInfo.attribute.propid = "value";
        tempInfo.attribute.datasetid = "viewdataset";
        tempInfo.attribute.columnid = json_fieldarray.fields[i].id;
        BindArray.push(tempInfo);

        ModelInfo.Components = ComponentArray;
        ModelArray.push(ModelInfo);
    }

    tempInfo = new Object();
    tempInfo.tag = "Bind";
    tempInfo.Bind = BindArray;
    BindsArray.push(tempInfo);

    if(json_generationattr.attributes.use_triggerbutton == "true") {
        ModelInfo = new Object();
        ComponentArray = new Array();
        ModelInfo.fieldid = "";

        tempInfo = new Object();
        tempInfo.tag = "Button";

        var buttonwidth = 70;
        tempInfo.attribute = new Object();
        tempInfo.attribute.id = "btn_Trigger";
        tempInfo.attribute.right = 0;
        tempInfo.attribute.top = 0;
```

```
        tempInfo.attribute.width = buttonwidth;
        tempInfo.attribute.height = 20;
        tempInfo.attribute.text = "Search"
        ComponentArray.push(tempInfo);
        ModelInfo.Components = ComponentArray;
        ModelArray.push(ModelInfo);
    }

    tempInfo = new Object();
    tempInfo.tag = "View";
    tempInfo.attribute = json_contents.View.attribute;
    tempInfo.Model = ModelArray;
    tempInfo.Objects = ObjectsArray;
    tempInfo.Bind = BindsArray;

    View.View = tempInfo;
    var vResult = JSON.stringify(View);
    return vResult;
};

fn_GetViewAttributeList = function () {
    var attrArray = new Array();
    var attrObj = new Object();
    attrObj.id = "use_triggerbutton";
    attrObj.edittype = "Boolean";
    attrObj.defaultvalue = "true";
    attrObj.description = "Create a button for data inquiry (true/false)";
    attrArray.push(attrObj);

    var attr = new Object();
    attr.attributecount = attrArray.length;
    attr.attributes = attrArray;

    var strJson = JSON.stringify(attr);
    trace("RE : "+strJson);
    return strJson;
};
```

viewSampleGrid

Create the Grid component and the Dataset object according to the field information and then bind the Dataset object to the Grid component. The fn_GetViewAttributeList function is not used.

```
fn_GetViewGenerationResult = function (fieldarray, contents, generationattr) {
    var vResult = "";
    var json_contents = JSON.parse(contents);
    var json_fieldarray = JSON.parse(fieldarray);
    //var json_generationattr = JSON.parse(generationattr);

    var View = new Object(); // JSON Object
    var tempInfo;

    var ModelArray = new Array();
    var ModelInfo = new Object();
    ModelInfo.fieldid = "";

    var ComponentArray = new Array();
    var ObjectsArray = new Array();
    var ObjectArray = new Array();
    var DatasetArray = new Array();
    var ColumnArray = new Array();

    var GridColumnsArray = new Array();
    var GridRowsArray = new Array();
    var GridBandHeadArray = new Array();
    var GridBandBodyArray = new Array();


    for (var i in json_fieldarray.fields) {
        if(ModelInfo.fieldid != "") {
            ModelInfo.fieldid += ",";
        }
        ModelInfo.fieldid += json_fieldarray.fields[i].id;

        tempInfo = new Object();
        tempInfo.tag = "Column";
        tempInfo.attribute = new Object();
        tempInfo.attribute.id = json_fieldarray.fields[i].id;
        tempInfo.attribute.type = json_fieldarray.fields[i].datatype;
```

```
        tempInfo.attribute.size = json_fieldarray.fields[i].datasize;
        ColumnArray.push(tempInfo);

        tempInfo = new Object();
        tempInfo.tag = "Column";
        tempInfo.attribute = new Object();
        tempInfo.attribute.size = "80";
        GridColumnsArray.push(tempInfo);

        tempInfo = new Object();
        tempInfo.tag = "Cell";
        tempInfo.attribute = new Object();
        tempInfo.attribute.col = i;
        tempInfo.attribute.text = json_fieldarray.fields[i].label;
        GridBandHeadArray.push(tempInfo);

        tempInfo = new Object();
        tempInfo.tag = "Cell";
        tempInfo.attribute = new Object();
        tempInfo.attribute.col = i;
        tempInfo.attribute.text = "bind:"+json_fieldarray.fields[i].id;
        tempInfo.attribute.edittype = "text";
        GridBandBodyArray.push(tempInfo);
    }
    var GridInfo = new Object();
    GridInfo.tag = "Formats";
    GridInfo.Formats = new Array();
    var formatInfo = new Object();
    formatInfo.tag = "Format";
    formatInfo.attribute = new Object();
    formatInfo.attribute.id = "default";
    formatInfo.Format = new Array();

    tempInfo = new Object();
    tempInfo.tag = "Columns";
    tempInfo.Columns = GridColumnsArray;
    formatInfo.Format.push(tempInfo);

    tempInfo = new Object();
    tempInfo.tag = "Row";
    tempInfo.attribute = new Object();
```

```
    tempInfo.attribute.band = "head";
    tempInfo.attribute.size = "24";
    GridRowsArray.push(tempInfo);
    tempInfo = new Object();
    tempInfo.tag = "Row";
    tempInfo.attribute = new Object();
    tempInfo.attribute.size = "24";
    GridRowsArray.push(tempInfo);

    tempInfo = new Object();
    tempInfo.tag = "Rows";
    tempInfo.Columns = GridRowsArray;
    formatInfo.Format.push(tempInfo);

    tempInfo = new Object();
    tempInfo.tag = "Band";
    tempInfo.attribute = new Object();
    tempInfo.attribute.id = "head";
    tempInfo.Columns = GridBandHeadArray;
    formatInfo.Format.push(tempInfo);

    tempInfo = new Object();
    tempInfo.tag = "Band";
    tempInfo.attribute = new Object();
    tempInfo.attribute.id = "body";
    tempInfo.Columns = GridBandBodyArray;
    formatInfo.Format.push(tempInfo);
    GridInfo.Formats.push(formatInfo);

    tempInfo = new Object();
    tempInfo.tag = "Grid";
    tempInfo.attribute = new Object();
    tempInfo.attribute.id = "Grid"+i;
    tempInfo.attribute.left = 0;
    tempInfo.attribute.top = 0;
    tempInfo.attribute.right = 0;
    tempInfo.attribute.bottom = 0;
    tempInfo.attribute.binddataset = "viewdataset";
    tempInfo.Grid = new Array();
    tempInfo.Grid.push(GridInfo);
    ComponentArray.push(tempInfo);
```

```
    ModelInfo.Components = ComponentArray;
    ModelArray.push(ModelInfo);

    tempInfo = new Object();
    tempInfo.tag = "View";
    tempInfo.attribute = json_contents.View.attribute;
    tempInfo.Model = ModelArray;
    tempInfo.Objects = ObjectsArray;

    View.View = tempInfo;
    var vResult = JSON.stringify(View);
    return vResult;
};

fn_GetViewAttributeList = function () {
    return null;
};
```

27.3 Creating Action

This step is to set the function that will actually operate on the screen created using the Model and the View template. When a specific event occurs in the object specified as Trigger, the specified action is processed. In the example, when the button is clicked in the search window, the data is displayed in the Grid component. According to the Action property value for data search, the Alert dialog box is displayed if the specific property value is selected when the data search is successful.



Action is created in the form of the user module and is then installed in Nexacro Studio.
You can create the Action module by creating the Invisible Object project in the Nexacro module developer.

27.3.1 Creating Action Module Project

1. Run the Nexacro module developer beta.

> If you did not create the shortcut when installing Nexacro Studio, then run the "nexacromoduledeveloper.exe" file in the installation folder.

2. Select the menu [File > New > Project] to run Project Wizard.

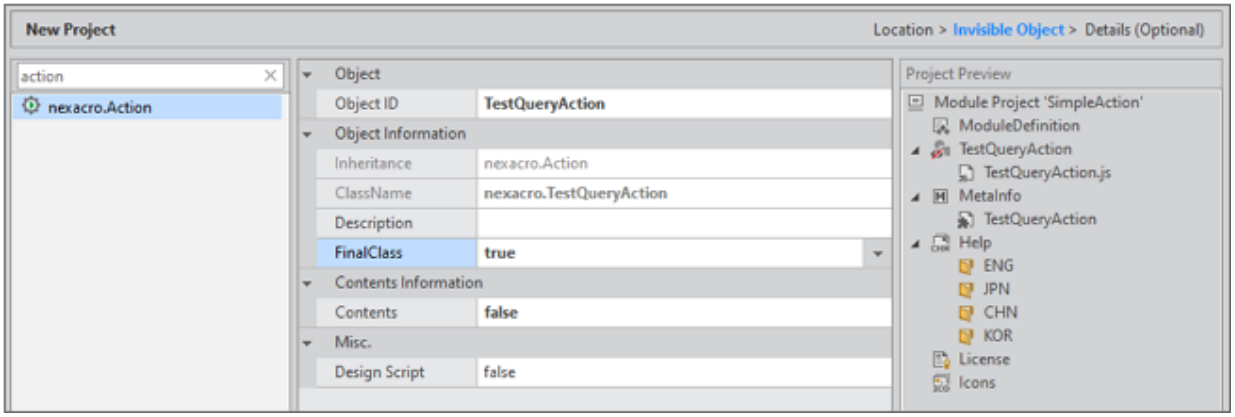
3. Select Invisible Object from the Module item.



4. Enter Project Name and click the [Next] button.

5. Enter Object ID. Object ID is set to the Action object name.

   In the example, "TestQueryAction" is entered as Object ID. ClassName is automatically populated when Object ID is entered. Change the value of the FinalClass property to true and the value of the Contents property to false.



6 Click the [Finish] button.

The project is created. Detailed option settings are not set in this example. The detailed properties of the object can be set at any time after project creation.

27.3.2 Adding Action Object

When creating the project, one default object is created and the object to deploy in the same module must be added.

1 Select the menu [File > New > Invisible Object] to run Invisible Object Wizard.

2 Enter Object ID.

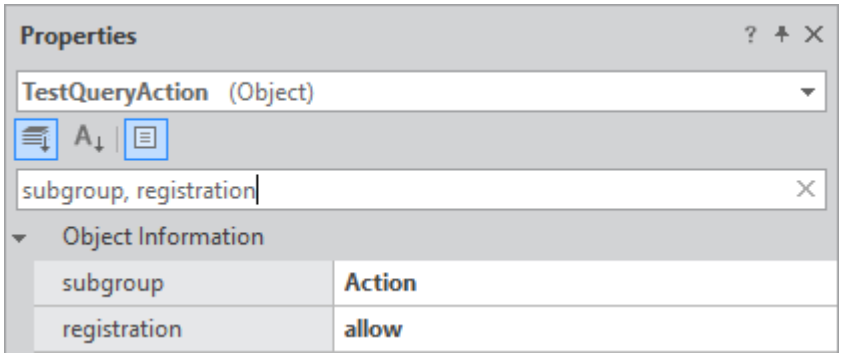
The name of the Action object to add is "TestAlertAction". Change the value of the FinalClass property to true and the value of the Contents property to false.

3 Click the [Finish] button.

27.3.3 Editing Action Object Property

1 Select the TestQueryAction and TestAlertAction object MetaInfo items in the Project Explorer and modify the property values below in the properties window.

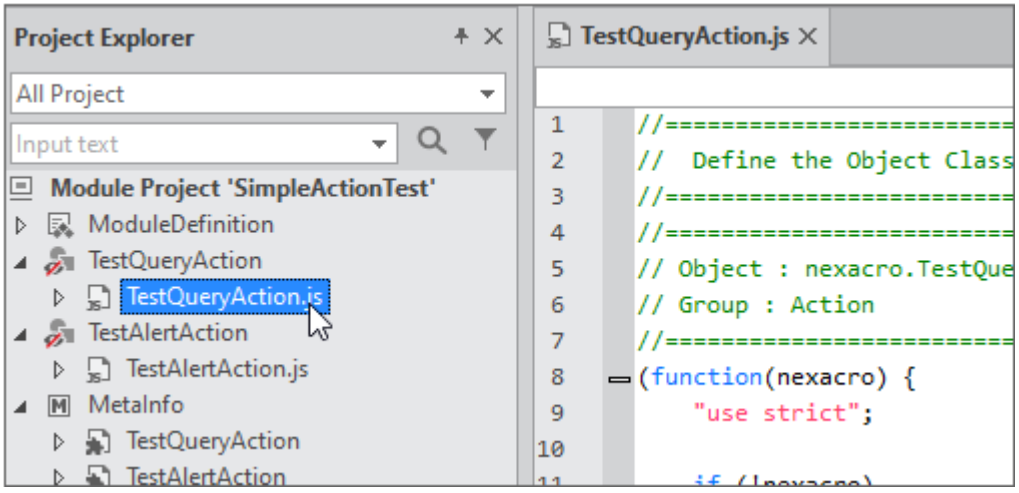
```
subgroup: Action
registration: allow
```



2 Save your changes.

27.3.4 Editing Action Object Script

1 Double-click the Action object script file to be edited in Project Explorer or select the [Edit] item from the context menu.



2 Modify the script file according to the Action object.

The Action script inherits and implements nexacro.Action. The basic structure is as follows and the run function is executed when the event occurs. The necessary information is collected and processed within the run function.



The information needed in the script can be accessed using the Action object property and method.

Action Object Properties, Methods, & Events

```
(function(nexacro) {
    "use strict";

    if (!nexacro)
        return;
    if (nexacro.TestQueryAction)
        return nexacro.TestQueryAction;

    var TestQueryAction = function(id, parent)
    {
        nexacro.Action.call(this, id, parent);
    };

    var _pTestQueryAction = nexacro._createPrototype(nexacro.Action,
TestQueryAction);
    TestQueryAction.prototype = _pTestQueryAction;
    _pTestQueryAction._type_name = "TestQueryAction";
    _pTestQueryAction.uservalue = null;

    _pTestQueryAction.destroy = function ()
    {
        nexacro.Action.prototype.destroy.call(this);
        this.uservalue = null;
    };

    _pTestQueryAction.set_uservalue = function (v)
    {
        if (v && this.uservalue!== v)
        {
            this.uservalue= v;
        }
    };

    _pTestQueryAction.run = function ()
    {
    };

    nexacro.TestQueryAction = TestQueryAction;
    return TestQueryAction;
}) (nexacro);
```

The script used for TestQueryAction, TestAlertAction is as follows.

TestQueryAction Script

When the trigger occurs (when the button is clicked) the run function is called.

The _getArgumentList function is called to receive a combination of the contents information passed as a parameter and the column value stored in the Dataset object set to the viewdataset property, and the XML file for data loading is checked with the value entered in samplefilename, and the transaction method is executed.

In the example, since the XML file is called directly, the value transmitted to the parameter is output using the trace method and not used.

TestQueryAction.js

```
(function(nexacro) {
    "use strict";

    if (!nexacro)
        return;
    if (nexacro.TestQueryAction)
        return nexacro.TestQueryAction;

    var TestQueryAction = function(id, parent)
    {
        nexacro.Action.call(this, id, parent);
    };

    var _pTestQueryAction = nexacro._createPrototype(nexacro.Action,
TestQueryAction);
    TestQueryAction.prototype = _pTestQueryAction;
    _pTestQueryAction._type_name = "TestQueryAction";

    _pTestQueryAction.samplefilename = ""; // XML file name
    _pTestQueryAction.alerttype = false; // TestAlertAction condition
    _pTestQueryAction.targetobj = null;

    _pTestQueryAction.destroy = function ()
    {
        nexacro.Action.prototype.destroy.call(this);
        this.samplefilename = null;
        this.alerttype = null;
        this.targetobj = null;
```

```
    };

    _pTestQueryAction.set_samplefilename = function (v)
    {
        if (v && this.samplefilename !== v)
        {
            this.samplefilename = v;
        }
    };

    _pTestQueryAction.set_alerttype = function (v)
    {
        if (v && this.alerttype !== v)
        {
            this.alerttype = v;
        }
    };

    _pTestQueryAction.set_targetobj = function (v)
    {
        if (v && this.targetobj !== v)
        {
            this.targetobj = v;
        }
    };


    _pTestQueryAction._getArgumentList = function (strType)
    {
        var argumentlist = this.getContents(strType);
        if (!argumentlist) {
            return;
        }
        if(strType == "model")
        {
            for (var i = 0; i < argumentlist.length; i++)
            {
                var tempDataset = this
.parent.lookup(argumentlist[i].viewid).getViewDataset();
                argumentlist[i].value =
tempDataset.getColumn(tempDataset.rowposition, argumentlist[i].fieldid);
```

```
            }
        }
        return argumentlist;
    };

    _pTestQueryAction.run = function ()
    {
        var targetview = this.getTargetView();
        var viewdataset = targetview.viewdataset;
        var argumentlist = "";

        var modellist = this._getArgumentList ("model");
        var extralist = this._getArgumentList ("extra");
        var outdataset = viewdataset + "=ds_" + viewdataset;

        var callbackfn = "fn_callback";

        if (modellist)
        {
            for (var i = 0; modellist.length > i; i++)
            {
                if (modellist[i].value)
                {
                    argumentlist += modellist[i].name + "=" +
modellist[i].value;
                    argumentlist += " ";
                }
            }
        }

        if (extralist)
        {
            for (var i = 0; extralist.length > i; i++)
            {
                if (extralist[i].value)
                {
                    argumentlist += extralist[i].name + "=" +
extralist[i].value;
                    argumentlist += " ";
                }
            }
```

```
        }

        // transaction callback function (onsuccess, onerror event)
        var targetaction = this;
        targetview.form[callbackfn] = function (strSvcID, nErrorCode,
strErrorMag)
        {
            if (nErrorCode >= 0)
            {
                targetaction.error = "callback-success";
                targetaction.on_fire_onsuccess();
            }
            else
            {
                targetaction.error = "callback-error";
                targetaction.on_fire_onerror();
            }
        }

        argumentlist += "dsname=" + viewdataset + " ";
        argumentlist.trim();
        var serviceurl = "Sample::"+this.samplefilename+".xml";
        trace("argumentlist: "+argumentlist);
        trace("outdataset: "+outdataset);
        trace("serviceurl: "+serviceurl);

        targetview.form.transaction("TEST", serviceurl, "", outdataset,
argumentlist, callbackfn);
    };

    nexacro.TestQueryAction = TestQueryAction;

    return TestQueryAction;

}) (nexacro);
```

TestAlertAction Script

In the TestAlertAction script, the message property value is included as the alert method parameter and is then executed when the run function is executed.

TestAlertAction.js

```
(function(nexacro) {
    "use strict";

    if (!nexacro)
        return;
    if (nexacro.TestAlertAction)
        return nexacro.TestAlertAction;

    var TestAlertAction = function(id, parent)
    {
        nexacro.Action.call(this, id, parent);
    };

    var _pTestAlertAction = nexacro._createPrototype(nexacro.Action,
TestAlertAction);
    TestAlertAction.prototype = _pTestAlertAction;
    _pTestAlertAction._type_name = "TestAlertAction";

    _pTestAlertAction.message = "";

    _pTestAlertAction.destroy = function ()
    {
        nexacro.Action.prototype.destroy.call(this);

        this.message = null;
    };

    _pTestAlertAction.set_message = function (v)
    {
        this.message = v;
    };

    _pTestAlertAction.run = function ()
    {
        alert(this.message);
    };

    nexacro.TestAlertAction = TestAlertAction;

    return TestAlertAction;
```

```
}) (nexacro);
```

27.3.5 Editing Action Object Meta-Info Property

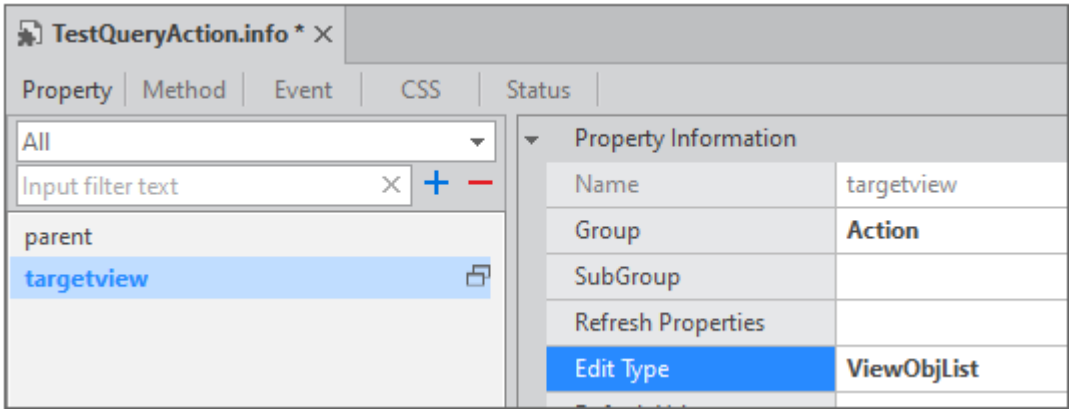
To deploy and use Action as the module, the meta-info file must be created.

1. Double-click the Action object MetaInfo item to be edited in Project Explorer or select the [Edit] item from the context menu.

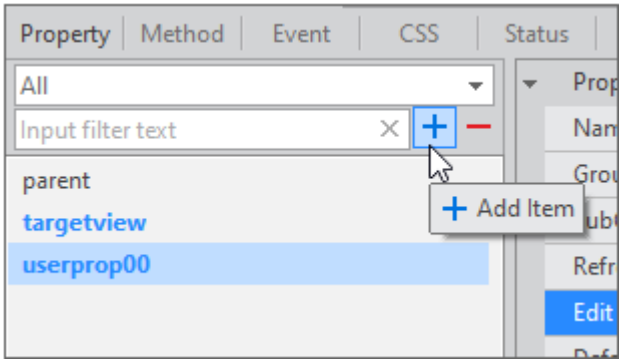
2. Modify the details of the targetview property.

   The parent, targetview properties are the default properties of nexacro.Action and are displayed as default values in meta-info when creating the Action object. Select the targetview item and then modify the information in the property window.

   ```
   Group: Action
   Edit Type: ViewObjList
   ```



3. Click the [+] button in the [Property] tab to add the property of the Action object.



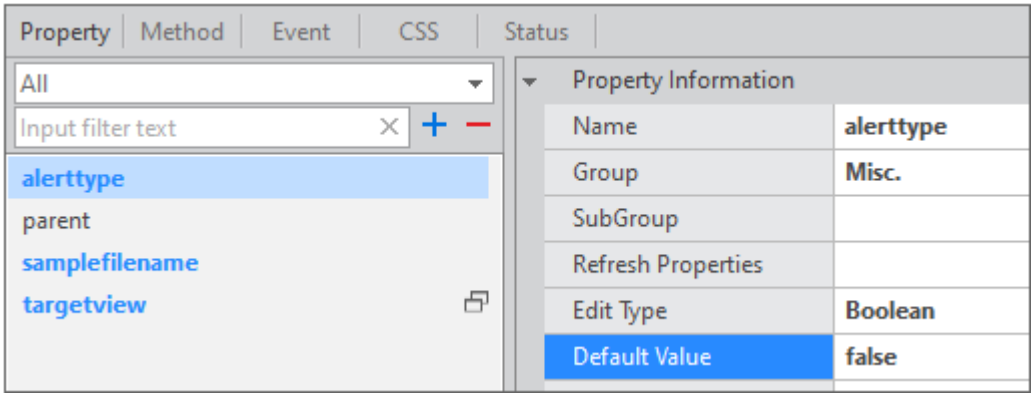


When editing the Action script, if you use the Add menu to add the property, then the information is also added to meta-info, but if you directly edit the script, then you need to add it to the meta-info information.

For the TestQueryAction object, add two properties as shown below.

```
Name: samplefilename
Group: Misc.
Edit Type: String

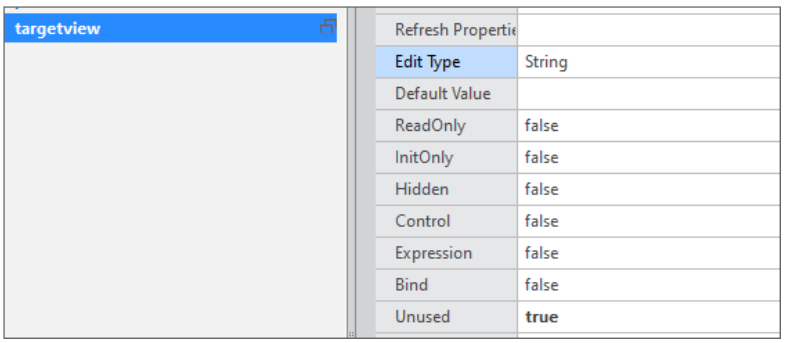
Name: alerttype
Group: Misc.
Edit Type: Boolean
Default Value: false
```



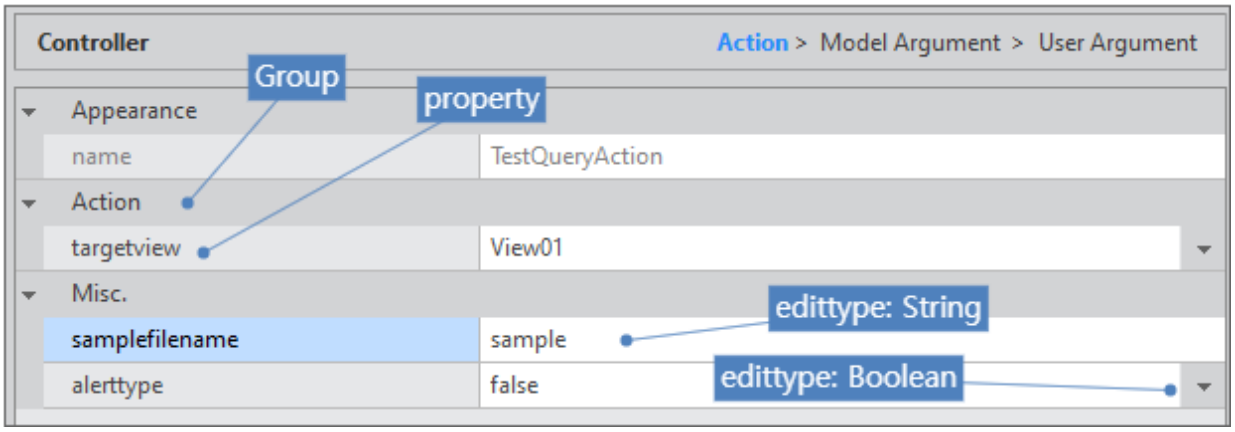
For the TestAlertAction object, add one property as shown below.

```
Name: message
Group: Misc.
Edit Type: String
```

The targetview property is not used in the TestAlertAction object. Select the property and then change the Unused item to true.



The contents set in the meta-info file are processed as follows in Controller Wizard when connecting Action.



group

You can specify the group name shown in Controller Wizard. This is auxiliary information that is provided so that the user can easily recognize the property.

edittype

In the ObjectInfo tag, set edittype to "action".

```
<ObjectInfo ... edittype="action" ...
```

You can set the user input type by specifying edittype when setting each property. The function to select from the list without entering the value as the text is also supported. The types of edittype that can be used in Action are as follows.

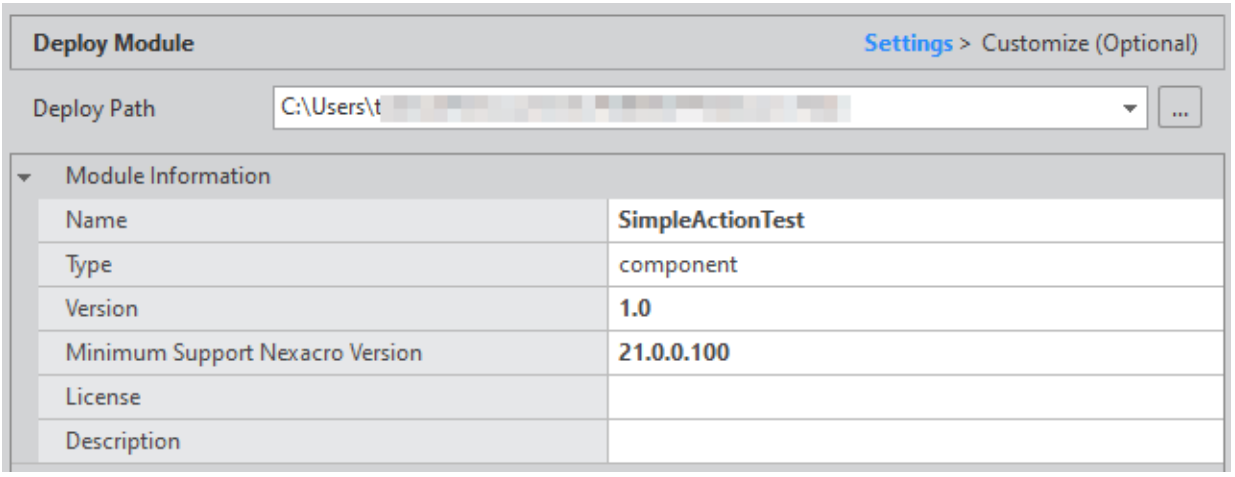
edittype	Description
String	Enters character strings.
Number	Enters numbers.
Boolean	Selects true, false values.
ModelServiceID	Provides the Model list.
ViewObjList	Provides the list of the View components that can be used in the current Form. Setting the refreshinfo item together will affect sub-items when the selected list is changed.
ViewChildObjList	Provides the list of components or objects contained in the View component selected in ViewObjList.

27.3.6 Creating Action Module File

1. Select [Deploy > Module Package] from the menu.

2. Check the path to create the module file and then click the [Next] button.

   Enter information such as the Version as necessary.



3. Click the [Deploy] button to create the xmodule file.

28.

QuickCode detailed specifications and references

28.1 Action Object Properties, Methods, & Events

28.1.1 Properties

Property Name	Description
targetview	This is the property that sets the View component to perform Action.

28.1.2 Methods

Method Name	Description
getContents	Returns the contents object transmitted when Action is executed or the value object corresponding to the key. `Action.getContents([strKey])` - If the strKey parameter is not set, then the contents object is returned. - The strKey parameter can be set to either "model" or "extra".
getTargetView	Returns the View component corresponding to the targetview property value. `Action.getTargetView()`
run	Executes Action. `Action.run()`

28.1.3 Events

Event Name	Description
onsuccess	This is the event for processing Trigger callback when the run method execution is successful. `onsuccess(obj:nexacro.Action,e:nexacro.ActionEventInfo);`

Event Name	Description
onerror	This is the event for processing Trigger callback when the run method execution fails. `onerror(obj:nexacro.Action,e:nexacro.ActionEventInfo);`

28.2 Item Information Available for the Trigger Type

In the example, "click" was selected from the context menu so that the trigger works when the Button is clicked. Selecting the "click" item is similar to writing the Button component's onclick event function. The list below shows the available trigger types.

type	event	Description
Click	onclick	Occurs when the target object (component) mouse is clicked.
Row Change	onrowposchanged	This occurs when the rowposition of a Dataset object is changed.
Value Change	oncolumnchanged	This occurs when the column value of a Dataset object is changed.
Action Success	onsuccess	Occurs when the execution of the selected Action object is successful. The Trigger View item selects a Form object, and the Trigger Object item selects an Action object.
Action Fail	onerror	Occurs when the execution of the selected Action object fails. The Trigger View item selects a Form object, and the Trigger Object item selects an Action object.
Model Load Success	onmodelloadsuccess	This occurs when the data of a Dataset object is loaded.
Model Load Fail	onmodelloadfail	This occurs when the data of a Dataset object is failed to be loaded. (This occurs when there is no data too.)
Form init	onload	Occurs when loading the Form object.
External Input	onextendedcommand	Occurs when the event is transmitted from the device defined in DeviceAdaptors.

The context menu displayed when an Action object is dragged and dropped onto a component/object is shown below.

Component/Object	Context menu
Component (Excluding the View component)	(none) Click External Input

Component/Object	Context menu
View Component	(none) Click (Component) Model Load Success Model Load Fail External Input
Action	(none) Action Success Action Fail External Input
Dataset Object	(none) Row Change Value Change Model Load Success Model Load Fail External Input
Form	(none) Form Init External Input

28.3 Trigger Condition Reserved Words & Samples

You can set the Trigger occurring condition when Action is connected. The reserved words that can be used are as follows.

Reserved Word	Description
triggerview	This is the object set as triggerview of the Trigger item when setting Action.
triggerobj	This is the object set as triggerobj of the Trigger item when setting Action.
form	This is the parent Form object that contains Trigger.

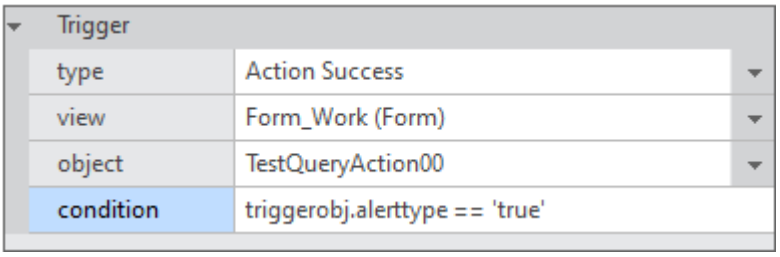
It can be applied in the following manner.

```
triggerview.name=='view_search' && triggerobj.name == 'btn_Trigger00_00' &&
form.name == 'sample_MasterDetail_V'
```

A certain property of triggerobj can be applied as the condition. For example, if Type is "Action Success", then the triggerobj is Action. When the corresponding Action is processed, whether or not to process the next

Action according to the property value can be determined.

```
triggerobj.alerttype == 'true'
```



28.4 fn_GetViewGenerationResult Return Value JSON Format

The contents parameter transmitted when calling the fn_GetViewGenerationResult function or the value returned after calling the function is processed in JSON format as shown below. The contents parameter converts the XML code of the current screen to JSON and then transmits it, and the JSON code returned after calling the function is converted to the XML code in Nexacro Studio to be reflected on the screen.

The contents parameter is converted inside Nexacro Studio, but the JSON code to be returned must be written within the fn_GetViewGenerationResult function according to the format.

28.4.1 Basic Format

The returned JSON code configures the screen within the View component and then sets the Dataset object, binding item information, and script. The basic format is as follows.

XML	JSON

<View>
  <Layouts>
  <Layout/>
  </Layouts>
  <InitValue/>
  <Objects/>
  <Bind/>
  <Script/>
</View>
	{
	  "View": {
	  "Model": [
	  {
	  "fieldid": "",
	  "Components": [
	  {}
	  ]
	  }

XML	JSON

	],
	"InitValue": [
	{}
	],
	"Objects": [
	{}
	],
	"Bind": [
	{}
	],
	"Script": {
	}
	}
	}

You can exclude items that are not in use. For example, if the structure has only one Dataset object, then the JSON code can be processed as follows.

```
{
    "View": {
        "Objects":[
            {}
        ]
    }
}
```



View Template only supports the Default layout.
If the View Template is used in the Form to which the layout has been added, then the fn_GetViewGenerationResult function may not be able to process the information.



In the XML file created after calling the fn_GetViewGenerationResult function, tags such as ModelInfo and ViewTemplateInfo are additionally created. The area is the code that manages the information related to the connected Model, View Template, and if it is arbitrarily modified, then the previous setting information may not be imported when the screen UI is created with the Model and View template.

28.4.2 Conversion Rules

What the fn_GetViewGenerationResult return value JSON code needs to create is the XML tag to be used in Nexacro Studio. Write the JSON code that is connected to each tag according to the rules below.

- Configure under the component Model property placed in the top-level View component.
- One tag is configured with one JSON object.
- Child tags are configured with the same JSON array as the Tag name of the parent tag.

Configure under the component Model property placed in the top-level View component.

Components placed in the View component have the connected fieldid information. Therefore, they have the fieldid property and the Components property under the Model property, and the actual component is configured under the Components property.

If there is no fielded connected to the component, then the Model property has only one JSON object and lists the components under the Components property. If there are two or more connected fieldids, then the related information is created under ModelInfo when creating XML with JSON objects with as many as the number of fielded as shown below.

XML

```
<View>
    <Layouts>
        <Layout>
            <Button id="Btn01"/>
            <Button id="Btn02"/>
    </Layouts>
    <ModelInfo>
        <Components>
             <Component id="Btn01" fieldid="ID"/>
             <Component id="Btn02" fieldid="DATA"/>
        </Components>
    </ModelInfo>
</View>
```

JSON

```
{
    "View": {
        "Model": [
            {
                "fieldid": "ID",
                "Components": [
                    {
                        "tag": "Button",
                        "attribute": {
                            "id": "Btn01"
                        }
                    }
                ]
            },
            {
                "fieldid": "DATA",
                "Components": [

                    {
                        "tag": "Button",
                        "attribute": {
                            "id": "Btn02"
```

XML	JSON
	} }] }] } }

One tag is configured with one JSON object.

The tag name is processed as the tag property of the JSON object and the tag property is processed as the attribute property.

```
<Tag Name Property1="Property 1 Value" Property 2="Property 2 Value "Property
3="Property 3 Value"/>
```

```
{
  "tag": " Tag Name ",
  "attribute": {
    "Property1": "Property1 Value",
    "Property2": "Property2 Value",
    "Property3": "Property3 Value"
  }
}
```

As shown below, one tag is processed as one JSON object, regardless of the component or object.

XML	JSON

<Dataset id="Dataset00"/>	{
	"tag":"Dataset",
	"attribute":{
	"id":"Dataset00"
	}
	}

<Button id="Button00"
taborder="0"
text="Button00"
left="209"
top="12"
width="90"
height="30"
	{
	"tag": "Button",
	"attribute": {
	"id": "Button00",
	"taborder": "0",
	"text": "Button00",

XML	JSON
onclick="View00_Button00_onclick"/>	"left": "209", "top": "12", "width": "90", "height": "30", "onclick": "View00_Button00_onclick" } }

<BindItem id="item0"
compid="Button00"
propid="text"
datasetid="Dataset00"
columnid="Column0"/>
	{
	"tag": "BindItem",
	"attribute": {
	"id": "item0",
	"compid": "Button00",
	"propid": "text",
	"datasetid": "Dataset00",
	"columnid": "Column0"
	}
	}

Child tags are configured with the same JSON array as the Tag name of the parent tag.

In the case of the container component including the child component, it has the same name as the component name at the same level as the tag and the attribute properties, and the property value is processed as the JSON array.

Even if it is not the container component, then it is processed in the same manner when it has the child tag structure like the Grid component.

```
<Parent Tag Name>
    <Child Tag Name Property 1="Property1 Value" Property2="Property2 Value"
Property3="Property3 Value"/>
</Parent Tag Name>
```

```
{
    "tag": "Parent Tag Name",
    "attribute": {
    },
    "Parent Tag Name": [
        {
            "tag": "Child Tag Name",
```

```
            "attribute": {
                "Property1": "Property1 Value",
                "Property2": "Property2 Value",
                "Property3": "Property3 Value"
            }
        }
}
```

In the example code below, some properties of the component have been excluded to help you understand more clearly.

XML	JSON

<Div id="Div00" text="Div00">
  <Layouts>
  <Layout>
  <Button id="Button00" text="Button00"/>
  </Layout>
  </Layouts>
</Div>
	{
	  "tag": "Div",
	  "attribute": {
	  "id": "Div00"
	  },
	  "Div": [
	  {
	  "Components": [
	  {
	  "tag": "Button",
	  "attribute": {
	  "id": "Button00",
	  "text": "Button00"
	  }
	  }
	  ]
	  }
	  ]
	}

<Bind>
  <BindItem id="item0" compid="Button00" propid="text" datasetid="Dataset00" columnid="Column0"/>
  <BindItem id="item1" compid="Button01" propid="text" datasetid="Dataset00" columnid="Column0"/>
</Bind>
	{
	  "tag": "Bind",
	  "Bind": [
	  {
	  "tag": "BindItem",
	  "attribute": {
	  "id": "item0",
	  "compid": "Button00",
	  "propid": "text",
	  "datasetid": "Dataset00",
	  "columnid": "Column0"
	  }
	  },

XML	JSON

	<pre>{
	"tag": "BindItem",
	"attribute": {
	"id": "item1",
	"compid": "Button01",
	"propid": "text",
	"datasetid": "Dataset00",
	"columnid": "Column0"
	}
	}
	]
	}</pre>

<pre><Dataset id="Dataset00">
<ColumnInfo>
<Column id="Column0" type="STRING" size="256"/>
</ColumnInfo>
<Rows>
<Row/>
</Rows>
</Dataset></pre>
	<pre>{
	"tag": "Dataset",
	"attribute": {
	"id": "Dataset00"
	},
	"Dataset": [
	{
	"tag": "ColumnInfo",
	"ColumnInfo": [
	{
	"tag": "Column",
	"attribute": {
	"id": "Column0",
	"type": "STRING",
	"size": "256"
	}
	}
	]
	},
	{
	"tag": "Rows",
	"Rows": [
	{
	"tag": "Row"
	}
	]
	}
	]
	}</pre>

In the case of the Grid component, a few steps of child tags are included to specify the format to be displayed

on the screen.

XML	JSON

<Grid id="Grid00" binddataset="Dataset00">
  <Formats>
  <Format id="default">
  <Columns>
  <Column size="80"/>
  </Columns>
  <Rows>
  <Row size="24" band="head"
/>
  <Row size="24"/>
  </Rows>
  <Band id="head">
  <Cell text="Column0"/>
  </Band>
  <Band id="body">
  <Cell text="bind:Column0"/>
  </Band>
  </Format>
  </Formats>
</Grid>
	{
	  "tag": "Grid",
	  "attribute": {
	  "id": "Grid00",
	  "binddataset": "Dataset01"
	  },
	  "Grid": [
	  {
	  "tag": "Formats",
	  "Formats": [
	  {
	  "tag": "Format",
	  "attribute": {
	  "id": "default"
	  },
	  "Format": [
	  {
	  "tag":
	"Columns",
	  "Columns": [
	  {
	  "tag":
	"Column",
	  "attribute": {
	  "size": "80"
	  }
	  }
	  ]
	  },
	  {
	  "tag": "Rows",
	  "Rows": [
	  {
	  "tag":
	"Row",
	  "attribute": {
	  "size": "24",
	  "band": "head"

XML	JSON

	}
	},
	{
	"tag": "Row",
	"attribute": {
	"size": "24"
	}
	}
	]
	},
	{
	"tag": "Band",
	"attribute": {
	"id": "head"
	},
	"Band": [
	{
	"tag": "Cell",
	"attribute": {
	"text": "Column0"
	}
	}
	]
	},
	{
	"tag": "Band",
	"attribute": {
	"id": "body"
	},
	"Band": [
	{
	"tag": "Cell",
	"attribute": {

XML	JSON
	"text": "bind:Column0" } }] }] }] }] }

As the Grid component has child tags in several steps, the JSON code may look complicated, but the conversion rules are the same. If the Grid component in the above table is separated by tag, then it is as follows.

XML	JSON

<Grid id="Grid00" binddataset="Dataset00">
<Formats>
</Formats>
</Grid>
	{
	"tag": "Grid",
	"attribute": {
	"id": "Grid00",
	"binddataset": "Dataset01"
	},
	"Grid": [
	{
	"tag": "Formats"
	}
	]
	}

<Formats>
<Format id="default">
</Format>
</Formats>
	{
	"tag": "Formats",
	"Formats": [
	{
	"tag": "Format"
	}
	]
	}

<Format id="default">
<Columns>
</Columns>
<Rows>
</Rows>
<Band id="head">
</Band>
	{
	"tag": "Format",
	"attribute": {
	"id": "default"
	},
	"Format": [
	{

XML	JSON

</Format>	"tag": "Columns"
	},
	{
	"tag": "Rows"
	},
	{
	"tag": "Band",
	"attribute": {
	"id": "head"
	}
	}
	]
	}

<Columns>
 	
<Column size="80"/>
 	
</Columns>
	{
	"tag": "Columns",
	"Columns": [
	{
	"tag": "Column",
	"attribute": {
	"size": "80"
	}
	}
	]
	}

You can also create and use the function that creates the JSON object or the array of JSON objects for each tag unit as shown below. For areas that need to be processed based on field information, such as the Grid or Dataset object, the code readability can be improved by transmitting the field information to a separate function.

```
{
    "tag": "Dataset",
    "attribute": { "id": "ds_sample" },
    "Dataset": [
        {
            "tag": "ColumnInfo",
            "ColumnInfo": fn_GetDsColumns(oFieldArray)
        },
        {
            "tag": "Rows",
            "Rows": fn_GetDsRows(oFieldArray)
        }
```

```
    ]
}
```

28.4.3 Conversion Rule Exception (Container Component)

View Template only supports the Default layout. Therefore, the container component supports one property called Components instead of the Layouts, Layout tags.

XML	JSON

<Div id="Div00" text="Div00">
  <Layouts>
  <Layout>
  <Button id="Button00" text="Button00"/>
  </Layout>
  </Layouts>
</Div>
	{
	  "tag": "Div",
	  "attribute": {
	  "id": "Div00"
	  },
	  "Div": [
	  {
	  "Components": [
	  {
	  "tag": "Button",
	  "attribute": {
	  "id": "Button00",
	  "text": "Button00"
	  }
	  }
	  ]
	  }
	  ]
	}

You can also follow the basic conversion rules of configuring one tag as one JSON object, as shown below. Instead of Components property, use Layouts and Layout properties.

XML	JSON

<Div id="Div00" text="Div00">
  <Layouts>
  <Layout>
  <Button id="Button00" text="Button00"/>
  </Layout>
  </Layouts>
</Div>
	{
	  "tag": "Div",
	  "attribute": {
	  "id": "Div00"
	  },
	  "Div": [
	  {
	  "tag": "Layouts",
	  "Layouts": [
	  {

XML	JSON

	"tag": "Layout",
	"Layout": [
	{
	"tag": "Button",
	"attribute": {
	"id": "Button00",
	"text": "Button00"
	}
	}
	]
	}
	]
	}
	]
	}

28.4.4 Processing Event

The event can be processed through Action, but it can also be processed by adding the Script tag to the return value. In this case, you need to configure to call the script included in the View component.

When processing the event in Nexacro Studio, it is differentiated from being registered in the Form script in that it is processed as the innerform script of the View component. Therefore, when writing the onclick event, you need to access the ID of the corresponding View component.

> "Event Name": {View Component ID}.form.{Event Function Name}"onclick": "View00.form.btnAdd_onclick"

```
{
    "View":{
        "tag":"View",
        "attribute":{
            "id": "View00"
        },
        "Model": [
            {
                "fieldid": "",
```

```
                "Components": [
                    {
                        "tag": "Button",
                        "attribute": {
                            "id": "Button00",
                            "text": "Button00",
                            "onclick": "View00.form.btnAdd_onclick"
                        }
                    }
                ]
            }
        ],
        "Script": {
            "tag": "Script",
            "attribute": {
                "type": "xscript5.1"
            },
        "value": "this.btnAdd_onclick =
function(obj:nexacro.Button,e:nexacro.ClickEventInfo) {}"
}
```

28.5 View Layout Editor

If you set an item whose edittype is "Layout" in the fn_GetViewAttributeList function of the View template, you can use the View Layout Editor to set the layout information when setting the View Attributes item in the View Generation Wizard.

The set value is passed to the fn_GetViewGenerationResult function as a generationattr parameter and can be used to configure the View screen.

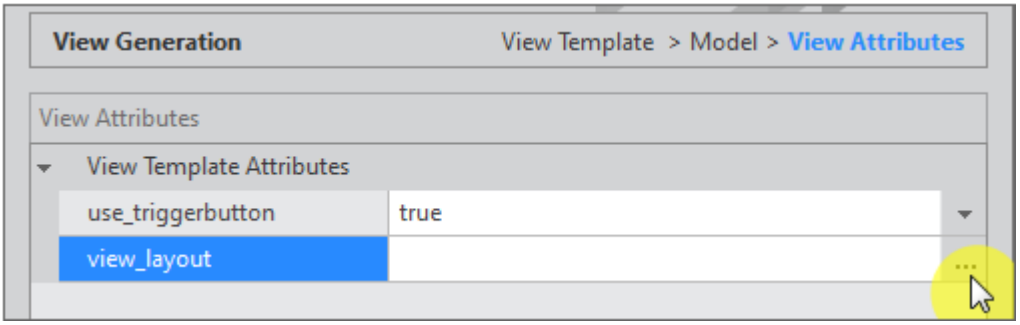


The layout information set in the View Layout Editor is passed as a parameter, and you must directly implement it to use the layout information when developing a View template.

An example of the case where the JSON value returned by the fn_GetViewAttributeList function is as follows.

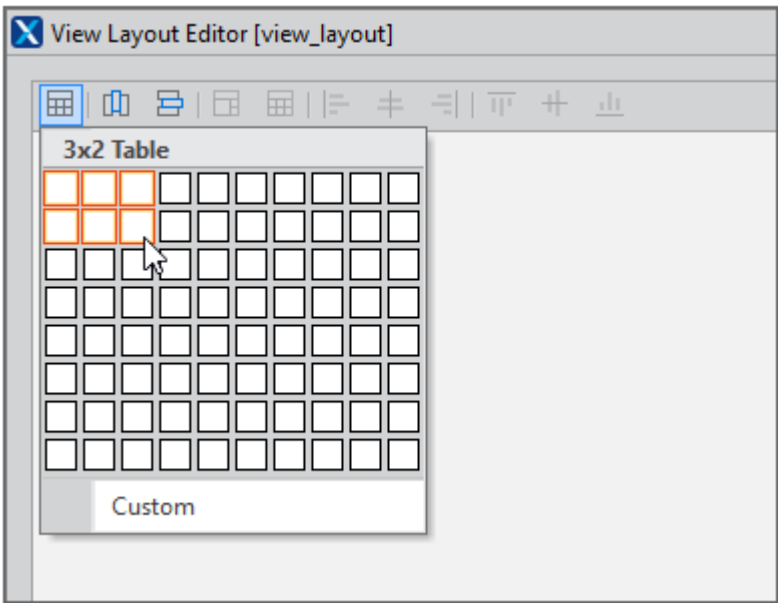
```
{
    "attributecount": 2,
    "attributes": [
        {
            "id": "use_triggerbutton",
            "edittype": "Boolean",
            "defaultvalue": "true"
        },
        {
            "id": "view_layout",
            "edittype": "Layout",
            "userproperties": {
                "propertycount": "1",
                "properties": [
                    {
                        "id": "gap",
                        "edittype": "Number",
                        "defaultvalue": 5
                    }
                ]
            }
        }
    ]
}
```

In the form design window, connect the model to the view component. View Layout Editor can be executed by selecting View Template in the View Generation Wizard and clicking the button displayed next to the item (view_layout) whose edittype is set to “Layout” in the View Attributes item setting window.

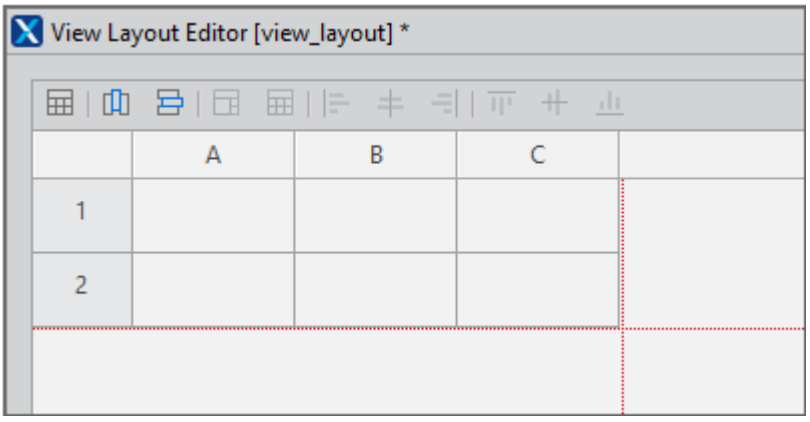


After launching View Layout Editor, click the first visible "Create Table" button and set the basic Column and Row of the layout. Up to 10 Columns and 8 Rows combinations can be selected with mouse operation in the checkerboard window, and for more values, select “Custom” and then directly enter the number of Columns and Rows.

The layout range that can be entered is limited to 1 to 30 for both Column and Row.

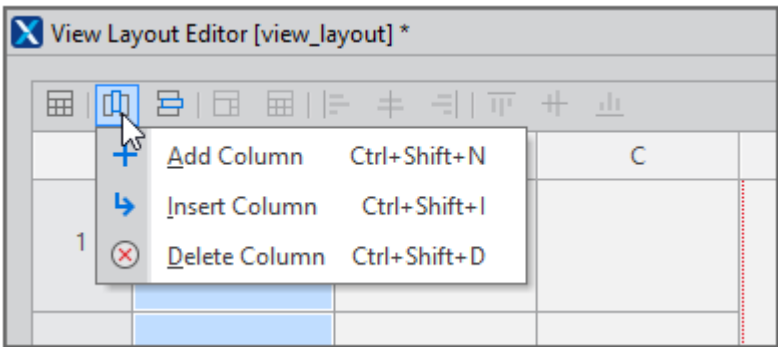


Column and Row sizes are automatically set to fill the View component. The size of each column and row can be changed in the property window. The red dotted border indicates the size of the View component.

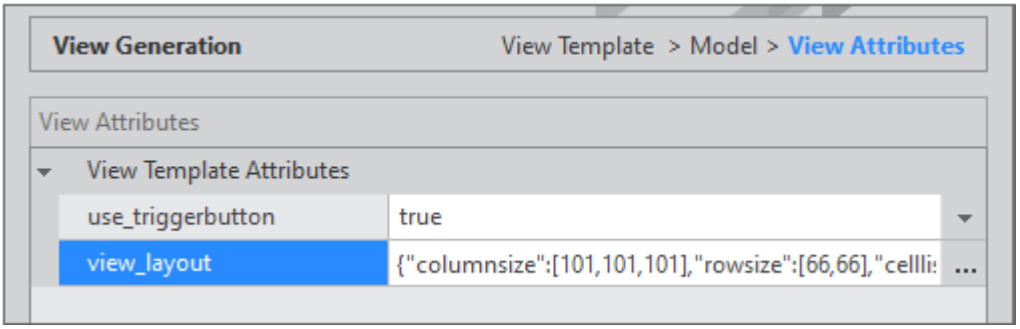


If you set the column and row size to -1, the value set for the column and row size is subtracted from the size of the view component, and the size of the remaining view components is automatically divided and set. For example, if the width of the View component is 400 and column A is set to 200 and columns B and C are set to -1, columns B and C are set to 100 ((400-200)/2) and displayed in the Editor.
The width display is a function limited to the Editor screen display, and the value return value is passed as -1, so the View Template developer must directly implement it.

After creating the basic table, you can add, insert or delete columns and rows from the "Edit Column", "Edit Row" button or the context menu.



If you click the [OK] button after setting the layout, the value of the item whose edittype is set as "Layout" is set.



28.5.1 Set cell properties

You can set cell properties after selecting cells in the View Layout Editor.

The properties that can be set by default are as follows.

- halign (left | center | right): property to horizontally align the model field to be displayed in the cell
- valign (top | middle | bottom): property to vertically align the model field to be displayed in the cell
- direction (horizontal | vertical): model field direction property to display in cell

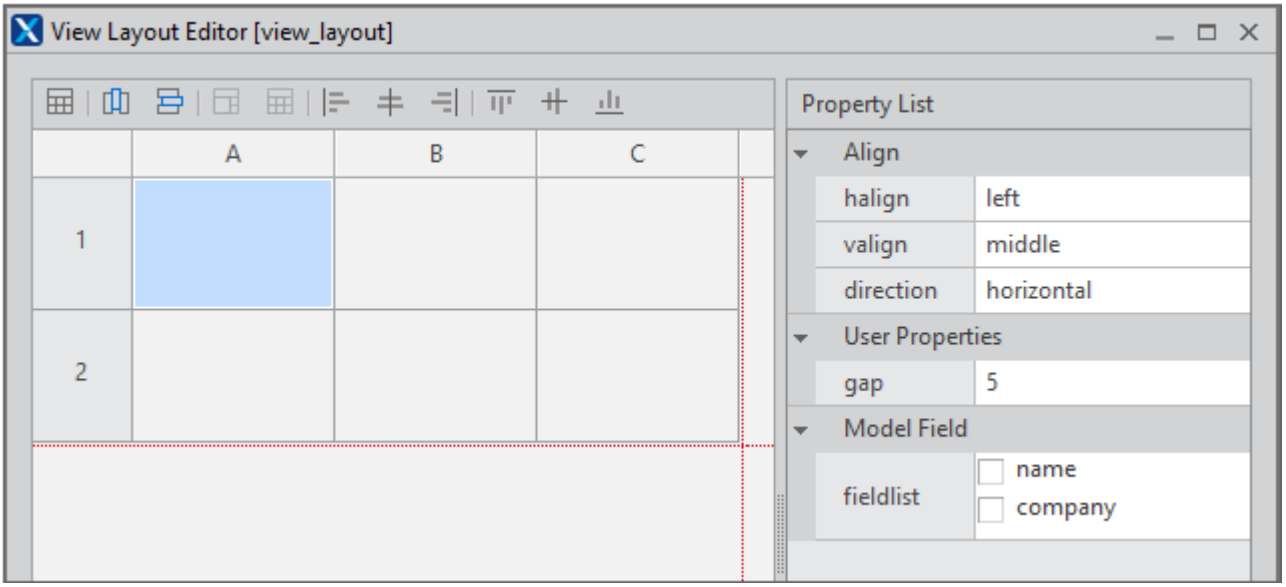
If you selected the selected field value when setting the model, you can set the field value to be applied to the cell.

- fieldlist: selected model field value

If the fn_GetViewAttributeList function returned userproperties values, you can set additional User Properties properties.

- User Properties: property information set by userproperties

If you select two model field values (name, company) and return one userproperties value (gap), the cell properties that can be set are as follows.



28.5.2 Merge or Unmerge Cells

Similar to the Grid Editor, it supports cell merge and release functions. You can merge cells by selecting two or more cells and selecting "Merget Cells" from the context menu, or you can break merged cells by selecting merged cells and selecting "Split Cell" from the context menu.

When merging cells, the column and row index information is set to a small value.

28.5.3 value return value JSON format

The value return value JSON format is shown below. The return value can be passed to the fn_GetViewGenerationResult function as a generationattr parameter and used to configure the View screen.

- columnsize - Column size array
- rowsize - Row size array
- celllist - array of cell properties
  - col - Column index
  - row - Row index
  - direction
  - User Properties
  - colspan - when cells are combined horizontally
  - rowspan - when cells are merged vertically
  - fieldlist - set field value array

```
{
    "columnsize":[101,101,101],
    "rowsize":[66,66],
    "celllist":[
        {
            "col":"0",
            "row":"0",
            "halign":"left",
            "valign":"top",
            "direction":"horizontal",
            "gap":"5",
            "fieldlist":["name"]
        },
        {
            "col":"0",
            "row":"1",
            "halign":"center",
            "valign":"middle",
            "direction":"vertical",
            "gap":"5",
            "colspan":"2",
            "fieldlist":["name","company"]
        }
    ]
}
```

28.6 XML <-> JSON Converter

It supports the ability to convert XML code to JSON code or JSON code to XML code.

It can be used to check the result of the View component information being passed as the contents parameter to the View template fn_GetViewGenerationResult function, or to create JSON data to be returned from the fn_GetViewGenerationResult function.

28.6.1 Where features are available

The XML <-> JSON Converter feature can be accessed in several ways.

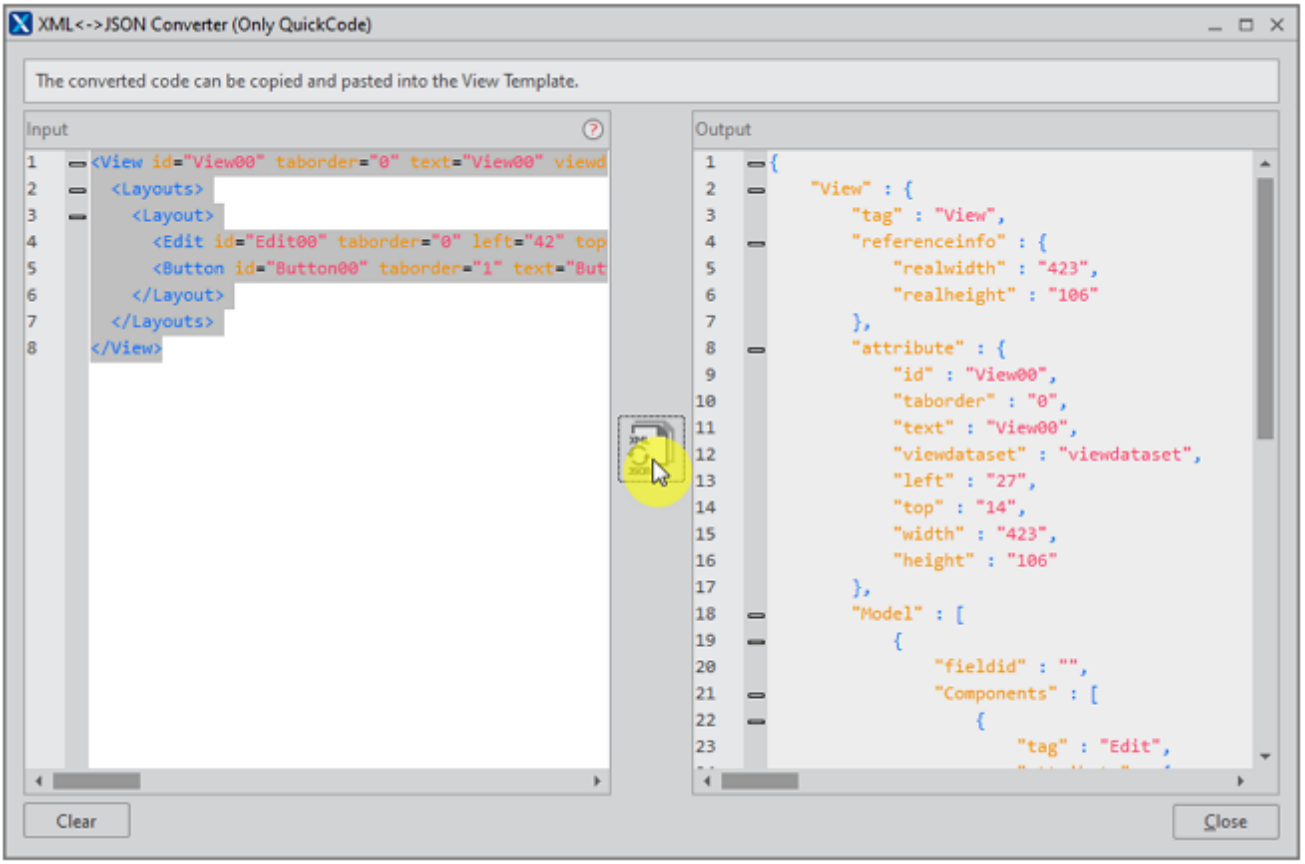
- Nexacro Studio menu [QuickCode > XML<->JSON Converter]
- context menu
  - Form design view, Source window
  - When selecting a component (invisible object) in the Form design view
  - When selecting a code block in the Form Source window
  - View Template code editing window
  - When selecting Form in Project Explorer
- Button at the top of the code editing window
  - When editing the View Template code

28.6.2 Automatically select code to convert

Provides the ability to automatically select the code to convert based on the approach.

- Form design view
  - When a specific component (object) is not selected
  - Layout XML code corresponding to the current layout
  - Corresponding Layout XML code according to the selected Layout
  - When a specific component (object) is selected
    Selected component (object) XML code (multiple selection is also supported)
- Form source view
  - select the XML code region to convert
  - Treat the selected code as code to convert
- View template editor
  - Select the JSON or XML code area to convert
  - Treat the selected code as code to convert

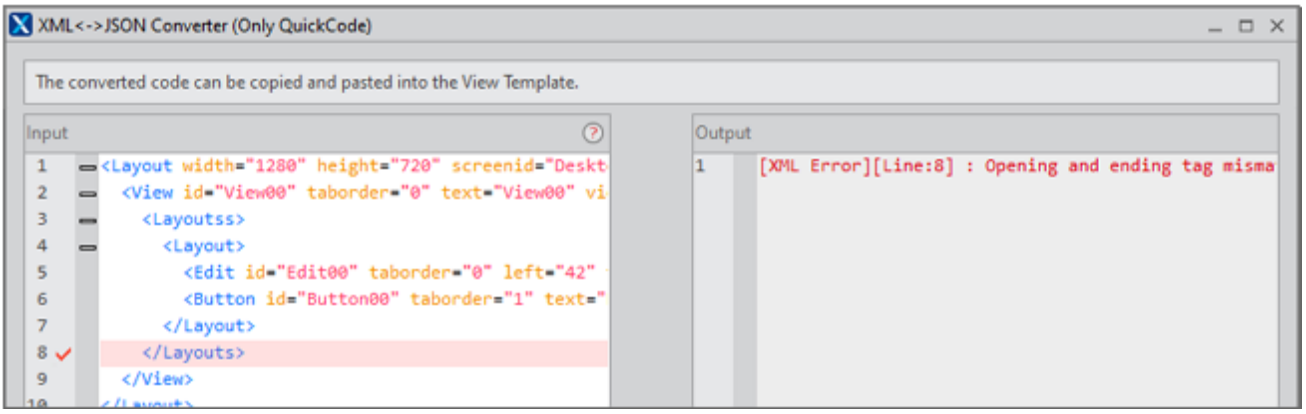
The automatically selected code is copied to the input area and converted to JSON code when the convert button in the middle is clicked. The converted JSON code is copied to the clipboard.



28.6.3 Direct code entry

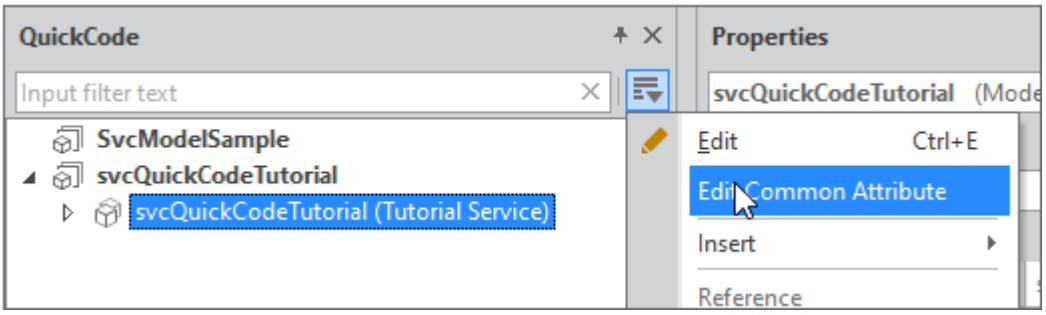
You can convert by directly entering XML or JSON code in the input area. It automatically recognizes XML or JSON code and handles XML->JSON or JSON->XML conversions.

□□ □□□ □□□ □□□ □□ □□ □□ □□ □□ □ Output □□ □□ □□□□ □□□□□.

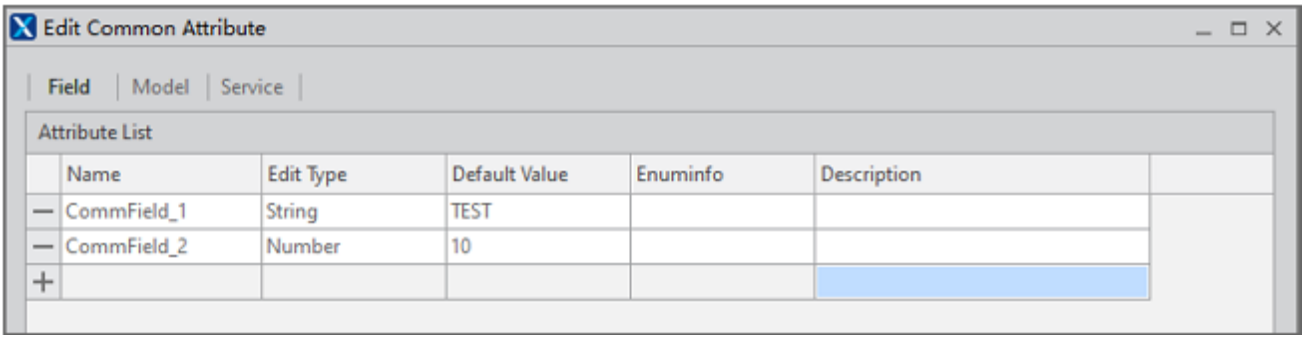


28.7 Edit Attribute

If you select "Edit Common Attribute" from the QuickCode panel menu, you can add attributes to Service, Model, and Field within the project. Attributes are added to existing items as well as when new items are created.

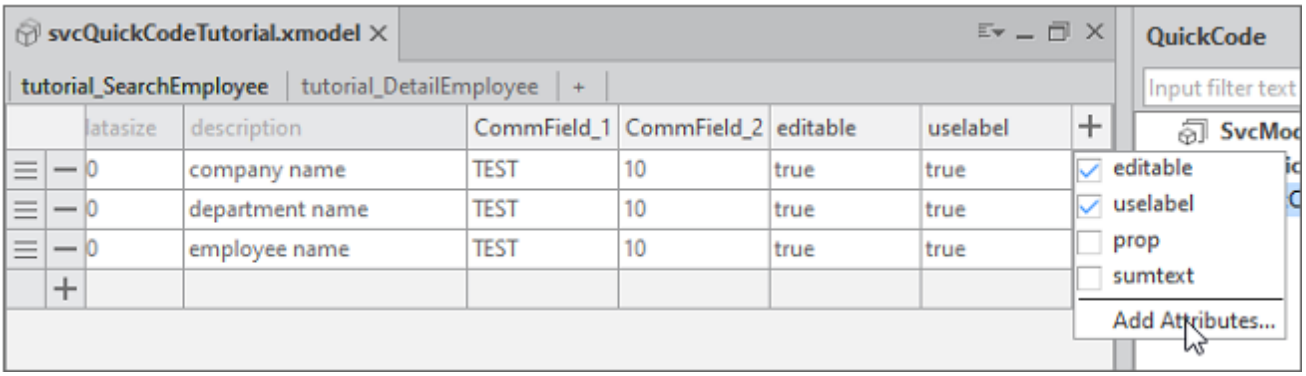


When the Edit Common Attribute window is executed, select "Field", "Model", and "Service" items and add Attribute.

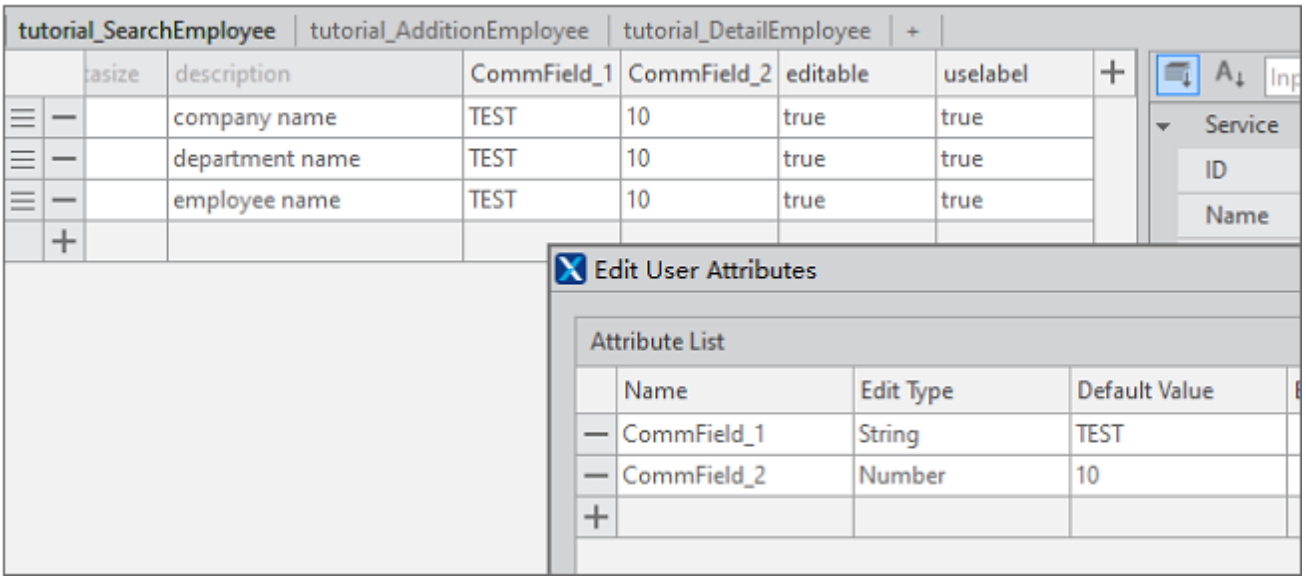


28.7.1 Edit Field Attributes

If you select "Add Attributes" in the Model edit window, the "Edit User Attributes" window will be displayed.

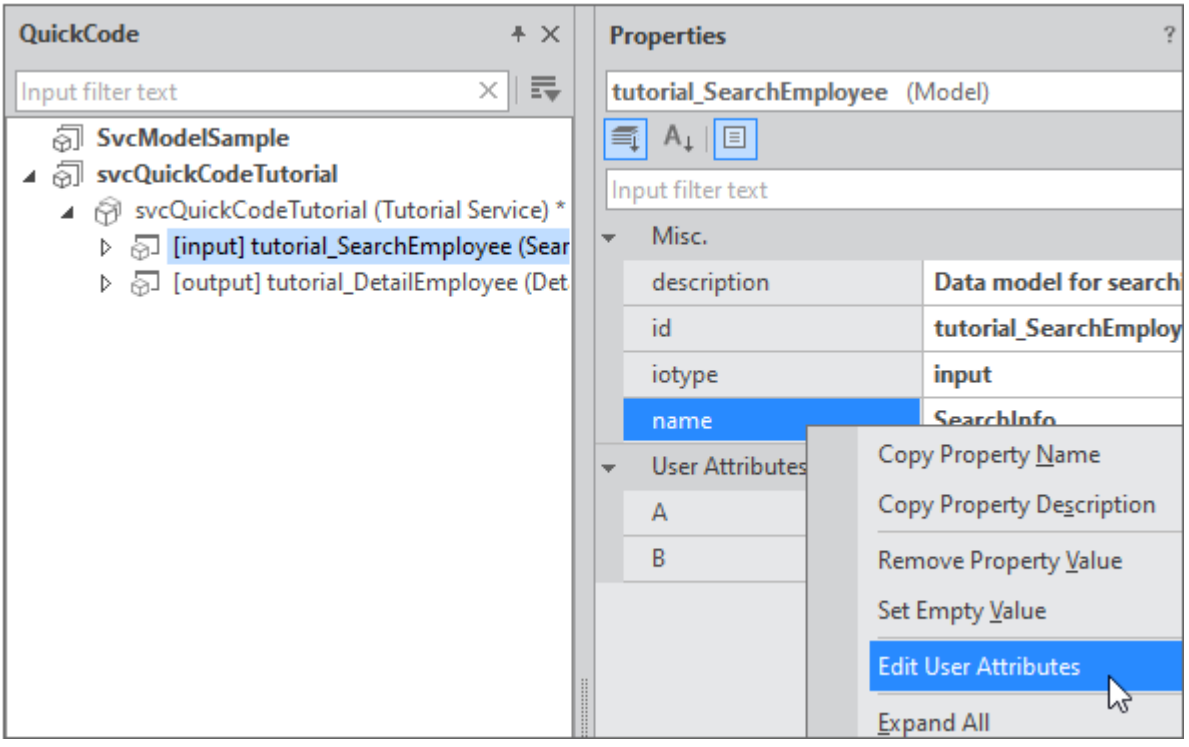


In the "Edit User Attributes" window, you can edit or delete items set as "Common Attribute" and add Field Attributes to be used only in the model.

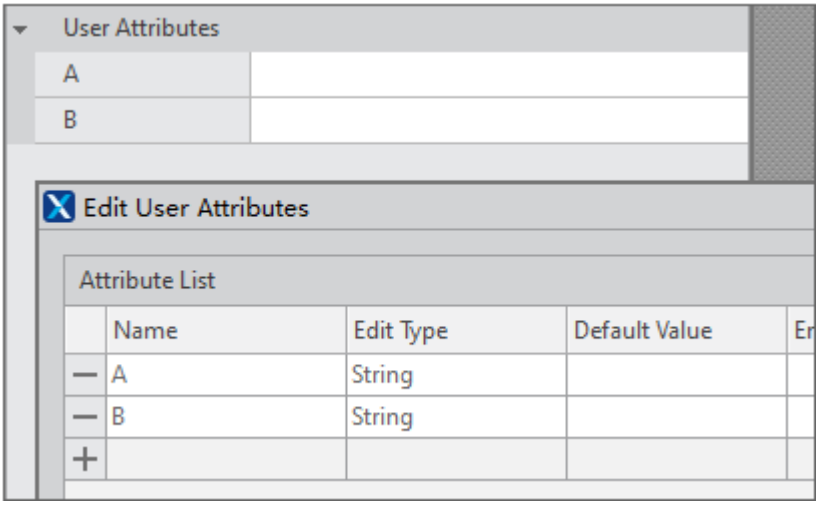


28.7.2 Edit Model Attribute

When selecting a model in the QuickCode panel, Model Attribute is displayed in the properties window.



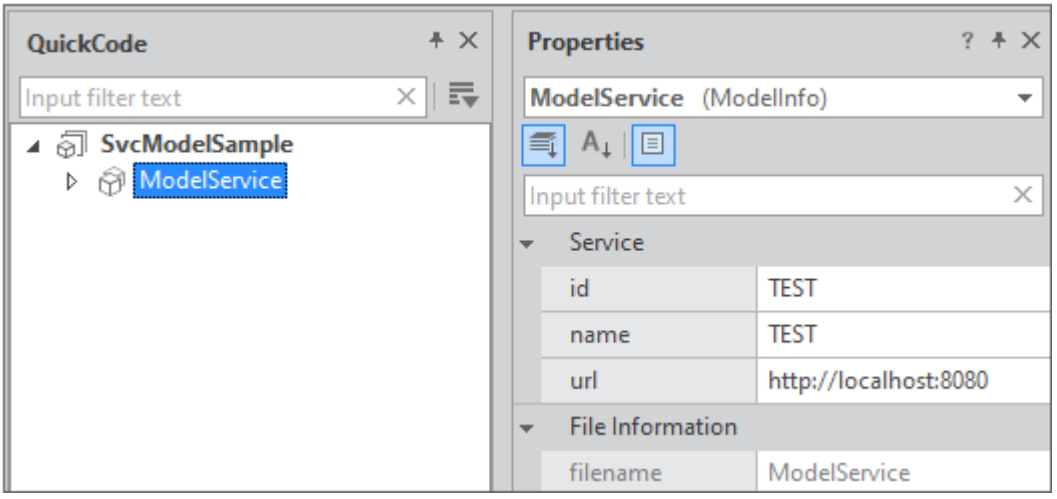
Select the "Edit User Attributes" item from the context menu to display the "Edit User Attributes" window.



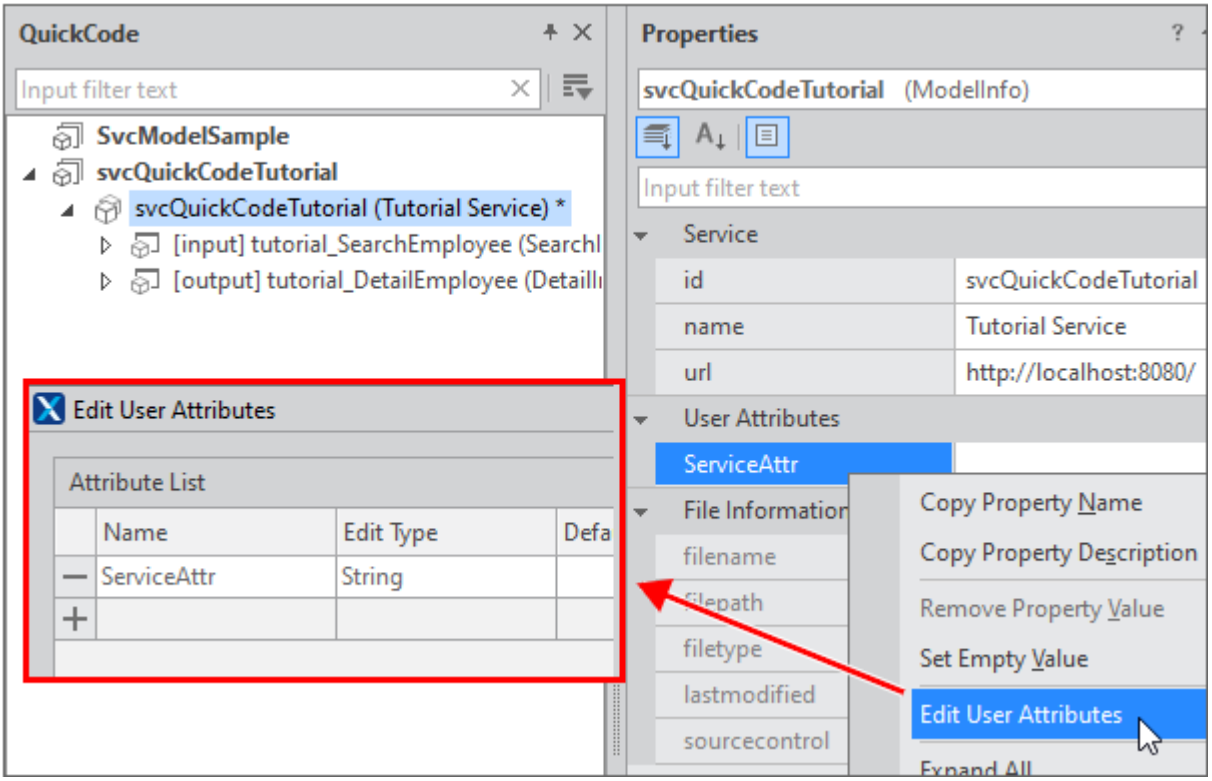
In the "Edit User Attributes" window, you can edit or delete the items set as "Common Attribute" and add attributes to be used only in the model.

28.7.3 Edit Service Attribute

Double-click the Service item in the QuickCode panel or select "Edit" from the context menu to activate the item in the Service group in the property window.



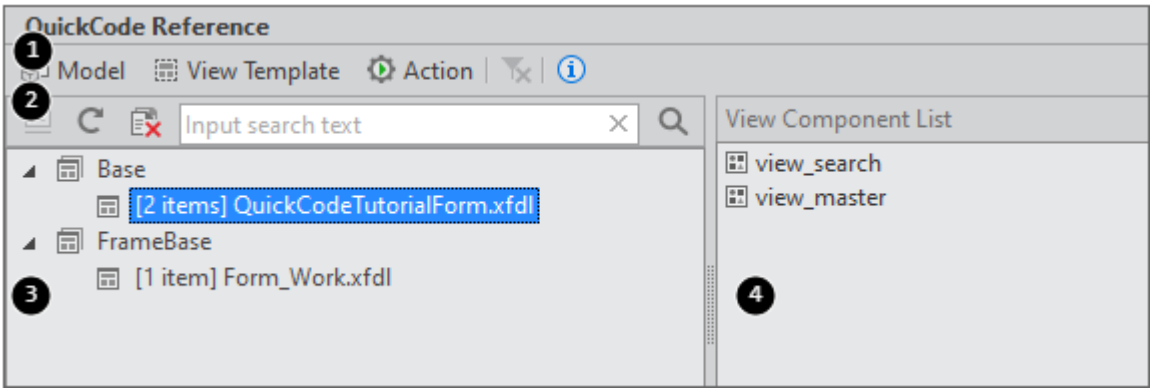
Select the "Edit User Attributes" item from the context menu of the property window to display the "Edit User Attributes" window.



In the "Edit User Attributes" window, you can edit or delete items set as "Common Attribute" and add attributes to be used only in the corresponding Service.

28.8 QuickCode Reference

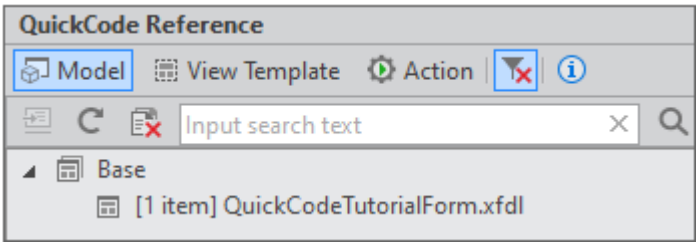
QuickCode Reference provides a function to find Form objects and View components that use the corresponding items within a project when modifying Model, View Template, and Action items. You can select whether to display the panel in the menu [QuickCode > View > QuickCode Reference].



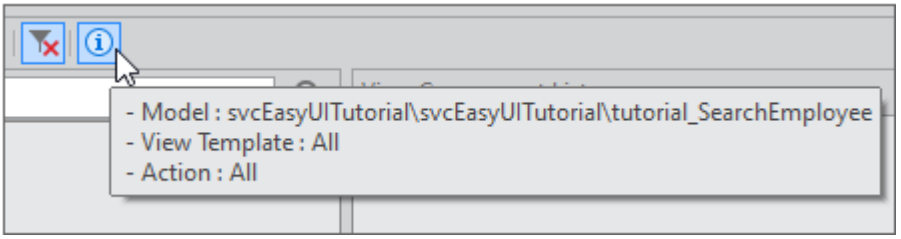
		Description
1	filter toolbar	Show selected items in the QuickCode panel or delete filters.
2	main toolbar	Go to Component: Moves the focus to the selected View component. Refresh: Requeries the Reference list again. Clear All: Deletes the Reference list. Find: Executes the keyword search function.
3	Reference list	Displays a list of Form objects that use the View component in the project. When a filter is applied, only applicable lists are displayed.
4	View Component list	Displays a list of View components that meet the conditions in the selected Form object.

28.8.1 filter

When "Reference" is selected from the context menu after selecting Model, View Template, or Action in the QuickCode panel, a filter can be added to display only the results using that item. Applied filter items are displayed in blue. Clicking on an item marked in blue turns off that filter.

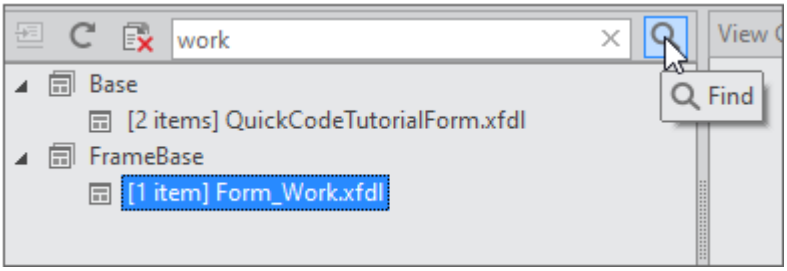


If you select the information display icon, you can check the currently applied filter information.



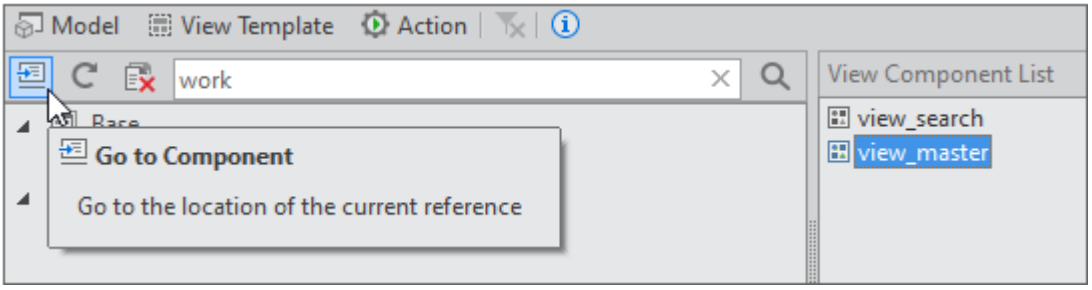
28.8.2 search

If you still have a lot of results after applying filters, keyword search can help you find what you're looking for. After entering a keyword, click the Search button to highlight items corresponding to the keyword.



28.8.3 Move the View component

Selecting a View component from the search results and clicking the "Go to Component" button or double-clicking a View component executes the Form object containing the View component and moves the focus to the View component.



28.9 Querying the Model service

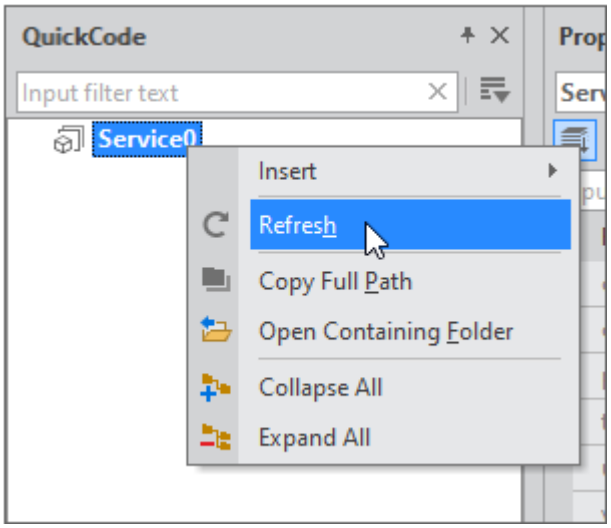
If you register URL, ServiceList, and DataSchema when registering a model type service in TypeDefinition, you can search data from the service and import Model Group, Model Service, and Model items.

28.9.1 Querying the Model Group, Model service

1. Add service item in TypeDefinition.

```
Type: model
URL: https://edu.tobesoft.com/developer_guide/model/
ServiceList: ModelList.jsp
DataSchema: ModelList.jsp
```

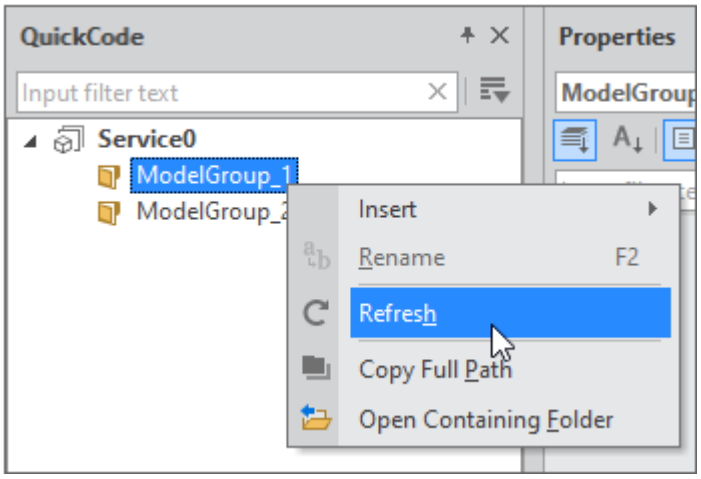
2 Select the service item in the QuickCode panel and select the "Refresh" item in the context menu.



When selecting an item, the service set in the service URL and ServiceList is searched. The request format is shown below.

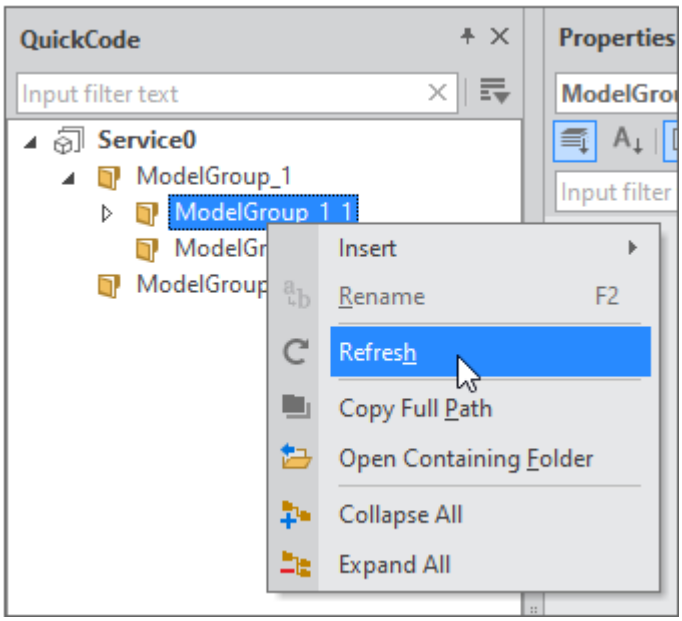
```
[URL][ServiceList]?group=/
https://edu.tobesoft.com/developer_guide/N/Service/modellist.jsp?group=/
```

3 Select the resulting Model Group, Model Service and select the "Refresh" item from the context menu.



When selecting an item, the service set in the service URL and ServiceList is searched. The request format is shown below.

```
[URL][ServiceList]?group=[Selected Model Group or Model Service]
https://edu.tobesoft.com/developer_guide/N/Service/modellist.jsp?group=Mod
elGroup_1
```

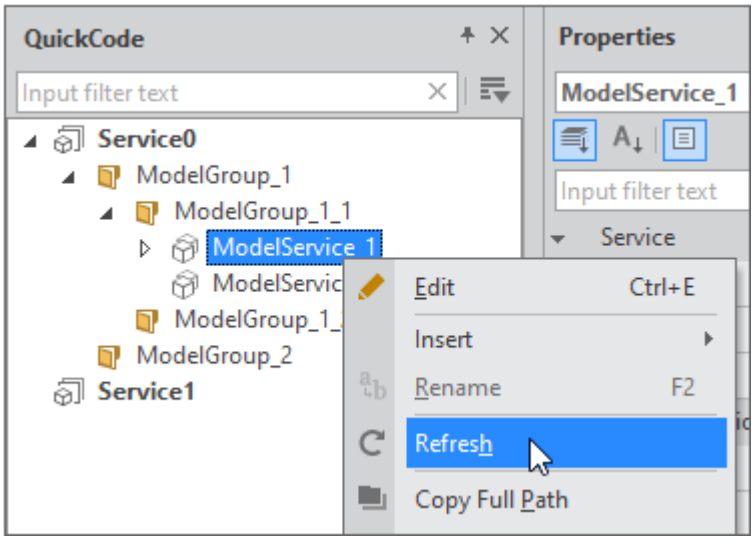


If there is a model group under the model group, the service is searched by displaying the path with "/" character. The request format is shown below.

```
[URL][ServiceList]?group=[parent Model Group]/[selected Model Group]
https://edu.tobesoft.com/developer_guide/N/Service/modellist.jsp?group=Mod
elGroup_1/ModelGroup_1_1
```

28.9.2 Querying the Model

1. Select the resulting Model Service and choose the "Refresh" item from the context menu.

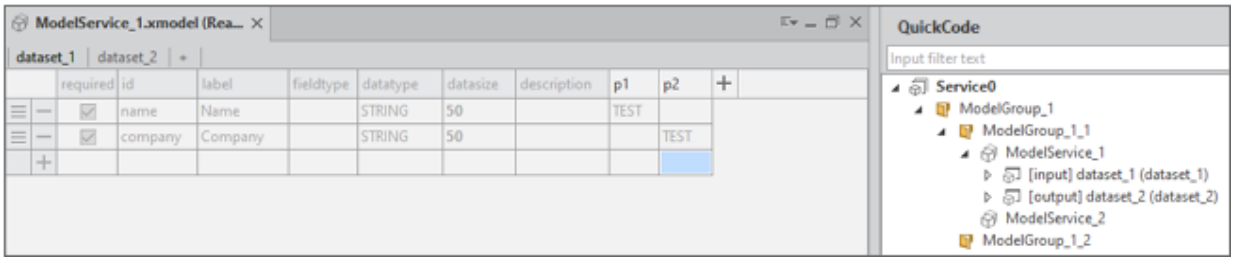


When looking up Model, use DataSchema instead of ServiceList in service item to look up information.

The request format is shown below.

```
[URL][DataSchema]?group=[Model Group]&id[Selected Model Service]
https://edu.tobesoft.com/developer_guide/N/Service/modellist.jsp?group=ModelGroup_1/ModelGroup_1_1&id=ModelService_1
```

2 You can select the Model item and check the field information.



Model items retrieved from the service cannot be edited.
All functions that add new fields or attributes are displayed as inactive.

28.9.3 Response data format - ErrorCode, ErrorMsg

Tag	Value	Required	Description
Success			
ErrorCode	0	O	
ErrorMsg		X	On success, the message is not processed.
Error			
ErrorCode	-1	O	
ErrorMsg		O	Received messages are displayed on the screen.

28.9.4 Response data format - Model Group, Model Service

Tag	Attribute	Required	Description
ModelGroup	id	O	Set the id value.
	type	O	group: Model Group service: Model Service

[URL][ServiceList]?group=/

```
<?xml version='1.0' encoding='utf-8'?>
<Root>
    <ErrorCode>0</ErrorCode>
    <ErrorMsg>Success</ErrorMsg>

    <ModelGroup id="ModelGroup_1" type="group"/>
    <ModelGroup id="Service_1" type="service"/>
</Root>
```

[URL][ServiceList]?group=ModelGroup_1

```
<?xml version='1.0' encoding='utf-8'?>
<Root>
    <ErrorCode>0</ErrorCode>
    <ErrorMsg>Success</ErrorMsg>

    <ModelGroup id="ModelGroup_1">
        <ModelGroup id="ModelGroup_1_1" type="group"/>
        <ModelGroup id="Service_1_1" type="service"/>
    </ModelGroup>
</Root>
```

28.9.5 Response Data Format - Model

Tag	Attribute	Required	Description
ModelInfo	version		"1.2"
Models			Model parent tag
Model	id	O	id
	name		name
	iotype		Input/output type (input/output)
	[attr]		ModelUserAttributes > Attribute
Field	id	O	id
	label	O	label information
	fieldtype	O	type (FreeText/SelectOne/SelectAny/MultiMedia)
	datatype	O	data type (STRING/INT/FLOAT/BIGDECIMAL/DATE/TIME/DATETIME/BLOB)

Tag	Attribute	Required	Description
	datesize	O	data size
	description		explanation
	required	O	required (true/false)
	[attr]		FieldUserAttributes > Attribute
FieldUserAttributes			Attribute parent tag when setting FieldUserAttributes
ModelUserAttributes			Attribute parent tag when setting ModelUserAttributes
Service	id	O	id
	name		name
	url	O	Business Service to use when requesting transaction
	[attr]		ServiceUserAttributes > Attribute
ServiceUserAttributes			Attribute parent tag when setting ServiceUserAttributes

ModelUserAttributes, FieldUserAttributes, ServiceUserAttributes □□ □□□ □□ Attribute □□□ □□□ □□ □□□ □□□□.

Tag	Attribute	Required	Description
Attribute	name	O	name
	edittype	O	edit type (Boolean/Enum/Number/String/ViewObjList)
	defaultvalue		default value
	enuminfo		Enum information
	description		explanation

> [URL][DataSchema]?group=ModelGroup_1&id=Service_1

```
<ModelInfo version="1.2">
    <Models>
        <Model id="dataset_1" name="dataset_1" iotype="input" p1="TEST">
            <Field id="name" datatype="STRING" datasize="50"  label="Name"
fieldType="FreeText"  description=""  required="true"  p1="TEST"  p2="" />
            <Field id="company" datatype="STRING" datasize="50"  label=
"Company"  fieldType="FreeText"  description=""  required="true"  p1=""  p2=
"TEST" />
            <FieldUserAttributes>
                <Attribute name="p1" edittype="String"/>
```

```
                <Attribute name="p2" edittype="String"/>
            </FieldUserAttributes>
            <ModelUserAttributes>
                <Attribute name="p1" edittype="String"/>
            </ModelUserAttributes>
        </Model>
        <Model id="dataset_2" name="dataset_2" iotype="output">
            <Field id="B1" datatype="INT" datasize="10"  label="1"  fieldType=
"FreeText"  description=""  required="true" />
            <Field id="B2" datatype="STRING" datasize="50"  label="1"
fieldType="FreeText"  description=""  required="true" />
        </Model>
    </Models>
    <Service id="svcSearchEmployee" name="SearchEmployee" url=
"http://localhost:8080/searchEmployeeInfo.jsp?" svc_attr01=""/>
    <ServiceUserAttributes>
       <Attribute name="svc_attr01" edittype="String"/>
    </ServiceUserAttributes>
</ModelInfo>
```

28.9.6 JSP code example

This is the JSP code used to demonstrate how it works. Returns XML code when searching for a specific Model Group, Model Serivce, or Model item.

```
<%@ page contentType = "text/xml" %>
<%
    String str_group = request.getParameter("group")==null?""
:request.getParameter("group");
    String str_id = request.getParameter("id")==null?"":request.getParameter(
"id");
    if(str_id.equals(""))
    {
        if(str_group.equals("/"))
        {
%>
<Root>
  <ErrorCode>0</ErrorCode>
  <ErrorMsg>Success</ErrorMsg>
```

```
  <ModelGroup id="ModelGroup_1" type="group"/>
  <ModelGroup id="ModelGroup_2" type="group"/>
</Root>

<%
        } else if(str_group.equals("ModelGroup_1")){
%>
<Root>
  <ErrorCode>0</ErrorCode>
  <ErrorMsg>Success</ErrorMsg>

  <ModelGroup id="ModelGroup_1">
    <ModelGroup id="ModelGroup_1_1" type="group"/>
    <ModelGroup id="ModelGroup_1_2" type="group"/>
  </ModelGroup>
</Root>
<%
        } else if(str_group.equals("ModelGroup_1/ModelGroup_1_1")){
%>
<Root>
  <ErrorCode>0</ErrorCode>
  <ErrorMsg>Success</ErrorMsg>

  <ModelGroup id="ModelGroup_1_1">
    <ModelGroup id="ModelService_1" type="service"/>
    <ModelGroup id="ModelService_2" type="service"/>
  </ModelGroup>
</Root>
<%
        } else if(str_group.equals("ModelGroup_2")){
%>
<Root>
  <ErrorCode>0</ErrorCode>
  <ErrorMsg>Success</ErrorMsg>

  <ModelGroup id="ModelGroup_2">
    <ModelGroup id="ModelService_3" type="service"/>
    <ModelGroup id="ModelService_4" type="service"/>
  </ModelGroup>
</Root>
```

```
<%
        } else {
%>
<Root>
  <ErrorCode>-1</ErrorCode>
  <ErrorMsg>Error</ErrorMsg>
</Root>
<%
        }
    } else if(str_id.equals("ModelService_1")){
%>
<Root>
  <ErrorCode>0</ErrorCode>
  <ErrorMsg>Success</ErrorMsg>

<ModelInfo version="1.2">
<Models>
<Model id="dataset_1" name="dataset_1" iotype="input" p1="TEST">
    <Field id="name" datatype="STRING" datasize="50"  label="Name"  fieldType=
"FreeText"  description=""  required="true"  p1="TEST"  p2="" />
    <Field id="company" datatype="STRING" datasize="50"  label="Company"
fieldType="FreeText"  description=""  required="true"  p1=""  p2="TEST" />
<FieldUserAttributes>
    <Attribute name="p1" edittype="String"/>
    <Attribute name="p2" edittype="String"/>
</FieldUserAttributes>
<ModelUserAttributes>
    <Attribute name="p1" edittype="String"/>
</ModelUserAttributes>
</Model>
<Model id="dataset_2" name="dataset_2" iotype="output">
    <Field id="B1" datatype="INT" datasize="10"  label="1"  fieldType=
"FreeText"  description=""  required="true" />
    <Field id="B2" datatype="STRING" datasize="50"  label="1"  fieldType=
"FreeText"  description=""  required="true" />
</Model>
</Models>
<Service id="svcSearchEmployee" name="SearchEmployee" url=
"http://localhost:8080/searchEmployeeInfo.jsp?" svc_attr01=""/>
    <ServiceUserAttributes>
       <Attribute name="svc_attr01" edittype="String"/>
```

```
    </ServiceUserAttributes>
</ModelInfo>
</Root>
<%
    } else if(str_id.equals("ModelService_3")){
%>
<Root>
  <ErrorCode>0</ErrorCode>
  <ErrorMsg>Success</ErrorMsg>

<ModelInfo version="1.2">
<Models>
<Model id="dataset_1" name="dataset_1" iotype="input">
    <Field id="name" datatype="STRING" datasize="50"  label="Name"  fieldType=
"FreeText"  description=""  required="true" />
    <Field id="company" datatype="STRING" datasize="50"  label="Company"
fieldType="FreeText"  description=""  required="true" />
</Model>
</Models>
</ModelInfo>
</Root>
<%
    } else {
%>
<Root>
  <ErrorCode>-1</ErrorCode>
  <ErrorMsg>Error</ErrorMsg>
</Root>
<%
    }
%>
```

Part III.

Console Tools

29.

Nexacro Deploy

29.1 Generating Code

If you develop apps based on Nexacro, all those programs will be converted into either JavaScript files or CSS files. This conversion process is called generation. Generated apps can be run on web browsers powered by operating systems. Otherwise, they are available in the Nexacro runtime environment (NRE).

To deploy a project that you develop through Nexacro Studio, you need to convert the codes of the project so that they are ready to be deployed. This conversion process is called "generation". You can generate codes through the menu [Generate] or [Deploy] in Nexacro Studio. However, it is better to use Nexacro Deploy when a project's scale is large or when you use an automatic build system. Nexacro Deploy is a tool for automatic generation and deployment.

29.2 Types of Nexacro Deploy

You can select and use the appropriate Nexacro Deploy according to your purpose.

	nexacrodeploy.exe	**Nexacro Deploy JAVA**
distribution method	Install with Nexacro Studio	Distributed separately as a jar file
system requirements	Same as Nexacro Studio	Install JDK 1.8.0_131 or higher Need to set JAVA_HOME environment variable
executable file name	Tools\nexacrodeploy.exe	bin\start.bat bin\start.sh

29.3 Options

29.3.1 Required Options

The below four options must be specified.

```
nexacrodeploy.exe -P [ARG] -O [ARG] -B [ARG] -GENERATERULE [ARG]
start.bat -P [ARG] -O [ARG] -B [ARG] -GENERATERULE [ARG]
start.sh  -P [ARG] -O [ARG] -B [ARG] -GENERATERULE [ARG]
```

Option	Description
-P [ARG]	Specifies the path to the XPRJ file that contains a project to process.
-O [ARG]	Specifies the output path where generate files are stored. Output path will be created automatically if nothing is specified.
-B [ARG]	Specifies the path to the library files needed for generating files.
-GENERATERULE [ARG]	Specify the path where the Generate Rule file is located.



The generated (deployed) file in the path specified with the -O(-D) option is not deleted even if you change the option specifying the work target or execute Generate(deploy) after deleting the file from the project. If necessary, the operator must manually delete the file.



If a file is deleted or renamed directly in the Nexacro Studio Project Explorer, the file created in the Ouputs path is also deleted or the file name is changed. This function is not a Nexacro Deploy function, but a function supported by Nexacro Studio.



The -CSSRULE option in the previous versions, which specified the path for the CSS Rule files, are no longer supported.
Rule files required for Generate, including CSS Rule files, have been added. The -GENERATERULE option has been changed to be required.

Delete the -CSSRULE option from the existing script and add the -GENERATERULE option.
The Generate Rule file is located under the [SDK > {24.X.X.X} > generate] folder and the path can be directed to another location by the user.

29.3.2 Options for Specifying Target Files

If you do not set the below options, the deployment tool will work on all the files in a project.

Option	Description
-FILE [ARG...]	Specifies a list of paths to files to generate. Specifies IDs of themes to generate. Separated by a comma or semicolon when multiple inputs exist. The entire list of files must be enclosed in double quotation marks while each individual file is in single quotation marks.
-SERVICE [ARG...]	Specifies the service prefix ID list to Generate. Separates multiple inputs with ','(Comma) or ';'(Semicolon).
-BOOTSTRAP [ARG...]	Specifies the OS to which the created bootstrap files will apply. Separated by a comma or semicolon when multiple inputs exist.
-MODULE	Used to generate only module files.



If you do not specify the -FILE, -SERVICE, -BOOTSTRAP or -MODULE option, the deployment tool will deal with all the cases related to the project.



If there is a file that affects the bootstrap file such as the XADL file or environment.xml among the files specified with the -FILE option, then the bootstrap file may be recreated.



If multiple files are specified in the -FILE or -BOOTSTRAP option, there must be no white space between them.

```
(X) -FILE "'C:\a.xfdl', 'C:\b.xfdl'
(O) -FILE "'C:\a.xfdl','C:\b.xfdl'
```

29.3.3 Generate Options

Option	Description
-REGENERATE	Generates all files regardless of changes in files. If you do not specify this option, unchanged files will not be generated.
-BROWSER [ARG...]	Specifies certain web browsers for which XCSS and Xtheme files will be generated. Separated by a comma or semicolon when multiple inputs exist.

Option	Description
	Note: you cannot specify a browser that is not supported by Nexacro.
-UNARCHIVE [ARG...]	The operating system Bootstrap file that has been set will not be archived. (A Run.html file is created instead of a Run.zip file). Android,iOS Android iOS
-JSVERSION	Set script validation rules to be applied when generating. If no option value is specified, the default is "ECMAScript 2015". You can choose "ECMAScript 5" or "ECMAScript 2015". When using the -JSVERSION option, Regenerate is performed on related items regardless of whether the file has been modified or not.
-DEFER	When creating the Bootstrap file, set the defer attribute within the Script tag.
-SPLASH	Sets whether or not to include a script to display Splash Loader when bootstrap file is created.
-RTL	When theme and xcss file are generated, a results file where RTL has been applied will be included.



You can specify below browsers as arguments of the -BROWSER option.
NRE
Nexacro Browser (deprecated)
Internet Explorer 11
Internet Explorer 10
Chrome
Firefox
Opera
Safari

Internet Explorer 10 and 11 are not supported if the library set in the -B option is Version 24.0.0.100 and up. Files will not be generated even if it is specified as an option.

If you want to specify the NRE and Chrome, write code as below.

```
nexacrodeploy.exe -P -O -B -GENERATERULE -BROWSER "NRE,Chrome"
start.bat -P -O -B -GENERATERULE -BROWSER "NRE,Chrome"
start.sh -P -O -B -GENERATERULE -BROWSER "NRE,Chrome"
```



It will generate files for all the browsers if you do not specify the -BROWSER option.



Currently, this command is applicable only to iOS, whose bootstrap file is Run.zip. The Run.html file will be created instead of Run.zip.



If multiple browsers are specified in the -BROWSER option, there must be no white space between them.

```
(X) -BROWSER "NRE, Chrome"
(O) -BROWSER "NRE,Chrome"
```

29.3.4 Deploy Options

Option	Description
-D [ARG]	Specifies the output path where deployed files are stored. Output path will be created automatically if a specified path is non-existent. Deployment will be conducted if this option is used. You cannot use the path specified by the '-O' option.
-MERGE	Creates a single JS file by merging the JS files defined in a JSON file.
-MERGEXCSS	Multiple xcss files added to the application will be combined into a single file.
-COMPRESS	Compresses JS files when deploying.
-SHRINK	Obfuscates variable names. A function will not be obfuscated if it contains an eval statement.
-IGNORECOMPRESS [ARG]	Specifies path for the *.ignorecompress* file, which stores a list of files that are not subject to compression. Note: this option is available only when the '-COMPRESS' option is applied.
-IGNOREEVAL	Conducts obfuscation regardless of eval functions. (not recommended) Note: this option is available only when the '-SHRINK' option is applied.
-COMPILE	Compiles files that will be used in the NRE. Note: only the NRE supports the files produced as a result of applying this option.
-PRJURL [ARG]	Specifies the Project URl to load the App. Note: Option for iOS only.

29.3.5 Other options

Option	Description
-L [ARG]	Specifies the path to a log file to save the process. Output path will be created automatically if nothing is specified. Creating a log file may increase the processing time.
-H | -? | -HELP	Provides the help for Nexacro Deploy.

29.4 Examples



The usage example was written based on nexacrodeploy.exe.

29.4.1 Generate

Generate entire Application

- You can generate an Application based on *C:\TestGenerate\TestGenerate.xprj* file and produce the result to the *E:\ResultGenerate* folder.
- You also can generate the files necessary to run the app (the console does not generate bootstrap files except for the ones related to the execution of the NRE or web browsers.)
- The generation is conducted by referring to the path of base libraries, specified as the -B option.

```
nexacrodeploy.exe  -P "C:\TestGenerate\TestGenerate.xprj" -O
"E:\ResultGenerate" -B "C:\TestGenerate\nexacrolib" -GENERATERULE
"C:\TestGenerate\generate"
```

Generate only certain file

- You can generate the *C:\TestGenerate\aa.xfdl* file specified as the -FILE option and produce the result to the *E:\ResultGenerate* folder.

```
nexacrodeploy.exe -P "C:\TestGenerate\TestGenerate.xprj" -O "E:\ResultGenerate"
-B "C:\TestGenerate\nexacrolib" -GENERATERULE "C:\TestGenerate\generate" -FILE
"C:\TestGenerate\aa.xfdl"
```

- You also can generate a theme file by specifying the ID of the target theme file.

```
nexacrodeploy.exe -P "C:\TestGenerate\TestGenerate.xprj" -O "E:\ResultGenerate"
-B "C:\TestGenerate\nexacrolib" -GENERATERULE "C:\TestGenerate\generate" -FILE
"theme::default"
```

- You can generate *n* number of files and produce the result to the *E:\ResultGenerate* folder.

```
nexacrodeploy.exe -P "C:\TestGenerate\TestGenerate.xprj" -O "E:\ResultGenerate"
-B "C:\TestGenerate\nexacrolib" -GENERATERULE "C:\TestGenerate\generate" -FILE
"'C:\TestGenerate\aa.xfdl','C:\TestGenerate\bb.xfdl','C:\TestGenerate\cc.xfdl'"
```

When generating only specific services

- Generate only the files corresponding to the Base service specified with the -SERVICE option and then create them under the "E:\ResultGenerate" folder.

```
nexacrodeploy.exe -P "C:\TestGenerate\TestGenerate.xprj" -O "E:\ResultGenerate"
-B "C:\TestGenerate\nexacrolib" -GENERATERULE "C:\TestGenerate\generate"
-SERVICE "Base"
```

- Generate n number of services and then create them under the "E:\ResultGenerate" folder.

```
nexacrodeploy.exe -P "C:\TestGenerate\TestGenerate.xprj" -O "E:\ResultGenerate"
-B "C:\TestGenerate\nexacrolib" -GENERATERULE "C:\TestGenerate\generate"
-SERVICE "Base,FrameBase"
```

Generate only module files

- You can generate only the module files in use located at *C:\TestGenerate\nexacrolib* and produce the result to the *E:\ResultGenerate* folder.

```
nexacrodeploy.exe -P "C:\TestGenerate\TestGenerate.xprj" -O "E:\ResultGenerate"
-B "C:\TestGenerate\nexacrolib" -GENERATERULE "C:\TestGenerate\generate"
-MODULE
```

Generate only bootstrap files

- You can generate bootstrap files of all OS types needed to run your app and produce the result to the *E:\ResultGenerate* folder.

```
nexacrodeploy.exe -P "C:\TestGenerate\TestGenerate.xprj" -O "E:\ResultGenerate"
-B "C:\TestGenerate\nexacrolib" -GENERATERULE "C:\TestGenerate\generate"
-BOOTSTRAP
```

Generate bootstrap files but not archive them

- You can generate bootstrap files of all OS types needed to run your app and produce the result to the *E:\ResultGenerate* folder. However, the result will be the *Run.html* file rather than *Run.zip*.

```
nexacrodeploy.exe -P "C:\TestGenerate\TestGenerate.xprj" -O "E:\ResultGenerate"
-B "C:\TestGenerate\nexacrolib" -GENERATERULE "C:\TestGenerate\generate"
-BOOTSTRAP -UNARCHIVE "Android,iOS"
```

29.4.2 Deploy

Deploy entire Application

- The application deployer generates files and produces the result to the path specified as the -O option. Then, it creates the files for deployment in the directory specified as the -D option.
- In this case, all the options are applied to the generation process, which produces the result to the path specified as the -O option. (-R, -BROWSER, etc.)
- The files in the directory of the -D option are produced based on the files in the directory of the -O option.

```
nexacrodeploy.exe  -P "C:\TestGenerate\TestGenerate.xprj" -O
"E:\ResultGenerate" -B "C:\TestGenerate\nexacrolib" -GENERATERULE
"C:\TestGenerate\generate" -D "E:\ResultDeploy"
```

Module merge option

- You can create one JavaScript file by combining the JavaScript files in the *nexacrolib* folder. When you do so, the JavaScript files in *nexacrolib* will change their names in tandem with each relevant JSON file.
- You can create the *Framework.json* file by merging all the JavaScript files in the *framework* folder.

```
nexacrodeploy.exe  -P "C:\TestGenerate\TestGenerate.xprj" -O
"E:\ResultGenerate" -B "C:\TestGenerate\nexacrolib" -GENERATERULE
"C:\TestGenerate\generate" -D "E:\ResultDeploy" -MERGE
```

Compress option

- You can compress JavaScript files by removing their comments and whitespaces and thus making each file one-line code.

```
nexacrodeploy.exe  -P "C:\TestGenerate\TestGenerate.xprj" -O
"E:\ResultGenerate" -B "C:\TestGenerate\nexacrolib" -GENERATERULE
"C:\TestGenerate\generate" -D "E:\ResultDeploy" -COMPRESS
```

- While compressing JavaScript files to make each file one-line code with their comments and whitespaces removed, you can also obfuscate the variables of those JS files at the same time. However, a function will not be obfuscated if it contains an eval statement.

```
nexacrodeploy.exe  -P "C:\TestGenerate\TestGenerate.xprj" -O
"E:\ResultGenerate" -B "C:\TestGenerate\nexacrolib" -GENERATERULE
"C:\TestGenerate\generate" -D "E:\ResultDeploy" -COMPRESS -SHRINK
```

- While compressing JavaScript files to make each file one-line code with their comments and whitespaces removed, you can also obfuscate the variables of those files at the same time. Moreover, you can insist on obfuscation by ignoring eval statements included in functions.

```
nexacrodeploy.exe  -P "C:\TestGenerate\TestGenerate.xprj" -O
"E:\ResultGenerate" -B "C:\TestGenerate\nexacrolib" -GENERATERULE
"C:\TestGenerate\generate" -D "E:\ResultDeploy" -COMPRESS -SHRINK -IGNOREEVAL
```

- Script (JS) files are compressed with 1line code with notes and spaces removed.
- The js files designated in the ".ignorecompress" file are not compressed.

  The ".ignorecompress" file is automatically created when [Ignore compress files] is set in the Nexacro Studio's Compress option. If the file is created manually, the ".ignorecompress" file should be located in the same path as the xprj file and should be set as an XML file as follows.

  Create files with js extensions to be excluded when compressing based on the output path (it will not work properly if only the folder is set).

```
nexacrodeploy.exe  -P "C:\TestGenerate\TestGenerate.xprj" -O
"E:\ResultGenerate" -B "C:\TestGenerate\nexacrolib" -GENERATERULE
"C:\TestGenerate\generate" -D "E:\ResultDeploy" -COMPRESS -IGNORECOMPRESS
"C:\TestGenerate\.ignorecompress"
```

```
<?xml version="1.0" encoding="utf-8"?>
<CompressConfigue version="1.0">
  <IgnoreCompress>
    <File path=".\Service0\test01.js"/>
    <File path=".\Service0\test02.js"/>
```

```
  </IgnoreCompress>
</CompressConfigue>
```

Compile option

- You can compress deployment files by encoding them with the format applicable only to the NRE. The resulting encoded files cannot be read through other editor programs.

```
nexacrodeploy.exe  -P "C:\TestGenerate\TestGenerate.xprj" -O
"E:\ResultGenerate" -B "C:\TestGenerate\nexacrolib" -GENERATERULE
"C:\TestGenerate\generate" -D "E:\ResultDeploy" -COMPILE
```

29.4.3 Others

Save generation result as log file

- You can generate an Application based on *C:\TestGenerate\TestGenerate.xprj* file and produce the result to the *E:\ResultGenerate* folder. Then, you can save the details of the generation in the form of the *C:\Log.txt* file.

```
nexacrodeploy.exe  -P "C:\TestGenerate\TestGenerate.xprj" -O
"E:\ResultGenerate" -B "C:\TestGenerate\nexacrolib" -GENERATERULE
"C:\TestGenerate\generate" -L "C:\log.txt"
```